Hashtag Treinamentos

CURSO COMPLETO

SQL

IMPRESSIONADOR

Microsoft® SQL Server®

SUMÁRIO

MÓDULO 01	BANCO DE DADOS E SQL	004
MÓDULO 02	INSTALAÇÃO DO SQL SERVER	037
MÓDULO 03	INTRODUÇÃO AO SQL SERVER	069
MÓDULO 04	ORDENANDO E FILTRANDO DADOS	088
MÓDULO 05	FUNÇÕES DE AGREGAÇÃO	114
MÓDULO 06	CRIANDO AGRUPAMENTOS	128
MÓDULO 07	JOINS	149
MÓDULO 08	GROUP BY + JOINS	197
MÓDULO 09	VARIÁVEIS	218
MÓDULO 10	MANIPULANDO STRINGS E DATAS	244
MÓDULO 11	FUNÇÕES CONDICIONAIS	275
MÓDULO 12	VIEWS	299
MÓDULO 13	CRUD	316

Próxima Página

SUMÁRIO

MÓDULO 14	SUBQUERIES E CTE'S	351
MÓDULO 15	LOOPS	393
MÓDULO 16	WINDOW FUNCTIONS	406
MÓDULO 17	REGEX (REGULAR EXPRESSIONS)	470
MÓDULO 18	CONSTRAINTS	487
MÓDULO 19	SEQUENCES	520
MÓDULO 20	TRANSACTIONS	543
MÓDULO 21	FUNCTIONS	590
MÓDULO 22	PROCEDURES	609
MÓDULO 23	TRIGGERS DML	652
MÓDULO 24	TRIGGERS DDL	685
MÓDULO 25	PIVOT TABLE	695

MÓDULO 1

SQL

Banco de Dados e SQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 1 BANCO DE DADOS E SQL

01	TEMA 1 - O que é um dado?	006
02	TEMA 2 - O que é um banco de dados?	011
03	TEMA 3 - O que é o SQL?	017
04	TEMA 4 - Sistemas de Banco de Dados?	021
05	TEMA 5 - SQL vs MySQL, SQL Server, Oracle e PostgreSQL?	025
06	TEMA 6 - SQL, T-SQL, PL-SQL e pgSQL	028
07	TEMA 7 - Por que aprender SQL?	031
08	TEMA 8 - Profissionais na área de BD	035

TEMA 1

O que é um dado?

O que é um dado?

O que é um dado?

O que é um dado?

Um dado é uma informação que permite chegar ao conhecimento de algo. Estamos rodeados de dados.

Observe a imagem ao lado: concluímos que são bolas de futebol, certo? Logo, o primeiro dado que temos delas é que são utilizadas para jogar futebol. Além disso, temos outras informações sobre elas. Todas têm um tamanho, há uma determinada quantidade delas, provavelmente elas têm um custo associado, um tempo de uso, etc.

Qualquer objeto possui informações associadas. Nós, por exemplo, temos um nome, uma data de nascimento, um peso, altura e assim vai. Todas essas informações são dados.

Por esses exemplos iniciais, chegamos a uma conclusão: existem diferentes tipos de dados.

O que é um dado?

Em resumo, um dado **é uma informação** sobre alguma coisa. Em uma empresa, os dados cumprem um papel tão importante quanto qualquer outro bem.

Porém, para extrair o melhor de um dado, não basta tê-lo, é preciso organizá-lo para então entendê-lo.

> **Os dados são o novo petróleo.**

O que é um dado?

Os dados estão em todo lugar

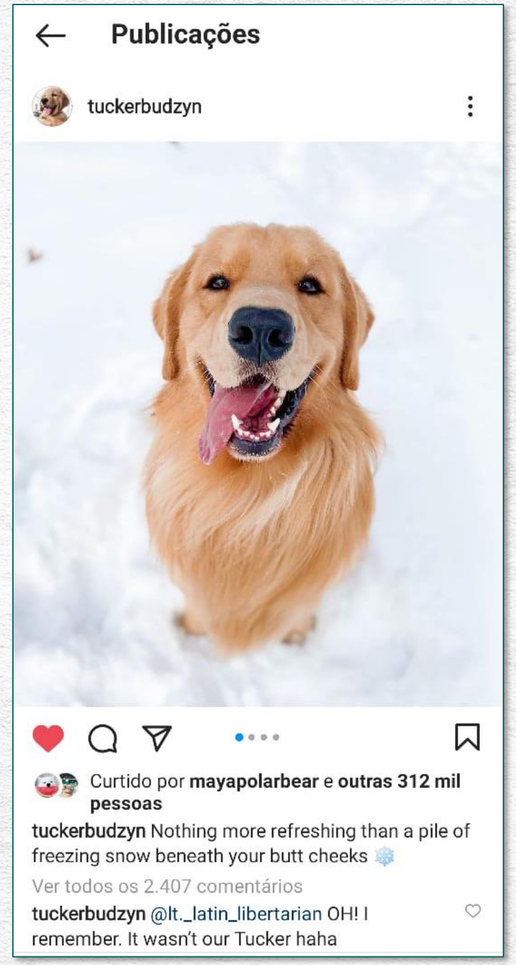
Imagina que você curta uma determinada foto no Instagram, como a foto mostrada ao lado.

Pra você, pra mim e para qualquer outro usuário comum do aplicativo se trata de uma mera curtida em uma foto...

... Porém, para o Instagram, essa curtida vale ouro.

Uma vez que você curte a publicação ao lado, o Instagram passa a ter um dado muito importante sobre você: **você gosta de cachorros**. Com esse dado, ou seja, com essa informação, o Instagram vai começar a mostrar a você mais fotos de cachorros. E se você continuar curtindo essas fotos, o Instagram vai mostrar mais e mais fotos de cachorros para você, até chegar a um ponto de você se perguntar "uau, como o Instagram sabe que eu gosto de cachorros?"

Só que não é mágica, é apenas o Instagram armazenando esses dados, entendendo um padrão e tomando uma ação a partir disso.



O que é um dado?

Todas as empresas possuem um grande volume de informações/dados que precisam ser organizados e armazenados.

Vamos tomar como exemplo a situação anterior do Instagram. O aplicativo possui mais de 100 milhões de usuários. Imagine a quantidade de dados que precisam ser armazenados:

- Perfis
- Fotos
- Vídeos
- Mensagens
- Etc..

A partir dessas informações, o Instagram vai entender quais são as suas preferências como usuário, o que você gosta ou não, o que deve mostrar a você ou não e como prender a sua atenção, a fim de atingir um objetivo simples: **gerar o maior engajamento possível**.

Não é só o Instagram que vive de dados

Não é só o Instagram que tem o desafio diário de manipular tantos dados. Qualquer empresa precisa manipular diariamente centenas, milhares ou até milhões de dados.

É claro que o volume de dados vai variar de acordo com o tamanho da empresa, mas de forma geral, toda empresa precisa ter os dados organizados e centralizados.

TEMA 2

O que é um banco de dados?

O que é um banco de dados?

O que é um banco de dados?

O que é um banco de dados?

Como vimos anteriormente, um dado é uma informação que nos permite chegar ao conhecimento de algo. Um dado isolado por si só já tem um grande valor. Em conjunto com outros dados então, ele é mais valioso ainda.

Vamos então tentar entender o que é um Banco de Dados de uma maneira bem simples. Imagine a imagem abaixo. Um dado carrega consigo alguma informação. Quando temos vários dados, precisamos organizar em uma tabela. E quando temos várias tabelas, temos um banco de dados.

Dado

- 1 Compra online
- 1 transferência bancária
- 1 curtida em uma foto do Instagram

Tabela

- Diversas compras online
- Diversas transferência bancárias
- Diversas curtidas em fotos do Instagram

Banco de Dados

- Informações de compras online, usuários, etc
- Informações de contas bancárias, histórico de transferências, dados dos usuários, etc
- Curtidas em fotos, informações de preferências dos usuários, informações dos perfis de usuários, etc

Índice Geral

Índice deste Módulo

O que é um banco de dados?

Com os dados bem organizados e estruturados, o Instagram será capaz de melhorar a experiência de cada usuário do aplicativo, mostrando temas mais relevantes de acordo com as preferências de cada usuário.

Dado

O registro de 1 curtida feita pelo usuário.

Data	Usuário	Curtidas	Categoria
10/05/2021	Joãozinho	1	Cachorro

Tabela

Lugar onde serão reunidos todas as curtidas dos usuários.

Data	Usuário	Curtidas	Categoria
10/05/2021	Joãozinho	1	Cachorro
10/05/2021	Mariazinha	3	Gato
11/05/2021	Joãozinho	5	Cachorro
13/05/2021	Mariazinha	10	Panda
15/05/2021	Joãozinho	6	Cachorro

Banco de Dados

Conjunto de várias tabelas que se relacionam, com informações de curtidas, usuários e perfis.

Data	Usuário	Curtidas	Categoria
10/05/2021	Joãozinho	1	Cachorro
10/05/2021	Mariazinha	3	Gato
11/05/2021	Joãozinho	5	Cachorro
13/05/2021	Mariazinha	10	Panda
15/05/2021	Joãozinho	6	Cachorro

Usuário	Idade	Cidade
Joãozinho	21	Rio de Janeiro
Mariazinha	25	São Paulo

Categoria	Perfis
Cachorro	30
Gato	10
Panda	5

Índice Geral
Índice deste Módulo

O que é um banco de dados?

Um outro exemplo seria um banco de dados da Nubank. Com ele, a empresa é capaz de entender o perfil de consumo de cada usuário e definir quais terão prioridade em um possível aumento de limite.

Dado

O registro de 1 compra feita pelo usuário.

Data	Usuário	Loja	Valor
10/05/2021	Joãozinho	Ifood	R$ 150,00

Tabela

Lugar onde serão reunidos todos os dados.

Data	Usuário	Loja	Valor
10/05/2021	Joãozinho	Ifood	R$ 150,00
10/05/2021	Mariazinha	Uber	R$ 10,30
11/05/2021	Joãozinho	Magalu	R$ 5.000,00
13/05/2021	Mariazinha	Uber	R$ 8,90
15/05/2021	Joãozinho	Rapi	R$ 120,00

Banco de Dados

Conjunto de várias tabelas que se relacionam, com informações de compras e usuários.

Data	Usuário	Compra	Valor
10/05/2021	Joãozinho	Ifood	R$ 150,00
10/05/2021	Mariazinha	Uber	R$ 10,30
11/05/2021	Joãozinho	Magalu	R$ 5.000,00
13/05/2021	Mariazinha	Uber	R$ 8,90
15/05/2021	Joãozinho	Rapi	R$ 120,00

Usuário	Idade	Limite
Joãozinho	21	R$ 7.000,00
Mariazinha	25	R$ 6.500,00

Loja	Vendas
Ifood	1000
Uber	560
Magalu	5
Rapi	800

O que é um banco de dados?

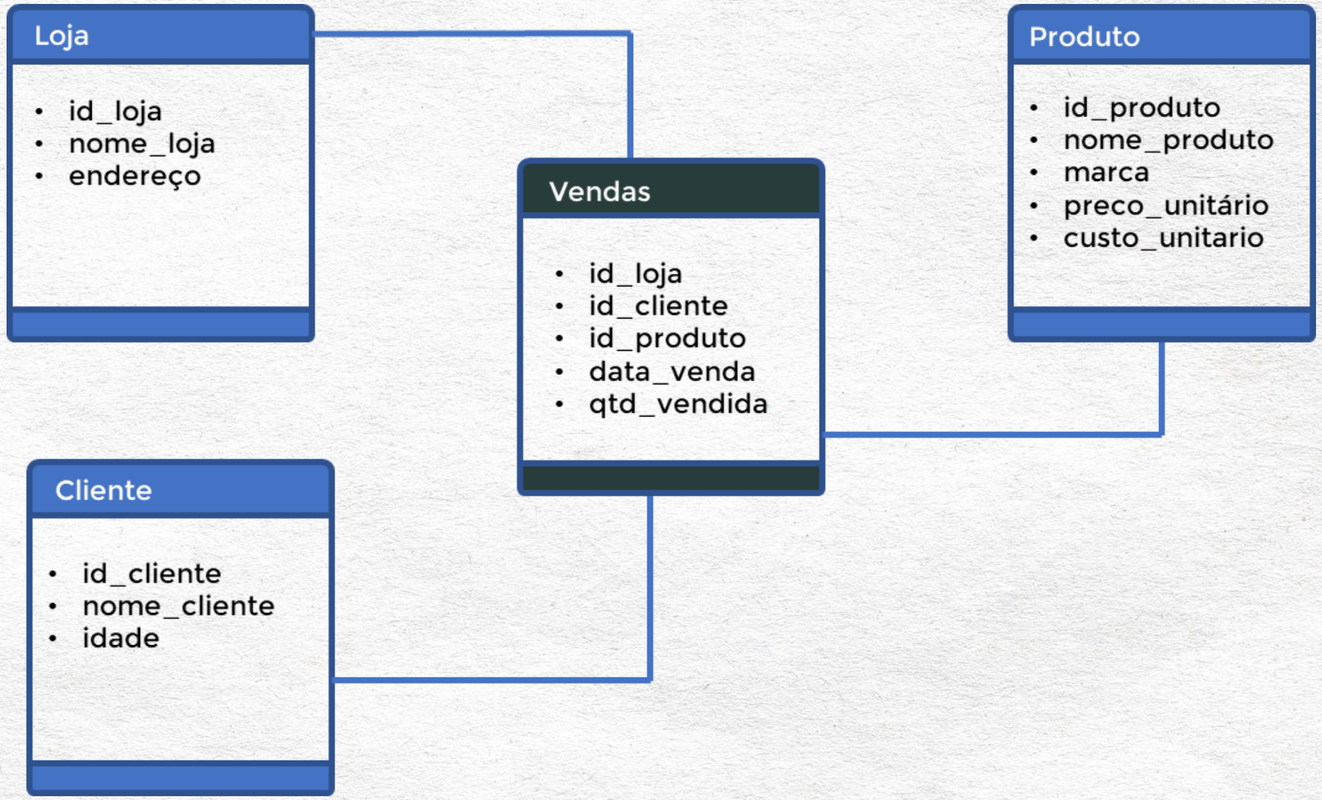
> **Bancos de dados** são conjuntos de tabelas, com alguma relação entre si, com dados sobre pessoas, lugares ou coisas.
>
> Estes dados organizados permitem a compreensão de um determinado fenômeno na empresa, seja a preferência dos usuários em uma rede social, seja o perfil de consumo em um aplicativo de transações financeiras.

O que é um banco de dados?

O desenho esquemático de um banco de dados é algo como o mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais serão o foco do nosso curso, até por serem o tipo de bancos de dados mais comumente encontrados no mercado.



TEMA 3

O que é o SQL?

O que é o SQL?

O que é o SQL?

O que é o SQL?

Como vimos, dados são o coração de qualquer negócio. Uma empresa como o Facebook precisa manipular, consultar e armazenar bilhões de informações dos usuários, como publicações, fotos, stories, mensagens privadas, etc.

Para que isso seja possível, é necessário algum sistema de banco de dados para manipular todas essas informações.

E quem vai permitir que a gente consiga buscar todos esses dados em bancos de dados, adicionar novos dados, excluir esses dados e atualizar esses dados é o **SQL**.

O SQL é uma linguagem padrão para trabalhar com bancos de dados relacionais. Será através do SQL que seremos capazes de consultar e manipular os dados dos nossos bancos de dados.

SQL significa: “**Structured Query Language**”. Se trata de uma linguagem de programação que permite a manipulação dos dados em um banco de dados.

O que é o SQL?

O SQL é uma linguagem que muitos profissionais acabam precisando aprender: seja quem usa Excel de forma pesada e acaba migrando as informações para um Banco de Dados, seja um cientista de dados que usa o Python para agregar os dados das diferentes fontes de informação.

> SQL significa “**Structured Query Language**”. Se trata de uma linguagem de programação utilizada para armazenar, consultar, adicionar e excluir informações em um banco de dados.

O que é o SQL?

Imagine que você tem uma tabela em um banco de dados com as vendas de alguns produtos (pode até ser feita uma analogia a uma planilha do Excel).

id	produto	data_venda	valor
1	televisão	2021-03-20	1500
2	computador	2021-03-22	2300
3	celular	2021-03-25	800
4	ps4	2021-03-28	3100
5	tablet	2021-03-28	650

Se você quer buscar todas as vendas com um valor maior que 1000 reais, o código SQL que você deve usar é o seguinte:

```
SELECT * FROM tabela_vendas WHERE valor > 1000
```

O resultado do código anterior será uma nova tabela, mostrando apenas os produtos onde o valor de venda é maior que 1000 reais.

id	produto	data_venda	valor
1	televisão	2021-03-20	1500
2	computador	2021-03-22	2300
4	ps4	2021-03-28	3100

Sem perceber, o que acabamos de fazer anteriormente foi um exemplo de uma query, ou seja, uma consulta ao banco de dados, usando um código SQL para isso.

Relembrando: SQL significa “Structured Query Language”, traduzindo, “linguagem de consulta estruturada”.

TEMA 4

Sistemas de Banco de Dados?

Sistemas de Banco de Dados?

Sistemas de Banco de Dados?

Sistemas de Gerenciamento de Bancos de Dados

Agora que já sabemos o que é o SQL, surge a seguinte dúvida: o que é um Sistema de Gerenciamento de Banco de Dados, ou SGBD?

Um SGBD permite ao desenvolvedor trabalhar com diferentes tabelas de um banco de dados através de uma interface. Essa interface seria basicamente um programa que nos permite fazer a leitura de tabelas de um banco de dados e utilizar o SQL para manipular esses dados, tudo de uma maneira bem visual e *user-friendly*.

Um SGBD é composto essencialmente por 2 partes:

um **servidor**, onde vamos conseguir armazenar os nossos bancos de dados.

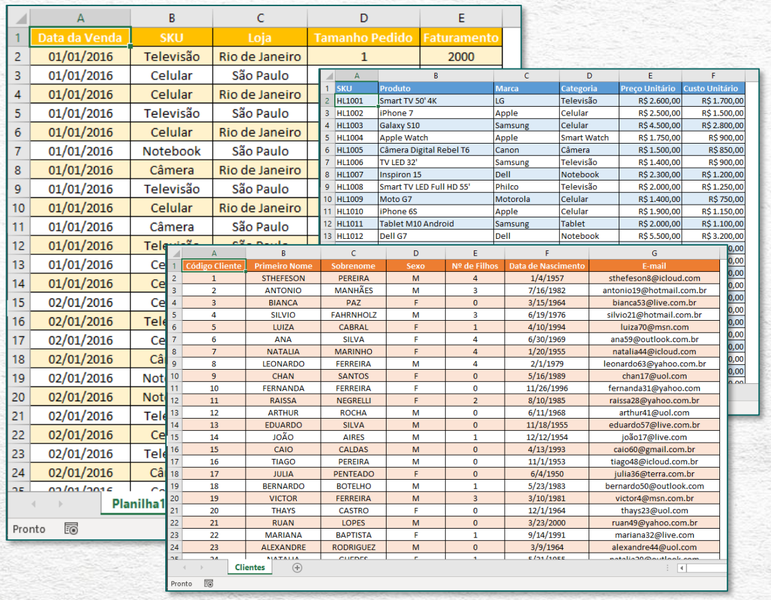
uma **interface** amigável que nos permite escrever os códigos em SQL para acessar os bancos de dados.

Sistemas de Gerenciamento de Bancos de Dados

E o que dizer sobre o Excel e o Access?

Em uma visão bem simplista, podemos considerar o Excel como uma simplificação de um SGBD. Por meio dele, conseguimos criar diferentes planilhas e tabelas, relacionando-as entre si por meio de fórmulas.

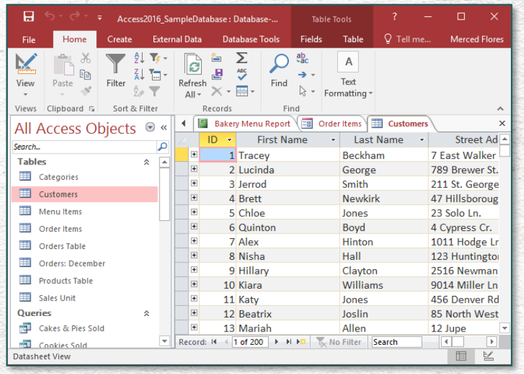
O Access já seria um programa com mais similaridades a um SGBD, porém com muitas limitações também, pensando em uma empresa que lida com alto volume de dados e que necessita uma maior segurança de armazenamento.



Data da Venda	SKU	Loja	Tamanho Pedido	Faturamento
01/01/2016	Televisão	Rio de Janeiro	1	2000
01/01/2016	Celular	São Paulo		
01/01/2016	Celular	Rio de Janeiro		
01/01/2016	Televisão	São Paulo		
01/01/2016	Celular	Rio de Janeiro		
01/01/2016	Notebook	São Paulo		
01/01/2016	Câmera	Rio de Janeiro		
01/01/2016	Televisão	São Paulo		
01/01/2016	Celular	Rio de Janeiro		
01/01/2016	Câmera	São Paulo		

SKU	Produto	Marca	Categoria	Preço Unitário	Custo Unitário
HL1001	Smart TV 50' 4K	LG	Televisão	R$ 2.600,00	R$ 1.700,00
HL1002	iPhone 7	Apple	Celular	R$ 2.500,00	R$ 1.500,00
HL1003	Galaxy S10	Samsung	Celular	R$ 4.500,00	R$ 2.800,00
HL1004	Apple Watch	Apple	Smart Watch	R$ 1.750,00	R$ 900,00
HL1005	Câmera Digital Rebel T6	Canon	Câmera	R$ 1.500,00	R$ 850,00
HL1006	TV LED 32'	Samsung	Televisão	R$ 1.400,00	R$ 900,00
HL1007	Inspiron 15	Dell	Notebook	R$ 2.300,00	R$ 1.200,00
HL1008	Smart TV LED Full HD 55'	Philco	Televisão	R$ 2.000,00	R$ 1.250,00
HL1009	Moto G7	Motorola	Celular	R$ 1.400,00	R$ 750,00
HL1010	iPhone 6S	Apple	Celular	R$ 1.900,00	R$ 1.150,00
HL1011	Tablet M10 Android	Samsung	Tablet	R$ 2.000,00	R$ 1.100,00
HL1012	Dell G7	Dell	Notebook	R$ 5.500,00	R$ 3.200,00

Código Cliente	Primeiro Nome	Sobrenome	Sexo	Nº de Filhos	Data de Nascimento	E-mail
1	STHEFESON	PEREIRA	M	4	1/4/1957	sthefeson8@icloud.com
2	ANTONIO	MANHÃES	M	3	7/16/1982	antonio19@hotmail.com.br
3	BIANCA	PAZ	F	0	3/15/1964	bianca53@live.com.br
4	SILVIO	FAHRNHOLZ	M	3	6/19/1976	silvio21@hotmail.com.br
5	LUIZA	CABRAL	F	1	4/10/1994	luiza70@msn.com
6	ANA	SILVA	F	4	6/30/1969	ana59@outlook.com.br
7	NATALIA	MARINHO	F	4	1/20/1955	natalia44@icloud.com
8	LEONARDO	FERREIRA	M	4	2/1/1979	leonardo63@yahoo.com.br
9	CHAN	SANTOS	F	0	5/16/1989	chan17@uol.com
10	FERNANDA	FERREIRA	F	0	11/26/1996	fernanda31@yahoo.com
11	RAISSA	NEGRELLI	F	2	8/10/1985	raissa28@yahoo.com.br
12	ARTHUR	ROCHA	M	0	6/11/1968	arthur41@uol.com
13	EDUARDO	SILVA	M	0	11/18/1955	eduardo57@live.com.br
14	JOÃO	AIRES	M	1	12/12/1954	joão17@live.com
15	CAIO	CALDAS	M	0	4/13/1993	caio60@gmail.com.br
16	TIAGO	PEREIRA	M	0	11/1/1953	tiago48@icloud.com.br
17	JULIA	PENTEADO	F	0	6/4/1950	julia36@terra.com.br
18	BERNARDO	BOTELHO	M	1	5/23/1983	bernardo50@outlook.com
19	VICTOR	FERREIRA	M	3	3/10/1981	victor4@msn.com.br
20	THAYS	CASTRO	F	0	12/1/1964	thays23@uol.com
21	RUAN	LOPES	M	0	3/23/2000	ruan49@yahoo.com.br
22	MARIANA	BAPTISTA	F	1	9/14/1991	mariana32@live.com
23	ALEXANDRE	RODRIGUEZ	M	0	3/9/1964	alexandre44@uol.com



Índice Geral

Índice deste Módulo

Sistemas de Gerenciamento de Bancos de Dados

Existem alguns SGBDs para Bancos de Dados Relacionais que são muito utilizados por grandes empresas. Abaixo, temos os 4 principais programas para SGBDs.

É importante que fique claro que **todos esses SGBDs** utilizam o SQL como linguagem de programação.

MySQL

É um SGBD relacional de código aberto, usado na maioria das aplicações gratuitas para gerir suas bases de dados.

A interface de código utilizada é o MySQL Workbench.

Oracle

A Oracle é uma das maiores empresas de tecnologia do mundo.

O SGBD da Oracle é focado em empresas de médio e grande porte.

A interface de código utilizada é o SQL Developer.

SQL Server

O SQL Server é o SGBD criado pela Microsoft, também para bancos de dados relacionais.

A interface de código utilizada é o SSMS.

PostgreSQL

O PostgreSQL é um SGBD relacional, criado em 1989 e ainda um dos mais utilizados no mundo.

A interface de código utilizada é o pgAdmin.

Índice Geral
Índice deste Módulo

TEMA 5

SQL vs MySQL, SQL Server, Oracle e PostgreSQL?

SQL vs MySQL, SQL Server, Oracle e PostgreSQL?

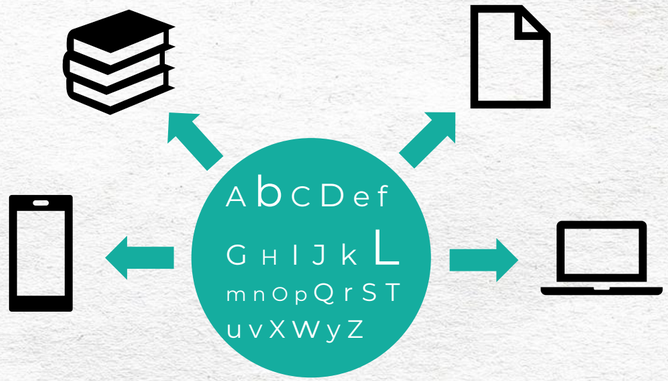
SQL vs MySQL, SQL Server, Oracle e PostgreSQL?

SQL vs. MySQL, SQL Server, Oracle e PostgreSQL

Uma das dúvidas mais comuns de quem começa a estudar SQL é: o que eu devo aprender? SQL ou MySQL? SQL ou SQL Server?

A resposta é: esse questionamento não faz sentido!

Para você entender a diferença entre estes termos, imagine que você queira aprender a se comunicar com outras pessoas. Você vai concordar que o alfabeto é muito importante para esse objetivo, certo? Mas de nada serve o alfabeto se não tivermos um local onde podemos combinar essas letras e formar palavras que permitam a comunicação.



Por isso, além do alfabeto, precisamos também de um lugar para escrever as palavras: pode ser uma folha de papel, um caderno, o nosso celular, um computador, e assim vai.

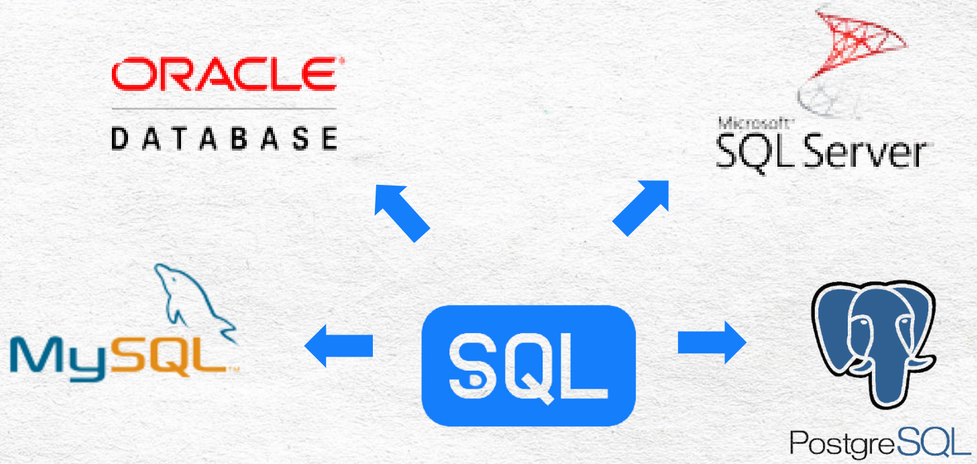
A nossa decisão então não será entre aprender o alfabeto ou usar uma folha de papel, ou aprender o alfabeto ou usar um caderno. O alfabeto será importante em qualquer situação: ele é a base da nossa comunicação.

SQL vs. MySQL, SQL Server, Oracle e PostgreSQL

Fazendo um paralelo com o SQL, o SQL é como se fosse o nosso alfabeto, e os diferentes SGBDs serão os lugares onde conseguiremos escrever esses códigos em SQL para acessar nossos bancos de dados.

Por isso, a pergunta correta não é: aprender SQL ou MySQL? E sim, qual sistema de gerenciamento de banco de dados usar: MySQL, SQL Server, Oracle ou PostgreSQL? O SQL você vai precisar aprender em qualquer um dos casos.

> O SQL é a linguagem de programação usada para manipular os bancos de dados.
>
> Já o MySQL, SQL Server, Oracle Database e PostgreSQL são sistemas de bancos de dados, um ambiente onde vamos utilizar o SQL.



TEMA 6

SQL, T-SQL, PL-SQL e pgSQL

SQL, T-SQL, PL-SQL e pgSQL

SQL, T-SQL, PL-SQL e pgSQL

SQL, T-SQL, PL-SQL e PL-pgSQL: do que se tratam?

O SQL é a linguagem de consulta usada para comunicação com dados em um banco de dados. É usado como a linguagem padrão em todos os sistemas de gerenciamento de banco de dados relacional.

No entanto, o SQL possui algumas vertentes que dependem de cada sistema de gerenciamento de banco de dados: cada SGBD incorpora à linguagem algumas funcionalidades e recursos extras.

Apesar de algumas diferenças, a base para todas as demais variantes do SQL seguem uma lógica semelhante, mas vale a pena entender que elas existem e que, caso você ouça por ai, você entenda minimamente sobre cada uma delas.

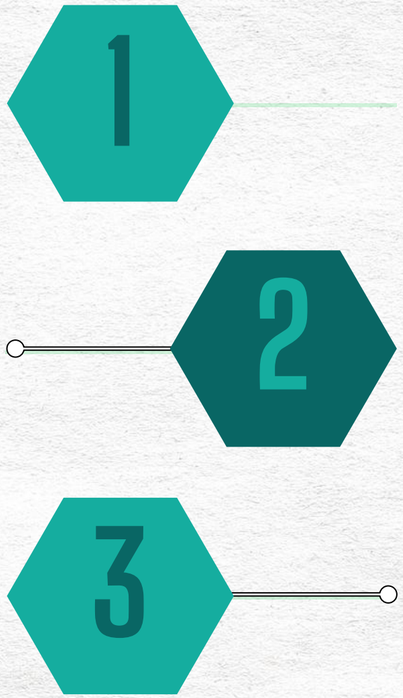
SQL, T-SQL, PL-SQL e PL-pgSQL: do que se tratam?

Abaixo, um resumo de cada uma dessas variações.

PL-SQL

PL-SQL ou Procedural Language/SQL é outra forma estendida de SQL que é usada pela Oracle para seu banco de dados.

A principal diferença entre o T-SQL e o PL-SQL é a maneira como eles lidam com variáveis, procedimentos armazenados e funções integradas. T-SQL também é considerado mais fácil e simples de entender, enquanto PL-SQL incorpora maior complexidade.



T-SQL

TSQL, ou T-SQL (abreviação de Transaction-SQL), é uma versão aprimorada do SQL que contém algumas extensões. O TSQL foi originalmente desenvolvido pela Sybase e agora é propriedade da Microsoft.

O T-SQL adiciona alguns recursos avançados ao SQL para torná-lo mais poderoso, como variáveis declaradas, controle de transação, tratamento de erros e exceções, operações de string, processamento de data e hora, etc.

PL-pgSQL

PL-PgSQL é uma linguagem procedural específica do PostgreSQL baseada em SQL. Semelhante ao T-SQL e PL-SQL, ele adiciona alguns recursos avançados ao SQL, como loops, variáveis, tratamento de erros e exceções, etc.

TEMA 7

Por que aprender SQL?

Por que aprender SQL?

Por que aprender SQL?

Por que aprender SQL?

O SQL possui aplicações em diferentes áreas e profissões. O profissional que tiver conhecimento não só em ferramentas como o Excel, Power BI ou Python, mas também em SQL, terá muitas oportunidades de atuação.

O motivo para aprender SQL é simples: garantir uma maior versatilidade no conhecimento e ser um profissional diferenciado e requisitado no mercado.

A seguir, listamos algumas áreas que precisam manipular um alto volume de dados, e consequentemente precisarão de sistemas de bancos de dados para auxiliar neste trabalho.

Por que aprender SQL?

Setor financeiro

Apps de banco, sistemas de pagamento e empresas de investimento

Imagine todo o volume de dados por trás de todas as operações financeiras que fazemos no dia a dia: abertura de contas, pagamentos, transferências, depósitos, investimentos variados, etc.
Para que tudo funcione perfeitamente, é necessário que as empresas tenham um controle dos dados através de sistemas de bancos de dados.

Redes sociais

Facebook, Instagram, Linkedin, TikTok

Redes sociais são mais um exemplo de um local onde encontramos um alto volume de dados. Elas armazenam bilhões de dados de usuários, localizações, preferências, fotos, etc.
A partir desses dados, os aplicativos entendem o comportamento de cada usuário e buscam proporcionar a melhor experiência possível para cada um.

Aplicativos de música e Streaming

Spotify, Netflix, Globoplay, Amazon Prime

Os Bancos de Dados auxiliam no armazenamento de informações de milhares de bibliotecas, álbuns, músicas, artistas, filmes, séries e gêneros e utilizam esses dados para cruzar com as informações de preferências dos usuários.
Todo o trabalho de pesquisa e sugestão de músicas, séries e filmes tem por trás um trabalho complexo de manipulação e cruzamento de dados.

Redes de pesquisa

Youtube e Google

Na mesma linha das redes sociais, os mecanismos de busca na internet também se utilizam de uma grande quantidade de dados para sugerir vídeos, postagens, sites, etc, de acordo com as buscas e comportamento na web.

Por que aprender SQL?

Data science

Data Science, ou Ciência de Dados, é uma área voltada para o estudo e análise de dados econômicos, financeiros e sociais, com o objetivo de chegar ao conhecimento de algo. Já é de se esperar que essa é uma das áreas que mais necessita de um bom gerenciamento dos dados.

Setor de energia

O setor de energia é uma área com muitas oportunidades no mercado, envolvendo empresas de geração, transmissão, distribuição e comercialização de energia.
Imagine o volume de dados que este setor precisa manipular. Sem bancos de dados bem estruturados, não seria algo viável.

Business Intelligence e Análise de Dados

A base para se trabalhar com BI e criar relatórios, gráficos e dashboards que auxiliam na tomada de decisão de um negócio é ter os dados do negócio bem estruturado.

Marketing

Profissionais da área de marketing precisam manipular dados sobre seus clientes, como jornada, histórico de compras, interesse no produto. Dependendo do tamanho/alcance do negócio ou da empresa, será necessário trabalhar com um alto volume de dados.

TEMA 8

Profissionais na área de BD

Profissionais na área de BD

Profissionais na área de BD

Profissionais em bancos de dados

Um profissional com conhecimento em Bancos de Dados pode atuar em diferentes segmentos. Abaixo, listamos as principais áreas de atuação. Observe que há uma grande variedade para que trabalha com dados.

Administrador de Dados (AD):
Profissional que conhece a fundo o processo empresarial, define as tabelas que serão criadas e os relacionamentos entre essas tabelas.

Desenvolvedor de Banco de Dados (DB):
O DB tem um grande conhecimento de lógica de programação e responsável por criar rotinas e gerenciar processos que são feitos dentro do próprio SGBD.

Administrador de Banco de Dados (DBA):
A função de um DBA é garantir que o Banco de Dados esteja sempre funcionando, garantindo sua disponibilidade e otimização.

Analista de Business Intelligence:
Responsável pela coleta, organização e análise das informações de mercado que dão suporte à gestão do negócio.

Cientista de Dados:
Especialista com habilidade para analisar grande volume de dados e interpretar informações de valor e apoiar na tomada de decisão dos negócios.

Analista de Dados:
Responsável por coletar, compilar e interpretar dados, promovendo estratégias para a empresa.

MÓDULO 2

SQL

Instalação do SQL Server, SSMS e conhecendo o Banco de Dados

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 2 INSTALAÇÃO DO SQL SERVER

01	TEMA 9 - Instalando o SQL Server	039
02	TEMA 10 - Instalando o SSMS	046
03	TEMA 11 - Download do BD Contoso	056
04	TEMA 12 - Conhecendo o BD Contoso	064
05	TEMA 13 - Explorando as bases de dados	067

TEMA 9

Instalando o SQL Server

Instalando o SQL Server

Instalando o SQL Server

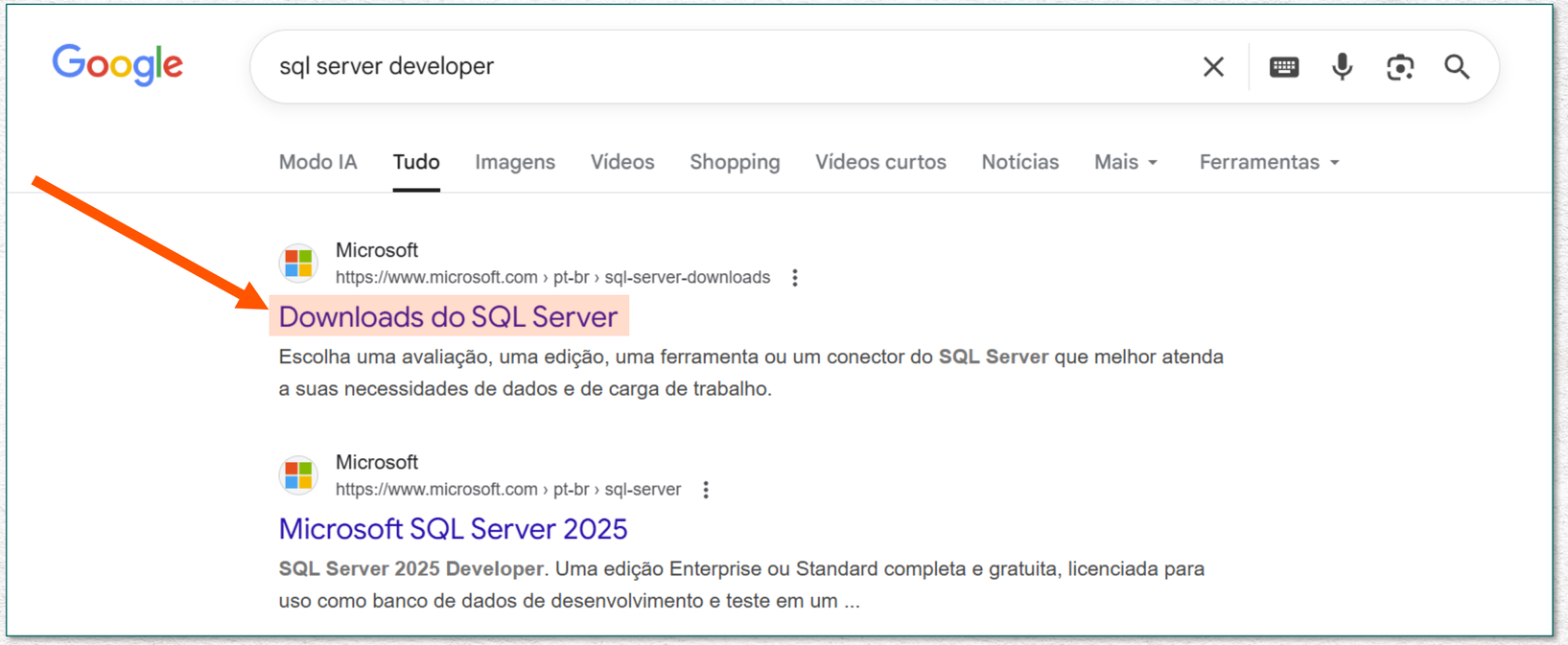
Instalando o SQL Server

Vamos instalar as **versões mais atuais** do **SQL Server** (o **servidor** em si) e do **SSMS** (a **interface** que usamos para acessar os bancos de dados armazenados no servidor):

- **SQL Server 2025**
- **SQL Server Management Studio 22**

Para fazer o download do SQL Server, basta procurar no Google por "**sql server developer**".

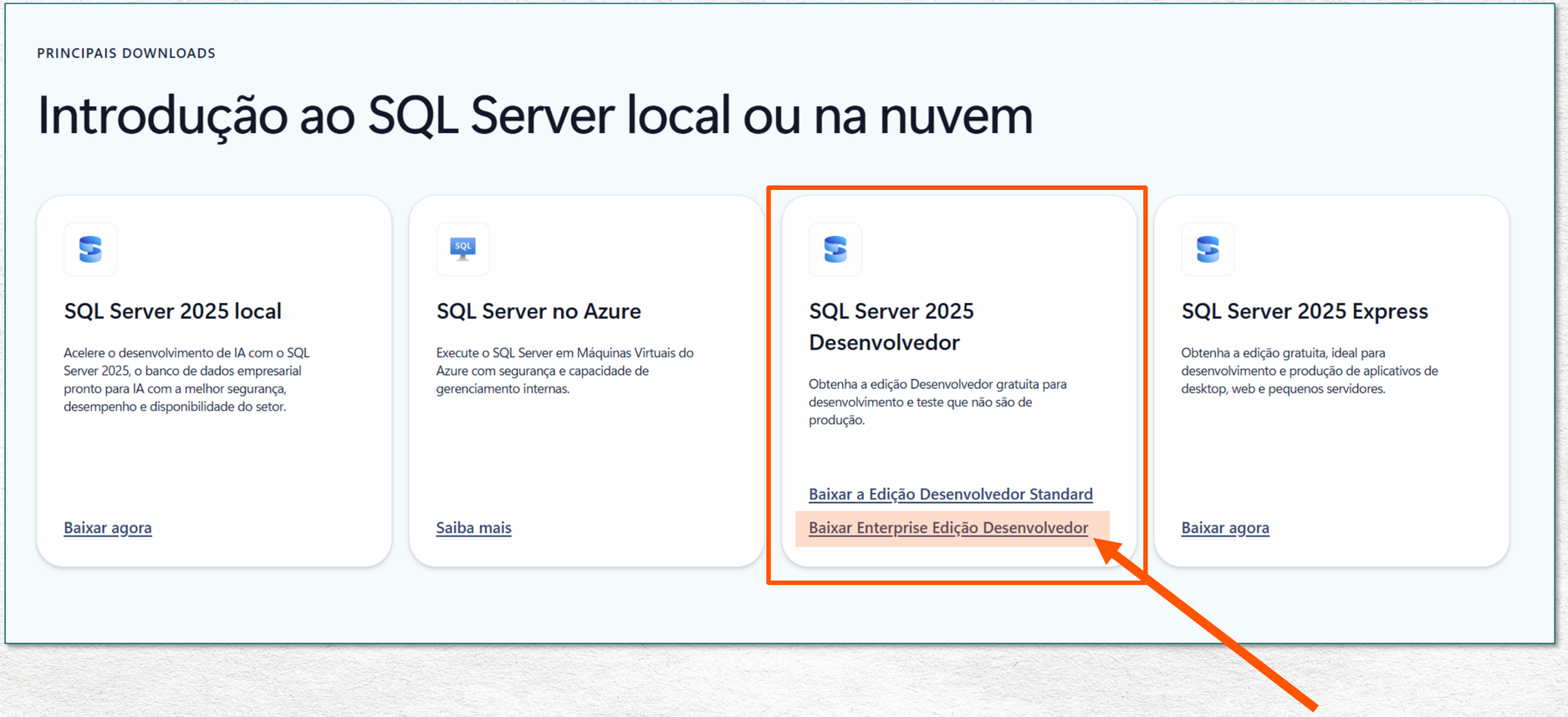
Clique no link oficial da Microsoft ("**Downloads do SQL Server**"):



Instalando o SQL Server

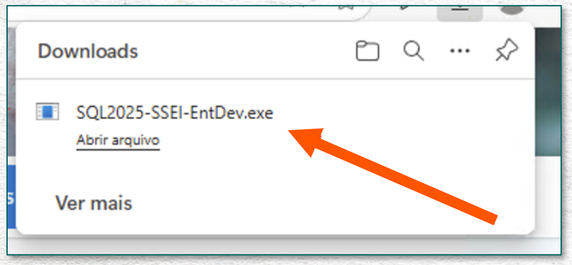
Na página que se abrir, procure pelo quadrado com a opção **“SQL Server 2025 Desenvolvedor”.**

Dentro dele, clique no link de texto escrito **“Baixar Enterprise Edição Desenvolvedor”:**



Instalando o SQL Server

Abra o arquivo executável baixado (**SQL2025-SSEI-EntDev.exe**):



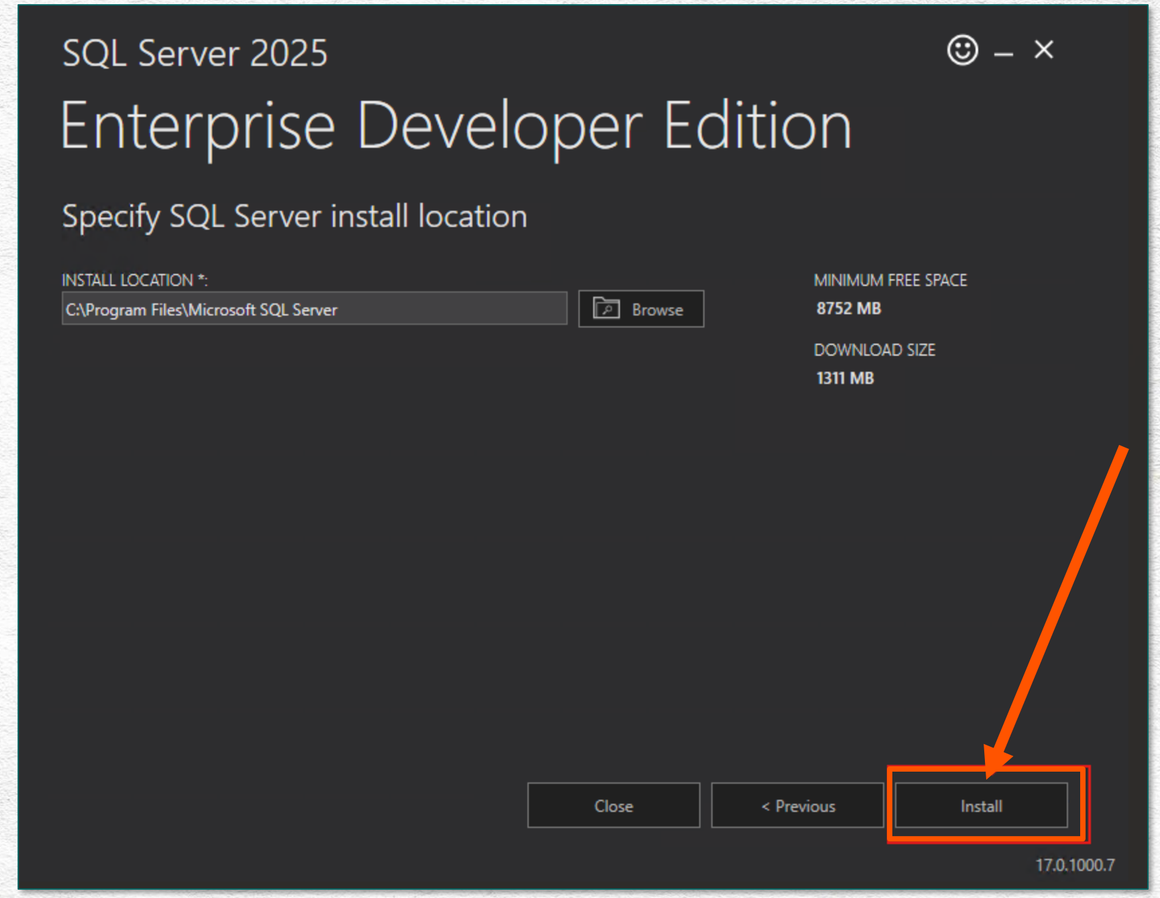
Na tela que se abrir, escolha a opção Básica (**Basic**).

Instalando o SQL Server

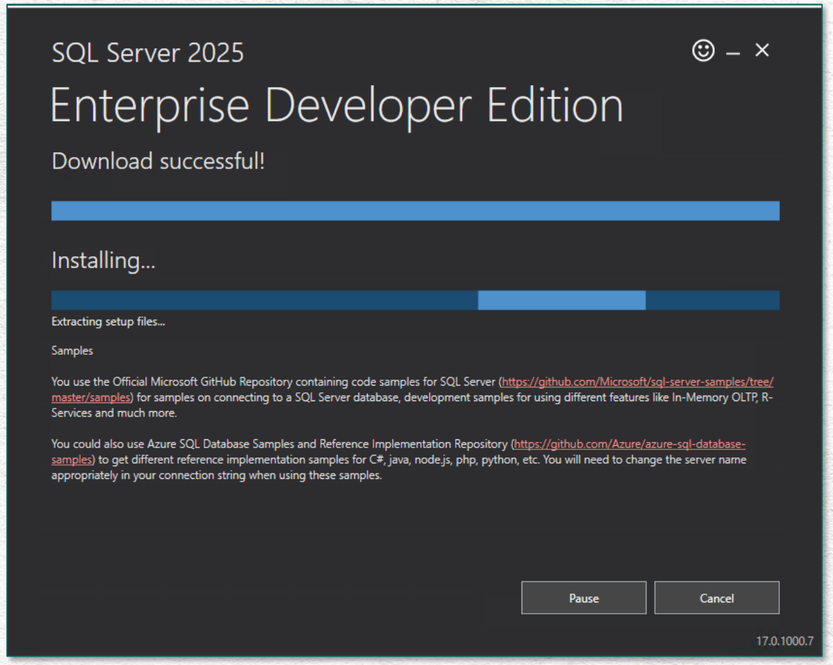
Clique em **Accept** para aceitar os termos da licença:

Instalando o SQL Server

Na tela seguinte, apenas clique em **Install** para efetuar a instalação...



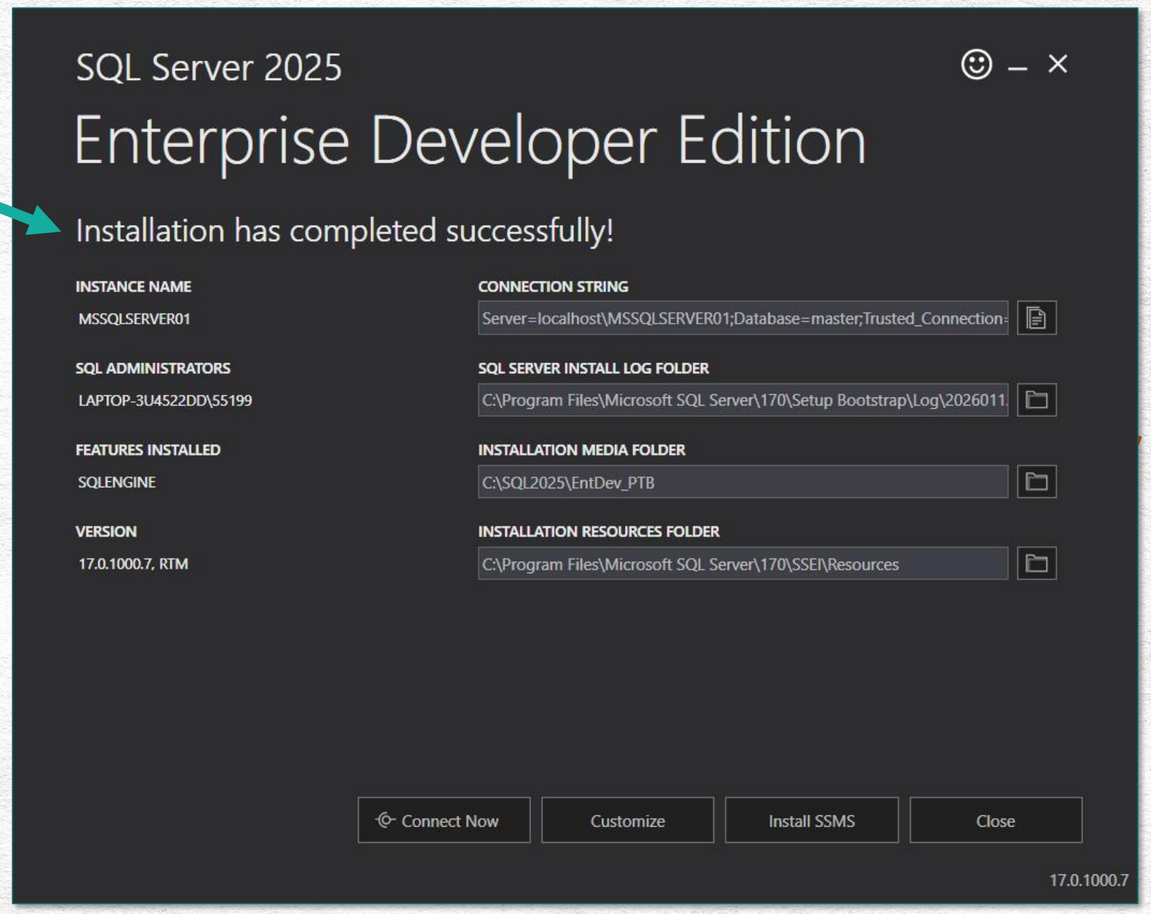
... e aguarde. Isso pode demorar um pouco:



Instalando o SQL Server

Ao finalizar, aparecerá uma tela com a mensagem **"Installation has completed successfully!"**.

NÃO FECHE ESSA JANELA AINDA!



TEMA 10

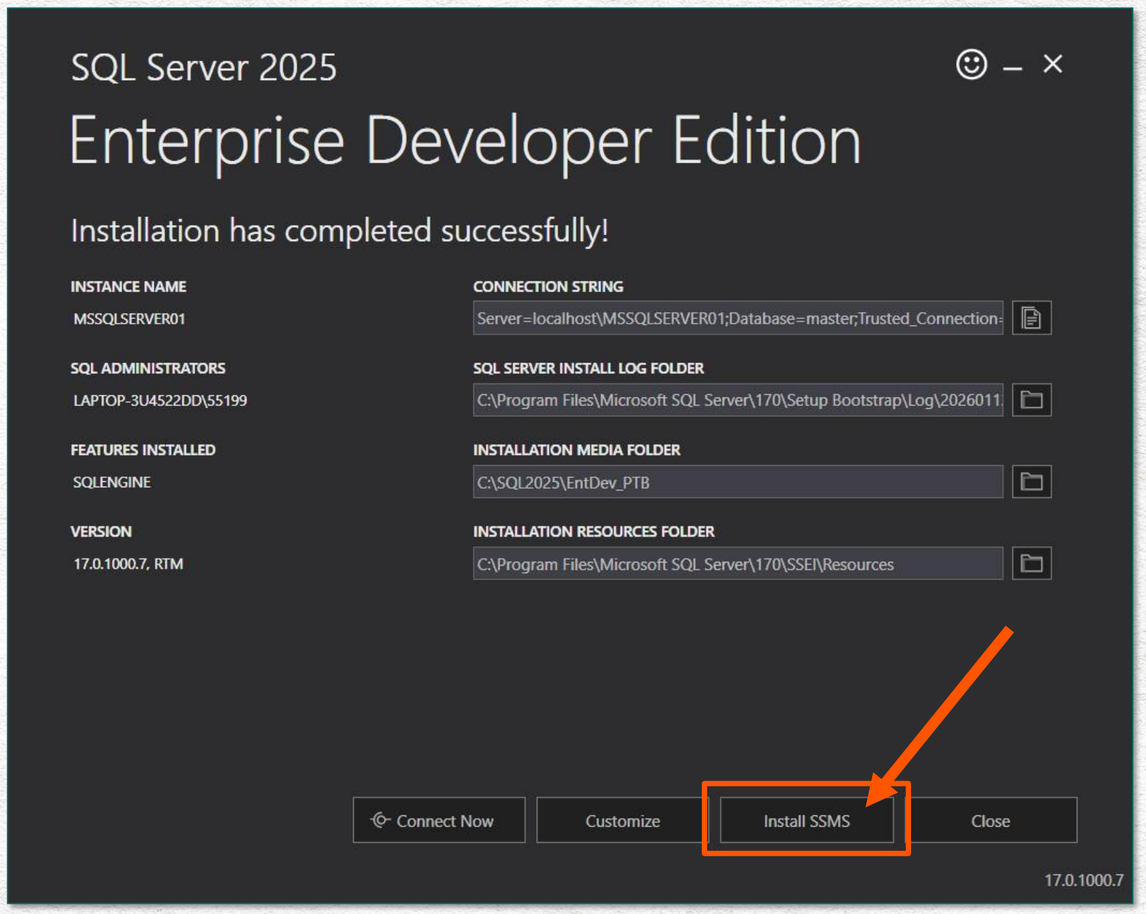
Instalando o SSMS

Instalando o SSMS

Instalando o SSMS

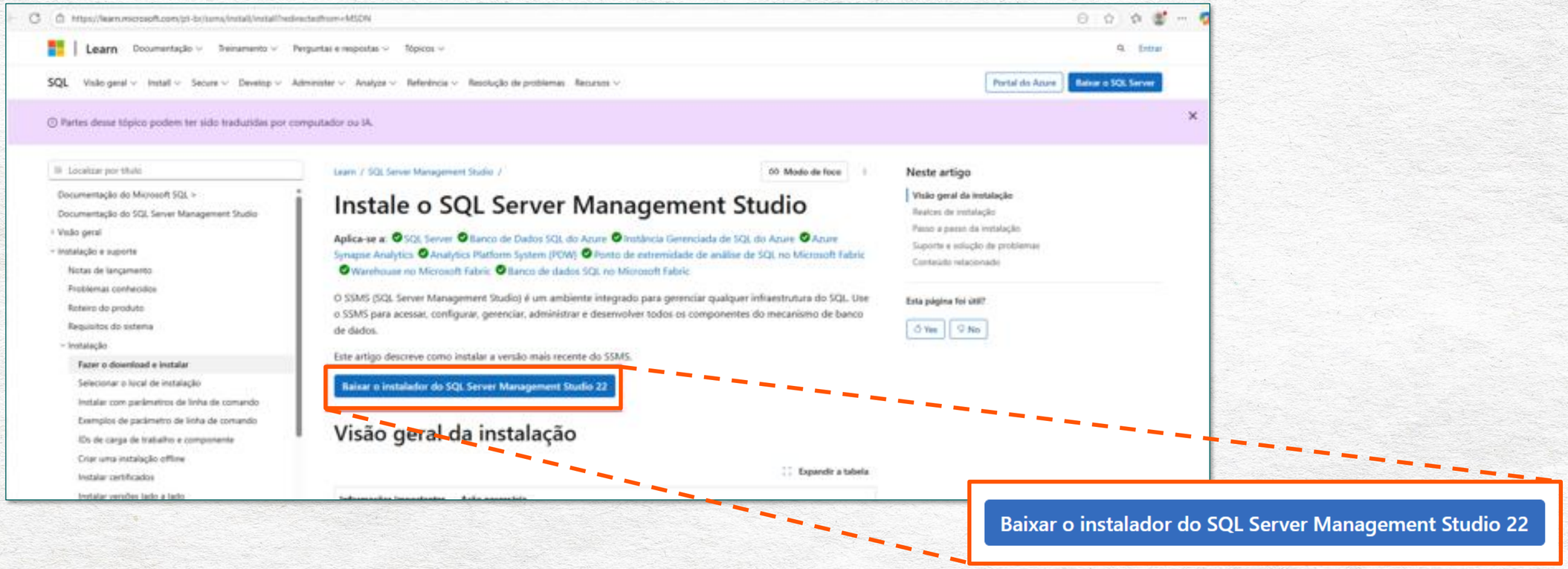
Instalando o SQL Server Management Studio (SSMS)

Na janela de sucesso da instalação do SQL Server (da etapa anterior), clique no botão **Install SSMS**:



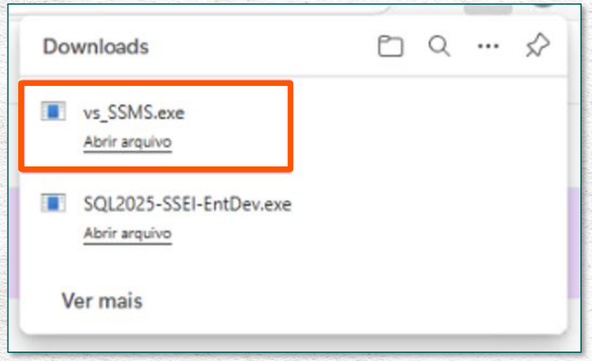
Instalando o SQL Server Management Studio (SSMS)

Você será levado ao site da Microsoft. Clique no botão azul **"Baixar o instalador do SQL Server Management Studio 22"**:



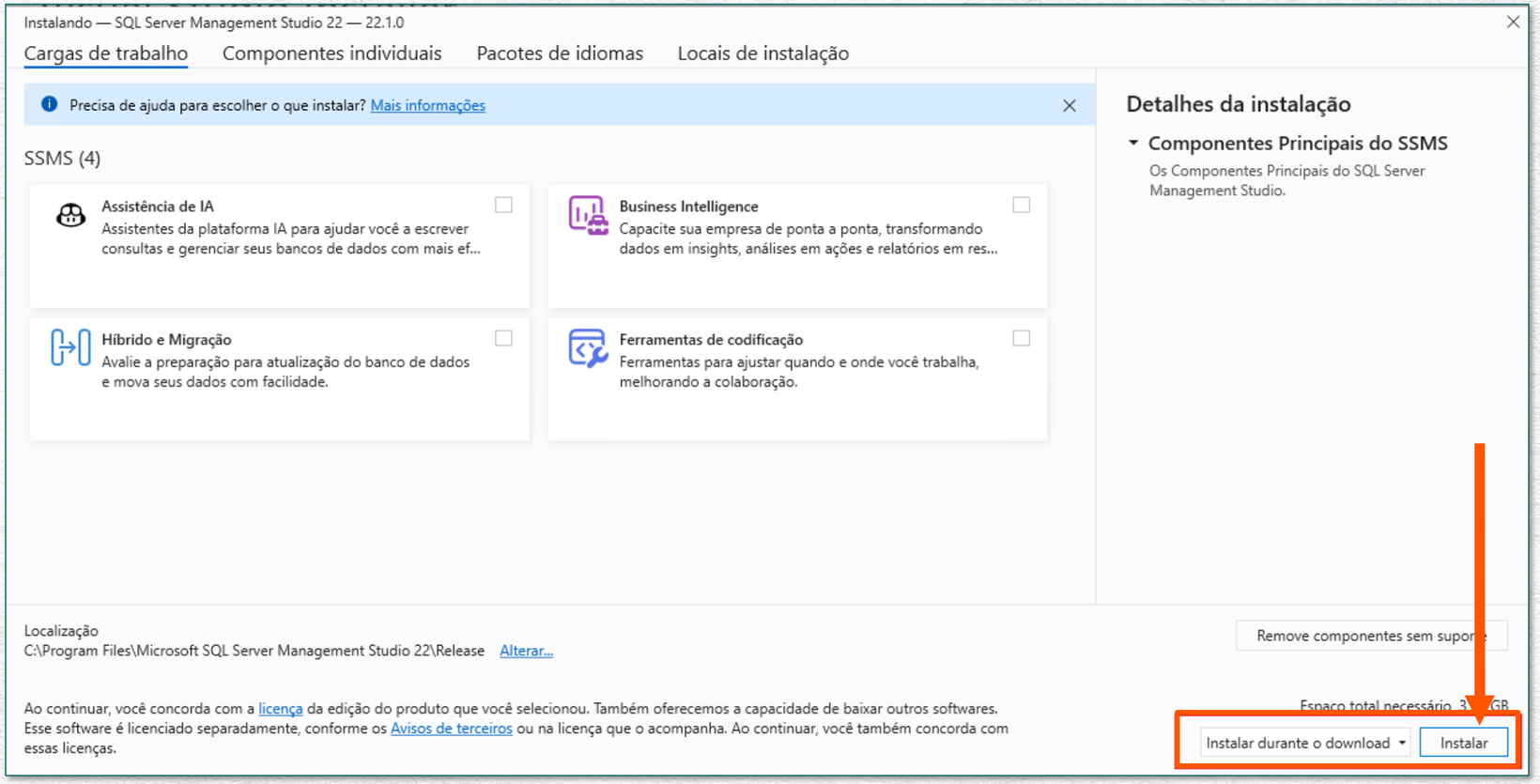
Instalando o SQL Server Management Studio (SSMS)

Execute o arquivo baixado (**vs_SSMS.exe**):



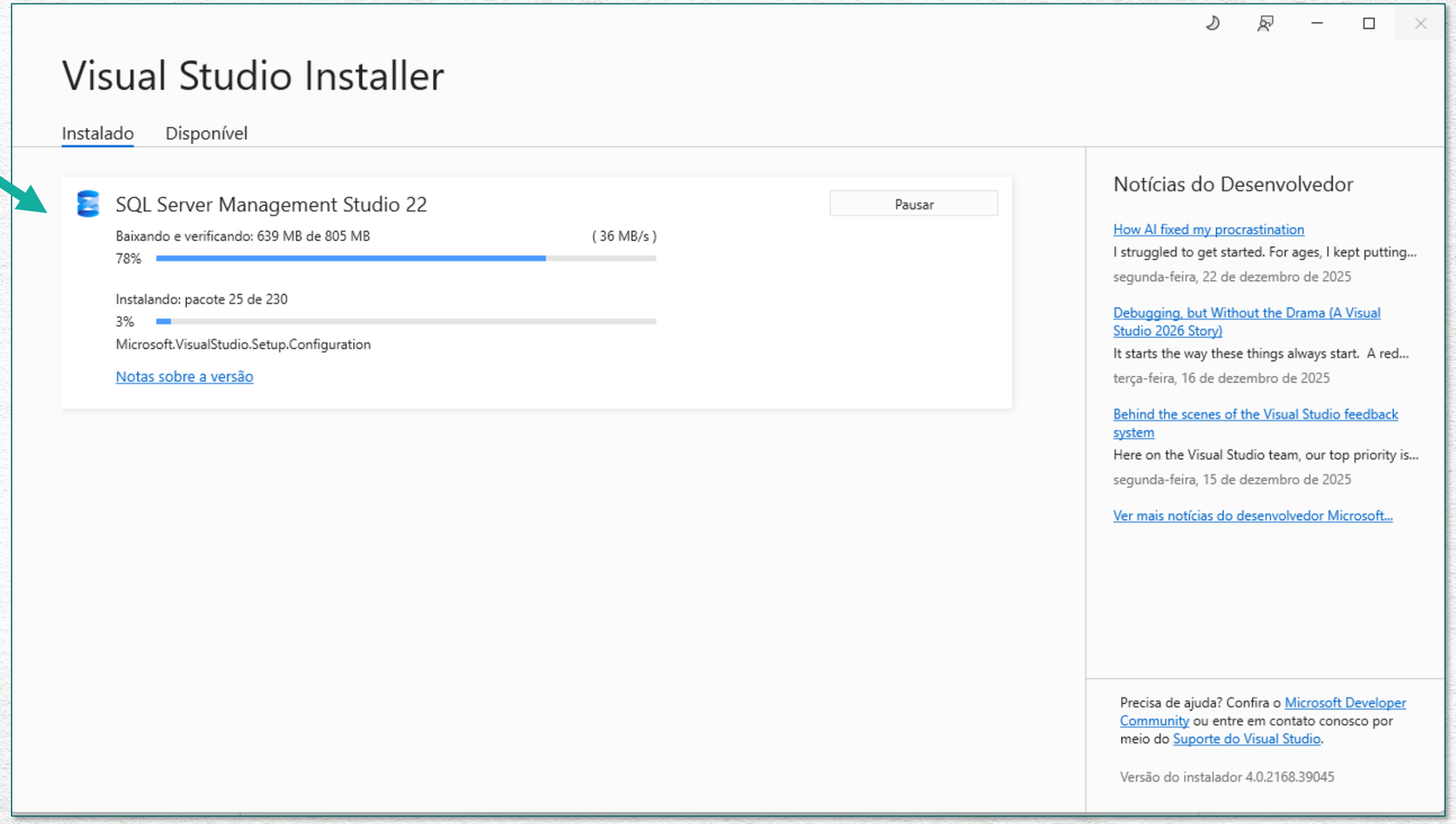
Obs.: O instalador agora usa a interface do **Visual Studio Installer**.

Uma janela se abrirá mostrando os componentes. Apenas clique no botão **Instalar** no canto inferior direito da tela:



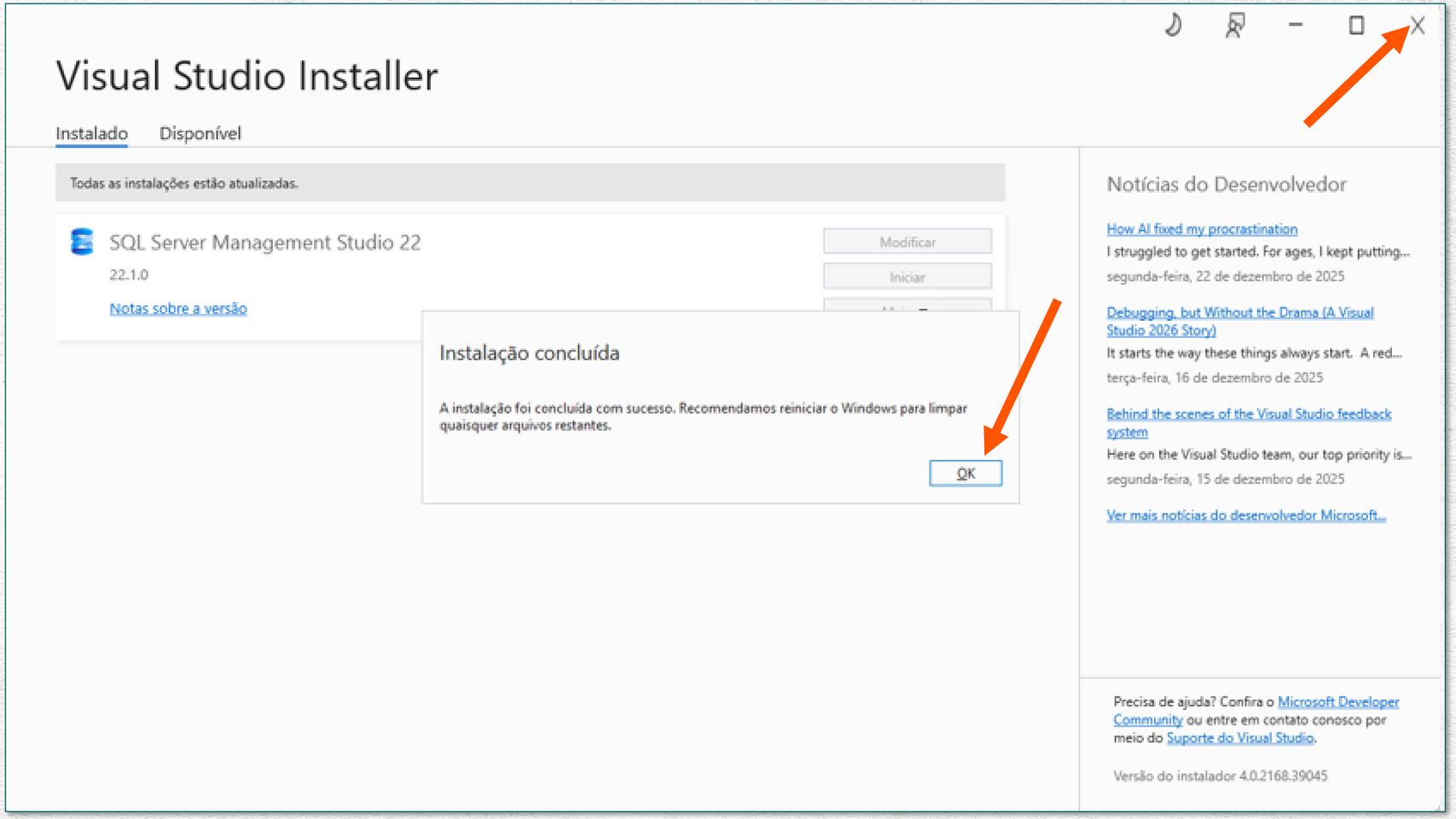
Instalando o SQL Server Management Studio (SSMS)

Aguarde o término da instalação...



Instalando o SQL Server Management Studio (SSMS)

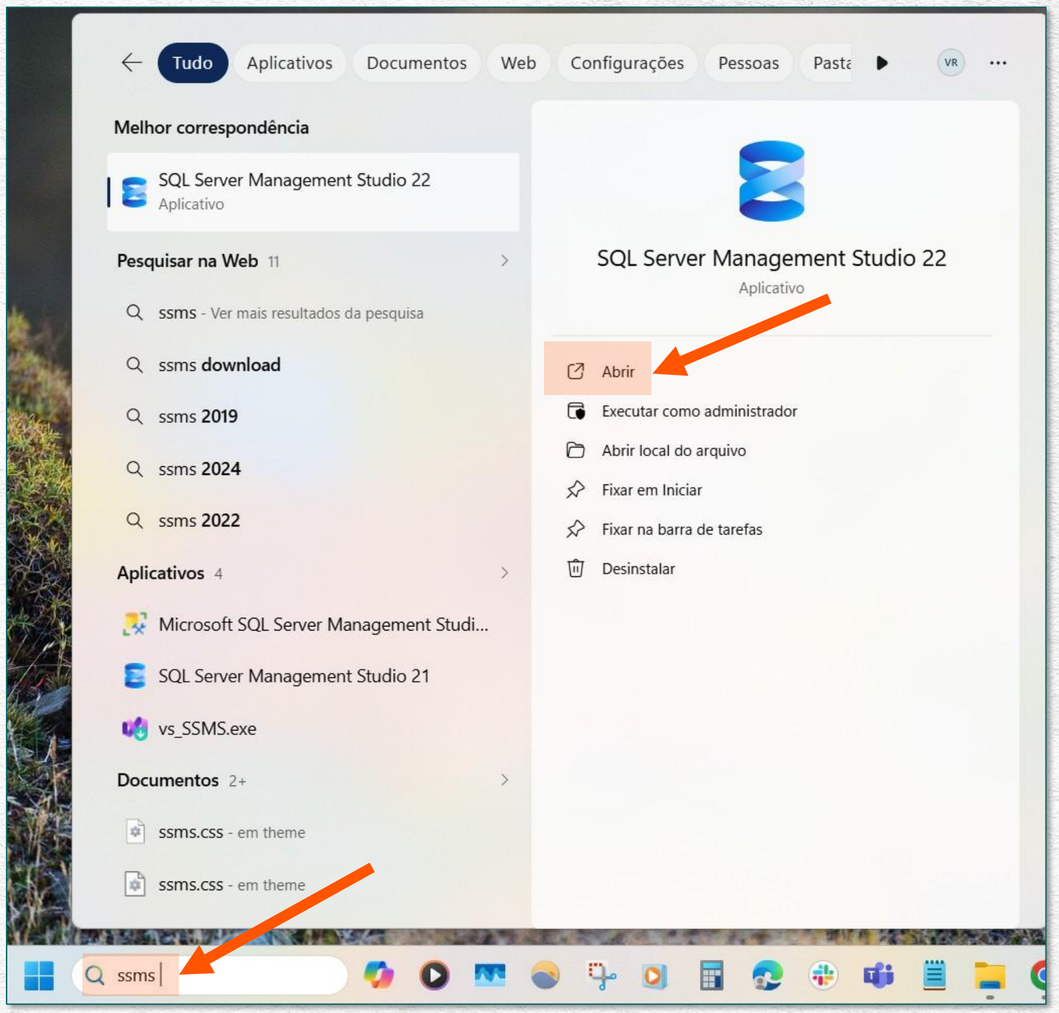
Quando aparecer a mensagem de **"Instalação concluída"**, clique em **OK** e, em seguida, **feche** a janela do instalador:



Instalando o SQL Server Management Studio (SSMS)

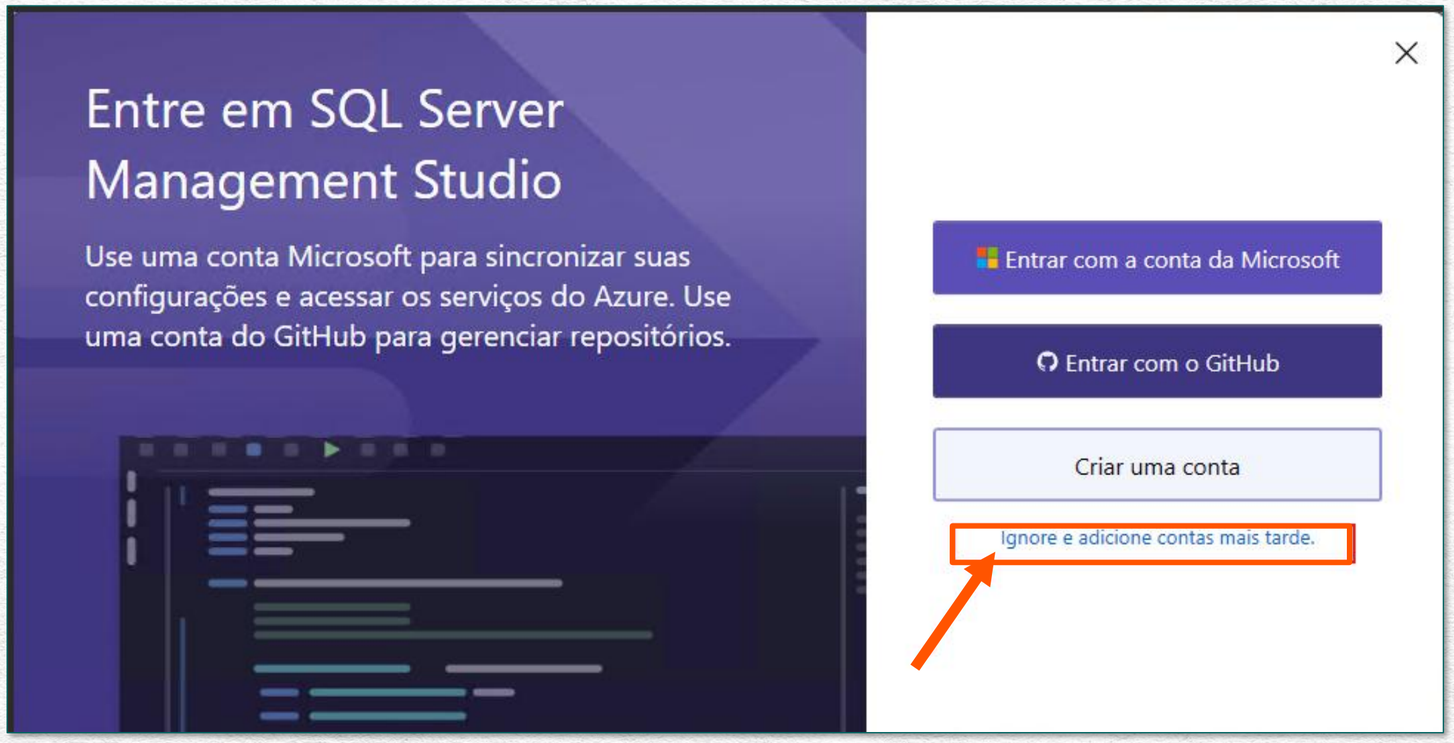
Agora podemos conectar o SSMS (**interface**) ao SQL Server (**servidor**) pela primeira vez.

Para isso, no menu Iniciar do Windows, procure por **"SSMS"** ou **"SQL Server Management Studio 22"** e abra o aplicativo:



Instalando o SQL Server Management Studio (SSMS)

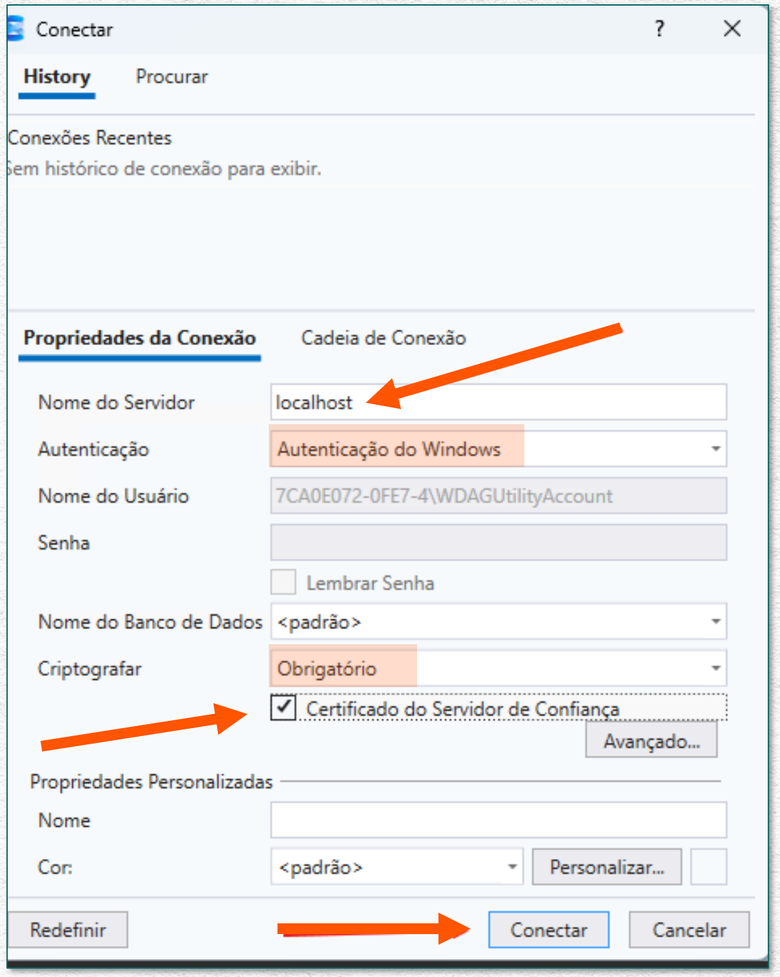
Na primeira abertura, pode aparecer uma janela pedindo para entrar com uma conta Microsoft ou GitHub. Se isso acontecer, clique no link inferior: **"Ignore e adicione contas mais tarde"**:



Instalando o SQL Server Management Studio (SSMS)

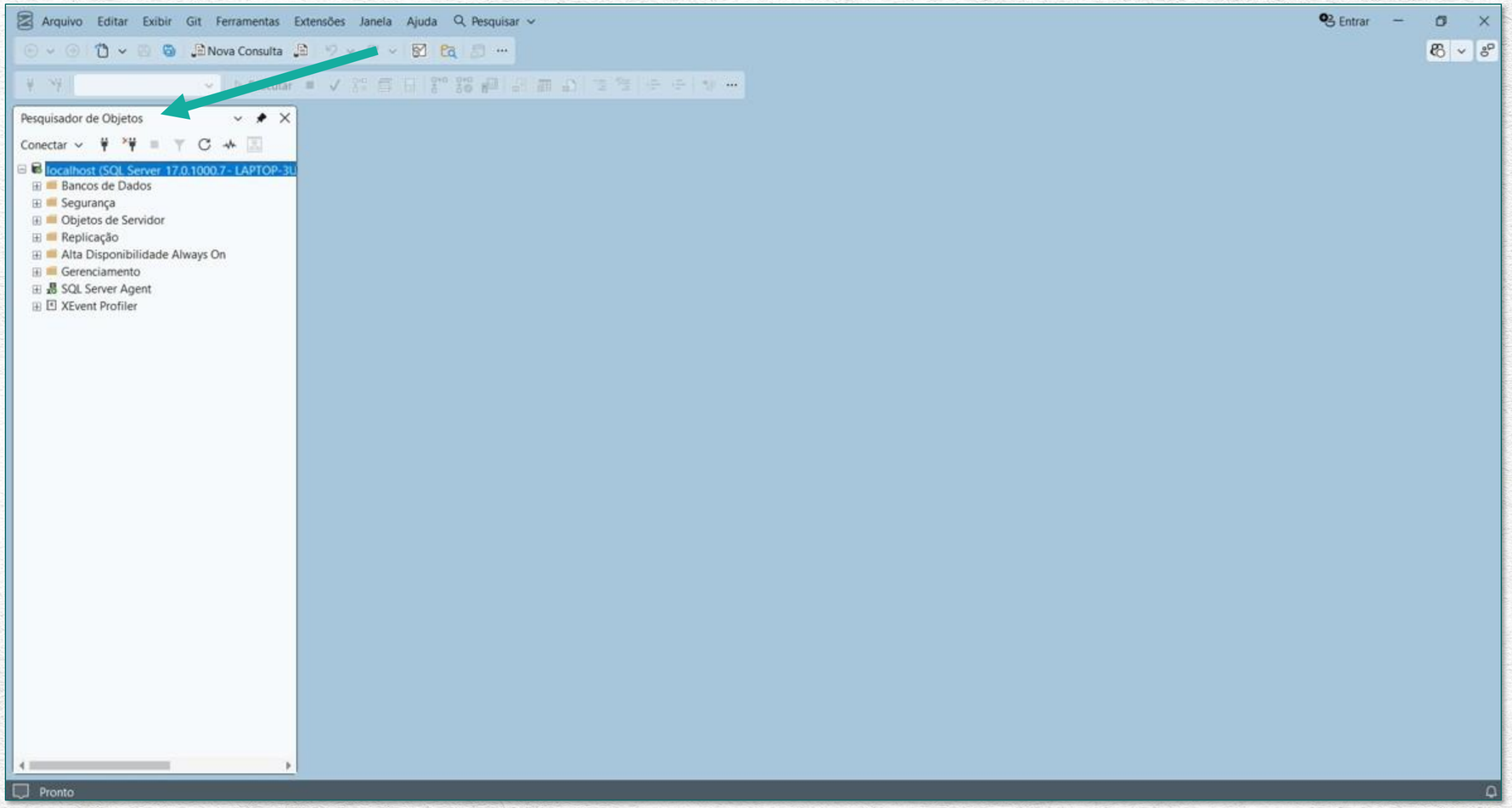
A janela de conexão se abrirá. Configure exatamente assim:

- **Nome do Servidor:** digite **localhost**;
- **Autenticação:** mantenha selecionada a opção **Autenticação do Windows**;
- **Criptografar:** mantenha selecionada a opção **Obrigatório**;
- **Importante:** Marque a caixinha **"Certificado do Servidor de Confiança"** (Trust Server Certificate). **Sem isso, a conexão pode falhar**;
- Clique em **Conectar**.



Instalando o SQL Server Management Studio (SSMS)

Pronto! Ao conectar, você verá o **"Pesquisador de Objetos"** à esquerda da tela:



Seu SQL Server está instalado, e agora podemos seguir para a próxima aula para aprendermos como **restaurar o banco de dados Contoso**, que utilizaremos ao longo dos módulos de SQL Server do curso. ☺

TEMA 11

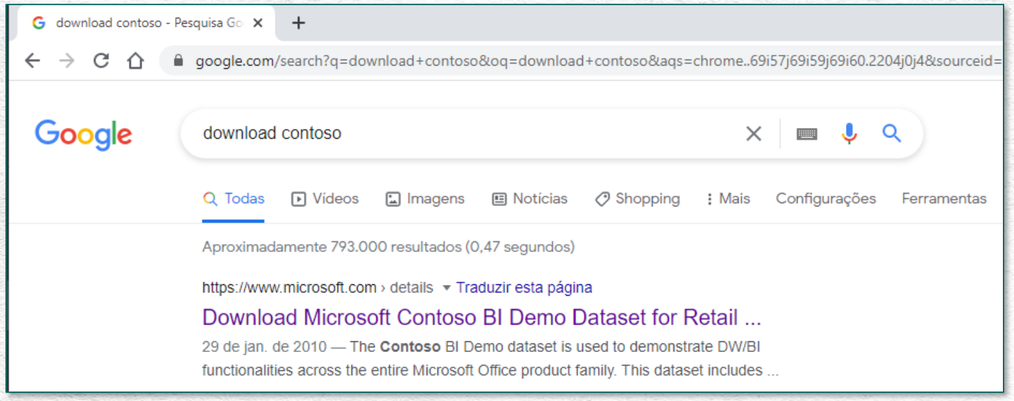
Download do BD Contoso

Download do BD Contoso

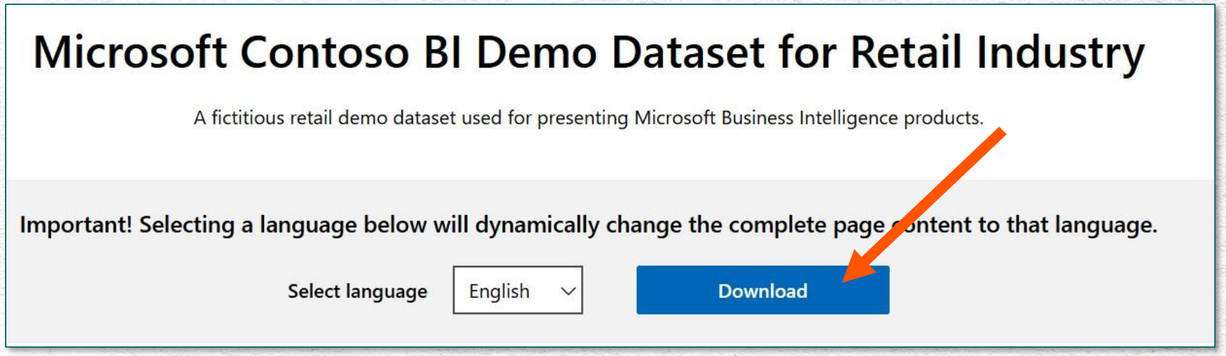
Download do BD Contoso

Download do BD Contoso e abrindo no SSMS

Para fazer o download do BD Contoso, basta procurar no Google por "**download contoso**". Clique no link mostrado abaixo (geralmente é o primeiro que aparece):



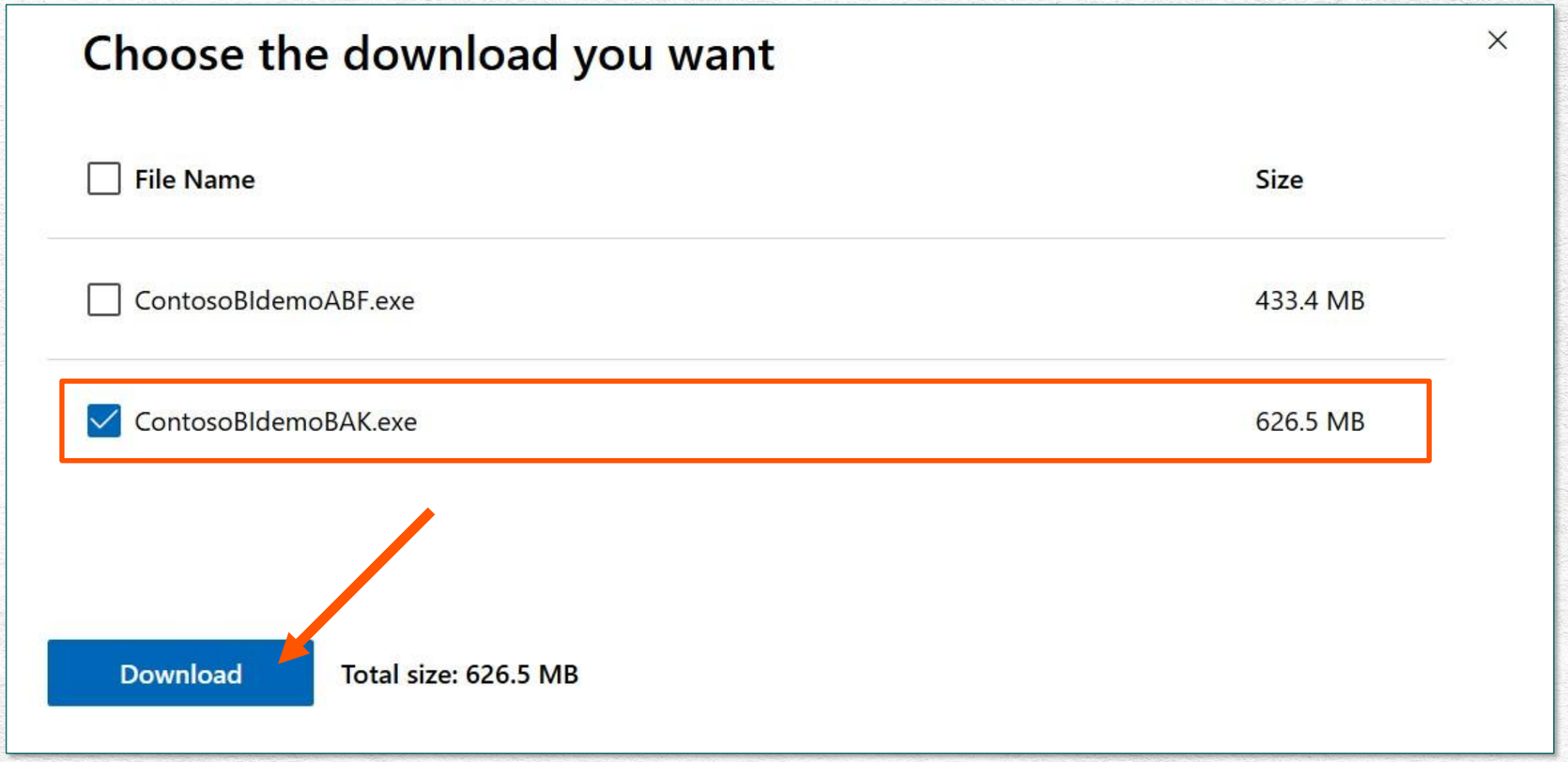
Na página que se abrir, clique em **Download**:



Download do BD Contoso e abrindo no SSMS

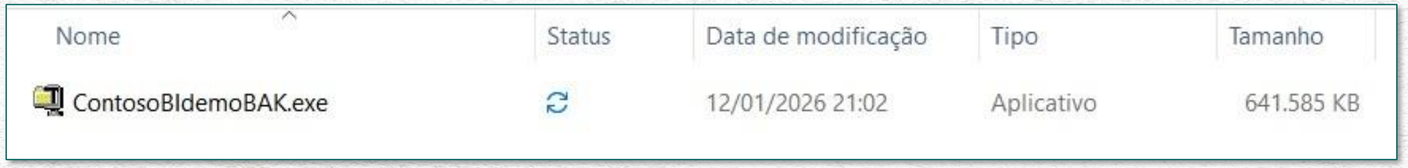
O arquivo a ser baixado deve ser o **ContosoBIdemoBAK.exe**.

Selecione-o e clique em **Download** novamente:



Download do BD Contoso e abrindo no SSMS

Descompacte o arquivo ContosoBIdemoBAK.exe baixado (para isso, use um descompactador. Pode ser o padrão do Windows ou algum de sua preferência, como o WinRar ou 7-Zip):

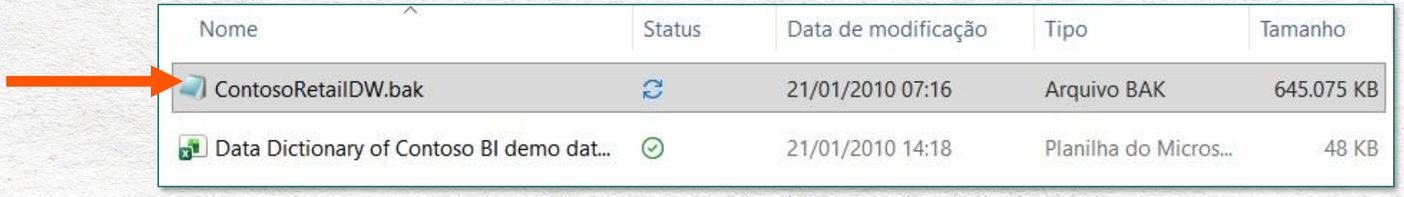


ARQUIVO BAIXADO COMPACTADO

Ao descompactar, aparecerá uma **pasta** no seu computador com o mesmo nome do arquivo que você descompactou:

PASTA DESCOMPACTADA QUE CONTÉM O ARQUIVO .BAK

Dentro dela, você encontrará dois arquivos. Utilizaremos no SSMS o arquivo **ContosoRetailDW.bak** indicado abaixo:

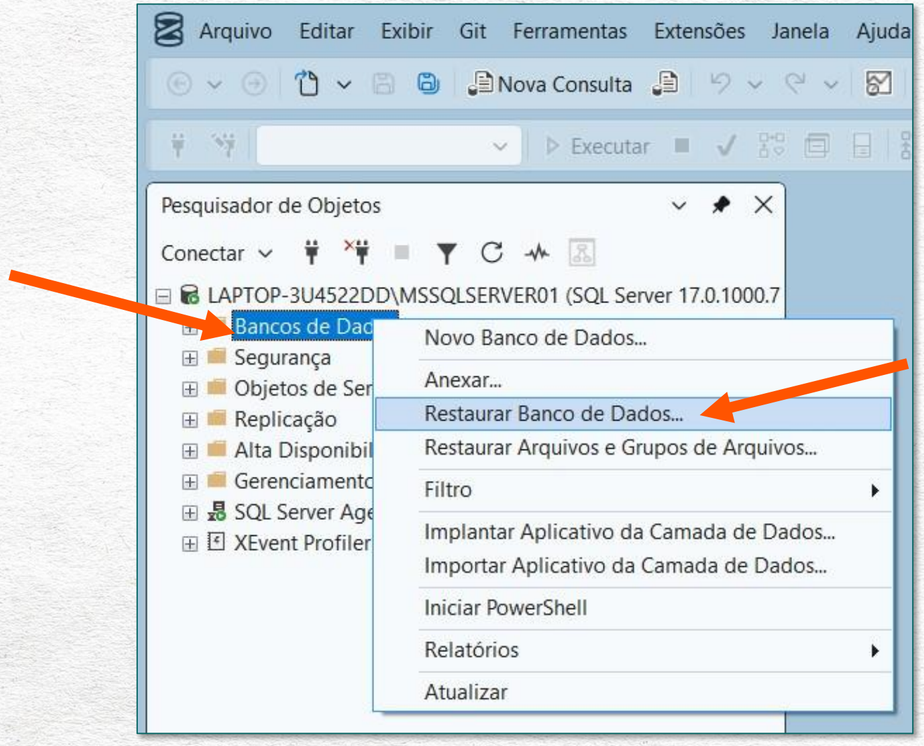


ARQUIVO NO FORMATO .BAK QUE UTILIZAREMOS NO SSMS PARA IMPORTAR O CONTOSO

Índice Geral
Índice deste Módulo

Download do BD Contoso e abrindo no SSMS

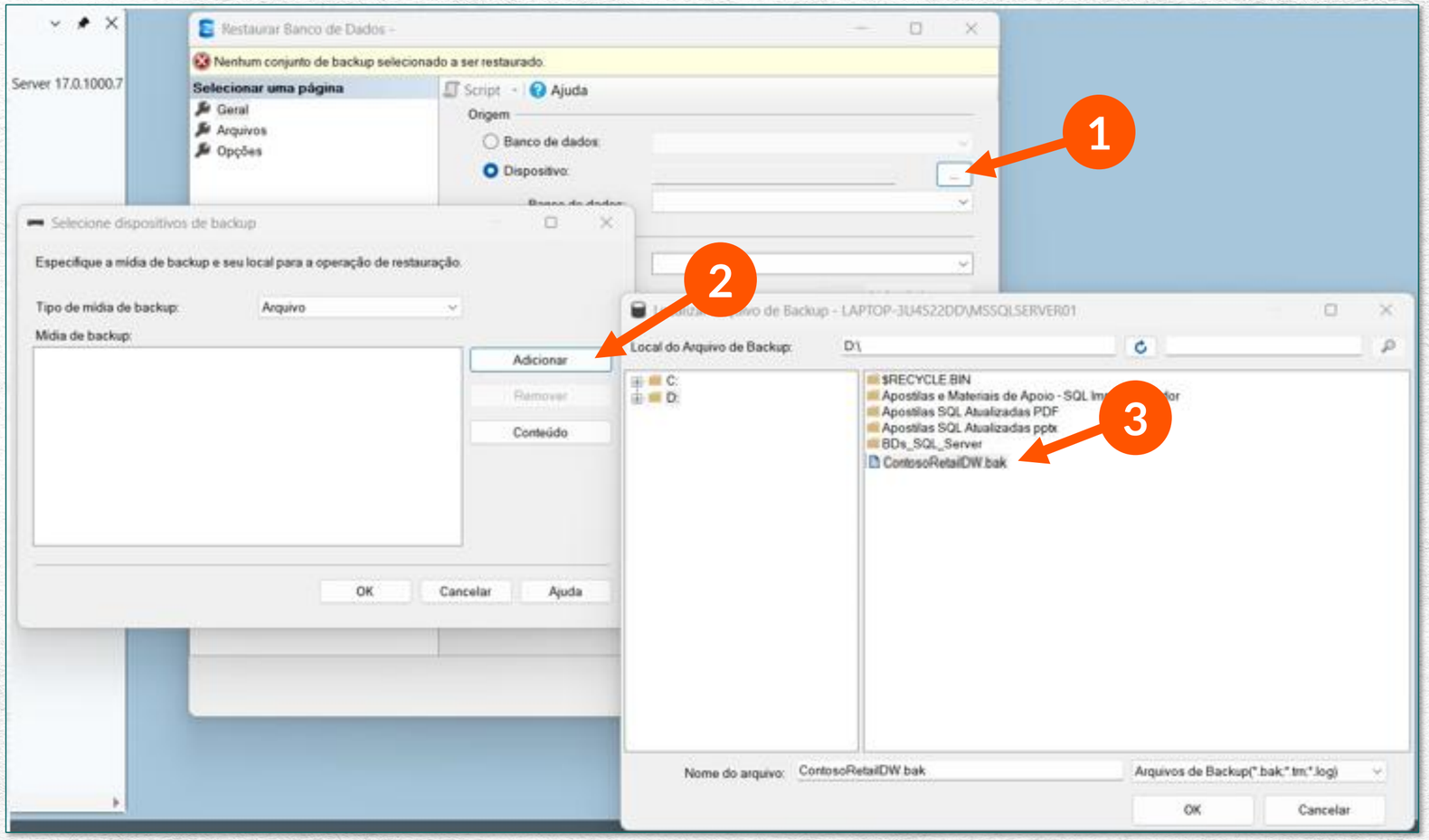
Com o SSMS já conectado, para importar o BD Contoso, clique com o botão direito em **Banco de Dados > Restaurar Banco de Dados...**



Download do BD Contoso e abrindo no SSMS

Na janela que se abrir, troque a Origem para '**Dispositivo**' e clique nos **três pontinhos**.

Na janela seguinte, clique em **Adicionar** e, na próxima, procure o **arquivo BAK do Contoso** dentro da pasta onde você o salvou:

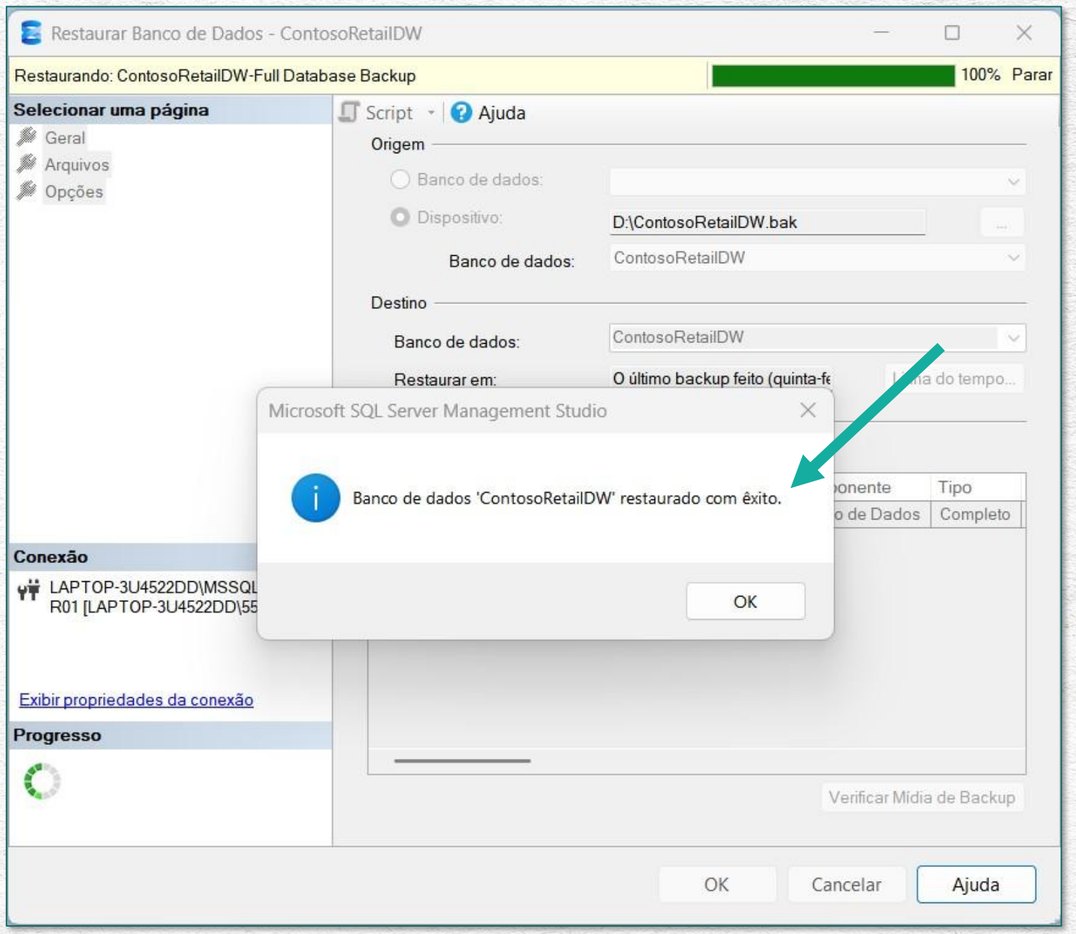


Índice Geral

Índice deste Módulo

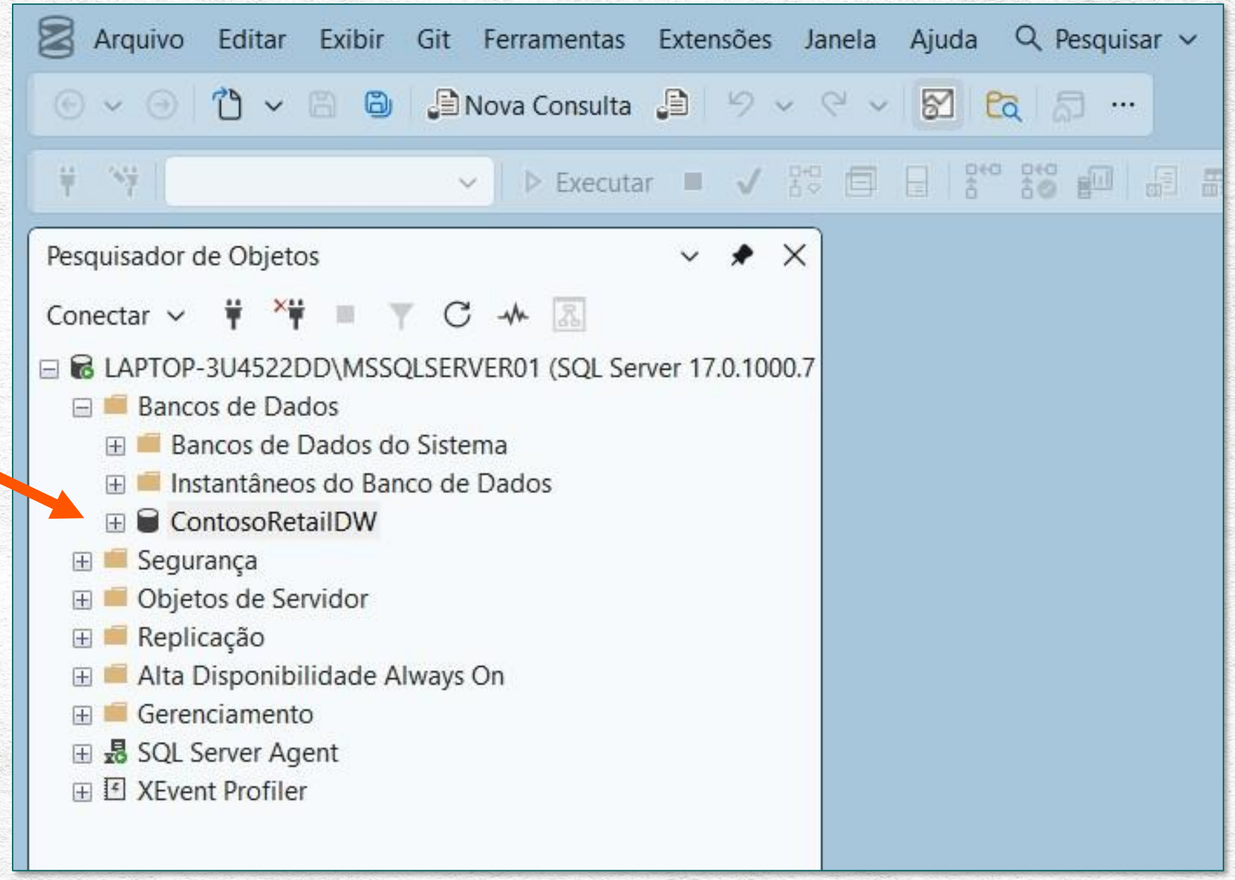
Download do BD Contoso e abrindo no SSMS

Depois é só clicar em **OK** em todas as janelas abertas até aparecer a mensagem de restauração efetuada com êxito:



Download do BD Contoso e abrindo no SSMS

Pronto! O banco de dados Contoso foi restaurado e já pode ser acessado no SSMS:



TEMA 12

Conhecendo o BD Contoso

Conhecendo o BD Contoso

Conhecendo o BD Contoso

Conhecendo o BD Contoso

Vamos conhecer um pouco mais sobre a Contoso.

- A empresa Contoso Corporation é uma empresa multinacional (fictícia) com sede em Paris e com escritórios espalhados ao longo de todo o mundo.
- A empresa é uma organização de fabricação, vendas e suporte, com mais de **100.000 produtos**.
- Por ser uma empresa multinacional, utiliza um SGBD da Microsoft (SQL Server) para centralizar e manipular o grande volume de dados associado ao negócio.
- Possui mais de **25 mil funcionários**, espalhados ao longo das dezenas de escritórios no mundo.

Conhecendo o BD Contoso

O Banco de Dados possui uma série de informações, divididas em diferentes tabelas. As principais são destacadas abaixo:

dimCustomer

Tabela contendo informações dos **clientes**, tais como: Primeiro Nome, Data de Nascimento, Email, etc.

dimEmployee

Tabela contendo informações dos **funcionários**, tais como: Primeiro Nome, Cargo, Departamento, etc.

dimProduct

Tabela contendo informações dos **produtos**, tais como: Nome do Produto, Marca, Cor, etc.

dimStore

Tabela contendo informações das **lojas**, tais como: Nome, Data de Abertura, Contato, etc.

factSales

Tabela contendo informações de **vendas**, tais como: Produto vendido, Quantidade, Preço, etc.

factOnlineSales

Tabela contendo informações de **vendas online**, muito semelhante à tabela factSales.

TEMA 13

Explorando as bases de dados

Explorando as bases de dados

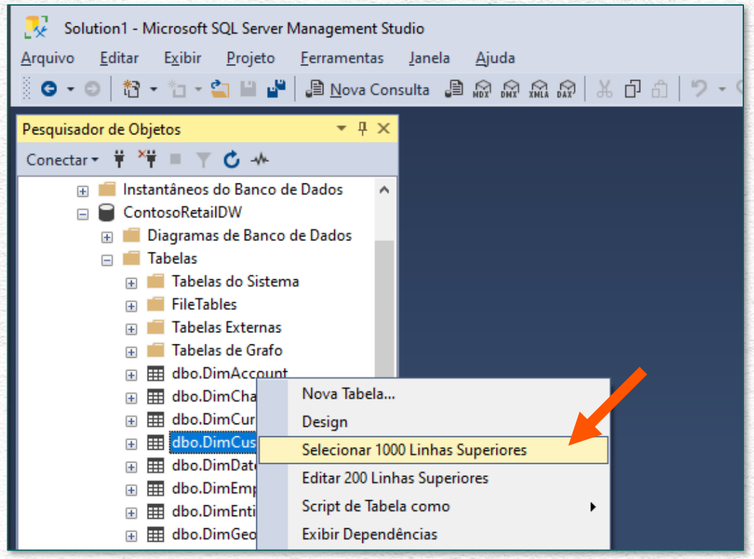
Explorando as bases de dados

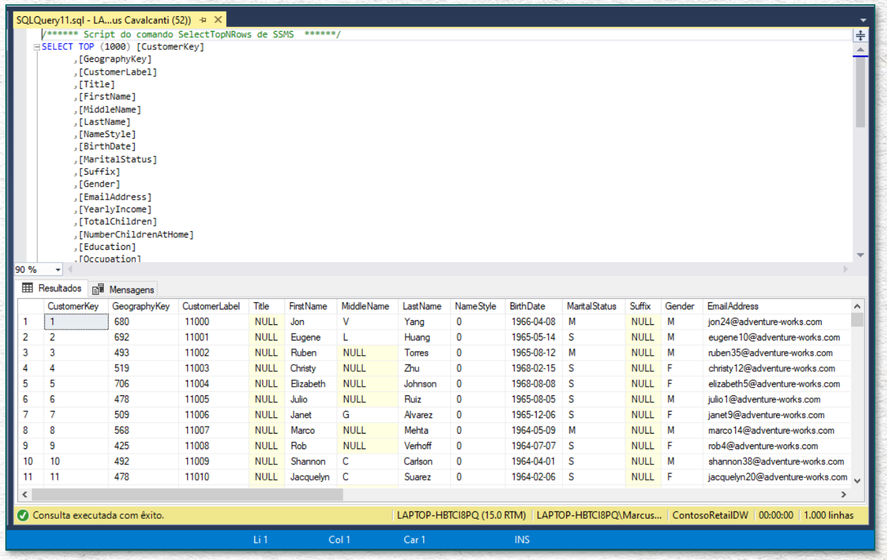
Como explorar as bases de dados pela primeira vez

Sempre que você for trabalhar em cima de bancos de dados, antes de criar qualquer código SQL, tenha o bom costume de explorar os dados à sua disposição.

A maneira mais simples de fazer isso é clicar com o botão direito em cada tabela e escolher a opção 'Selecionar 1000 linhas superiores'.

Isso vai te dar uma visão dos dados de cada tabela. Essa prática é importante para você conhecer os dados de uma tabela quando você está trabalhando com Bancos de Dados pela primeira vez.





MÓDULO 3

SQL

Introdução ao SQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 3 INTRODUÇÃO AO SQL SERVER

01	O comando SELECT	071
02	O comando SELECT *	072
03	O comando SELECT TOP(N)	073
04	O comando SELECT DISTINCT	074
05	Comentando o código	075
06	AS: Renomeando (aliasing) colunas	076
07	EXERCÍCIOS	077
08	GABARITOS	080

O comando SELECT

O SELECT é um comando para selecionar dados de uma tabela.

Ao lado, temos a estrutura para selecionar apenas colunas específicas da tabela.

Seleciona uma única coluna:

```
SQLQuery1.sql - LAP...us Cavalcanti (55))*
-- Seleciona a coluna FirstName
-- da tabela dimCustomer
SELECT FirstName
FROM dimCustomer
```

Resultados | Mensagens

	FirstName
1	Jon
2	Eugene
3	Ruben
4	Christy
5	Elizabeth
6	Julio
7	Janet
8	Marco
9	Rob
10	Shannon
11	Jacquelyn
12	Curtis

Seleciona mais de uma coluna:

```
SQLQuery1.sql - LAP...us Cavalcanti (55))*
-- Seleciona a coluna
-- FirstName da tabela dimCustomer
SELECT FirstName, LastName, EmailAddress
FROM dimCustomer
```

	FirstName	LastName	EmailAddress
1	Jon	Yang	jon24@adventure-works.com
2	Eugene	Huang	eugene10@adventure-works.com
3	Ruben	Torres	ruben35@adventure-works.com
4	Christy	Zhu	christy12@adventure-works.com
5	Elizabeth	Johnson	elizabeth5@adventure-works.com
6	Julio	Ruiz	julio1@adventure-works.com
7	Janet	Alvarez	janet9@adventure-works.com
8	Marco	Mehta	marco14@adventure-works.com
9	Rob	Verhoff	rob4@adventure-works.com
10	Shannon	Carlson	shannon38@adventure-works.com
11	Jacquelyn	Suarez	jacquelyn20@adventure-works.com
12	Curtis	Lu	curtis9@adventure-works.com

O comando SELECT*

O comando SELECT * permite selecionar TODAS as colunas de uma determinada tabela. Não é muito recomendável para tabelas muito grandes. O objetivo deste comando é ter uma rápida visualização de toda a tabela.

Será melhor usado em conjunto com o comando TOP.

Seleciona toda a tabela:

```
SQLQuery1.sql - LAP...us Cavalcanti (55))*
-- Seleciona todas as linhas e colunas
-- da tabela dimCustomer
SELECT *
FROM dimCustomer
```

Resultados | Mensagens

	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	1	680	11000	NULL	Jon	V	Yang	0	1966-04-08	M	NULL	M	jon24@adventure-works.com
2	2	692	11001	NULL	Eugene	L	Huang	0	1965-05-14	S	NULL	M	eugene10@adventure-works.c
3	3	493	11002	NULL	Ruben	NULL	Torres	0	1965-08-12	M	NULL	M	ruben35@adventure-works.co
4	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.cc
5	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.c
6	6	478	11005	NULL	Julio	NULL	Ruiz	0	1965-08-05	S	NULL	M	julio1@adventure-works.com
7	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com
8	8	568	11007	NULL	Marco	NULL	Mehta	0	1964-05-09	M	NULL	M	marco14@adventure-works.co
9	9	425	11008	NULL	Rob	NULL	Verhoff	0	1964-07-07	S	NULL	F	rob4@adventure-works.com
10	10	492	11009	NULL	Shannon	C	Carlson	0	1964-04-01	S	NULL	M	shannon38@adventure-works.
11	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F	jacquelyn20@adventure-works

O comando SELECT TOP(N)

O comando SELECT TOP(N) permite que a gente selecione apenas as N primeiras linhas de uma tabela.

Selecionar as TOP(N) linhas da tabela:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Retorna as 10 primeiras
-- linhas da tabela
SELECT TOP(10) ProductName
FROM dimProduct
```

Resultados | Mensagens

	ProductName
1	Contoso 512MB MP3 Player E51 Silver
2	Contoso 512MB MP3 Player E51 Blue
3	Contoso 1G MP3 Player E100 White
4	Contoso 2G MP3 Player E200 Silver
5	Contoso 2G MP3 Player E200 Red
6	Contoso 2G MP3 Player E200 Black
7	Contoso 2G MP3 Player E200 Blue
8	Contoso 4G MP3 Player E400 Silver
9	Contoso 4G MP3 Player E400 Black
10	Contoso 4G MP3 Player E400 Green

Selecionar as TOP(N) % linhas da tabela :

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Retorna as 10% primeiras linhas da tabela.
-- Se a tabela tem 100 linhas, ele retorna
-- as 10 primeiras
SELECT TOP(10) PERCENT ProductName
FROM dimProduct
```

Resultados | Mensagens

	ProductName
1	Contoso 512MB MP3 Player E51 Silver
2	Contoso 512MB MP3 Player E51 Blue
3	Contoso 1G MP3 Player E100 White
4	Contoso 2G MP3 Player E200 Silver
5	Contoso 2G MP3 Player E200 Red
6	Contoso 2G MP3 Player E200 Black
7	Contoso 2G MP3 Player E200 Blue
8	Contoso 4G MP3 Player E400 Silver
9	Contoso 4G MP3 Player E400 Black
10	Contoso 4G MP3 Player E400 Green
11	Contoso 4G MP3 Player E400 Orange
12	Contoso 4GB Flash MP3 Player E401 Blue

252 linhas

O comando SELECT DISTINCT

O comando SELECT DISTINCT retorna os valores distintos de uma tabela.

Retorna todas as linhas da tabela:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Retorna todas as linhas
-- da coluna ColorName
SELECT ColorName
FROM dimProduct
```

Resultados | Mensagens

	ColorName
1	Silver
2	Blue
3	White
4	Silver
5	Red
6	Black
7	Blue
8	Silver
9	Black
10	Green
11	Orange
12	Blue

Retorna valores repetidos!

Retorna apenas as cores únicas da tabela:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Retorna apenas os valores
-- distintos da coluna ColorName
SELECT DISTINCT ColorName
FROM dimProduct
```

Resultados | Mensagens

	ColorName
1	Purple
2	Azure
3	Transparent
4	Silver
5	Green
6	Pink
7	Grey
8	Red
9	Brown
10	Gold
11	Black
12	Blue

Nenhum valor repetido!

Comentando o código

Comentários são uma boa prática para garantir um bom entendimento de um código.

Conforme vamos criando consultas cada vez mais complexas, os comentários podem ser muito úteis para ajudar no entendimento do que está sendo feito.

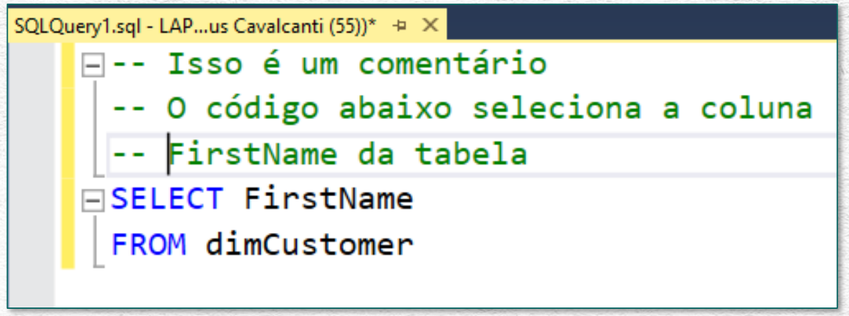
Existem duas formas de comentar códigos:

Na opção 1), utilizamos o hífen duplo para comentar uma única linha de código.

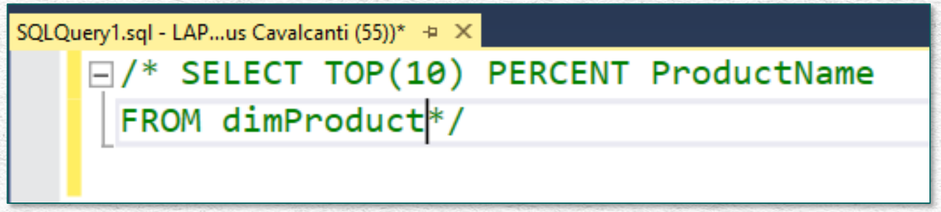
Já na opção 2), utilizamos uma barra seguida de um asterisco para identificar onde começa um comentário, e utilizamos o asterisco seguido de uma barra para identificar onde termina um comentário.

Ao lado, um exemplo de cada situação.

(1) O -- comenta linha por linha do código



(2) O /* ... */ comenta mais de 1 linha por vez



AS: Renomeando (aliasing) colunas

Uma coluna de uma tabela pode ser renomeada por meio do comando AS. O nome da nova coluna pode ser escrito sem as aspas simples, apenas se for um texto único. Se precisar renomear com um nome composto, utilize as aspas simples.

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```sql
-- Retorna 3 colunas da tabela dimProduct,
-- e renomeia cada uma delas
SELECT
ProductName AS Produto,
BrandName AS Marca,
ColorName AS Cor
FROM DimProduct
```

Resultados | Mensagens

	Produto	Marca	Cor
1	Contoso 512MB MP3 Player E51 Silver	Contoso	Silver
2	Contoso 512MB MP3 Player E51 Blue	Contoso	Blue
3	Contoso 1G MP3 Player E100 White	Contoso	White
4	Contoso 2G MP3 Player E200 Silver	Contoso	Silver
5	Contoso 2G MP3 Player E200 Red	Contoso	Red
6	Contoso 2G MP3 Player E200 Black	Contoso	Black
7	Contoso 2G MP3 Player E200 Blue	Contoso	Blue
8	Contoso 4G MP3 Player E400 Silver	Contoso	Silver
9	Contoso 4G MP3 Player E400 Black	Contoso	Black
10	Contoso 4G MP3 Player E400 Green	Contoso	Green
11	Contoso 4G MP3 Player E400 Orange	Contoso	Orange
12	Contoso 4GB Flash MP3 Player E401 Blue	Contoso	Blue

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```sql
-- Retorna 3 colunas da tabela dimProduct,
-- e renomeia cada uma delas
SELECT
ProductName AS 'Nome Produto',
BrandName AS 'Nome Marca',
ColorName AS 'Nome Cor'
FROM DimProduct
```

	Nome Produto	Nome Marca	Nome Cor
1	Contoso 512MB MP3 Player E51 Silver	Contoso	Silver
2	Contoso 512MB MP3 Player E51 Blue	Contoso	Blue
3	Contoso 1G MP3 Player E100 White	Contoso	White
4	Contoso 2G MP3 Player E200 Silver	Contoso	Silver
5	Contoso 2G MP3 Player E200 Red	Contoso	Red
6	Contoso 2G MP3 Player E200 Black	Contoso	Black
7	Contoso 2G MP3 Player E200 Blue	Contoso	Blue
8	Contoso 4G MP3 Player E400 Silver	Contoso	Silver
9	Contoso 4G MP3 Player E400 Black	Contoso	Black
10	Contoso 4G MP3 Player E400 Green	Contoso	Green
11	Contoso 4G MP3 Player E400 Orange	Contoso	Orange
12	Contoso 4GB Flash MP3 Player E401 Blue	Contoso	Blue

EXERCÍCIOS

1

Questão 1

Você é responsável por controlar os dados de clientes e de produtos da sua empresa. O que você precisará fazer é confirmar se:

a. Existem 2.517 produtos cadastrados na base e, se não tiver, você deverá reportar ao seu gestor para saber se existe alguma defasagem no controle dos produtos.

b. Até o mês passado, a empresa tinha um total de 19.500 clientes na base de controle. Verifique se esse número aumentou ou reduziu.

2

Questão 2

Você trabalha no setor de marketing da empresa Contoso e acaba de ter uma ideia de oferecer descontos especiais para os clientes no dia de seus aniversários. Para isso, você vai precisar listar todos os clientes e as suas respectivas datas de nascimento, além de um contato.

a. Selecione as colunas: CustomerKey, FirstName, EmailAddress, BirthDate da tabela dimCustomer.

b. Renomeie as colunas dessa tabela usando o alias (comando AS).

EXERCÍCIOS

3

Questão 3

A Contoso está comemorando aniversário de inauguração de 10 anos e pretende fazer uma ação de premiação para os clientes. A empresa quer presentear os primeiros clientes desde a inauguração.

Você foi alocado para levar adiante essa ação. Para isso, você terá que fazer o seguinte:

a. A Contoso decidiu presentear os primeiros 100 clientes da história com um vale compras de R$ 10.000. Utilize um comando em SQL para retornar uma tabela com os primeiros 100 primeiros clientes da tabela dimCustomer (selecione todas as colunas).

b. A Contoso decidiu presentear os primeiros 20% de clientes da história com um vale compras de R$ 2.000. Utilize um comando em SQL para retornar 10% das linhas da sua tabela dimCustomer (selecione todas as colunas).

c. Adapte o código do item a) para retornar apenas as 100 primeiras linhas, mas apenas as colunas FirstName, EmailAddress, BirthDate.

d. Renomeie as colunas anteriores para nomes em português

EXERCÍCIOS

4

Questão 4

A empresa Contoso precisa fazer contato com os fornecedores de produtos para repor o estoque. Você é da área de compras e precisa descobrir quem são esses fornecedores.

Utilize um comando em SQL para retornar apenas os nomes dos fornecedores na tabela dimProduct e renomeie essa nova coluna da tabela.

5

Questão 5

O seu trabalho de investigação não para. Você precisa descobrir se existe algum produto registrado na base de produtos que ainda não tenha sido vendido. Tente chegar nessa informação.

Obs: caso tenha algum produto que ainda não tenha sido vendido, você não precisa descobrir qual é, é suficiente saber se teve ou não algum produto que ainda não foi vendido.

GABARITOS

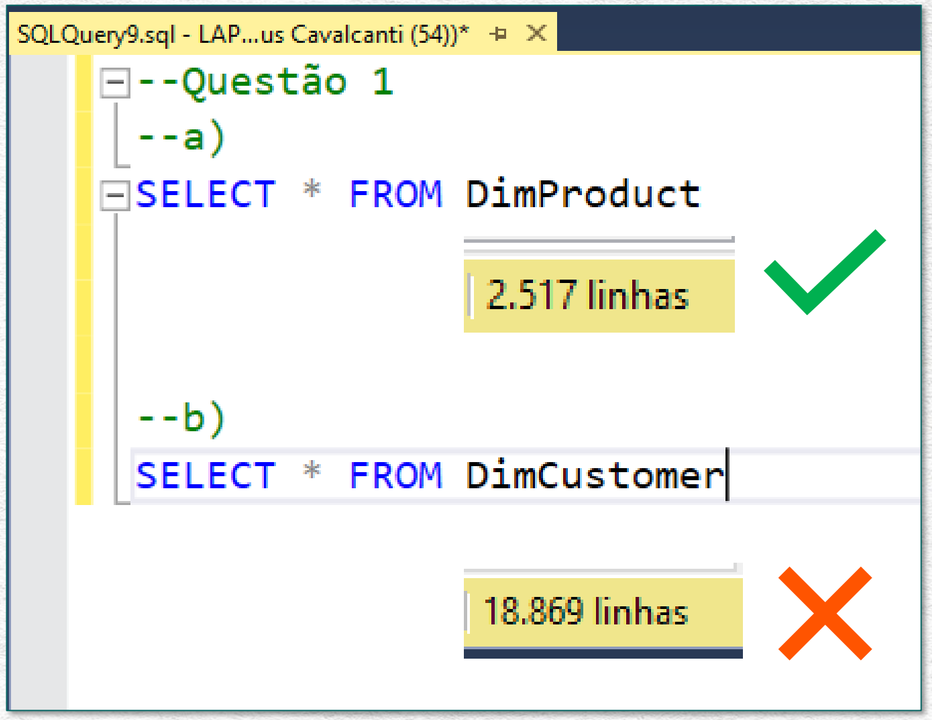
1

Questão 1

Você é responsável por controlar os dados de clientes e de produtos da sua empresa. O que você precisará fazer é confirmar se:

a. Existem 2.517 produtos cadastrados na base e, se não tiver, você deverá reportar ao seu gestor para saber se existe alguma defasagem no controle dos produtos. **A quantidade de produtos se manteve.**

b. Até o mês passado, a empresa tinha um total de 19.500 clientes na base de controle. Verifique se esse número aumentou ou reduziu. **A quantidade de clientes reduziu!**



GABARITOS

2

Questão 2

Você trabalha no setor de marketing da empresa Contoso e acaba de ter uma ideia de oferecer descontos especiais para os clientes no dia de seus aniversários. Para isso, você vai precisar listar todos os clientes e as suas respectivas datas de nascimento, além de um contato.

a. Selecione as colunas: CustomerKey, FirstName, EmailAddress, BirthDate da tabela dimCustomer.

b. Renomeie as colunas dessa tabela usando o alias (comando AS).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 2
--a) e b)

SELECT
    CustomerKey AS 'ID do Cliente',
    FirstName AS 'Nome',
    EmailAddress AS 'E-mail',
    BirthDate AS 'Data de Nascimento'
FROM
    DimCustomer
```

161 %

Resultados | Mensagens

	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	1	680	11000	NULL	Jon	V	Yang	0	1966-04-08	M	NULL	M	jon24@adventure-works.com
2	2	692	11001	NULL	Eugene	L	Huang	0	1965-05-14	S	NULL	M	eugene10@adventure-works.c
3	3	493	11002	NULL	Ruben	NULL	Torres	0	1965-08-12	M	NULL	M	ruben35@adventure-works.cor
4	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.co
5	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.c
6	6	478	11005	NULL	Julio	NULL	Ruiz	0	1965-08-05	S	NULL	M	julio1@adventure-works.com
7	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:06 | 18.869 linhas

GABARITOS

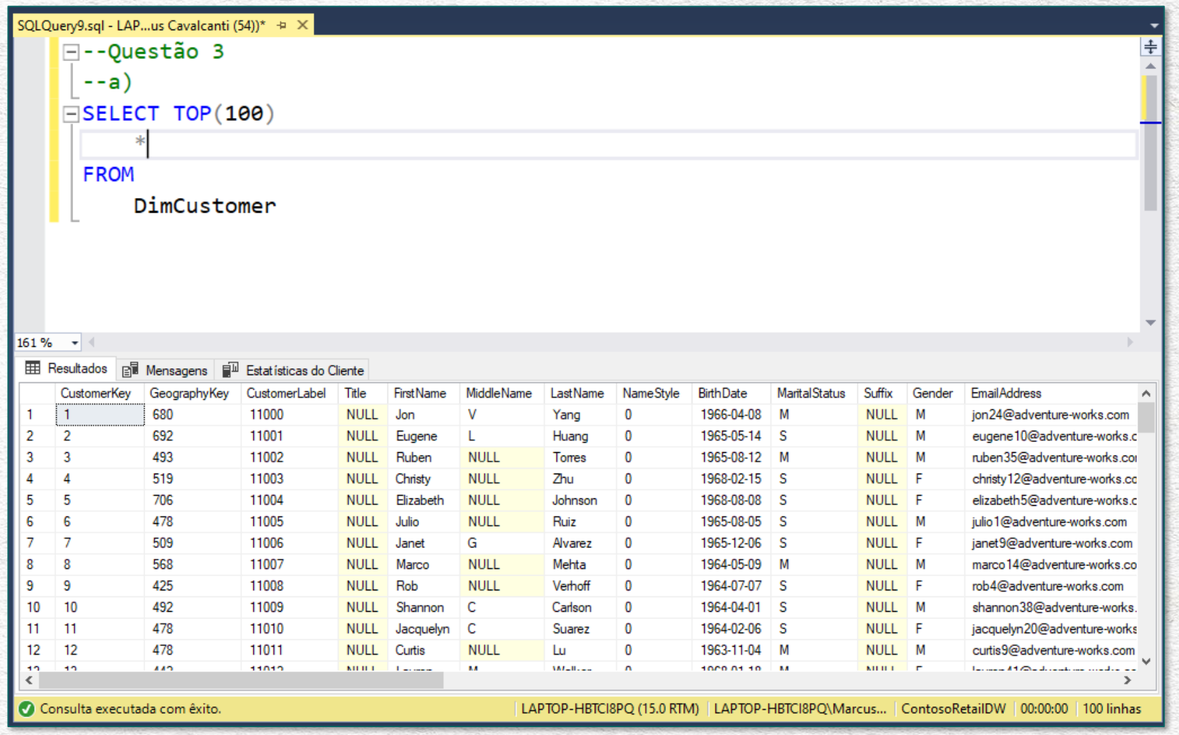
3

Questão 3

A Contoso está comemorando aniversário de inauguração de 10 anos e pretende fazer uma ação de premiação para os clientes. A empresa quer presentear os primeiros clientes desde a inauguração.

Você foi alocado para levar adiante essa ação. Para isso, você terá que fazer o seguinte:

a. A Contoso decidiu presentear os primeiros 100 clientes da história com um vale compras de R$ 10.000. Utilize um comando em SQL para retornar uma tabela com os primeiros 100 primeiros clientes da tabela dimCustomer (selecione todas as colunas).



	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	1	680	11000	NULL	Jon	V	Yang	0	1966-04-08	M	NULL	M	jon24@adventure-works.com
2	2	692	11001	NULL	Eugene	L	Huang	0	1965-05-14	S	NULL	M	eugene10@adventure-works.c
3	3	493	11002	NULL	Ruben	NULL	Torres	0	1965-08-12	M	NULL	M	ruben35@adventure-works.co
4	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.cc
5	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.c
6	6	478	11005	NULL	Julio	NULL	Ruiz	0	1965-08-05	S	NULL	M	julio1@adventure-works.com
7	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com
8	8	568	11007	NULL	Marco	NULL	Mehta	0	1964-05-09	M	NULL	M	marco14@adventure-works.co
9	9	425	11008	NULL	Rob	NULL	Verhoff	0	1964-07-07	S	NULL	F	rob4@adventure-works.com
10	10	492	11009	NULL	Shannon	C	Carlson	0	1964-04-01	S	NULL	M	shannon38@adventure-works.
11	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F	jacquelyn20@adventure-works
12	12	478	11011	NULL	Curtis	NULL	Lu	0	1963-11-04	M	NULL	M	curtis9@adventure-works.com

GABARITOS

3

Questão 3

A Contoso está comemorando aniversário de inauguração de 10 anos e pretende fazer uma ação de premiação para os clientes. A empresa quer presentear os primeiros clientes desde a inauguração.

Você foi alocado para levar adiante essa ação. Para isso, você terá que fazer o seguinte:

a. A Contoso decidiu presentear os primeiros 100 clientes da história com um vale compras de R$ 10.000. Utilize um comando em SQL para retornar uma tabela com os primeiros 100 primeiros clientes da tabela dimCustomer (selecione todas as colunas).

b. A Contoso decidiu presentear os primeiros 20% de clientes da história com um vale compras de R$ 2.000. Utilize um comando em SQL para retornar 20% das linhas da sua tabela dimCustomer (selecione todas as colunas).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 3
--b)
SELECT TOP(20) PERCENT
    *
FROM
    DimCustomer
```

161 %

Resultados | Mensagens | Estatísticas do Cliente

	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	1	680	11000	NULL	Jon	V	Yang	0	1966-04-08	M	NULL	M	jon24@adventure-works.com
2	2	692	11001	NULL	Eugene	L	Huang	0	1965-05-14	S	NULL	M	eugene10@adventure-works.c
3	3	493	11002	NULL	Ruben	NULL	Torres	0	1965-08-12	M	NULL	M	ruben35@adventure-works.co
4	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.cc
5	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.c
6	6	478	11005	NULL	Julio	NULL	Ruiz	0	1965-08-05	S	NULL	M	julio1@adventure-works.com
7	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com
8	8	568	11007	NULL	Marco	NULL	Mehta	0	1964-05-09	M	NULL	M	marco14@adventure-works.co
9	9	425	11008	NULL	Rob	NULL	Verhoff	0	1964-07-07	S	NULL	F	rob4@adventure-works.com
10	10	492	11009	NULL	Shannon	C	Carlson	0	1964-04-01	S	NULL	M	shannon38@adventure-works.
11	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F	jacquelyn20@adventure-works
12	12	478	11011	NULL	Curtis	NULL	Lu	0	1963-11-04	M	NULL	M	curtis9@adventure-works.com

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 3.774 linhas

GABARITOS

3

Questão 3

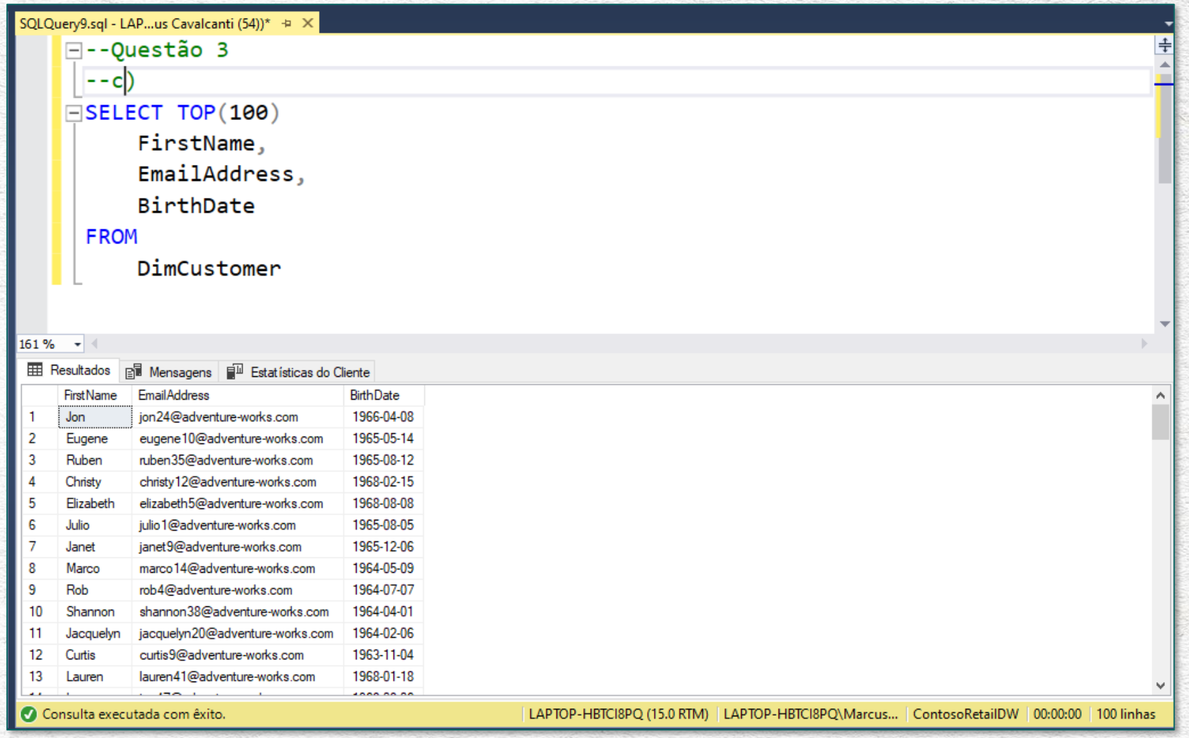
A Contoso está comemorando aniversário de inauguração de 10 anos e pretende fazer uma ação de premiação para os clientes. A empresa quer presentear os primeiros clientes desde a inauguração.

Você foi alocado para levar adiante essa ação. Para isso, você terá que fazer o seguinte:

a. A Contoso decidiu presentear os primeiros 100 clientes da história com um vale compras de R$ 10.000. Utilize um comando em SQL para retornar uma tabela com os primeiros 100 primeiros clientes da tabela dimCustomer (selecione todas as colunas).

b. A Contoso decidiu presentear os primeiros 20% de clientes da história com um vale compras de R$ 2.000. Utilize um comando em SQL para retornar 20% das linhas da sua tabela dimCustomer (selecione todas as colunas).

c. Adapte o código do item a) para retornar apenas as 100 primeiras linhas, mas apenas as colunas FirstName, EmailAddress, BirthDate.



	FirstName	EmailAddress	BirthDate
1	Jon	jon24@adventure-works.com	1966-04-08
2	Eugene	eugene10@adventure-works.com	1965-05-14
3	Ruben	ruben35@adventure-works.com	1965-08-12
4	Christy	christy12@adventure-works.com	1968-02-15
5	Elizabeth	elizabeth5@adventure-works.com	1968-08-08
6	Julio	julio1@adventure-works.com	1965-08-05
7	Janet	janet9@adventure-works.com	1965-12-06
8	Marco	marco14@adventure-works.com	1964-05-09
9	Rob	rob4@adventure-works.com	1964-07-07
10	Shannon	shannon38@adventure-works.com	1964-04-01
11	Jacquelyn	jacquelyn20@adventure-works.com	1964-02-06
12	Curtis	curtis9@adventure-works.com	1963-11-04
13	Lauren	lauren41@adventure-works.com	1968-01-18

GABARITOS

3

Questão 3

A Contoso está comemorando aniversário de inauguração de 10 anos e pretende fazer uma ação de premiação para os clientes. A empresa quer presentear os primeiros clientes desde a inauguração.

Você foi alocado para levar adiante essa ação. Para isso, você terá que fazer o seguinte:

a. A Contoso decidiu presentear os primeiros 100 clientes da história com um vale compras de R$ 10.000. Utilize um comando em SQL para retornar uma tabela com os primeiros 100 primeiros clientes da tabela dimCustomer (selecione todas as colunas).

b. A Contoso decidiu presentear os primeiros 20% de clientes da história com um vale compras de R$ 2.000. Utilize um comando em SQL para retornar 20% das linhas da sua tabela dimCustomer (selecione todas as colunas).

c. Adapte o código do item a) para retornar apenas as 100 primeiras linhas, mas apenas as colunas FirstName, EmailAddress, BirthDate.

d. Renomeie as colunas anteriores para nomes em português

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 3
--d)
SELECT TOP(100)
    FirstName AS 'Nome',
    EmailAddress AS 'Email',
    BirthDate AS 'Data Nascimento'
FROM
    DimCustomer
```

133 %

Resultados | Mensagens | Estatísticas do Cliente

	Nome	Email	Data Nascimento
1	Jon	jon24@adventure-works.com	1966-04-08
2	Eugene	eugene10@adventure-works.com	1965-05-14
3	Ruben	ruben35@adventure-works.com	1965-08-12
4	Christy	christy12@adventure-works.com	1968-02-15
5	Elizabeth	elizabeth5@adventure-works.com	1968-08-08
6	Julio	julio1@adventure-works.com	1965-08-05
7	Janet	janet9@adventure-works.com	1965-12-06
8	Marco	marco14@adventure-works.com	1964-05-09
9	Rob	rob4@adventure-works.com	1964-07-07
10	Shannon	shannon38@adventure-works.com	1964-04-01
11	Jacquelyn	jacquelyn20@adventure-works.com	1964-02-06
12	Curtis	curtis9@adventure-works.com	1963-11-04
13	Lauren	lauren41@adventure-works.com	1968-01-18

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 100 linhas

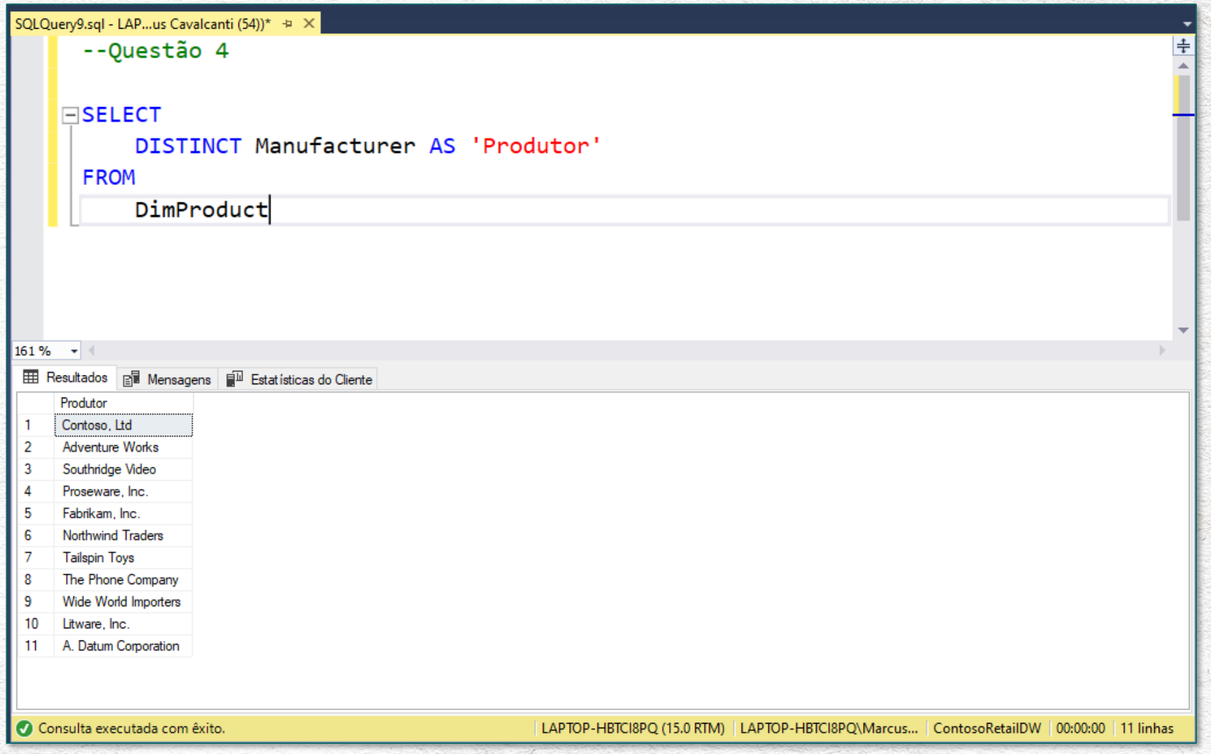
GABARITOS

4

Questão 4

A empresa Contoso precisa fazer contato com os fornecedores de produtos para repor o estoque. Você é da área de compras e precisa descobrir quem são esses fornecedores.

Utilize um comando em SQL para retornar apenas os nomes dos fornecedores na tabela dimProduct e renomeie essa nova coluna da tabela.



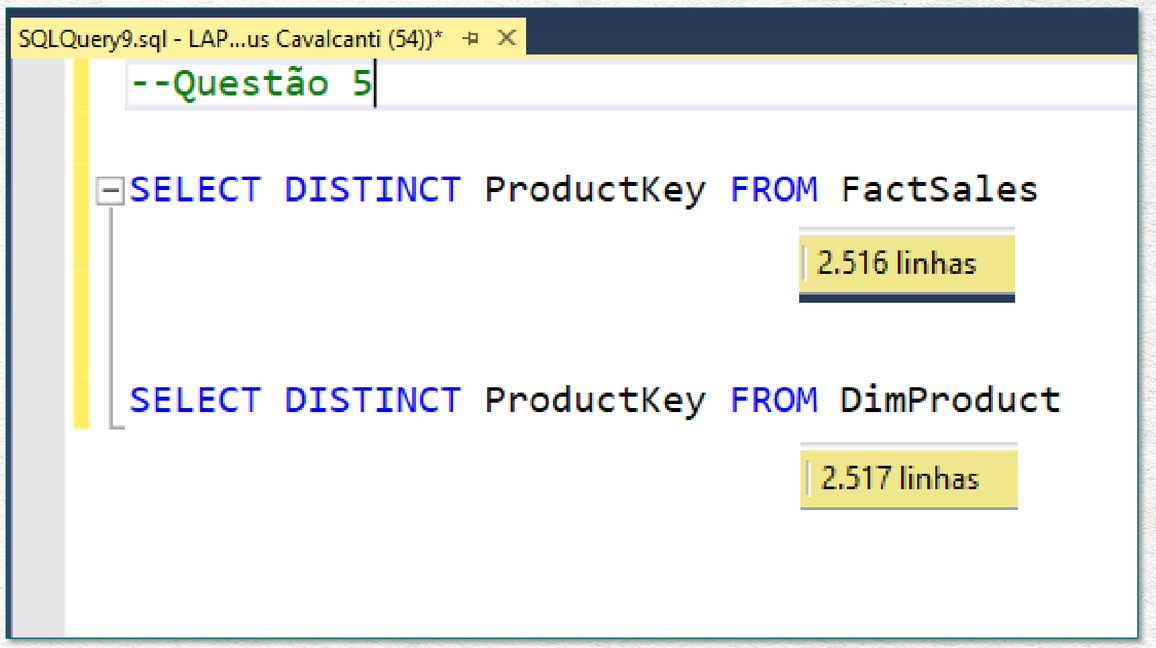
GABARITOS

5

Questão 5

O seu trabalho de investigação não para. Você precisa descobrir se existe algum produto registrado na base de produtos que ainda não tenha sido vendido. Tente chegar nessa informação.

Obs: caso tenha algum produto que ainda não tenha sido vendido, você não precisa descobrir qual é, é suficiente saber se teve ou não algum produto que ainda não foi vendido. **Existe 1 produto que está registrado na tabela DimProduct e que não foi vendido.**



MÓDULO 4

SQL

Ordenando e filtrando dados

O QUE VEREMOS NESTE MÓDULO

MÓDULO 4 ORDENANDO E FILTRANDO DADOS

01	ORDER BY: Ordenando uma tabela	090
02	WHERE ...LIKE: Um filtro especial	093
03	WHERE ...AND, OR e NOT: Filtrar por mais de uma condição	095
04	WHERE ...IN: Especificando mais de um critério por vez	096
05	WHERE ...(NOT) BETWEEN: Como filtrar entre valores	097
06	WHERE ...IS (NOT) NULL: Filtrando valores vazios	098
07	EXERCÍCIOS	099
08	GABARITOS	104

ORDER BY: Ordenando uma tabela

O ORDER BY é usado para ordenar os valores de uma tabela em ordem crescente ou decrescente.

Por padrão, o ORDER BY ordena os dados em ordem crescente (ASC). Para ordenar de forma decrescente, usamos o DESC.

```
-- Exemplo 1: Selecione as 100 primeiras linhas da tabela DimStore e ordene de acordo com a coluna de quantidade de funcionários
```

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
SELECT TOP(100)
    StoreName,
    EmployeeCount
FROM DimStore
-- ordena de forma crescente de
-- acordo com a coluna EmployeeCount
ORDER BY EmployeeCount
```

Resultados | Mensagens

	StoreName	EmployeeCount
1	Contoso Cheshire Store	NULL
2	Contoso Europe Online Store	7
3	Contoso Asia Online Store	8
4	Contoso Strasbourg Store	10
5	Contoso Manchester Store	11
6	Contoso Nantes Store	11
7	Contoso Venezia Store	11
8	Contoso Paris Store	12
9	Contoso Europe Reseller	12
10	Contoso Marseille Store	13
11	Contoso Naples Store	13
12	Contoso Berlin Store	13

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
SELECT TOP(100)
    StoreName,
    EmployeeCount
FROM DimStore
-- ordena de forma decrescente
-- de acordo com a coluna EmployeeCount
ORDER BY EmployeeCount DESC
```

Resultados | Mensagens

	StoreName	EmployeeCount
1	Contoso North America Online Store	325
2	Contoso Guangzhou Store	131
3	Contoso Catalog Store	120
4	Contoso Shanghai No.1 Store	95
5	Contoso Shanghai No.2 Store	95
6	Contoso Ottawa No.1 Store	95
7	Contoso Ottawa No.2 Store	95
8	Contoso Toronto No.1 Store	95
9	Contoso Toronto No.2 Store	95
10	Contoso Toronto No.3 Store	95
11	Contoso Montreal No.1 Store	95
12	Contoso Montreal No.2 Store	95

ORDER BY: Ordenando uma tabela

Também é possível ordenar uma tabela por uma coluna de textos.

```
-- Exemplo 2: Selecione as 100 primeiras linhas da tabela DimStore e ordene de acordo com a coluna de StoreName
```

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
SELECT
TOP(100) StoreKey, StoreName
FROM DimStore
-- ordena de forma crescente (A-Z),
-- de acordo com a coluna StoreName
ORDER BY StoreName
```

Resultados | Mensagens

	StoreKey	StoreName
1	114	Contoso Albany Store
2	184	Contoso Alexandria Store
3	240	Contoso Amsterdam Store
4	183	Contoso Anchorage Store
5	191	Contoso Annapolis Store
6	43	Contoso Appleton Store
7	93	Contoso Arlington Store
8	298	Contoso Ashgabat No.2 Store
9	264	Contoso Ashgabat No.1 Store
10	307	Contoso Asia Online Store
11	310	Contoso Asia Reseller
12	250	Contoso Athens Store

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
SELECT
TOP(100) StoreKey, StoreName
FROM DimStore
-- ordena de forma decrescente (Z-A),
-- de acordo com a coluna StoreName
ORDER BY StoreName DESC
```

Resultados | Mensagens

	StoreKey	StoreName
1	204	Contoso York Store
2	269	Contoso Yokohama Store
3	254	Contoso Yerevan Store
4	6	Contoso Yakima Store
5	148	Contoso Worcester No.2 Store
6	147	Contoso Worcester No.1 Store
7	182	Contoso Winchester Store
8	21	Contoso Wheat Ridge Store
9	22	Contoso Westminster Store
10	208	Contoso West Yorkshire Store
11	54	Contoso Waukesha No.2 Store
12	50	Contoso Waukesha No.1 Store

ORDER BY: Ordenando uma tabela

Também é possível ordenar uma tabela por mais de uma coluna.

```
-- Exemplo 3: Selecione as 100 primeiras linhas da tabela dimProduct e ordene de acordo com as coluna de UnitCost (DESC) e Weight (ASC)
```

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
SELECT TOP(100)
    ProductName,
    Weight,
    UnitCost
FROM DimProduct
-- ordena de forma crescente (A-Z)
ORDER BY UnitCost DESC, Weight ASC
```

Resultados | Mensagens

	ProductName	Weight	UnitCost
1	Litware Refrigerator L1200 Orange	36	1060,22
2	Fabrikam Refrigerator 24.7CuFt X9800 Green	38,1	1060,22
3	Litware Refrigerator 24.7CuFt X980 Blue	56	1060,22
4	Fabrikam Refrigerator 24.7CuFt X9800 Brown	56,8	1060,22
5	Fabrikam Refrigerator 24.7CuFt X9800 Orange	58	1060,22
6	Litware Refrigerator 24.7CuFt X980 Silver	68	1060,22
7	Litware Refrigerator 24.7CuFt X980 Brown	70	1060,22
8	Fabrikam Refrigerator 24.7CuFt X9800 White	71	1060,22
9	Litware Refrigerator 24.7CuFt X980 Green	71	1060,22
10	Fabrikam Refrigerator 24.7CuFt X9800 Blue	75	1060,22
11	Litware Refrigerator 24.7CuFt X980 Grey	132	1060,22
12	Litware Refrigerator 24.7CuFt X980 White	175	1060,22

WHERE ...LIKE: Um filtro especial

O LIKE é usado em conjunto com o WHERE para procurar por um determinado padrão em uma coluna.

Existem 2 caracteres especiais usados em conjunto com o LIKE:

O sinal de porcentagem (%) representa zero, um ou múltiplos caracteres.
O underline (_) representa um único caractere.

Aplicação do LIKE	Descrição
WHERE CustomerName LIKE 'a%'	Encontra qualquer valor que começa com "a"
WHERE CustomerName LIKE '%a'	Encontra qualquer valor que termina com "a"
WHERE CustomerName LIKE '%or%'	Encontra qualquer valor que tenha "or" em qualquer posição da palavra
WHERE CustomerName LIKE '_r%'	Encontra qualquer valor que tenha "r" a partir do segundo caractere
WHERE CustomerName LIKE 'a_%'	Encontra qualquer valor que comece com "a" e tenha pelo menos 2 caracteres
WHERE CustomerName LIKE 'a__%'	Encontra qualquer valor que comece com "a" e tenha pelo menos 3 caracteres
WHERE CustomerName LIKE 'a%o'	Encontra qualquer valor que comece com "a" e termine com "o"

WHERE ...LIKE: Um filtro especial

O LIKE é usado em conjunto com o WHERE para procurar por um determinado padrão em uma coluna.

Existem 2 caracteres especiais usados em conjunto com o LIKE:

- O sinal de porcentagem (%) representa zero, um ou múltiplos caracteres.
- O underline (_) representa um único caractere.

Exemplos:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Seleciona todas as linhas da tabela onde
-- o nome do produto contenha a palavra "MP3"

SELECT
*
FROM
dimProduct

WHERE ProductName LIKE '%MP3%'
```

Resultados | Mensagens

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName	Class
1	1	0101001	Contoso 512MB MP3 Player E51 Silver	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
2	2	0101002	Contoso 512MB MP3 Player E51 Blue	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
3	3	0101003	Contoso 1G MP3 Player E100 White	1GB flash memory and USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
4	4	0101004	Contoso 2G MP3 Player E200 Silver	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
5	5	0101005	Contoso 2G MP3 Player E200 Red	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
6	6	0101006	Contoso 2G MP3 Player E200 Black	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
7	7	0101007	Contoso 2G MP3 Player E200 Blue	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
8	8	0101008	Contoso 4G MP3 Player E400 Silver	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Contoso	1
9	9	0101009	Contoso 4G MP3 Player E400 Black	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Contoso	1
10	10	0101010	Contoso 4G MP3 Player E400 Green	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Contoso	1
11	11	0101011	Contoso 4G MP3 Player E400 Orange	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Contoso	1

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 35 linhas

WHERE ...AND, OR e NOT: Filtrar por mais de uma condição

O comando WHERE pode ser combinado com os operadores AND, OR e NOT.

Os operadores AND e OR são usados para filtrar linhas da tabela baseado em mais de uma condição.

- O AND mostra as linhas da tabela se todas as condições forem atendidas.
- O OR mostra as linhas da tabela se pelo menos uma das condições for atendida.
- O NOT simplesmente mostra o oposto do que for considerado no filtro. Ex: mostra todas as linhas que NÃO forem da marca "Contoso".

Exemplos:

```
-- Seleciona todas as linhas da tabela em que a marca é Contoso E a cor é Prata.

SELECT * FROM dimProduct

WHERE BrandName = 'Contoso' AND ColorName = 'Silver'
```

```
-- Seleciona todas as linhas da tabela em que a cor é Azul OU a cor é Prata.

SELECT * FROM dimProduct

WHERE ColorName = 'Blue' OR ColorName = 'Silver'
```

```
-- Seleciona todas as linhas da tabela em que a cor NÃO é azul.

SELECT  * FROM dimProduct

WHERE NOT ColorName = 'Blue'
```

WHERE ...IN: Especificando mais de um critério por vez

O operador IN permite que sejam especificados múltiplos critérios dentro do WHERE.

O IN é uma alternativa reduzida ao OR.

Exemplos:

```
SQLQuery1.sql - LAP...us Cavalcanti (55))*
-- Seleciona todas as linhas da tabela
-- com produtos das cores Azul, Prata e Preta.

SELECT * FROM dimProduct

WHERE ColorName IN ('Blue', 'Silver', 'Black')
```

Resultados | Mensagens

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName
1	2	0101002	Contoso 512MB MP3 Player E51 Blue	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso
2	6	0101006	Contoso 2G MP3 Player E200 Black	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso
3	7	0101007	Contoso 2G MP3 Player E200 Blue	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso
4	9	0101009	Contoso 4G MP3 Player E400 Black	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso
5	12	0101012	Contoso 4GB Flash MP3 Player E401 Blue	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso
6	13	0101013	Contoso 4GB Flash MP3 Player E401 Black	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso
7	20	0101020	Contoso 8GB MP3 Player new model M820 Black	2" LCD with blue-white LED, 320x240-pixel, plays m...	1	Contoso, Ltd	Contoso
8	21	0101021	Contoso 8GB MP3 Player new model M820 Blue	2" LCD with blue-white LED, 320x240-pixel, plays m...	1	Contoso, Ltd	Contoso
9	24	0101024	Contoso 16GB Mp5 Player M1600 Blue	3" 16:9 TFT Touch screen, 16GB flash memory, pla...	1	Contoso, Ltd	Contoso
10	25	0101025	Contoso 16GB Mp5 Player M1600 Black	3" 16:9 TFT Touch screen, 16GB flash memory, pla...	1	Contoso, Ltd	Contoso
11	33	0101033	Contoso 32GB Video MP3 Player M3200 Black	4.3" Touch screen, 32GB flash memory, beyond 30 ...	1	Contoso, Ltd	Contoso

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 802 linhas

Observe que a quantidade de linhas é bem menor, **pois estamos considerando apenas os produtos de 3 marcas** específicas.

Exemplos:

```
SQLQuery1.sql - LAP...us Cavalcanti (55))*
-- Seleciona todas as linhas que NÃO correspondem
-- às cores Azul, Prata e Preta.

SELECT * FROM dimProduct

WHERE ColorName NOT IN ('Blue', 'Silver', 'Black')
```

Resultados | Mensagens

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandN
1	1	0101001	Contoso 512MB MP3 Player E51 Silver	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Conto
2	3	0101003	Contoso 1G MP3 Player E100 White	1GB flash memory and USB driver plays MP3 and WMA	1	Contoso, Ltd	Conto
3	4	0101004	Contoso 2G MP3 Player E200 Silver	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Conto
4	5	0101005	Contoso 2G MP3 Player E200 Red	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Conto
5	8	0101008	Contoso 4G MP3 Player E400 Silver	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Conto
6	10	0101010	Contoso 4G MP3 Player E400 Green	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Conto
7	11	0101011	Contoso 4G MP3 Player E400 Orange	4GB flash memory and FM Radio, LCD Display with 7-...	1	Contoso, Ltd	Conto
8	14	0101014	Contoso 4GB Flash MP3 Player E401 Silver	1.8" color LCD, play MP3, WMA and Video MTV, and...	1	Contoso, Ltd	Conto
9	15	0101015	Contoso 4GB Flash MP3 Player E401 White	1.8" color LCD, play MP3, WMA and Video MTV, and...	1	Contoso, Ltd	Conto
10	16	0101016	Contoso 8GB Super-Slim MP3/Video Player M800 White	2" color LCD, Touchpad, Plays music, video, photos ...	1	Contoso, Ltd	Conto
11	17	0101017	Contoso 8GB Super-Slim MP3/Video Player M800 Red	2" color LCD, Touchpad, Plays music, video, photos ...	1	Contoso, Ltd	Conto

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 1.715 linhas

WHERE ...(NOT) BETWEEN: Como filtrar entre valores

O BETWEEN seleciona valores em um intervalo. Estes valores podem ser números, textos ou datas.

O BETWEEN é inclusivo: ou seja, os valores dos extremos (valor inicial e valor final) também são incluídos no intervalo.

Exemplos:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Seleciona todas as vendas da tabela
-- com quantidade vendida entre 10 e 15.

SELECT TOP(1000) * FROM factSales
WHERE SalesQuantity BETWEEN 10 AND 15
```

Resultados | Mensagens

	SalesKey	DateKey	channelKey	StoreKey	ProductKey	PromotionKey	CurrencyKey	UnitCost	UnitPrice	SalesQuantity	ReturnQuantity	ReturnAmount	Discou
1	12	2007-04-29 00:00:00.000	1	119	543	1	1	116,75	229,00	10	0	0,00	0
2	13	2007-07-25 00:00:00.000	1	171	739	3	1	78,19	236,00	12	0	0,00	0
3	14	2008-12-16 00:00:00.000	1	16	1269	13	1	25,47	49,96	13	0	0,00	1
4	15	2008-10-17 00:00:00.000	2	199	1788	1	1	21,92	43,00	10	0	0,00	0
5	19	2008-07-16 00:00:00.000	1	292	519	1	1	205,09	619,00	12	0	0,00	0
6	25	2007-06-06 00:00:00.000	1	108	2351	1	1	183,94	399,99	10	0	0,00	0
7	27	2009-04-29 00:00:00.000	1	186	2483	1	1	160,95	350,00	10	0	0,00	0
8	28	2009-12-23 00:00:00.000	1	4	1368	22	1	18,48	40,19	13	0	0,00	0
9	30	2008-12-20 00:00:00.000	1	302	363	14	1	321,44	699,00	13	0	0,00	1
10	34	2007-05-24 00:00:00.000	1	214	1494	1	1	95,65	208,00	10	0	0,00	0
11	35	2007-10-04 00:00:00.000	1	55	1992	1	1	71,37	139,99	10	1	139,99	0

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 1.000 linhas

WHERE ...IS (NOT) NULL: Filtrando valores vazios

Podemos encontrar valores nulos (em branco) em uma tabela. Para filtrar esses valores, podemos usar os comandos IS NULL ou IS NOT NULL.

Exemplos:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Seleciona todas as linhas
-- onde o primeiro nome é nulo
SELECT * FROM dimCustomer
WHERE firstName IS NULL
```

Resultados | Mensagens

	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress	YearlyIncome
1	18761	424	CS424	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
2	18762	430	CS430	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
3	18763	431	CS431	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
4	18764	432	CS432	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
5	18765	433	CS433	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
6	18766	434	CS434	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
7	18767	436	CS436	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
8	18768	438	CS438	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
9	18769	440	CS440	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
10	18770	441	CS441	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00
11	18771	442	CS442	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	NULL	NULL	10000000,00

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:01 | 385 linhas

Exemplos:

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Seleciona todas as linhas
-- onde o primeiro nome NÃO é nulo
SELECT * FROM dimCustomer
WHERE firstName IS NOT NULL
```

Resultados | Mensagens

	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	1	680	11000	NULL	Jon	V	Yang	0	1966-04-08	M	NULL	M	jon24@adventure-works.com
2	2	692	11001	NULL	Eugene	L	Huang	0	1965-05-14	S	NULL	M	eugene10@adventure-works.c
3	3	493	11002	NULL	Ruben	NULL	Torres	0	1965-08-12	M	NULL	M	ruben35@adventure-works.co
4	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.cc
5	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.c
6	6	478	11005	NULL	Julio	NULL	Ruiz	0	1965-08-05	S	NULL	M	julio1@adventure-works.com
7	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com
8	8	568	11007	NULL	Marco	NULL	Mehta	0	1964-05-09	M	NULL	M	marco14@adventure-works.co
9	9	425	11008	NULL	Rob	NULL	Verhoff	0	1964-07-07	S	NULL	F	rob4@adventure-works.com
10	10	492	11009	NULL	Shannon	C	Carlson	0	1964-04-01	S	NULL	M	shannon38@adventure-works.
11	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F	jacquelyn20@adventure-works

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 18.484 linhas

EXERCÍCIOS

1

Questão 1

Você é o gerente da área de compras e precisa criar um relatório com as TOP 100 vendas, de acordo com a quantidade vendida. Você precisa fazer isso em 10min pois o diretor de compras solicitou essa informação para apresentar em uma reunião.

Utilize seu conhecimento em SQL para buscar essas TOP 100 vendas, de acordo com o total vendido (SalesAmount).

2

Questão 2

Os TOP 10 produtos com maior UnitPrice possuem exatamente o mesmo preço. Porém, a empresa quer diferenciar esses preços de acordo com o peso (Weight) de cada um.

O que você precisará fazer é ordenar esses top 10 produtos, de acordo com a coluna de UnitPrice e, além disso, estabelecer um critério de desempate, para que seja mostrado na ordem, do maior para o menor.

Caso ainda assim haja um empate entre 2 ou mais produtos, pense em uma forma de criar um segundo critério de desempate (além do peso).

EXERCÍCIOS

3

Questão 3

Você é responsável pelo setor de logística da empresa Contoso e precisa dimensionar o transporte de todos os produtos em categorias, de acordo com o peso.

Os produtos da categoria A, com peso acima de 100kg, deverão ser transportados na primeira leva.

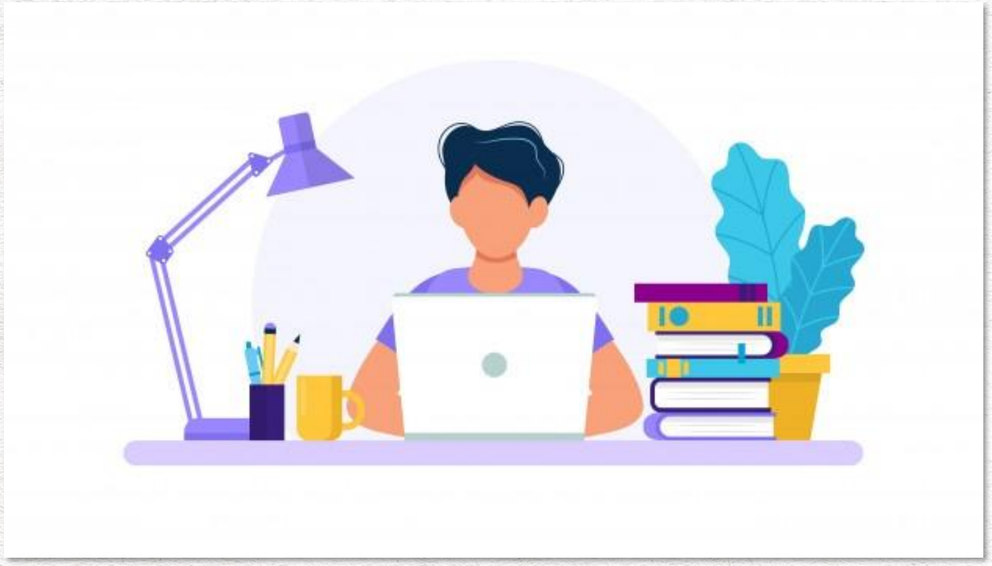
Faça uma consulta no banco de dados para descobrir quais são estes produtos que estão na categoria A.

a. Você deverá retornar apenas 2 colunas nessa consulta: Nome do Produto e Peso.

b. Renomeie essas colunas com nomes mais intuitivos.

c. Ordene esses produtos do mais pesado para o mais leve.

EXERCÍCIOS



4

Questão 4

Você foi alocado para criar um relatório das lojas registradas atualmente na Contoso.

a. Descubra quantas lojas a empresa tem no total. Na consulta que você deverá fazer à tabela DimStore, retorne as seguintes informações: StoreName, OpenDate, EmployeeCount.

b. Renomeeie as colunas anteriores para deixar a sua consulta mais intuitiva.

c. Dessas lojas, descubra quantas (e quais) lojas ainda estão ativas.

5

Questão 5

O gerente da área de controle de qualidade notificou à Contoso que todos os produtos Home Theater da marca Litware, disponibilizados para venda no dia 15 de março de 2009, foram identificados com defeitos de fábrica.

O que você deverá fazer é identificar os ID's desses produtos e repassar ao gerente para que ele possa notificar as lojas e consequentemente solicitar a suspensão das vendas desses produtos.

EXERCÍCIOS

6

Questão 6

Imagine que você precise extrair um relatório da tabela DimStore, com informações de lojas. Mas você precisa apenas das lojas que não estão mais funcionando atualmente.

a. Utilize a coluna de Status para filtrar a tabela e trazer apenas as lojas que não estão mais funcionando.

b. Agora imagine que essa coluna de Status não existe na sua tabela. Qual seria a outra forma que você teria de descobrir quais são as lojas que não estão mais funcionando.

7

Questão 7

De acordo com a quantidade de funcionários, cada loja receberá uma determinada quantidade de máquinas de café. As lojas serão divididas em 3 categorias:

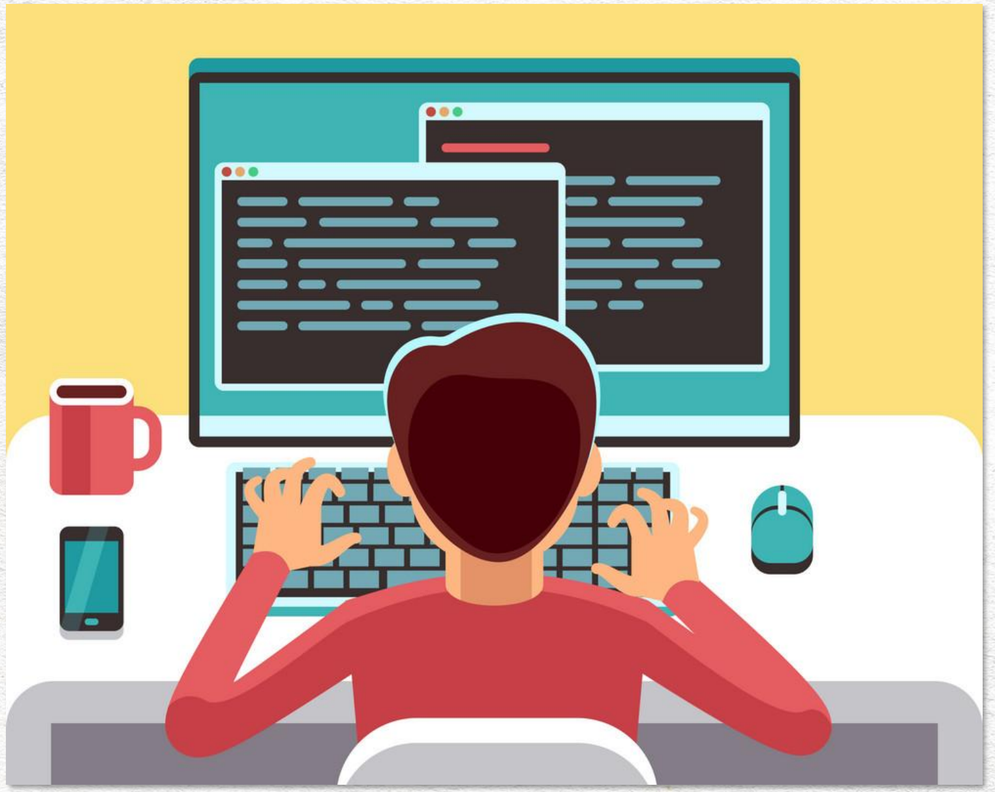
CATEGORIA 1: De 1 a 20 funcionários -> 1 máquina de café

CATEGORIA 2: De 21 a 50 funcionários -> 2 máquinas de café

CATEGORIA 3: Acima de 51 funcionários -> 3 máquinas de café

Identifique, para cada caso, quais são as lojas de cada uma das 3 categorias acima (basta fazer uma verificação).

EXERCÍCIOS



8

Questão 8

A empresa decidiu que todos os produtos LCD receberão um super desconto no próximo mês. O seu trabalho é fazer uma consulta à tabela DimProduct e retornar os ID's, Nomes e Preços de todos os produtos LCD existentes.

9

Questão 9

Faça uma lista com todos os produtos das cores: Green, Orange, Black, Silver e Pink. Estes produtos devem ser exclusivamente das marcas: Contoso, Litware e Fabrikam.

10

Questão 10

A empresa possui 16 produtos da marca Contoso, da cor Silver e com um UnitPrice entre 10 e 30. Descubra quais são esses produtos e ordene o resultado em ordem decrescente de acordo com o preço (UnitPrice).

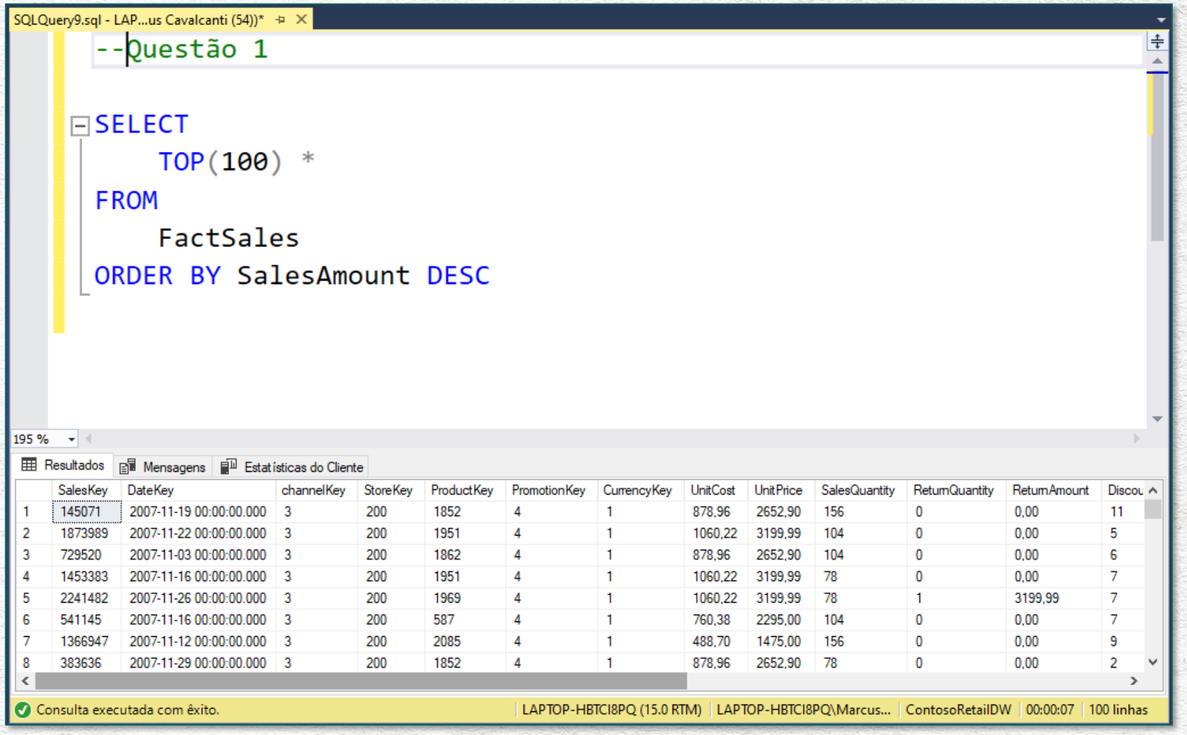
GABARITOS

1

Questão 1

Você é o gerente da área de compras e precisa criar um relatório com as TOP 100 vendas, de acordo com a quantidade vendida. Você precisa fazer isso em 10min pois o diretor de compras solicitou essa informação para apresentar em uma reunião.

Utilize seu conhecimento em SQL para buscar essas TOP 100 vendas, de acordo com o total vendido (SalesAmount).



	SalesKey	DateKey	channelKey	StoreKey	ProductKey	PromotionKey	CurrencyKey	UnitCost	UnitPrice	SalesQuantity	ReturnQuantity	ReturnAmount	Discou
1	145071	2007-11-19 00:00:00.000	3	200	1852	4	1	878,96	2652,90	156	0	0,00	11
2	1873989	2007-11-22 00:00:00.000	3	200	1951	4	1	1060,22	3199,99	104	0	0,00	5
3	729520	2007-11-03 00:00:00.000	3	200	1862	4	1	878,96	2652,90	104	0	0,00	6
4	1453383	2007-11-16 00:00:00.000	3	200	1951	4	1	1060,22	3199,99	78	0	0,00	7
5	2241482	2007-11-26 00:00:00.000	3	200	1969	4	1	1060,22	3199,99	78	1	3199,99	7
6	541145	2007-11-16 00:00:00.000	3	200	587	4	1	760,38	2295,00	104	0	0,00	7
7	1366947	2007-11-12 00:00:00.000	3	200	2085	4	1	488,70	1475,00	156	0	0,00	9
8	383636	2007-11-29 00:00:00.000	3	200	1852	4	1	878,96	2652,90	78	0	0,00	2

GABARITOS

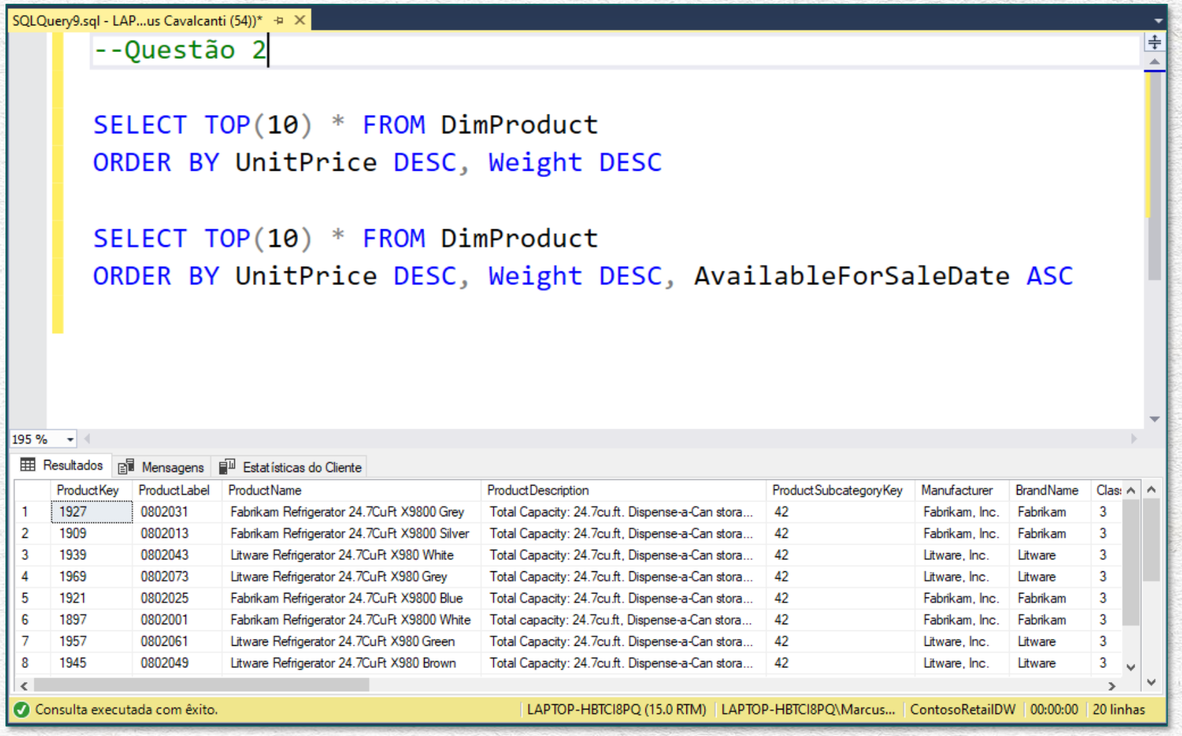
2

Questão 2

Os TOP 10 produtos com maior UnitPrice possuem exatamente o mesmo preço. Porém, a empresa quer diferenciar esses preços de acordo com o peso (Weight) de cada um.

O que você precisará fazer é ordenar esses top 10 produtos, de acordo com a coluna de UnitPrice e, além disso, estabelecer um critério de desempate, para que seja mostrado na ordem, do maior para o menor.

Caso ainda assim haja um empate entre 2 ou mais produtos, pense em uma forma de criar um segundo critério de desempate (além do peso).



	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName	Clas
1	1927	0802031	Fabrikam Refrigerator 24.7CuFt X9800 Grey	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Fabrikam, Inc.	Fabrikam	3
2	1909	0802013	Fabrikam Refrigerator 24.7CuFt X9800 Silver	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Fabrikam, Inc.	Fabrikam	3
3	1939	0802043	Litware Refrigerator 24.7CuFt X980 White	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Litware, Inc.	Litware	3
4	1969	0802073	Litware Refrigerator 24.7CuFt X980 Grey	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Litware, Inc.	Litware	3
5	1921	0802025	Fabrikam Refrigerator 24.7CuFt X9800 Blue	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Fabrikam, Inc.	Fabrikam	3
6	1897	0802001	Fabrikam Refrigerator 24.7CuFt X9800 White	Total capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Fabrikam, Inc.	Fabrikam	3
7	1957	0802061	Litware Refrigerator 24.7CuFt X980 Green	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Litware, Inc.	Litware	3
8	1945	0802049	Litware Refrigerator 24.7CuFt X980 Brown	Total Capacity: 24.7cu.ft. Dispense-a-Can stora...	42	Litware, Inc.	Litware	3

GABARITOS

3

Questão 3

Você é responsável pelo setor de logística da empresa Contoso e precisa dimensionar o transporte de todos os produtos em categorias, de acordo com o peso.

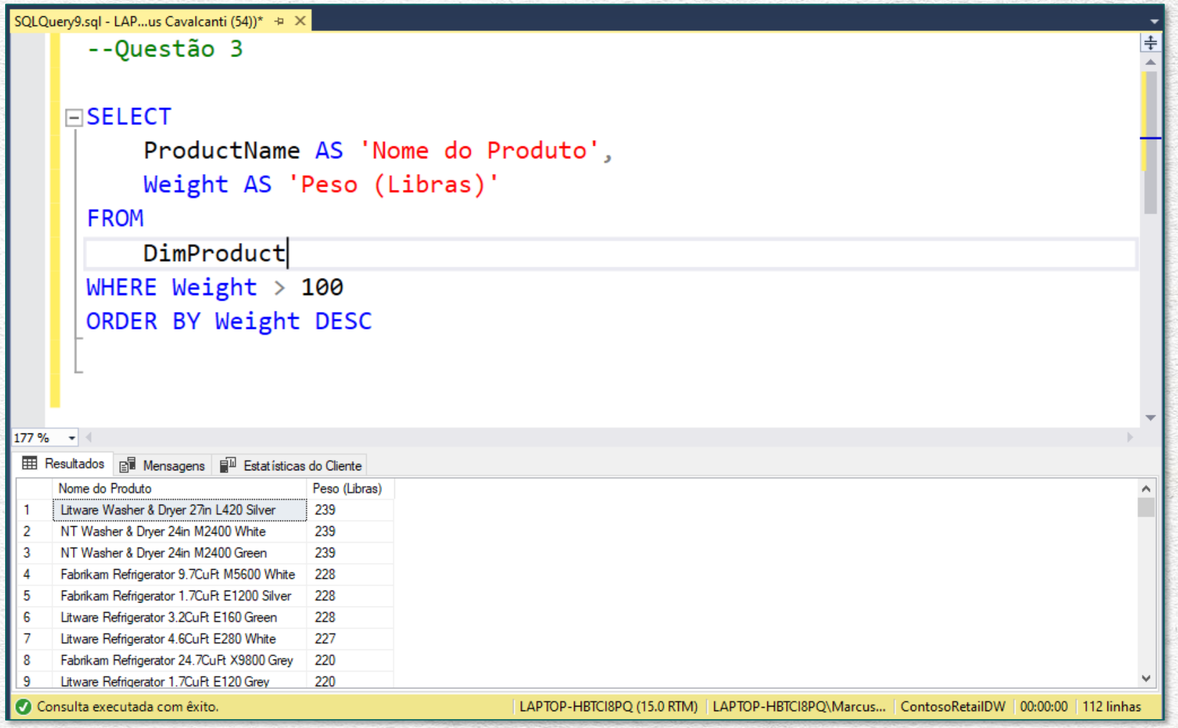
Os produtos da categoria A, com peso acima de 100kg, deverão ser transportados na primeira leva.

Faça uma consulta no banco de dados para descobrir quais são estes produtos que estão na categoria A.

a. Você deverá retornar apenas 2 colunas nessa consulta: Nome do Produto e Peso.

b. Renomeie essas colunas com nomes mais intuitivos.

c. Ordene esses produtos do mais pesado para o mais leve.



GABARITOS

4

Questão 4

Você foi alocado para criar um relatório das lojas registradas atualmente na Contoso.

a. Descubra quantas lojas a empresa tem no total. Na consulta que você deverá fazer à tabela DimStore, retorne as seguintes informações: StoreName, OpenDate, EmployeeCount.

b. Renomeeie as colunas anteriores para deixar a sua consulta mais intuitiva.

c. Dessas lojas, descubra quantas (e quais) lojas ainda estão ativas.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 4

SELECT
    StoreName AS 'Nome da Loja',
    OpenDate AS 'Data de abertura',
    EmployeeCount AS 'Qtd. Funcionários'
FROM
    DimStore
WHERE StoreType = 'Store' AND Status = 'On'
```

177 %

Resultados | Mensagens | Estatísticas do Cliente

	Nome da Loja	Data de abertura	Qtd. Funcionários
1	Contoso Seattle No.1 Store	2004-04-12 00:00:00.000	17
2	Contoso Seattle No.2 Store	2004-02-14 00:00:00.000	25
3	Contoso Kennewick Store	2004-02-12 00:00:00.000	26
4	Contoso Bellevue Store	2004-03-01 00:00:00.000	19
5	Contoso Redmond Store	2004-04-02 00:00:00.000	33
6	Contoso Yakima Store	2005-05-15 00:00:00.000	47
7	Contoso Granger Store	2004-06-16 00:00:00.000	22
8	Contoso Sunnyside Store	2004-07-02 00:00:00.000	17
9	Contoso Toppenish Store	2004-08-18 00:00:00.000	25

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 287 linhas

GABARITOS

5

Questão 5

O gerente da área de controle de qualidade notificou à Contoso que todos os produtos Home Theater da marca Litware, disponibilizados para venda no dia 15 de março de 2009, foram identificados com defeitos de fábrica.

O que você deverá fazer é identificar os ID's desses produtos e repassar ao gerente para que ele possa notificar as lojas e consequentemente solicitar a suspensão das vendas desses produtos.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 5

SELECT
    *
FROM
    DimProduct
WHERE BrandName = 'Litware' AND AvailableForSaleDate = '15/03/2009'
```

177 %

Resultados | Mensagens | Estatísticas do Cliente

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName
1	198	0203007	Litware Home Theater System 4.1 Channel M411 Black	4.1 Channel Front Surround. 1080p/720p up conve...	11	Litware, Inc.	Litware
2	214	0203023	Litware Home Theater System 4.1 Channel M411 Silver	4.1 channel front surround, 1080p/720p up convers...	11	Litware, Inc.	Litware
3	230	0203039	Litware Home Theater System 4.1 Channel M411 Brown	4.1 Channel front surround. 1080p/720p up conver...	11	Litware, Inc.	Litware

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 3 linhas

GABARITOS

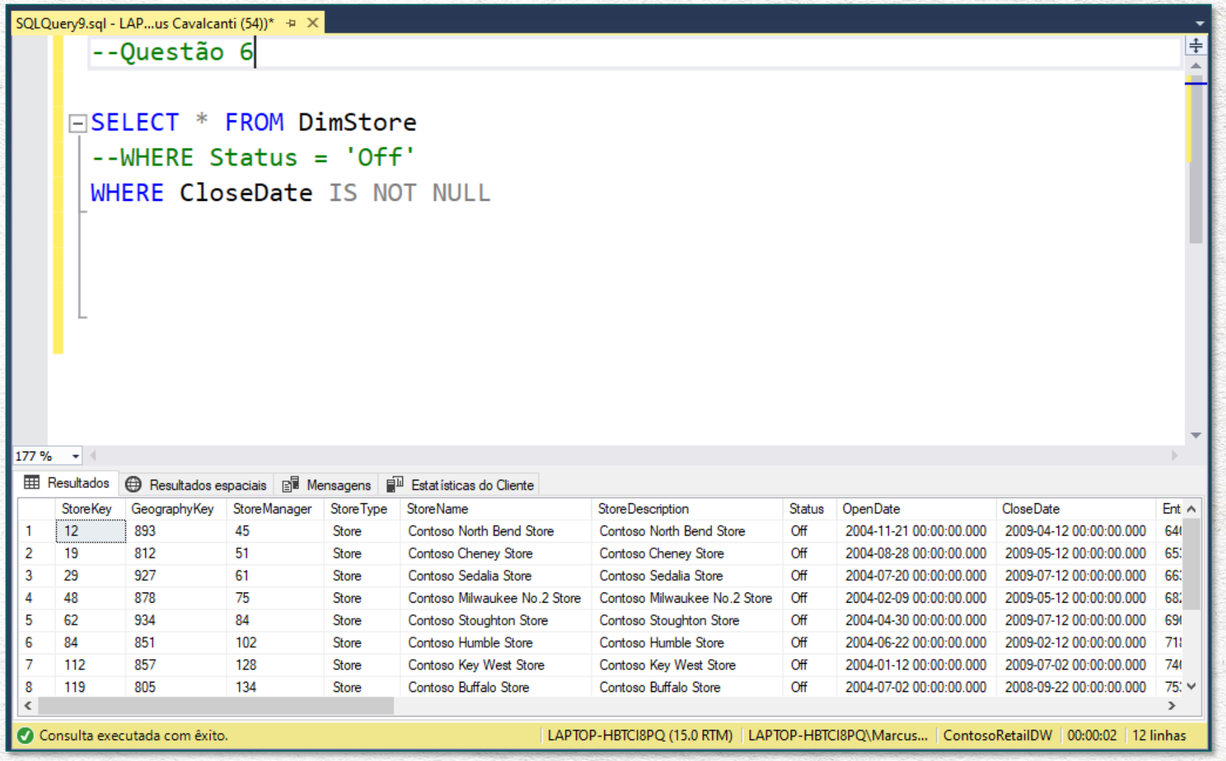
6

Questão 6

Imagine que você precise extrair um relatório da tabela DimStore, com informações de lojas. Mas você precisa apenas das lojas que não estão mais funcionando atualmente.

a. Utilize a coluna de Status para filtrar a tabela e trazer apenas as lojas que não estão mais funcionando.

b. Agora imagine que essa coluna de Status não existe na sua tabela. Qual seria a outra forma que você teria de descobrir quais são as lojas que não estão mais funcionando.



	StoreKey	GeographyKey	StoreManager	StoreType	StoreName	StoreDescription	Status	OpenDate	CloseDate	Ent
1	12	893	45	Store	Contoso North Bend Store	Contoso North Bend Store	Off	2004-11-21 00:00:00.000	2009-04-12 00:00:00.000	64
2	19	812	51	Store	Contoso Cheney Store	Contoso Cheney Store	Off	2004-08-28 00:00:00.000	2009-05-12 00:00:00.000	65
3	29	927	61	Store	Contoso Sedalia Store	Contoso Sedalia Store	Off	2004-07-20 00:00:00.000	2009-07-12 00:00:00.000	66
4	48	878	75	Store	Contoso Milwaukee No.2 Store	Contoso Milwaukee No.2 Store	Off	2004-02-09 00:00:00.000	2009-05-12 00:00:00.000	68
5	62	934	84	Store	Contoso Stoughton Store	Contoso Stoughton Store	Off	2004-04-30 00:00:00.000	2009-07-12 00:00:00.000	69
6	84	851	102	Store	Contoso Humble Store	Contoso Humble Store	Off	2004-06-22 00:00:00.000	2009-02-12 00:00:00.000	71
7	112	857	128	Store	Contoso Key West Store	Contoso Key West Store	Off	2004-01-12 00:00:00.000	2009-07-02 00:00:00.000	74
8	119	805	134	Store	Contoso Buffalo Store	Contoso Buffalo Store	Off	2004-07-02 00:00:00.000	2008-09-22 00:00:00.000	75

GABARITOS

7

Questão 7

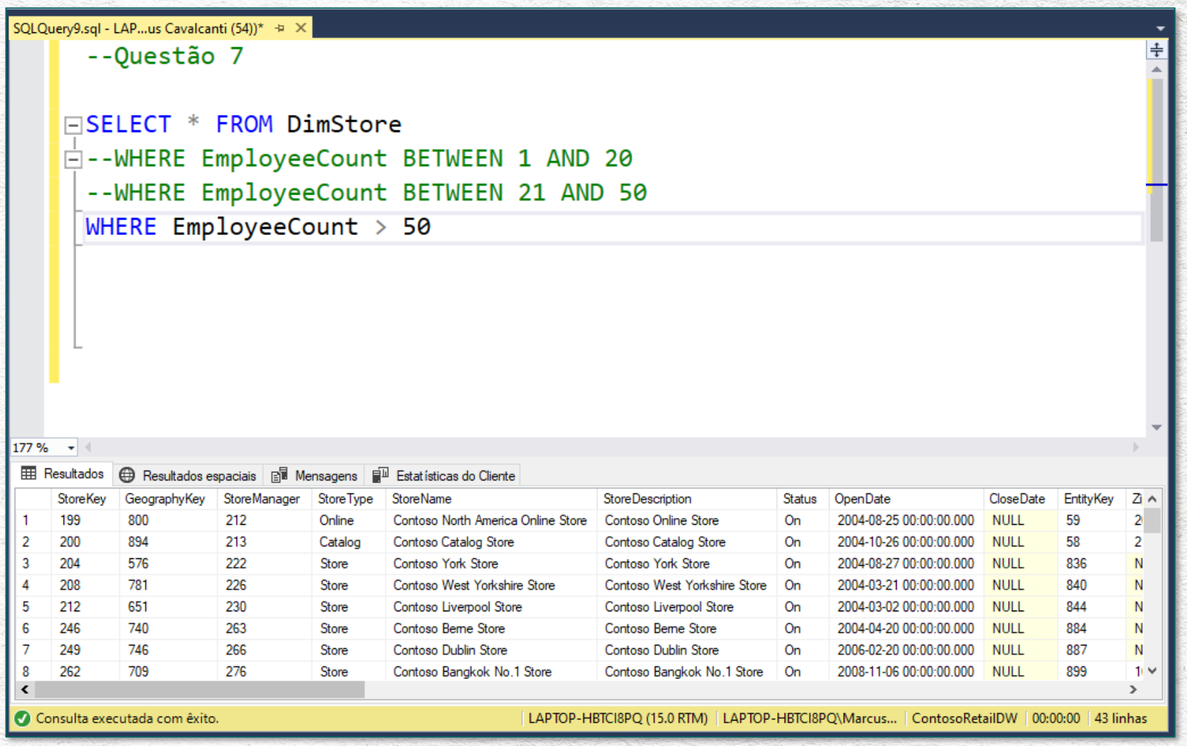
De acordo com a quantidade de funcionários, cada loja receberá uma determinada quantidade de máquinas de café. As lojas serão divididas em 3 categorias:

CATEGORIA 1: De 1 a 20 funcionários -> 1 máquina de café

CATEGORIA 2: De 21 a 50 funcionários -> 2 máquinas de café

CATEGORIA 3: Acima de 51 funcionários -> 3 máquinas de café

Identifique, para cada caso, quais são as lojas de cada uma das 3 categorias acima (basta fazer uma verificação).



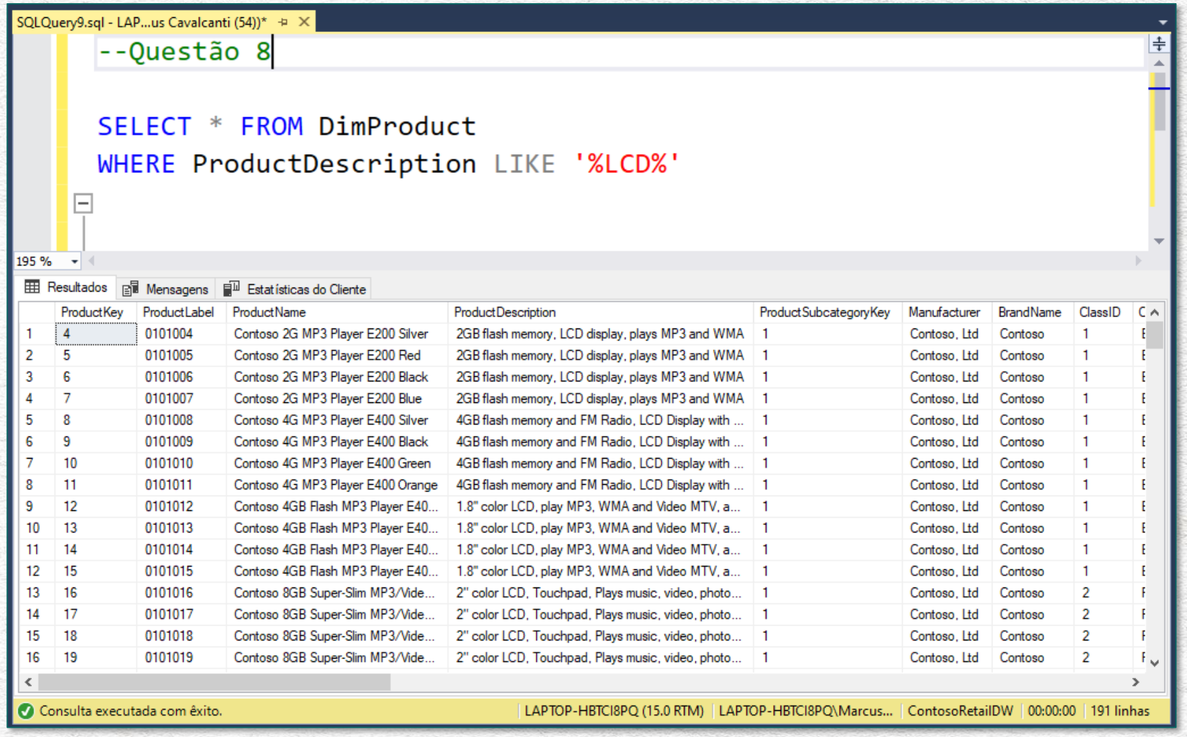
	StoreKey	GeographyKey	StoreManager	StoreType	StoreName	StoreDescription	Status	OpenDate	CloseDate	EntityKey	Z
1	199	800	212	Online	Contoso North America Online Store	Contoso Online Store	On	2004-08-25 00:00:00.000	NULL	59	2
2	200	894	213	Catalog	Contoso Catalog Store	Contoso Catalog Store	On	2004-10-26 00:00:00.000	NULL	58	2
3	204	576	222	Store	Contoso York Store	Contoso York Store	On	2004-08-27 00:00:00.000	NULL	836	N
4	208	781	226	Store	Contoso West Yorkshire Store	Contoso West Yorkshire Store	On	2004-03-21 00:00:00.000	NULL	840	N
5	212	651	230	Store	Contoso Liverpool Store	Contoso Liverpool Store	On	2004-03-02 00:00:00.000	NULL	844	N
6	246	740	263	Store	Contoso Berne Store	Contoso Berne Store	On	2004-04-20 00:00:00.000	NULL	884	N
7	249	746	266	Store	Contoso Dublin Store	Contoso Dublin Store	On	2006-02-20 00:00:00.000	NULL	887	N
8	262	709	276	Store	Contoso Bangkok No.1 Store	Contoso Bangkok No.1 Store	On	2008-11-06 00:00:00.000	NULL	899	1

GABARITOS

8

Questão 8

A empresa decidiu que todos os produtos LCD receberão um super desconto no próximo mês. O seu trabalho é fazer uma consulta à tabela DimProduct e retornar os ID's, Nomes e Preços de todos os produtos LCD existentes.



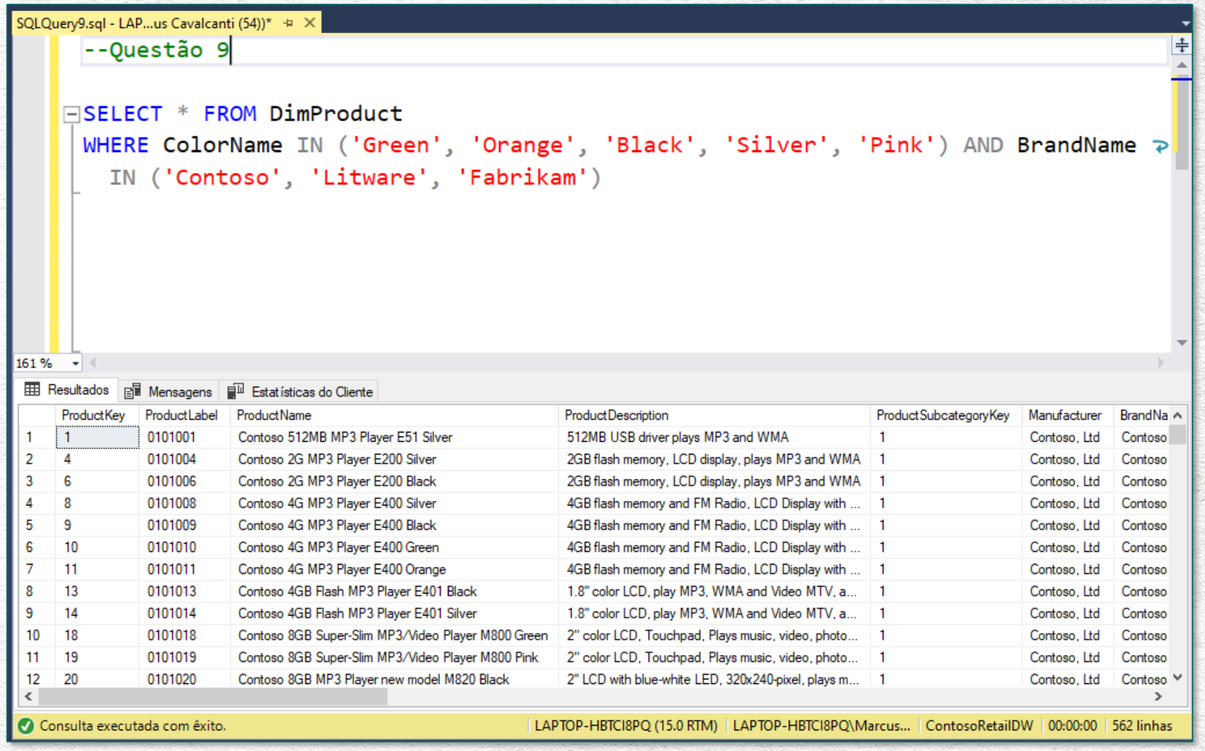
	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName	ClassID
1	4	0101004	Contoso 2G MP3 Player E200 Silver	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
2	5	0101005	Contoso 2G MP3 Player E200 Red	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
3	6	0101006	Contoso 2G MP3 Player E200 Black	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
4	7	0101007	Contoso 2G MP3 Player E200 Blue	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
5	8	0101008	Contoso 4G MP3 Player E400 Silver	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso	1
6	9	0101009	Contoso 4G MP3 Player E400 Black	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso	1
7	10	0101010	Contoso 4G MP3 Player E400 Green	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso	1
8	11	0101011	Contoso 4G MP3 Player E400 Orange	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso	1
9	12	0101012	Contoso 4GB Flash MP3 Player E40...	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso	1
10	13	0101013	Contoso 4GB Flash MP3 Player E40...	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso	1
11	14	0101014	Contoso 4GB Flash MP3 Player E40...	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso	1
12	15	0101015	Contoso 4GB Flash MP3 Player E40...	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso	1
13	16	0101016	Contoso 8GB Super-Slim MP3/Vide...	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso	2
14	17	0101017	Contoso 8GB Super-Slim MP3/Vide...	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso	2
15	18	0101018	Contoso 8GB Super-Slim MP3/Vide...	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso	2
16	19	0101019	Contoso 8GB Super-Slim MP3/Vide...	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso	2

GABARITOS

9

Questão 9

Faça uma lista com todos os produtos das cores: Green, Orange, Black, Silver e Pink. Estes produtos devem ser exclusivamente das marcas: Contoso, Litware e Fabrikam.



	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandNa
1	1	0101001	Contoso 512MB MP3 Player E51 Silver	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso
2	4	0101004	Contoso 2G MP3 Player E200 Silver	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso
3	6	0101006	Contoso 2G MP3 Player E200 Black	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso
4	8	0101008	Contoso 4G MP3 Player E400 Silver	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso
5	9	0101009	Contoso 4G MP3 Player E400 Black	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso
6	10	0101010	Contoso 4G MP3 Player E400 Green	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso
7	11	0101011	Contoso 4G MP3 Player E400 Orange	4GB flash memory and FM Radio, LCD Display with ...	1	Contoso, Ltd	Contoso
8	13	0101013	Contoso 4GB Flash MP3 Player E401 Black	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso
9	14	0101014	Contoso 4GB Flash MP3 Player E401 Silver	1.8" color LCD, play MP3, WMA and Video MTV, a...	1	Contoso, Ltd	Contoso
10	18	0101018	Contoso 8GB Super-Slim MP3/Video Player M800 Green	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso
11	19	0101019	Contoso 8GB Super-Slim MP3/Video Player M800 Pink	2" color LCD, Touchpad, Plays music, video, photo...	1	Contoso, Ltd	Contoso
12	20	0101020	Contoso 8GB MP3 Player new model M820 Black	2" LCD with blue-white LED, 320x240-pixel, plays m...	1	Contoso, Ltd	Contoso

GABARITOS

10

Questão 10

A empresa possui 16 produtos da marca Contoso, da cor Silver e com um UnitPrice entre 10 e 30. Descubra quais são esses produtos e ordene o resultado em ordem decrescente de acordo com o preço (UnitPrice).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 10

SELECT * FROM DimProduct
WHERE BrandName = 'Contoso' AND ColorName = 'Silver' AND Weight BETWEEN 10 AND
  30
ORDER BY UnitPrice DESC
```

161 %

Resultados | Mensagens | Estatísticas do Cliente

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	Brand
1	2122	0805007	Contoso Coffee Maker Super-Auto 12C X1250 Silver	Combination 12 cup super-auto coffee maker, 1250-wa...	45	Contoso, Ltd	Conto
2	2100	0804016	Contoso Water Heater 7.2GPM X1800 Silver	Delivers 7.2 gallons per minute, Can save up to 50% of...	44	Contoso, Ltd	Conto
3	2101	0804017	Contoso Water Heater 4.3GPM M1250 Silver	Delivers up to 4.3 gallons per minute. Endless hot water	44	Contoso, Ltd	Conto
4	2102	0804018	Contoso Water Heater 4.0GPM M1250 Silver	Provides 4 gallons of hot water per minute, Includes on...	44	Contoso, Ltd	Conto
5	2103	0804019	Contoso Water Heater 2.6GPM E0900 Silver	Provides 2.6 gallons of hot water per minute, average ...	44	Contoso, Ltd	Conto
6	2124	0805009	Contoso Coffee Maker 12C M1000 Silver	Combination 12 cup coffee maker, 24-hour clock/timer...	45	Contoso, Ltd	Conto
7	257	0203066	Contoso Home Theater System 5.1 Channel M1500 Si...	1000 watts over 5 channels, 1 center-channel speaker	11	Contoso, Ltd	Conto
8	265	0203074	Contoso Home Theater System 2.1 Channel M1230 Si...	2.1 channel home theater system, includes 2 front spe...	11	Contoso, Ltd	Conto
9	2104	0804020	Contoso Water Heater 1.5GPM E0800 Silver	Delivers 1.5 gallons per minute, 99-percent efficient	44	Contoso, Ltd	Conto
10	607	0305069	Contoso Screen 106in M060 Silver	16:9 aspect ratio-106in, electric/motorized screen for h...	19	Contoso, Ltd	Conto
11	1623	0602053	Contoso DVD Recorder L210 Silver	DVD and VHS Recorder with two way dubbing, Recor...	35	Contoso, Ltd	Conto
12	2126	0805011	Contoso Coffee Maker 5C E0900 Silver	5-Cup Capacity with glass carafe, digital soft touch pad...	45	Contoso, Ltd	Conto

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 16 linhas

MÓDULO 5

SQL

Funções de Agregação

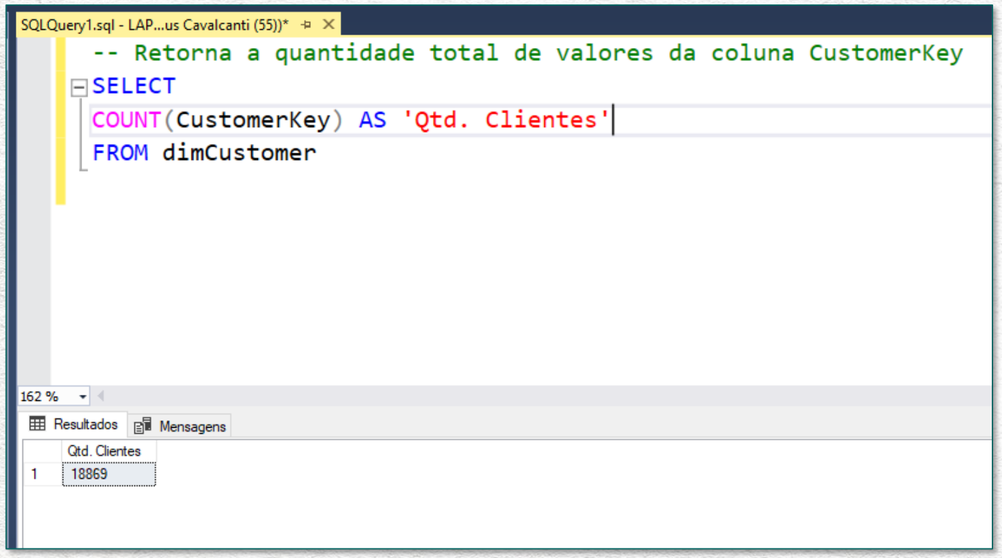
O QUE VEREMOS NESTE MÓDULO

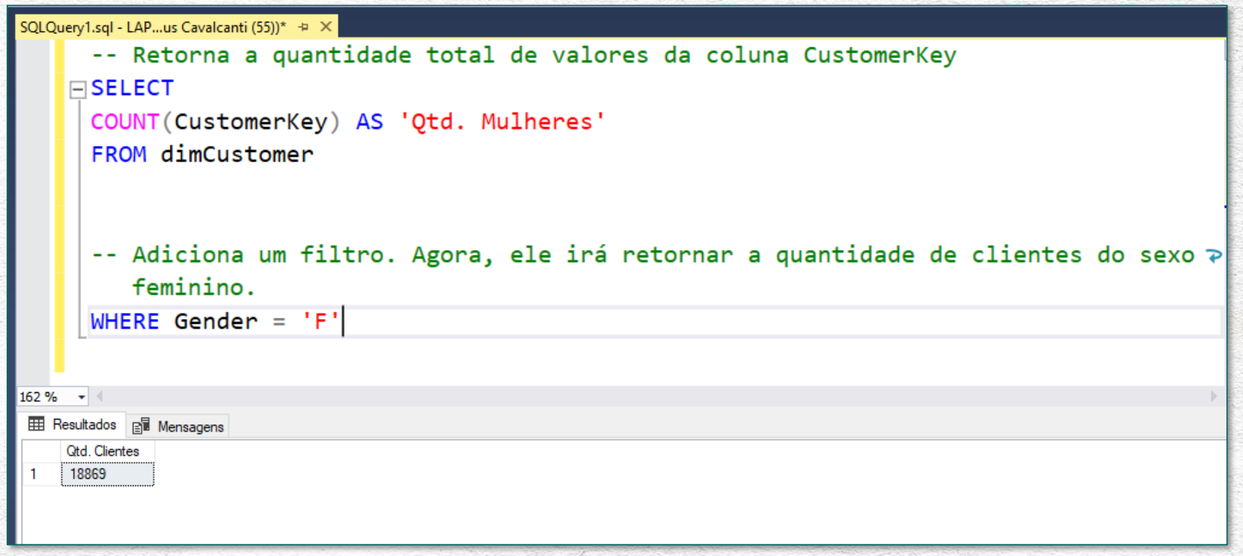
MÓDULO 5 FUNÇÕES DE AGREGAÇÃO

01	COUNT: Contando valores em uma tabela	116
02	COUNT+DISTINCT: Contando valores distintos	117
03	SUM: Somando os valores de uma coluna	118
04	AVG: Tirando a média dos valores de uma coluna	119
05	MIN e MAX: Mínimo e Máximo valor de uma coluna	120
06	EXERCÍCIOS	121
07	GABARITOS	123

COUNT: Contando valores em uma tabela

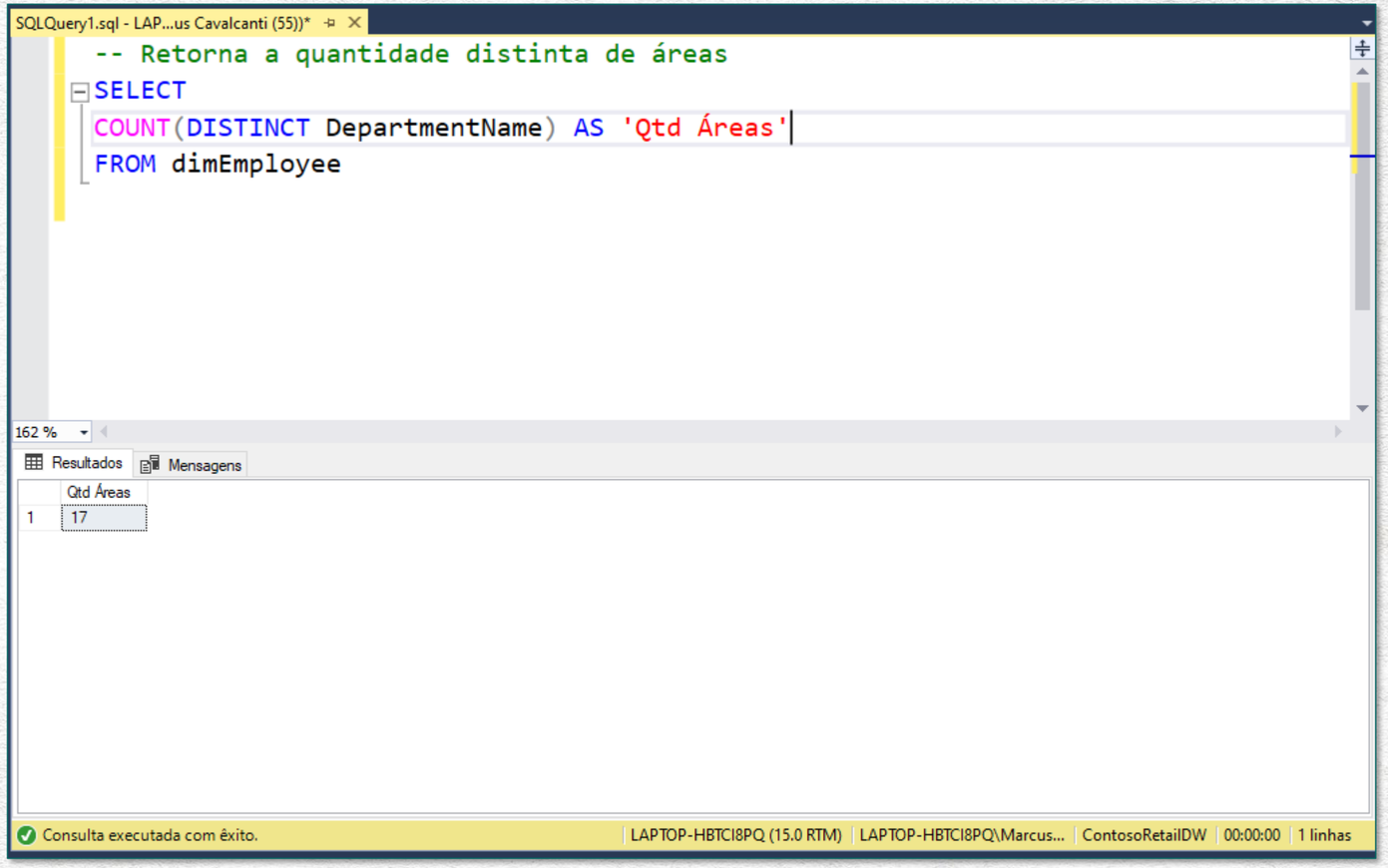
A função COUNT() retorna a quantidade de linhas de uma coluna. É possível fazer uma contagem e especificar um determinado critério.





COUNT+DISTINCT: Contando valores distintos

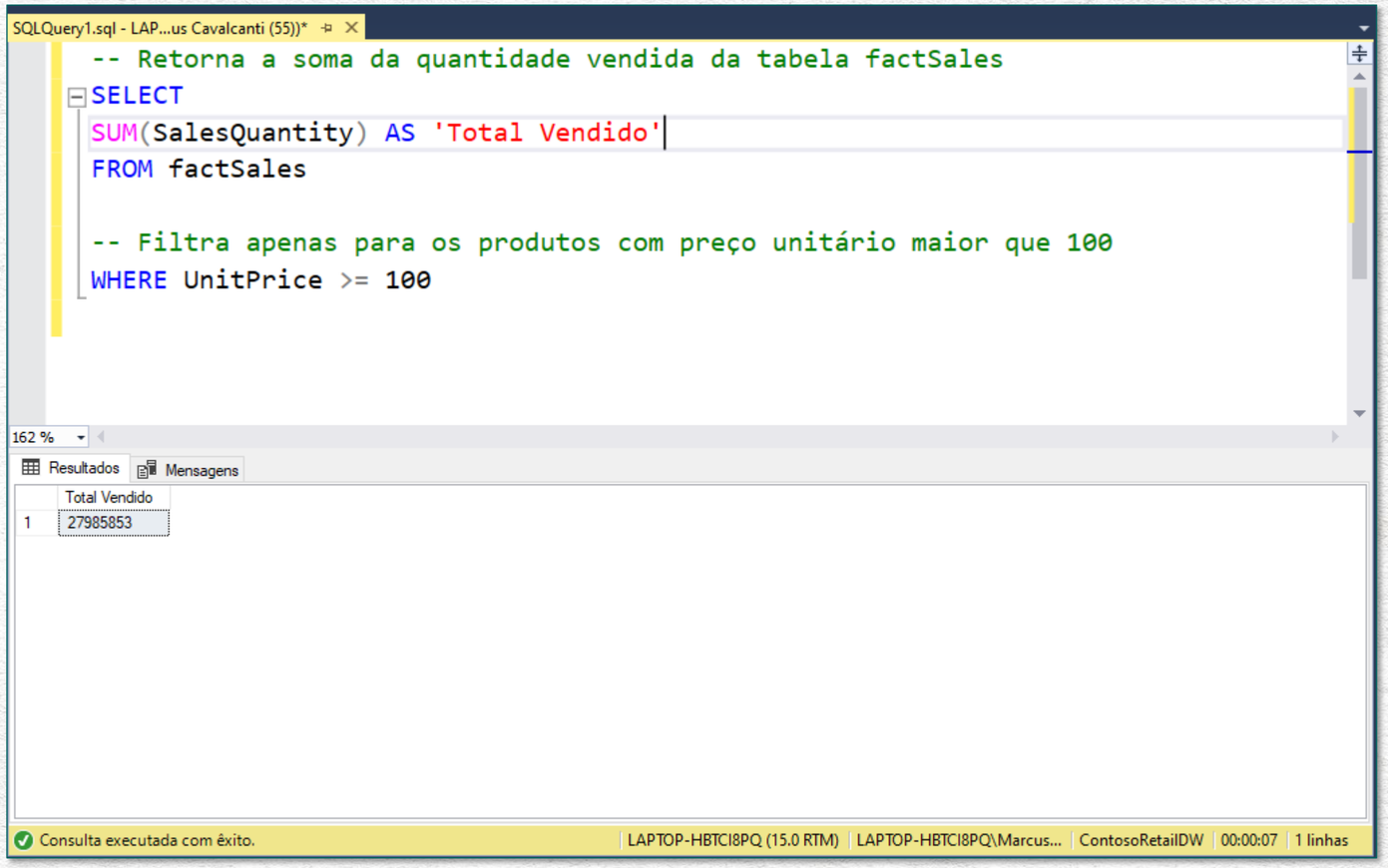
Podemos usar o COUNT() combinado com o DISTINCT() para fazer uma contagem distinta dos valores de uma coluna.



SUM: Somando os valores de uma coluna

Para retornar a soma total de uma determinada coluna, usamos o SUM().

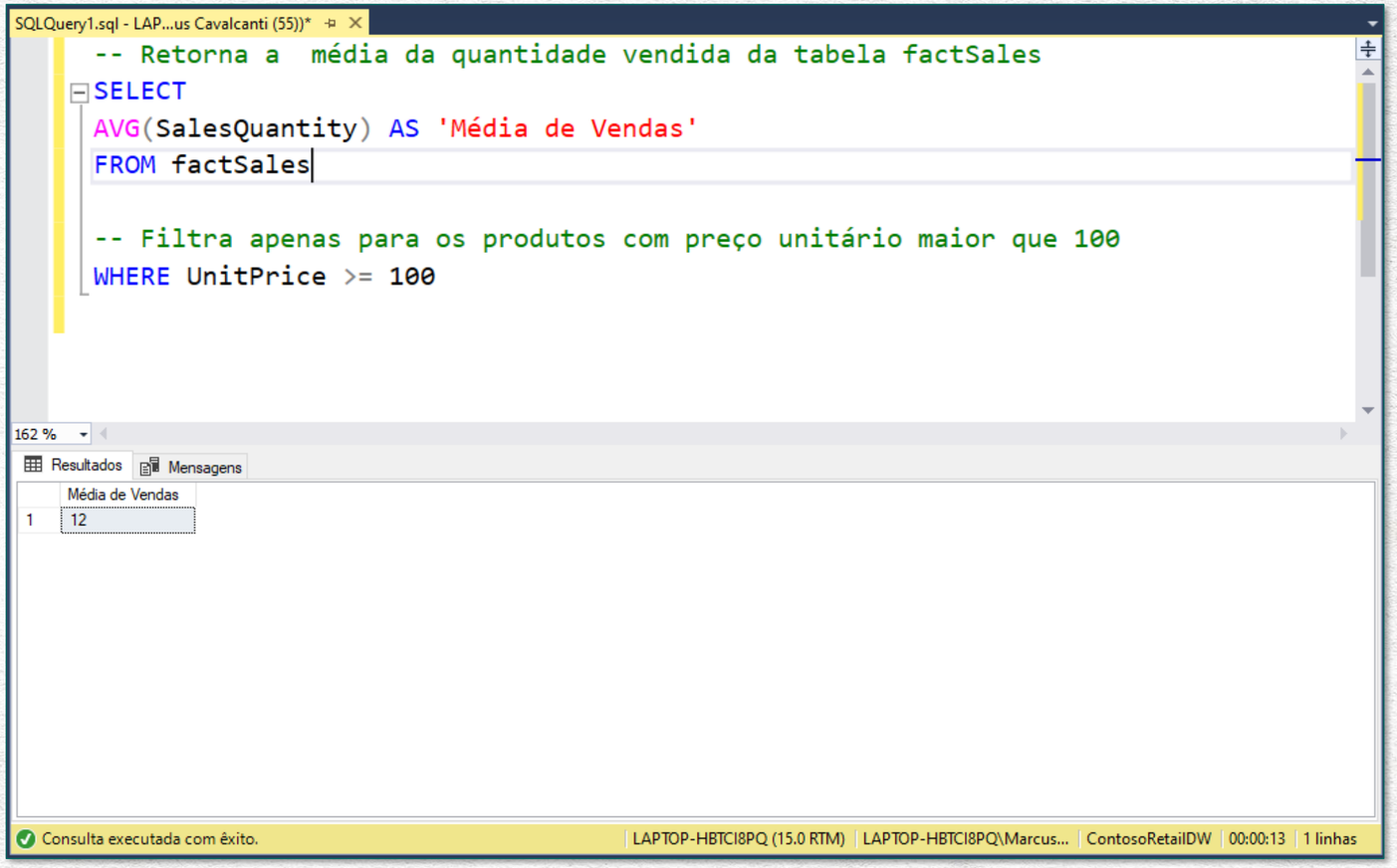
Também é possível fazer uma soma e especificar um determinado critério.



AVG: Tirando a média dos valores de uma coluna

Para retornar a média de uma determinada coluna, usamos o AVG().

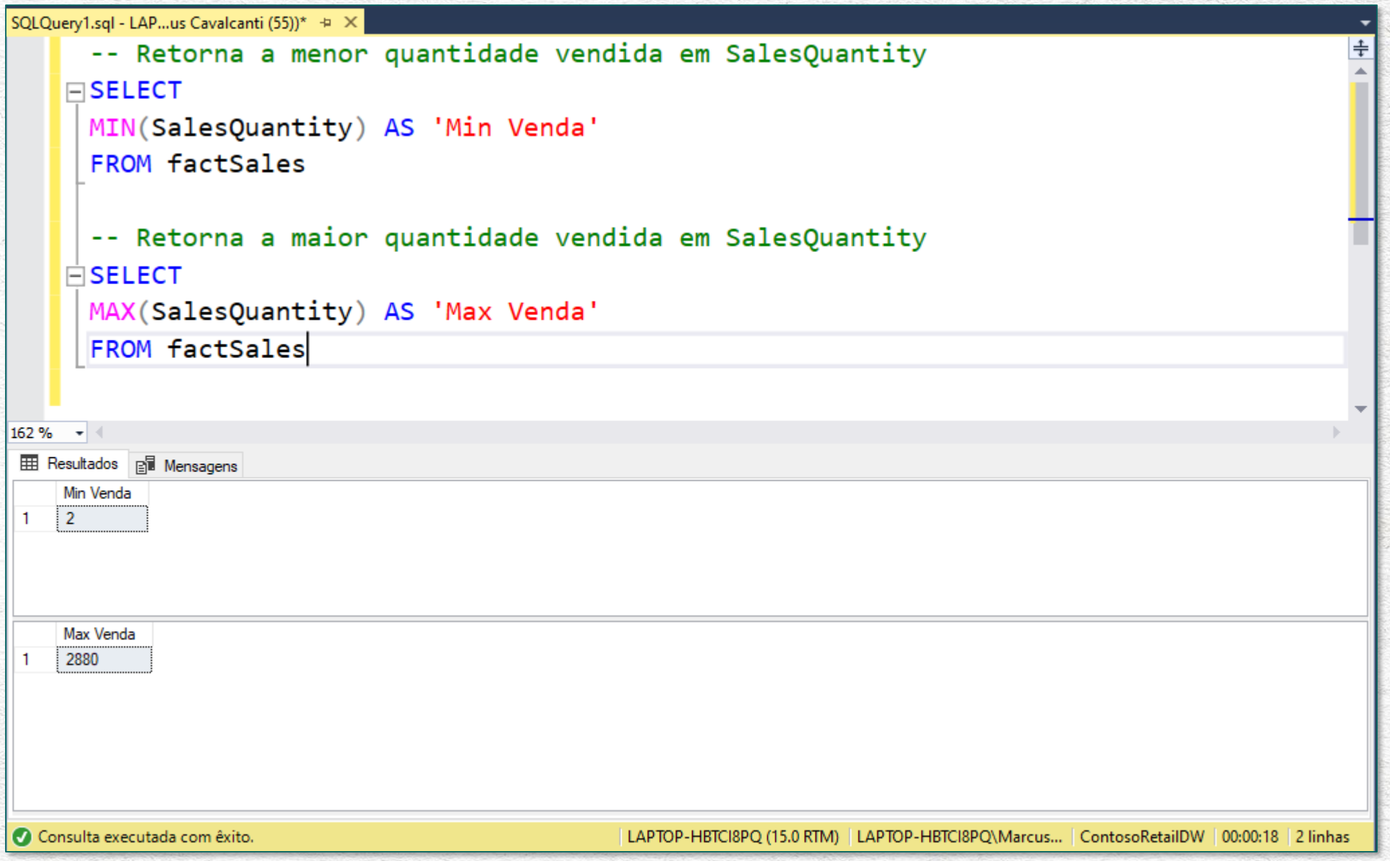
Também é possível fazer uma média e especificar um determinado critério.



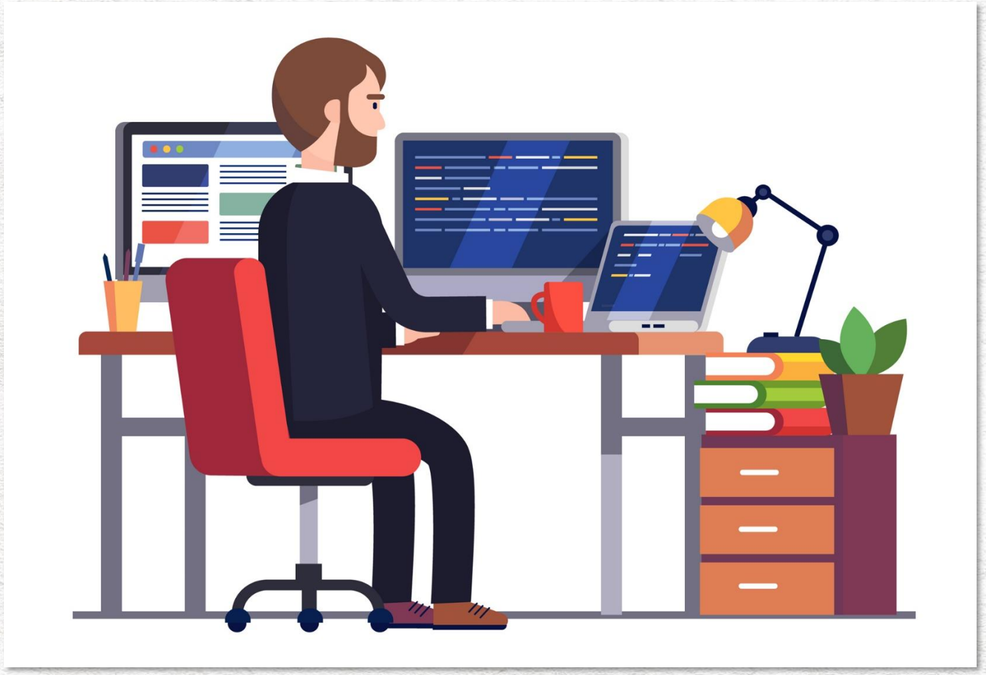
MIN e MAX: Mínimo e Máximo valor de uma coluna

O MIN() retorna o mínimo valor de uma determinada coluna.

O MAX() retorna o máximo valor de uma determinada coluna.



EXERCÍCIOS



1 Questão 1

O gerente comercial pediu a você uma análise da Quantidade Vendida e Quantidade Devolvida para o canal de venda mais importante da empresa: Store.

Utilize uma função SQL para fazer essas consultas no seu banco de dados. Obs: Faça essa análise considerando a tabela FactSales.

2 Questão 2

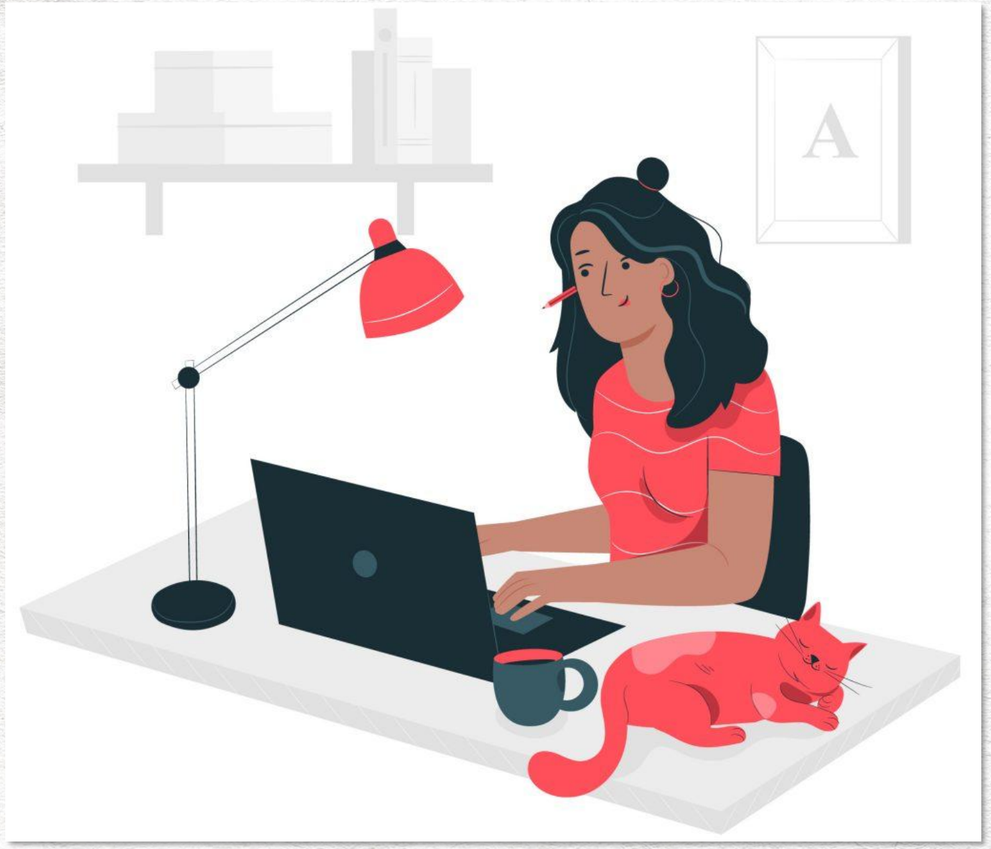
Uma nova ação no setor de Marketing precisará avaliar a média salarial de todos os clientes da empresa, mas apenas de ocupação Professional. Utilize um comando SQL para atingir esse resultado.

3 Questão 3

Você precisará fazer uma análise da quantidade de funcionários das lojas registradas na empresa. O seu gerente te pediu os seguintes números e informações:

a. Quantos funcionários tem a loja com mais funcionários?
b. Qual é o nome dessa loja?
c. Quantos funcionários tem a loja com menos funcionários?
d. Qual é o nome dessa loja?

EXERCÍCIOS



4

Questão 4

A área de RH está com uma nova ação para a empresa, e para isso precisa saber a quantidade total de funcionários do sexo Masculino e do sexo Feminino.

a. Descubra essas duas informações utilizando o SQL.

b. O funcionário e a funcionária mais antigos receberão uma homenagem. Descubra as seguintes informações de cada um deles: Nome, E-mail, Data de Contratação.

5

Questão 5

Agora você precisa fazer uma análise dos produtos. Será necessário descobrir as seguintes informações:

a. Quantidade distinta de cores de produtos.
b. Quantidade distinta de marcas
c. Quantidade distinta de classes de produto

Para simplificar, você pode fazer isso em uma mesma consulta.

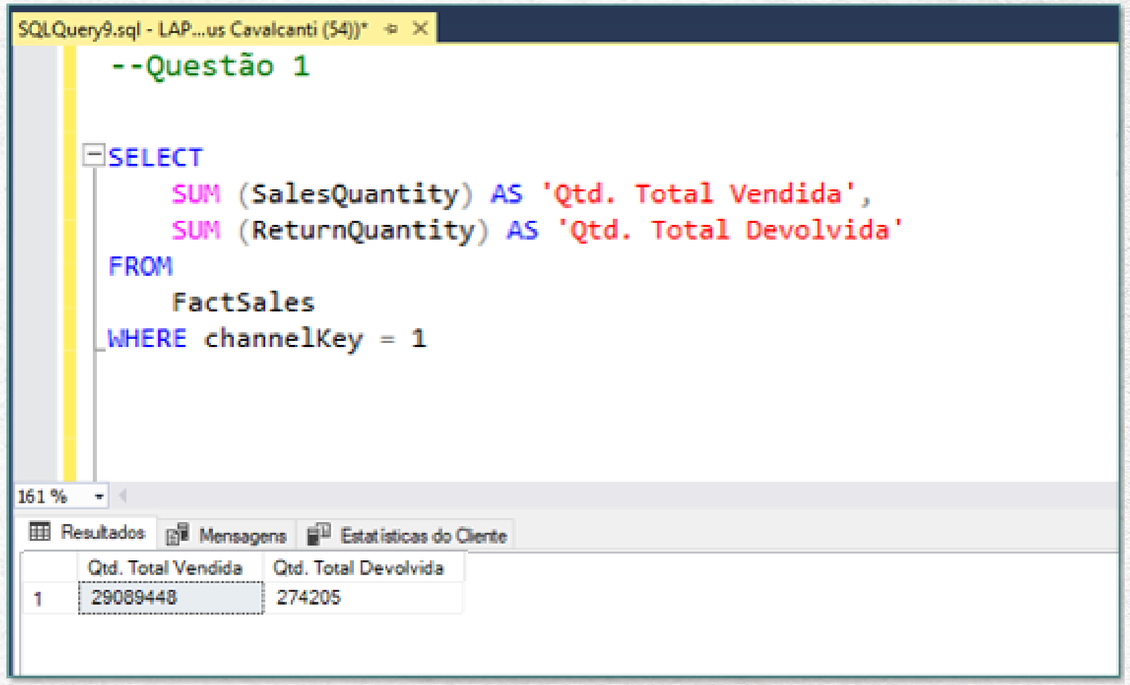
GABARITOS

1

Questão 1

O gerente comercial pediu a você uma análise da Quantidade Vendida e Quantidade Devolvida para o canal de venda mais importante da empresa: Store.

Utilize uma função SQL para fazer essas consultas no seu banco de dados. Obs: Faça essa análise considerando a tabela FactSales.

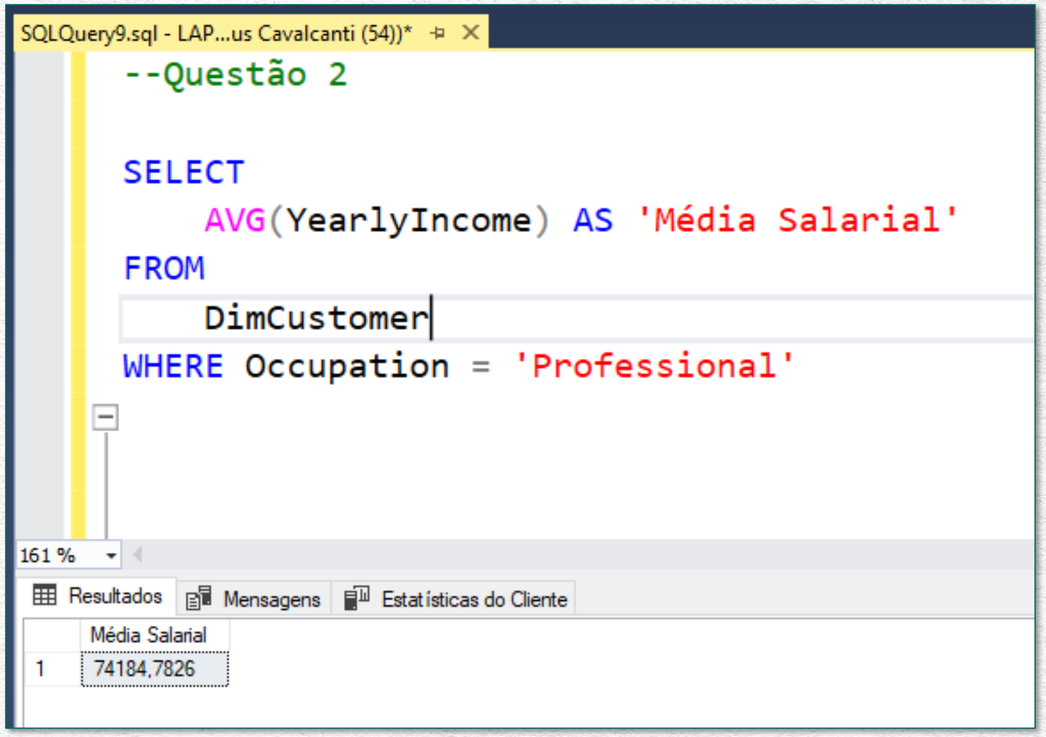


GABARITOS

2

Questão 2

Uma nova ação no setor de Marketing precisará avaliar a média salarial de todos os clientes da empresa, mas apenas de ocupação Professional. Utilize um comando SQL para atingir esse resultado.



GABARITOS

3

Questão 3

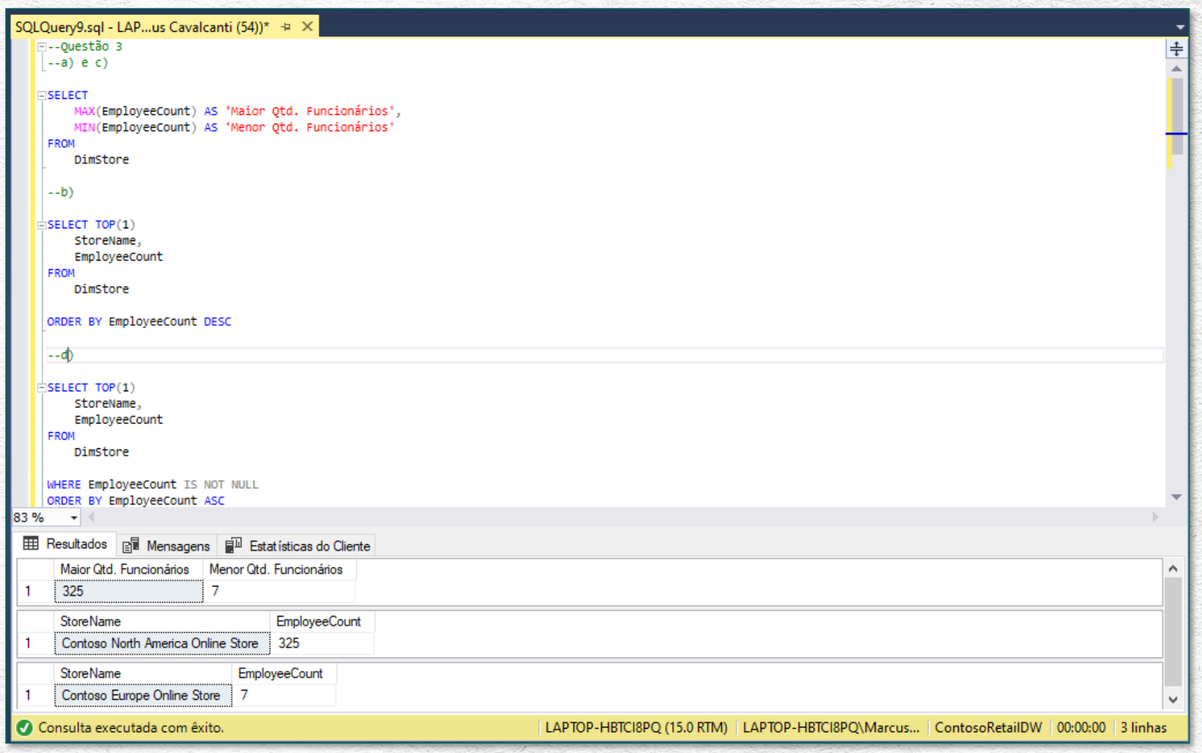
Você precisará fazer uma análise da quantidade de funcionários das lojas registradas na empresa. O seu gerente te pediu os seguintes números e informações:

a. Quantos funcionários tem a loja com mais funcionários?

b. Qual é o nome dessa loja?

c. Quantos funcionários tem a loja com menos funcionários?

d. Qual é o nome dessa loja?



GABARITOS

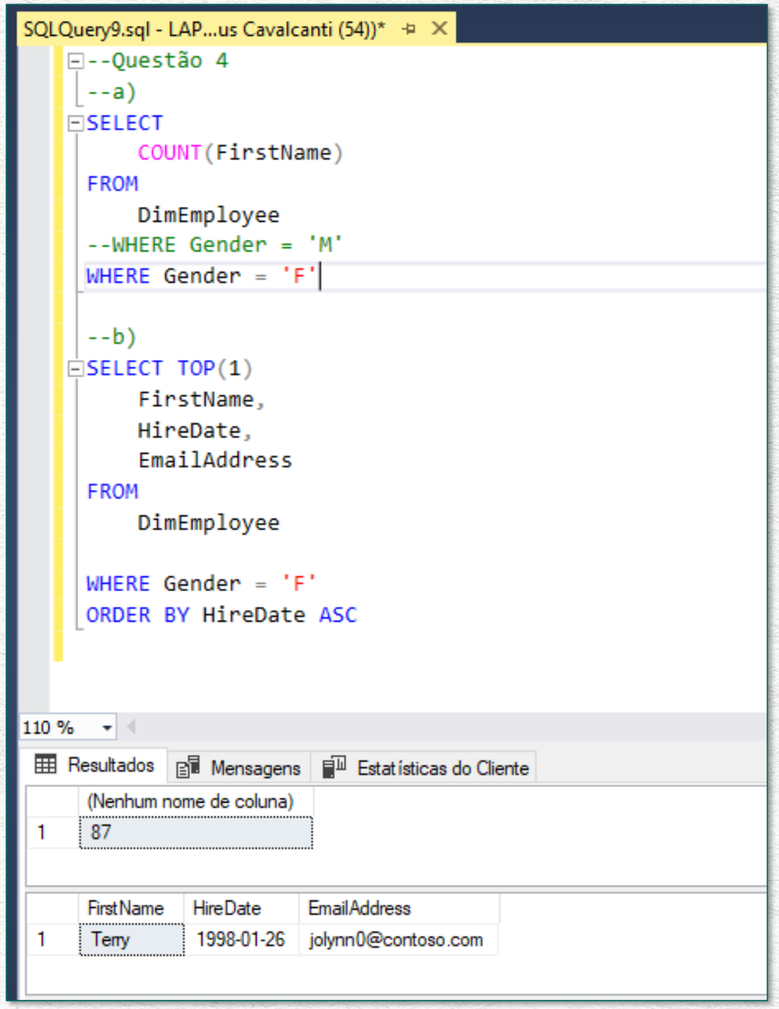
4

Questão 4

A área de RH está com uma nova ação para a empresa, e para isso precisa saber a quantidade total de funcionários do sexo Masculino e do sexo Feminino.

a. Descubra essas duas informações utilizando o SQL.

b. O funcionário e a funcionária mais antigos receberão uma homenagem. Descubra as seguintes informações de cada um deles: Nome, E-mail, Data de Contratação.



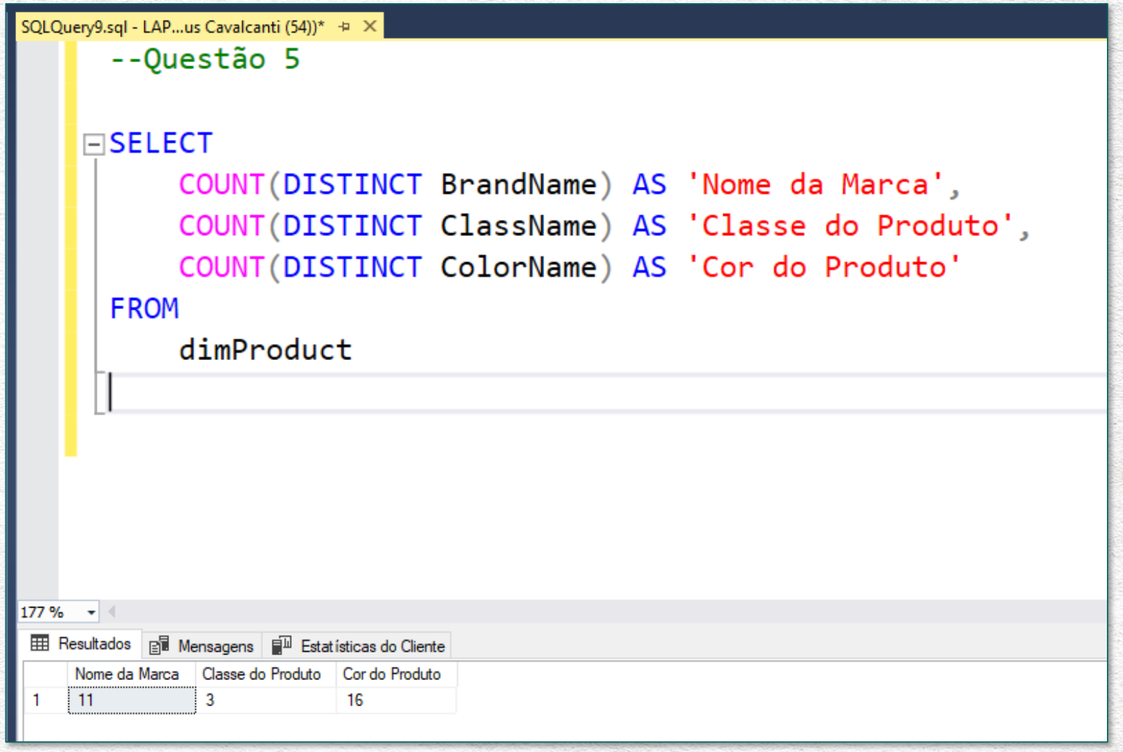
GABARITOS

5 Questão 5

Agora você precisa fazer uma análise dos produtos. Será necessário descobrir as seguintes informações:

a. Quantidade distinta de cores de produtos.
b. Quantidade distinta de marcas
c. Quantidade distinta de classes de produto

Para simplificar, você pode fazer isso em uma mesma consulta.



MÓDULO 6

SQL

Criando agrupamentos

O QUE VEREMOS NESTE MÓDULO

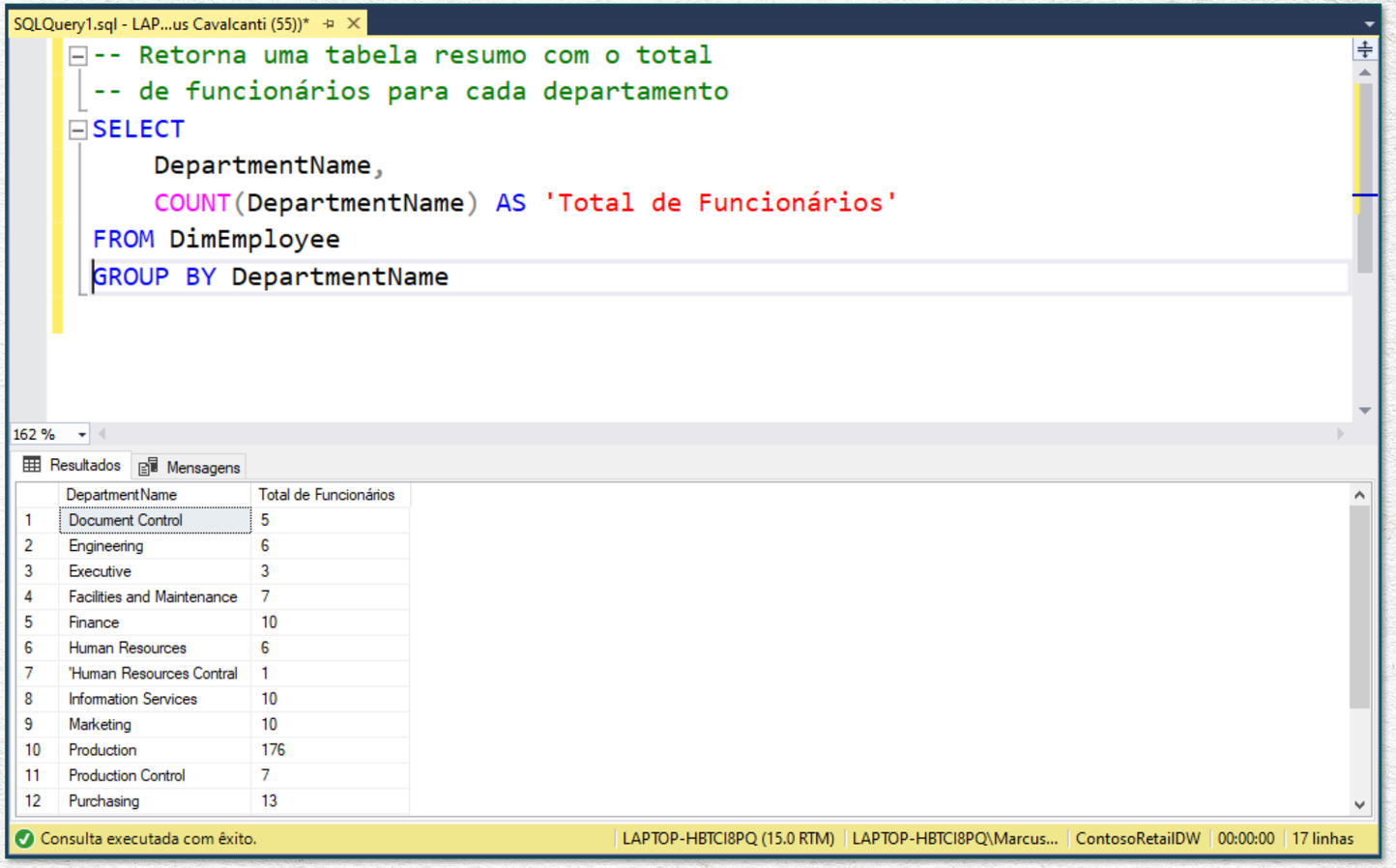
MÓDULO 6 CRIANDO AGRUPAMENTOS

01	GROUP BY: Criando agrupamentos	130
02	GROUP BY+ORDER BY: Ordenando agrupamentos	131
03	GROUP BY+WHERE: Filtrando a tabela principal antes de agrupar	132
04	GROUP BY+HAVING: Filtrando um agrupamento	133
05	WHERE vs. HAVING: Qual é a diferença?	134
06	EXERCÍCIOS	135
07	GABARITOS	139

GROUP BY: Criando agrupamentos

O comando GROUP BY permite agrupar valores de acordo com uma coluna, por exemplo “total de funcionários de acordo com o departamento".

O GROUP BY é usado junto com funções de agregação (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar valores de acordo com uma ou mais colunas.



	DepartmentName	Total de Funcionários
1	Document Control	5
2	Engineering	6
3	Executive	3
4	Facilities and Maintenance	7
5	Finance	10
6	Human Resources	6
7	'Human Resources Contral	1
8	Information Services	10
9	Marketing	10
10	Production	176
11	Production Control	7
12	Purchasing	13

GROUP BY+ORDER BY: Ordenando agrupamentos

O comando GROUP BY combinado com o ORDER BY permite que a tabela agrupada seja também ordenada.

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
--Agrupa a tabela DimStore de acordo com a contagem
-- de funcionários e ordena de acordo com essa quantidade
SELECT
    StoreType,
    SUM(EmployeeCount) AS 'Qtd. Total Funcionários'
FROM
    DimStore
GROUP BY StoreType
ORDER BY SUM(EmployeeCount) DESC
```

162 %

Resultados | Mensagens

	StoreType	Qtd. Total Funcionários
1	Store	10535
2	Online	340
3	Catalog	120
4	Reseller	44

GROUP BY+WHERE: Filtrando a tabela principal antes de agrupar

O comando GROUP BY combinado com o WHERE permite que a tabela principal seja filtrada antes de o agrupamento ser criado.

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Cria um agrupamento por Cor do Produto,
-- mas antes considera apenas os produtos da marca Contoso
SELECT
    ColorName AS 'Cor do Produto',
    COUNT(ColorName) AS 'Qtd. Total'
FROM
    DimProduct
WHERE BrandName = 'Contoso'
GROUP BY ColorName
```

162 %

Resultados | Mensagens

	Cor do Produto	Qtd. Total
1	Black	174
2	Blue	45
3	Brown	15
4	Gold	10
5	Green	15
6	Grey	95
7	Orange	8
8	Pink	21
9	Purple	1
10	Red	36
11	Silver	109
12	Silver Grey	4

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 15 linhas

Índice Geral

Índice deste Módulo

GROUP BY+HAVING: Filtrando um agrupamento

O comando GROUP BY combinado com o HAVING permite que a tabela agrupada seja filtrada e que apenas os valores totais que atendam ao critério do HAVING sejam considerados.

SQLQuery1.sql - LAP...us Cavalcanti (55))*

```sql
-- Cria um agrupamento por Marca.
-- Em seguida, é utilizado o HAVING para
-- filtrar a tabela agrupada.
SELECT
    BrandName AS 'Marca',
    COUNT(BrandName) AS 'Total Marca'
FROM
    DimProduct
GROUP BY BrandName
HAVING COUNT(BrandName) >= 200
```

147 %

Resultados | Mensagens

	Marca	Total Marca
1	Litware	264
2	Contoso	710
3	Proseware	244
4	Fabrikam	267

WHERE vs. HAVING: Qual é a diferença?

É muito importante que fique claro pra você a diferença entre o comando WHERE e o HAVING.

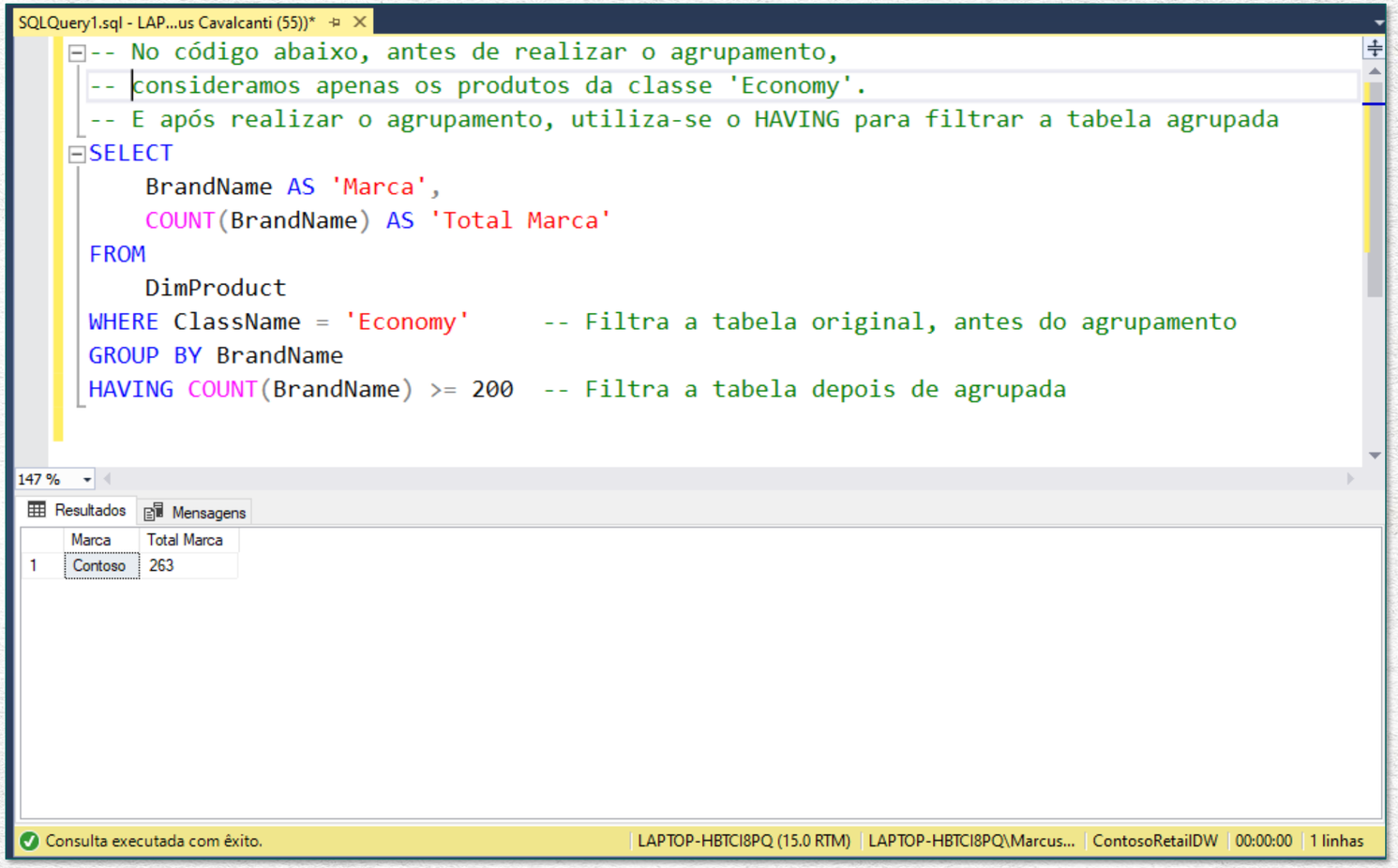
Para entender a diferença entre os dois, vale lembrar que o GROUP BY **permite que a gente crie um agrupamento a partir de uma tabela principal**.

O comando WHERE será utilizado para filtrar a tabela principal antes de criar o agrupamento.

Já o HAVING será utilizado para filtrar a tabela agrupada.

Portanto, podemos entender que:

- O **WHERE** será utilizado antes do agrupamento ser feito.
- O **HAVING** será um filtro utilizado após o agrupamento ser feito.



EXERCÍCIOS

1 Questão 1

a. Faça um resumo da quantidade vendida (SalesQuantity) de acordo com o canal de vendas (channelkey).

b. Faça um agrupamento mostrando a quantidade total vendida (SalesQuantity) e quantidade total devolvida (Return Quantity) de acordo com o ID das lojas (StoreKey).

c. Faça um resumo do valor total vendido (SalesAmount) para cada canal de venda, mas apenas para o ano de 2007.

2 Questão 2

Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesAmount) por produto (ProductKey).

a. A tabela final deverá estar ordenada de acordo com a quantidade vendida e, além disso, mostrar apenas os produtos que tiveram um resultado final de vendas maior do que $5.000.000.

b. Faça uma adaptação no exercício anterior e mostre os Top 10 produtos com mais vendas. Desconsidere o filtro de $5.000.000 aplicado.

EXERCÍCIOS

3

Questão 3

a. Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o ID (CustomerKey) do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

b. Feito isso, faça um agrupamento de total vendido (SalesQuantity) por ID do produto e descubra quais foram os top 3 produtos mais comprados pelo cliente da letra a).

4

Questão 4

a. Faça um agrupamento e descubra a quantidade total de produtos por marca.

b. Determine a média do preço unitário (UnitPrice) para cada ClassName.

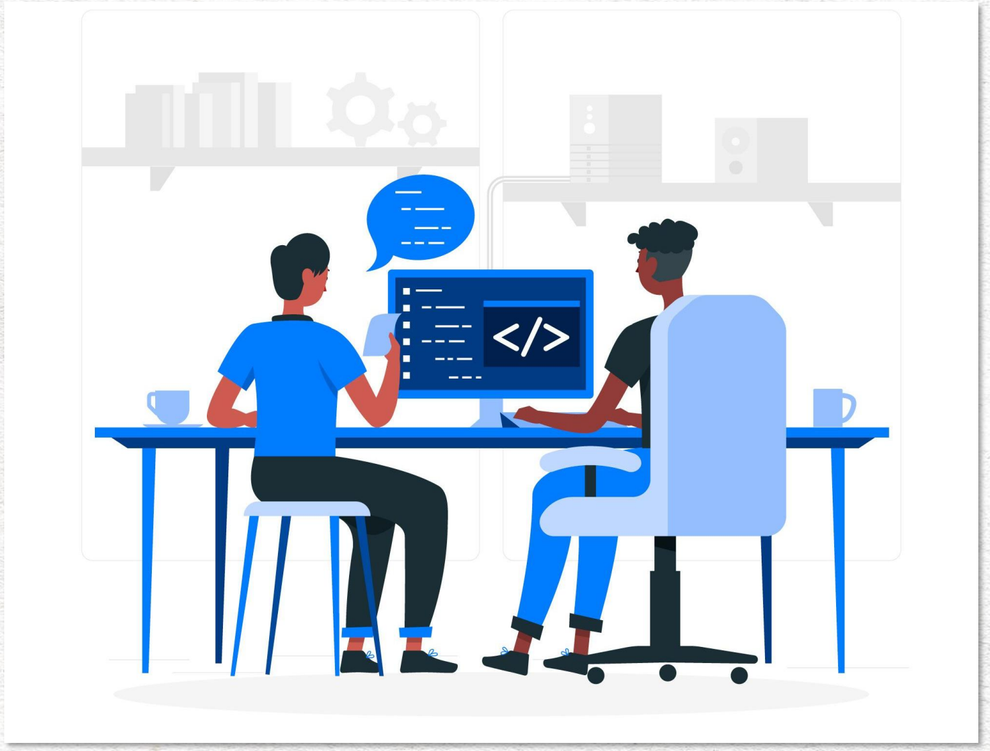
c. Faça um agrupamento de cores e descubra o peso total que cada cor de produto possui.

5

Questão 5

Você deverá descobrir o peso total para cada tipo de produto (StockTypeName). A tabela final deve considerar apenas a marca 'Contoso' e ter os seus valores classificados em ordem decrescente.

EXERCÍCIOS



6 Questão 6

Você seria capaz de confirmar se todas as marcas dos produtos possuem à disposição todas as 16 opções de cores?

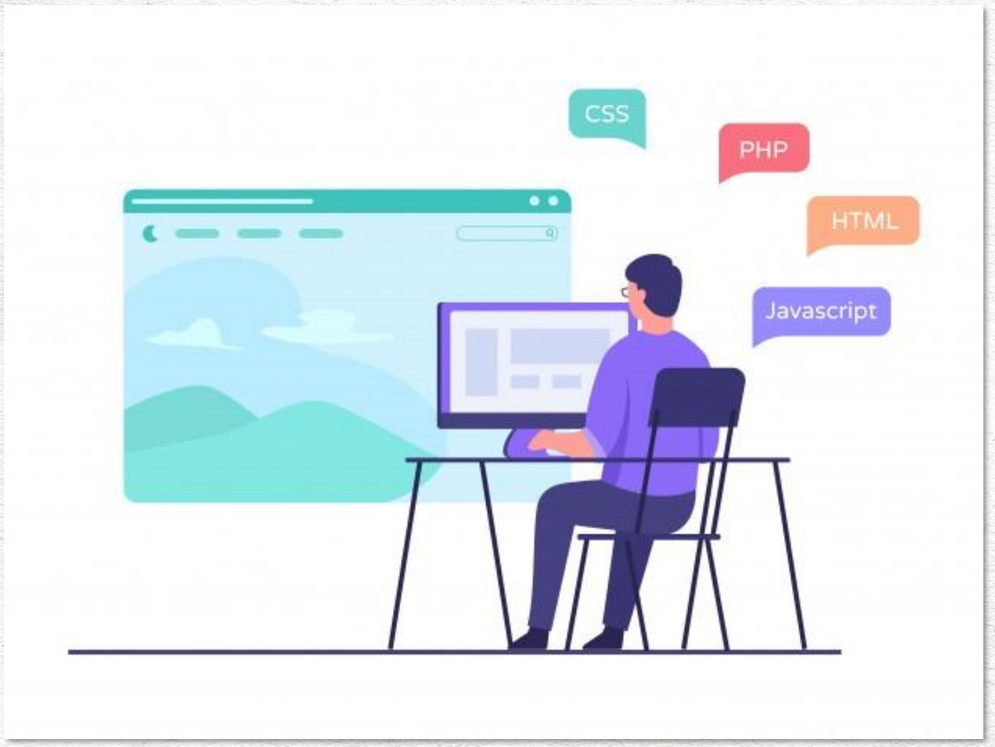
7 Questão 7

Faça um agrupamento para saber o total de clientes de acordo com o Sexo e também a média salarial de acordo com o Sexo. Corrija qualquer resultado “inesperado” com os seus conhecimentos em SQL.

8 Questão 8

Faça um agrupamento para descobrir a quantidade total de clientes e a média salarial de acordo com o seu nível escolar. Utilize a coluna Education da tabela DimCustomer para fazer esse agrupamento.

EXERCÍCIOS



9

Questão 9

Faça uma tabela resumo mostrando a quantidade total de funcionários de acordo com o Departamento (DepartmentName). Importante: Você deverá considerar apenas os funcionários ativos.

10

Questão 10

Faça uma tabela resumo mostrando o total de VacationHours para cada cargo (Title). Você deve considerar apenas as mulheres, dos departamentos de Production, Marketing, Engineering e Finance, para os funcionários contratados entre os anos de 1999 e 2000.

GABARITOS

1

Questão 1

a. Faça um resumo da quantidade vendida (SalesQuantity) de acordo com o canal de vendas (channelkey).

b. Faça um agrupamento mostrando a quantidade total vendida (SalesQuantity) e quantidade total devolvida (Return Quantity) de acordo com o ID das lojas (StoreKey).

c. Faça um resumo do valor total vendido (SalesAmount) para cada canal de venda, mas apenas para o ano de 2007.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 1
--a)
SELECT
    channelKey,
    SUM(SalesQuantity)
FROM
    FactSales
GROUP BY channelKey

--b)
SELECT
    StoreKey,
    SUM(SalesQuantity),
    SUM(ReturnQuantity)
FROM
    FactSales
GROUP BY StoreKey

--c)
SELECT
    channelKey,
    SUM(SalesQuantity)
FROM
    FactSales
WHERE DateKey BETWEEN '01/01/2007' AND '31/12/2007'
GROUP BY channelKey
```

GABARITOS

2

Questão 2

Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesAmount) por produto (ProductKey).

a. A tabela final deverá estar ordenada de acordo com a quantidade vendida e, além disso, mostrar apenas os produtos que tiveram um resultado final de vendas maior do que $5.000.000.

b. Faça uma adaptação no exercício anterior e mostre os Top 10 produtos com mais vendas. Desconsidere o filtro de $5.000.000 aplicado.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 2
--a)
SELECT
    ProductKey,
    SUM(SalesAmount)
FROM
    FactSales
GROUP BY ProductKey
HAVING SUM(SalesAmount) >= 5000000
ORDER BY SUM(SalesAmount) DESC

--b)
SELECT TOP(10)
    ProductKey,
    SUM(SalesAmount)
FROM
    FactSales
GROUP BY ProductKey
ORDER BY SUM(SalesAmount) DESC
```

GABARITOS

3

Questão 3

a. Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o ID (CustomerKey) do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

b. Feito isso, faça um agrupamento de total vendido (SalesQuantity) por ID do produto e descubra quais foram os top 3 produtos mais comprados pelo cliente da letra a).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 3
--a)
SELECT TOP(1)
    CustomerKey,
    SUM(SalesQuantity)
FROM
    FactOnlineSales
GROUP BY CustomerKey
ORDER BY SUM(SalesQuantity) DESC

--b)
SELECT
    TOP (3)
    ProductKey AS 'ID do Produto',
    SUM (SalesQuantity) AS 'Total Vendido'
FROM
    FactOnlineSales
WHERE CustomerKey = 19037
GROUP BY ProductKey
ORDER BY SUM (SalesQuantity) DESC
```

GABARITOS

4

Questão 4

a. Faça um agrupamento e descubra a quantidade total de produtos por marca.

b. Determine a média do preço unitário (UnitPrice) para cada ClassName.

c. Faça um agrupamento de cores e descubra o peso total que cada cor de produto possui.

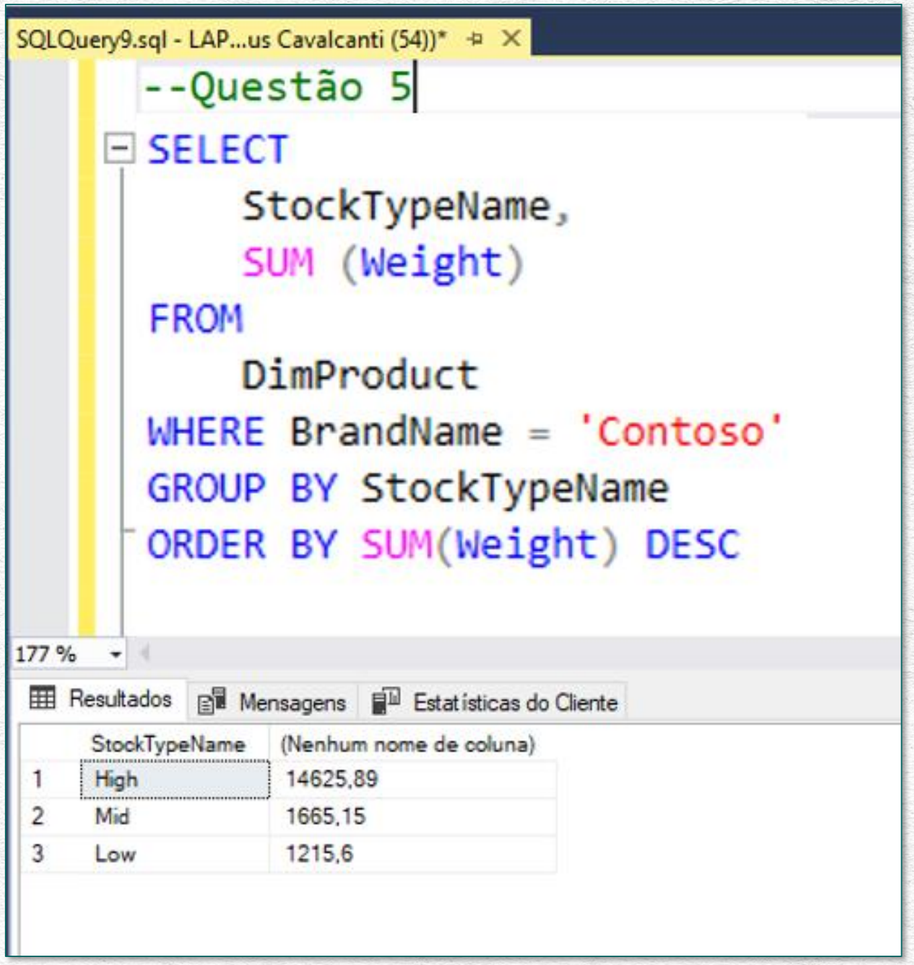
SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 4
--a)
SELECT
    BrandName,
    COUNT(*)
FROM
    DimProduct
GROUP BY BrandName

--b)
SELECT
    ClassName,
    AVG(UnitPrice)
FROM
    DimProduct
GROUP BY ClassName

--c)
SELECT
    ColorName,
    SUM(Weight)
FROM
    DimProduct
GROUP BY ColorName
```

GABARITOS

5 Questão 5

Você deverá descobrir o peso total para cada tipo de produto (StockTypeName). A tabela final deve considerar apenas a marca 'Contoso' e ter os seus valores classificados em ordem decrescente.



GABARITOS

6

Questão 6

Você seria capaz de confirmar se todas as marcas dos produtos possuem à disposição todas as 16 opções de cores? **Nenhuma marca possui um exemplar de cada cor.**

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 6
SELECT
    BrandName,
    COUNT(DISTINCT ColorName)
FROM
    DimProduct
GROUP BY BrandName
```

177 %

Resultados | Mensagens | Estatísticas do Cliente

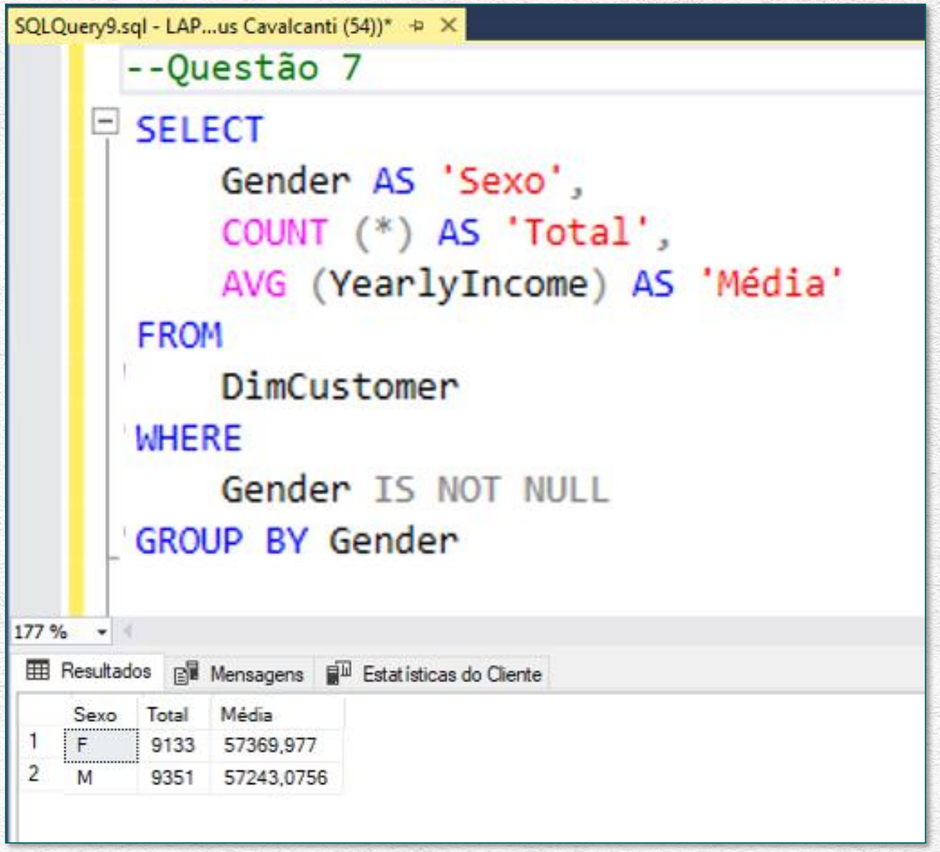
	BrandName	(Nenhum nome de coluna)
1	A. Datum	10
2	Adventure Works	7
3	Contoso	15
4	Fabrikam	12
5	Litware	12
6	Northwind Traders	9
7	Proseware	7
8	Southridge Video	10
9	Tailspin Toys	11
10	The Phone Company	6
11	Wide World Importers	12

GABARITOS

7

Questão 7

Faça um agrupamento para saber o total de clientes de acordo com o Sexo e também a média salarial de acordo com o Sexo. Corrija qualquer resultado “inesperado” com os seus conhecimentos em SQL.



GABARITOS

8

Questão 8

Faça um agrupamento para descobrir a quantidade total de clientes e a média salarial de acordo com o seu nível escolar. Utilize a coluna Education da tabela DimCustomer para fazer esse agrupamento.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 8
SELECT
    Education AS 'Nível Escolar',
    COUNT(Education) AS 'Qtd. Total por Escolaridade',
    AVG(YearlyIncome) AS 'Média Salarial'
FROM
    DimCustomer
WHERE Education IS NOT NULL
GROUP BY Education
```

177 %

Resultados | Mensagens | Estatísticas do Cliente

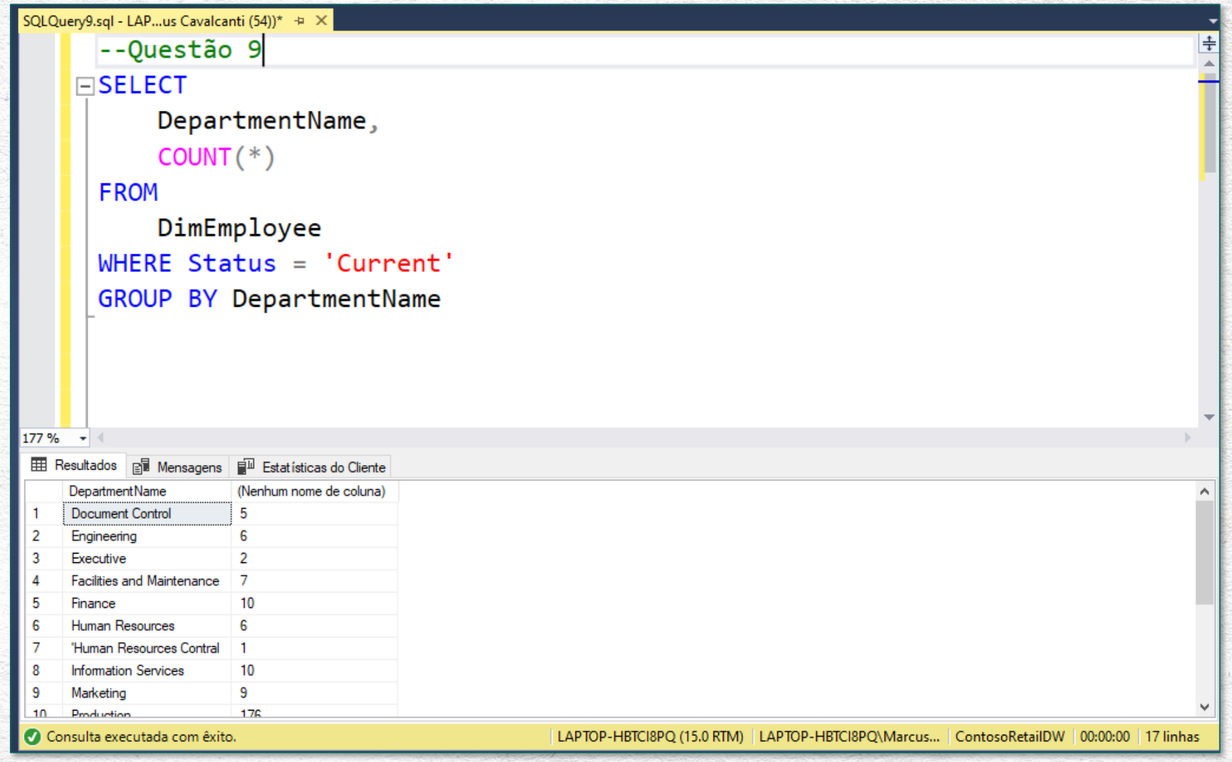
	Nível Escolar	Qtd. Total por Escolaridade	Média Salarial
1	High School	3294	49049,7874
2	Partial High School	1581	39468,6907
3	Partial College	5064	55211,2954
4	Graduate Degree	3189	66095,9548
5	Bachelors	5356	64395,0709

GABARITOS

9

Questão 9

Faça uma tabela resumo mostrando a quantidade total de funcionários de acordo com o Departamento (DepartmentName). Importante: Você deverá considerar apenas os funcionários ativos.



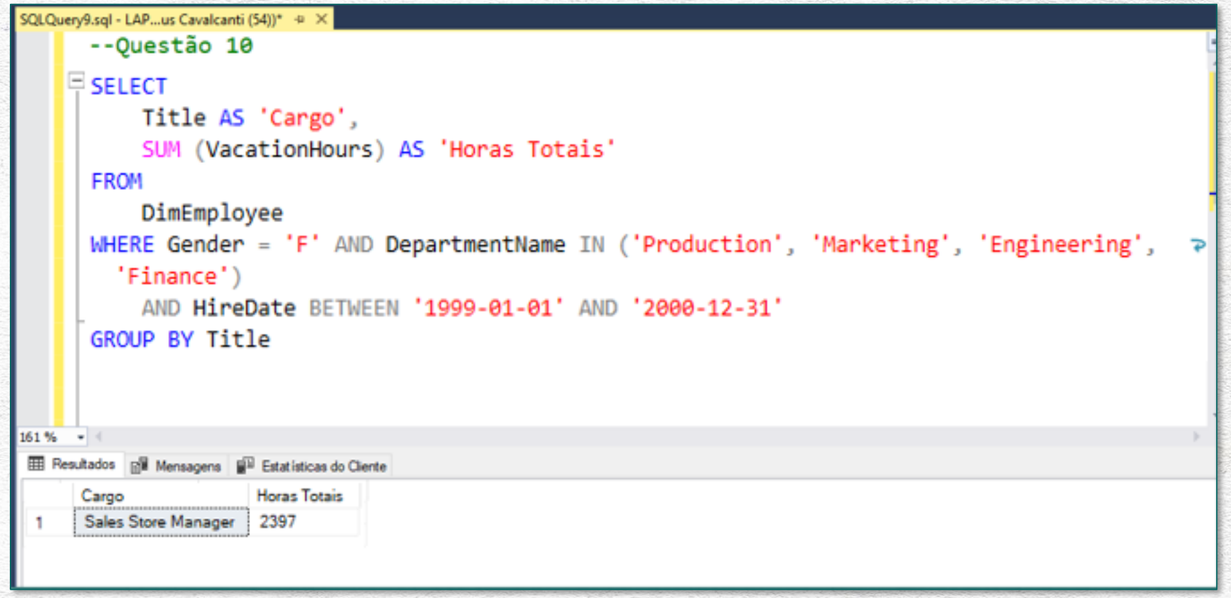
	DepartmentName	(Nenhum nome de coluna)
1	Document Control	5
2	Engineering	6
3	Executive	2
4	Facilities and Maintenance	7
5	Finance	10
6	Human Resources	6
7	'Human Resources Contral	1
8	Information Services	10
9	Marketing	9
10	Production	176

GABARITOS

10

Questão 10

Faça uma tabela resumo mostrando o total de VacationHours para cada cargo (Title). Você deve considerar apenas as mulheres, dos departamentos de Production, Marketing, Engineering e Finance, para os funcionários contratados entre os anos de 1999 e 2000.



MÓDULO 7

SQL

JOINs

O QUE VEREMOS NESTE MÓDULO

MÓDULO 7 JOINS

01	Chave Primária vs. Chave Estrangeira	151
02	Tabela Fato vs. Tabela Dimensão	152
03	JOIN	155
04	LEFT (OUTER) JOIN	156
05	RIGHT (OUTER) JOIN	157
06	INNER (OUTER) JOIN	158
07	FULL (OUTER) JOIN	159
08	LEFT (ANTI) JOIN	160
09	RIGHT (ANTI) JOIN	161
10	FULL (ANTI) JOIN	162
11	LEFT, RIGHT e INNER JOIN: exemplos práticos	163
12	FULL JOIN: exemplos práticos	169
13	LEFT, RIGHT e FULL ANTI JOIN: exemplos práticos	170
14	Como definir quem é a LEFT TABLE e a RIGHT TABLE	173
15	LEFT JOIN ou INNER JOIN: qual usar?	175
16	CROSS JOIN	176
17	Múltiplos Joins	177
18	UNION e UNION ALL	179
19	EXERCÍCIOS	181
20	GABARITOS	187

Chave Primária vs. Chave Estrangeira

Uma Chave Primária é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.

Já uma Chave Estrangeira é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

Como pode ser visto abaixo, a tabela **Categoria** possui uma coluna chamada id_subcategoria, com valores que não se repetem. Essa será a **Chave Primária**. Já na tabela de Produtos, a coluna de id_subcategoria também aparece, mas os valores se repetem. Isso porque podemos ter mais de um produto pertencendo à mesma categoria. Na tabela de **Produtos**, a coluna de id_subcategoria vai ser a **Chave Estrangeira** e vai permitir a gente relacionar os valores dessa coluna com a Chave Primária da tabela de Categoria.

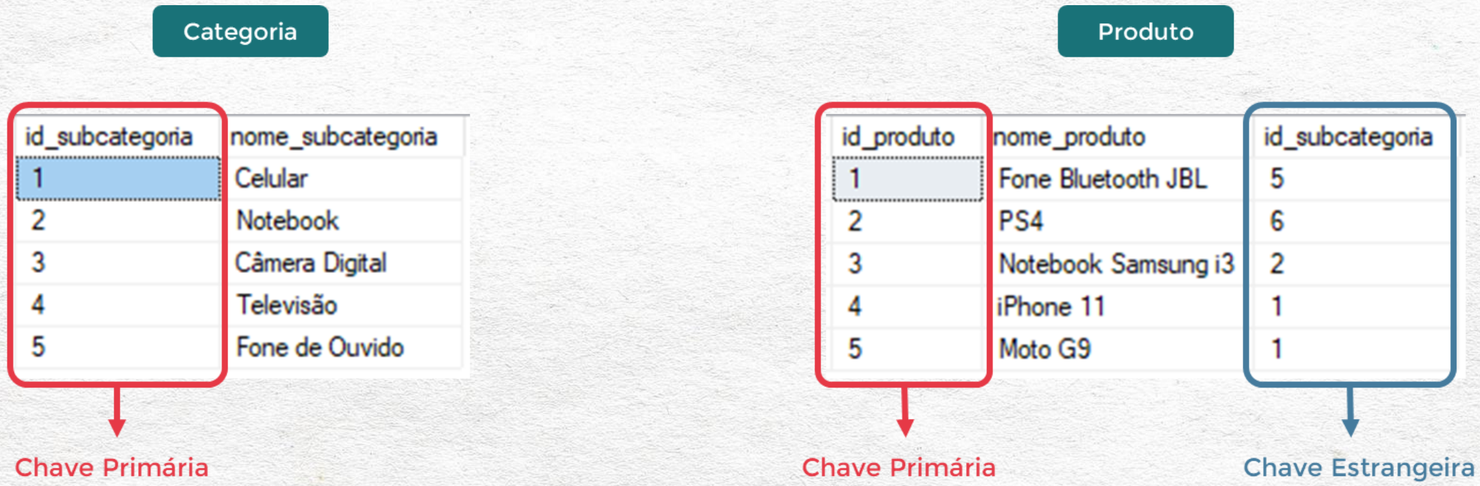


Tabela Fato vs. Tabela Dimensão

Você provavelmente deve ter notado em nossas tabelas do banco de dados Contoso que os nomes das tabelas possuem um certo padrão: alguns nomes começam com a inicial **Dim** e outros com a inicial **Fact**. Chegou a hora de entender essa nomenclatura.

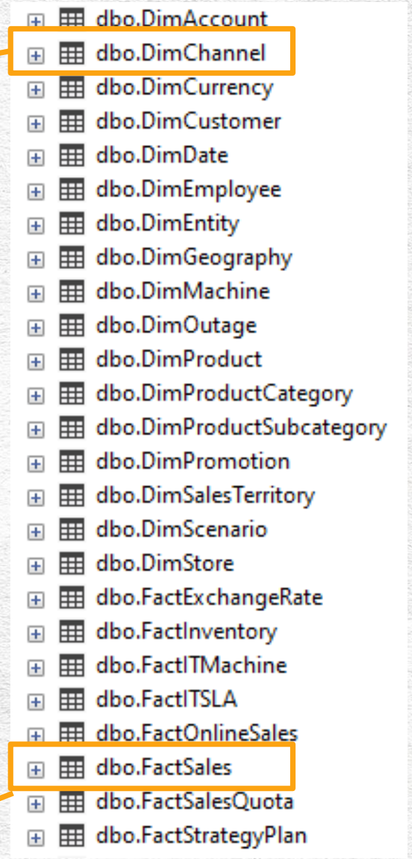
Vamos analisar as tabelas DimChannel e FactSales do nosso banco de dados Contoso, conforme mostrado no print ao lado.

A tabela DimChannel resume os 4 canais de venda da empresa Contoso, e a tabela FactSales é uma tabela que contém as vendas dos produtos.

Essas duas tabelas possuem características diferentes, que vamos entender agora.

ChannelKey	ChannelName
1	Store
2	Online
3	Catalog
4	Reseller

DimChannel



SalesKey	DateKey	ChannelKey	SalesQuantity
1	2007-01-02 00:00:00.000	1	8
2	2007-02-12 00:00:00.000	4	4
3	2008-01-24 00:00:00.000	1	9
4	2008-01-13 00:00:00.000	2	8
5	2008-01-22 00:00:00.000	2	24
6	2007-07-02 00:00:00.000	3	36
7	2007-11-19 00:00:00.000	4	6
8	2008-04-10 00:00:00.000	2	9
9	2008-07-14 00:00:00.000	2	24
10	2009-04-10 00:00:00.000	4	18

FactSales

Tabela Fato vs. Tabela Dimensão

Uma Tabela Dimensão é uma tabela que contém características de um determinado elemento: lojas, produtos, funcionários, clientes, etc.

Nesta tabela, nenhum dos elementos principais irá se repetir. É onde vamos encontrar nossas **chaves primárias**.

Uma Tabela Fato é uma tabela que vai registrar os fatos ou acontecimentos de uma empresa/negócio em determinados períodos de tempo (vendas, devoluções, aberturas de chamados, receitas, despesas, etc).

Geralmente é uma tabela com milhares de informações e composta essencialmente por colunas de ID usadas para buscar as informações complementares de uma tabela dimensão, conhecidas como **chaves estrangeiras**.

No exemplo ao lado, a FactSales é a nossa tabela Fato e a DimChannel é a nossa tabela Dimensão.

Agora faz sentido o porquê dessas iniciais '**Fact**' e '**Dim**' no nome das tabelas?

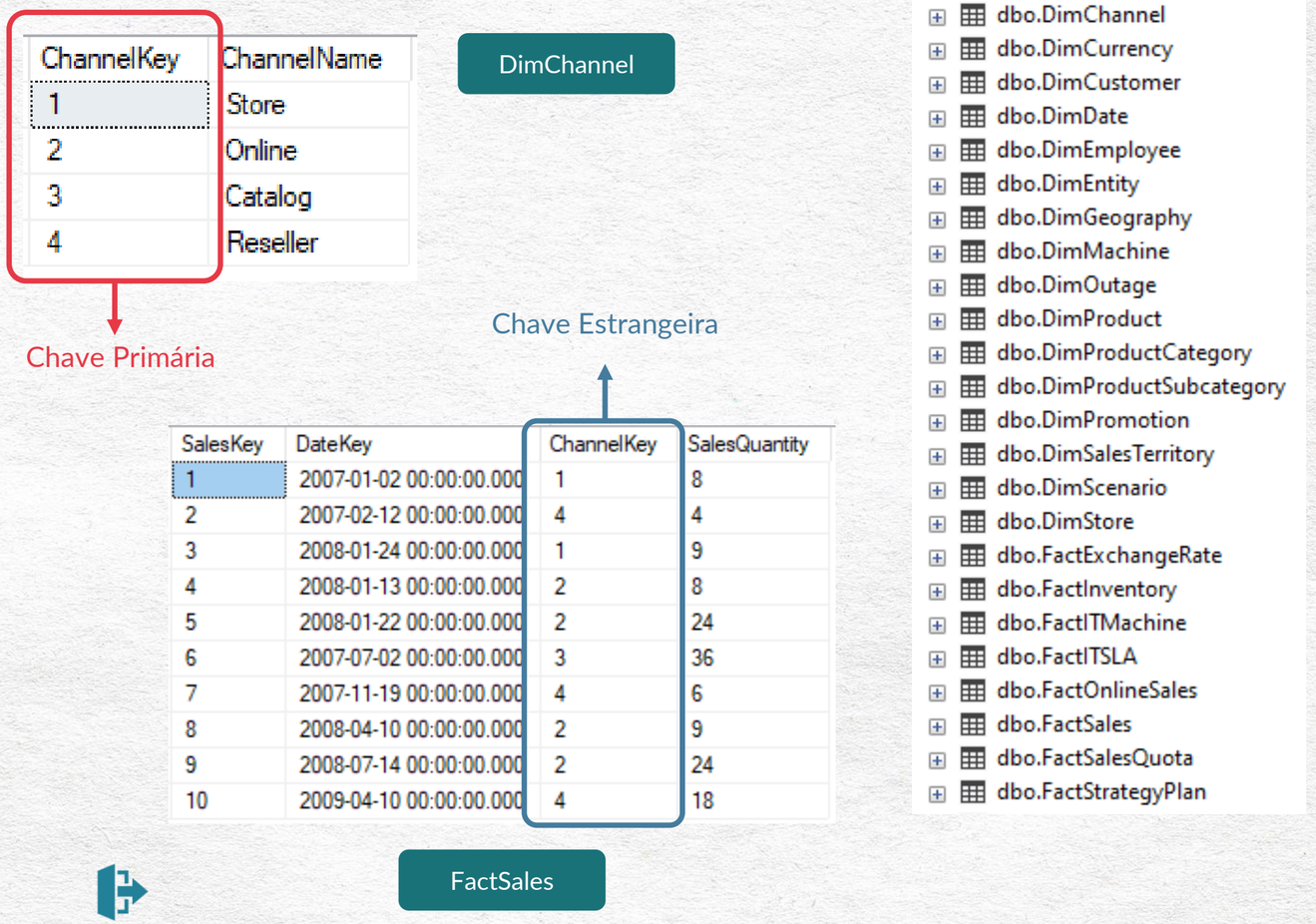


Tabela Fato vs. Tabela Dimensão

Não necessariamente uma relação acontece entre uma fato e uma dimensão.

Duas dimensões também podem se relacionar, como é o caso do exemplo ao lado.

O que não fazemos é uma relação entre duas fatos.

DimSubcategory

ProductSubcategoryKey	ProductSubcategoryName
1	MP4&MP3
2	Recorder
3	Radio
4	Recording Pen
5	Headphones

Chave Primária

Chave Estrangeira

ProductKey	ProductName	ProductSubcategoryKey
1	Contoso 512MB MP3 Player E51 Silver	1
2	Contoso 512MB MP3 Player E51 Blue	1
3	Contoso 1G MP3 Player E100 White	1
4	Contoso 2G MP3 Player E200 Silver	1
5	Contoso 2G MP3 Player E200 Red	1
6	Contoso 2G MP3 Player E200 Black	1
7	Contoso 2G MP3 Player E200 Blue	1
8	Contoso 4G MP3 Player E400 Silver	1
9	Contoso 4G MP3 Player E400 Black	1
10	Contoso 4G MP3 Player E400 Green	1

DimProduct

JOIN

Nas aulas anteriores vimos que existem dois tipos de tabelas: Dimensão e Fato. Essas tabelas podem ser relacionadas através de uma coluna: na tabela Dimensão, identificamos a chave Primária, que será relacionada com a chave Estrangeira da tabela Fato.

Mas pra que servem essas relações?

Essas relações serão criadas por meio do que chamamos de JOIN's. A tradução literal dessa palavra é "juntar", "unir". Os JOINs vão nos permitir fazer exatamente isso: juntar as nossas tabelas Fato e Dimensão, de forma a complementar as informações umas das outras.

Existem diversos tipos de Joins no SQL, os quais estão listados ao lado.

- LEFT OUTER JOIN
- RIGHT OUTER JOIN
- INNER JOIN
- FULL OUTER JOIN
- LEFT ANTI JOIN
- RIGHT ANTI JOIN
- FULL ANTI JOIN

Veremos mais a frente que os Joins mais utilizados serão o LEFT JOIN e o INNER JOIN. Porém, vamos aprender na prática como todos eles funcionam.

LEFT (OUTER) JOIN

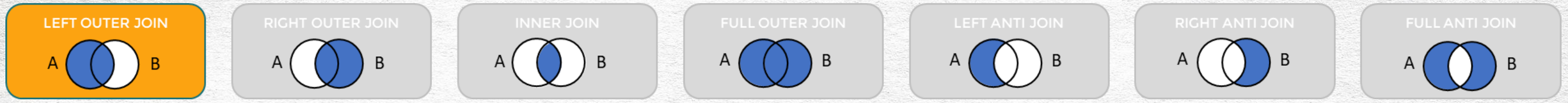


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Todos da Tabela A + Interseção com a Tabela B

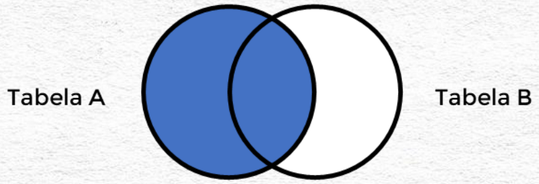


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	Fone Bluetooth JBL	5	Fone de Ouvido
2	PS4	6	NULL
3	Notebook Samsung i3	2	Notebook
4	iPhone 11	1	Celular
5	Moto G9	1	Celular

O LEFT JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá todas as linhas exclusivas apenas da tabela A, mais as linhas que são a interseção entre a Tabela A e a Tabela B.

Nenhuma linha presente apenas na Tabela B será incluída na tabela resultante desse JOIN.

RIGHT (OUTER) JOIN

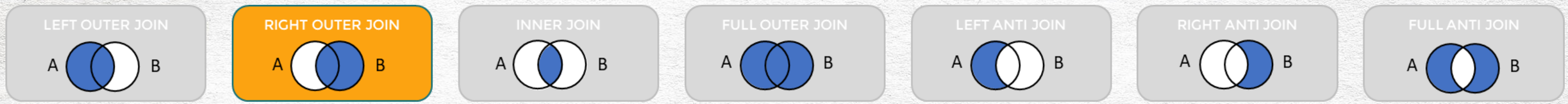


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Todos da Tabela B + Interseção com a Tabela A

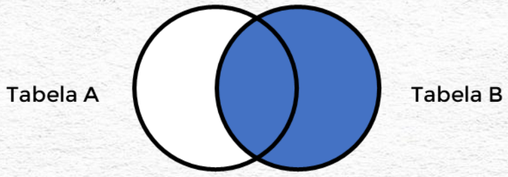


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
4	iPhone 11	1	Celular
5	Moto G9	1	Celular
3	Notebook Samsung i3	2	Notebook
NULL	NULL	NULL	Câmera Digital
NULL	NULL	NULL	Televisão
1	Fone Bluetooth JBL	5	Fone de Ouvido

O RIGHT JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá todas as linhas exclusivas apenas da tabela B, mais as linhas que são a interseção entre a Tabela A e a Tabela B.

Nenhuma linha presente apenas na Tabela A será incluída na tabela resultante desse JOIN.

INNER (OUTER) JOIN

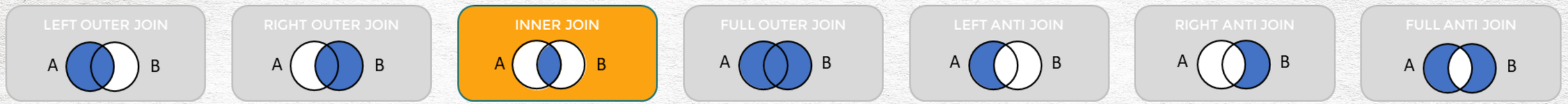


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Apenas a interseção entre Tabelas A e B

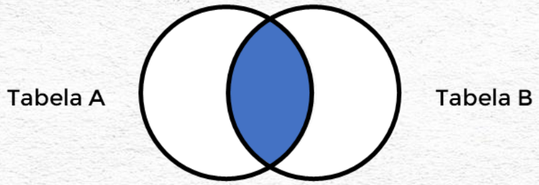


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	Fone Bluetooth JBL	5	Fone de Ouvido
3	Notebook Samsung i3	2	Notebook
4	iPhone 11	1	Celular
5	Moto G9	1	Celular

O INNER JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá **apenas** as linhas que são a interseção entre a Tabela A e a Tabela B.

FULL (OUTER) JOIN

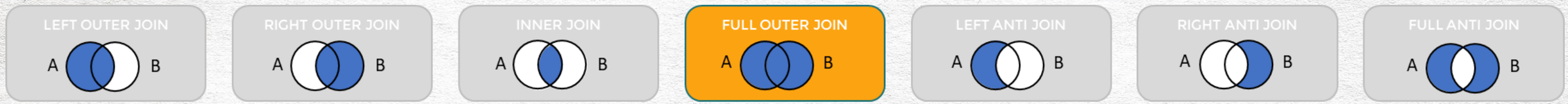


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Todas as linhas da Tabela A + Todas as linhas da Tabela B

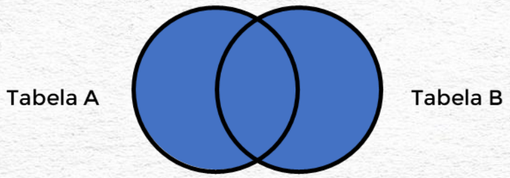


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	Fone Bluetooth JBL	5	Fone de Ouvido
2	PS4	6	NULL
3	Notebook Samsung i3	2	Notebook
4	iPhone 11	1	Celular
5	Moto G9	1	Celular
NULL	NULL	NULL	Câmera Digital
NULL	NULL	NULL	Televisão

O FULL JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá todas as linhas da Tabela A e todas as linhas da Tabela B.

LEFT (ANTI) JOIN

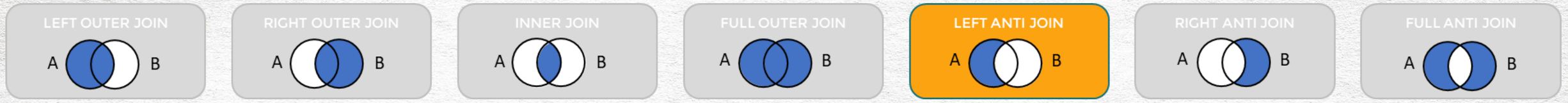


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Apenas as linhas que aparecem em A

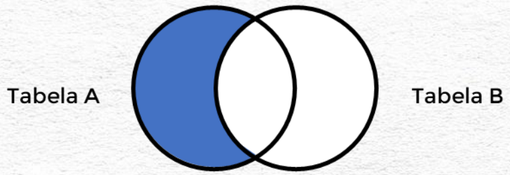


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
2	PS4	6	NULL

O LEFT (ANTI) JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá **apenas as linhas que só existem na Tabela A**.

RIGHT (ANTI) JOIN

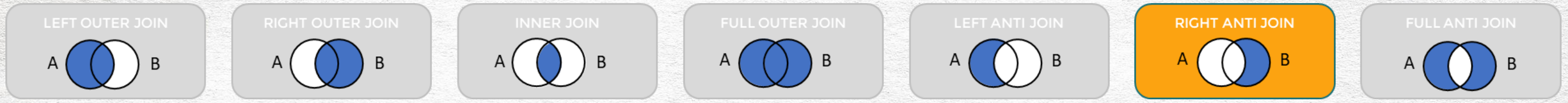


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Apenas as linhas que aparecem em B

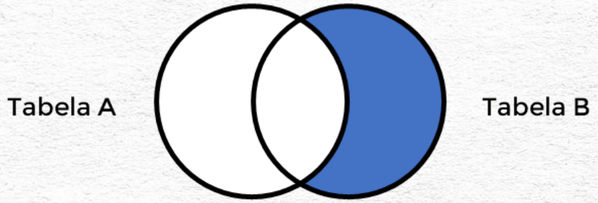


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
NULL	NULL	NULL	Câmera Digital
NULL	NULL	NULL	Televisão

O RIGHT (ANTI) JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá **apenas as linhas que só existem na Tabela B**.

FULL (ANTI) JOIN

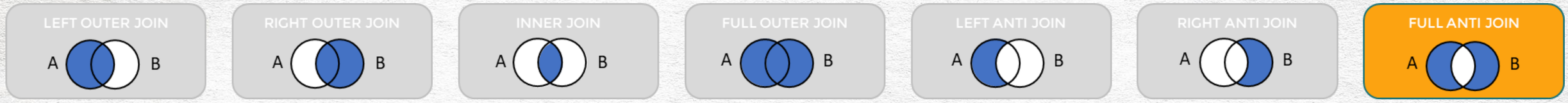


Tabela A

id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Apenas as linhas que aparecem em A + Apenas as linhas que aparecem em B

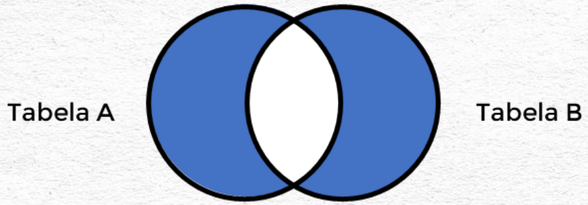


Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido

Tabela C

id_produto	nome_produto	id_subcategoria	nome_subcategoria
2	PS4	6	NULL
NULL	NULL	NULL	Câmera Digital
NULL	NULL	NULL	Televisão

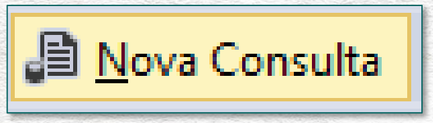
O FULL (ANTI) JOIN vai nos permitir relacionar duas tabelas (Tabela A e Tabela B) e criar uma nova tabela (Tabela C) que é a junção das duas.

A tabela resultante desse JOIN terá **apenas as linhas que só existem na Tabela A ou na Tabela B**.

LEFT, RIGHT e INNER JOIN: exemplos práticos

Para fazer os exemplos a seguir, utilize os códigos a seguir para criar as tabelas de exemplo. **Cada código deve ser executado em uma Consulta diferente**.

Crie uma Nova Consulta e utilize o primeiro código abaixo. Nele, vamos apenas criar um novo Banco de Dados para armazenar nossas Tabelas de exemplo (Obs: entraremos mais a fundo na parte de criação de tabelas e bancos de dados no módulo CRUD).



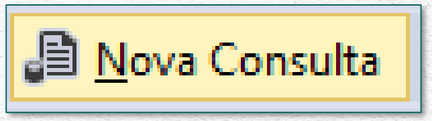
Consulta 1

SQLQuery1.sql - LA...5)) - não conectado* | SQLQuery2.sql - LAP...us Cavalcanti (54))* | SQLQuery3.sql - LAP...us Cavalcanti (68))*

```
CREATE DATABASE Joins  --Cria um banco de dados novo
GO
USE Joins  --Seleciona o banco de dados onde vamos adicionar as tabelas
```

LEFT, RIGHT e INNER JOIN: exemplos práticos

Na consulta 2, utilize o código abaixo para criar a tabela de **Produtos**, com 5 produtos de exemplo e as seguintes informações: **id_produto**, **nome_produto** e **id_subcategoria**. (Obs: entraremos mais a fundo na parte de criação de tabelas e bancos de dados no módulo CRUD).



Consulta 2

SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))* | SQLQuery3.sql - LAP...us Cavalcanti (68))*

```
USE Joins

CREATE TABLE produtos(
id_produto int,
nome_produto varchar(30),
id_subcategoria int)

INSERT INTO produtos(id_produto, nome_produto, id_subcategoria)
VALUES
(1, 'Fone Bluetooth JBL', 5),
(2, 'PS4', 6),
(3, 'Notebook Samsung i3', 2),
(4, 'iPhone 11', 1),
(5, 'Moto G9', 1)
```

LEFT, RIGHT e INNER JOIN: exemplos práticos

Na consulta 3, utilize o código abaixo para criar a tabela de **Subcategoria**, com 5 subcategorias de exemplo. Essa tabela terá as seguintes informações: **id_subcategoria** e **nome_subcategoria**. (Obs: entraremos mais a fundo na parte de criação de tabelas e bancos de dados no módulo CRUD).

Consulta 3

SQLQuery1.sql - LAP...us Cavalcanti (55))* SQLQuery2.sql - LAP...us Cavalcanti (54))* SQLQuery3.sql - LAP...us Cavalcanti (68))*

```
USE Joins   --Seleciona o banco de dados onde vamos adicionar as tabelas

CREATE TABLE subcategoria(
id_subcategoria int,
nome_subcategoria varchar(30))

INSERT INTO subcategoria(id_subcategoria, nome_subcategoria)
VALUES
(1, 'Celular'),
(2, 'Notebook'),
(3, 'Câmera Digital'),
(4, 'Televisão'),
(5, 'Fone de Ouvido')
```

LEFT, RIGHT e INNER JOIN: exemplos práticos

Na consulta ao lado, realizamos um **LEFT JOIN**.

Utilizamos um SELECT comum para selecionar as colunas desejadas. **Um detalhe importante é que quando selecionamos uma coluna que é comum às duas tabelas (no caso, a coluna id_subcategoria) precisamos especificar qual é a tabela que queremos considerar**.

Em relação à estrutura do JOIN, temos sempre o padrão abaixo: uma igualdade, onde do lado esquerdo precisamos informar a chave estrangeira da relação, e do lado direito precisamos informar a chave primária da relação entre as tabelas.

```
FROM
    produtos
LEFT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

chave estrangeira chave primária

Por fim, repare que a tabela resultante contém um produto que existe apenas na tabela **produto**, o PS4.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us C

```
-- Exemplo: LEFT JOIN
SELECT
    id_produto,
    nome_produto,
    produtos.id_subcategoria,
    nome_subcategoria
FROM
    produtos
LEFT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

133 %

Resultados | Mensagens

	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	1	Fone Bluetooth JBL	5	Fone de Ouvido
2	2	PS4	6	NULL
3	3	Notebook Samsung i3	2	Notebook
4	4	iPhone 11	1	Celular
5	5	Moto G9	1	Celular

LEFT, RIGHT e INNER JOIN: exemplos práticos

Na consulta ao lado, realizamos um **RIGHT JOIN**.

A estrutura é muito semelhante ao LEFT JOIN (apenas trocamos o LEFT por RIGHT).

O resultado final é uma tabela contendo também as subcategorias que existem apenas na tabela da direita (Câmera Digital e Televisão).

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- Exemplo: LEFT JOIN
SELECT
    id_produto,
    nome_produto,
    produtos.id_subcategoria,
    nome_subcategoria
FROM
    produtos
RIGHT JOIN subcategoria
    ON produtos.id_subcategoria = subcategoria.id_subcategoria
```

133 %

Resultados | Mensagens

	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	4	iPhone 11	1	Celular
2	5	Moto G9	1	Celular
3	3	Notebook Samsung i3	2	Notebook
4	NULL	NULL	NULL	Câmera Digital
5	NULL	NULL	NULL	Televisão
6	1	Fone Bluetooth JBL	5	Fone de Ouvido

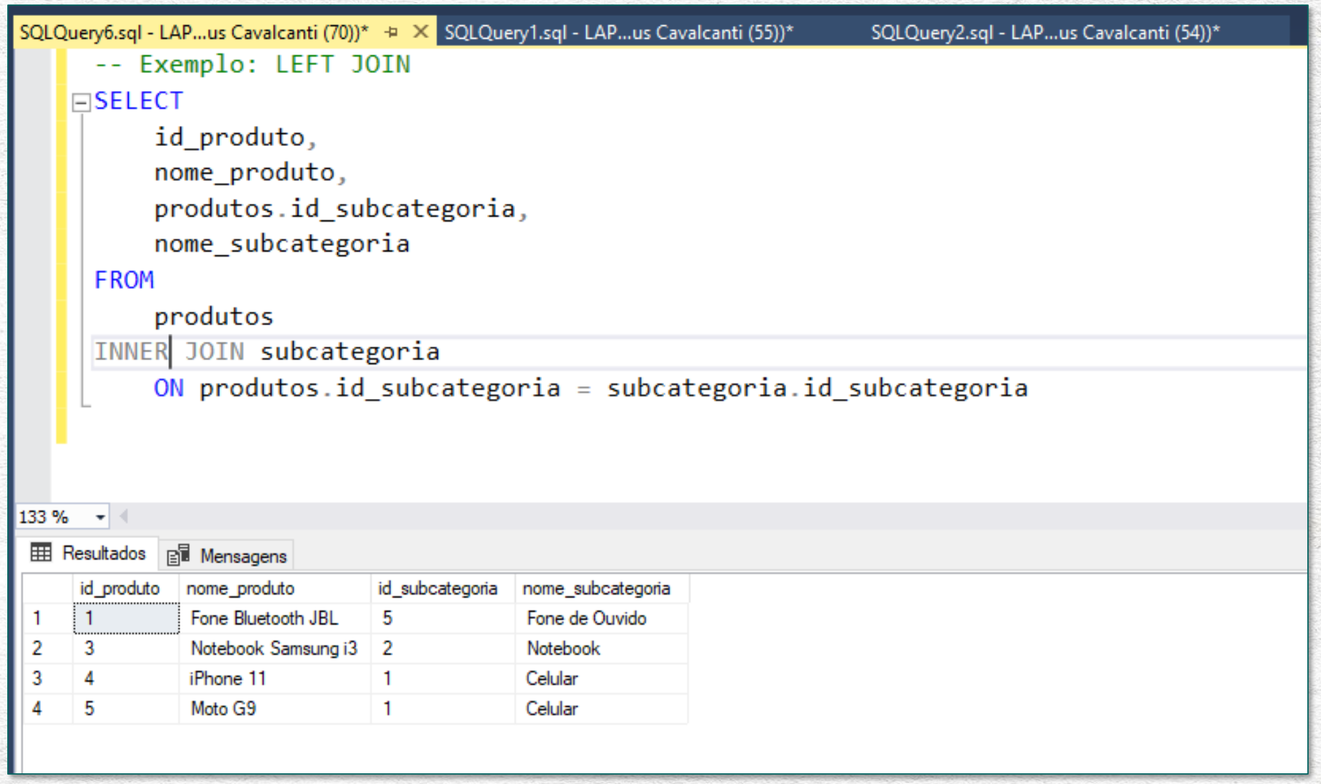
LEFT, RIGHT e INNER JOIN: exemplos práticos

Por fim, realizamos um **INNER JOIN**.

A estrutura é muito semelhante ao LEFT/RIGHT JOIN (apenas trocamos o LEFT/RIGHT por INNER).

O resultado final é uma tabela contendo apenas as informações presentes nas duas tabelas, ou seja, a interseção.

Nessa tabela resultante não teremos nem o produto PS4 (que existe apenas na tabela **produtos**, nem as subcategorias Câmera Digital e Televisão da tabela **subcategoria**).



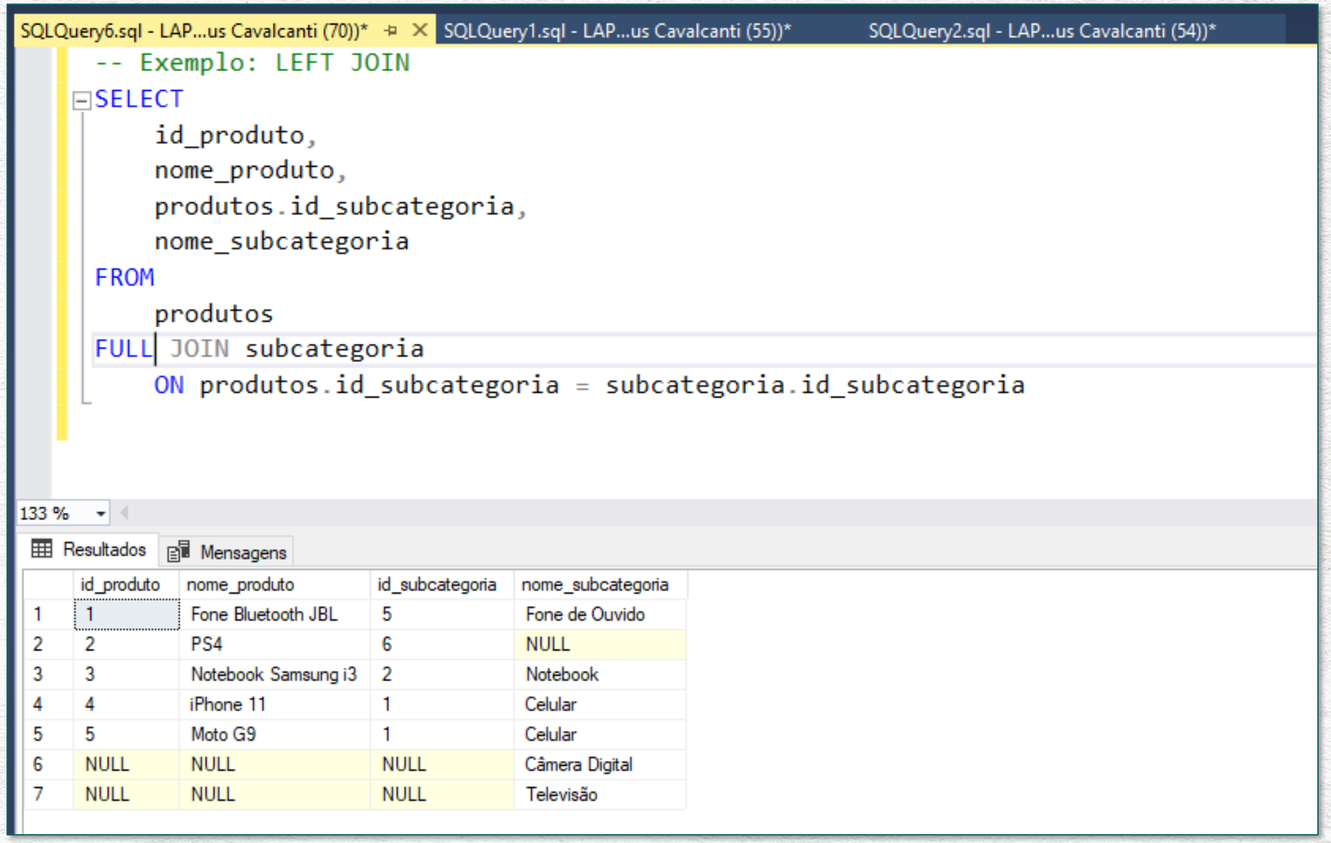
	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	1	Fone Bluetooth JBL	5	Fone de Ouvido
2	3	Notebook Samsung i3	2	Notebook
3	4	iPhone 11	1	Celular
4	5	Moto G9	1	Celular

FULL JOIN: exemplos práticos

Ao lado, realizamos um **FULL JOIN**.

Esse JOIN retornará **todas as linhas das duas tabelas**: tanto a linha do produto PS4 (que existe apenas na tabela produto) quanto as subcategorias Câmera Digital e Televisão (que existem apenas na tabela subcategoria).

Essa seria uma junção completa das duas tabelas.



	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	1	Fone Bluetooth JBL	5	Fone de Ouvido
2	2	PS4	6	NULL
3	3	Notebook Samsung i3	2	Notebook
4	4	iPhone 11	1	Celular
5	5	Moto G9	1	Celular
6	NULL	NULL	NULL	Câmera Digital
7	NULL	NULL	NULL	Televisão

LEFT, RIGHT e FULL ANTI JOIN: exemplos práticos

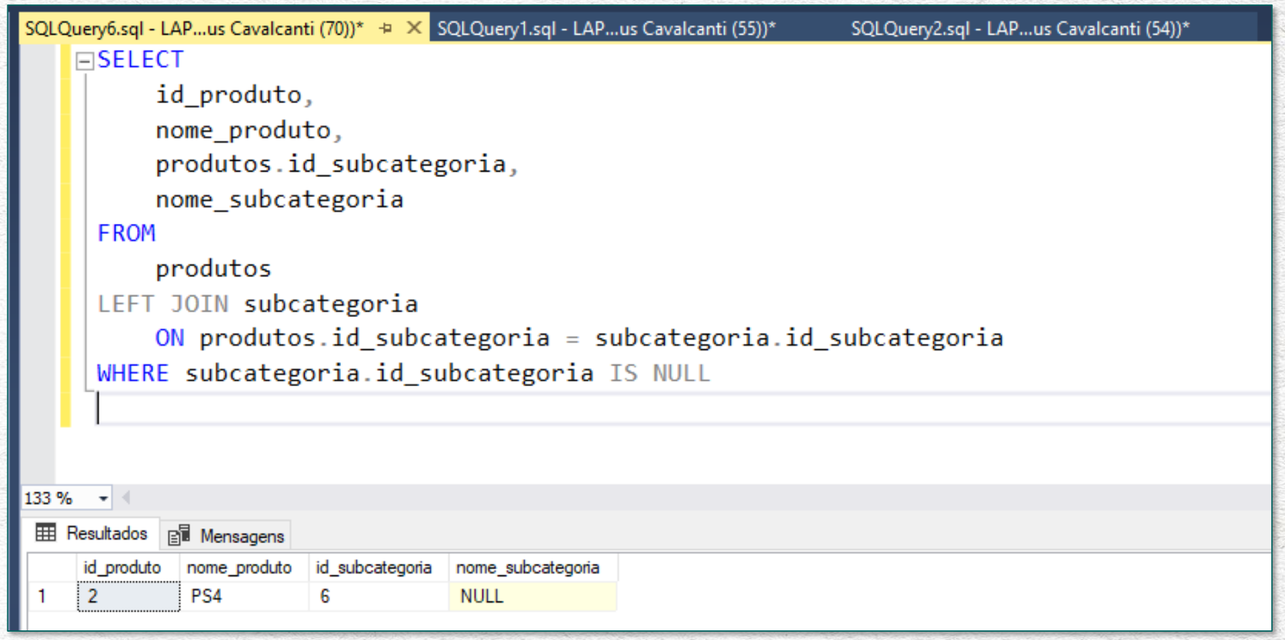
Outro conjunto de JOINs são os chamados ANTI JOIN: LEFT ANTI JOIN, RIGTH ANTI JOIN e FULL ANTI JOIN.

Eles não são tão utilizados na prática, mas é importante que você saiba o que cada um faz.

Ao lado, realizamos um **LEFT ANTI JOIN**.

Este JOIN vai retornar os produtos que existem apenas na tabela da esquerda (**produtos**): no caso, o produto PS4.

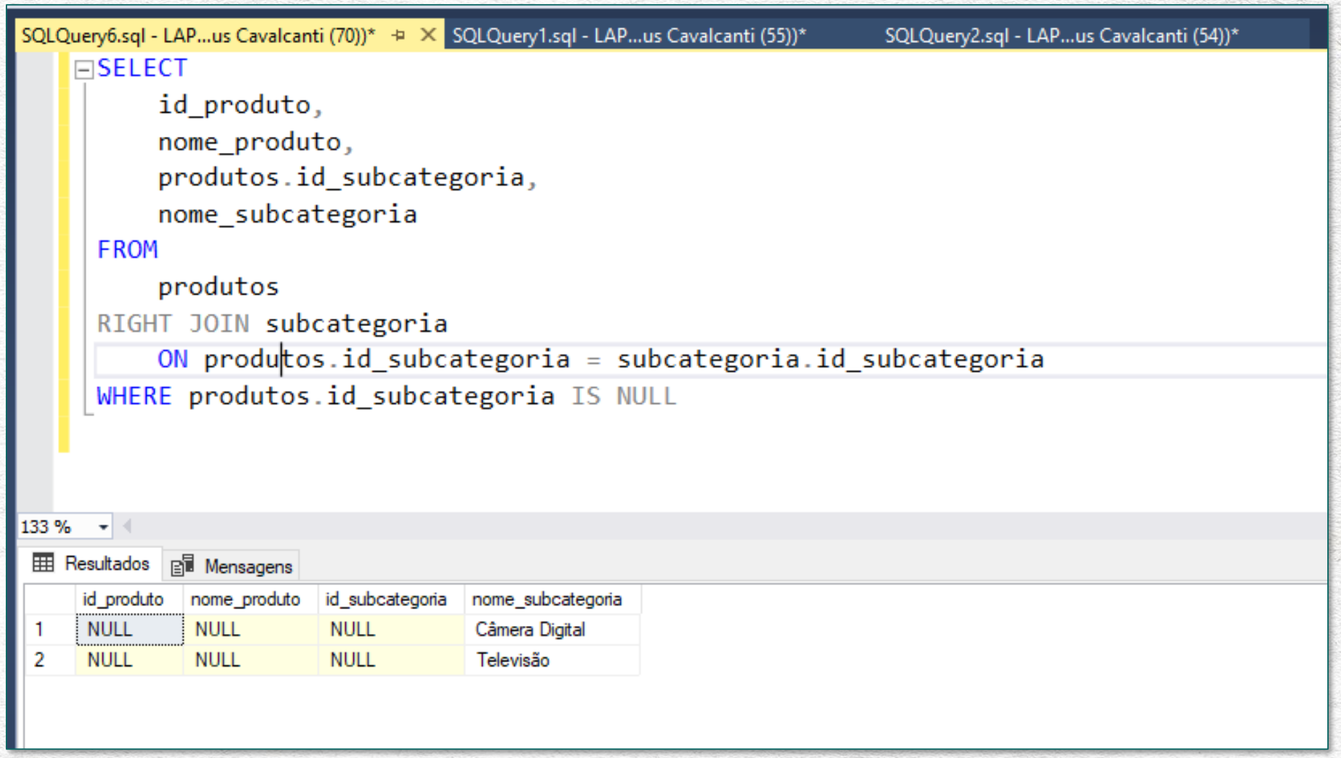
Observe que não existe um comando chamado LEFT ANTI JOIN. Para criá-lo, precisamos combinar o LEFT JOIN com um filtro WHERE, onde consideramos todas as subcategorias (tabela da direita) que são nulos.



LEFT, RIGHT e FULL ANTI JOIN: exemplos práticos

Ao lado, realizamos um **RIGHT ANTI JOIN**.

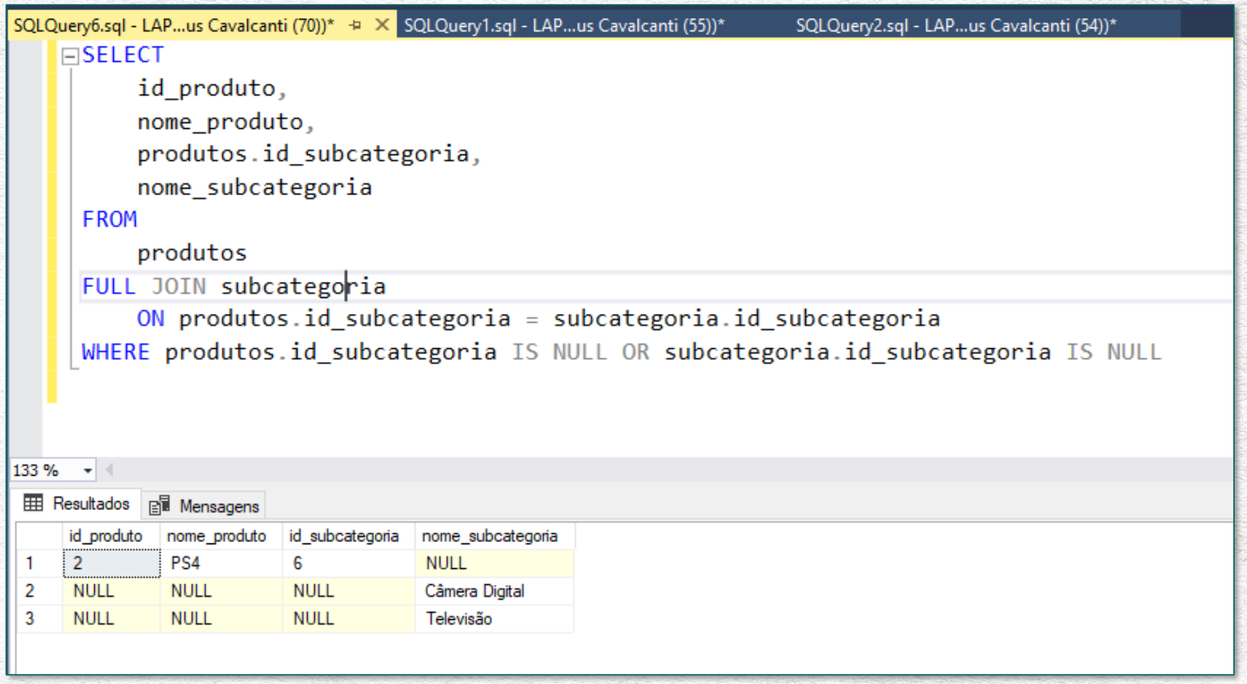
Ele terá um comportamento semelhante ao LEFT ANTI JOIN, com a diferença que a tabela resultante terá as linhas que estão presentes apenas na tabela da direita (tabela subcategoria).



LEFT, RIGHT e FULL ANTI JOIN: exemplos práticos

Por fim, realizamos um **FULL ANTI JOIN**.

Este JOIN retornará todas as linhas que estão presentes em apenas uma das tabelas, como é mostrado na imagem ao lado.

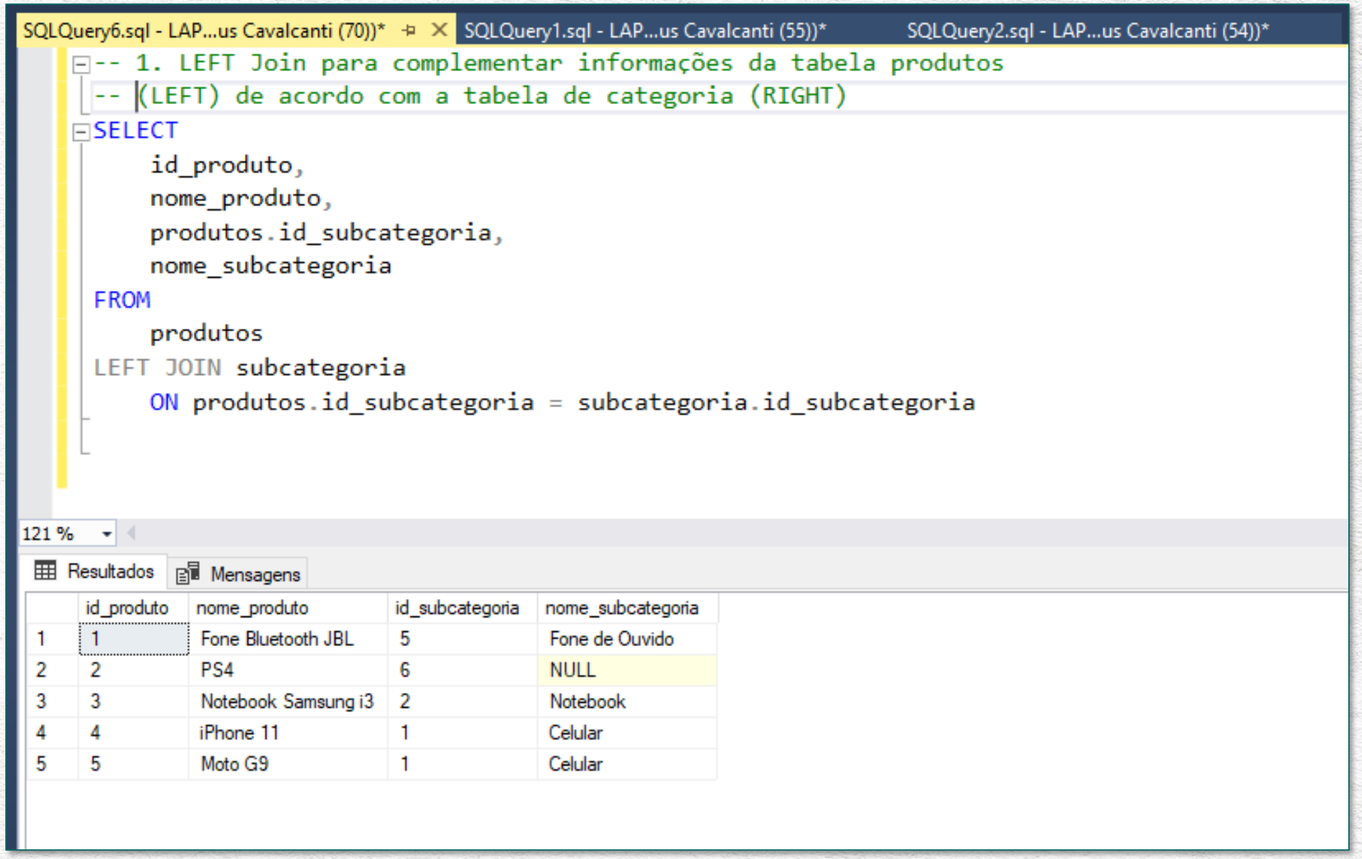


Como definir quem é a LEFT TABLE e a RIGHT TABLE

Uma dúvida comum é se existe alguma diferença em escolher quem será a LEFT TABLE ou a RIGHT TABLE.

Observe o exercício ao lado. Utilizamos o LEFT JOIN para retornar a junção entre as tabelas **produtos** e **subcategoria**.

Seria possível chegar no mesmo resultado utilizando o RIGHT JOIN, como veremos na página a seguir.



	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	1	Fone Bluetooth JBL	5	Fone de Ouvido
2	2	PS4	6	NULL
3	3	Notebook Samsung i3	2	Notebook
4	4	iPhone 11	1	Celular
5	5	Moto G9	1	Celular

Como definir quem é a LEFT TABLE e a RIGHT TABLE

Observe que conseguimos obter a mesma tabela utilizando também o RIGHT JOIN.

Para isso, precisamos fazer algumas alterações: a tabela de produtos, que antes era a tabela da esquerda, virou a tabela da direita.

Já a tabela de subcategoria, que antes era a tabela da direita, virou a tabela da esquerda.

Com essas alterações, conseguimos chegar no mesmo resultado, utilizando o RIGHT JOIN.

O que vale a gente pensar no momento é que esse join tinha como finalidade complementar as informações de subcategoria em nossa tabela de produtos. É mais intuitivo pensar que queremos complementar a tabela da esquerda com as informações da tabela da direita. E para chegar nesse resultado, usamos o LEFT JOIN.

O RIGHT JOIN nos leva a uma lógica um pouco mais confusa, por isso preferimos utilizar o LEFT JOIN, por ser mais intuitivo.

A lição que fica é: podemos chegar no mesmo resultado usando tanto o LEFT JOIN quanto o RIGHT JOIN, mas o mais comum de ser utilizado é o LEFT JOIN.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- 2. Obtendo o LEFT Join do caso 1 usando um RIGHT Join e invertendo as tabelas de lado

SELECT
    id_produto,
    nome_produto,
    produtos.id_subcategoria,
    nome_subcategoria
FROM
    subcategoria
RIGHT JOIN produtos
    ON subcategoria.id_subcategoria = produtos.id_subcategoria
```

121 %

Resultados | Mensagens

	id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	1	Fone Bluetooth JBL	5	Fone de Ouvido
2	2	PS4	6	NULL
3	3	Notebook Samsung i3	2	Notebook
4	4	iPhone 11	1	Celular
5	5	Moto G9	1	Celular

LEFT JOIN ou INNER JOIN: qual usar?

Em alguns casos, o LEFT JOIN e o INNER JOIN podem trazer o mesmo resultado.

Caso não haja nenhuma informação na tabela A que não esteja presente na tabela B, os resultados serão iguais. Para ficar claro, pense nas seguintes situações:

Situação 1

Se a Tabela A (esquerda) possui os produtos 1 a 10, e na Tabela B (direita) também temos os produtos de 1 a 10, se fizermos um INNER JOIN ou um LEFT JOIN, teremos o mesmo resultado, pois não há em nenhuma das tabelas um produto que exista apenas em uma dessas tabelas.

Situação 2

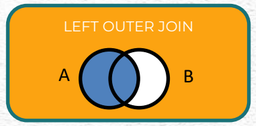
Se a Tabela A (esquerda) possui os produtos de 1 a 11, e na Tabela B (direita) temos apenas os produtos de 1 a 10, repare que há 1 produto na Tabela A que só existe na tabela A. Portanto, se usamos o LEFT JOIN, a tabela resultante terá 1 produto a mais (o que existe apenas na tabela da esquerda) enquanto o INNER JOIN vai retornar apenas os produtos que estão presentes nas duas tabelas (e deixará 1 produto da Tabela A de fora).

Tabela A

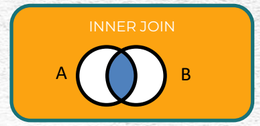
id_produto	nome_produto	id_subcategoria
1	Fone Bluetooth JBL	5
2	PS4	6
3	Notebook Samsung i3	2
4	iPhone 11	1
5	Moto G9	1

Tabela B

id_subcategoria	nome_subcategoria
1	Celular
2	Notebook
3	Câmera Digital
4	Televisão
5	Fone de Ouvido



id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	Fone Bluetooth JBL	5	Fone de Ouvido
2	PS4	6	NULL
3	Notebook Samsung i3	2	Notebook
4	iPhone 11	1	Celular
5	Moto G9	1	Celular



id_produto	nome_produto	id_subcategoria	nome_subcategoria
1	Fone Bluetooth JBL	5	Fone de Ouvido
3	Notebook Samsung i3	2	Notebook
4	iPhone 11	1	Celular
5	Moto G9	1	Celular

LEFT JOIN

No exemplo acima, a tabela resultante terá também o produto PS4, que está presente apenas na tabela A (esquerda).

INNER JOIN

No exemplo acima, a tabela resultante terá apenas os produtos que estão presentes nas duas tabelas, excluindo o PS4, que está presente apenas na Tabela A.

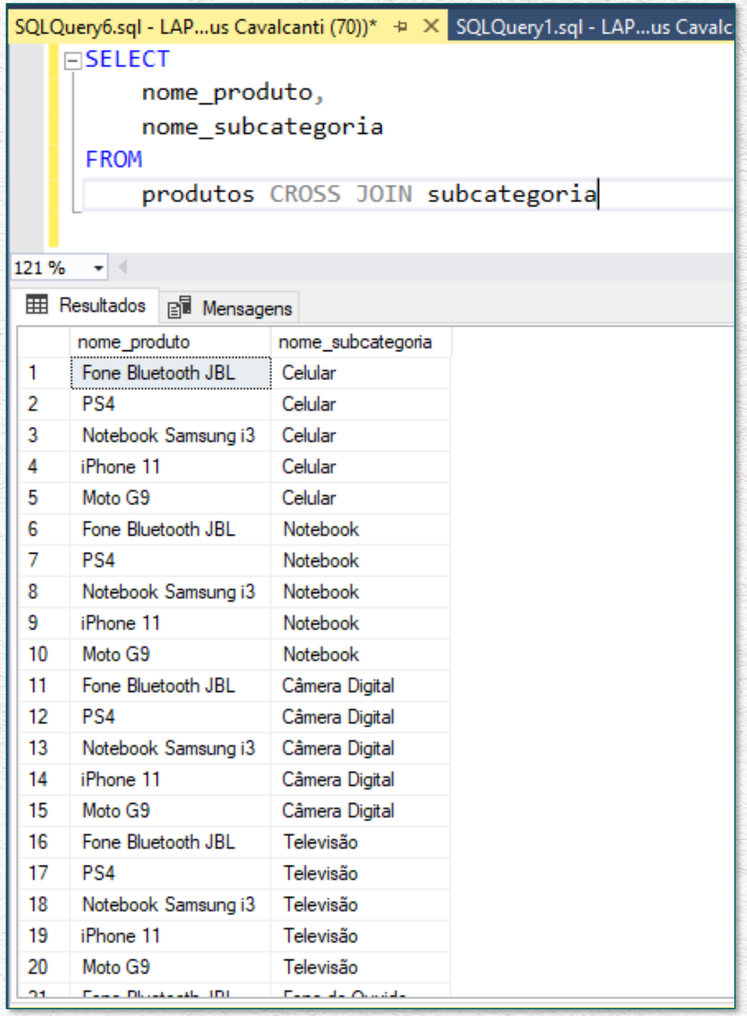
CROSS JOIN

Um outro JOIN que pode ser útil é o **CROSS JOIN**.

Com este JOIN, conseguimos criar uma tabela que é a combinação de todos os valores entre duas colunas.

Observe a imagem ao lado. Utilizando o CROSS JOIN com as colunas nome_produto e nome_subcategoria, é feita uma combinação de cada produto e cada subcategoria.

Essa combinação é conhecida como produto cartesiano: uma combinação um a um de todos os valores de duas ou mais colunas.



	nome_produto	nome_subcategoria
1	Fone Bluetooth JBL	Celular
2	PS4	Celular
3	Notebook Samsung i3	Celular
4	iPhone 11	Celular
5	Moto G9	Celular
6	Fone Bluetooth JBL	Notebook
7	PS4	Notebook
8	Notebook Samsung i3	Notebook
9	iPhone 11	Notebook
10	Moto G9	Notebook
11	Fone Bluetooth JBL	Câmera Digital
12	PS4	Câmera Digital
13	Notebook Samsung i3	Câmera Digital
14	iPhone 11	Câmera Digital
15	Moto G9	Câmera Digital
16	Fone Bluetooth JBL	Televisão
17	PS4	Televisão
18	Notebook Samsung i3	Televisão
19	iPhone 11	Televisão
20	Moto G9	Televisão

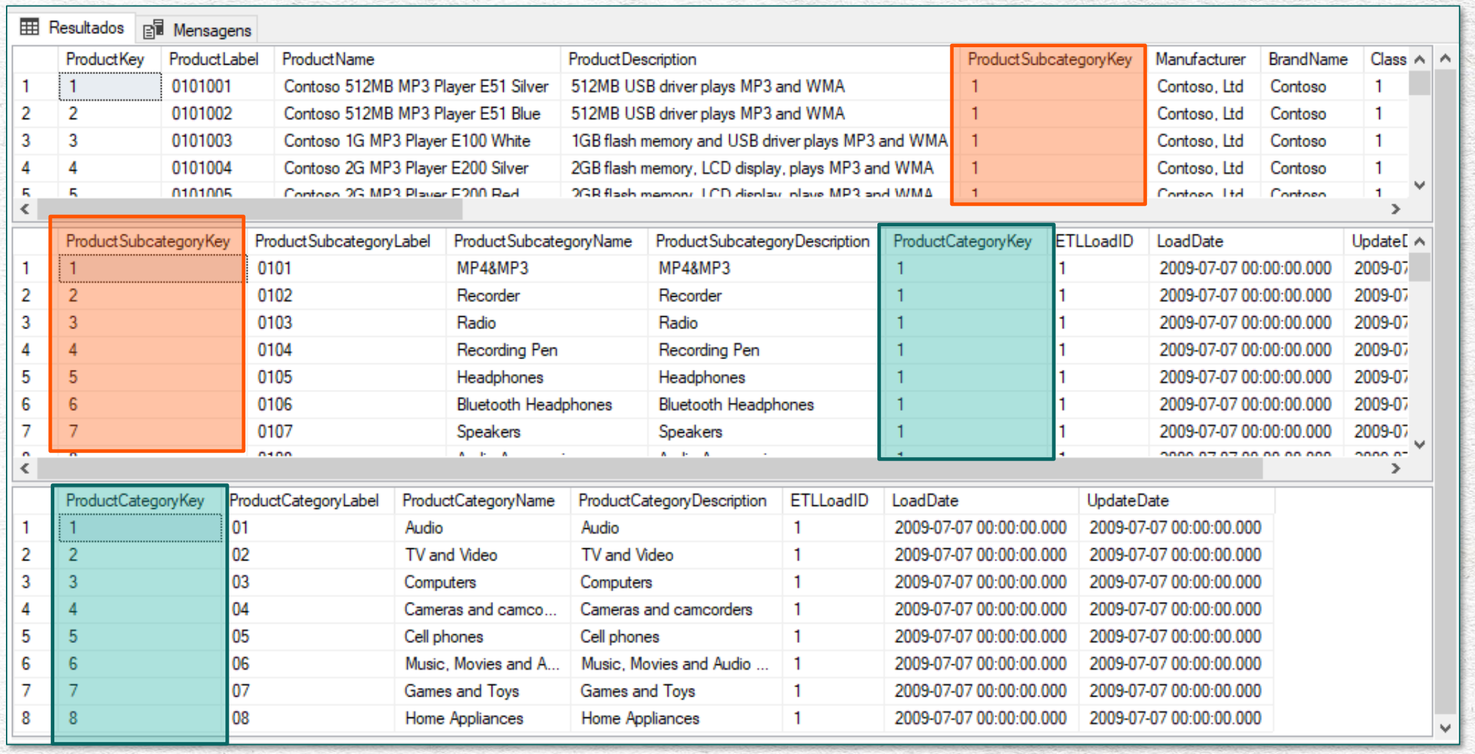
Múltiplos Joins

Em algumas situações, precisamos utilizar múltiplos Joins.

Em nosso banco de dados Contoso, temos uma tabela chamada DimProduct, que contém as informações dos produtos.

Porém, essa tabela não possui informações das categorias desses produtos, apenas as informações de ids das subcategorias.

Portanto, sabemos apenas de forma indireta qual é a categoria do produto: de acordo com o id da subcategoria, conseguimos descobrir qual é a categoria.



Resultados | Mensagens

	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName	Class
1	1	0101001	Contoso 512MB MP3 Player E51 Silver	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
2	2	0101002	Contoso 512MB MP3 Player E51 Blue	512MB USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
3	3	0101003	Contoso 1G MP3 Player E100 White	1GB flash memory and USB driver plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
4	4	0101004	Contoso 2G MP3 Player E200 Silver	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1
5	5	0101005	Contoso 2G MP3 Player E200 Red	2GB flash memory, LCD display, plays MP3 and WMA	1	Contoso, Ltd	Contoso	1

	ProductSubcategoryKey	ProductSubcategoryLabel	ProductSubcategoryName	ProductSubcategoryDescription	ProductCategoryKey	ETLLoadID	LoadDate	UpdateD
1	1	0101	MP4&MP3	MP4&MP3	1	1	2009-07-07 00:00:00.000	2009-07
2	2	0102	Recorder	Recorder	1	1	2009-07-07 00:00:00.000	2009-07
3	3	0103	Radio	Radio	1	1	2009-07-07 00:00:00.000	2009-07
4	4	0104	Recording Pen	Recording Pen	1	1	2009-07-07 00:00:00.000	2009-07
5	5	0105	Headphones	Headphones	1	1	2009-07-07 00:00:00.000	2009-07
6	6	0106	Bluetooth Headphones	Bluetooth Headphones	1	1	2009-07-07 00:00:00.000	2009-07
7	7	0107	Speakers	Speakers	1	1	2009-07-07 00:00:00.000	2009-07

	ProductCategoryKey	ProductCategoryLabel	ProductCategoryName	ProductCategoryDescription	ETLLoadID	LoadDate	UpdateDate
1	1	01	Audio	Audio	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
2	2	02	TV and Video	TV and Video	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
3	3	03	Computers	Computers	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
4	4	04	Cameras and camco...	Cameras and camcorders	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
5	5	05	Cell phones	Cell phones	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
6	6	06	Music, Movies and A...	Music, Movies and Audio ...	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
7	7	07	Games and Toys	Games and Toys	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000
8	8	08	Home Appliances	Home Appliances	1	2009-07-07 00:00:00.000	2009-07-07 00:00:00.000

Múltiplos Joins

Para levar uma informação da tabela de categoria até a tabela de Produtos, precisaremos fazer dois joins: um para partir da tabela de produtos e chegar na tabela de subcategoria, e outro para sair da tabela de subcategoria e chegar na de categoria.

O resultado de uma consulta que reúne as informações de nome do produto (DimProduct), nome da subcategoria (DimProductSubcategory) e nome da categoria (DimProductCategory) é mostrado na imagem ao lado.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
SELECT
    ProductKey,
    ProductName,
    DimProductSubcategory.ProductSubcategoryKey,
    ProductSubcategoryName,
    DimProductSubcategory.ProductCategoryKey,
    DimProductCategory.ProductCategoryName
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
INNER JOIN DimProductCategory
    ON DimProductSubcategory.ProductCategoryKey = DimProductCategory.ProductCategoryKey
```

121 %

Resultados | Mensagens

	ProductKey	ProductName	ProductSubcategoryKey	ProductSubcategoryName	ProductCategoryKey	ProductCategoryName
1	1	Contoso 512MB MP3 Player E51 Silver	1	MP4&MP3	1	Audio
2	2	Contoso 512MB MP3 Player E51 Blue	1	MP4&MP3	1	Audio
3	3	Contoso 1G MP3 Player E100 White	1	MP4&MP3	1	Audio
4	4	Contoso 2G MP3 Player E200 Silver	1	MP4&MP3	1	Audio
5	5	Contoso 2G MP3 Player E200 Red	1	MP4&MP3	1	Audio
6	6	Contoso 2G MP3 Player E200 Black	1	MP4&MP3	1	Audio
7	7	Contoso 2G MP3 Player E200 Blue	1	MP4&MP3	1	Audio
8	8	Contoso 4G MP3 Player E400 Silver	1	MP4&MP3	1	Audio
9	9	Contoso 4G MP3 Player E400 Black	1	MP4&MP3	1	Audio
10	10	Contoso 4G MP3 Player E400 Green	1	MP4&MP3	1	Audio
11	11	Contoso 4G MP3 Player E400 Orange	1	MP4&MP3	1	Audio

UNION e UNION ALL

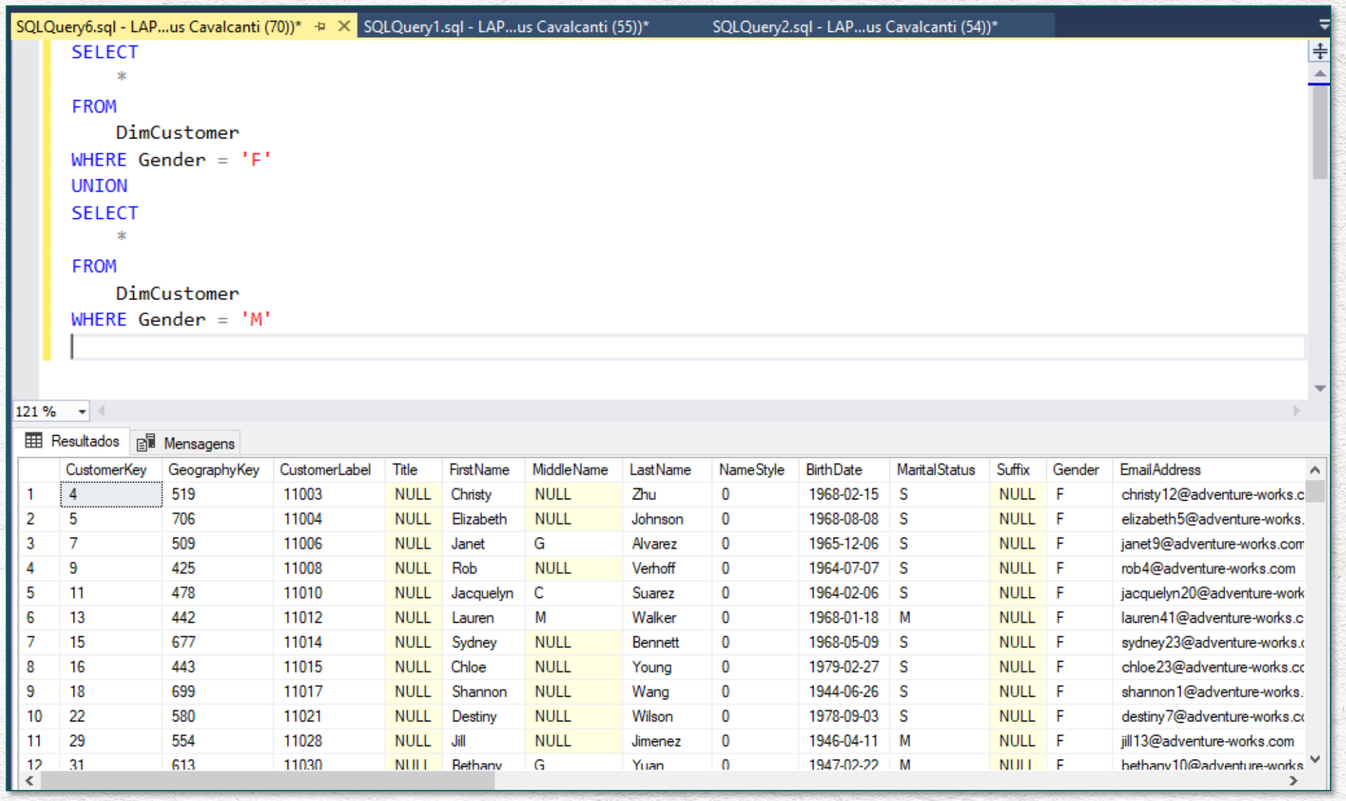
Existem dois outros comandos usamos para junção de tabelas: UNION e UNION ALL.

Estes dois permitem que a gente junte duas ou mais tabelas, como se estivéssemos empilhando as tabelas, ou seja, colocando uma tabela em cima da outra.

Ao lado, temos um exemplo do UNION. Imagine que tivéssemos duas tabelas separadas por gênero, com informações de clientes.

Poderíamos juntar as duas tabelas usando o UNION, chegando no resultado de uma tabela única, contendo informações tanto dos clientes do sexo masculino quanto do sexo feminino.

Um detalhe importante sobre o uso do UNION é que as tabelas a serem juntadas precisam ter as mesmas colunas, na mesma ordem.



	CustomerKey	GeographyKey	CustomerLabel	Title	FirstName	MiddleName	LastName	NameStyle	BirthDate	MaritalStatus	Suffix	Gender	EmailAddress
1	4	519	11003	NULL	Christy	NULL	Zhu	0	1968-02-15	S	NULL	F	christy12@adventure-works.c
2	5	706	11004	NULL	Elizabeth	NULL	Johnson	0	1968-08-08	S	NULL	F	elizabeth5@adventure-works.
3	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F	janet9@adventure-works.com
4	9	425	11008	NULL	Rob	NULL	Verhoff	0	1964-07-07	S	NULL	F	rob4@adventure-works.com
5	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F	jacquelyn20@adventure-work
6	13	442	11012	NULL	Lauren	M	Walker	0	1968-01-18	M	NULL	F	lauren41@adventure-works.c
7	15	677	11014	NULL	Sydney	NULL	Bennett	0	1968-05-09	S	NULL	F	sydney23@adventure-works.c
8	16	443	11015	NULL	Chloe	NULL	Young	0	1979-02-27	S	NULL	F	chloe23@adventure-works.co
9	18	699	11017	NULL	Shannon	NULL	Wang	0	1944-06-26	S	NULL	F	shannon1@adventure-works.
10	22	580	11021	NULL	Destiny	NULL	Wilson	0	1978-09-03	S	NULL	F	destiny7@adventure-works.co
11	29	554	11028	NULL	Jill	NULL	Jimenez	0	1946-04-11	M	NULL	F	jill13@adventure-works.com
12	31	613	11030	NULL	Bethany	G	Yuan	0	1947-02-22	M	NULL	F	bethany10@adventure-works

UNION e UNION ALL

Além do UNION, temos o **UNION ALL**.

Ele terá um resultado muito semelhante ao UNION, com apenas uma diferença: **todas as linhas duplicadas serão incluídas**.

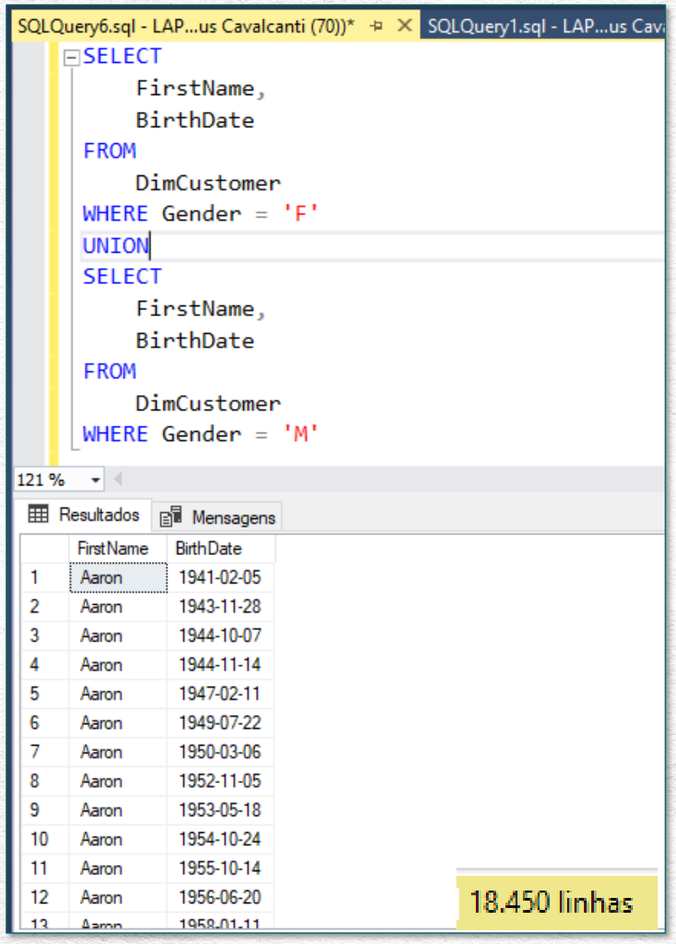
Observe o exemplo ao lado. Temos uma união de duas tabelas, também contendo informações separadas de clientes por gênero, só que dessa vez apenas as colunas de FirstName e BirthDate.

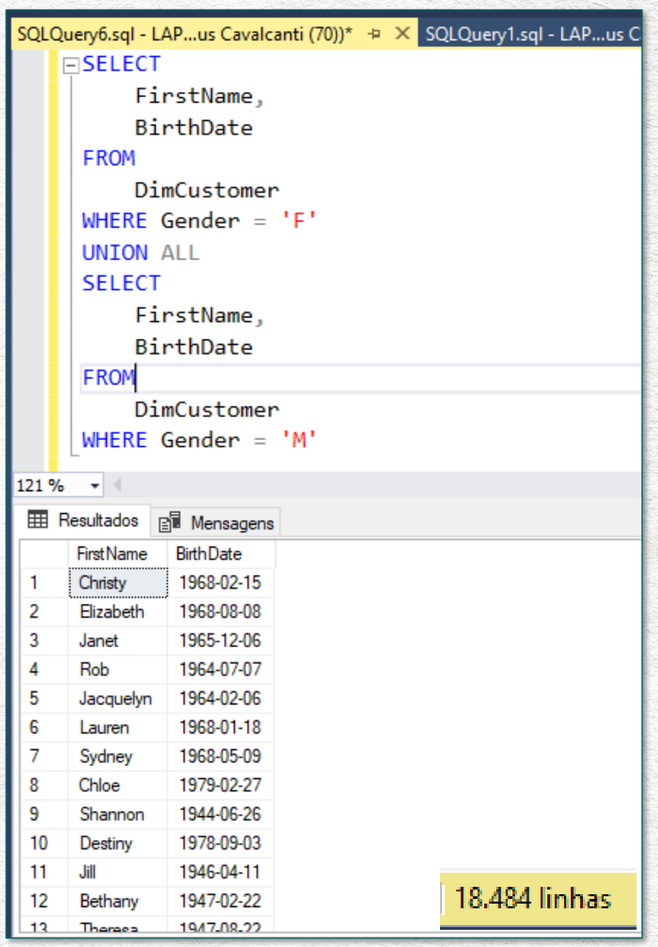
No UNION (imagem à esquerda), o resultado mostra 18.450 linhas. Já com o UNION ALL, temos um resultado de 18.484.

Qual é a diferença?

O UNION retorna uma tabela removendo todos os duplicados, enquanto o UNION ALL mantém os duplicados.

Isso significa que essa união possui pessoas que nasceram no mesmo dia, e possuem o mesmo primeiro nome. Por isso, no caso do UNION, a quantidade de linhas resultado foi menor.





EXERCÍCIOS

1

Questão 1

Utilize o INNER JOIN para trazer os nomes das subcategorias dos produtos, da tabela DimProductSubcategory para a tabela DimProduct.

2

Questão 2

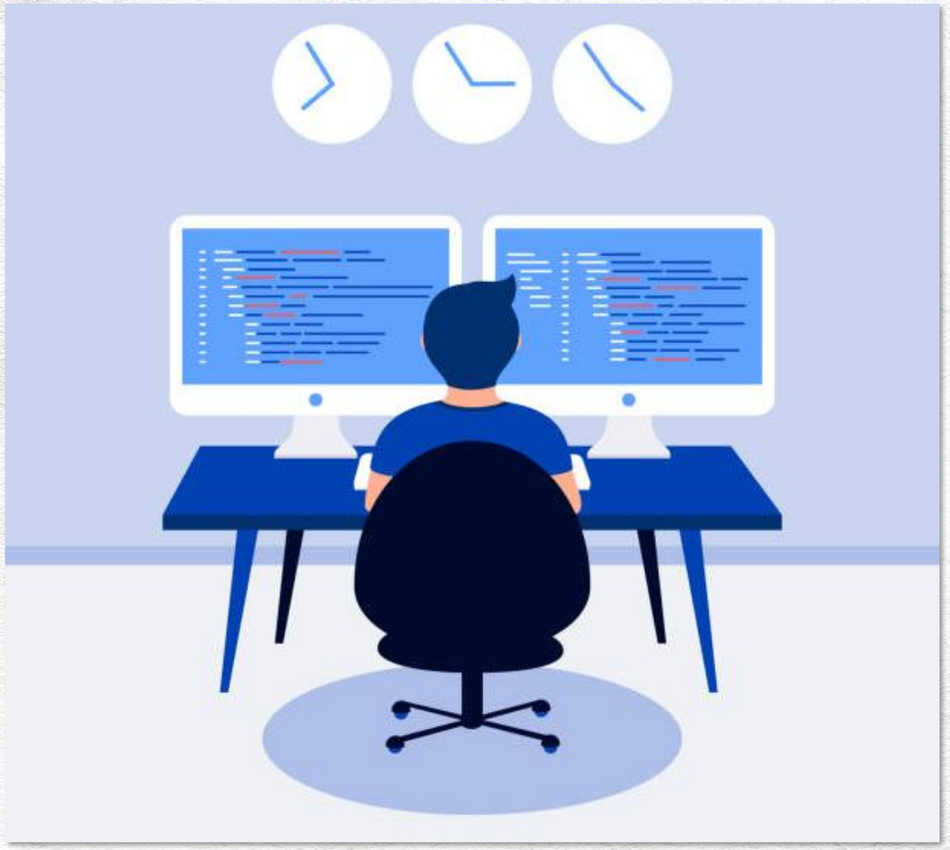
Identifique uma coluna em comum entre as tabelas DimProductSubcategory e DimProductCategory. Utilize essa coluna para complementar informações na tabela DimProductSubcategory a partir da DimProductCategory. Utilize o LEFT JOIN.

3

Questão 3

Para cada loja da tabela DimStore, descubra qual o Continente e o Nome do País associados (de acordo com DimGeography). Seu SELECT final deve conter apenas as seguintes colunas: StoreKey, StoreName, EmployeeCount, ContinentName e RegionCountryName. Utilize o LEFT JOIN neste exercício.

EXERCÍCIOS



4 Questão 4

Complementa a tabela DimProduct com a informação de ProductCategoryDescription. Utilize o LEFT JOIN e retorne em seu SELECT apenas as 5 colunas que considerar mais relevantes.

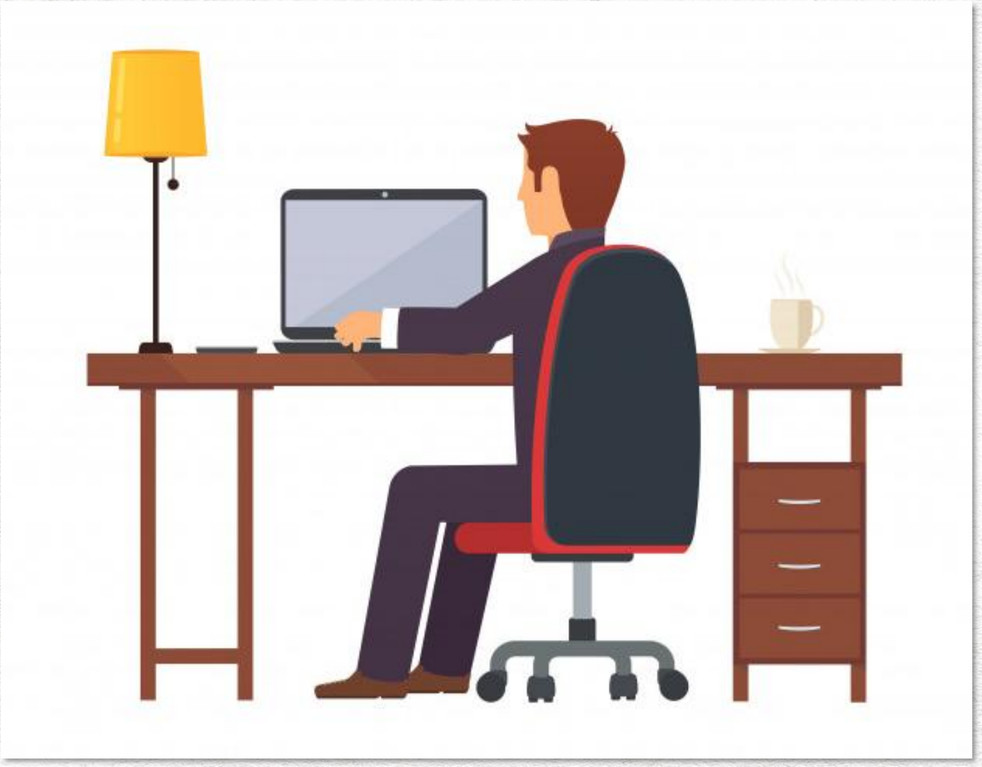
5 Questão 5

A tabela FactStrategyPlan resume o planejamento estratégico da empresa. Cada linha representa um montante destinado a uma determinada AccountKey.

a. Faça um SELECT das 100 primeiras linhas de FactStrategyPlan para reconhecer a tabela.

b. Faça um INNER JOIN para criar uma tabela contendo o AccountName para cada AccountKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas:
   - StrategyPlanKey
   - DateKey
   - AccountName
   - Amount

EXERCÍCIOS



6

Questão 6

Vamos continuar analisando a tabela FactStrategyPlan. Além da coluna AccountKey que identifica o tipo de conta, há também uma outra coluna chamada ScenarioKey. Essa coluna possui a numeração que identifica o tipo de cenário: Real, Orçado e Previsão.

Faça um INNER JOIN para criar uma tabela contendo o ScenarioName para cada ScenarioKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas:

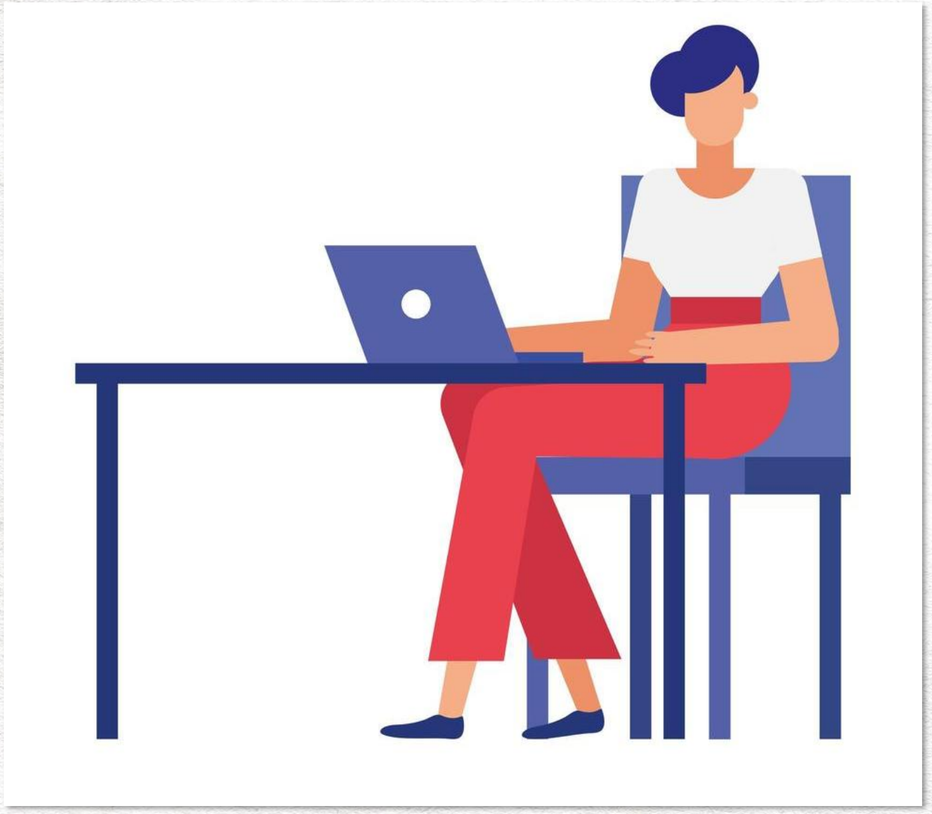
- StrategyPlanKey
- DateKey
- ScenarioName
- Amount

7

Questão 7

Algumas subcategorias não possuem nenhum exemplar de produto. Identifique que subcategorias são essas.

EXERCÍCIOS



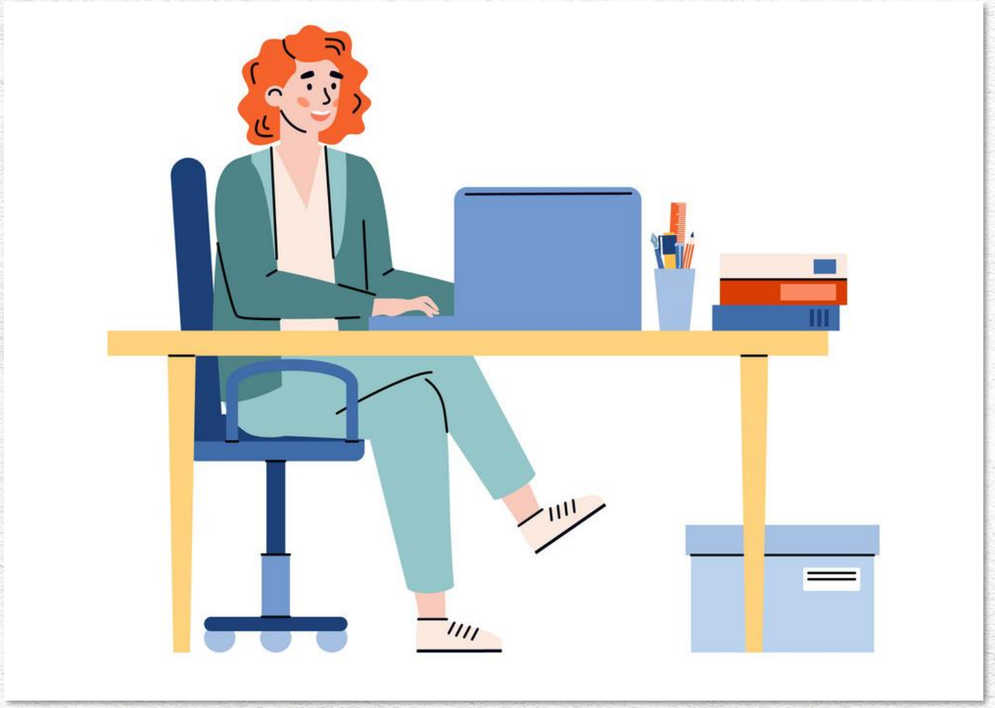
8

Questão 8

A tabela abaixo mostra a combinação entre Marca e Canal de Venda, para as marcas Contoso, Fabrikam e Litware. Crie um código SQL para chegar no mesmo resultado.

BrandName	ChannelName
Contoso	Catalog
Contoso	Online
Contoso	Reseller
Contoso	Store
Fabrikam	Catalog
Fabrikam	Online
Fabrikam	Reseller
Fabrikam	Store
Litware	Catalog
Litware	Online
Litware	Reseller
Litware	Store

EXERCÍCIOS



9

Questão 9

Neste exercício, você deverá relacionar as tabelas FactOnlineSales com DimPromotion. Identifique a coluna que as duas tabelas têm em comum e utilize-a para criar esse relacionamento.

Retorne uma tabela contendo as seguintes colunas:

- OnlineSalesKey
- DateKey
- PromotionName
- SalesAmount

A sua consulta deve considerar apenas as linhas de vendas referentes a produtos com desconto (PromotionName <> 'No Discount'). Além disso, você deverá ordenar essa tabela de acordo com a coluna DateKey, em ordem crescente.

EXERCÍCIOS

10

Questão 10

A tabela abaixo é resultado de um Join entre a tabela FactSales e as tabelas: DimChannel, DimStore e DimProduct.

Recrie esta consulta e classifique em ordem crescente de acordo com SalesAmount.

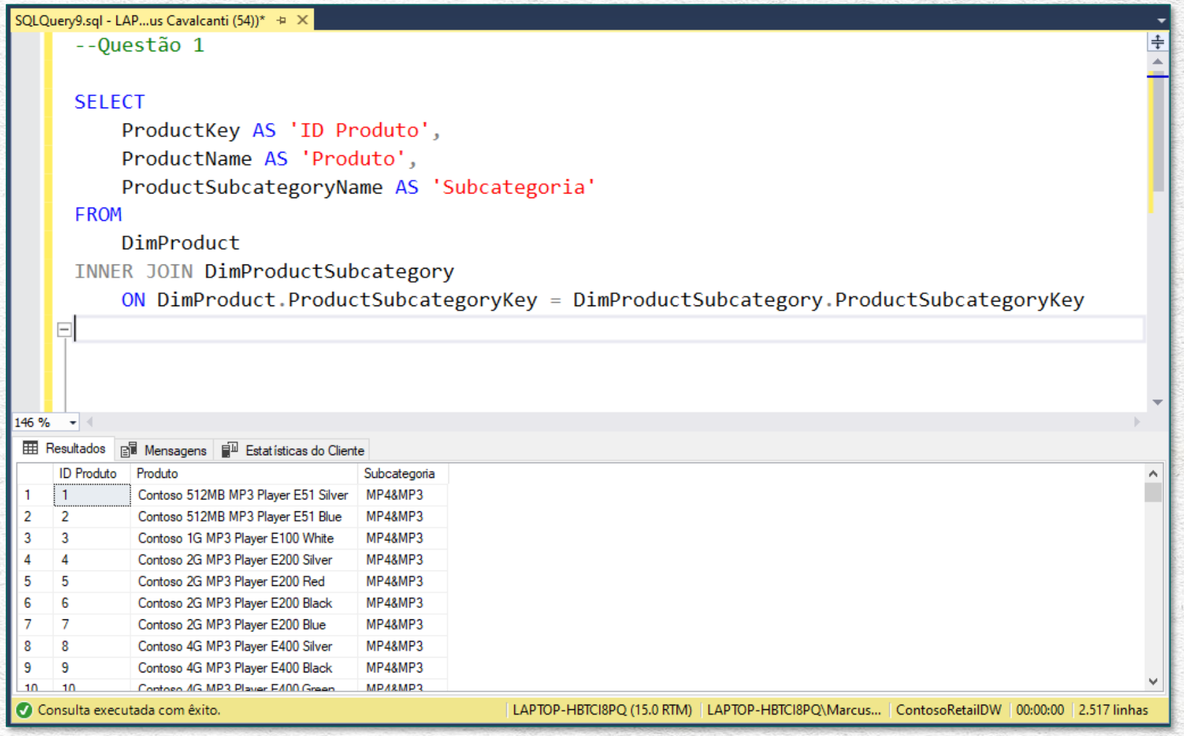
SalesKey	ChannelName	StoreName	ProductName	SalesAmount
145071	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	408016,02
1873989	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	329598,97
729520	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Green	272718,12
1453383	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	245119,234
2241482	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Grey	245119,234
541145	Catalog	Contoso Catalog Store	Contoso Projector 1080p X980 White	235467,00
1366947	Catalog	Contoso Catalog Store	Contoso Water Heater 7.2GPM X1800 White	227445,00
383636	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	205865,04
686002	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Blue	204803,88
517342	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Blue	202681,56
2883639	Catalog	Contoso Catalog Store	Proseware Projector 1080p DLP86 White	190923,60
1498028	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Green	190212,93
3106199	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Brown	188799,41
1623258	Catalog	Contoso Catalog Store	Litware Washer & Dryer 24in M260 Blue	186890,40
2850262	Catalog	Contoso Catalog Store	Adventure Works Desktop PC3.0 MS300 Silver	182441,00

GABARITOS

1

Questão 1

Utilize o INNER JOIN para trazer os nomes das subcategorias dos produtos, da tabela DimProductSubcategory para a tabela DimProduct.



GABARITOS

2

Questão 2

Identifique uma coluna em comum entre as tabelas DimProductSubcategory e DimProductCategory. Utilize essa coluna para complementar informações na tabela DimProductSubcategory a partir da DimProductCategory. Utilize o LEFT JOIN.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 2

SELECT
    ProductSubcategoryKey AS 'ID Subcategoria',
    ProductSubcategoryName AS 'Subcategoria',
    ProductCategoryName AS 'Categoria'
FROM
    DimProductSubcategory
LEFT JOIN DimProductCategory
    ON DimProductSubcategory.ProductCategoryKey = DimProductCategory.ProductCategoryKey
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	ID Subcategoria	Subcategoria	Categoria
1	1	MP4&MP3	Audio
2	2	Recorder	Audio
3	3	Radio	Audio
4	4	Recording Pen	Audio
5	5	Headphones	Audio
6	6	Bluetooth Headphones	Audio
7	7	Speakers	Audio
8	8	Audio Accessories	Audio
9	9	Televisions	TV and Video
10	10	VCD & DVD	TV and Video

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:02 | 44 linhas

GABARITOS

3

Questão 3

Para cada loja da tabela DimStore, descubra qual o Continente e o Nome do País associados (de acordo com DimGeography). Seu SELECT final deve conter apenas as seguintes colunas: StoreKey, StoreName, EmployeeCount, ContinentName e RegionCountryName. Utilize o LEFT JOIN neste exercício.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 3

SELECT
    StoreKey AS 'ID Loja',
    StoreName AS 'Loja',
    Employeecount AS 'Qtd. Funcionários',
    ContinentName AS 'Continente',
    RegionCountryName AS 'País'
FROM
    DimStore
LEFT JOIN DimGeography
    ON DimStore.GeographyKey = DimGeography.GeographyKey
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	ID Loja	Loja	Qtd. Funcionários	Continente	País
1	4	Contoso Bellevue Store	19	North America	United States
2	156	Contoso Cambridge Store	26	Europe	United Kingdom
3	88	Contoso Cedar Park Store	19	North America	United States
4	214	Contoso Leeds Store	25	Europe	United Kingdom
5	201	Contoso London Store	17	Europe	United Kingdom
6	215	Contoso Paris Store	12	Europe	France
7	309	Contoso Europe Reseller	12	Europe	France
8	17	Contoso Spokane Store	25	North America	United States
9	253	Contoso Sydney No.1 Store	20	Asia	Australia
10	300	Contoso Sydney No.2 Store	95	Asia	Australia

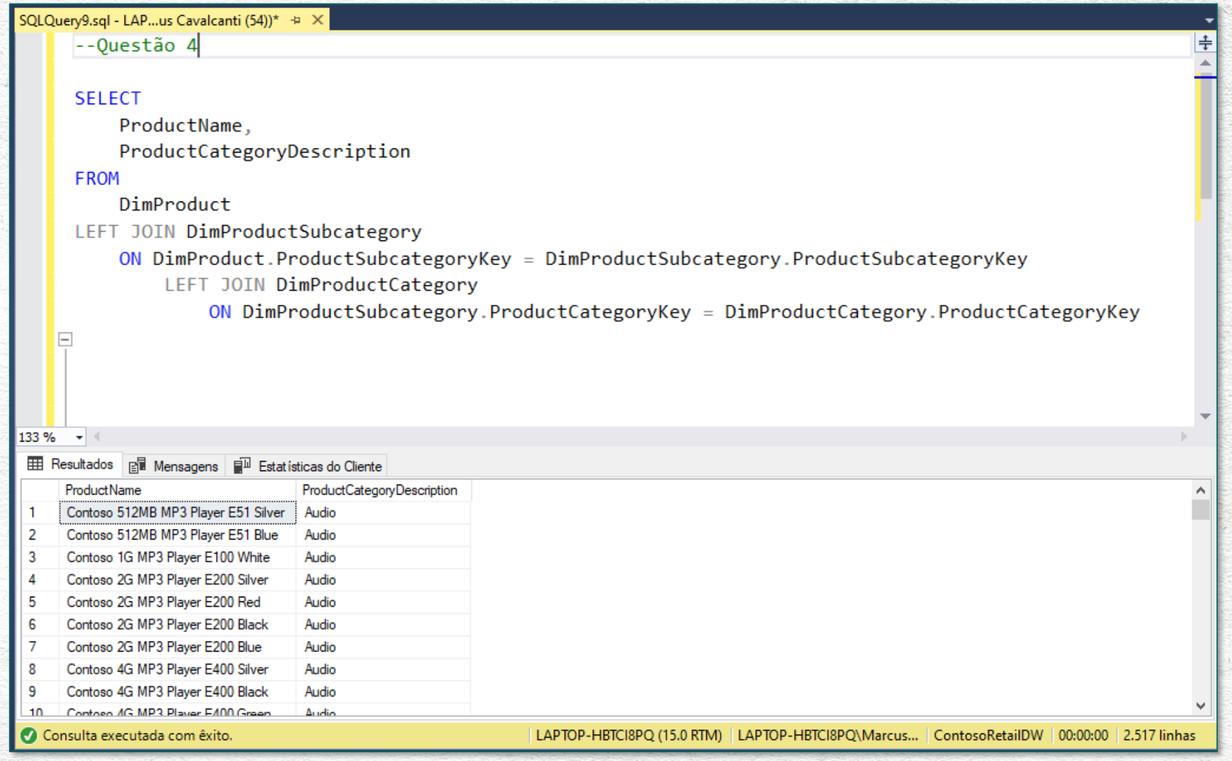
Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:25 | 306 linhas

GABARITOS

4

Questão 4

Complementa a tabela DimProduct com a informação de ProductCategoryDescription. Utilize o LEFT JOIN e retorne em seu SELECT apenas as 5 colunas que considerar mais relevantes.



	ProductName	ProductCategoryDescription
1	Contoso 512MB MP3 Player E51 Silver	Audio
2	Contoso 512MB MP3 Player E51 Blue	Audio
3	Contoso 1G MP3 Player E100 White	Audio
4	Contoso 2G MP3 Player E200 Silver	Audio
5	Contoso 2G MP3 Player E200 Red	Audio
6	Contoso 2G MP3 Player E200 Black	Audio
7	Contoso 2G MP3 Player E200 Blue	Audio
8	Contoso 4G MP3 Player E400 Silver	Audio
9	Contoso 4G MP3 Player E400 Black	Audio
10	Contoso 4G MP3 Player E400 Green	Audio

GABARITOS

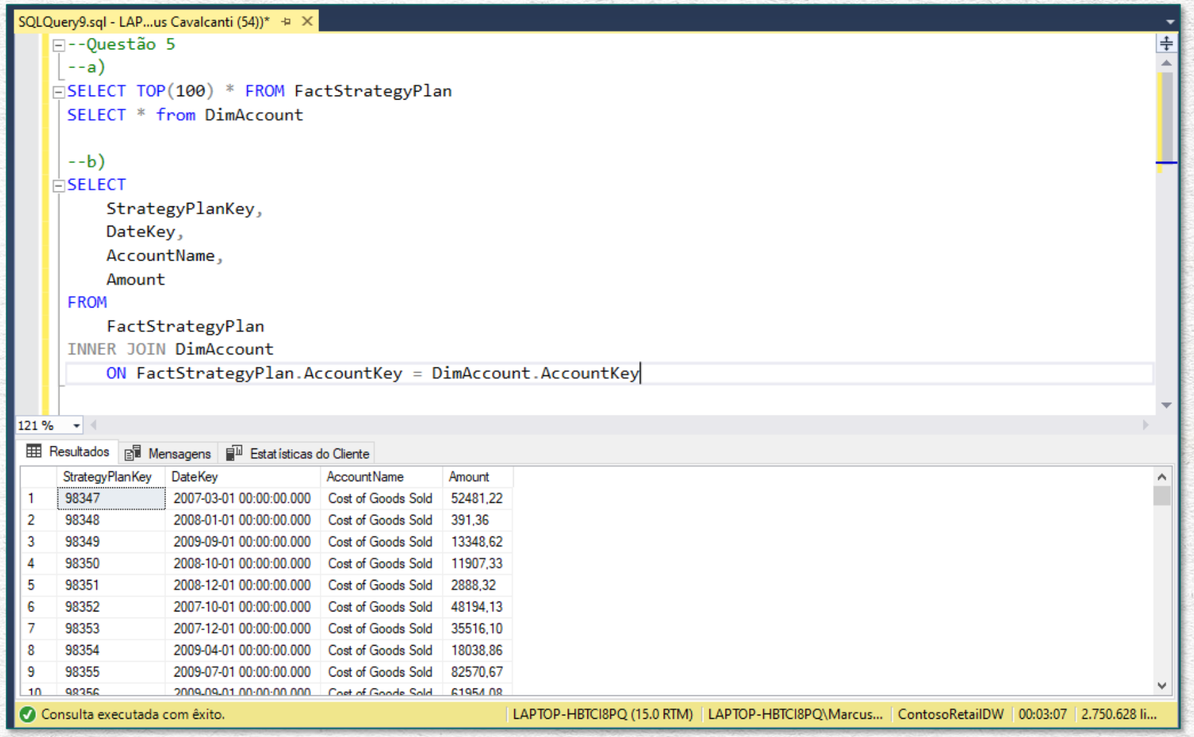
5

Questão 5

A tabela FactStrategyPlan resume o planejamento estratégico da empresa. Cada linha representa um montante destinado a uma determinada AccountKey.

a. Faça um SELECT das 100 primeiras linhas de FactStrategyPlan para reconhecer a tabela.

b. Faça um INNER JOIN para criar uma tabela contendo o AccountName para cada AccountKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas:
   - StrategyPlanKey
   - DateKey
   - AccountName
   - Amount



	StrategyPlanKey	DateKey	AccountName	Amount
1	98347	2007-03-01 00:00:00.000	Cost of Goods Sold	52481,22
2	98348	2008-01-01 00:00:00.000	Cost of Goods Sold	391,36
3	98349	2009-09-01 00:00:00.000	Cost of Goods Sold	13348,62
4	98350	2008-10-01 00:00:00.000	Cost of Goods Sold	11907,33
5	98351	2008-12-01 00:00:00.000	Cost of Goods Sold	2888,32
6	98352	2007-10-01 00:00:00.000	Cost of Goods Sold	48194,13
7	98353	2007-12-01 00:00:00.000	Cost of Goods Sold	35516,10
8	98354	2009-04-01 00:00:00.000	Cost of Goods Sold	18038,86
9	98355	2009-07-01 00:00:00.000	Cost of Goods Sold	82570,67
10	98356	2009-09-01 00:00:00.000	Cost of Goods Sold	61954,08

GABARITOS

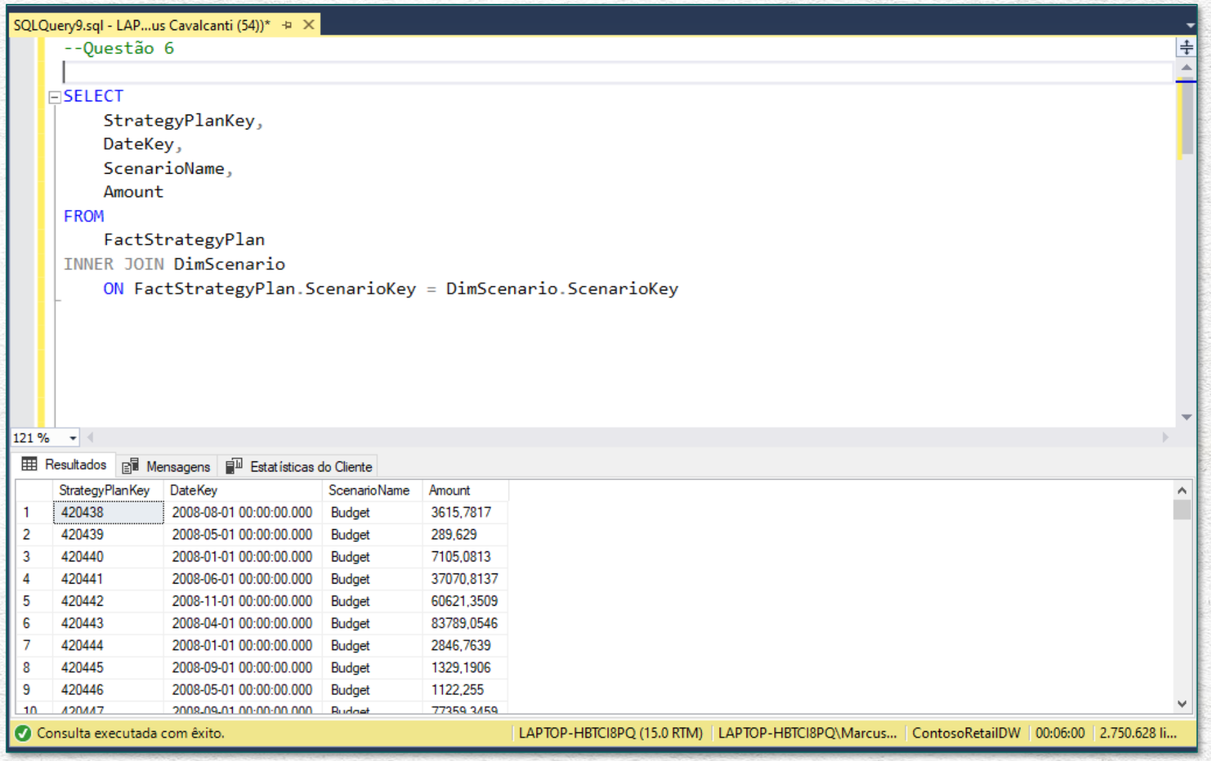
6

Questão 6

Vamos continuar analisando a tabela FactStrategyPlan. Além da coluna AccountKey que identifica o tipo de conta, há também uma outra coluna chamada ScenarioKey. Essa coluna possui a numeração que identifica o tipo de cenário: Real, Orçado e Previsão.

Faça um INNER JOIN para criar uma tabela contendo o ScenarioName para cada ScenarioKey da tabela FactStrategyPlan. O seu SELECT final deve conter as colunas:

- StrategyPlanKey
- DateKey
- ScenarioName
- Amount



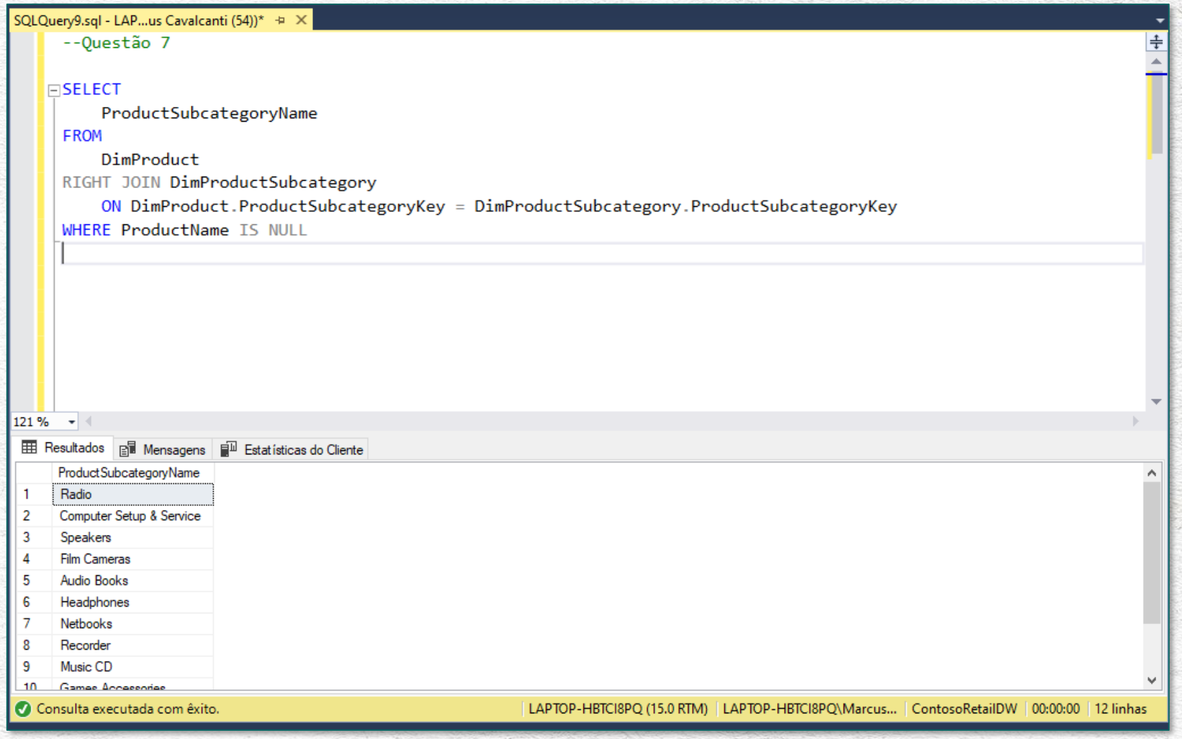
	StrategyPlanKey	DateKey	ScenarioName	Amount
1	420438	2008-08-01 00:00:00.000	Budget	3615,7817
2	420439	2008-05-01 00:00:00.000	Budget	289,629
3	420440	2008-01-01 00:00:00.000	Budget	7105,0813
4	420441	2008-06-01 00:00:00.000	Budget	37070,8137
5	420442	2008-11-01 00:00:00.000	Budget	60621,3509
6	420443	2008-04-01 00:00:00.000	Budget	83789,0546
7	420444	2008-01-01 00:00:00.000	Budget	2846,7639
8	420445	2008-09-01 00:00:00.000	Budget	1329,1906
9	420446	2008-05-01 00:00:00.000	Budget	1122,255
10	420447	2008-09-01 00:00:00.000	Budget	77359,3459

GABARITOS

7

Questão 7

Algumas subcategorias não possuem nenhum exemplar de produto. Identifique que subcategorias são essas.



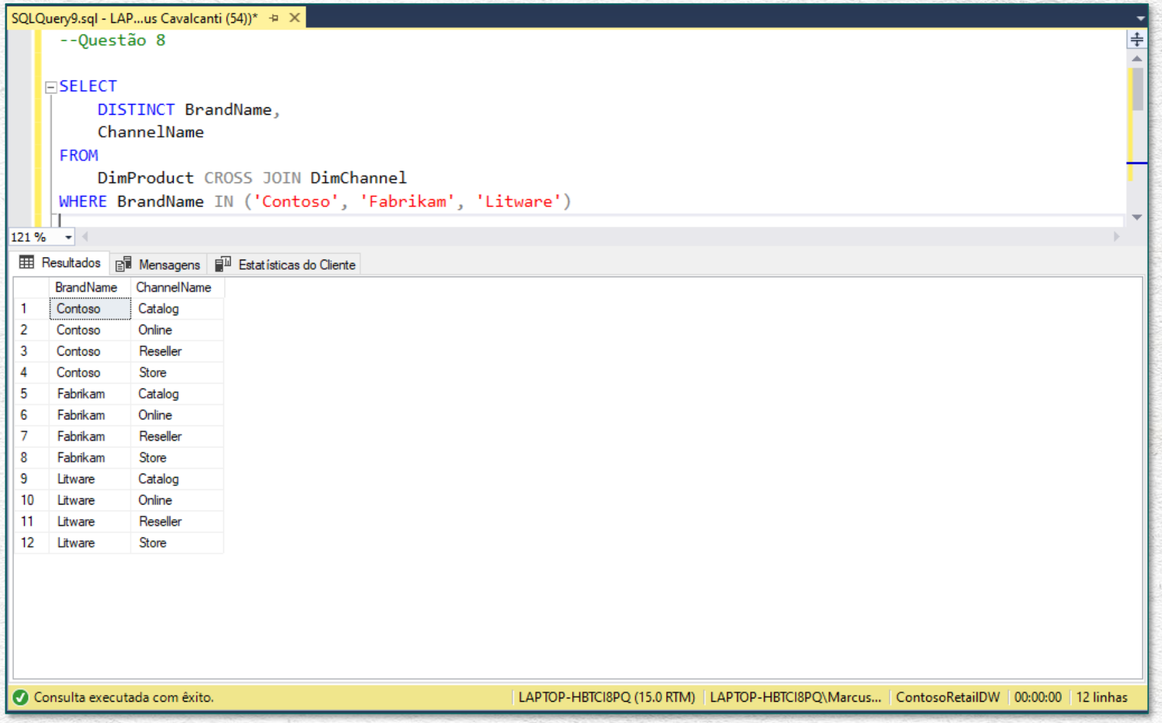
GABARITOS

8

Questão 8

A tabela abaixo mostra a combinação entre Marca e Canal de Venda, para as marcas Contoso, Fabrikam e Litware. Crie um código SQL para chegar no mesmo resultado.

BrandName	ChannelName
Contoso	Catalog
Contoso	Online
Contoso	Reseller
Contoso	Store
Fabrikam	Catalog
Fabrikam	Online
Fabrikam	Reseller
Fabrikam	Store
Litware	Catalog
Litware	Online
Litware	Reseller
Litware	Store



GABARITOS

9

Questão 9

Neste exercício, você deverá relacionar as tabelas FactOnlineSales com DimPromotion. Identifique a coluna que as duas tabelas têm em comum e utilize-a para criar esse relacionamento.

Retorne uma tabela contendo as seguintes colunas:

- OnlineSalesKey
- DateKey
- PromotionName
- SalesAmount

A sua consulta deve considerar apenas as linhas de vendas referentes a produtos com desconto (PromotionName <> 'No Discount'). Além disso, você deverá ordenar essa tabela de acordo com a coluna DateKey, em ordem crescente.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 9

SELECT TOP(1000)
    OnlineSalesKey,
    DateKey,
    PromotionName,
    SalesAmount
FROM
    FactOnlineSales
INNER JOIN DimPromotion
    ON FactOnlineSales.PromotionKey = DimPromotion.PromotionKey
WHERE PromotionName <> 'No Discount'
ORDER BY DateKey ASC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	OnlineSalesKey	DateKey	PromotionName	SalesAmount
1	31963608	2007-01-01 00:00:00.000	North America Spring Promotion	595,65
2	19560484	2007-01-01 00:00:00.000	European Holiday Promotion	10,36
3	31963609	2007-01-01 00:00:00.000	North America Spring Promotion	565,25
4	31541784	2007-01-01 00:00:00.000	North America Spring Promotion	48,4405
5	31963610	2007-01-01 00:00:00.000	European Holiday Promotion	501,60
6	19560485	2007-01-01 00:00:00.000	European Holiday Promotion	10,36
7	31963611	2007-01-01 00:00:00.000	European Holiday Promotion	476,00
8	31541785	2007-01-01 00:00:00.000	North America Spring Promotion	39,6435
9	31963612	2007-01-01 00:00:00.000	Asian Holiday Promotion	532,95
10	19560486	2007-01-01 00:00:00.000	European Holiday Promotion	10,36
11	31963613	2007-01-01 00:00:00.000	Asian Holiday Promotion	505,75
12	31541786	2007-01-01 00:00:00.000	Asian Holiday Promotion	43,3415

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:02:11 | 1.000 linhas

GABARITOS

10

Questão 10

A tabela abaixo é resultado de um Join entre a tabela FactSales e as tabelas: DimChannel, DimStore e DimProduct.

Recrie esta consulta e classifique em ordem decrescente de acordo com SalesAmount.

SalesKey	ChannelName	StoreName	ProductName	SalesAmount
145071	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	408016,02
1873989	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	329598,97
729520	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Green	272718,12
1453383	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	245119,234
2241482	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Grey	245119,234
541145	Catalog	Contoso Catalog Store	Contoso Projector 1080p X980 White	235467,00
1366947	Catalog	Contoso Catalog Store	Contoso Water Heater 7.2GPM X1800 White	227445,00
383636	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	205865,04
686002	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Blue	204803,88
517342	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Blue	202681,56
2883639	Catalog	Contoso Catalog Store	Proseware Projector 1080p DLP86 White	190923,60
1498028	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Green	190212,93
3106199	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Brown	188799,41
1623258	Catalog	Contoso Catalog Store	Litware Washer & Dryer 24in M260 Blue	186890,40
2850262	Catalog	Contoso Catalog Store	Adventure Works Desktop PC3.0 MS300 Silver	182441,00

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 10
SELECT TOP(100)
    SalesKey,
    ChannelName,
    StoreName,
    ProductName,
    SalesAmount
FROM
    factSales
INNER JOIN DimChannel
    ON FactSales.channelKey = DimChannel.channelKey
INNER JOIN DimStore
    ON FactSales.StoreKey = DimStore.StoreKey
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
ORDER BY SalesAmount DESC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	SalesKey	ChannelName	StoreName	ProductName	SalesAmount
1	145071	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	408016,02
2	1873989	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	329598,97
3	729520	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Green	272718,12
4	2241482	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Grey	245119,234
5	1453383	Catalog	Contoso Catalog Store	Litware Refrigerator 24.7CuFt X980 Silver	245119,234
6	541145	Catalog	Contoso Catalog Store	Contoso Projector 1080p X980 White	235467,00
7	1366947	Catalog	Contoso Catalog Store	Contoso Water Heater 7.2GPM X1800 White	227445,00
8	383636	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Silver	205865,04
9	686002	Catalog	Contoso Catalog Store	NT Washer & Dryer 27in L2700 Blue	204803,88

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:04:15 | 100 linhas

MÓDULO 8

SQL

Group By + Joins
Aplicações

O QUE VEREMOS NESTE MÓDULO

MÓDULO 8 GROUP BY + JOINS

01	GROUP BY + INNER JOIN	199
02	EXERCÍCIOS	200
03	GABARITOS	204

GROUP BY + INNER JOIN

Nos últimos dois módulos, aprendemos a criar agrupamentos com o **GROUP BY** e também aprendemos a fazer relações/junções entre tabelas, por meio dos **JOINs**.

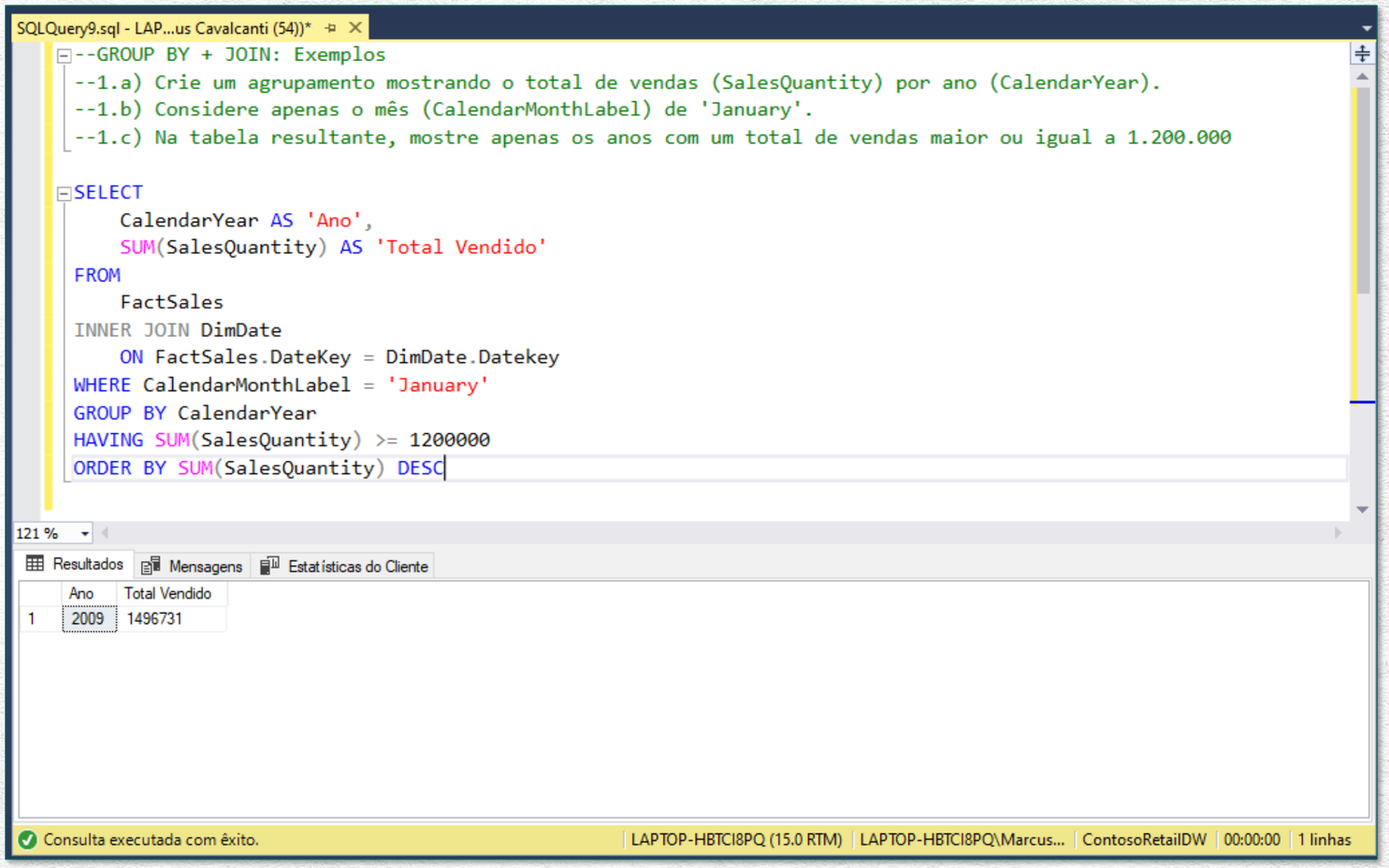
Nesse módulo, vamos praticar com alguns casos que combinam os dois conceitos: criar agrupamentos utilizando o Join.

Ao lado, temos um exemplo dessa combinação (enunciado do exercício está na própria imagem). Não há nenhuma novidade em relação a combinação do GROU BY e do JOIN.

Na verdade, é até mais fácil do que parece. Para criar agrupamentos, aproveitando as relações criadas com o JOIN, basta adicionar um GROUP BY logo ao final do código da consulta, seguindo exatamente a mesma ordem que já aprendemos.

Aqui também valem os filtros WHERE e HAVING, combinados com o GROUP BY.

Agora, vamos praticar!



Índice Geral
Índice deste Módulo

EXERCÍCIOS

1 Questão 1

a. Faça um resumo da quantidade vendida (Sales Quantity) de acordo com o nome do canal de vendas (ChannelName). Você deve ordenar a tabela final de acordo com SalesQuantity, em ordem decrescente.

b. Faça um agrupamento mostrando a quantidade total vendida (Sales Quantity) e quantidade total devolvida (Return Quantity) de acordo com o nome das lojas (StoreName).

c. Faça um resumo do valor total vendido (Sales Amount) para cada mês (CalendarMonthLabel) e ano (CalendarYear).

2 Questão 2

Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesAmount) por produto.

a. Descubra qual é a cor de produto que mais é vendida (de acordo com SalesQuantity).

b. Quantas cores tiveram uma quantidade vendida acima de 3.000.000.

EXERCÍCIOS

3 Questão 3

Crie um agrupamento de quantidade vendida (SalesQuantity) por categoria do produto (ProductCategoryName).

Obs: Você precisará fazer mais de 1 INNER JOIN, dado que a relação entre FactSales e DimProductCategory não é direta.

4 Questão 4

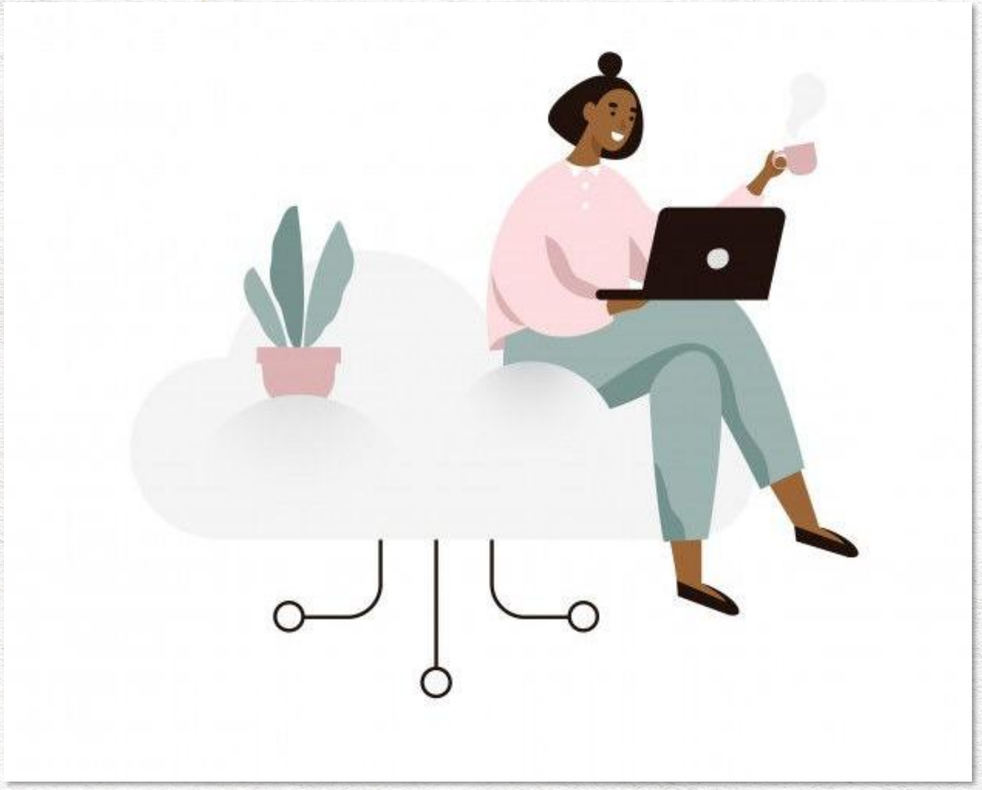
a. Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o nome completo do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

b. Feito isso, faça um agrupamento de produtos e descubra quais foram os top 10 produtos mais comprados pelo cliente da letra a, considerando o nome do produto.

5 Questão 5

Faça um resumo mostrando o total de produtos comprados (Sales Quantity) de acordo com o sexo dos clientes.

EXERCÍCIOS



6 Questão 6

Faça uma tabela resumo mostrando a taxa de câmbio média de acordo com cada CurrencyDescription. A tabela final deve conter apenas taxas entre 10 e 100.

7 Questão 7

Descubra o valor total na tabela FactStrategyPlan destinado aos cenários: Actual e Budget.

8 Questão 8

Faça uma tabela resumo mostrando o resultado do planejamento estratégico (FactStrategyPlan) por ano.

EXERCÍCIOS

9

Questão 9

Faça um agrupamento de quantidade de produtos por ProductSubcategoryName. Leve em consideração em sua análise apenas a marca Contoso e a cor Silver.

10

Questão 10

Faça um agrupamento duplo de quantidade de produtos por BrandName e ProductSubcategoryName. A tabela final deverá ser ordenada de acordo com a coluna BrandName.

GABARITOS

1

Questão 1

a. Faça um resumo da quantidade vendida (Sales Quantity) de acordo com o nome do canal de vendas (ChannelName). Você deve ordenar a tabela final de acordo com SalesQuantity, em ordem decrescente.

b. Faça um agrupamento mostrando a quantidade total vendida (Sales Quantity) e quantidade total devolvida (Return Quantity) de acordo com o nome das lojas (StoreName).

c. Faça um resumo do valor total vendido (Sales Amount) para cada mês (CalendarMonthLabel) e ano (CalendarYear).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 1
--a)

SELECT
    ChannelName AS 'Canal de Venda',
    SUM(SalesQuantity) AS 'Total Vendido'
FROM
    FactSales
INNER JOIN DimChannel
    ON FactSales.channelKey = DimChannel.ChannelKey
GROUP BY ChannelName
ORDER BY SUM(SalesQuantity)
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Canal de Venda	Total Vendido
1	Catalog	4454373
2	Reseller	7270570
3	Online	12506063
4	Store	29089448

GABARITOS

1

Questão 1

a. Faça um resumo da quantidade vendida (Sales Quantity) de acordo com o nome do canal de vendas (ChannelName). Você deve ordenar a tabela final de acordo com SalesQuantity, em ordem decrescente.

b. Faça um agrupamento mostrando a quantidade total vendida (Sales Quantity) e quantidade total devolvida (Return Quantity) de acordo com o nome das lojas (StoreName).

c. Faça um resumo do valor total vendido (Sales Amount) para cada mês (CalendarMonthLabel) e ano (CalendarYear).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 1
--b)
SELECT
    StoreName AS 'Loja',
    SUM(SalesQuantity) AS 'Total Vendido',
    SUM(ReturnQuantity) AS 'Total Vendido'
FROM
    FactSales
INNER JOIN DimStore
    ON FactSales.StoreKey = DimStore.StoreKey
GROUP BY StoreName
ORDER BY StoreName
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Loja	Total Vendido	Total Vendido
1	Contoso Albany Store	94833	856
2	Contoso Alexandria Store	94804	865
3	Contoso Amsterdam Store	82194	946
4	Contoso Anchorage Store	96089	1016
5	Contoso Annapolis Store	94081	867
6	Contoso Appleton Store	92402	865
7	Contoso Arlington Store	93675	898
8	Contoso Ashgabat No.2 Store	160581	1384
9	Contoso Ashgabat No.1 Store	163541	1450
10	Contoso Asia Online Store	4232977	34326
11	Contoso Asia Reseller	2452197	20677
12	Contoso Athens Store	86461	945
13	Contoso Atlantic City Store	90935	902
14	Contoso Attleboro Store	92652	805

GABARITOS

1

Questão 1

a. Faça um resumo da quantidade vendida (Sales Quantity) de acordo com o nome do canal de vendas (ChannelName). Você deve ordenar a tabela final de acordo com SalesQuantity, em ordem decrescente.

b. Faça um agrupamento mostrando a quantidade total vendida (Sales Quantity) e quantidade total devolvida (Return Quantity) de acordo com o nome das lojas (StoreName).

c. Faça um resumo do valor total vendido (Sales Amount) para cada mês (CalendarMonthLabel) e ano (CalendarYear).

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 1
--c)
SELECT
    CalendarYear AS 'Ano',
    CalendarMonthLabel AS 'Mês',
    SUM(SalesAmount) AS 'Faturamento Total'
FROM
    FactSales
INNER JOIN DimDate
    ON FactSales.DateKey = DimDate.DateKey
GROUP BY CalendarYear, CalendarMonthLabel, CalendarMonth
ORDER BY CalendarMonth ASC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Ano	Mês	Faturamento Total
1	2007	January	269835263,2337
2	2007	February	298215968,3498
3	2007	March	300486926,9015
4	2007	April	400160331,5966
5	2007	May	423429127,7894
6	2007	June	409797545,5526
7	2007	July	389617372,2722
8	2007	August	388429827,1124
9	2007	September	379144599,5604
10	2007	October	423213240,8428
11	2007	November	453750209,243

GABARITOS

2

Questão 2

Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesAmount) por produto.

a. Descubra qual é a cor de produto que mais é vendida (de acordo com SalesQuantity).

b. Quantas cores tiveram uma quantidade vendida acima de 3.000.000.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 2
--a)
SELECT
    ColorName AS 'Cor',
    SUM(SalesQuantity) AS 'Qtd.Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ColorName
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Cor	Qtd.Vendida
1	Azure	251937
2	Purple	65736
3	Silver	9726178
4	Brown	1302543
5	Transparent	399520
6	Red	3223538
7	Blue	3179880
8	Orange	981342
9	Silver Grey	286107
10	Yellow	1329259
11	White	10897426

GABARITOS

2

Questão 2

Você precisa fazer uma análise de vendas por produtos. O objetivo final é descobrir o valor total vendido (SalesAmount) por produto.

a. Descubra qual é a cor de produto que mais é vendida (de acordo com SalesQuantity).

b. Quantas cores tiveram uma quantidade vendida acima de 3.000.000.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 2
--b)
SELECT
    ColorName AS 'Cor',
    SUM(SalesQuantity) AS 'Qtd.Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ColorName
HAVING SUM(SalesQuantity) >= 3000000
ORDER BY SUM(SalesQuantity) DESC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Cor	Qtd.Vendida
1	Black	13230557
2	White	10897426
3	Silver	9726178
4	Grey	4784400
5	Red	3223538
6	Blue	3179880

GABARITOS

3

Questão 3

Crie um agrupamento de quantidade vendida (SalesQuantity) por categoria do produto (ProductCategoryName).

Obs: Você precisará fazer mais de 1 INNER JOIN, dado que a relação entre FactSales e DimProductCategory não é direta.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 3

SELECT
    ProductCategoryName AS 'Categoria',
    SUM(SalesQuantity) AS 'Qtd.Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
        INNER JOIN DimProductSubcategory
            ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
                INNER JOIN DimProductCategory
                    ON DimProductSubcategory.ProductCategoryKey = DimProductCategory.ProductCategoryKey
GROUP BY ProductCategoryName
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Categoria	Qtd.Vendida
1	Games and Toys	6040572
2	Audio	1353298
3	Cell phones	13459636
4	Music, Movies and Audio Books	1523415
5	TV and Video	3449196
6	Cameras and camcorders	6551189
7	Computers	10563676
8	Home Appliances	10379472

GABARITOS

4

Questão 4

a. Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o nome completo do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

b. Feito isso, faça um agrupamento de produtos e descubra quais foram os top 10 produtos mais comprados pelo cliente da letra a, considerando o nome do produto.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 4
--a)
SELECT
    DimCustomer.CustomerKey AS 'Categoria',
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    SUM(SalesQuantity) AS 'Qtd.Vendida'
FROM
    FactOnlineSales
INNER JOIN DimCustomer
    ON FactOnlineSales.CustomerKey = DimCustomer.CustomerKey
WHERE CustomerType = 'Person'
GROUP BY DimCustomer.CustomerKey, FirstName, LastName
ORDER BY SUM(SalesQuantity) DESC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

	Categoria	Nome	Sobrenome	Qtd.Vendida
1	7665	Robert	Long	376
2	17994	Destiny	Perry	359
3	3735	Kevin	Edwards	358
4	12667	Olivia	Anderson	355
5	4250	Abigail	Anderson	353
6	13928	Kaitlyn	Harris	353
7	17989	Harold	Perez	352
8	17783	Cody	Howard	351
9	11348	Abigail	Long	351
10	12790	Ryan	Walker	350

GABARITOS

4 Questão 4

a. Você deve fazer uma consulta à tabela FactOnlineSales e descobrir qual é o nome completo do cliente que mais realizou compras online (de acordo com a coluna SalesQuantity).

b. Feito isso, faça um agrupamento de produtos e descubra quais foram os top 10 produtos mais comprados pelo cliente da letra a, considerando o nome do produto.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 4
--b)
SELECT
    ProductName AS 'Produto',
    SUM(SalesQuantity) AS 'Qtd.Vendida'
FROM
    FactOnlineSales
INNER JOIN DimProduct
    ON FactOnlineSales.ProductKey = DimProduct.ProductKey
WHERE CustomerKey = 7665
GROUP BY ProductName
ORDER BY SUM(SalesQuantity) DESC
```

121 %

Resultados | Mensagens | Estatísticas do Cliente

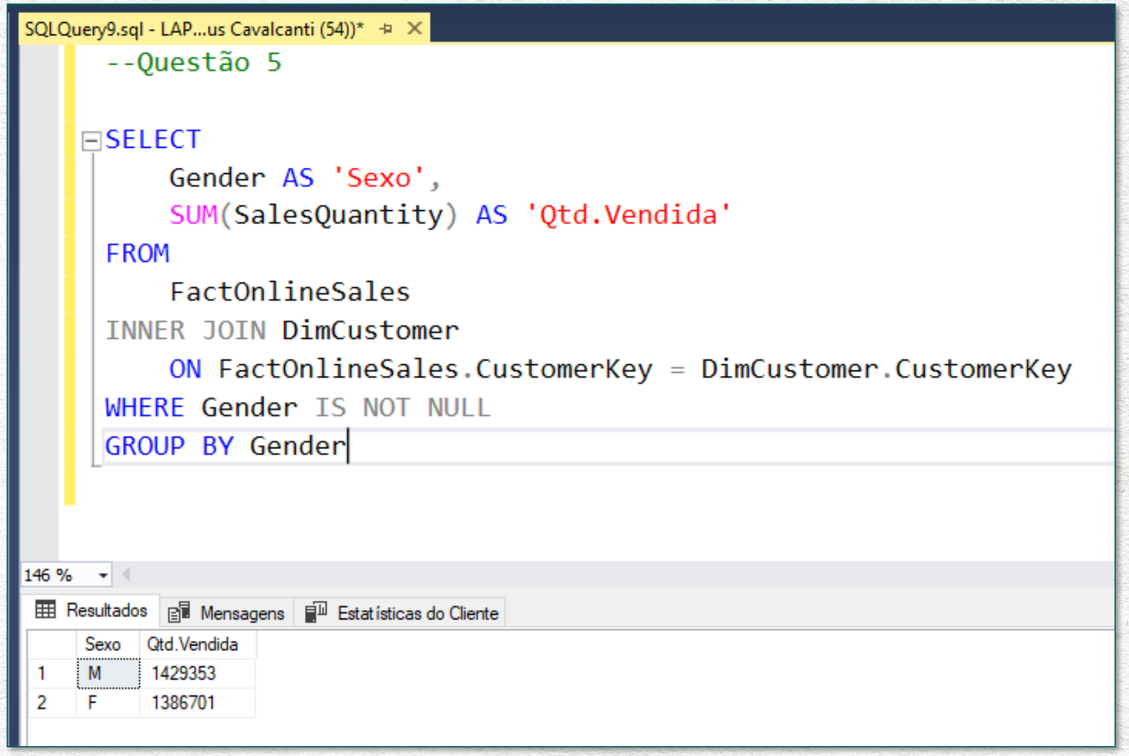
	Produto	Qtd.Vendida
1	Adventure Works 26" 720p LCD HDTV M140 Silver	37
2	SV 16xDVD M360 Black	37
3	Contoso Telephoto Conversion Lens X400 Silver	24
4	A. Datum SLR Camera X137 Grey	24
5	NT Bluetooth Stereo Headphones E52 Blue	17
6	Contoso 4G MP3 Player E400 Silver	16
7	Contoso Optical USB Mouse M45 White	15
8	SV Keyboard E90 White	15
9	SV 40GB USB2.0 Portable Hard Disk E400 Silver	9
10	Contoso USB Cable M250 White	9

GABARITOS

5

Questão 5

Faça um resumo mostrando o total de produtos comprados (Sales Quantity) de acordo com o sexo dos clientes.



GABARITOS

6

Questão 6

Faça uma tabela resumo mostrando a taxa de câmbio média de acordo com cada CurrencyDescription. A tabela final deve conter apenas taxas entre 10 e 100.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 6

SELECT
    CurrencyDescription,
    AVG(AverageRate) AS 'Taxa Média'
FROM
    FactExchangeRate
INNER JOIN DimCurrency
    ON FactExchangeRate.CurrencyKey = DimCurrency.CurrencyKey
GROUP BY CurrencyDescription
HAVING AVG(AverageRate) BETWEEN 10 AND 100
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	CurrencyDescription	Taxa Média
1	Bhutan Ngultrum	45,3475367741935
2	Indian Rupee	43,3835251612903
3	Russian Ruble	26,4664903225806
4	Syrian Pound	48,6909677419355
5	Taiwan Dollar	31,9811390322581
6	Thai Baht	32,636354516129

GABARITOS

7

Questão 7

Descubra o valor total na tabela FactStrategyPlan destinado aos cenários: Actual e Budget.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 7

SELECT
    ScenarioName AS 'Cenario',
    SUM(Amount) AS 'Total'
FROM
    FactStrategyPlan
INNER JOIN DimScenario
    ON FactStrategyPlan.ScenarioKey = DimScenario.ScenarioKey
GROUP BY ScenarioName
HAVING ScenarioName IN ('Actual', 'Budget')
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	Cenario	Total
1	Actual	22412098587,5267
2	Budget	21041994176,9634

GABARITOS

8

Questão 8

Faça uma tabela resumo mostrando o resultado do planejamento estratégico (FactStrategyPlan) por ano.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 8

SELECT
    CalendarYear AS 'Ano',
    SUM(Amount) AS 'Total'
FROM
    FactStrategyPlan
INNER JOIN DimDate
    ON FactStrategyPlan.DateKey = DimDate.DateKey
GROUP BY CalendarYear
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	Ano	Total
1	2007	23909967021,2265
2	2008	21500169509,0507
3	2009	20483426084,2657

GABARITOS

9

Questão 9

Faça um agrupamento de quantidade de produtos por ProductSubcategoryName. Leve em consideração em sua análise apenas a marca Contoso e a cor Silver.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 9

SELECT
    ProductSubcategoryName AS 'Subcategoria',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
WHERE BrandName = 'Contoso' AND ColorName = 'Silver'
GROUP BY ProductSubcategoryName
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	Subcategoria	Qtd. Produtos
1	Air Conditioners	7
2	Cameras & Camcorders Accessories	17
3	Cell phones Accessories	7
4	Coffee Machines	6
5	Computers Accessories	8
6	Digital SLR Cameras	4
7	Home Theater System	13
8	Microwaves	6
9	Movie DVD	13
10	MP4&MP3	6

GABARITOS

10

Questão 10

Faça um agrupamento duplo de quantidade de produtos por BrandName e ProductSubcategoryName. A tabela final deverá ser ordenada de acordo com a coluna BrandName.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 10

SELECT
    BrandName AS 'Marca',
    ProductSubcategoryName AS 'Subcategoria',
    COUNT(*) AS 'Qtd. Produtos'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
GROUP BY BrandName, ProductSubcategoryName
ORDER BY BrandName ASC
```

146 %

Resultados | Mensagens | Estatísticas do Cliente

	Marca	Subcategoria	Qtd. Produtos
1	A. Datum	Digital Cameras	100
2	A. Datum	Digital SLR Cameras	32
3	Adventure Works	Coffee Machines	24
4	Adventure Works	Desktops	22
5	Adventure Works	Lamps	40
6	Adventure Works	Laptops	28
7	Adventure Works	Monitors	28
8	Adventure Works	Televisions	50
9	Contoso	Air Conditioners	35
10	Contoso	Cameras & Camcorders Accessories	69

MÓDULO 9

SQL

Variáveis

O QUE VEREMOS NESTE MÓDULO

MÓDULO 9 VARIÁVEIS

01	Tipos de dados e operações básicas	220
02	CAST e SQL_VARIANT_PROPERTY	222
03	FORMAT	223
04	ROUND, FLOOR e CEILING	224
05	DECLARE e SET	225
06	Utilizando uma variável em uma consulta (Ex. 1)	228
07	Utilizando uma variável em uma consulta (Ex. 2)	229
08	Armazenando o resultado da consulta em uma variável	230
09	PRINT	231
10	Armazenando um registro da consulta em uma variável	232
11	Acumulando valores dentro de uma variável	233
12	Variáveis globais	234
13	EXERCÍCIOS	235
14	GABARITOS	239

Tipos de dados e operações básicas

Todo dado possui um tipo específico: ele pode ser um número, um texto ou uma data.

Em resumo, temos abaixo os 4 principais tipos de dados:

1 Inteiro (INT)

Qualquer número que seja representado apenas pela sua parte inteira, sem casas decimais. Exemplos: 1, 100, 589.

A forma como o SQL vai entender que estamos trabalhando com um número inteiro será através do identificador **INT**

2 Decimal (FLOAT)

Qualquer número que contenha casas decimais. Exemplos: 10.33, 90.91, 410.87.

A forma como o SQL vai entender que estamos trabalhando com um número decimal será através do identificador **FLOAT**.

3 Texto (VARCHAR)

Toda cadeia de caracteres que pode ser interpretada como um texto. Exemplos: ‘Carla’, ‘Motorola’, ‘Pastel’, ‘44’.

A forma como o SQL vai entender que estamos trabalhando com um texto será através do identificador **VARCHAR**

4 Data (DATETIME)

Qualquer dado no formado de data. Exemplos: ‘01/01/2021’, ‘23/03/2021’.

A forma como o SQL vai entender que estamos trabalhando com uma data será através do identificador **DATETIME**

Tipos de dados e operações básicas

Os tipos de dados determinam quais operações conseguimos fazer com os dados: operações matemáticas, concatenação, etc.

Ao lado, temos alguns exemplos de operações que conseguimos fazer com os diferentes tipos de dados.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))*

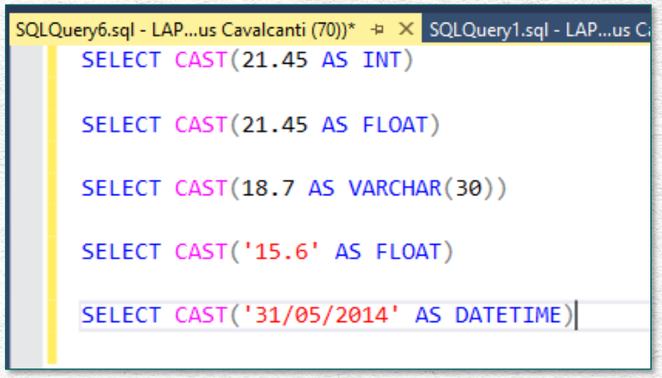
```
-- Operações básicas

SELECT 10 AS 'Número'

SELECT 'Marcus' AS 'Nome'

SELECT '21/06/2021' AS 'Data'

-- Operações com números

SELECT 10+20
SELECT 20-5
SELECT 31*40
SELECT 431.0/23

-- Operações com textos

SELECT 'SQL' + ' ' + 'Impressionador'
```

CAST e SQL_VARIANT_PROPERTY

A função **CAST** nos permite especificar o tipo de um dado. Ao lado, temos alguns exemplos.

Observe na forma como o SQL entende cada tipo de dado:

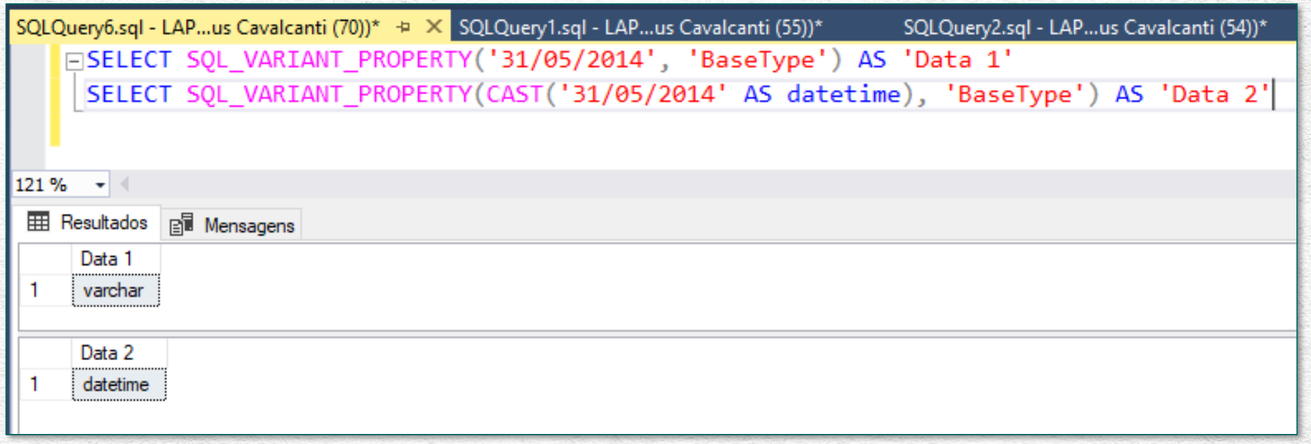
- **INT**: inteiro
- **FLOAT**: decimal
- **VARCHAR**: texto/string
- **DATETIME**: data



Um comando muito útil para se descobrir o tipo dos dados é o **SQL_VARIANT_PROPERTY**.

Esta função nos pede dois argumentos: o **primeiro argumento** é o dado que queremos descobrir o tipo, e o **segundo argumento** é a informação que queremos: **BaseType**, ou seja, o tipo do dado.

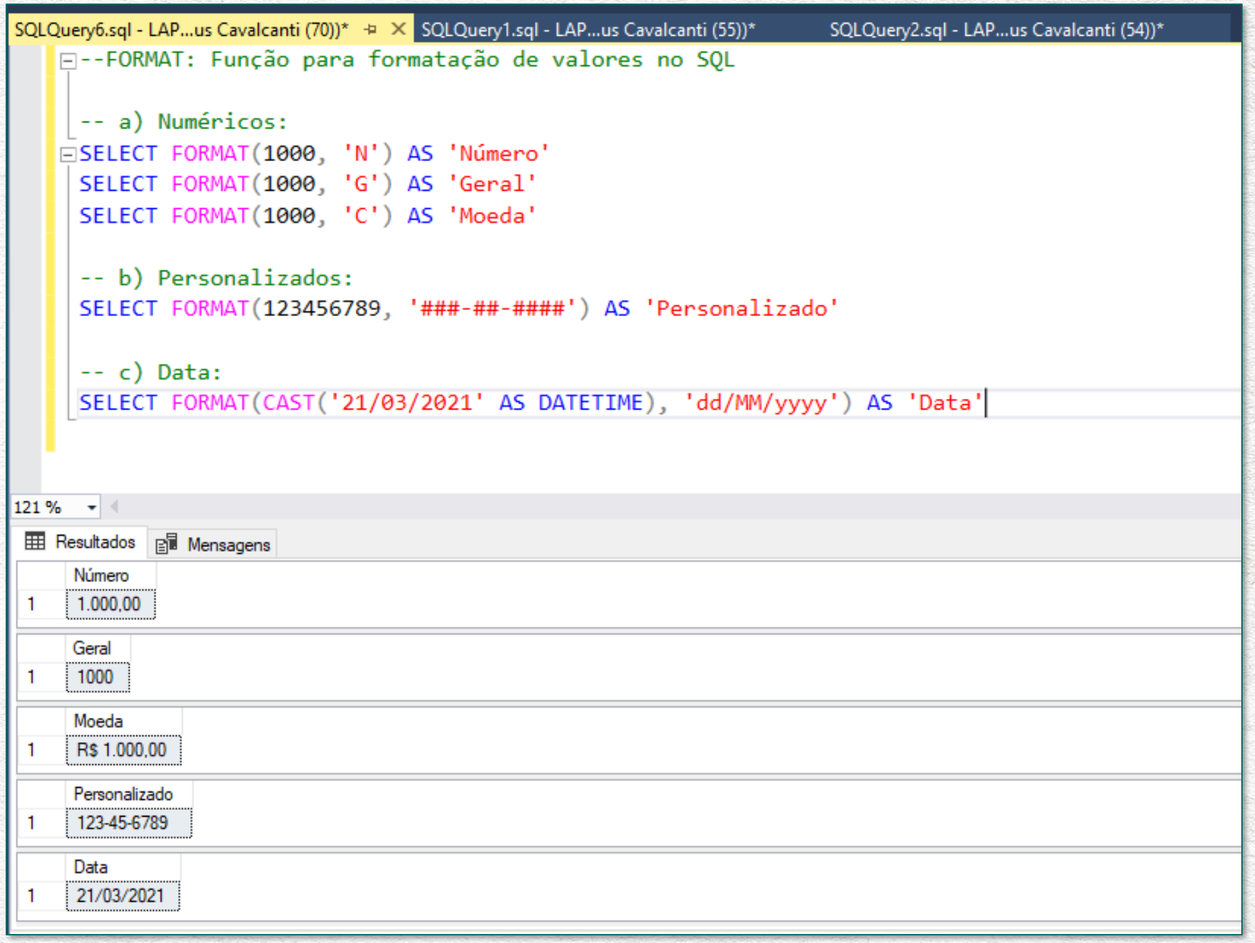
O resultado pode ser visto ao lado. Na primeira linha, passar apenas a data '31/05/2014' faz com que o dado seja armazenado como texto (**varchar**). Em contrapartida, quando usamos o CAST para forçar o tipo do dado para data, temos como resultado o **datetime**.



FORMAT

A função **FORMAT** nos permite formatar um valor a partir de uma determinado código de formatação.

Ao lado, temos os principais formatos que podemos utilizar com valores.



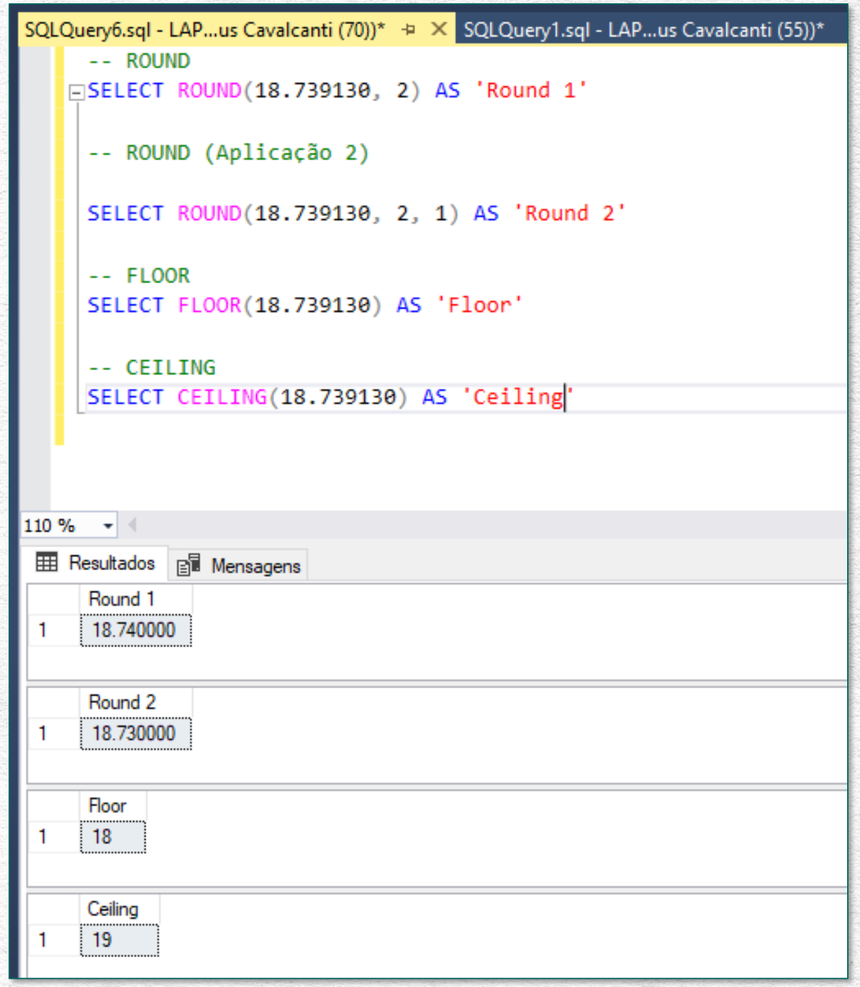
ROUND, FLOOR e CEILING: Funções de arredondamento

Existem algumas funções muito úteis para arredondamento de valores: ROUND, FLOOR e CEILING.

Essas funções são bem intuitivas:

- ROUND: arredonda um valor com uma quantidade de casas decimais.
- FLOOR: arredonda um valor para baixo, ou seja, para um número inteiro logo abaixo.
- CEILING: arredonda um valor para cima, ou seja, para um número inteiro logo acima.

Na imagem ao lado, é possível visualizar o resultado dessas funções, considerando o valor 18.739130.



Obs.: A função ROUND possui uma aplicação além de apenas arredondar um valor de acordo com um número de casas decimais. Na aplicação 2 da imagem ao lado, utilizamos um terceiro argumento (número 1) para informar que o que queremos na verdade não é arredondar, e sim cortar o número na segunda casa decimal. Então em vez do valor ficar em 18.74, ele fica em 18.73.

Índice Geral
Índice deste Módulo

DECLARE e SET

Chegou a hora de entender o que são variáveis no SQL e como declará-las.

1. O que é uma variável?

Uma variável é um objeto que armazena o valor de um dado. Imagine por exemplo a sua carteira. A sua carteira é capaz de armazenar uma determinada quantidade de dinheiro: 10 reais, 20 reais, 100 reais. A carteira é como se fosse a variável, o local onde armazenamos um valor, no caso, o dinheiro.

Variáveis são muito úteis para tornar nossos cálculos mais automáticos. Sempre que precisarmos utilizar um mesmo valor diversas vezes ao longo do código, podemos utilizar variáveis pro caso da gente precisar alterar estes valores de uma vez. Com variáveis, alteramos uma vez só e o código ficará muito mais otimizado.

2. Declarando uma variável

Declaramos uma variável dentro do SQL conforme a estrutura abaixo:

```
DECLARE @var TIPO
SET @var = valor
```

Ou seja, toda declaração começa com um DECLARE, seguido do nome da variável (iniciando com o @), um TIPO (que pode ser INT, FLOAT, VARCHAR ou DATETIME, e na linha seguinte atribuímos um valor a essa variável.

Exemplo:

```
DECLARE @quantidade INT
SET @quantidade = 100
```

DECLARE e SET

3. Declarando mais de uma variável

Também é possível declarar mais de uma variável por vez:

```
DECLARE @var1 TIPO1, @var2 TIPO2
SET @var1 = valor1
SET @var2 = valor2
```

Exemplo:

```
DECLARE @quantidade INT, @preco FLOAT, @faturamento FLOAT
SET @quantidade = 100
SET @preco = 8.99
SET @faturamento = @quantidade * @preco
```

DECLARE e SET

4. Exemplos

Ao lado, temos alguns exemplos de variáveis. Tente exercitar!

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...

```sql
-- Exemplo 1: Declare uma variável chamada 'idade' e armazene o valor 30

DECLARE @idade INT
SET @idade = 30

SELECT @idade

-- Exemplo 2: Declare uma variável chamada 'preco' e armazene o valor 10.89

DECLARE @preco FLOAT
SET @preco = 10.89
SELECT @preco

DECLARE @preco2 FLOAT
SET @preco2 = 10.89
SELECT @preco2

-- Exemplo 3: Declare uma variável chamada 'nome' e armazene o valor 'Mateus'

DECLARE @nome VARCHAR(50)
SET @nome = 'Mateus'
SELECT @nome

-- Exemplo 4: Declare uma variável chamada 'data' e armazene uma data.

DECLARE @data DATETIME
SET @data = '2021-10-28'
SELECT @data
```

Utilizando uma variável em uma consulta (Exemplo 1)

É possível utilizar variáveis dentro de consultas, como mostrado no exemplo ao lado.

Criamos uma variável **@varDesconto** para armazenar um determinado valor e usamos em nossa consulta para calcular o preço com desconto em cima dos produtos.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQ

```
-- Aplique um desconto de 10% em todos os preços dos produtos.
-- Sua consulta final deve conter as colunas ProductKey, ProductName,
-- UnitPrice e Preço com Desconto.

DECLARE @varDesconto FLOAT
SET @varDesconto = 0.1

SELECT
    ProductKey AS 'ID',
    ProductName AS 'Nome do Produto',
    UnitPrice AS 'Preço',
    UnitPrice * (1 - @varDesconto) AS 'Preço com Desconto'
FROM
    DimProduct
```

100 %

Resultados | Mensagens

	ID	Nome do Produto	Preço	Preço com Desconto
1	1	Contoso 512MB MP3 Player E51 Silver	12,99	11,691
2	2	Contoso 512MB MP3 Player E51 Blue	12,99	11,691
3	3	Contoso 1G MP3 Player E100 White	14,52	13,068
4	4	Contoso 2G MP3 Player E200 Silver	21,57	19,413
5	5	Contoso 2G MP3 Player E200 Red	21,57	19,413
6	6	Contoso 2G MP3 Player E200 Black	21,57	19,413
7	7	Contoso 2G MP3 Player E200 Blue	21,57	19,413
8	8	Contoso 4G MP3 Player E400 Silver	59,99	53,991
9	9	Contoso 4G MP3 Player E400 Black	59,99	53,991
10	10	Contoso 4G MP3 Player E400 Green	59,99	53,991
11	11	Contoso 4G MP3 Player E400 Orange	59,99	53,991
12	12	Contoso 4GB Flash MP3 Player E401 Blue	77,68	69,912

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0

Utilizando uma variável em uma consulta (Exemplo 2)

Ao lado, temos mais um exemplo de como usar variáveis dentro de consultas.

Dessa vez, criamos uma variável **@varData** para tornar o filtro das consultas mais automático.

Assim, sempre que a gente quiser analisar uma nova data, alteramos o valor da variável uma única vez, e todos os lugares que utilizarem esse valor dentro do código serão atualizados automaticamente.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))*

```
-- Crie uma variável de data para otimizar a consulta abaixo.
DECLARE @varData DATETIME
SET @varData = '01/01/1980'

SELECT
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    BirthDate AS 'Nascimento',
    'Cliente' AS 'Tipo'
FROM
    DimCustomer
WHERE BirthDate >= @varData

UNION

SELECT
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    BirthDate AS 'Nascimento',
    'Funcionário' AS 'Tipo'
FROM
    DimEmployee
WHERE BirthDate >= @varData
ORDER BY Nascimento
```

83 %

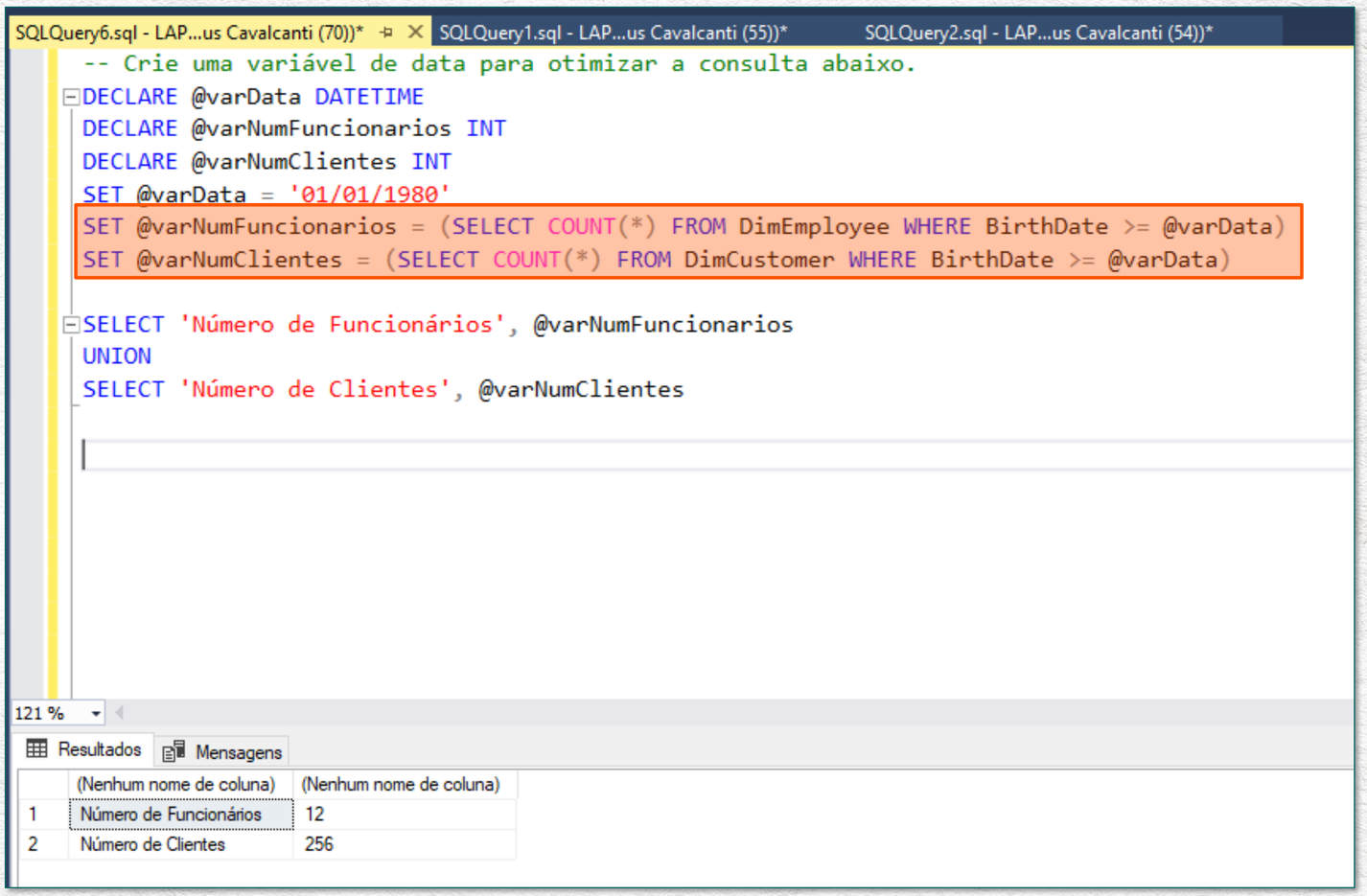
Resultados | Mensagens

	Nome	Sobrenome	Nascimento	Tipo
1	Angela	Kelly	1980-01-06	Cliente
2	Arthur	Fernandez	1980-01-06	Cliente
3	Caroline	Russell	1980-01-06	Cliente
4	Christine	Luo	1980-01-06	Cliente
5	Terrence	Xu	1980-01-06	Cliente
6	Ian	Price	1980-01-08	Cliente
7	Manuel	Fernandez	1980-01-11	Cliente
8	Toni	Sai	1980-01-11	Cliente

Armazenando o resultado de uma consulta em uma variável

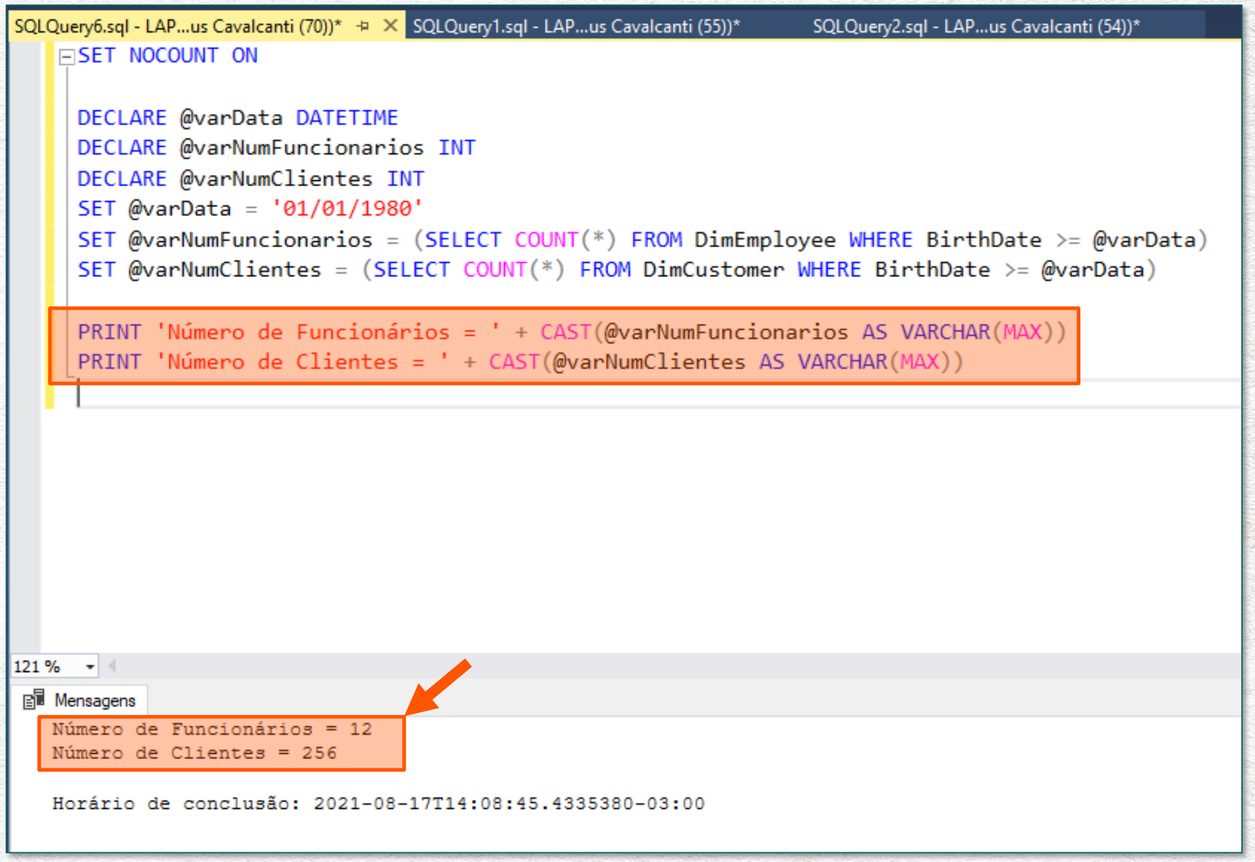
Podemos também armazenar o resultado de uma consulta dentro de uma variável.

No exemplo ao lado, armazenamos dentro de duas variáveis os resultados de dois SELECTs: um que calcula a quantidade de funcionários em uma empresa, e o outro que calcula a quantidade de clientes, ambos considerando uma data de nascimento.



PRINT

Um outro comando útil é o **PRINT**. Podemos criar um código para printar um determinado valor na caixa de mensagens, indicada na imagem abaixo:



Armazenando um registro de uma consulta em uma variável

Variáveis também nos permitem armazenar determinados registros dentro de uma tabela.

Como assim registros?

Registros são as informações que temos dentro de uma tabela. Por meio de um filtro ou top(n) é possível acessar esses registros.

E pra fechar, conseguimos armazenar esses registros dentro de variáveis.

No exemplo ao lado, armazenamos dentro de duas variáveis o nome e o total vendido para o produto mais vendido. Utilizamos para isso o TOPN e o ORDER BY.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- Qual é o nome do produto que teve a maior quantidade de vendas
SELECT TOP(100) * FROM FactSales

DECLARE @varProdutoMaisVendido INT
DECLARE @varTotalMaisVendido FLOAT

SELECT TOP(1)
    @varProdutoMaisVendido = ProductKey,
    @varTotalMaisVendido = SalesAmount
FROM
    FactSales
ORDER BY SalesAmount DESC

SELECT @varProdutoMaisVendido AS 'Produto', @varTotalMaisVendido AS 'Total'

SELECT
    ProductName
FROM
    DimProduct
WHERE ProductKey = @varProdutoMaisVendido

SELECT * FROM DimProduct
```

100 %

Resultados | Mensagens

	SalesKey	DateKey	channelKey	StoreKey	ProductKey	PromotionKey	CurrencyKey	UnitCost	UnitPrice	SalesQuantity	ReturnQuantity	ReturnAmount	Dis
1	1	2007-01-02 00:00:00.000	1	209	956	10	1	91,05	198,00	8	0	0,00	1
2	2	2007-02-12 00:00:00.000	4	308	766	2	1	10,15	19,90	4	0	0,00	1
3	3	2008-01-24 00:00:00.000	1	156	1175	11	1	209,03	410,00	9	0	0,00	3
4	4	2008-01-13 00:00:00.000	2	306	1429	10	1	132,90	289,00	8	0	0,00	1
5	5	2008-01-22 00:00:00.000	2	306	1133	10	1	144,52	436,20	24	0	0,00	3
6	6	2007-07-02 00:00:00.000	3	200	2365	3	1	183,94	399,99	36	0	0,00	10

Índice Geral

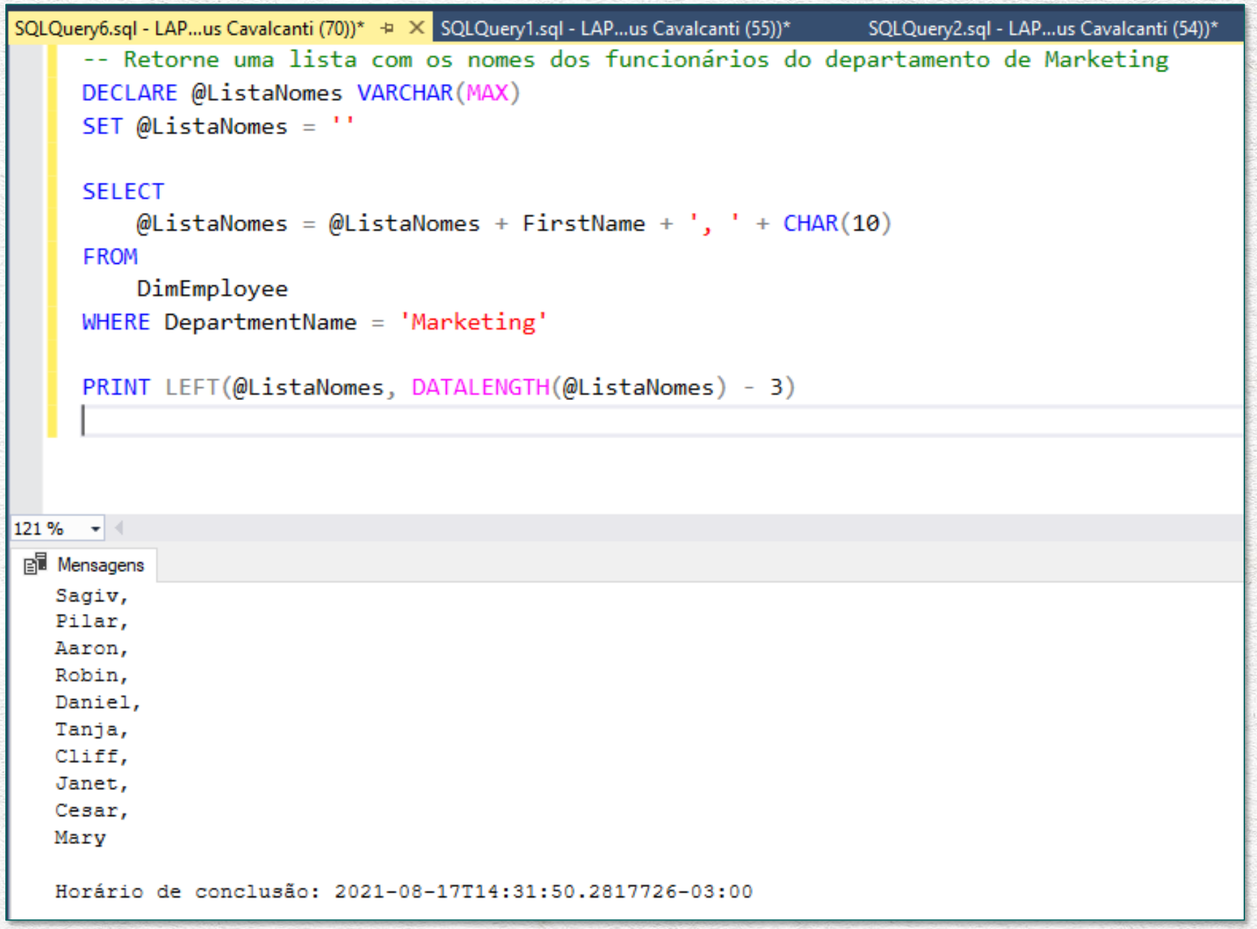
Índice deste Módulo

Acumulando valores dentro de uma variável

Variáveis também podem acumular mais de um valor de uma vez.

Observe na imagem ao lado. Temos a variável **@ListaNomes** e queremos armazenar dentro dela todos os funcionários do departamento de Marketing, isso de uma só vez.

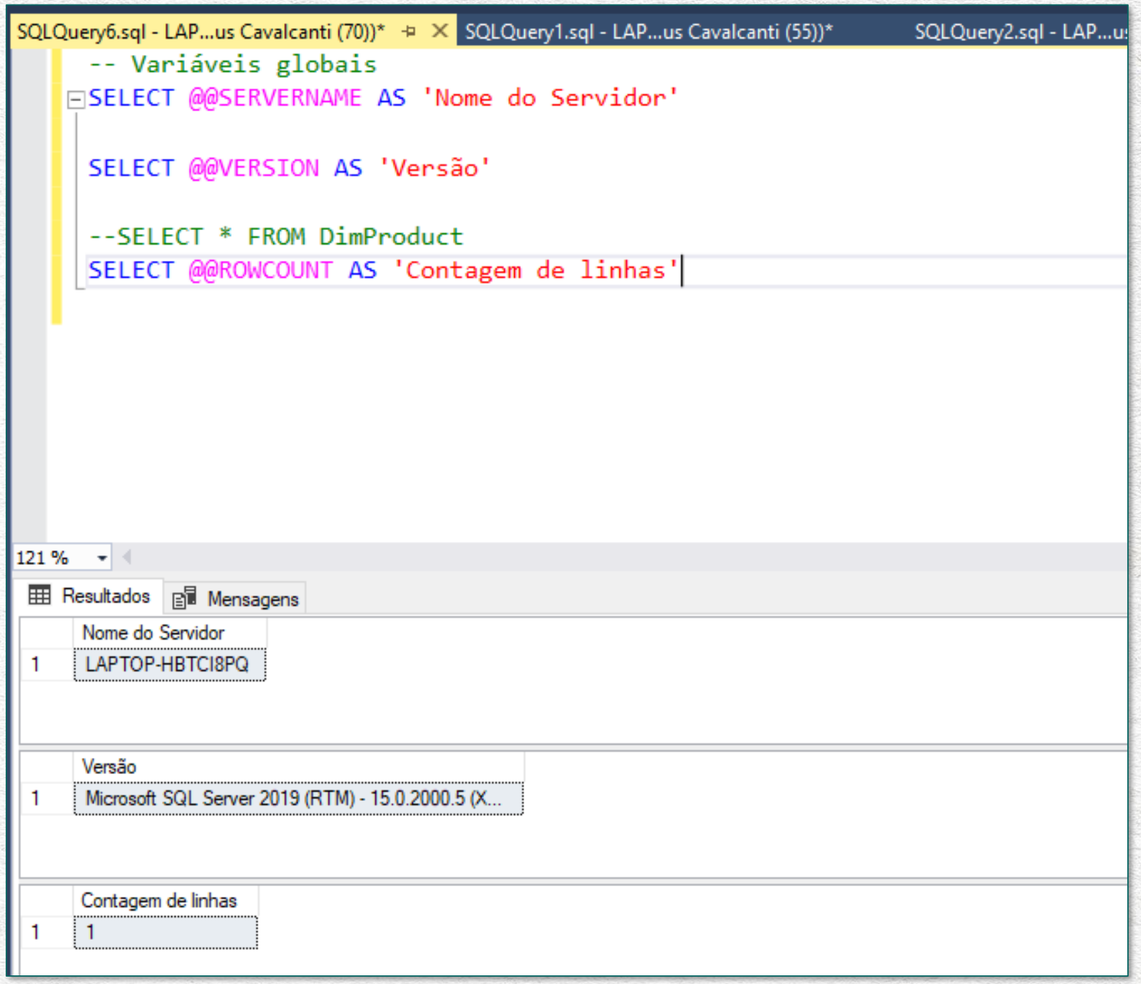
A estrutura para acumular os valores dentro da variável é mostrado ao lado.



Variáveis globais

O SQL possui algumas variáveis padrão, que podem ser acessadas quando utilizamos o @@, como mostrado ao lado.

No código da imagem, conseguimos acessar variáveis que retornam informações como: Nome do Servidor, Versão do Sistema ou até mesmo a contagem de linhas de uma tabela.



EXERCÍCIOS

1

Questão 1

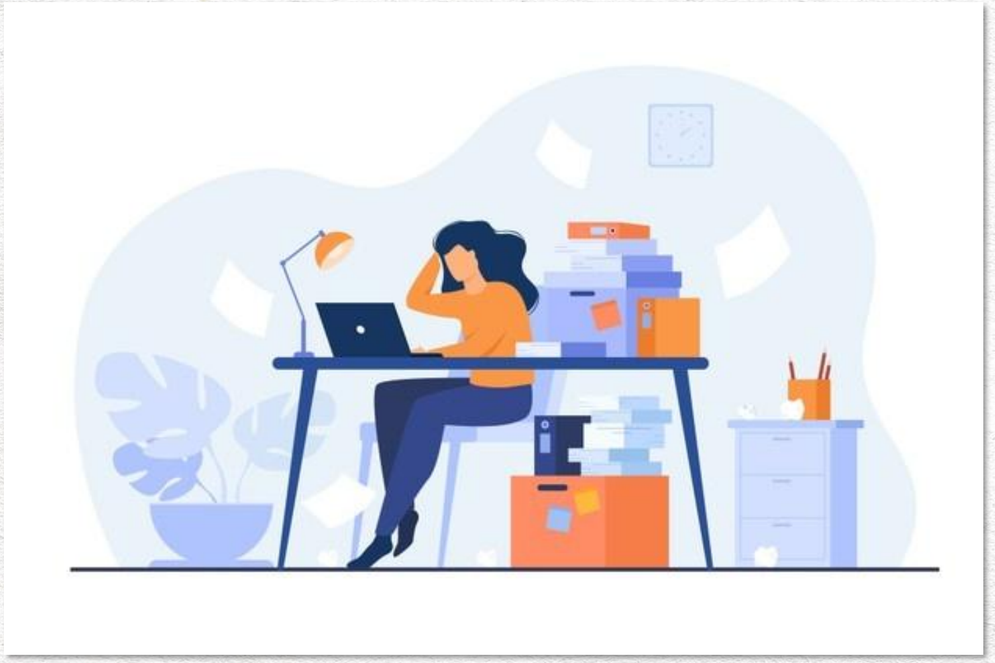
Declare 4 variáveis inteiras. Atribua os seguintes valores a elas:

valor1 = 10
valor2 = 5
valor3 = 34
valor4 = 7

a) Crie uma nova variável para armazenar o resultado da soma entre valor1 e valor2. Chame essa variável de soma.
b) Crie uma nova variável para armazenar o resultado da subtração entre valor3 e valor 4. Chame essa variável de subtracao.
c) Crie uma nova variável para armazenar o resultado da multiplicação entre o valor 1 e o valor4. Chame essa variável de multiplicacao.
d) Crie uma nova variável para armazenar o resultado da divisão do valor3 pelo valor4. Chame essa variável de divisao. Obs: O resultado deverá estar em decimal, e não em inteiro.

Arredonde o resultado da letra d) para 2 casas decimais.

EXERCÍCIOS



2

Questão 2

Para cada declaração das variáveis abaixo, atenção em relação ao tipo de dado que deverá ser especificado.

a. Declare uma variável chamada ‘produto’ e atribua o valor de ‘Celular’.

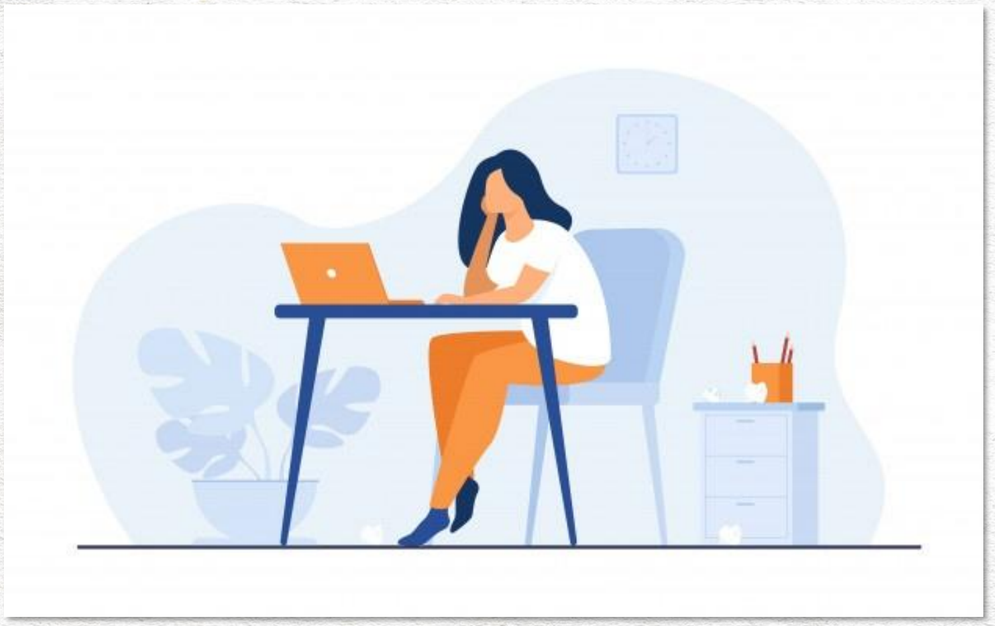
b. Declare uma variável chamada ‘quantidade’ e atribua o valor de 12.

c. Declare uma variável chamada ‘preco’ e atribua o valor 9.99.

d. Declare uma variável chamada ‘faturamento’ e atribua o resultado da multiplicação entre ‘quantidade’ e ‘preco’.

e. Visualize o resultado dessas 4 variáveis em uma única consulta, por meio do SELECT.

EXERCÍCIOS



3

Questão 3

Você é responsável por gerenciar um banco de dados onde são recebidos dados externos de usuários. Em resumo, esses dados são:

- Nome do usuário
- Data de nascimento
- Quantidade de pets que aquele usuário possui

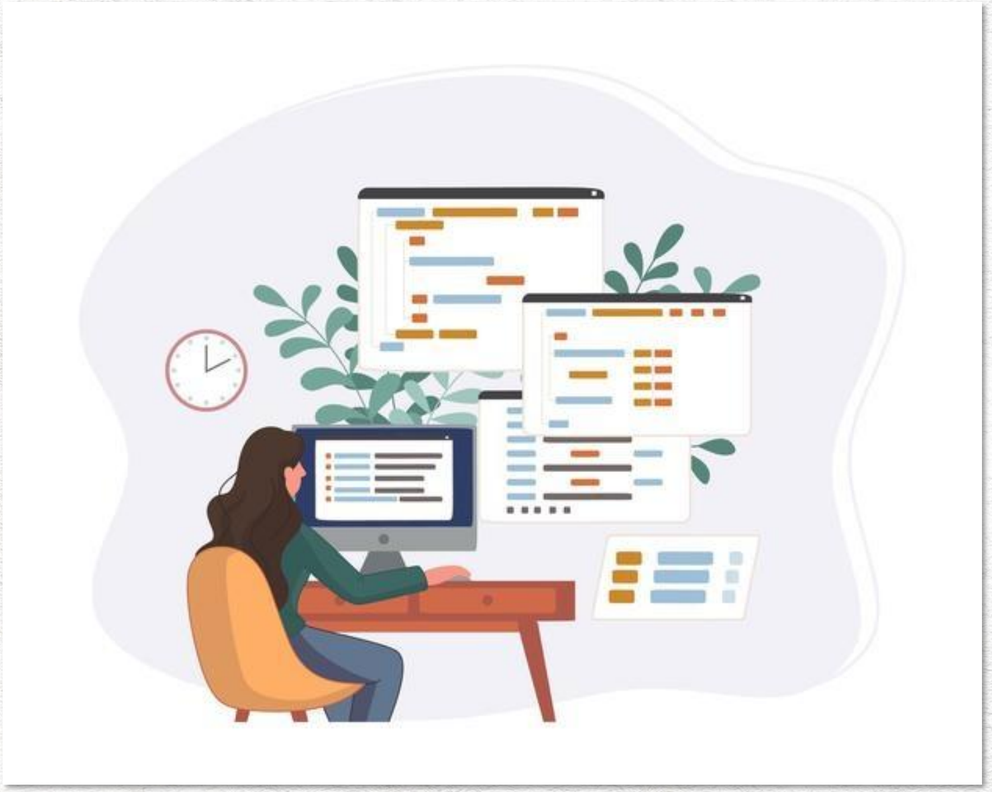
Você precisará criar um código em SQL capaz de juntar as informações fornecidas por este usuário. Para simular estes dados, crie 3 variáveis, chamadas: nome, data_nascimento e num_pets. Você deverá armazenar os valores 'André', '10/02/1998' e 2, respectivamente.

O resultado final a ser alcançado é mostrado no print abaixo:

(Nenhum nome de coluna)
Meu nome é André, nasci em 10/02/1998 e tenho 2 pets.

Dica: você precisará utilizar as funções CAST e FORMAT para chegar no resultado.

EXERCÍCIOS



4

Questão 4

Você acabou de ser promovido e o seu papel será realizar um controle de qualidade sobre as lojas da empresa.

A primeira informação que é passada a você é que o ano de 2008 foi bem complicado para a empresa, pois foi quando duas das principais lojas fecharam. O seu primeiro desafio é descobrir o nome dessas lojas que fecharam no ano de 2008, para que você possa entender o motivo e mapear planos de ação para evitar que outras lojas importantes tomem o mesmo caminho.

O seu resultado deverá estar estruturado em uma frase, com a seguinte estrutura:

'As lojas fechadas no ano de 2008 foram: ' + nome_das_lojas

Obs: utilize o comando PRINT (e não o SELECT!) para mostrar o resultado.

5

Questão 5

Você precisa criar uma consulta para mostrar a lista de produtos da tabela DimProduct para uma subcategoria específica: 'Lamps'.

Utilize o conceito de variáveis para chegar neste resultado.

GABARITOS

1

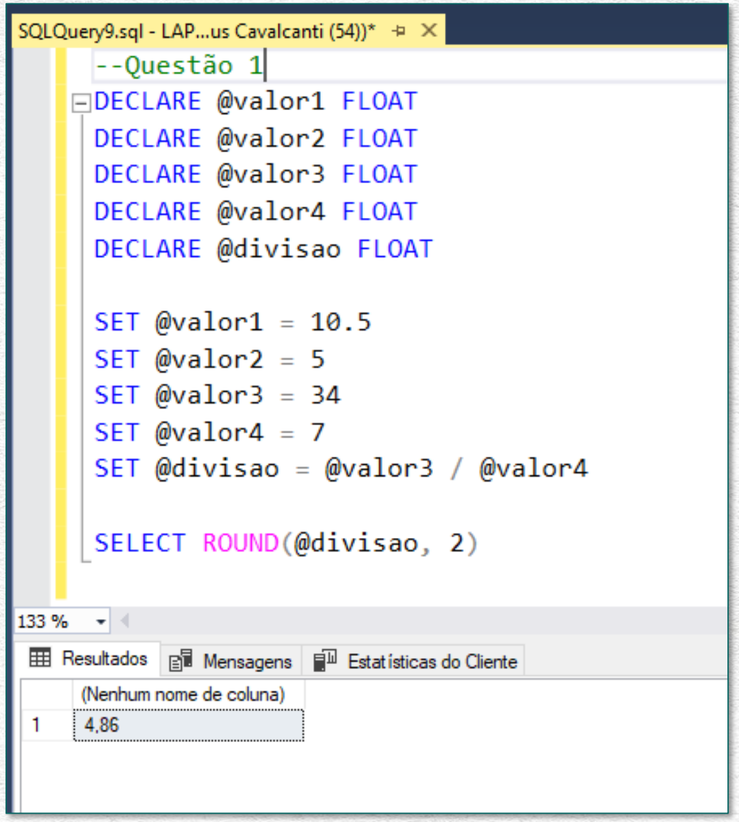
Questão 1

Declare 4 variáveis inteiras. Atribua os seguintes valores a elas:

valor1 = 10
valor2 = 5
valor3 = 34
valor4 = 7

a) Crie uma nova variável para armazenar o resultado da soma entre valor1 e valor2. Chame essa variável de soma.
b) Crie uma nova variável para armazenar o resultado da subtração entre valor3 e valor 4. Chame essa variável de subtracao.
c) Crie uma nova variável para armazenar o resultado da multiplicação entre o valor 1 e o valor4. Chame essa variável de multiplicacao.
d) Crie uma nova variável para armazenar o resultado da divisão do valor3 pelo valor4. Chame essa variável de divisao. Obs: O resultado deverá estar em decimal, e não em inteiro.

Arredonde o resultado da letra d) para 2 casas decimais.



GABARITOS

2

Questão 2

Para cada declaração das variáveis abaixo, atenção em relação ao tipo de dado que deverá ser especificado.

a. Declare uma variável chamada ‘produto’ e atribua o valor de ‘Celular’.

b. Declare uma variável chamada ‘quantidade’ e atribua o valor de 12.

c. Declare uma variável chamada ‘preco’ e atribua o valor 9.99.

d. Declare uma variável chamada ‘faturamento’ e atribua o resultado da multiplicação entre ‘quantidade’ e ‘preco’.

e. Visualize o resultado dessas 4 variáveis em uma única consulta, por meio do SELECT.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 2

DECLARE @produto VARCHAR(30) = 'Celular'
DECLARE @quantidade INT = 12
DECLARE @preco FLOAT = 9.99
DECLARE @faturamento FLOAT = @quantidade * @preco

SELECT
    @produto AS 'Produto',
    @quantidade AS 'Quantidade',
    @preco AS 'Preço',
    @faturamento AS 'Faturamento'
```

133 %

Resultados | Mensagens | Estatísticas do Cliente

	Produto	Quantidade	Preço	Faturamento
1	Celular	12	9,99	119,88

GABARITOS

3

Questão 3

Você é responsável por gerenciar um banco de dados onde são recebidos dados externos de usuários. Em resumo, esses dados são:

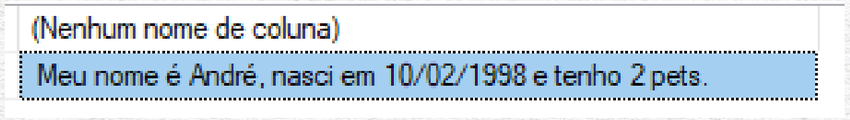
- Nome do usuário

- Data de nascimento

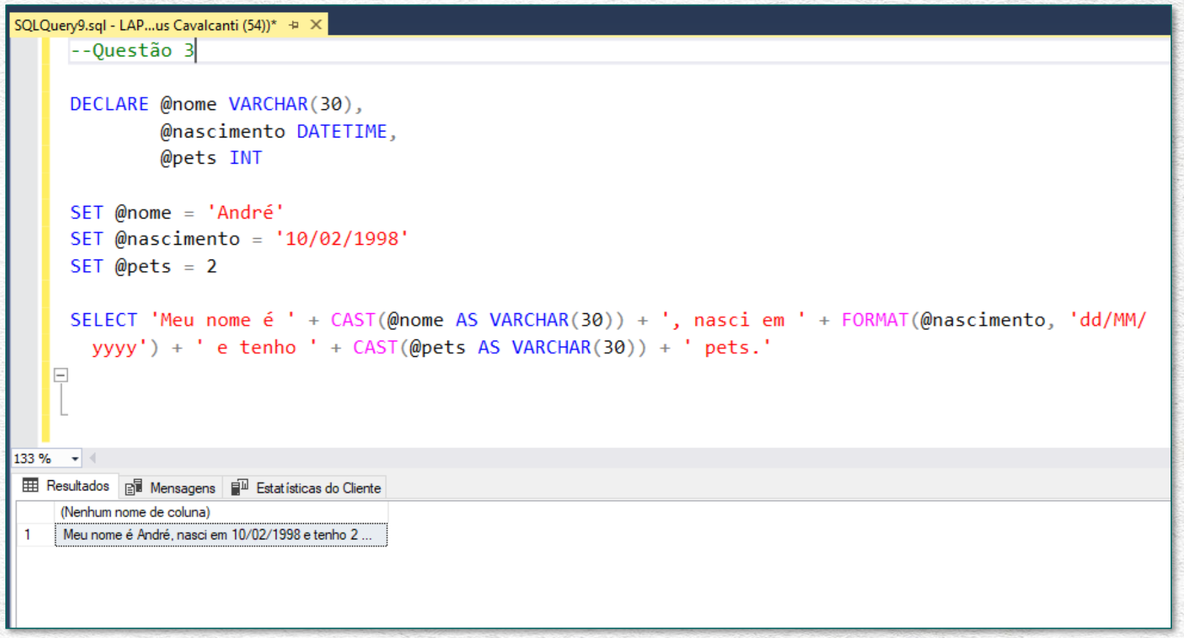
- Quantidade de pets que aquele usuário possui

Você precisará criar um código em SQL capaz de juntar as informações fornecidas por este usuário. Para simular estes dados, crie 3 variáveis, chamadas: nome, data_nascimento e num_pets. Você deverá armazenar os valores 'André', '10/02/1998' e 2, respectivamente.

O resultado final a ser alcançado é mostrado no print abaixo:



Dica: você precisará utilizar as funções CAST e FORMAT para chegar no resultado.



GABARITOS

4

Questão 4

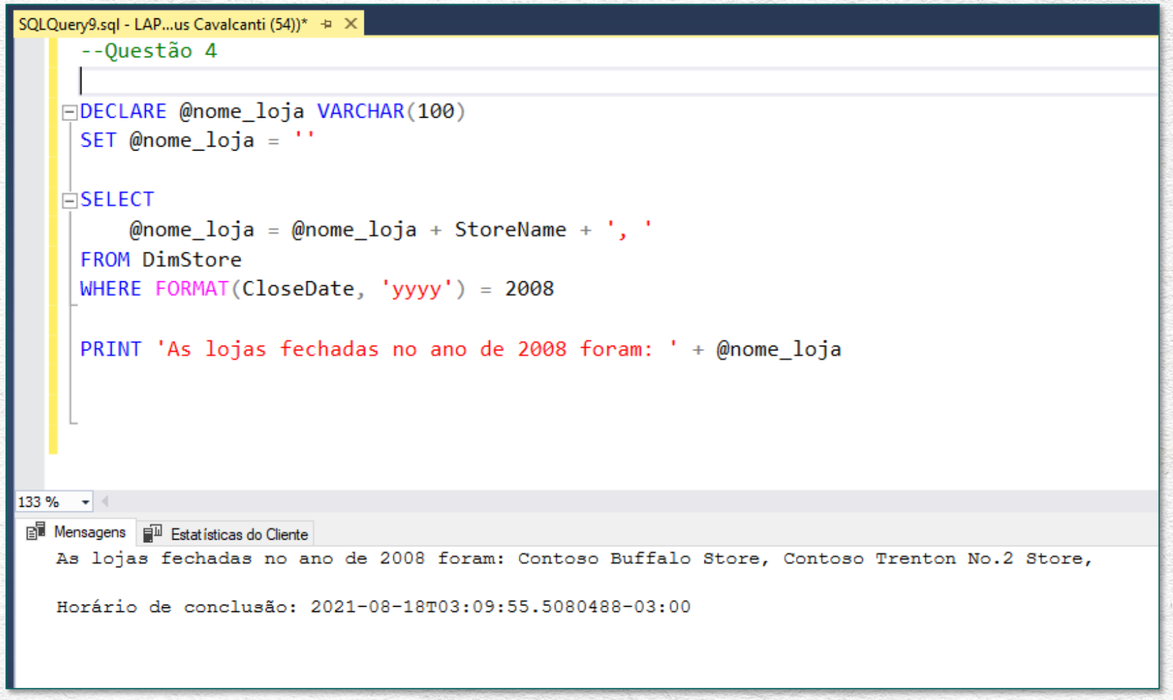
Você acabou de ser promovido e o seu papel será realizar um controle de qualidade sobre as lojas da empresa.

A primeira informação que é passada a você é que o ano de 2008 foi bem complicado para a empresa, pois foi quando duas das principais lojas fecharam. O seu primeiro desafio é descobrir o nome dessas lojas que fecharam no ano de 2008, para que você possa entender o motivo e mapear planos de ação para evitar que outras lojas importantes tomem o mesmo caminho.

O seu resultado deverá estar estruturado em uma frase, com a seguinte estrutura:

'As lojas fechadas no ano de 2008 foram: ' + nome_das_lojas

Obs: utilize o comando PRINT (e não o SELECT!) para mostrar o resultado.



GABARITOS

5

Questão 5

Você precisa criar uma consulta para mostrar a lista de produtos da tabela DimProduct para uma subcategoria específica: ‘Lamps’.

Utilize o conceito de variáveis para chegar neste resultado.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--Questão 5

SELECT * FROM DimProductSubcategory

DECLARE @id_subcategoria INT
DECLARE @subcategoria VARCHAR(100)

SET @subcategoria = 'Radio'
SET @id_subcategoria = (SELECT ProductSubcategoryKey FROM DimProductSubcategory WHERE ProductSubcategoryName =
  @subcategoria)

SELECT
    *
FROM
    DimProduct
WHERE ProductSubcategoryKey = @id_subcategoria
```

110 %

Resultados | Mensagens | Estatísticas do Cliente

	ProductSubcategoryKey	ProductSubcategoryLabel	ProductSubcategoryName	ProductSubcategoryDescription	ProductCategoryKey	ETLLoadID	LoadDate	UpdateD
1	1	0101	MP4&MP3	MP4&MP3	1	1	2009-07-07 00:00:00.000	2009-07
2	2	0102	Recorder	Recorder	1	1	2009-07-07 00:00:00.000	2009-07
3	3	0103	Radio	Radio	1	1	2009-07-07 00:00:00.000	2009-07
4	4	0104	Recording Pen	Recording Pen	1	1	2009-07-07 00:00:00.000	2009-07
5	5	0105	Headphones	Headphones	1	1	2009-07-07 00:00:00.000	2009-07
6	6	0106	Bluetooth Headphones	Bluetooth Headphones	1	1	2009-07-07 00:00:00.000	2009-07
7	7	0107	Speakers	Speakers	1	1	2009-07-07 00:00:00.000	2009-07
8	8	0108	Audio Accessories	Audio Accessories	1	1	2009-07-07 00:00:00.000	2009-07

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:00 | 44 linhas

MÓDULO 10

SQL

Manipulando Strings e Datas no SQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 10 MANIPULANDO STRINGS E DATAS

01	LEN e DATALENGTH	246
02	CONCAT	247
03	LEFT e RIGHT	248
04	REPLACE	249
05	TRANSLATE e STUFF	250
06	UPPER e LOWER	252
07	FORMAT	253
08	CHARINDEX e SUBSTRING	254
09	TRIM, LTRIM e RTRIM	255
10	DAY, MONTH e YEAR	256
11	DATEFROMPARTS	257
12	GETDATE e SYSDATETIME	258
13	DATENAME e DATEPART	259
14	DATEADD e DATEDIFF	260
15	EXERCÍCIOS	261
16	GABARITOS	266

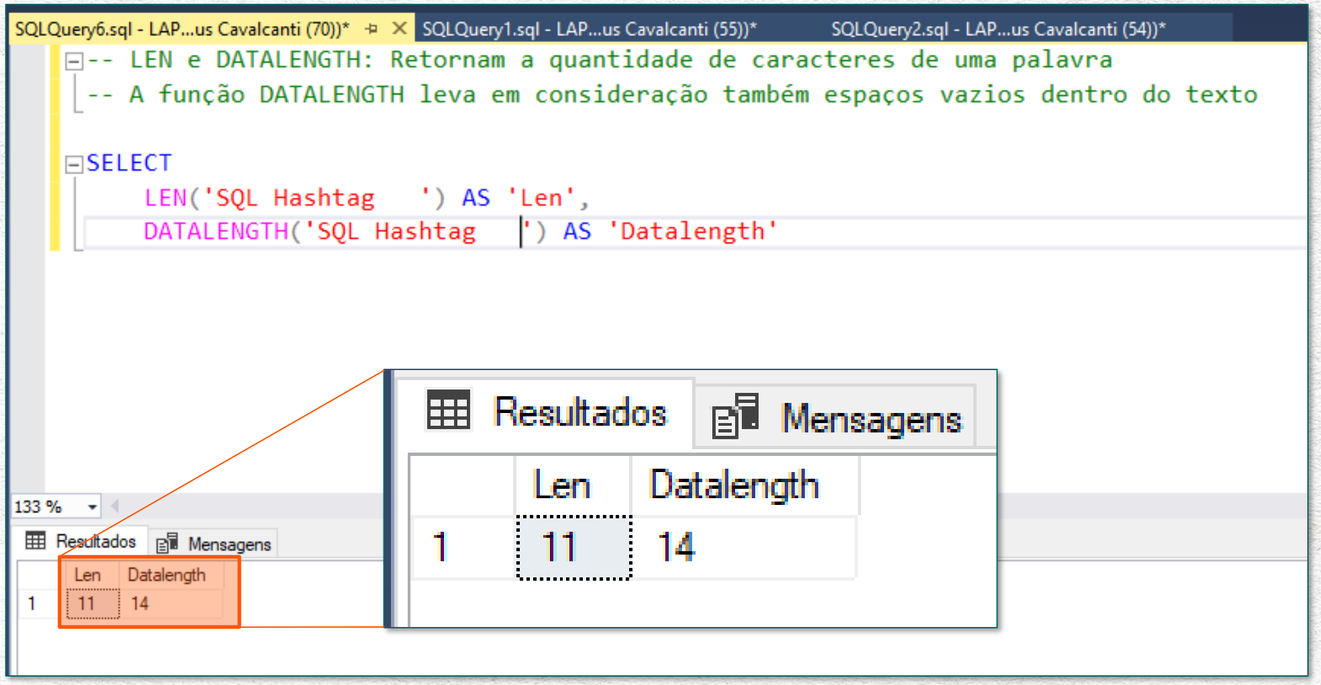
LEN e DATALENGTH

As funções **LEN** e **DATALENGTH** possuem resultados semelhantes: ambos retornam a quantidade de caracteres de um determinado texto.

Ao lado, é possível visualizar um exemplo.

A grande diferença entre os dois é sutil: o LEN retorna a quantidade de caracteres de um texto, porém ignora quando aparecem espaços a mais.

Já o DATALENGTH retorna a quantidade de caracteres, incluindo espaços a mais dentro do texto.



CONCAT

A função **CONCAT** permite juntar dois ou mais textos em um só.

Observe a imagem ao lado. A partir das colunas de Nome e Sobrenome da nossa tabela DimCustomer, podemos criar uma nova coluna, chamada Nome Completo, que é a junção dos dois textos: FirstName e LastName.

Para isso, usamos a função CONTAT.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- CONCAT ---> Permite juntar mais de um texto em uma única palavra
-- Faça uma consulta à tabela DimCustomer onde seja possível
-- visualizar o nome completo de cada cliente

SELECT
    FirstName AS 'Nome',
    LastName AS 'Sobrenome',
    EmailAddress AS 'E-mail',
    CONCAT(FirstName, ' ', LastName) AS 'Nome Completo'
FROM
    DimCustomer
```

133 %

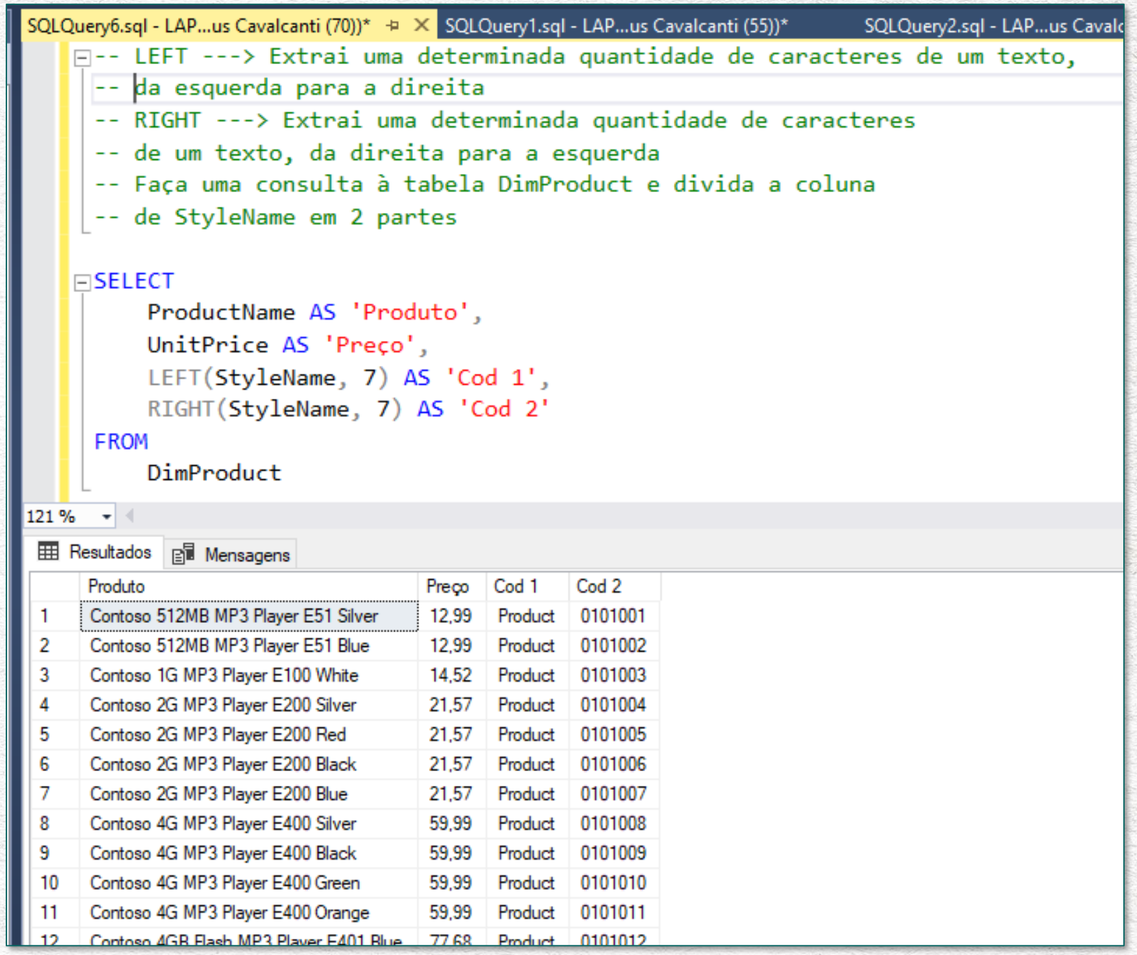
Resultados | Mensagens

	Nome	Sobrenome	E-mail	Nome Completo
1	Jon	Yang	jon24@adventure-works.com	Jon Yang
2	Eugene	Huang	eugene10@adventure-works.com	Eugene Huang
3	Ruben	Torres	ruben35@adventure-works.com	Ruben Torres
4	Christy	Zhu	christy12@adventure-works.com	Christy Zhu
5	Elizabeth	Johnson	elizabeth5@adventure-works.com	Elizabeth Johnson
6	Julio	Ruiz	julio1@adventure-works.com	Julio Ruiz
7	Janet	Alvarez	janet9@adventure-works.com	Janet Alvarez
8	Marco	Mehta	marco14@adventure-works.com	Marco Mehta
9	Rob	Verhoff	rob4@adventure-works.com	Rob Verhoff
10	Shannon	Carlson	shannon38@adventure-works.com	Shannon Carlson
11	Jacquelyn	Suarez	jacquelyn20@adventure-works.com	Jacquelyn Suarez
12	Curtis	Lu	curtis9@adventure-works.com	Curtis Lu

LEFT e RIGHT

As funções **LEFT** e **RIGHT** permitem extrair uma parte de um texto, sempre começando da esquerda pra direita, ou da direita pra esquerda, respectivamente.

Na imagem ao lado, temos um exemplo prático de como aplicar essas funções.



	Produto	Preço	Cod 1	Cod 2
1	Contoso 512MB MP3 Player E51 Silver	12,99	Product	0101001
2	Contoso 512MB MP3 Player E51 Blue	12,99	Product	0101002
3	Contoso 1G MP3 Player E100 White	14,52	Product	0101003
4	Contoso 2G MP3 Player E200 Silver	21,57	Product	0101004
5	Contoso 2G MP3 Player E200 Red	21,57	Product	0101005
6	Contoso 2G MP3 Player E200 Black	21,57	Product	0101006
7	Contoso 2G MP3 Player E200 Blue	21,57	Product	0101007
8	Contoso 4G MP3 Player E400 Silver	59,99	Product	0101008
9	Contoso 4G MP3 Player E400 Black	59,99	Product	0101009
10	Contoso 4G MP3 Player E400 Green	59,99	Product	0101010
11	Contoso 4G MP3 Player E400 Orange	59,99	Product	0101011
12	Contoso 4GB Flash MP3 Player E401 Blue	77,68	Product	0101012

REPLACE

A função **REPLACE** permite que a gente substitua um determinado texto por outro texto.

Na imagem ao lado, vemos uma aplicação prática do REPLACE.

Nela, substituímos os textos M e F por Masculino e Feminino, respectivamente.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- REPLACE ---> Substitui um determinado texto por outro texto
-- Crie uma consulta a partir de DimCustomer onde você retorna
-- o Nome Completo dos Clientes, a coluna de Sexo (Abrev) e uma
-- outra coluna de Sexo substituindo M por Masculino e F por Feminino

SELECT
    CONCAT(FirstName, ' ', LastName) AS 'Nome Completo',
    Gender AS 'Sexo (Abrev)',
    REPLACE(REPLACE(Gender, 'M', 'Masculino'), 'F', 'Feminino') AS 'Sexo (Extenso)'
FROM
    DimCustomer
```

121 %

Resultados | Mensagens

	Nome Completo	Sexo (Abrev)	Sexo (Extenso)
1	Jon Yang	M	Masculino
2	Eugene Huang	M	Masculino
3	Ruben Torres	M	Masculino
4	Christy Zhu	F	Feminino
5	Elizabeth Johnson	F	Feminino
6	Julio Ruiz	M	Masculino
7	Janet Alvarez	F	Feminino
8	Marco Mehta	M	Masculino
9	Rob Verhoff	F	Feminino
10	Shannon Carlson	M	Masculino
11	Jacquelyn Suarez	F	Feminino
12	Curtis Lu	M	Masculino

TRANSLATE e STUFF

Existem outras duas funções usadas para substituir textos: **TRANSLATE** e **STUFF**.

A função **TRANSLATE** permite que a gente informa uma sequência de caracteres antigos que desejamos substituir, além da sequência de caracteres novos que serão usados para substituir os caracteres antigos, seguindo exatamente a mesma sequência.

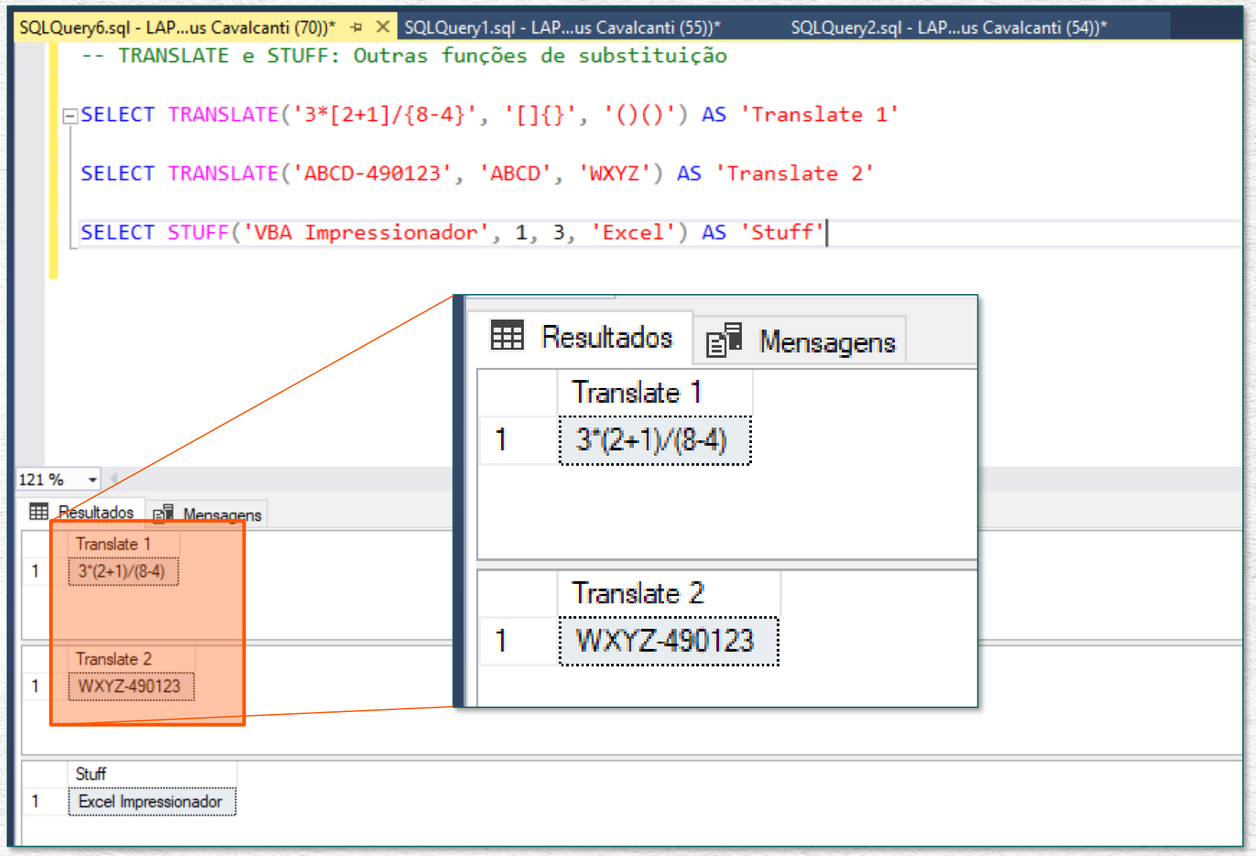
Observe os exemplos ao lado. Essa função pede 3 argumentos:

1. **Texto**
2. **Sequência antiga de caracteres**
3. **Sequência nova de caracteres**.

Na primeira aplicação do **TRANSLATE** (Translate 1), todos os caracteres [são substituídos por (, todos os caracteres] são substituídos pelo), e assim vai. Exatamente nessa ordem.

Já no exemplo Translate 2, sempre que a letra A aparecer, vamos substituir por W. Sempre que aparecer a letra B, vamos substituir por X, e assim vai.

Em resumo, essa substituição é feita caractere por caractere, exatamente na ordem especificada.



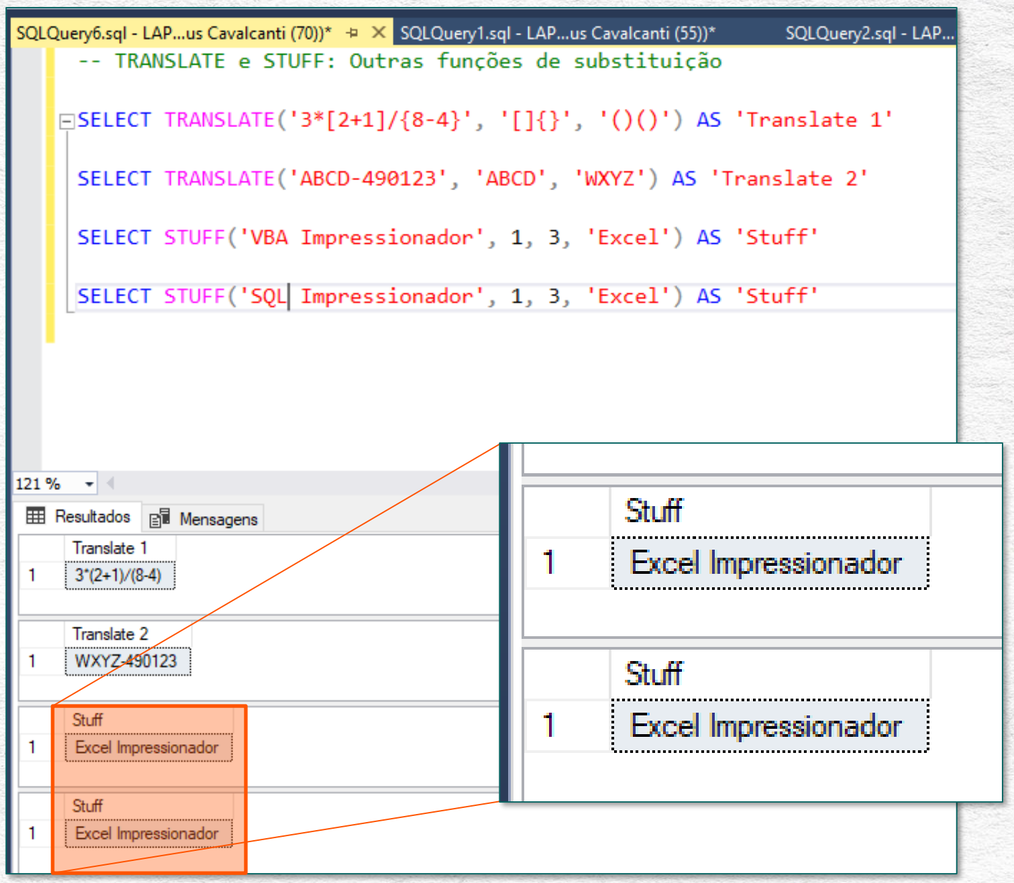
TRANSLATE e STUFF

A função **STUFF** permite que a gente substitua uma sequência de caracteres com um tamanho específico, por um outro texto.

Essa função pede 4 argumentos:

1. Texto
2. Posição inicial onde começa o texto antigo
3. Quantidade de caracteres do texto antigo
4. Novo texto

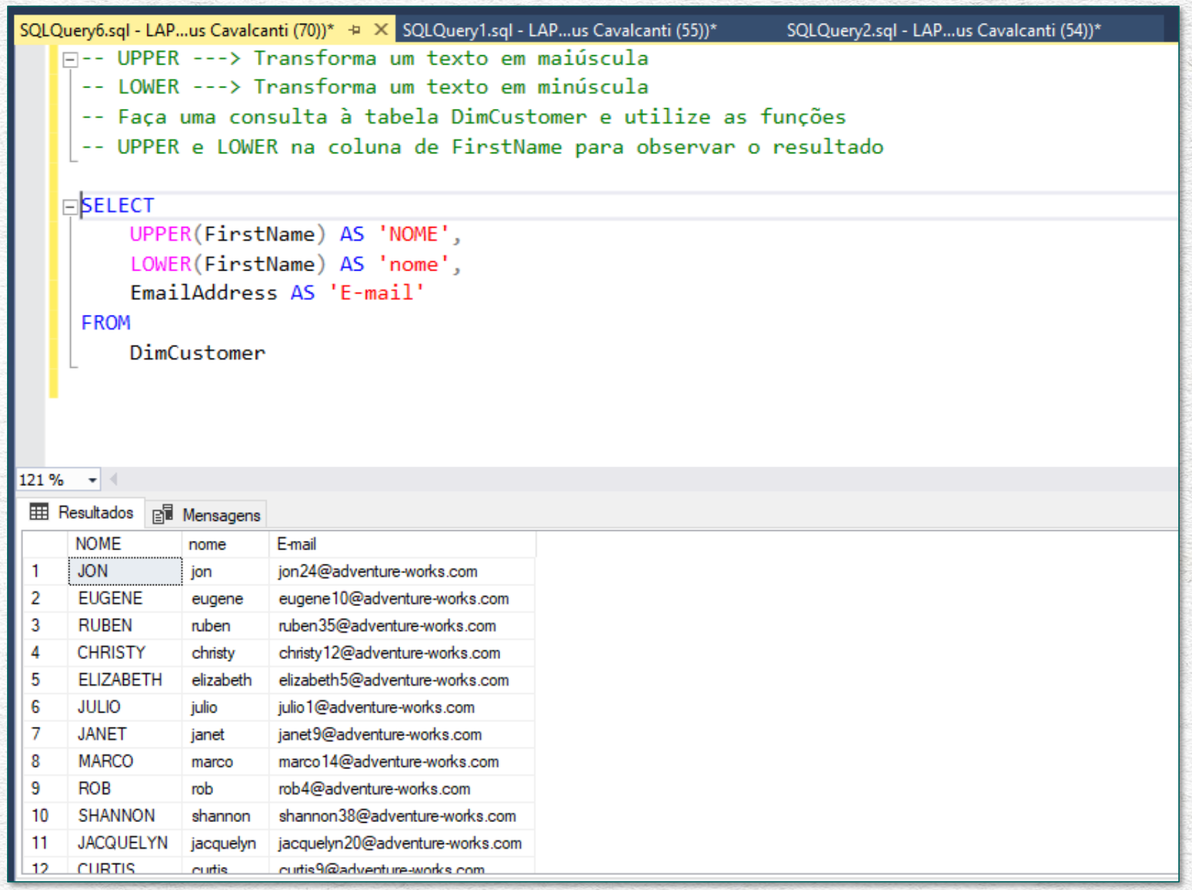
No exemplo ao lado, temos duas situações: VBA Impressionador e SQL Impressionador. Ambos tem 3 caracteres inicias (referentes ao nome do curso) e podemos utilizar esse padrão, junto com a função STUFF, para substituir sempre os 3 primeiros caracteres (independente do que esteja escrito nesses 3 primeiros caracteres) pelo texto 'Excel'.



UPPER e LOWER

As funções **UPPER** e **LOWER** são bastante intuitivas e nos permitem ajustar as palavras para que fiquem em letras maiúsculas e minúsculas.

Ao lado, temos uma aplicação dessas funções.



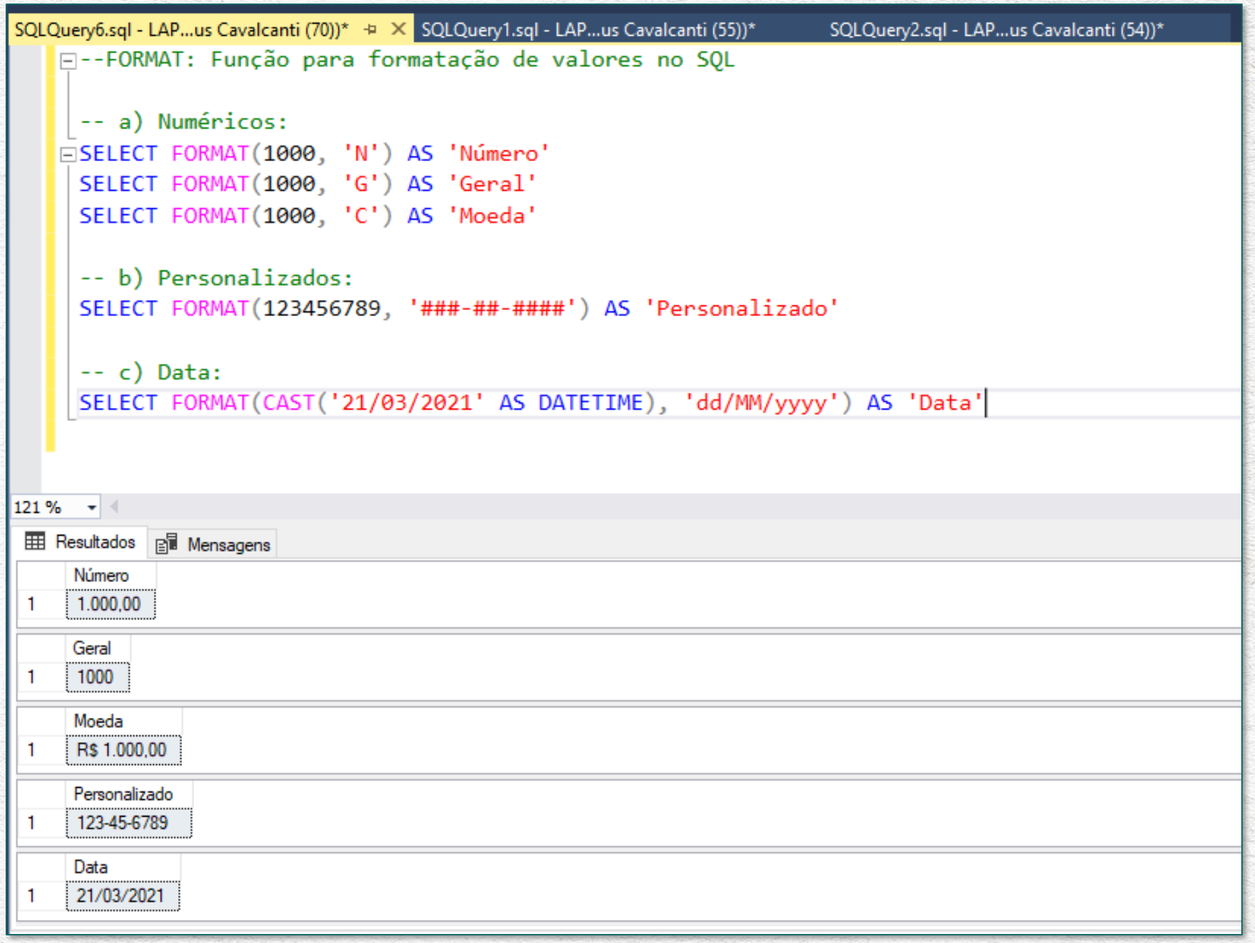
	NOME	nome	E-mail
1	JON	jon	jon24@adventure-works.com
2	EUGENE	eugene	eugene10@adventure-works.com
3	RUBEN	ruben	ruben35@adventure-works.com
4	CHRISTY	christy	christy12@adventure-works.com
5	ELIZABETH	elizabeth	elizabeth5@adventure-works.com
6	JULIO	julio	julio1@adventure-works.com
7	JANET	janet	janet9@adventure-works.com
8	MARCO	marco	marco14@adventure-works.com
9	ROB	rob	rob4@adventure-works.com
10	SHANNON	shannon	shannon38@adventure-works.com
11	JACQUELYN	jacquelyn	jacquelyn20@adventure-works.com
12	CURTIS	curtis	curtis9@adventure-works.com

FORMAT

A função **FORMAT** é uma função muito útil para realizar formatações personalizadas em nossas consultas.

Essa função nos pede 2 argumentos:
1. Valor a ser formatado
2. Código de formatação

As principais formatações são mostradas ao lado.



CHARINDEX e SUBSTRING

As funções **CHARINDEX** e **SUBSTRING** combinadas são muito úteis para manipular textos de forma mais avançada.

A função **CHARINDEX** nos permite descobrir a posição de uma determinada cadeira de caracteres dentro de um texto. Ela pede 2 argumentos:

1. Texto a ser procurado
2. Texto completo

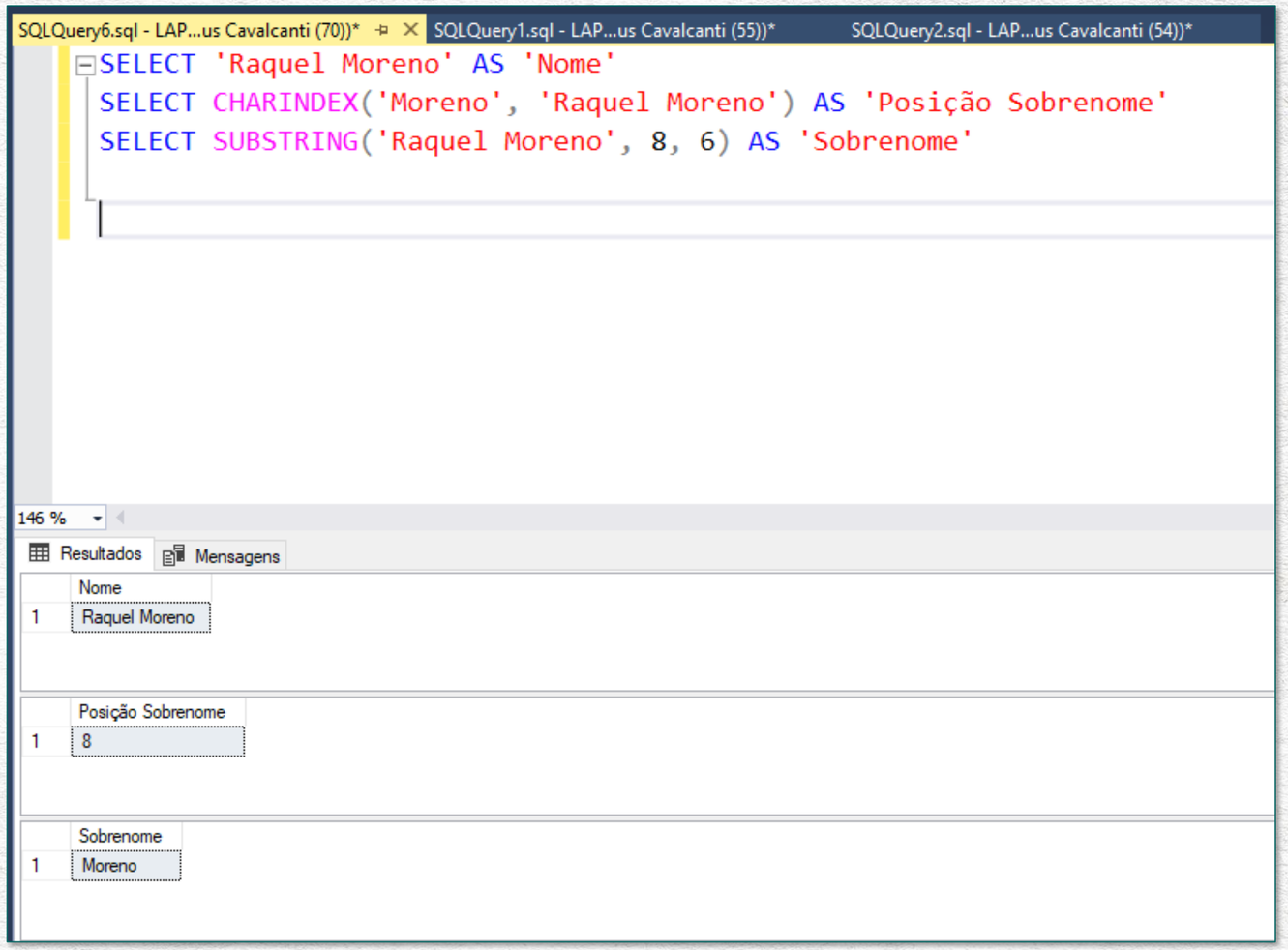
No exemplo ao lado, vemos que o texto 'Moreno' encontra-se na posição 8 do nome 'Raquel Moreno'.

Já a função **SUBSTRING** nos permite retornar uma cadeia de caracteres de dentro de um texto, a partir de uma posição inicial.

Essa função nos pede 3 argumentos:

1. Texto
2. Posição inicial do texto a ser extraído
3. Quantidade de caracteres a ser extraído

Na imagem ao lado, temos exemplos algumas aplicações dessas duas funções.



TRIM, LTRIM e RTRIM

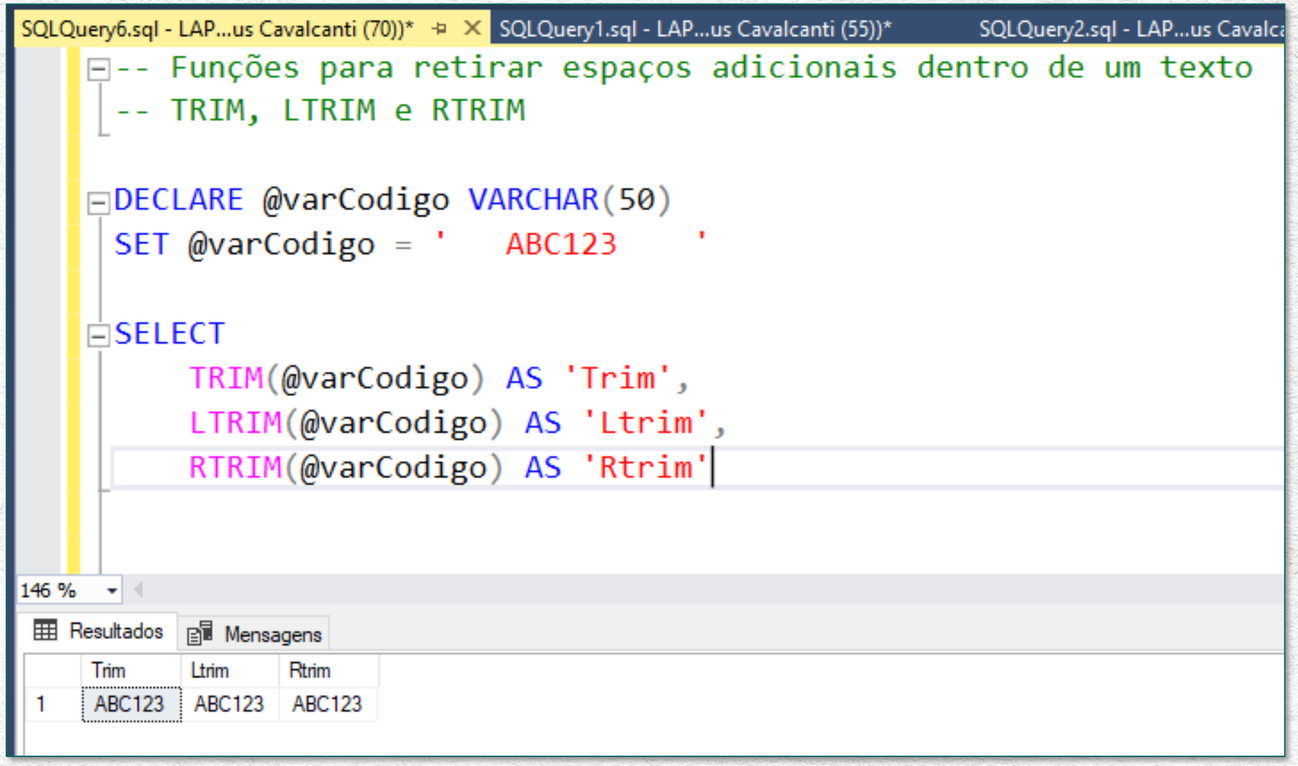
As funções **TRIM** e **LTRIM** e **RTRIM** são funções que nos permitem retirar espaços adicionais de um texto.

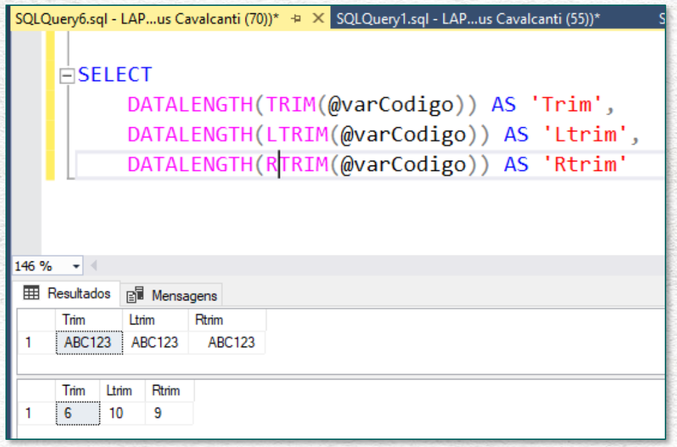
A função **TRIM** retira espaços adicionais à esquerda e à direita do texto.

A função **LTRIM** retira espaços adicionais à esquerda do texto.

A função **RTRIM** retira espaços adicionais à direita do texto.

Visualmente, parece que não há diferença na aplicação das 3 funções. Porém, se utilizarmos a função DATALENGTH sobre cada um dos resultados, vemos a diferença.

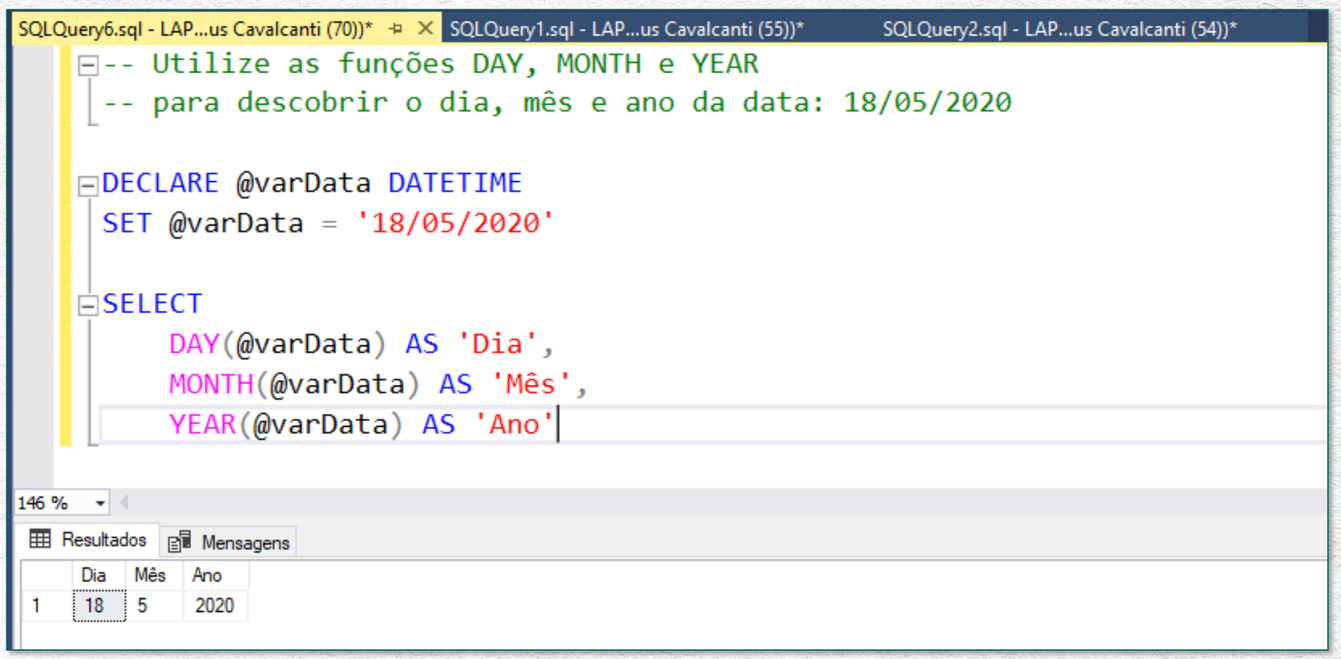




DAY, MONTH e YEAR

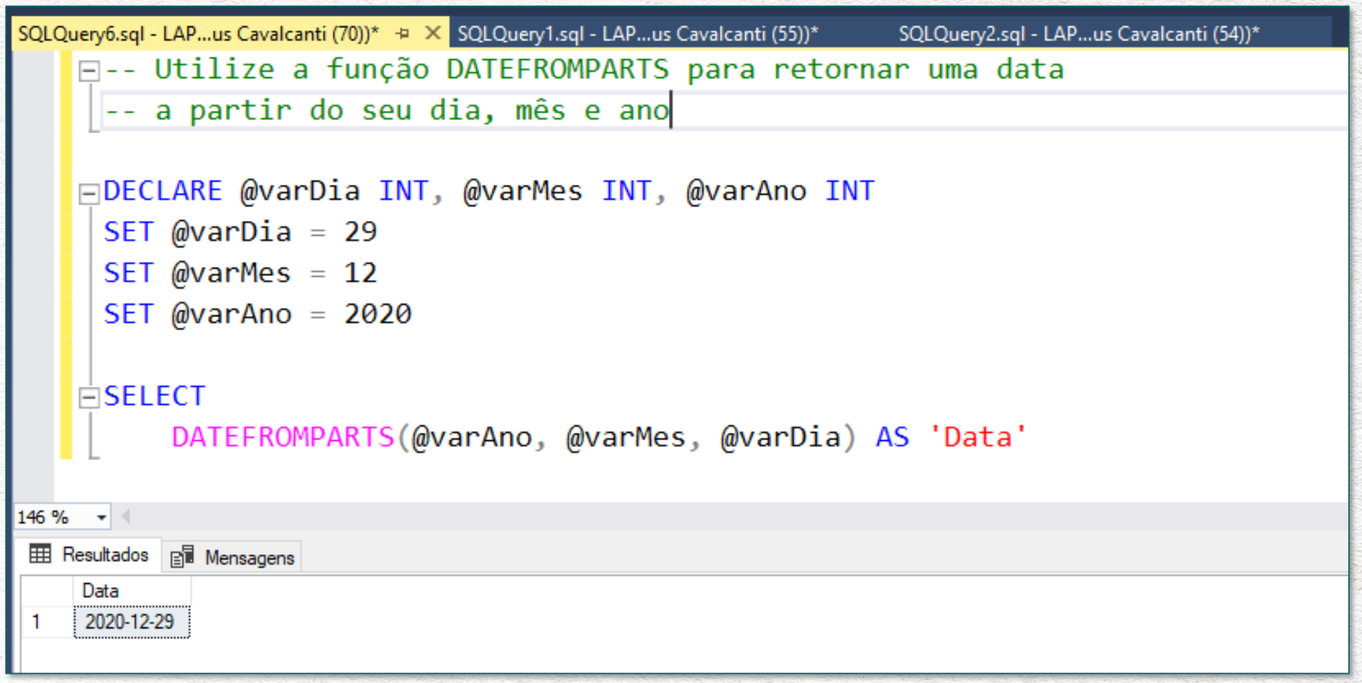
As funções **DAY** e **MONTH** e **YEAR** são funções que nos permitem descobrir o dia, mês e ano de uma data.

Ao lado, temos alguns exemplos do funcionamento dessas funções.



DATEFROMPARTS

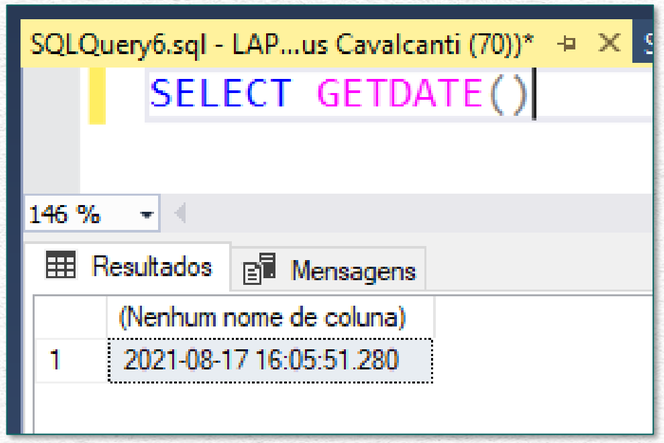
A função **DATEFROMPARTS** é uma função que permite retornar uma data completa a partir das informações separadas de dia, mês e ano.

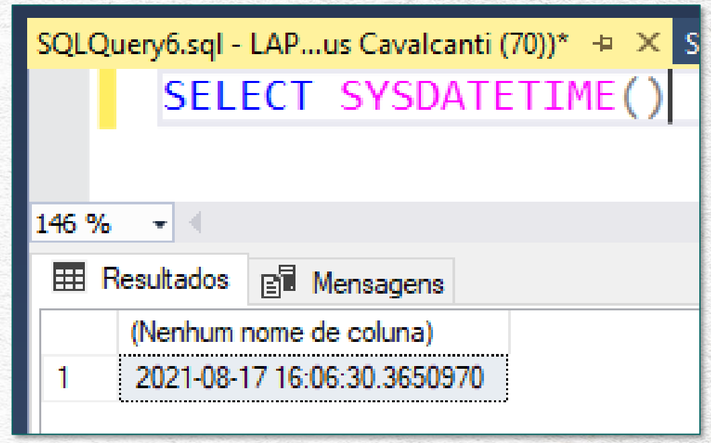


GETDATE e SYSDATETIME

As funções **GETDATE** e **SYSDATETIME** nos permitem retornar a data e hora atual do sistema.

A diferença entre elas é que a SYSDATETIME tem uma maior precisão em relação à GETDATE.





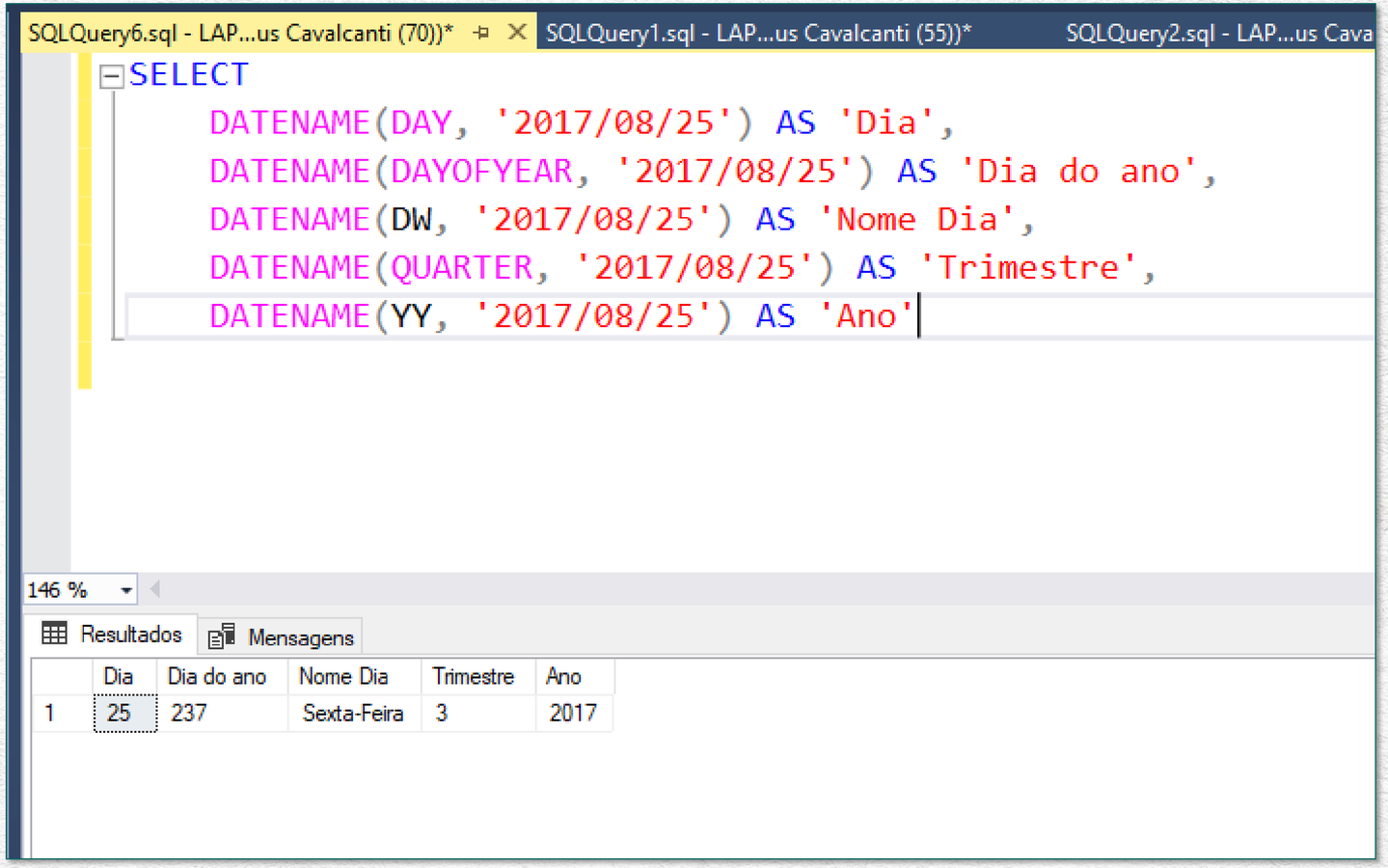
DATENAME e DATEPART

As funções **DATENAME** e **DATEPART** nos retornam informações de uma data, como o dia, mês, ano, trimestre, e assim vai.

As duas são usadas da mesma forma, com 2 argumentos:

1. Informação que queremos da data
2. Data

A diferença é que o resultado da função DATENAME é um valor do tipo texto (varchar) enquanto o resultado da função DATEPART é do tipo inteiro.



DATEADD e DATEDIFF

As funções DATEADD e DATEDIFF são funções que nos permitem realizar cálculos com datas.

A função **DATEADD** adiciona ou subtrai uma determinada quantidade de dias, meses ou anos a uma data. Essa função pede 3 argumentos:

1. Intervalo de tempo que queremos considerar (dia, mês, ano).
2. Quantidade de intervalos que queremos considerar.
3. Data na qual vamos adicionar ou subtrair os intervalos.

Já a função **DATEDIFF** permite o cálculo da diferença entre duas datas. Essa função pede 3 argumentos:

1. Intervalo de tempo que queremos subtrair (dia, mês, ano).
2. Data 1 a ser considerada no cálculo.
3. Data 2 a ser considerada no cálculo.

Ao lado, podemos ver exemplos práticos dessas funções.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalca

```
--DATEADD: Adiciona ou subtrai uma determinada quantidade
--de dias, meses ou anos a uma data.
--DATEDIFF: Calcula a diferença entre duas datas

DECLARE @varData1 DATETIME, @varData2 DATETIME, @varData3 DATETIME
SET @varData1 = '10/07/2020'
SET @varData2 = '05/03/2020'
SET @varData3 = '14/11/2020'

--DATEADD
SELECT
    DATEADD(MONTH, -1, @varData1) AS 'Dateadd'

--DATEDIFF
SELECT
    DATEDIFF(DAY, @varData2, @varData3) AS 'Dateadd'
```

121 %

Resultados | Mensagens

	Dateadd
1	2020-06-10 00:00:00.000

	Dateadd
1	254

EXERCÍCIOS

1

Questão 1

Quando estamos manipulando tabelas, é importante pensar em como os dados serão apresentados em um relatório. Imagine os nomes dos produtos da tabela DimProduct. Os textos são bem grandes e pode ser que mostrar os nomes completos dos produtos não seja a opção mais interessante, pois provavelmente não vão caber em um gráfico e a visualização ficará ruim.

a) Seu gestor te pede para listar todos os produtos para que seja criado um gráfico para ser apresentado na reunião diária de equipe. Faça uma consulta à tabela DimProduct que retorne (1) **o nome do produto** e (2) **a quantidade de caracteres que cada produto tem**, e ordene essa tabela do produto com a maior quantidade de caracteres para a menor.

b) Qual é a média de caracteres dos nomes dos produtos?

c) Analise a estrutura dos nomes dos produtos e verifique se seria possível reduzir o tamanho do nome dos produtos. (Dica: existem informações redundantes nos nomes dos produtos? Seria possível substituí-las?)

d) Qual é a média de caracteres nesse novo cenário?

EXERCÍCIOS

2

Questão 2

A coluna StyleName da tabela DimProduct possui alguns códigos identificados por números distintos, que vão de 0 até 9, como pode ser visto no exemplo abaixo:

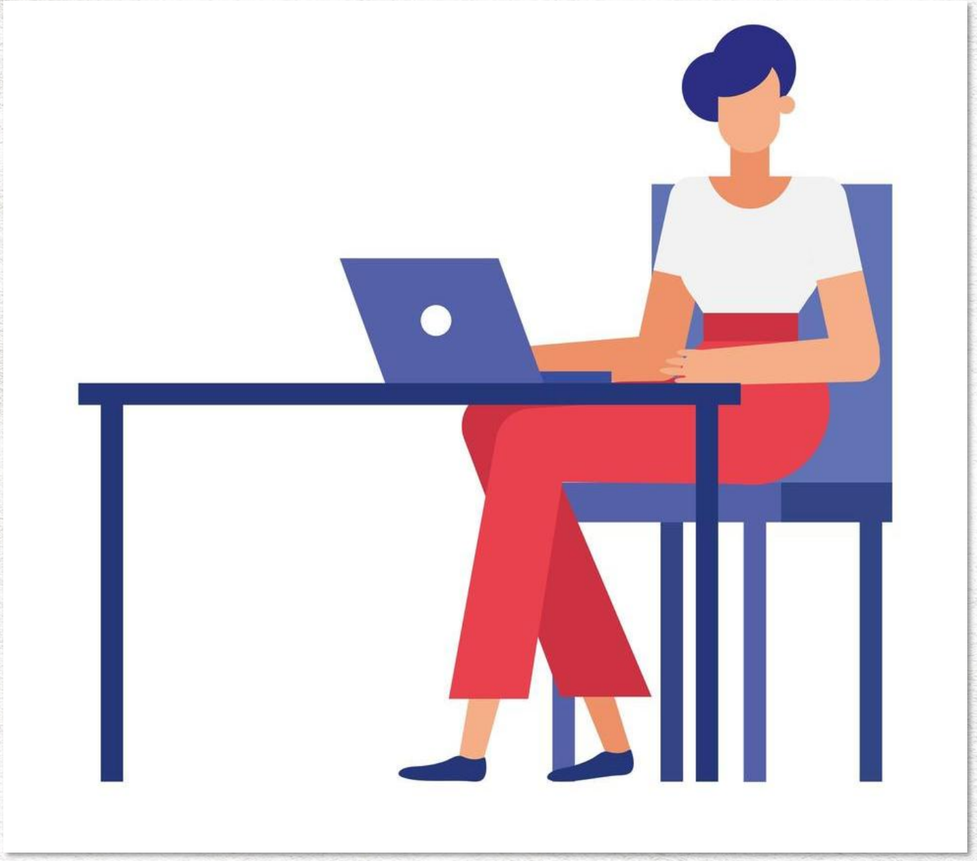
	StyleName
1	Product0101001
2	Product0101002
3	Product0101003
4	Product0101004
5	Product0101005
6	Product0101006
7	Product0101007
8	Product0101008
9	Product0101009

Porém, o setor de controle decidiu alterar a identificação do StyleName, e em vez de usar números, a ideia agora é passar a usar letras para substituir os números, conforme exemplo abaixo:

0 -> A, 1 -> B, 2 -> C, 3 -> D, 4 -> E, 5 -> F, 6 -> G, 7 -> H, 8 -> I, 9 - J

É de sua responsabilidade alterar os números por letras na coluna StyleName da tabela DimProduct. Utilize uma função que permita fazer essas substituições de forma prática e rápida.

EXERCÍCIOS



3

Questão 3

O setor de TI está criando um sistema para acompanhamento individual de cada funcionário da empresa Contoso. Cada funcionário receberá um login e senha. O login de cada funcionário será o ID do e-mail, como no exemplo abaixo:

EmailAddress
guy1@contoso.com
kevin0@contoso.com
roberto0@contoso.com
Kirk Nason@contoso.com

Já a senha será o **FirstName** + **o dia do ano em que o funcionário nasceu, em MAIÚSCULA**. Por exemplo, o funcionário com E-mail: mark0@contoso.com e data de nascimento 15/01/1990 deverá ter a seguinte senha:

Login: mark0
Senha: MARK15

O responsável pelo TI pediu a sua ajuda para retornar uma tabela contendo as seguintes colunas da tabela DimEmployee: Nome completo (FirstName + LastName), E-mail, ID do e-mail e Senha.

Portanto, faça uma consulta à tabela DimProduct e retorne esse resultado.

EXERCÍCIOS

4

Questão 4

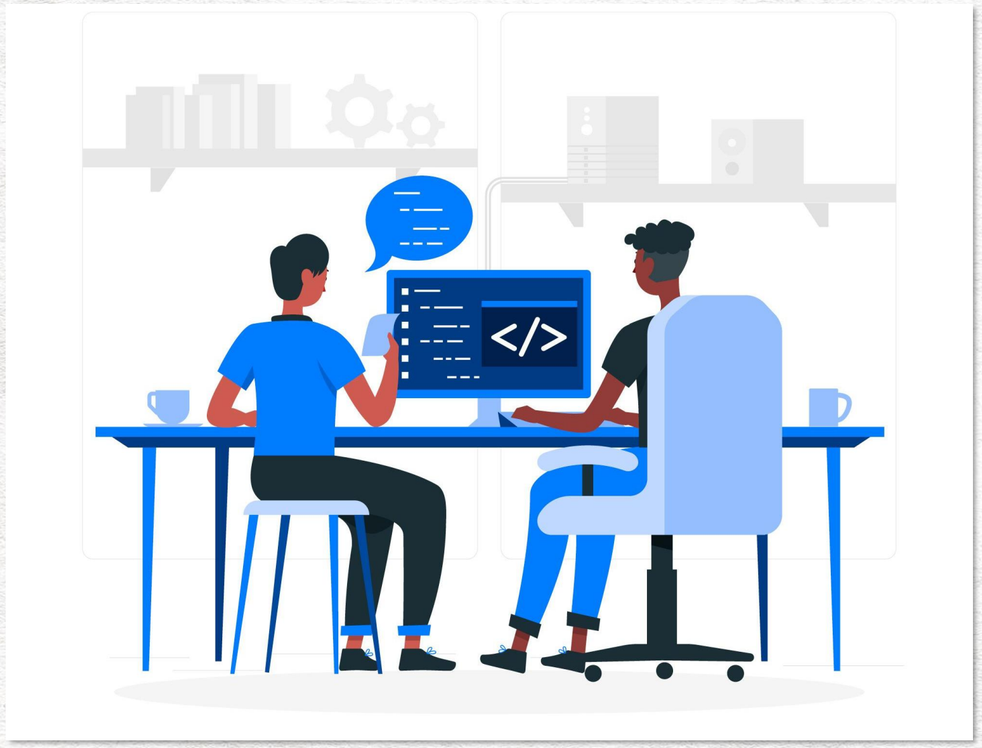
A tabela **DimCustomer** possui o primeiro registro de vendas no ano de 2001.

Como forma de reconhecer os clientes que compraram nesse ano, o setor de Marketing solicitou a você que retornasse uma tabela com todos os clientes que fizeram a sua primeira compra nesse ano para que seja enviada uma encomenda com um presente para cada um.

Para fazer esse filtro, você pode utilizar a coluna **DateFirstPurchase**, que contém a informação da data da primeira compra de cada cliente na tabela DimCustomer.

Você deverá retornar as colunas de FirstName, EmailAddress, AddressLine1 e DateFirstPurchase da tabela DimCustomer, considerando apenas os clientes que fizeram a primeira compra no ano de 2001.

EXERCÍCIOS



5

Questão 5

A tabela **DimEmployee** possui uma informação de data de contratação (HireDate). A área de RH, no entanto, precisa das informações dessas datas de forma **separada em dia, mês e ano**, pois será feita uma automatização para criação de um relatório de RH, e facilitaria muito se essas informações estivessem separadas em uma tabela.

Você deverá realizar uma consulta à tabela DimEmployee e retornar uma tabela contendo as seguintes informações: FirstName, EmailAddress, HireDate, além das colunas de Dia, Mês e Ano de contratação.

Obs. 1: **A coluna de Mês deve conter o nome do mês por extenso**, e não o número do mês.

Obs. 2: Lembre-se de nomear cada uma dessas colunas em sua consulta para garantir que o entendimento de cada informação ficará 100% claro.

6

Questão 6

Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.

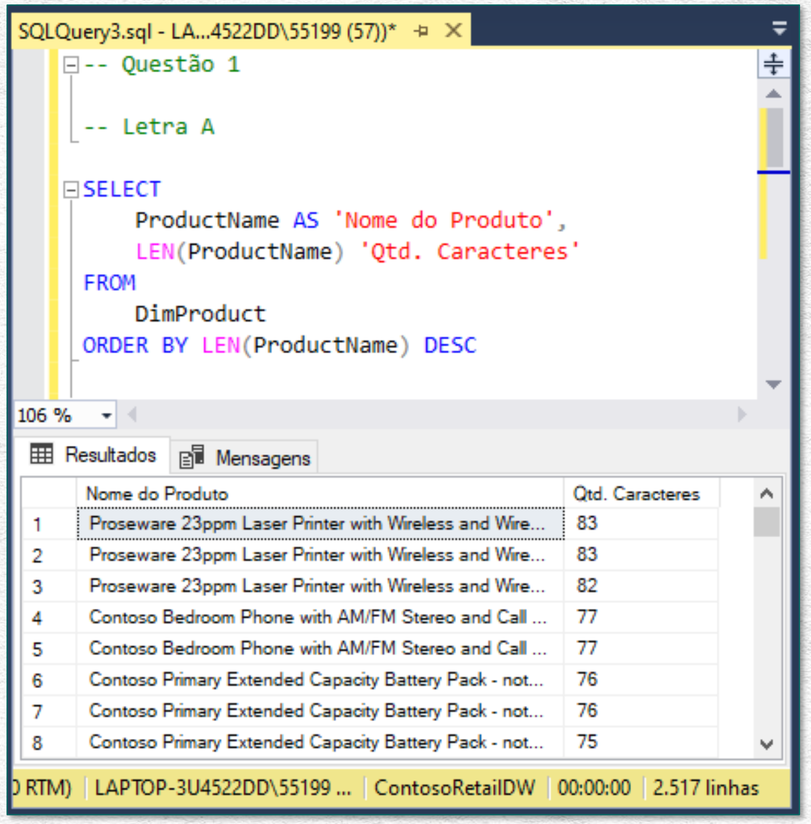
GABARITOS

1.a

Questão 1.a

Quando estamos manipulando tabelas, é importante pensar em como os dados serão apresentados em um relatório. Imagine os nomes dos produtos da tabela DimProduct. Os textos são bem grandes e pode ser que mostrar os nomes completos dos produtos não seja a opção mais interessante, pois provavelmente não vão caber em um gráfico e a visualização ficará ruim.

a) Seu gestor te pede para listar todos os produtos para que seja criado um gráfico para ser apresentado na reunião diária de equipe. Faça uma consulta à tabela DimProduct que retorne (1) **o nome do produto** e (2) **a quantidade de caracteres que cada produto tem**, e ordene essa tabela do produto com a maior quantidade de caracteres para a menor.

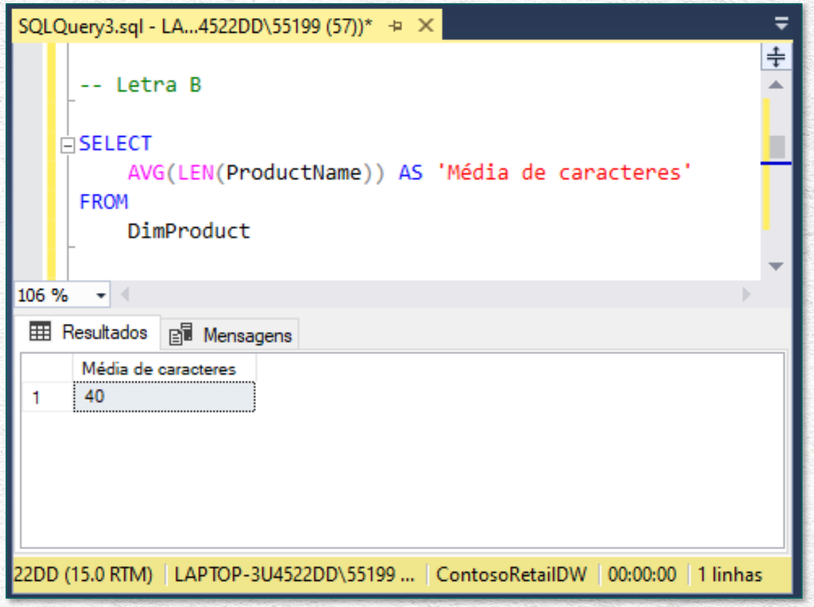


GABARITOS

1.b

Questão 1.b

b) Qual é a média de caracteres dos nomes dos produtos?

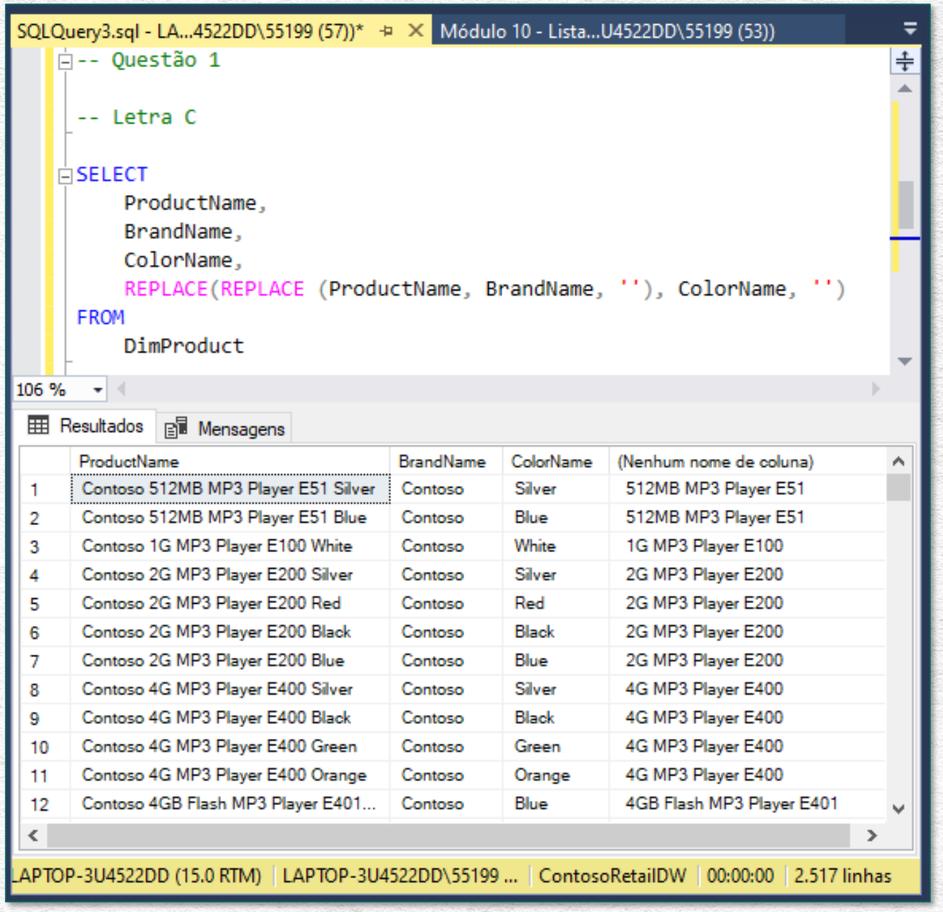


GABARITOS

1.c

Questão 1.c

c) Analise a estrutura dos nomes dos produtos e verifique se seria possível reduzir o tamanho do nome dos produtos. (Dica: existem informações redundantes nos nomes dos produtos? Seria possível substituí-las?)



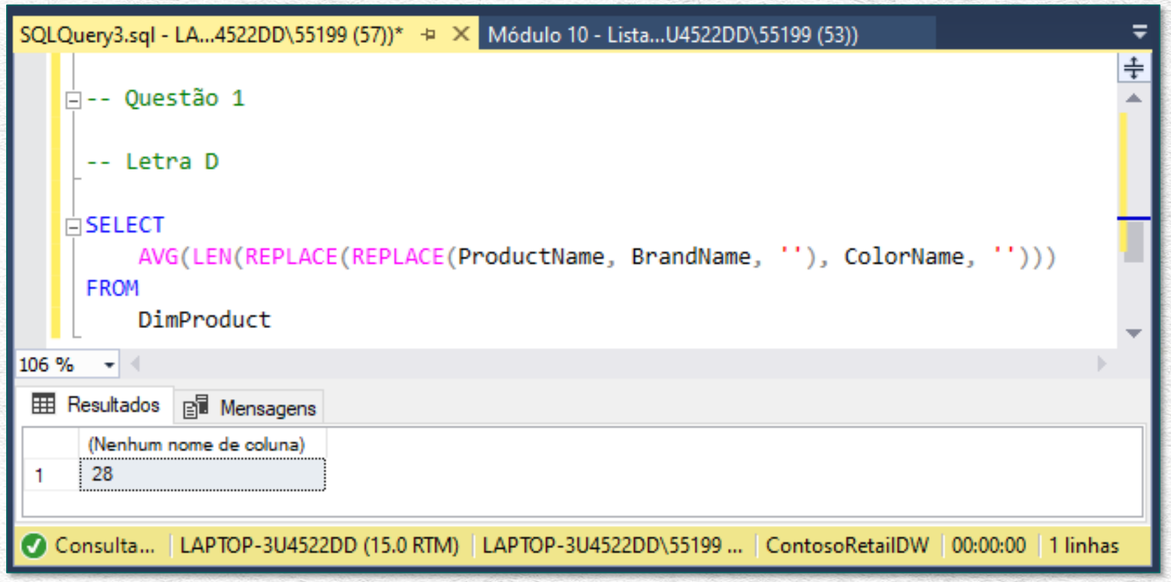
	ProductName	BrandName	ColorName	(Nenhum nome de coluna)
1	Contoso 512MB MP3 Player E51 Silver	Contoso	Silver	512MB MP3 Player E51
2	Contoso 512MB MP3 Player E51 Blue	Contoso	Blue	512MB MP3 Player E51
3	Contoso 1G MP3 Player E100 White	Contoso	White	1G MP3 Player E100
4	Contoso 2G MP3 Player E200 Silver	Contoso	Silver	2G MP3 Player E200
5	Contoso 2G MP3 Player E200 Red	Contoso	Red	2G MP3 Player E200
6	Contoso 2G MP3 Player E200 Black	Contoso	Black	2G MP3 Player E200
7	Contoso 2G MP3 Player E200 Blue	Contoso	Blue	2G MP3 Player E200
8	Contoso 4G MP3 Player E400 Silver	Contoso	Silver	4G MP3 Player E400
9	Contoso 4G MP3 Player E400 Black	Contoso	Black	4G MP3 Player E400
10	Contoso 4G MP3 Player E400 Green	Contoso	Green	4G MP3 Player E400
11	Contoso 4G MP3 Player E400 Orange	Contoso	Orange	4G MP3 Player E400
12	Contoso 4GB Flash MP3 Player E401...	Contoso	Blue	4GB Flash MP3 Player E401

GABARITOS

1.d

Questão 1.d

d) Qual é a média de caracteres nesse novo cenário?



GABARITOS

2 Questão 2

A coluna StyleName da tabela DimProduct possui alguns códigos identificados por números distintos, que vão de 0 até 9, como pode ser visto no exemplo abaixo:

	StyleName
1	Product0101001
2	Product0101002
3	Product0101003
4	Product0101004
5	Product0101005
6	Product0101006
7	Product0101007
8	Product0101008
9	Product0101009

Porém, o setor de controle decidiu alterar a identificação do StyleName, e em vez de usar números, a ideia agora é passar a usar letras para substituir os números, conforme exemplo abaixo:

0 -> A, 1 -> B, 2 -> C, 3 -> D, 4 -> E, 5 -> F, 6 -> G, 7 -> H, 8 -> I, 9 - J

É de sua responsabilidade alterar os números por letras na coluna StyleName da tabela DimProduct. Utilize uma função que permita fazer essas substituições de forma prática e rápida.

SQLQuery3.sql - LA...4522DD\55199 (57))*

```
-- Questão 2

SELECT
    ProductName,
    StyleName,
    TRANSLATE(StyleName, '0123456789', 'ABCDEFGHIJ')
FROM
    DimProduct
```

106 %

Resultados | Mensagens

	ProductName	StyleName	(Nenhum nome de coluna)
1	Contoso 512MB MP3 Player E51 Silver	Product0101001	ProductABABAAB
2	Contoso 512MB MP3 Player E51 Blue	Product0101002	ProductABABAAC
3	Contoso 1G MP3 Player E100 White	Product0101003	ProductABABAAD
4	Contoso 2G MP3 Player E200 Silver	Product0101004	ProductABABAAE
5	Contoso 2G MP3 Player E200 Red	Product0101005	ProductABABAAF
6	Contoso 2G MP3 Player E200 Black	Product0101006	ProductABABAAG
7	Contoso 2G MP3 Player E200 Blue	Product0101007	ProductABABAAH

522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 2.517 linhas

GABARITOS

3

Questão 3

O setor de TI está criando um sistema para acompanhamento individual de cada funcionário da empresa Contoso. Cada funcionário receberá um login e senha. O login de cada funcionário será o ID do e-mail, como no exemplo abaixo:

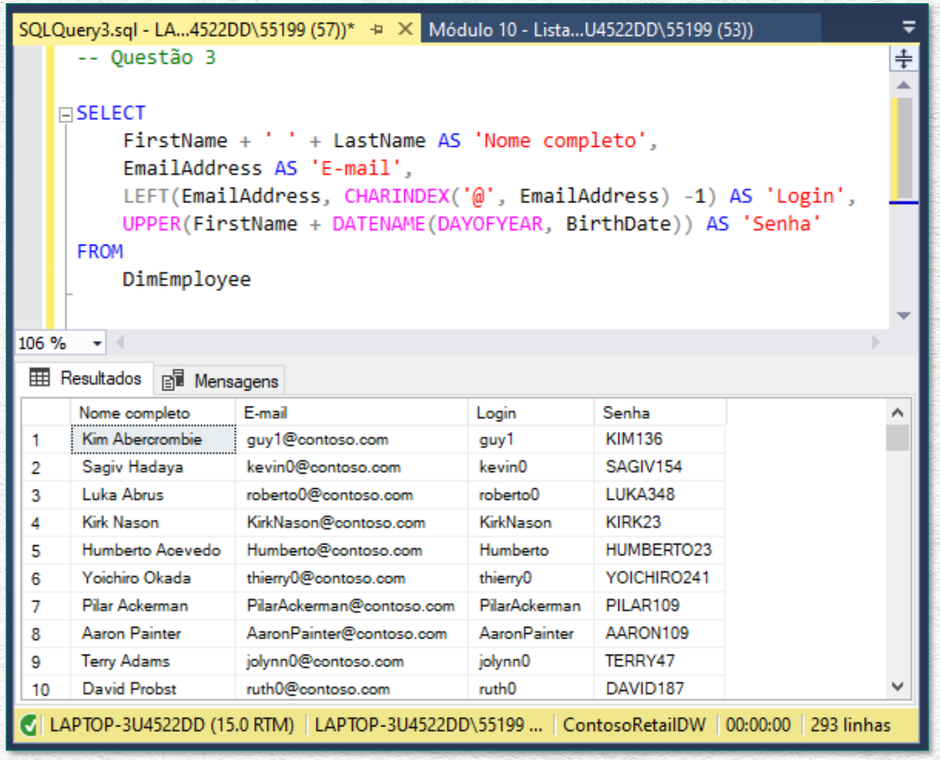
EmailAddress
guy1@contoso.com
kevin0@contoso.com
roberto0@contoso.com
Kirk Nason@contoso.com

Já a senha será o **FirstName** + **o dia do ano em que o funcionário nasceu, em MAIÚSCULA**. Por exemplo, o funcionário com E-mail: mark0@contoso.com e data de nascimento 15/01/1990 deverá ter a seguinte senha:

Login: mark0
Senha: MARK15

O responsável pelo TI pediu a sua ajuda para retornar uma tabela contendo as seguintes colunas da tabela DimEmployee: Nome completo (FirstName + LastName), E-mail, ID do e-mail e Senha.

Portanto, faça uma consulta à tabela DimProduct e retorne esse resultado.



GABARITOS

4

Questão 4

A tabela **DimCustomer** possui o primeiro registro de vendas no ano de 2001.

Como forma de reconhecer os clientes que compraram nesse ano, o setor de Marketing solicitou a você que retornasse uma tabela com todos os clientes que fizeram a sua primeira compra nesse ano para que seja enviada uma encomenda com um presente para cada um.

Para fazer esse filtro, você pode utilizar a coluna **DateFirstPurchase**, que contém a informação da data da primeira compra de cada cliente na tabela DimCustomer.

Você deverá retornar as colunas de FirstName, EmailAddress, AddressLine1 e DateFirstPurchase da tabela DimCustomer, considerando apenas os clientes que fizeram a primeira compra no ano de 2001.

SQLQuery3.sql - LA...4522DD\55199 (57))*

```sql
-- Questão 4

SELECT
    FirstName,
    EmailAddress,
    AddressLine1,
    DateFirstPurchase
FROM
    DimCustomer
WHERE YEAR(DateFirstPurchase) = 2001
```

106 %

Resultados | Mensagens

	FirstName	EmailAddress	AddressLine1	DateFirstPurchase
1	Jon	jon24@adventure-works.com	3761 N. 14th St	2001-07-22
2	Eugene	eugene10@adventure-works.com	2243 W St.	2001-07-18
3	Ruben	ruben35@adventure-works.com	5844 Linden Land	2001-07-10
4	Christy	christy12@adventure-works.com	1825 Village Pl.	2001-07-01
5	Elizabeth	elizabeth5@adventure-works.com	7553 Harness Circle	2001-07-26
6	Julio	julio1@adventure-works.com	7305 Humphrey Drive	2001-07-02
7	Janet	janet9@adventure-works.com	2612 Berry Dr	2001-07-27
8	Marco	marco14@adventure-works.com	942 Brook Street	2001-07-12
9	Rob	rob4@adventure-works.com	624 Peabody Road	2001-07-28

J4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 1.013 linhas

GABARITOS

5

Questão 5

A tabela **DimEmployee** possui uma informação de data de contratação (HireDate). A área de RH, no entanto, precisa das informações dessas datas de forma **separada em dia, mês e ano**, pois será feita uma automatização para criação de um relatório de RH, e facilitaria muito se essas informações estivessem separadas em uma tabela.

Você deverá realizar uma consulta à tabela DimEmployee e retornar uma tabela contendo as seguintes informações: FirstName, EmailAddress, HireDate, além das colunas de Dia, Mês e Ano de contratação.

Obs. 1: **A coluna de Mês deve conter o nome do mês por extenso**, e não o número do mês.

Obs. 2: Lembre-se de nomear cada uma dessas colunas em sua consulta para garantir que o entendimento de cada informação ficará 100% claro.

SQLQuery3.sql - LA...4522DD\55199 (57))* | Módulo 10 - Lista...U4522DD\55199 (53))

```
-- Questão 5

SELECT
    FirstName AS 'Nome',
    EmailAddress AS 'E-mail',
    HireDate AS 'Data da contratação',
    DAY(HireDate) AS 'Dia da contratação',
    DATENAME(MONTH, HireDate) AS 'Mês da contratação',
    YEAR(HireDate) AS 'Ano da contratação'
FROM
    DimEmployee
```

106 %

Resultados | Mensagens

	Nome	E-mail	Data da contratação	Dia da contratação	Mês da contratação	Ano da contratação
1	Kim	guy1@contoso.com	1996-07-31	31	Julho	1996
2	Sagiv	kevin0@contoso.com	1997-02-26	26	Fevereiro	1997
3	Luka	roberto0@contoso.com	1997-12-12	12	Dezembro	1997
4	Kirk	KirkNason@contoso.com	1998-01-05	5	Janeiro	1998
5	Humberto	Humberto@contoso.com	1998-01-05	5	Janeiro	1998
6	Yoichiro	thierry0@contoso.com	1998-01-11	11	Janeiro	1998
7	Pilar	PilarAckerman@contoso.com	1998-01-20	20	Janeiro	1998
8	Aaron	AaronPainter@contoso.com	1998-01-20	20	Janeiro	1998

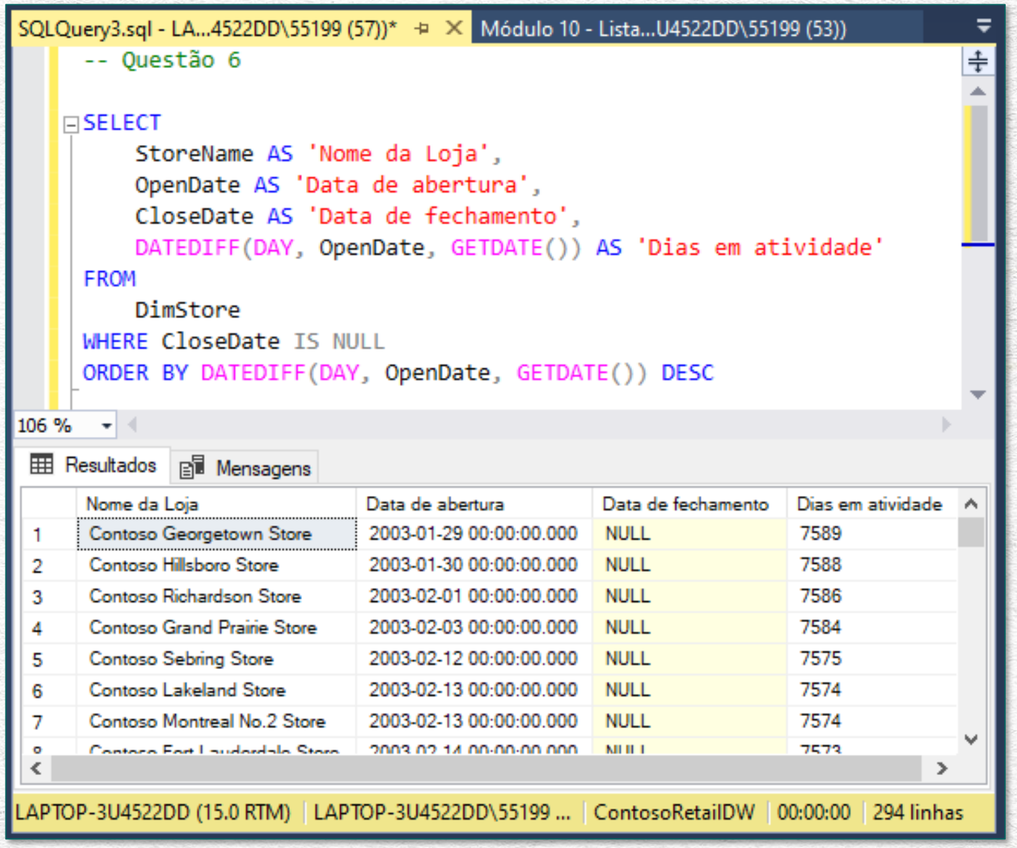
Consulta executa... | LAPTOP-3U4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 293 linhas

GABARITOS

6

Questão 6

Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.



MÓDULO 11

SQL

Funções Condicionais

O QUE VEREMOS NESTE MÓDULO

MÓDULO 11 FUNÇÕES CONDICIONAIS

01	CASE WHEN... ELSE	277
02	CASE WHEN... ELSE (mais de uma condição)	278
03	CASE + AND	279
04	CASE + OR	280
05	CASE Aninhado	281
06	CASE Aditivo	282
07	IIF: alternativa ao CASE	283
08	IIF Composto	284
09	ISNULL: Tratando valores nulos	285
10	EXERCÍCIOS	286
11	GABARITOS	292

CASE WHEN... ELSE

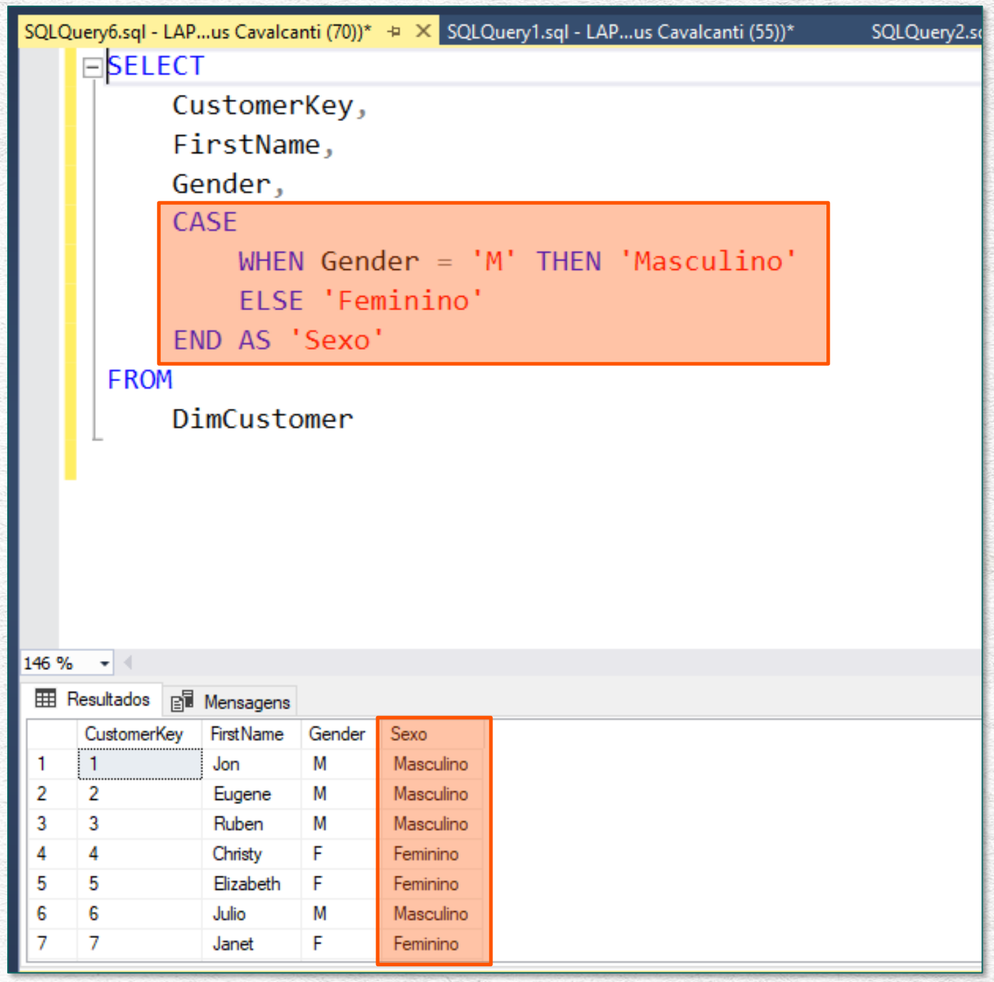
Assim como qualquer linguagem de programação, no SQL também podemos tratar condições.

A estrutura utilizada para tratamento de condições aqui é o CASE. A função CASE nos permite tratar condições e retornar um determinado valor, de acordo com o resultado de um ou mais testes lógicos. Essa função tem exatamente o mesmo objetivo das funções SE e IF que você já deve ter visto em algum momento.

Ao lado, temos um exemplo simples de como usar o CASE.

Ele será incluído em nosso código dentro de um SELECT como se fosse uma nova coluna.

No exemplo, utilizamos o CASE para verificar se o Gênero do cliente é igual a 'M'. Se for, queremos retornar o texto 'Masculino'. Caso contrário, retornamos o resultado 'Feminino'.



	CustomerKey	FirstName	Gender	Sexo
1	1	Jon	M	Masculino
2	2	Eugene	M	Masculino
3	3	Ruben	M	Masculino
4	4	Christy	F	Feminino
5	5	Elizabeth	F	Feminino
6	6	Julio	M	Masculino
7	7	Janet	F	Feminino

CASE WHEN... ELSE (mais de uma condição)

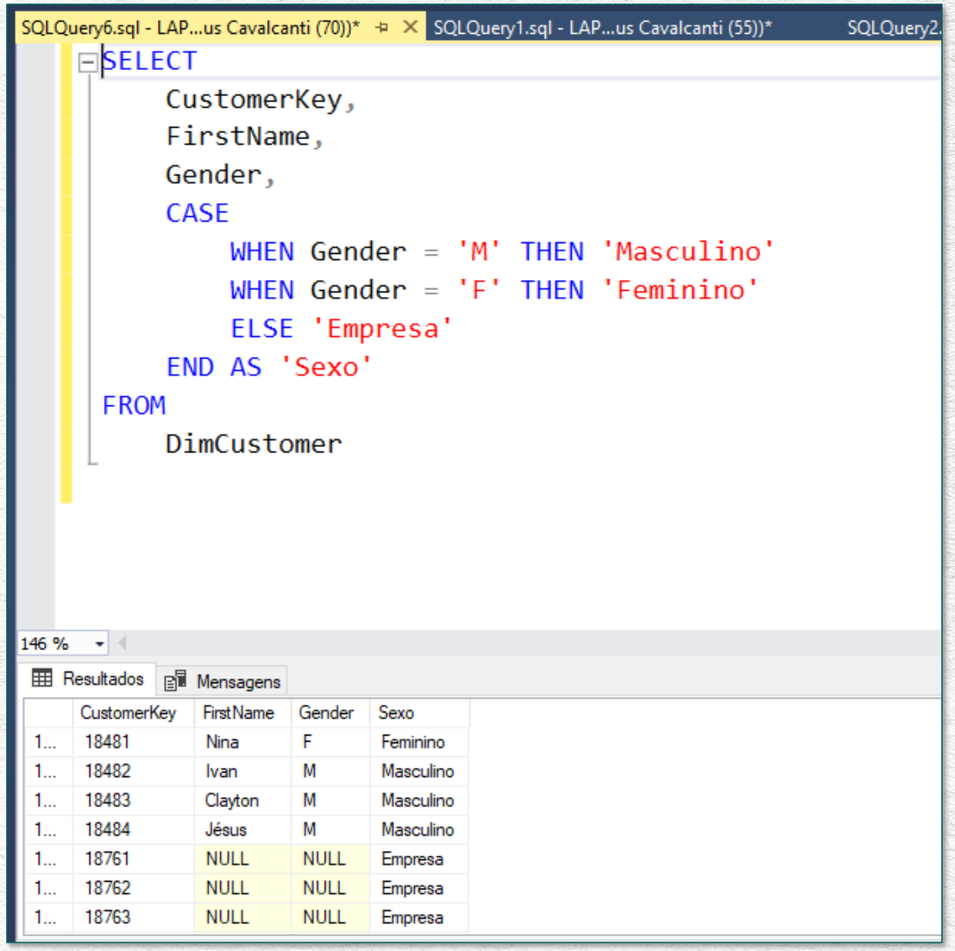
No exemplo anterior, vimos como utilizar o CASE com apenas um teste lógico. Ou seja, verificamos apenas uma vez o sexo do cliente. Caso o gênero não fosse 'M', já assumimos que seria 'F'.

Porém, alguns clientes na verdade são empresas, e na coluna de Gender o que temos é o valor NULL.

Neste caso, se utilizarmos o CASE criado no exemplo anterior, todas as empresas ficarão com o Sexto 'Feminino'.

O ideal seria que a gente fizesse um novo teste, para conseguir verificar se, caso o Gênero não seja nem 'M' nem 'F', ai sim podemos considerar como 'Empresa'.

Observe na imagem ao lado o resultado final deste CASE com dois testes lógicos.



	CustomerKey	FirstName	Gender	Sexo
1...	18481	Nina	F	Feminino
1...	18482	Ivan	M	Masculino
1...	18483	Clayton	M	Masculino
1...	18484	Jésus	M	Masculino
1...	18761	NULL	NULL	Empresa
1...	18762	NULL	NULL	Empresa
1...	18763	NULL	NULL	Empresa

CASE + AND

Podemos combinar o CASE com o AND para criar testes lógicos ainda mais avançados.

Observe o exemplo ao lado. Queremos criar uma nova coluna em nossa tabela para registrar o desconto de cada produto.

Os produtos que forem da marca Contoso E também forem da cor Red, receberão um desconto de 10%. Caso contrário, não recebem nenhum desconto.

A solução final é mostrada ao lado. Repare que a única diferença em relação aos exemplos anteriores foi que adicionamos um AND para nos permitir passar mais de um teste lógico.

Como o operador lógico AND, as duas condições (Marca igual à Contoso E Cor igual à Red) precisam ser satisfeitas para que o desconto seja aplicado.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
--CASE/AND: Exemplo
-- Aplique um desconto de 10% aos produtos que foram da marca CONTOSO
-- E TAMBÉM da cor Red
SELECT
    ProductName,
    BrandName,
    ColorName,
    UnitPrice,
    CASE
        WHEN BrandName = 'Contoso' AND ColorName = 'Red' THEN 0.1
        ELSE 0
    END AS 'Desconto'
FROM
    DimProduct
```

133 %

Resultados | Mensagens

	ProductName	BrandName	ColorName	UnitPrice	Desconto
1	Contoso 512MB MP3 Player E51 Silver	Contoso	Silver	12,99	0.0
2	Contoso 512MB MP3 Player E51 Blue	Contoso	Blue	12,99	0.0
3	Contoso 1G MP3 Player E100 White	Contoso	White	14,52	0.0
4	Contoso 2G MP3 Player E200 Silver	Contoso	Silver	21,57	0.0
5	Contoso 2G MP3 Player E200 Red	Contoso	Red	21,57	0.1
6	Contoso 2G MP3 Player E200 Black	Contoso	Black	21,57	0.0
7	Contoso 2G MP3 Player E200 Blue	Contoso	Blue	21,57	0.0
8	Contoso 4G MP3 Player E400 Silver	Contoso	Silver	59,99	0.0
9	Contoso 4G MP3 Player E400 Black	Contoso	Black	59,99	0.0
10	Contoso 4G MP3 Player E400 Green	Contoso	Green	59,99	0.0

CASE + OR

Podemos também combinar o CASE com o operador lógico OR.

No exemplo ao lado, os produtos que forem da marca Contoso OU da marca Fabrikam, receberão um desconto de 10%. Caso contrário, não recebem nenhum desconto.

Como o operador lógico OR, qualquer uma das duas condições poderá ser satisfeita (Marca igual à Contoso OU Marca igual à Fabrikam) para que o desconto seja aplicado.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (

```sql
--CASE/AND: Exemplo
-- Aplique um desconto de 10% aos produtos que foram da marca CONTOSO
-- OU da marca FABRIKAM
SELECT
    ProductName,
    BrandName,
    ColorName,
    UnitPrice,
    CASE
        WHEN BrandName = 'Contoso' OR BrandName = 'Fabrikam' THEN 0.1
        ELSE 0
    END AS 'Desconto'
FROM
    DimProduct
```

133 %

Resultados | Mensagens

	ProductName	BrandName	ColorName	UnitPrice	Desconto
1	Contoso 512MB MP3 Player E51 Silver	Contoso	Silver	12,99	0.1
2	Contoso 512MB MP3 Player E51 Blue	Contoso	Blue	12,99	0.1
3	Contoso 1G MP3 Player E100 White	Contoso	White	14,52	0.1
4	Contoso 2G MP3 Player E200 Silver	Contoso	Silver	21,57	0.1
5	Contoso 2G MP3 Player E200 Red	Contoso	Red	21,57	0.1
6	Contoso 2G MP3 Player E200 Black	Contoso	Black	21,57	0.1
7	Contoso 2G MP3 Player E200 Blue	Contoso	Blue	21,57	0.1
8	Contoso 4G MP3 Player E400 Silver	Contoso	Silver	59,99	0.1
9	Contoso 4G MP3 Player E400 Black	Contoso	Black	59,99	0.1
10	Contoso 4G MP3 Player E400 Green	Contoso	Green	59,99	0.1

CASE Aninhado

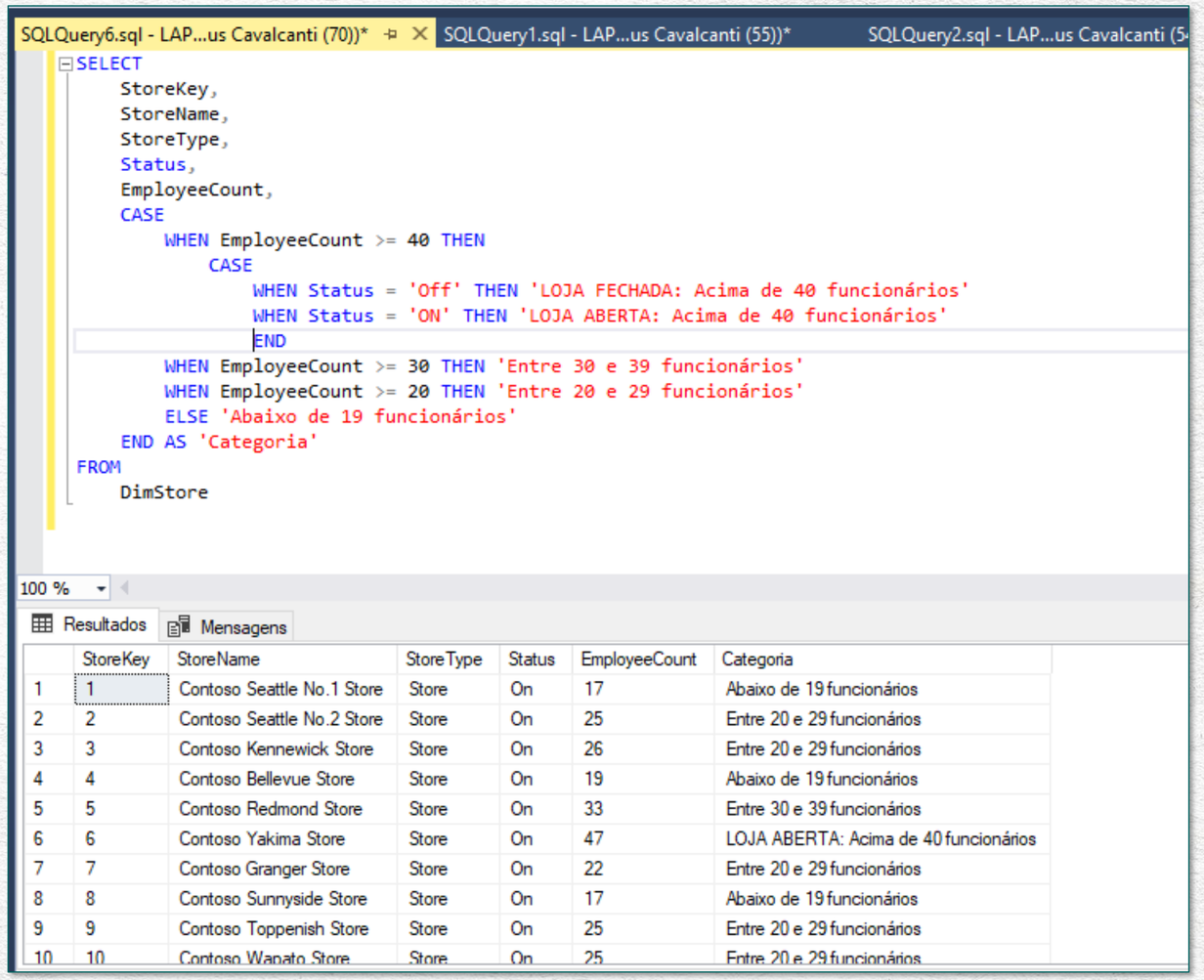
O CASE Aninhado nada mais é do que um CASE dentro de outro CASE. Em algumas situações ele pode ser muito útil.

No exemplo ao lado, criamos uma lógica onde, de acordo com a quantidade de funcionários da loja (EmployeeCount) estabelecemos um status para aquela loja:

- Acima de 40 funcionários
- Entre 30 e 39 funcionários
- Entre 20 e 29 funcionários
- Abaixo de 19 funcionários

Porém, especificamente para as lojas que têm mais de 40 funcionários, queremos fazer um novo teste e dividir em 'LOJA ABERTA' ou 'LOJA FECHADA'.

Para isso, utilizamos um novo CASE, como mostrado na imagem.



	StoreKey	StoreName	StoreType	Status	EmployeeCount	Categoria
1	1	Contoso Seattle No.1 Store	Store	On	17	Abaixo de 19 funcionários
2	2	Contoso Seattle No.2 Store	Store	On	25	Entre 20 e 29 funcionários
3	3	Contoso Kennewick Store	Store	On	26	Entre 20 e 29 funcionários
4	4	Contoso Bellevue Store	Store	On	19	Abaixo de 19 funcionários
5	5	Contoso Redmond Store	Store	On	33	Entre 30 e 39 funcionários
6	6	Contoso Yakima Store	Store	On	47	LOJA ABERTA: Acima de 40 funcionários
7	7	Contoso Granger Store	Store	On	22	Entre 20 e 29 funcionários
8	8	Contoso Sunnyside Store	Store	On	17	Abaixo de 19 funcionários
9	9	Contoso Toppenish Store	Store	On	25	Entre 20 e 29 funcionários
10	10	Contoso Wapato Store	Store	On	25	Entre 20 e 29 funcionários

CASE Aditivo

Uma outra aplicação do CASE é em uma situação especial, mostrada ao lado.

Nessa aplicação, conseguimos adicionar uma condição a uma condição anterior.

Por exemplo, no exercício ao lado, queremos aplicar um desconto de 10% aos produtos da categoria 'TV and Video'. E além disso, caso também seja da subcategoria 'Televisão', queremos aplicar mais 5%, fechando em 15% de desconto.

Criamos um primeiro CASE para verificar a categoria. Logo em seguida, somamos ao CASE que verifica se a subcategoria é televisão.

O resultado final é mostrado na imagem ao lado.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
-- CASE STATEMENT Aditivo
-- Os produtos da categoria 'TV and Video' terão um desconto de 10%
-- Se além de ser da categoria 'TV and Video', o produto for da subcategoria 'Televisions', receberá mais 5%. Total, 15%

SELECT
    ProductKey,
    ProductName,
    ProductCategoryName,
    ProductSubcategoryName,
    UnitPrice,
    CASE WHEN ProductCategoryName = 'TV and Video'
        THEN 0.10 ELSE 0.00 END
  + CASE WHEN ProductSubCategoryName = 'Televisions'
        THEN 0.05 ELSE 0.00 END
FROM DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
        INNER JOIN DimProductCategory
            ON DimProductSubcategory.ProductCategoryKey = DimProductCategory.ProductCategoryKey
```

100 %

Resultados | Mensagens

	ProductKey	ProductName	ProductCategoryName	ProductSubcategoryName	UnitPrice	(Nenhum nome de coluna)
1	1	Contoso 512MB MP3 Player E51 Silver	Audio	MP4&MP3	12,99	0.00
2	2	Contoso 512MB MP3 Player E51 Blue	Audio	MP4&MP3	12,99	0.00
3	3	Contoso 1G MP3 Player E100 White	Audio	MP4&MP3	14,52	0.00
4	4	Contoso 2G MP3 Player E200 Silver	Audio	MP4&MP3	21,57	0.00
5	5	Contoso 2G MP3 Player E200 Red	Audio	MP4&MP3	21,57	0.00
6	6	Contoso 2G MP3 Player E200 Black	Audio	MP4&MP3	21,57	0.00
7	7	Contoso 2G MP3 Player E200 Blue	Audio	MP4&MP3	21,57	0.00
8	8	Contoso 4G MP3 Player E400 Silver	Audio	MP4&MP3	59,99	0.00
9	9	Contoso 4G MP3 Player E400 Black	Audio	MP4&MP3	59,99	0.00
10	10	Contoso 4G MP3 Player E400 Green	Audio	MP4&MP3	59,99	0.00

Consulta executada com êxito. | LAPTOP-HBTCI8PQ (15.0 RTM) | LAPTOP-HBTCI8PQ\Marcus... | ContosoRetailDW | 00:00:0

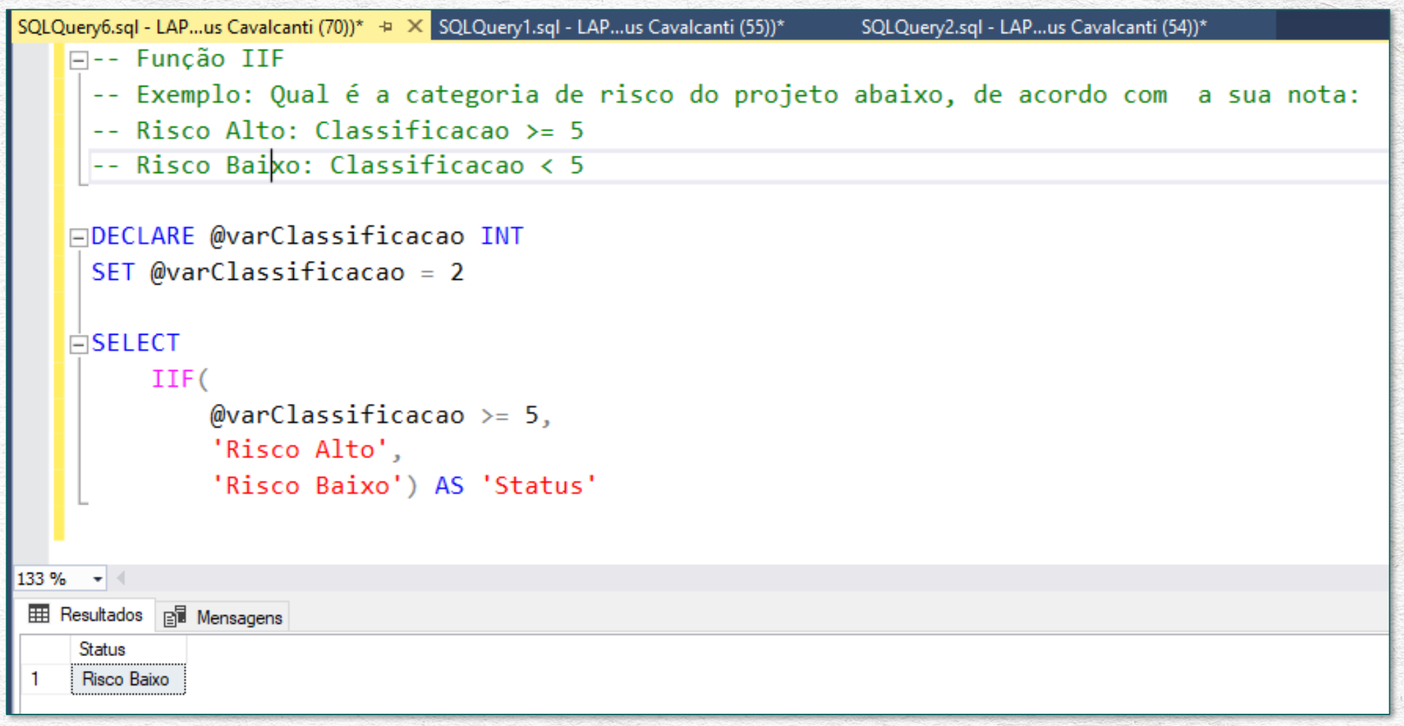
IIF: alternativa ao CASE

Uma alternativa ao CASE seria a função **IIF**. Ela é muito semelhante com a fórmula SE do Excel, pois pede exatamente 3 argumentos:

1. Teste Lógico
2. Valor se verdadeiro
3. Valor se falso

No exercício ao lado, queremos classificar um projeto de acordo com o seu nível de risco. Utilizamos para isso uma variável.

O resultado final é mostrado na imagem.



IIF Composto

É possível utilizar um IIF dentro de outro IIF, criando o que chamamos de **IIF composto**.

Com essa aplicação, tratamos vários resultados possíveis, de acordo com mais de um teste lógico.

No exemplo ao lado, queremos fazer uma consulta à tabela DimProduct e retornar o responsável pelos estoques dos produtos, de acordo com o StockTypeName.

Como temos 3 tipos de estoque e consequentemente 3 responsáveis diferentes, teremos que utilizar mais de 1 IIF para conseguir cobrir todas as situações possíveis.

O resultado final da consulta é mostrado ao lado.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```
--IIF Composto
--Faça um SELECT na tabela DimProduct e
--crie uma coluna que identifique os responsáveis
--por cada produto, de acordo com o seu estoque (StockTypeName)
--João é responsável pelos produtos High
--Maria é responsável pelos produtos Mid
--Luis é responsável pelos produtos Low

SELECT
    ProductKey,
    ProductName,
    StockTypeName,
    IIF(
        StockTypeName = 'High',
        'João',
        IIF(
            StockTypeName = 'Mid',
            'Maria',
            'Luis')
        ) AS 'Responsável'
FROM
    DimProduct
```

ISNULL: Tratando valores nulos

A função **ISNULL** nos permite retornar um resultado alternativo para o caso de um valor ser nulo (NULL).

Observe na imagem ao lado a coluna CityName. Diversas dessas cidades estão com o valor NULL.

Podemos utilizar a função ISNULL para verificar essa coluna (no 1º argumento) e caso o valor seja nulo, ele retornará um resultado alternativo.

No exemplo ao lado, o resultado alternativo para as cidades com o nome NULL será o texto 'Local desconhecido'.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

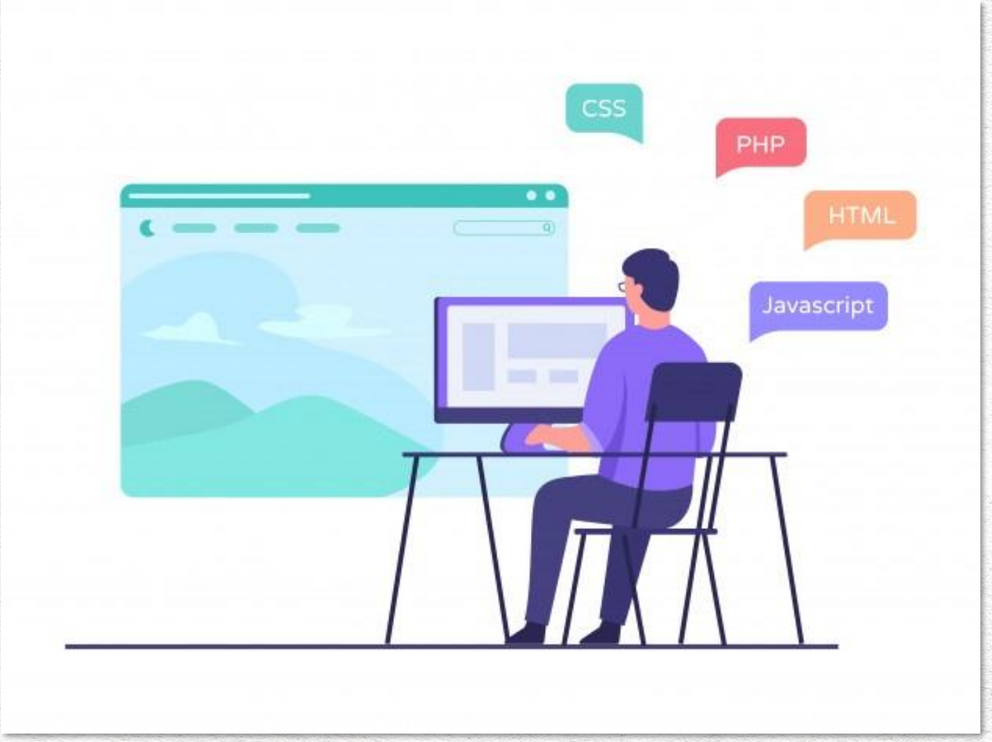
```
--ISNULL: Tratando valores nulos
SELECT
    GeographyKey,
    ContinentName,
    CityName,
    ISNULL(CityName, 'Local desconhecido')
FROM
    DimGeography
```

121 %

Resultados | Mensagens

	GeographyKey	ContinentName	CityName	(Nenhum nome de coluna)
148	413	Europe	NULL	Local desconhecido
149	414	Europe	NULL	Local desconhecido
150	415	North America	NULL	Local desconhecido
151	416	Europe	NULL	Local desconhecido
152	417	North America	NULL	Local desconhecido
153	418	Europe	NULL	Local desconhecido
154	419	Europe	NULL	Local desconhecido
155	420	Asia	NULL	Local desconhecido
156	421	North America	NULL	Local desconhecido
157	422	North America	NULL	Local desconhecido
158	423	Europe	Basings...	Basingstoke Hants
159	424	North America	Bellevue	Bellevue
160	425	Asia	Bendigo	Bendigo
161	426	Europe	Berks	Berks
162	430	North America	Beverly...	Beverly Hills
163	431	North America	Billings	Billings
164	432	North America	Biloxi	Biloxi

EXERCÍCIOS



1

Questão 1

O setor de vendas decidiu aplicar um desconto aos produtos de acordo com a sua classe. O percentual aplicado deverá ser de:

Economy -> 5%

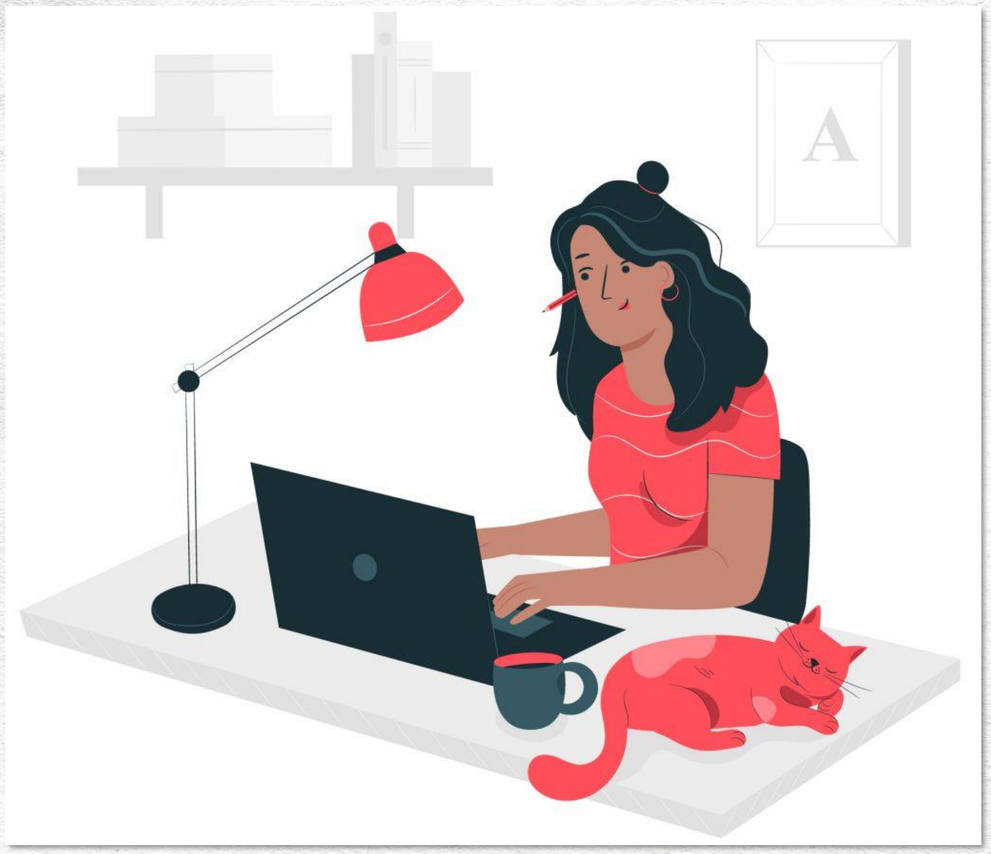
Regular -> 7%

Deluxe -> 9%

a) Faça uma consulta à tabela DimProduct que retorne as seguintes colunas: ProductKey, ProductName, e outras duas colunas que deverão retornar o % de Desconto e UnitPrice com desconto.

b) Faça uma adaptação no código para que os % de desconto de 5%, 7% e 9% sejam facilmente modificados (dica: utilize variáveis).

EXERCÍCIOS



2

Questão 2

Você ficou responsável pelo controle de produtos da empresa e deverá fazer uma análise da quantidade de produtos por Marca.

A divisão das marcas em categorias deverá ser a seguinte:

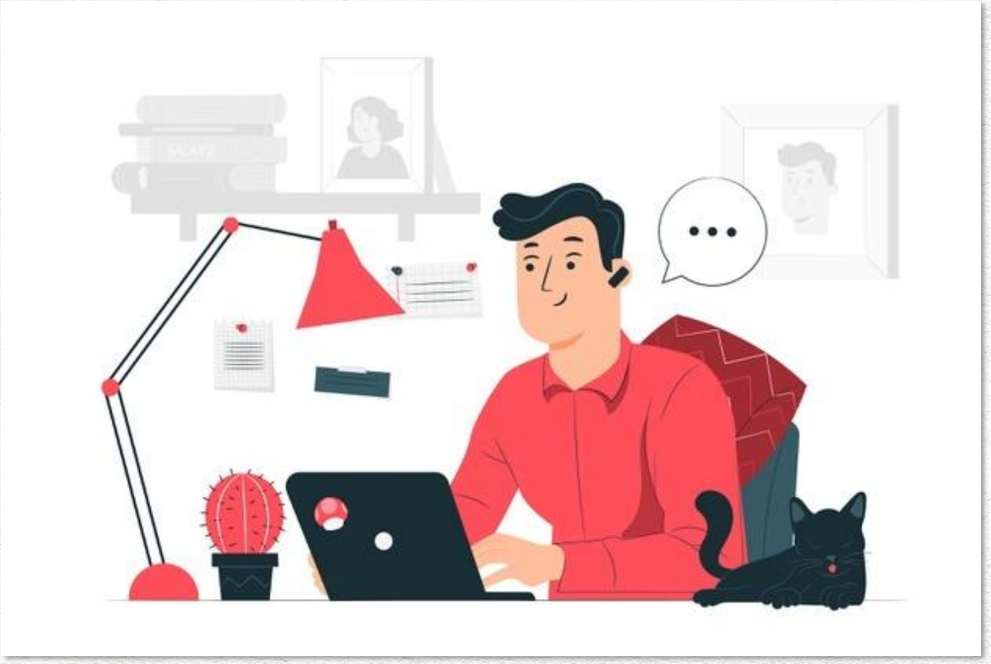
CATEGORIA A: Mais de 500 produtos

CATEGORIA B: Entre 100 e 500 produtos

CATEGORIA C: Menos de 100 produtos

Faça uma consulta à tabela DimProduct e retorne uma tabela com um agrupamento de Total de Produtos por Marca, além da coluna de Categoria, conforme a regra acima.

EXERCÍCIOS



3

Questão 3

Será necessário criar uma categorização de cada loja da empresa considerando a quantidade de funcionários de cada uma. A lógica a ser seguida será a lógica abaixo:

EmployeeCount >= 50; 'Acima de 50 funcionários'

EmployeeCount >= 40; 'Entre 40 e 50 funcionários'

EmployeeCount >= 30; 'Entre 30 e 40 funcionários'

EmployeeCount >= 20; 'Entre 20 e 30 funcionários'

EmployeeCount >= 40; 'Entre 10 e 20 funcionários'

Caso contrário: 'Abaixo de 10 funcionários’

Faça uma consulta à tabela DimStore que retorne as seguintes informações: StoreName, EmployeeCount e a coluna de categoria, seguindo a regra acima.

EXERCÍCIOS

4

Questão 4

O setor de logística deverá realizar um transporte de carga dos produtos que estão no depósito de Seattle para o depósito de Sunnyside.

Não se tem muitas informações sobre os produtos que estão no depósito, apenas se sabe que existem **100 exemplares de cada Subcategoria**. Ou seja, 100 laptops, 100 câmeras digitais, 100 ventiladores, e assim vai.

O gerente de logística definiu que **os produtos serão transportados por duas rotas distintas**. Além disso, a divisão dos produtos em cada uma das rotas será feita de acordo com as subcategorias (ou seja, todos os produtos de uma mesma subcategoria serão transportados pela mesma rota):

Rota 1: As subcategorias que tiverem uma soma total menor que 1000 kg deverão ser transportados pela Rota 1.

Rota 2: As subcategorias que tiverem uma soma total maior ou igual a 1000 kg deverão ser transportados pela Rota 2.

Você deverá realizar uma consulta à tabela DimProduct e fazer essa divisão das subcategorias por cada rota. Algumas dicas:

- **Dica 1**: A sua consulta deverá ter um total de 3 colunas: **Nome da Subcategoria**, **Peso Total** e **Rota**.
- **Dica 2**: Como não se sabe quais produtos existem no depósito, apenas que existem 100 exemplares de cada subcategoria, você deverá descobrir o peso médio de cada subcategoria e multiplicar essa média por 100, de forma que você descubra aproximadamente qual é o peso total dos produtos por subcategoria.
- **Dica 3**: Sua resposta final deverá ter um JOIN e um GROUP BY.

EXERCÍCIOS

5

Questão 5

O setor de marketing está com algumas ideias de ações para alavancar as vendas em 2021. Uma delas consiste em realizar sorteios entre os clientes da empresa.

Este sorteio será dividido em categorias:

‘Sorteio Mãe do Ano’: Nessa categoria vão participar todas as mulheres com filhos.

‘Sorteio Pai do Ano’: Nessa categoria vão participar todos os pais com filhos.

‘Caminhão de Prêmios’: Nessa categoria vão participar todas os demais clientes (homens e mulheres sem filhos).

Seu papel será realizar uma consulta à tabela DimCustomer e retornar as colunas:

- FirstName AS ‘Nome’
- Gender AS ‘Sexo’
- TotalChildren AS ‘Qtd. Filhos’
- EmailAdress AS ‘E-mail’
- Ação de Marketing: nessa coluna você deverá dividir os clientes de acordo com as categorias ‘Sorteio Mãe do Ano’, ‘Sorteio Pai do Ano’ e ‘Caminhão de Prêmios’.

EXERCÍCIOS

6

Questão 6

Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.

Atenção: lembre-se que **existem lojas que foram fechadas**.

GABARITOS

1.a

Questão 1.a

O setor de vendas decidiu aplicar um desconto aos produtos de acordo com a sua classe. O percentual aplicado deverá ser de:

Economy -> 5%

Regular -> 7%

Deluxe -> 9%

a) Faça uma consulta à tabela DimProduct que retorne as seguintes colunas: ProductKey, ProductName, e outras duas colunas que deverão retornar o % de Desconto e UnitPrice com desconto.

SQLQuery3.sql - LA...4522DD\55199 (57))* | Módulo 11 - Lista...U4522DD\55199 (52))*

```sql
-- Questão 1

-- Letra A

SELECT
    ProductKey AS 'ID Produto',
    ProductName AS 'Nome Produto',
    ClassName AS 'Classe',
    UnitPrice AS 'Preço',
    CASE
        WHEN ClassName = 'Economy' THEN (1 - 0.05) * UnitPrice
        WHEN ClassName = 'Regular' THEN (1 - 0.07) * UnitPrice
        ELSE (1 - 0.09) * UnitPrice
    END AS '% Desconto'
FROM
    DimProduct
```

106 %

Resultados | Mensagens

	ID Produto	Nome Produto	Classe	Preço	% Desconto
1	1	Contoso 512MB MP3 Player E51 Silver	Economy	12,99	12.340500
2	2	Contoso 512MB MP3 Player E51 Blue	Economy	12,99	12.340500
3	3	Contoso 1G MP3 Player E100 White	Economy	14,52	13.794000
4	4	Contoso 2G MP3 Player E200 Silver	Economy	21,57	20.491500
5	5	Contoso 2G MP3 Player E200 Red	Economy	21,57	20.491500
6	6	Contoso 2G MP3 Player E200 Black	Economy	21,57	20.491500
7	7	Contoso 2G MP3 Player E200 Blue	Economy	21,57	20.491500

OP-3U4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 2.517 linhas

GABARITOS

1.b

Questão 1.b

b) Faça uma adaptação no código para que os % de desconto de 5%, 7% e 9% sejam facilmente modificados (dica: utilize variáveis).

SQLQuery3.sql - LA...4522DD\55199 (57))* | Módulo 11 - Lista...U4522DD\55199 (52))*

```
-- Questão 1

-- Letra B

DECLARE @varDescEco FLOAT, @varDescReg FLOAT, @varDescDe FLOAT
SET @varDescEco = 0.05
SET @varDescReg = 0.07
SET @varDescDe = 0.09

SELECT
    ProductKey AS 'ID Produto',
    ProductName AS 'Nome Produto',
    ClassName AS 'Classe',
    UnitPrice AS 'Preço',
    CASE
        WHEN ClassName = 'Economy' THEN (1 - @varDescEco) * UnitPrice
        WHEN ClassName = 'Regular' THEN (1 - @varDescReg) * UnitPrice
        ELSE (1 - @varDescDe) * UnitPrice
    END AS '% Desconto'
FROM
    DimProduct
```

106 %

Resultados | Mensagens

	ID Produto	Nome Produto	Classe	Preço	% Desconto
1	1	Contoso 512MB MP3 Player E51 Silver	Economy	12,99	12,3405
2	2	Contoso 512MB MP3 Player E51 Blue	Economy	12,99	12,3405
3	3	Contoso 1G MP3 Player E100 White	Economy	14,52	13,794
4	4	Contoso 2G MP3 Player E200 Silver	Economy	21,57	20,4915
5	5	Contoso 2G MP3 Player E200 Red	Economy	21,57	20,4915
6	6	Contoso 2G MP3 Player E200 Black	Economy	21,57	20,4915
7	7	Contoso 2G MP3 Player E200 Blue	Economy	21,57	20,4915
8	8	Contoso 4G MP3 Player E400 Silver	Economy	59,99	56,9905
9	9	Contoso 4G MP3 Player E400 Black	Economy	59,99	56,9905

LAPTOP-3U4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 2.517 linhas

GABARITOS

2

Questão 2

Você ficou responsável pelo controle de produtos da empresa e deverá fazer uma análise da quantidade de produtos por Marca.

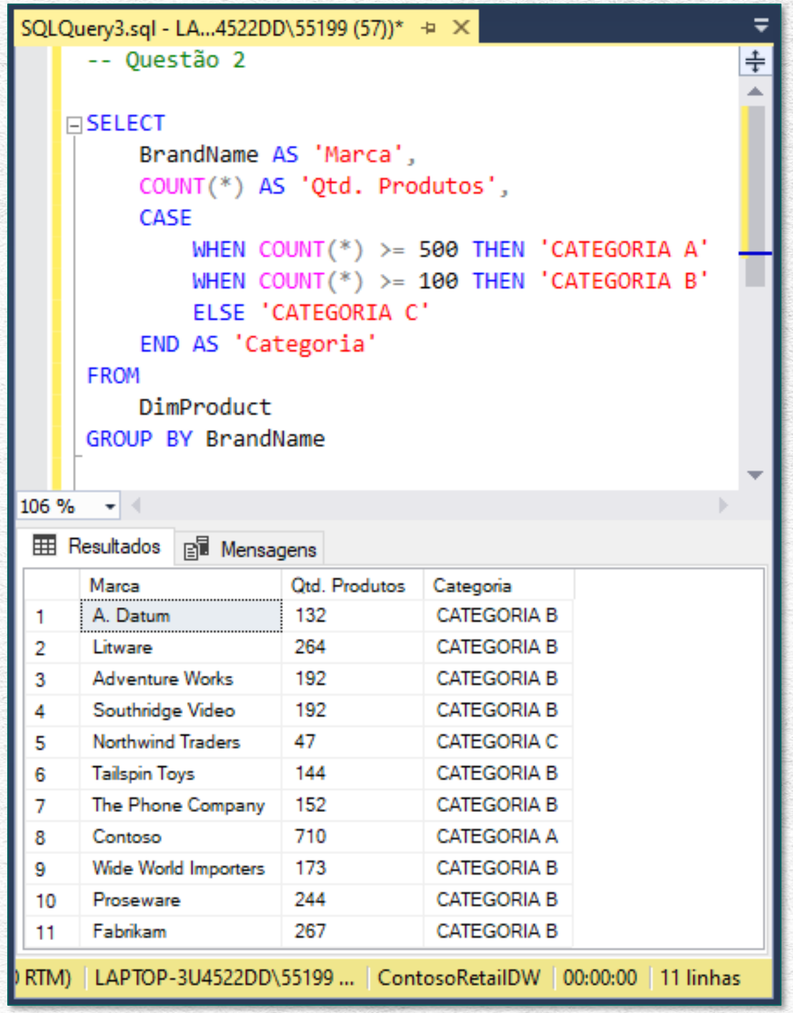
A divisão das marcas em categorias deverá ser a seguinte:

CATEGORIA A: Mais de 500 produtos

CATEGORIA B: Entre 100 e 500 produtos

CATEGORIA C: Menos de 100 produtos

Faça uma consulta à tabela DimProduct e retorne uma tabela com um agrupamento de Total de Produtos por Marca, além da coluna de Categoria, conforme a regra acima.



	Marca	Qtd. Produtos	Categoria
1	A. Datum	132	CATEGORIA B
2	Litware	264	CATEGORIA B
3	Adventure Works	192	CATEGORIA B
4	Southridge Video	192	CATEGORIA B
5	Northwind Traders	47	CATEGORIA C
6	Tailspin Toys	144	CATEGORIA B
7	The Phone Company	152	CATEGORIA B
8	Contoso	710	CATEGORIA A
9	Wide World Importers	173	CATEGORIA B
10	Proseware	244	CATEGORIA B
11	Fabrikam	267	CATEGORIA B

GABARITOS

3

Questão 3

Será necessário criar uma categorização de cada loja da empresa considerando a quantidade de funcionários de cada uma. A lógica a ser seguida será a lógica abaixo:

EmployeeCount >= 50; 'Acima de 50 funcionários'

EmployeeCount >= 40; 'Entre 40 e 50 funcionários'

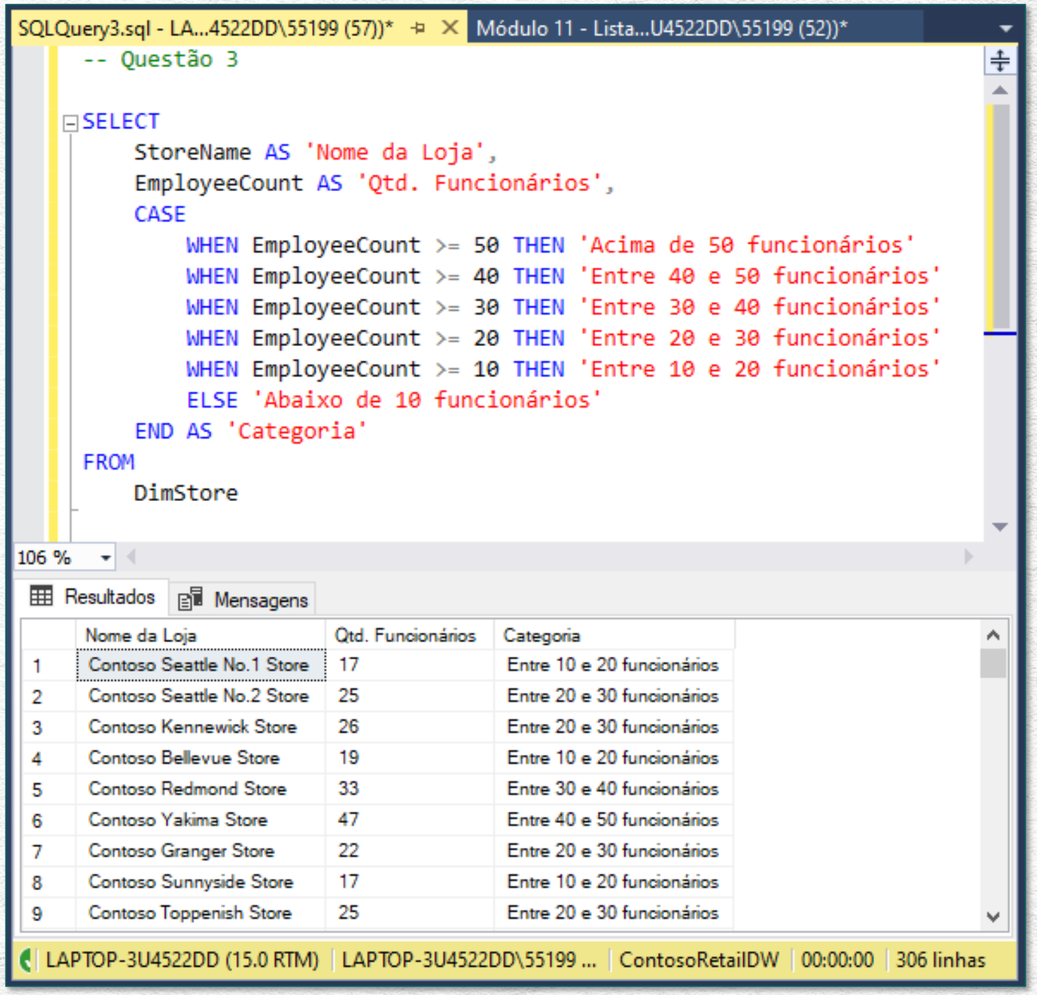
EmployeeCount >= 30; 'Entre 30 e 40 funcionários'

EmployeeCount >= 20; 'Entre 20 e 30 funcionários'

EmployeeCount >= 40; 'Entre 10 e 20 funcionários'

Caso contrário: 'Abaixo de 10 funcionários'

Faça uma consulta à tabela DimStore que retorne as seguintes informações: StoreName, EmployeeCount e a coluna de categoria, seguindo a regra acima.



GABARITOS

4 Questão 4

O setor de logística deverá realizar um transporte de carga dos produtos que estão no depósito de Seattle para o depósito de Sunnyside.

Não se tem muitas informações sobre os produtos que estão no depósito, apenas se sabe que existem **100 exemplares de cada Subcategoria**. Ou seja, 100 laptops, 100 câmeras digitais, 100 ventiladores, e assim vai.

O gerente de logística definiu que **os produtos serão transportados por duas rotas distintas**. Além disso, a divisão dos produtos em cada uma das rotas será feita de acordo com as subcategorias (ou seja, todos os produtos de uma mesma subcategoria serão transportados pela mesma rota):

Rota 1: As subcategorias que tiverem uma soma total menor que 1000 kg deverão ser transportados pela Rota 1.

Rota 2: As subcategorias que tiverem uma soma total maior ou igual a 1000 kg deverão ser transportados pela Rota 2.

Você deverá realizar uma consulta à tabela DimProduct e fazer essa divisão das subcategorias por cada rota. Algumas dicas:

- **Dica 1**: A sua consulta deverá ter um total de 3 colunas: **Nome da Subcategoria**, **Peso Total** e **Rota**.
- **Dica 2**: Como não se sabe quais produtos existem no depósito, apenas que existem 100 exemplares de cada subcategoria, você deverá descobrir o peso médio de cada subcategoria e multiplicar essa média por 100, de forma que você descubra aproximadamente qual é o peso total dos produtos por subcategoria.
- **Dica 3**: Sua resposta final deverá ter um JOIN e um GROUP BY.

SQLQuery3.sql - LA...4522DD\55199 (57))* | Módulo 11 - Lista...U4522DD\55199 (52))*

```
SELECT
    ProductSubcategoryName AS 'Nome Subcategoria',
    ROUND(AVG(Weight) * 100, 2) AS 'Peso Total',
    CASE
        WHEN ROUND(AVG(Weight) * 100, 2) >= 1000 THEN 'Rota 2'
        ELSE 'Rota 1'
    END AS 'Rota'
FROM
    DimProduct
INNER JOIN DimProductSubcategory
    ON DimProduct.ProductSubcategoryKey = DimProductSubcategory.ProductSubcategoryKey
GROUP BY ProductSubcategoryName
```

106 %

Resultados | Mensagens

	Nome Subcategoria	Peso Total	Rota
1	Air Conditioners	8938,71	Rota 2
2	Bluetooth Headphones	520,02	Rota 1
3	Boxed Games	140	Rota 1
4	Camcorders	215,15	Rota 1
5	Cameras & Camcorders Accessories	612,91	Rota 1
6	Car Video	492,17	Rota 1
7	Cell phones Accessories	434,44	Rota 1
8	Coffee Machines	2356,49	Rota 2
9	Computers Accessories	1575,21	Rota 2

Consulta executada co... | LAPTOP-3U4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 32 linhas

GABARITOS

5

Questão 5

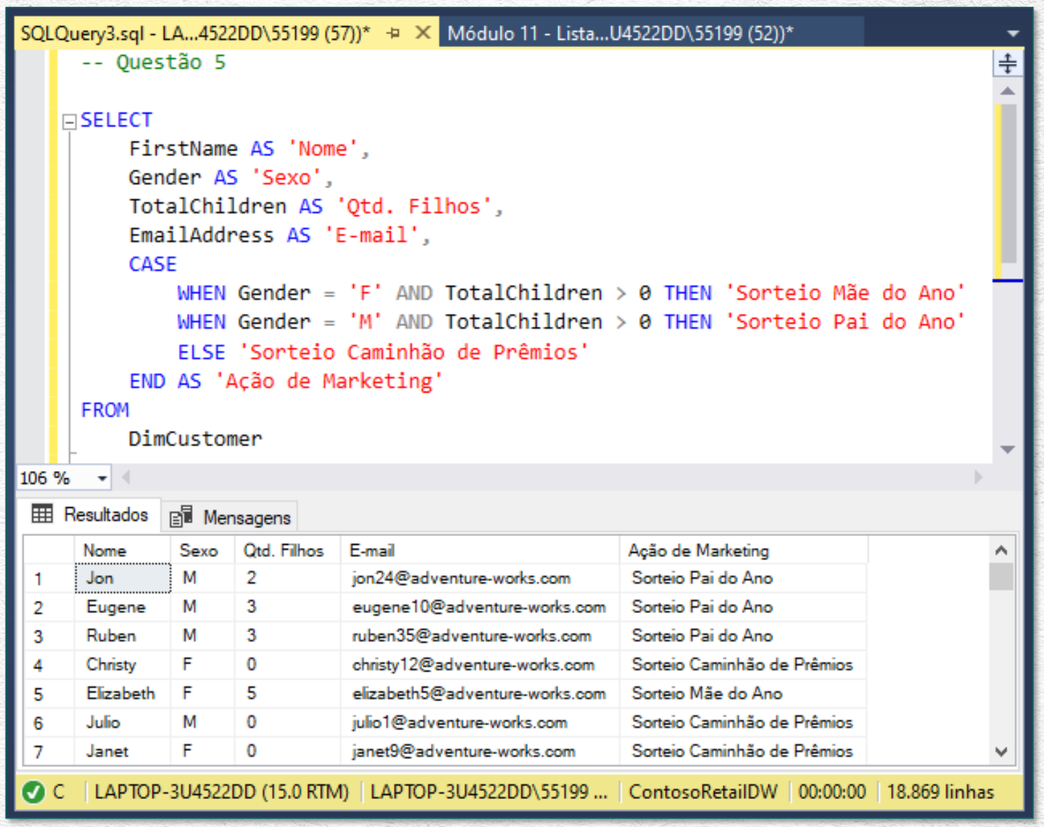
O setor de marketing está com algumas ideias de ações para alavancar as vendas em 2021. Uma delas consiste em realizar sorteios entre os clientes da empresa.

Este sorteio será dividido em categorias:

'Sorteio Mãe do Ano': Nessa categoria vão participar todas as mulheres com filhos.
'Sorteio Pai do Ano': Nessa categoria vão participar todos os pais com filhos.
'Caminhão de Prêmios': Nessa categoria vão participar todas os demais clientes (homens e mulheres sem filhos).

Seu papel será realizar uma consulta à tabela DimCustomer e retornar as colunas:

- FirstName AS 'Nome'
- Gender AS 'Sexo'
- TotalChildren AS 'Qtd. Filhos'
- EmailAdress AS 'E-mail'
- Ação de Marketing: nessa coluna você deverá dividir os clientes de acordo com as categorias 'Sorteio Mãe do Ano', 'Sorteio Pai do Ano' e 'Caminhão de Prêmios'.



	Nome	Sexo	Qtd. Filhos	E-mail	Ação de Marketing
1	Jon	M	2	jon24@adventure-works.com	Sorteio Pai do Ano
2	Eugene	M	3	eugene10@adventure-works.com	Sorteio Pai do Ano
3	Ruben	M	3	ruben35@adventure-works.com	Sorteio Pai do Ano
4	Christy	F	0	christy12@adventure-works.com	Sorteio Caminhão de Prêmios
5	Elizabeth	F	5	elizabeth5@adventure-works.com	Sorteio Mãe do Ano
6	Julio	M	0	julio1@adventure-works.com	Sorteio Caminhão de Prêmios
7	Janet	F	0	janet9@adventure-works.com	Sorteio Caminhão de Prêmios

GABARITOS

6 Questão 6

Descubra qual é a loja que possui o maior tempo de atividade (em dias). Você deverá fazer essa consulta na tabela DimStore, e considerar a coluna OpenDate como referência para esse cálculo.

Atenção: lembre-se que **existem lojas que foram fechadas**.

SQLQuery3.sql - LA...4522DD\55199 (57))* | Módulo 11 - Lista...U4522DD\55199 (52))*

```
-- Questão 6

SELECT
    StoreName,
    OpenDate,
    CloseDate,
    CASE
        WHEN CloseDate IS NULL THEN DATEDIFF(DAY, OpenDate, GETDATE())
        ELSE DATEDIFF(DAY, OpenDate, CloseDate)
    END AS 'Dias de atividade'
FROM DimStore
```

106 %

Resultados | Mensagens

	StoreName	OpenDate	CloseDate	Dias de atividade
10	Contoso Wapato Store	2004-09-19 00:00:00.000	NULL	6990
11	Contoso Cle Elum Store	2004-10-20 00:00:00.000	NULL	6959
12	Contoso North Bend Store	2004-11-21 00:00:00.000	2009-04...	1603
13	Contoso Snoqualmie Store	2004-12-22 00:00:00.000	NULL	6896
14	Contoso Fall City Store	2006-01-23 00:00:00.000	NULL	6499
15	Contoso Renton Store	2004-02-24 00:00:00.000	NULL	7198
16	Contoso Everett Store	2004-01-25 00:00:00.000	NULL	7228
17	Contoso Spokane Store	2004-03-26 00:00:00.000	NULL	7167

Co... | LAPTOP-3U4522DD (15.0 RTM) | LAPTOP-3U4522DD\55199 ... | ContosoRetailDW | 00:00:00 | 306 linhas

MÓDULO 12

SQL

Views

O QUE VEREMOS NESTE MÓDULO

MÓDULO 12 VIEWS

01	Introdução	301
02	O que é uma VIEW?	302
03	Por que criar uma VIEW?	303
04	CREATE VIEW: Criando a primeira VIEW	304
05	ALTER VIEW: Alterando uma VIEW criada	305
06	DROP VIEW: Excluindo uma VIEW	306
07	EXERCÍCIOS	307
08	GABARITOS	310

Introdução

Até aqui vimos como criar diferentes consultas aos bancos de dados. Utilizamos comandos como o SELECT, GROUP BY, JOINs, etc para criar tabelas como a mostrada ao lado.

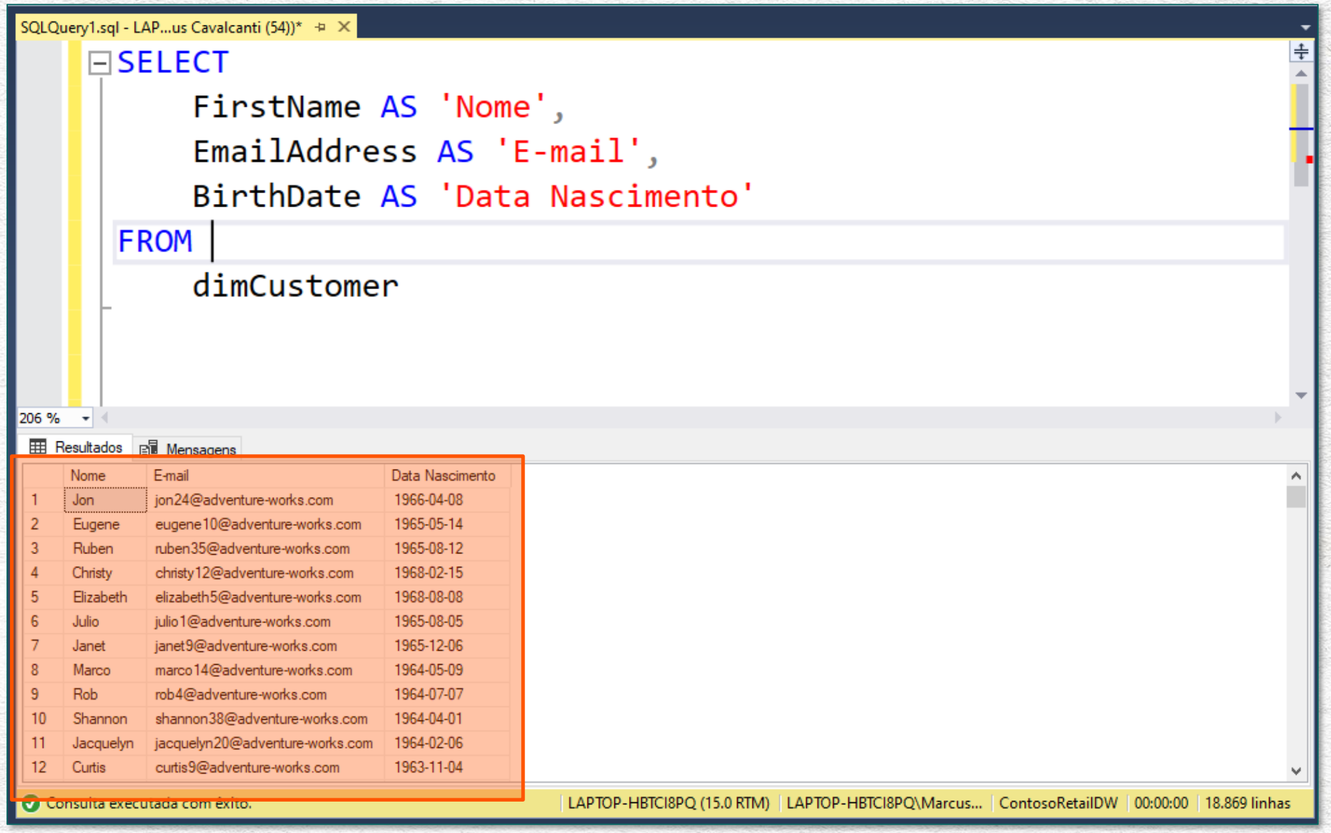
Mas pra onde foram todas essas tabelas que a gente criou? Elas estão em algum lugar?

A resposta é: elas não estão em nenhum lugar!

Tudo o que fizemos até agora foi apenas visualizar alguns dados das nossas tabelas do Banco de Dados, nada além disso. Quando executamos um SELECT e logo em seguida executamos outro SELECT, o resultado do primeiro é perdido.

Nenhuma das consultas que fizemos ficou salvo em algum lugar. Inclusive, diversas vezes precisamos criar as mesmas consultas, pois elas se perdem a cada novo SELECT, ou quando fechamos uma consulta e abrimos uma nova.

Existe uma solução pra gente conseguir salvar essas tabelas em algum lugar, e essa solução é a **View**.



O que é uma VIEW?

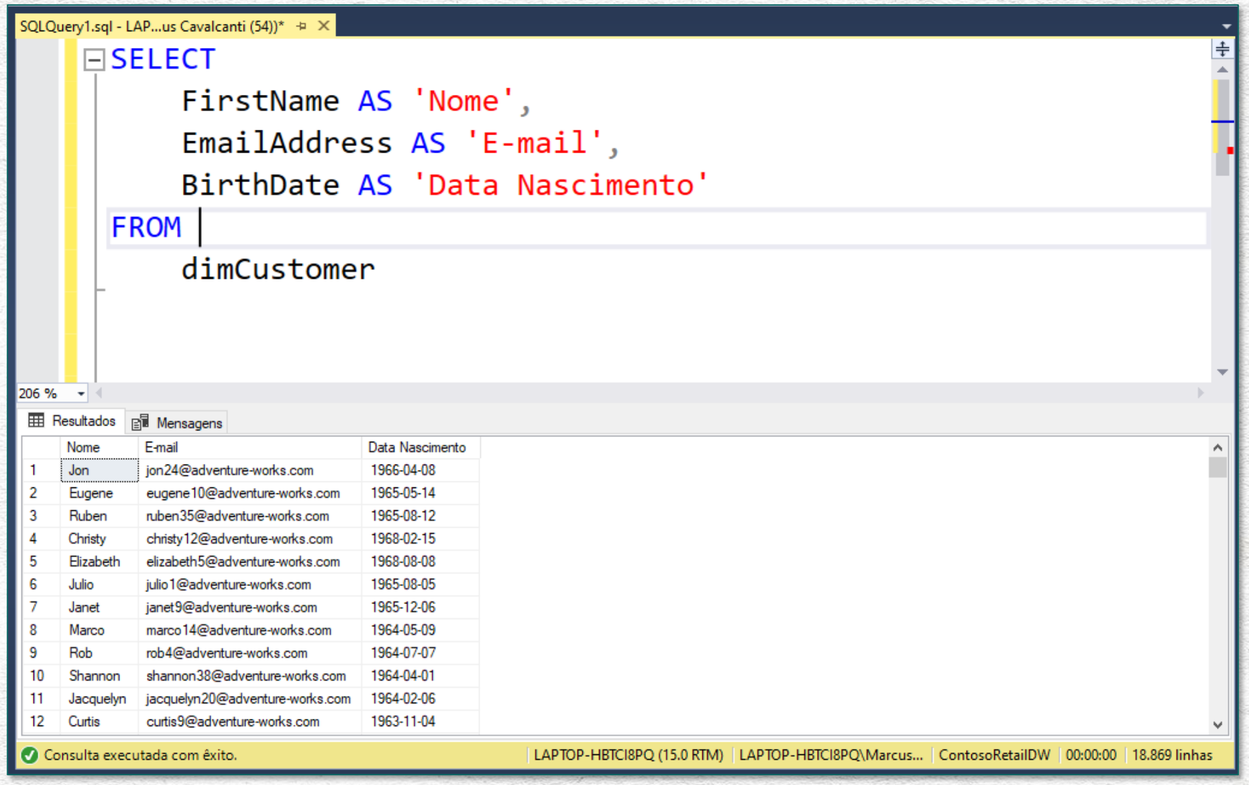
- Uma View (ou traduzindo, uma exibição), é uma tabela virtual criada a partir de uma consulta a uma ou mais tabelas (ou até mesmo de outras Views) no banco de dados.

- Ela contém linhas e colunas assim como uma tabela real. Nela podemos utilizar comandos como o JOIN, WHERE e outras funções.

- Sempre mostra resultados atualizados dos dados, ou seja, uma vez criada uma View, caso haja alterações no Banco de Dados, as Views são atualizadas automaticamente.

- Caso o servidor seja desligado (ou o SSMS fechado), a View continua armazenada no sistema.

Através de uma View, conseguimos armazenar o resultado de um SELECT (como por exemplo, a tabela à direita) e acessar sempre que precisar, como se fosse uma tabela, com a vantagem de não precisar criar esse SELECT do zero.



	Nome	E-mail	Data Nascimento
1	Jon	jon24@adventure-works.com	1966-04-08
2	Eugene	eugene10@adventure-works.com	1965-05-14
3	Ruben	ruben35@adventure-works.com	1965-08-12
4	Christy	christy12@adventure-works.com	1968-02-15
5	Elizabeth	elizabeth5@adventure-works.com	1968-08-08
6	Julio	julio1@adventure-works.com	1965-08-05
7	Janet	janet9@adventure-works.com	1965-12-06
8	Marco	marco14@adventure-works.com	1964-05-09
9	Rob	rob4@adventure-works.com	1964-07-07
10	Shannon	shannon38@adventure-works.com	1964-04-01
11	Jacquelyn	jacquelyn20@adventure-works.com	1964-02-06
12	Curtis	curtis9@adventure-works.com	1963-11-04

Por que criar uma VIEW?

São muitas as vantagens de uma View. Abaixo temos algumas das principais:

Reutilização

Sempre que necessário, podemos consultar aquela View, pois ela fica armazenada no sistema.

Segurança

Ao criar uma View, estamos ocultando linhas ou colunas da tabela original do banco de dados. Desta forma, apenas algumas informações relevantes serão visualizadas na View.

Ganho de tempo

Quando criamos Views, estamos poupando o tempo de recriar vários SELECTs, o que aumenta a produtividade.

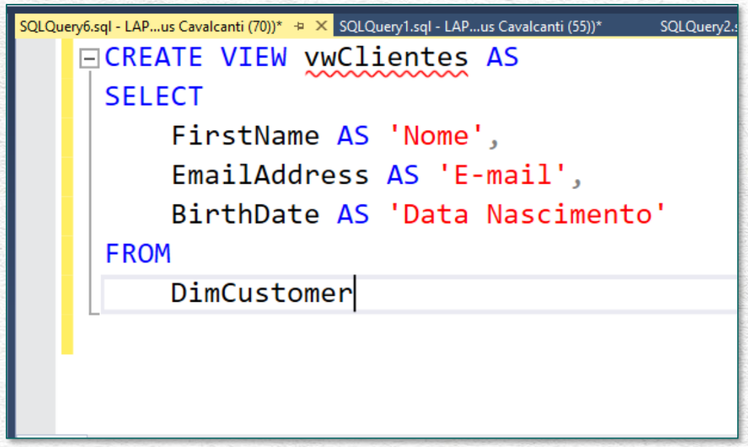
CREATE VIEW: Criando a primeira VIEW

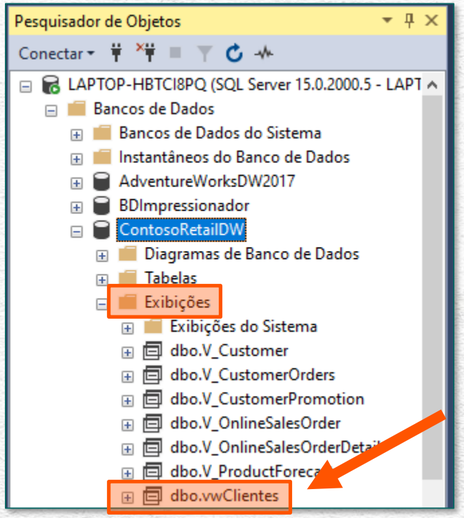
Para criar uma View, utilizamos o comando **CREATE VIEW**. Na imagem abaixo temos a estrutura padrão:

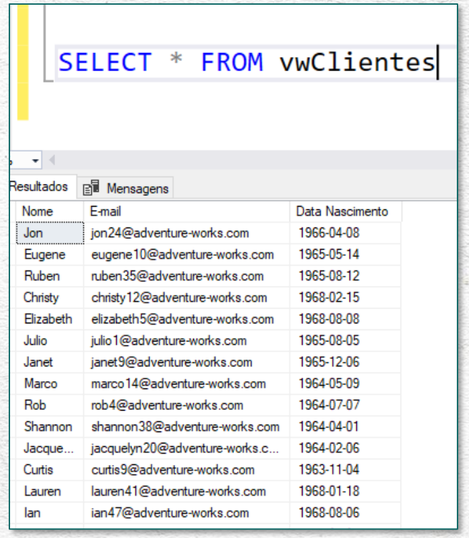
```
CREATE VIEW nome_da_view AS
SELECT
        Coluna1,
        Coluna2,
        Coluna3
FROM
        Tabela
```

Uma vez criada, a View ficará armazenada no banco de dados, na pasta de **Exibições**.

E para selecionar essa View, utilizamos o comando SELECT.







Nome	E-mail	Data Nascimento
Jon	jon24@adventure-works.com	1966-04-08
Eugene	eugene10@adventure-works.com	1965-05-14
Ruben	ruben35@adventure-works.com	1965-08-12
Christy	christy12@adventure-works.com	1968-02-15
Elizabeth	elizabeth5@adventure-works.com	1968-08-08
Julio	julio1@adventure-works.com	1965-08-05
Janet	janet9@adventure-works.com	1965-12-06
Marco	marco14@adventure-works.com	1964-05-09
Rob	rob4@adventure-works.com	1964-07-07
Shannon	shannon38@adventure-works.com	1964-04-01
Jacque...	jacquelyn20@adventure-works.c...	1964-02-06
Curtis	curtis9@adventure-works.com	1963-11-04
Lauren	lauren41@adventure-works.com	1968-01-18
Ian	ian47@adventure-works.com	1968-08-06

ALTER VIEW: Alterando uma VIEW criada

Para alterar uma View criada, usamos o comando **ALTER VIEW**.

Na imagem ao lado, temos um exemplo. A vwClientes, criada anteriormente, foi alterada para considerar apenas os clientes do sexo feminino.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.

```
ALTER VIEW vwClientes AS
SELECT
    FirstName AS 'Nome',
    EmailAddress AS 'E-mail',
    BirthDate AS 'Data Nascimento'
FROM
    dimCustomer
WHERE Gender = 'F'

SELECT * FROM vwClientes
```

146 %

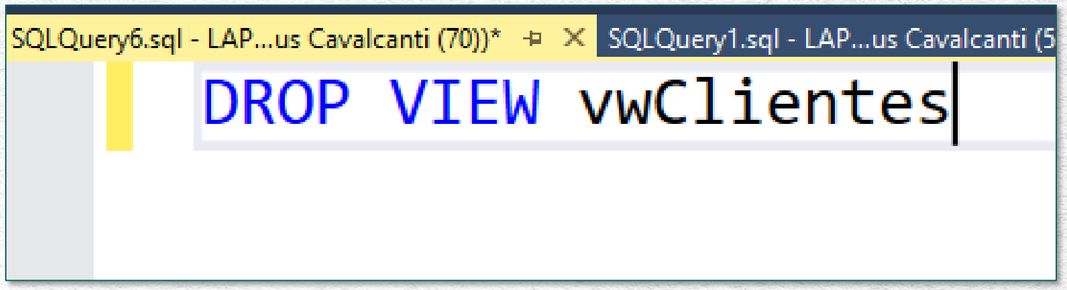
Resultados | Mensagens

	Nome	E-mail	Data Nascimento
1	Christy	christy12@adventure-works.com	1968-02-15
2	Elizabeth	elizabeth5@adventure-works.com	1968-08-08
3	Janet	janet9@adventure-works.com	1965-12-06
4	Rob	rob4@adventure-works.com	1964-07-07
5	Jacquelyn	jacquelyn20@adventure-works.com	1964-02-06
6	Lauren	lauren41@adventure-works.com	1968-01-18
7	Sydney	sydney23@adventure-works.com	1968-05-09

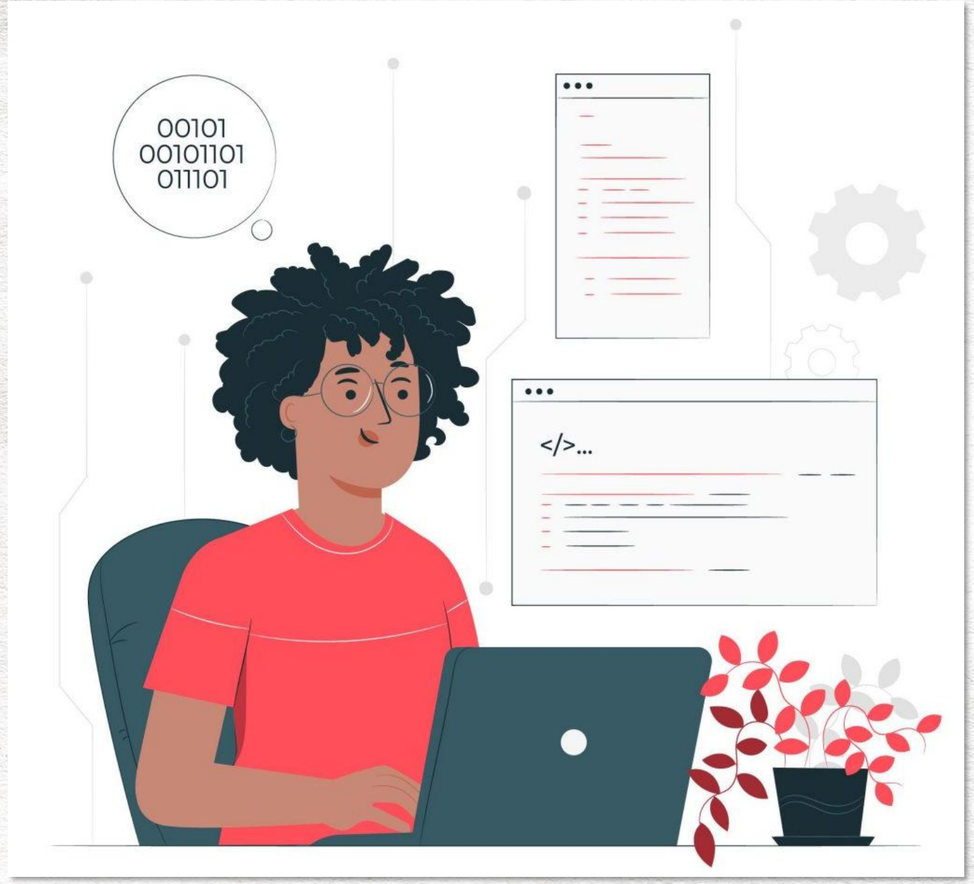
DROP VIEW: Excluindo uma VIEW

Para excluir uma View criada, usamos o comando **DROP VIEW**.

O comando é bem simples e é ilustrado na imagem ao lado.



EXERCÍCIOS



1 Questão 1

a. A partir da tabela DimProduct, crie uma View contendo as informações de ProductName, ColorName, UnitPrice e UnitCost, da tabela DimProduct. Chame essa View de vwProdutos.

b. A partir da tabela DimEmployee, crie uma View mostrando FirstName, BirthDate, DepartmentName. Chame essa View de vwFuncionarios.

c. A partir da tabela DimStore, crie uma View mostrando StoreKey, StoreName e OpenDate. Chame essa View de vwLojas.

2 Questão 2

Crie uma View contendo as informações de Nome Completo (FirstName + LastName), Gênero (por extenso), E-mail e Renda Anual (formatada com R$).

Utilize a tabela DimCustomer. Chame essa View de vwClientes.

EXERCÍCIOS

3

Questão 3

a. A partir da tabela DimStore, crie uma View que considera apenas as lojas ativas. Faça um SELECT de todas as colunas. Chame essa View de vwLojasAtivas.

b. A partir da tabela DimEmployee, crie uma View de uma tabela que considera apenas os funcionários da área de Marketing. Faça um SELECT das colunas: FirstName, EmailAddress e DepartmentName. Chame essa de vwFuncionariosMkt.

c. Crie uma View de uma tabela que considera apenas os produtos das marcas Contoso e Litware. Além disso, a sua View deve considerar apenas os produtos de cor Silver. Faça um SELECT de todas as colunas da tabela DimProduct. Chame essa View de vwContosoLitwareSilver.

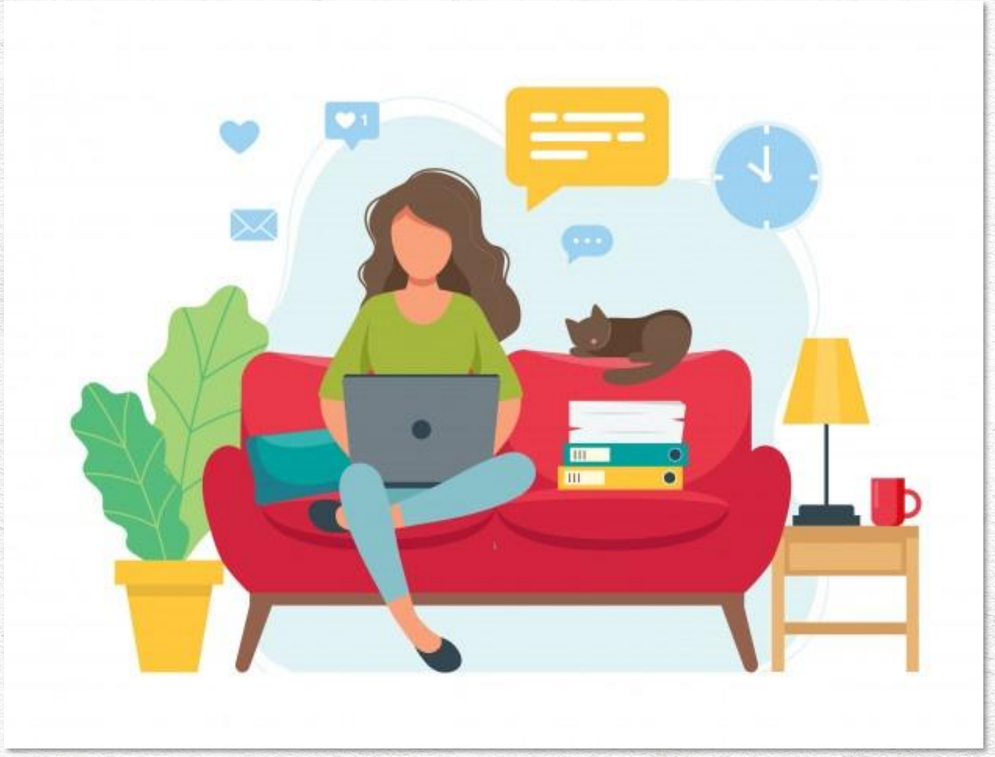
4

Questão 4

Crie uma View que seja o resultado de um agrupamento da tabela FactSales. Este agrupamento deve considerar o SalesQuantity (Quantidade Total Vendida) por Nome do Produto. Chame esta View de vwTotalVendidoProdutos.

OBS: Para isso, você terá que utilizar um JOIN para relacionar as tabelas FactSales e DimProduct.

EXERCÍCIOS



5

Questão 5

Faça as seguintes alterações nas tabelas da questão 1.

a. Na View criada na letra a da questão 1, adicione a coluna de BrandName.

b. Na View criada na letra b da questão 1, faça um filtro e considere apenas os funcionários do sexo feminino.

c. Na View criada na letra c da questão 1, faça uma alteração e filtre apenas as lojas ativas.

6

Questão 6

a. Crie uma View que seja o resultado de um agrupamento da tabela DimProduct. O resultado esperado da consulta deverá ser o total de produtos por marca. Chame essa View de vw_6a.

b. Altere a View criada no exercício anterior, adicionando o peso total por marca. Atenção: sua View final deverá ter então 3 colunas: Nome da Marca, Total de Produtos e Peso Total.

c. Exclua a View vw_6a.

GABARITOS

1

Questão 1

a. A partir da tabela DimProduct, crie uma View contendo as informações de ProductName, ColorName, UnitPrice e UnitCost, da tabela DimProduct. Chame essa View de vwProdutos.

b. A partir da tabela DimEmployee, crie uma View mostrando FirstName, BirthDate, DepartmentName. Chame essa View de vwFuncionarios.

c. A partir da tabela DimStore, crie uma View mostrando StoreKey, StoreName e OpenDate. Chame essa View de vwLojas.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 1
--a)
CREATE VIEW vw_1a AS
SELECT
    ProductName AS 'Nome do Produto',
    ColorName AS 'Cor',
    UnitPrice AS 'Preço Unitário',
    UnitCost AS 'Custo Unitário'
FROM
    DimProduct

--b)
CREATE VIEW vw_1b AS
SELECT
    FirstName AS 'Nome',
    BirthDate AS 'Data de Nascimento',
    DepartmentName AS 'Departamento'
FROM
    DimEmployee

--c)
CREATE VIEW vw_1c AS
SELECT
    StoreKey AS 'ID Loja',
    StoreName  AS 'Nome da Loja',
    OpenDate AS 'Data de Abertura'
FROM
    DimStore
```

GABARITOS

2

Questão 2

Crie uma View contendo as informações de Nome Completo (FirstName + LastName), Gênero (por extenso), E-mail e Renda Anual (formatada com R$).

Utilize a tabela DimCustomer. Chame essa View de vwClientes.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
CREATE VIEW vw_2 AS
SELECT
    CONCAT(FirstName, ' ', LastName) AS 'Nome Completo',
    REPLACE(REPLACE(Gender, 'M', 'Masculino'), 'F', 'Feminino') AS 'Gênero',
    EmailAddress AS 'E-mail',
    FORMAT(YearlyIncome, 'C') AS 'Renda Anual'
FROM
    DimCustomer
```

GABARITOS

3

Questão 3

a. A partir da tabela DimStore, crie uma View que considera apenas as lojas ativas. Faça um SELECT de todas as colunas. Chame essa View de vwLojasAtivas.

b. A partir da tabela DimEmployee, crie uma View de uma tabela que considera apenas os funcionários da área de Marketing. Faça um SELECT das colunas: FirstName, EmailAddress e DepartmentName. Chame essa de vwFuncionariosMkt.

c. Crie uma View de uma tabela que considera apenas os produtos das marcas Contoso e Litware. Além disso, a sua View deve considerar apenas os produtos de cor Silver. Faça um SELECT de todas as colunas da tabela DimProduct. Chame essa View de vwContosoLitwareSilver.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 3
--a)
CREATE VIEW vw_3a AS
SELECT
    *
FROM
    DimStore
WHERE Status = 'On'

--b)
CREATE VIEW vw_3b AS
SELECT
    FirstName,
    EmailAddress,
    DepartmentName
FROM
    DimEmployee
WHERE DepartmentName = 'Marketing'

--c)
CREATE VIEW vw_3c AS
SELECT
    *
FROM
    DimProduct
WHERE BrandName IN ('Contoso', 'Litware') AND ColorName = 'Silver'
```

GABARITOS

4

Questão 4

Crie uma View que seja o resultado de um agrupamento da tabela FactSales. Este agrupamento deve considerar o SalesQuantity (Quantidade Total Vendida) por Nome do Produto. Chame esta View de vwTotalVendidoProdutos.

OBS: Para isso, você terá que utilizar um JOIN para relacionar as tabelas FactSales e DimProduct.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 4

CREATE VIEW vw_4 AS
SELECT
    ProductName AS 'Nome do Produto',
    SUM(SalesQuantity) AS 'Qtd. Vendida'
FROM
    FactSales
INNER JOIN DimProduct
    ON FactSales.ProductKey = DimProduct.ProductKey
GROUP BY ProductName
```

GABARITOS

5

Questão 5

Faça as seguintes alterações nas tabelas da questão 1.

a. Na View criada na letra a da questão 1, adicione a coluna de BrandName.

b. Na View criada na letra b da questão 1, faça um filtro e considere apenas os funcionários do sexo feminino.

c. Na View criada na letra c da questão 1, faça uma alteração e filtre apenas as lojas ativas.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 5
--a)
ALTER VIEW vw_1a AS
SELECT
    ProductName AS 'Nome do Produto',
    BrandName AS 'Marca',
    ColorName AS 'Cor',
    UnitPrice AS 'Preço Unitário',
    UnitCost AS 'Custo Unitário'
FROM
    DimProduct

--b)
ALTER VIEW vw_1b AS
SELECT
    FirstName AS 'Nome',
    BirthDate AS 'Data de Nascimento',
    DepartmentName AS 'Departamento'
FROM
    DimEmployee
WHERE Gender = 'F'

--c)
ALTER VIEW vw_1c AS
SELECT
    StoreKey AS 'ID Loja',
    StoreName  AS 'Nome da Loja',
    OpenDate AS 'Data de Abertura'
FROM
    DimStore
WHERE Status = 'On'
```

GABARITOS

6

Questão 6

a. Crie uma View que seja o resultado de um agrupamento da tabela DimProduct. O resultado esperado da consulta deverá ser o total de produtos por marca. Chame essa View de vw_6a.

b. Altere a View criada no exercício anterior, adicionando o peso total por marca. Atenção: sua View final deverá ter então 3 colunas: Nome da Marca, Total de Produtos e Peso Total.

c. Exclua a View vw_6a.

SQLQuery9.sql - LAP...us Cavalcanti (54))*

```sql
--Questão 6
--a)
CREATE VIEW vw_6a AS
SELECT
    BrandName AS 'Marca',
    COUNT(*) AS 'Total'
FROM
    DimProduct
GROUP BY BrandName

--b)
ALTER VIEW vw_6a AS
SELECT
    BrandName AS 'Marca',
    COUNT(*) AS 'Total',
    SUM(Weight) AS 'Peso'
FROM
    DimProduct
GROUP BY BrandName

--c)
DROP VIEW vw_6a
```

MÓDULO 13

SQL

CRUD

O QUE VEREMOS NESTE MÓDULO

MÓDULO 13 CRUD

01	O que é CRUD?	318
02	Lembrando o que é um Banco de Dados...	319
03	Por que um Banco de Dados é importante	320
04	Como criar um Banco de Dados	321
05	Como criar uma tabela	323
06	CREATE e DROP DATABASE	331
07	USE DATABASE	332
08	CREATE TABLE	333
09	INSERT INTO	334
10	Existe ordem para adicionar dados no INSERT?	335
11	UPDATE	336
12	DELETE	337
13	Código para criação de uma tabela exemplo	338
14	ALTER TABLE: Adicionar coluna	339
15	ALTER TABLE: Alterando tipo de dados	340
16	ALTER TABLE: Excluindo uma coluna de uma tabela	341
17	EXERCÍCIOS	342
18	GABARITOS	345

O que é CRUD?

Operações **CRUD** são operações que conseguimos fazer em um Banco de Dados. Essa sigla significa o seguinte:

CREATE

- Permite criar Bancos de Dados, Tabelas ou Exibições (Views)

READ

- Permite ler os dados do banco de dados. Basicamente foi o que mais fizemos no Curso, através do SELECT.

UPDATE

- Permite atualizar os dados do banco de dados, tabelas ou views.

DELETE

- Permite deletar dados de um banco de dados, tabelas ou views.

No módulo anterior, vimos como criar Views (exibições). Essas exibições não são entendidas exatamente como tabelas do banco de dados.

Agora o que vamos fazer é aprender como criar tabelas propriamente ditas,

Lembrando o que é um Banco de Dados...

Um banco de dados é composto por um conjunto de tabelas (vide exemplo Contoso).

As tabelas armazenam dados em linhas e colunas.

ProductKey	ProductName	BrandName	UnitPrice
1	Contoso 512MB MP3 Player E51 Silver	Contoso	12,99

Uma linha abrange várias colunas que juntas descrevem alguma característica de um objeto. Desta forma, uma coluna é como se fosse uma categoria.

Portanto, cada objeto (registro/record) possui características que pertencem a essas categorias.

Agora o que vamos fazer é aprender como criar tabelas propriamente ditas,

Por que um Banco de Dados é importante

Como dito anteriormente, um Banco de Dados será usado para guardar todas as tabelas criadas. É importante entender que as tabelas de um banco de dados precisam ter algum tipo de relação.

Tomando como exemplo o Banco de Dados Contoso, todas as tabelas ali dentro têm algum tipo de relação. Não faria sentido, por exemplo, a gente criar uma tabela que contivesse informações de roupas da Riachuelo, sendo que é uma informação que não tem relação nenhuma com a Contoso.

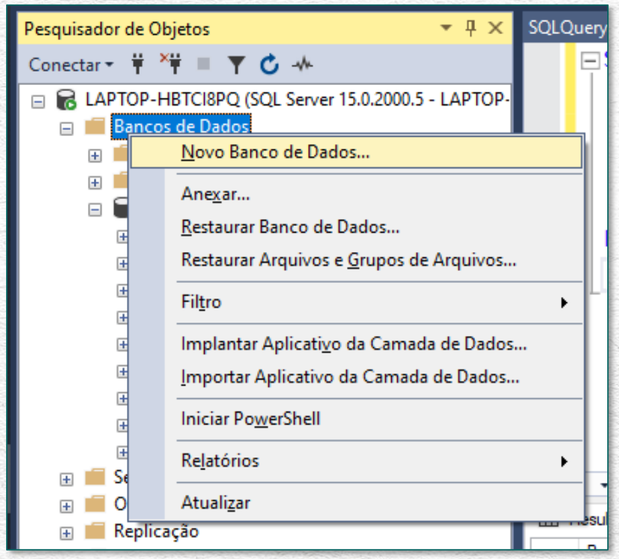
Como criar um Banco de Dados

CREATE DATABASE

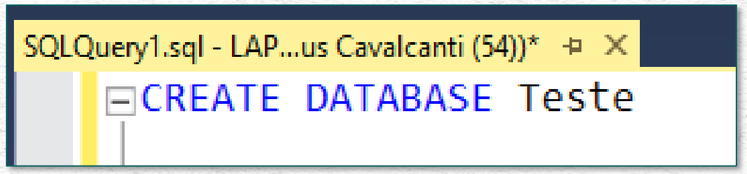
Vimos no módulo anterior como criar VIEWS, usando o comando CREATE VIEW. Para criar um Banco de Dados, a lógica é semelhante. Isso vale para tabelas.

Existem 2 formas de criar um Banco de Dados. Na **primeira opção**, podemos criar de forma manual clicando com o botão direito em cima da pasta Banco de Dados, e depois em **Novo Banco de Dados**... A **segunda opção** seria através de um código simples em SQL.

Manualmente

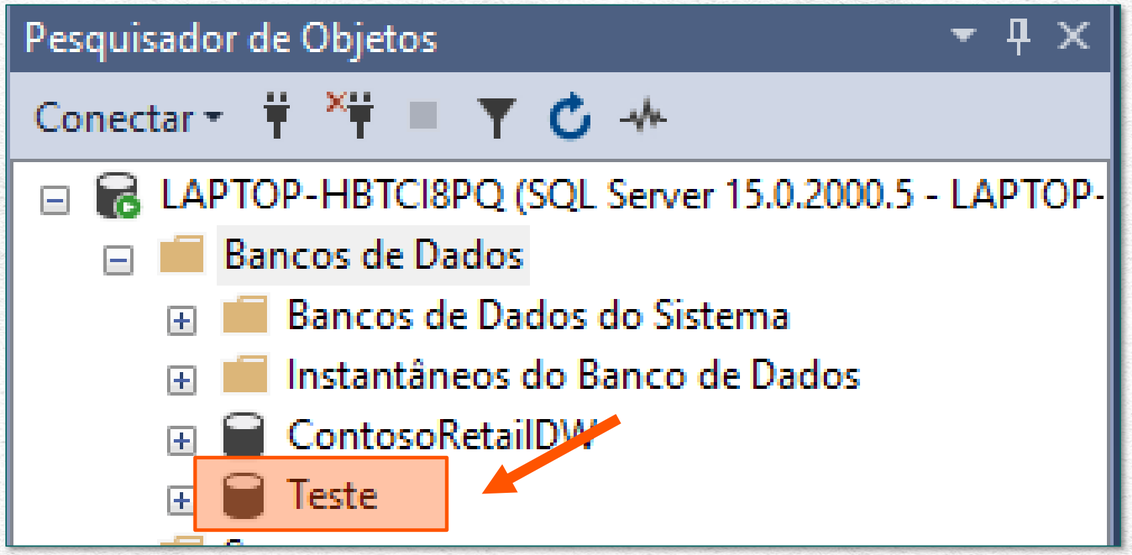


Usando SQL



Como criar um Banco de Dados

Uma vez criado, o Banco de Dados aparecerá da seguinte forma:



Como criar uma tabela

Agora que temos o nosso Banco de Dados, vamos começar a adicionar tabelas dentro dele. Mas antes, alguns pontos importantes que devemos ter em mente ao criar uma nova tabela:

1

Cada coluna da tabela vai ter um mesmo tipo de informação. Ou seja, uma coluna de Nome não pode conter outras informações além de nomes. O mesmo vale para qualquer outra coluna: é necessário seguir um padrão no tipo de informação que temos naquela coluna, nunca misturar!

CustomerKey	FirstName	BirthDate	YearlyIncome	UpdateDate
1	Jon	1966-04-08	90000,00	2009-10-01 00:00:00.000
2	Eugene	1965-05-14	60000,00	2009-10-01 00:00:00.000
3	Ruben	1965-08-12	60000,00	2009-10-01 00:00:00.000
4	Christy	1968-02-15	70000,00	2009-10-01 00:00:00.000
5	Elizabeth	1968-08-08	80000,00	2009-10-01 00:00:00.000
6	Julio	1965-08-05	70000,00	2009-10-01 00:00:00.000
7	Janet	1965-12-06	70000,00	2009-10-01 00:00:00.000
8	Marco	1964-05-09	60000,00	2009-10-01 00:00:00.000
9	Rob	1964-07-07	60000,00	2009-10-01 00:00:00.000
10	Shannon	1964-04-01	70000,00	2009-10-01 00:00:00.000

Como criar uma tabela

Agora que temos o nosso Banco de Dados, vamos começar a adicionar tabelas dentro dele. Mas antes, alguns pontos importantes que devemos ter em mente ao criar uma nova tabela:

2

As tabelas criadas devem ter uma estrutura particular: para adicionar uma nova informação nessa tabela (um novo cliente) adicionamos sempre uma nova linha! Se na sua tabela você adiciona uma nova coluna sempre que precisa adicionar um novo dado, reveja a sua estrutura.

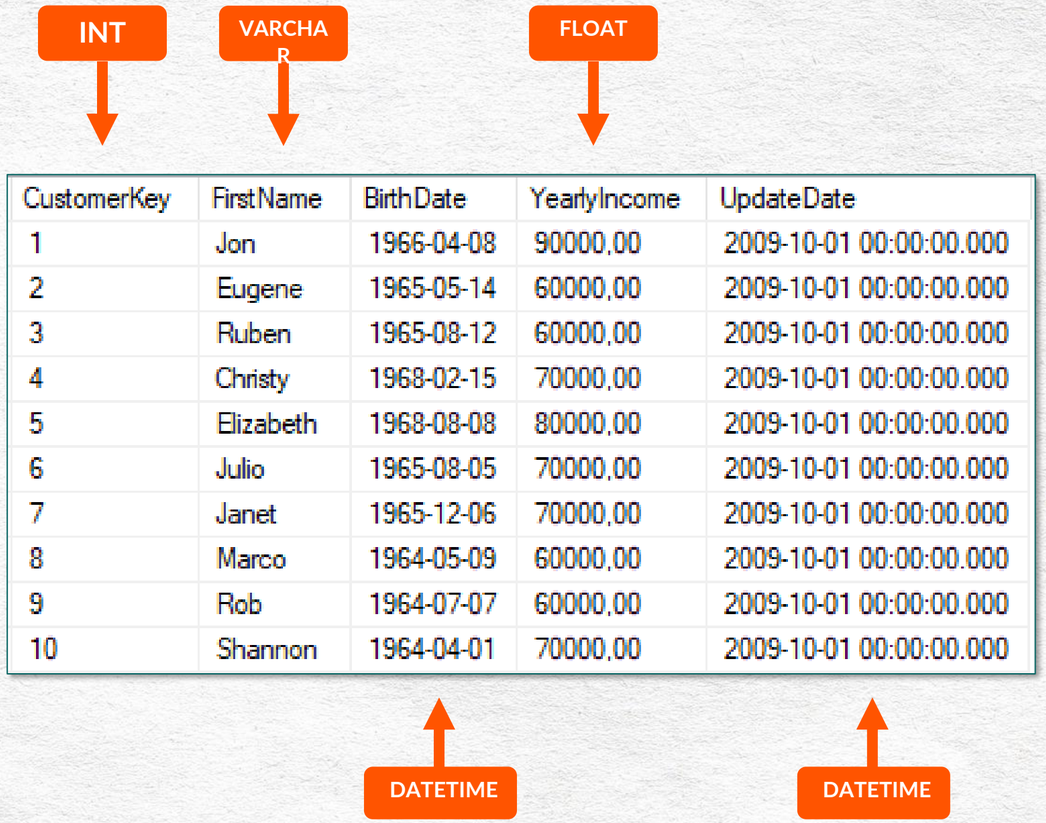
CustomerKey	FirstName	BirthDate	YearlyIncome	UpdateDate
1	Jon	1966-04-08	90000,00	2009-10-01 00:00:00.000
2	Eugene	1965-05-14	60000,00	2009-10-01 00:00:00.000
3	Ruben	1965-08-12	60000,00	2009-10-01 00:00:00.000
4	Christy	1968-02-15	70000,00	2009-10-01 00:00:00.000
5	Elizabeth	1968-08-08	80000,00	2009-10-01 00:00:00.000
6	Julio	1965-08-05	70000,00	2009-10-01 00:00:00.000
7	Janet	1965-12-06	70000,00	2009-10-01 00:00:00.000
8	Marco	1964-05-09	60000,00	2009-10-01 00:00:00.000
9	Rob	1964-07-07	60000,00	2009-10-01 00:00:00.000
10	Shannon	1964-04-01	70000,00	2009-10-01 00:00:00.000
11	Jacquelyn	1964-02-06	70000,00	2009-10-01 00:00:00.000

Como criar uma tabela

Agora que temos o nosso Banco de Dados, vamos começar a adicionar tabelas dentro dele. Mas antes, alguns pontos importantes que devemos ter em mente ao criar uma nova tabela:

3

Cada coluna vai ter um mesmo tipo de dado. Ou seja, na coluna de ID, sempre teremos um número; na coluna de Nome, sempre teremos textos, na coluna de data, sempre datas, e assim vai.



CustomerKey	FirstName	BirthDate	YearlyIncome	UpdateDate
1	Jon	1966-04-08	90000,00	2009-10-01 00:00:00.000
2	Eugene	1965-05-14	60000,00	2009-10-01 00:00:00.000
3	Ruben	1965-08-12	60000,00	2009-10-01 00:00:00.000
4	Christy	1968-02-15	70000,00	2009-10-01 00:00:00.000
5	Elizabeth	1968-08-08	80000,00	2009-10-01 00:00:00.000
6	Julio	1965-08-05	70000,00	2009-10-01 00:00:00.000
7	Janet	1965-12-06	70000,00	2009-10-01 00:00:00.000
8	Marco	1964-05-09	60000,00	2009-10-01 00:00:00.000
9	Rob	1964-07-07	60000,00	2009-10-01 00:00:00.000
10	Shannon	1964-04-01	70000,00	2009-10-01 00:00:00.000

Como criar uma tabela

Para criar as tabelas nos Bancos de Dados, vamos utilizar as operações CRUD da seguinte maneira:

1 CREATE TABLE

O comando CREATE TABLE vai nos permitir criar uma nova tabela. Ao lado, temos um exemplo da estrutura para utilização do comando. É neste momento que declaramos o tipo de cada dado.

```
CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(50),
    data_validade DATETIME,
    preco_produto FLOAT
)
```

Como criar uma tabela

Para criar as tabelas nos Bancos de Dados, vamos utilizar as operações CRUD da seguinte maneira:

2

INSERT INTO

Será através do comando INSERT INTO que vamos adicionar novos valores em nossa tabela. Além deste comando, precisaremos também do VALUES para especificar os valores adicionados.

```
INSERT INTO Produtos(id_produto, nome_produto, data_validade, preco_produto)
VALUES
    (1, 'Arroz', '2021-12-31', 22.50),
    (2, 'Feijão', '2021-12-31', 8.99)
```

Como criar uma tabela

Para criar as tabelas nos Bancos de Dados, vamos utilizar as operações CRUD da seguinte maneira:

3 UPDATE

Através do comando UPDATE conseguiremos atualizar um valor dentro de uma tabela. Geralmente esse comando é utilizado em conjunto com o WHERE, que vai nos possibilitar identificar a linha onde vamos fazer a alteração.

```
UPDATE Produtos
SET nome_produto = 'Macarrão'
WHERE id_produto = 3
```

Como criar uma tabela

Para criar as tabelas nos Bancos de Dados, vamos utilizar as operações CRUD da seguinte maneira:

4

DELETE

O comando DELETE é o que vai permitir a exclusão de dados em uma tabela. Observe que ainda não estamos falando de exclusão de tabelas, e sim exclusão de dados dentro de uma tabela.

```
DELETE
FROM Produtos
WHERE id_produto = 3
```

Como criar uma tabela

Para criar as tabelas nos Bancos de Dados, vamos utilizar as operações CRUD da seguinte maneira:

5

DROP TABLE

O comando DROP TABLE permite a exclusão definitiva de uma tabela dentro de um Banco de Dados.

```
DROP TABLE Produtos
```

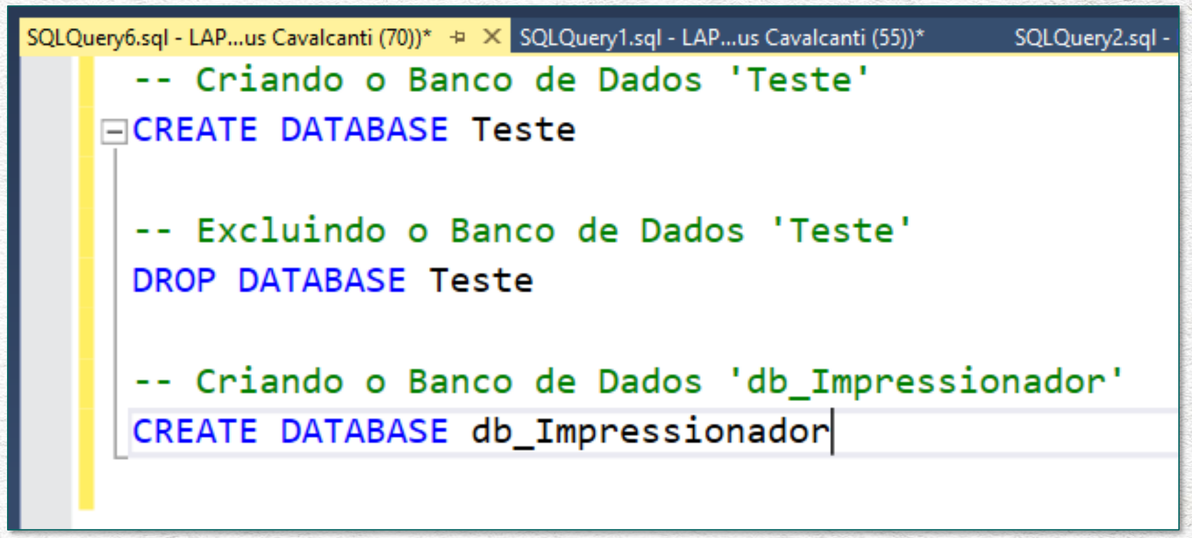
CREATE e DROP DATABASE

Vamos criar/excluir nosso primeiro Banco de Dados.

Utilize o comando **CREATE DATABASE** ao lado para criar o banco de dados ‘Teste’.

Em seguida, utilize o comando **DROP DATABASE** para excluir o banco de dados criado.

Por fim, crie um novo banco de dados chamado **db_Impressionador**. Este será o banco de dados onde criaremos nossas tabelas nas próximas aulas.

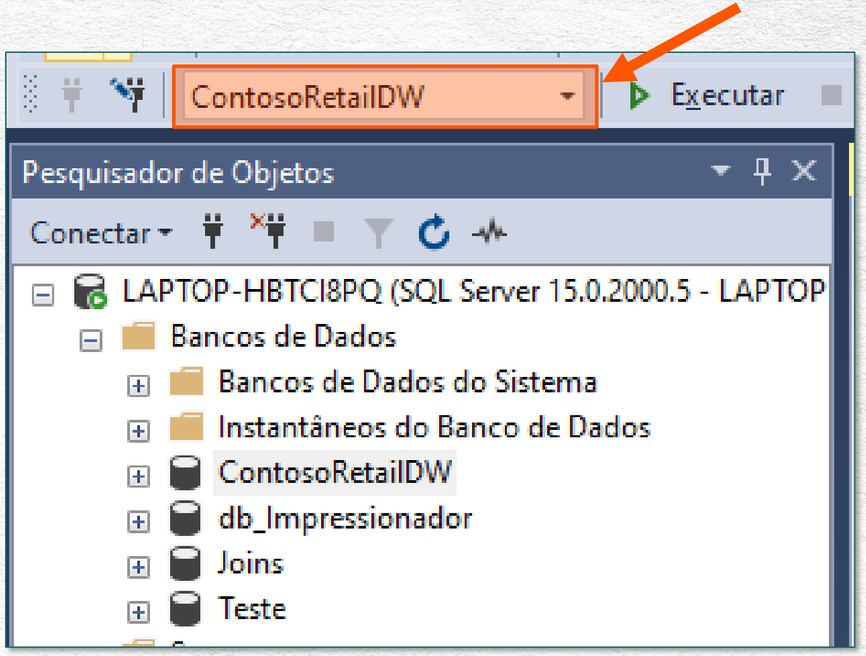


USE DATABASE

Agora que temos mais de um banco de dados, precisamos tomar cuidado antes de criar uma nova tabela.

O que podemos fazer para garantir que a tabela será criada no banco de dados certo seria alterar o nome do banco de dados no local indicado na imagem à direita.

Porém, essa seria uma solução manual.



O ideal será utilizar o comando USE, que nos permite identificar, logo no início do nosso código, qual o Banco de Dados queremos selecionar. Isso deixará essa seleção muito mais automática e evitará qualquer problema de criação de tabelas no banco de dados errado.

```
USE db_Impressionador
```

CREATE TABLE

Utilize o comando **CREATE TABLE** ao lado para criar a tabela ‘Produtos’.

A nova tabela deverá conter 4 colunas:

- id_produto (INT)
- nome_produto (VARCHAR)
- data_validade (DATETIME)
- preco_produto (FLOAT)

```
USE db_Impressionador

CREATE TABLE Produtos(
    id_produto INT,
    nome_produto VARCHAR(200),
    data_validade DATETIME,
    preco_produto FLOAT
)
```

INSERT INTO

Utilize o comando **INSERT INTO** ao lado para adicionar novos valores à tabela.

Um detalhe importante: não faz diferença a ordem em que os valores são adicionados dentro do comando VALUES, desde que essa ordem respeite a ordem das colunas dentro dos parênteses, logo após o nome da tabela.

```
INSERT INTO Produtos(id_produto, nome_produto,
  data_validade, preco_produto)
VALUES
    (1, 'Arroz', '31/12/2021', 22.50),
    (2, 'Feijão', '31/12/2021', 8.99)
```

Resultados | Mensagens

	id_produto	nome_produto	data_validade	preco_produto
1	1	Arroz	2021-12-31 00:00:00.000	22,5
2	2	Feijão	2021-12-31 00:00:00.000	8,99

```
INSERT INTO Produtos(id_produto, nome_produto,
  data_validade, preco_produto)
VALUES
    (1, 'Arroz', '31/12/2021', 22.50),
    (2, 'Feijão', '31/12/2021', 8.99)
```

Existe ordem correta para adicionar dados no INSERT?

Não existe ordem correta para adicionar os valores dentro do INSERT INTO. Observe na imagem ao lado que podemos adicionar os valores de forma diferente da ordem das colunas da tabela.

A única condição é que os valores dentro do VALUES sigam a mesma ordem dos argumentos informados dentro dos parênteses da tabela.

Inclusive, observe que podemos até mesmo não passar uma das informações (nome do produto) e não teremos nenhum erro.

```
INSERT INTO Produtos(id_produto, preco_produto,
  data_validade)
VALUES
    (3, 33.99, '31/12/2999')
```

Resultados | Mensagens

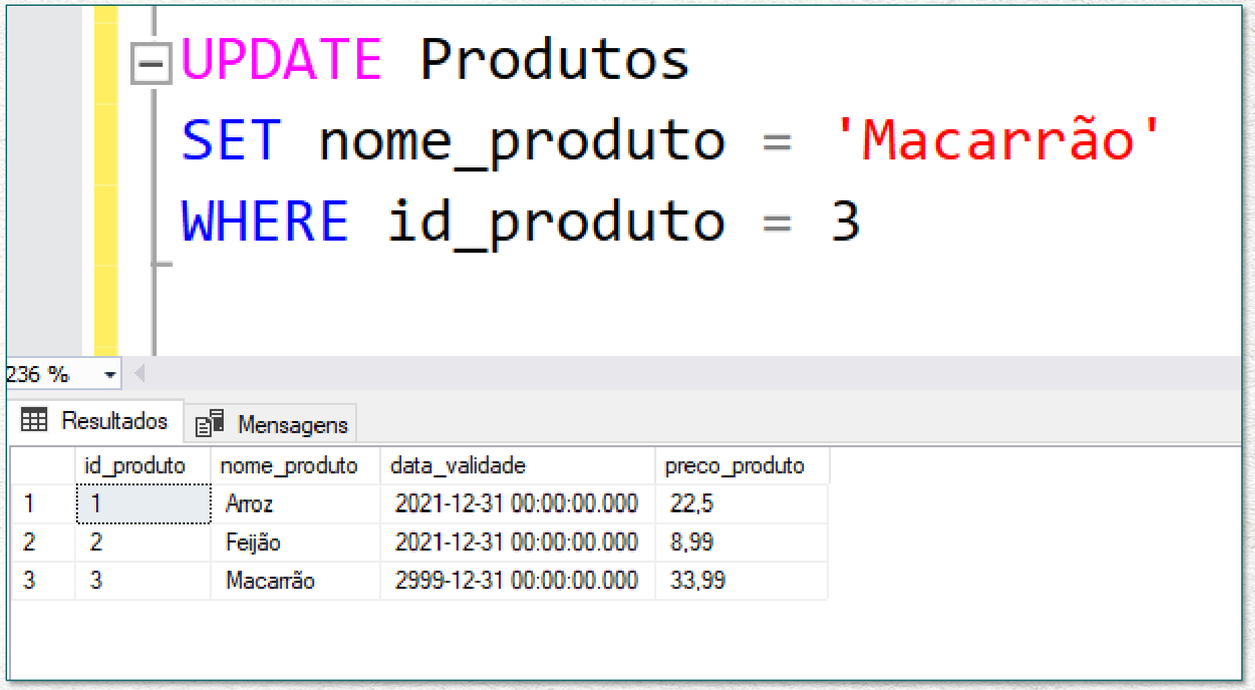
	id_produto	nome_produto	data_validade	preco_produto
1	1	Arroz	2021-12-31 00:00:00.000	22,5
2	2	Feijão	2021-12-31 00:00:00.000	8,99
3	3	NULL	2999-12-31 00:00:00.000	33,99

UPDATE

Utilize o comando **UPDATE** para atualizar um dado dentro de uma tabela.

O WHERE será fundamental para que a gente possa especificar exatamente qual é o id que queremos alterar.

Observe que, após o SET, deve ser informado o nome da coluna que queremos alterar o valor (no exemplo, **nome_produto**).



	id_produto	nome_produto	data_validade	preco_produto
1	1	Arroz	2021-12-31 00:00:00.000	22,5
2	2	Feijão	2021-12-31 00:00:00.000	8,99
3	3	Macarrão	2999-12-31 00:00:00.000	33,99

DELETE

Utilize o comando **DELETE** para deletar um dado dentro de uma tabela.

O WHERE será fundamental para que a gente possa especificar exatamente qual é o id que queremos deletar.

```
DELETE
FROM Produtos
WHERE id_produto = 3
```

236 %

Resultados | Mensagens

	id_produto	nome_produto	data_validade	preco_produto
1	1	Arroz	2021-12-31 00:00:00.000	22,5
2	2	Feijão	2021-12-31 00:00:00.000	8,99

Código para criação de uma tabela exemplo

Utilize o código ao lado para criar uma tabela exemplo de Funcionários. Essa tabela será usada para as próximas aplicações, onde veremos como alterar colunas de uma tabela.

SQLQuery6.sql - LAP...us Cavalcanti (70))* | SQLQuery1.sql - LAP...us Cavalcanti (55))* | SQLQuery2.sql - LAP...us Cavalcanti (54))*

```
USE db_Impressionador

CREATE TABLE Funcionarios(
    id_funcionario INT,
    nome_funcionario VARCHAR(100),
    salario FLOAT,
    data_nascimento DATETIME
)

INSERT INTO Funcionarios(id_funcionario, nome_funcionario, salario, data_nascimento)
VALUES
    (1, 'Lucas'     , 1500, '20/03/1990'),
    (2, 'Andressa'  , 2300, '07/12/1988'),
    (3, 'Felipe'    , 4000, '13/02/1993'),
    (4, 'Marcelo'   , 7100, '10/04/1993'),
    (5, 'Carla'     , 3200, '02/09/1986'),
    (6, 'Juliana'   , 5500, '21/01/1989'),
    (7, 'Mateus'    , 1900, '02/11/1993'),
    (8, 'Sandra'    , 3900, '09/05/1990'),
    (9, 'André'     , 1000, '13/03/1994'),
    (10, 'Julio'    , 4700, '05/07/1992')
```

ALTER TABLE: Adicionar coluna

Para adicionar uma coluna, utilizamos o comando **ALTER TABLE** em conjunto com o comando ADD, assim como mostrado no print ao lado.

```
ALTER TABLE Funcionarios
ADD cargo VARCHAR(100), bonus FLOAT
```

Em seguida, utilizamos o UPDATE em conjunto com o WHERE para atualizar os valores dessa coluna.

```
UPDATE Funcionarios
SET cargo = 'Analista'
WHERE id_funcionario = 1
```

121 %

Resultados | Mensagens

	id_funcionario	nome_funcionario	salario	data_nascimento	cargo	bonus
1	1	Lucas	1500	1990-03-20 00:00:00.000	Analista	NULL
2	2	Andressa	2300	1988-12-07 00:00:00.000	NULL	NULL
3	3	Felipe	4000	1993-02-13 00:00:00.000	NULL	NULL
4	4	Marcelo	7100	1993-04-10 00:00:00.000	NULL	NULL
5	5	Carla	3200	1986-09-02 00:00:00.000	NULL	NULL
6	6	Juliana	5500	1989-01-21 00:00:00.000	NULL	NULL
7	7	Mateus	1900	1993-11-02 00:00:00.000	NULL	NULL
8	8	Sandra	3900	1990-05-09 00:00:00.000	NULL	NULL
9	9	André	1000	1994-03-13 00:00:00.000	NULL	NULL
10	10	Julio	4700	1992-07-05 00:00:00.000	NULL	NULL

ALTER TABLE: Alterando tipo de dados

Para alterar o tipo de dados de uma coluna, basta seguir o exemplo da imagem ao lado.

```
ALTER TABLE Funcionarios
ALTER COLUMN salario INT
```

ALTER TABLE: Excluindo uma coluna de uma tabela

Para excluir uma ou mais colunas de uma tabela, utilizamos o comando **DROP COLUMN**, como mostrado no exemplo ao lado.

```
ALTER TABLE Funcionarios
DROP COLUMN cargo, bonus
```

EXERCÍCIOS

1

Questão 1

a) Crie um banco de dados chamado BD_Teste.
b) Exclua o banco de dados criado no item anterior.
c) Crie um banco de dados chamado Exercicios.

2

Questão 2

No banco de dados criado no exercício anterior, crie 3 tabelas, cada uma contendo as seguintes colunas.

Tabela 1 (dCliente): ID_Cliente, Nome_Cliente, Data_de_Nascimento
Tabela 2 (dGerente): ID_Gerente, Nome_Gerente, Data_de_Contratacao, Salario
Tabela 3 (fContratos): ID_Contrato, Data_de_Assinatura, ID_Cliente, ID_Gerente, Valor_do_Contrato

Lembre-se dos seguintes pontos:

a) Garantir que o Banco de Dados Exercicios está selecionado.

b) Definir qual será o tipo de dados mais adequado para cada coluna das tabelas. Lembrando que os tipos de dados mais comuns são: INT, FLOAT, VARCHAR e DATETIME.

Por fim, faça um SELECT para visualizar cada tabela.

EXERCÍCIOS

3

Questão 3

Em cada uma das 3 tabelas, adicione os seguintes valores:

TABELA dCliente

ID_Cliente	Nome_Cliente	Data_de_Nascimento
1	André Martins	1989-02-12 00:00:00.000
2	Bárbara Campos	1992-05-07 00:00:00.000
3	Carol Freitas	1985-04-23 00:00:00.000
4	Diego Cardoso	1994-10-11 00:00:00.000
5	Eduardo Pereira	1988-11-09 00:00:00.000
6	Fabiana Silva	1989-09-02 00:00:00.000
7	Gustavo Barb...	1993-06-27 00:00:00.000
8	Helen Viana	1990-02-11 00:00:00.000

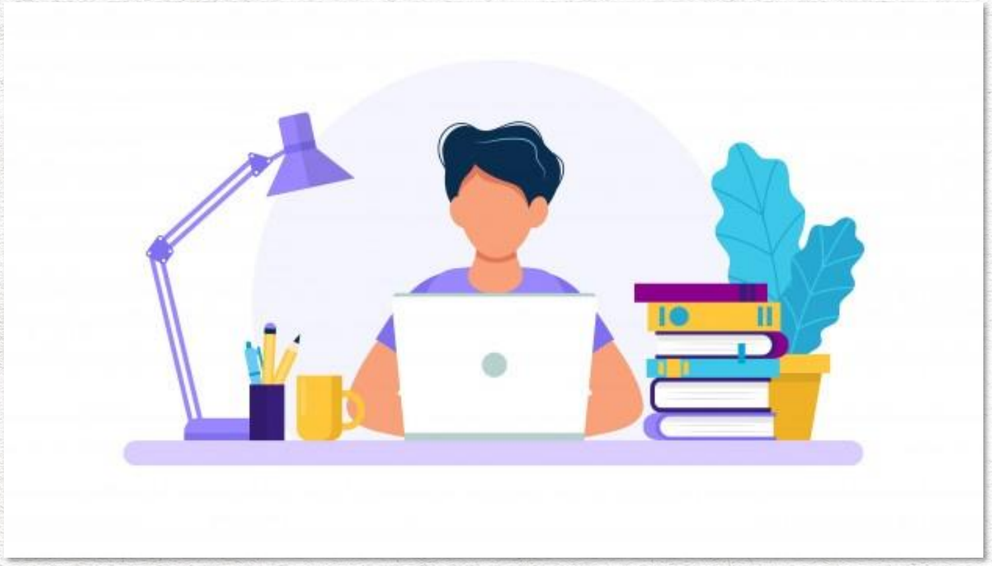
TABELA dGerente

ID_Gerente	Nome_Gerente	Data_de_Contratacao	Salario
1	Lucas Sampaio	2015-03-21 00:00:00.000	6700
2	Mariana Padilha	2011-01-10 00:00:00.000	9900
3	Nathália Santos	2018-10-03 00:00:00.000	7200
4	Otávio Costa	2017-04-18 00:00:00.000	11000

TABELA fContratos

ID_Contrato	Data_de_Assinatura	ID_Cliente	ID_Gerente	Valor_do_Contrato
1	2019-01-12 00:00:00.000	8	1	23000
2	2019-02-10 00:00:00.000	3	2	15500
3	2019-03-07 00:00:00.000	7	2	6500
4	2019-03-15 00:00:00.000	1	3	33000
5	2019-03-21 00:00:00.000	5	4	11100
6	2019-03-23 00:00:00.000	4	2	5500
7	2019-03-28 00:00:00.000	9	3	55000
8	2019-04-04 00:00:00.000	2	1	31000
9	2019-04-05 00:00:00.000	10	4	3400
10	2019-04-05 00:00:00.000	6	2	9200

EXERCÍCIOS



4

Questão 4

Novos dados deverão ser adicionados nas tabelas dCliente, dGerente e fContratos. Fique livre para adicionar uma nova linha em cada tabela contendo, respectivamente,

(1) um novo cliente (id cliente, nome e data de nascimento)

(2) um novo gerente (id gerente, nome, data de contratação e salário)

(3) um novo contrato (id, data assinatura, id cliente, id gerente, valor do contrato)

5

Questão 5

O contrato de ID igual a 4 foi registrado com alguns erros na tabela fContratos. Faça uma alteração na tabela atualizando os seguintes valores:

Data_de_Assinatura: 17/03/2019

ID_Gerente: 2

Valor_do_Contrato: 33500

6

Questão 6

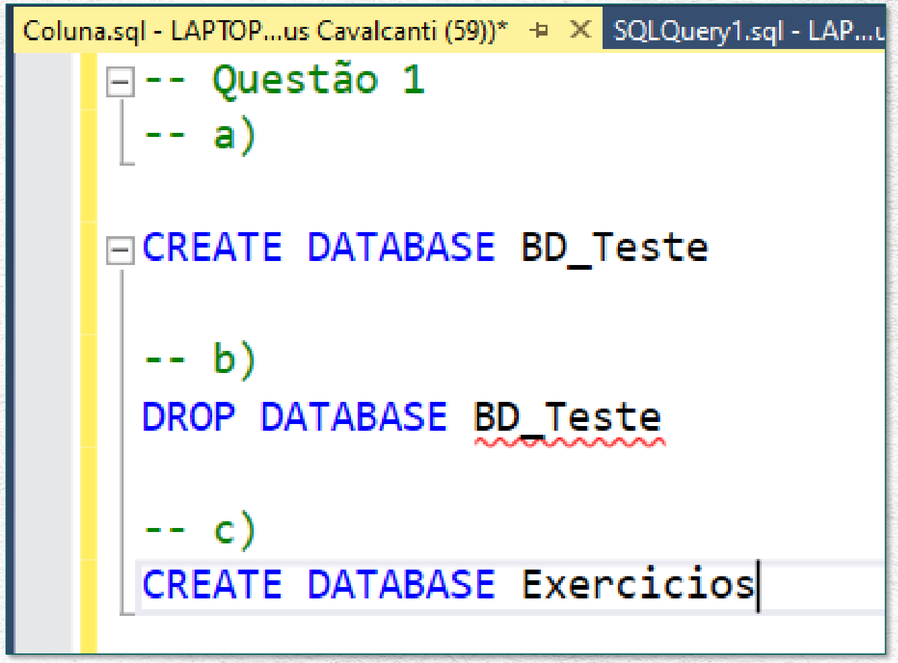
Delete a linha da tabela fContratos que você criou na questão 4.

GABARITOS

1

Questão 1

a) Crie um banco de dados chamado BD_Teste.
b) Exclua o banco de dados criado no item anterior.
c) Crie um banco de dados chamado Exercicios.



GABARITOS

2

Questão 2

No banco de dados criado no exercício anterior, crie 3 tabelas, cada uma contendo as seguintes colunas.

Tabela 1 (dCliente): ID_Cliente, Nome_Cliente, Data_de_Nascimento
Tabela 2 (dGerente): ID_Gerente, Nome_Gerente, Data_de_Contratacao, Salario
Tabela 3 (fContratos): ID_Contrato, Data_de_Assinatura, ID_Cliente, ID_Gerente, Valor_do_Contrato

Lembre-se dos seguintes pontos:

a) Garantir que o Banco de Dados Exercicios está selecionado.

b) Definir qual será o tipo de dados mais adequado para cada coluna das tabelas. Lembrando que os tipos de dados mais comuns são: INT, FLOAT, VARCHAR e DATETIME.

Por fim, faça um SELECT para visualizar cada tabela.

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LA

```sql
-- Questão 2

USE Exercicios

CREATE TABLE dCliente(
    ID_Cliente INT,
    Nome_Cliente VARCHAR(100),
    Data_de_Nascimento DATETIME
)

CREATE TABLE dGerente(
    ID_Gerente INT,
    Nome_Gerente VARCHAR(100),
    Data_de_Contratacao DATETIME,
    Salario FLOAT
)


CREATE TABLE fContratos(
    ID_Contrato INT,
    Data_de_Assinatura DATETIME,
    ID_Cliente INT,
    ID_Gerente INT,
    Valor_do_Contrato FLOAT
)

SELECT * FROM dCliente
SELECT * FROM dGerente
SELECT * FROM fContratos
```

GABARITOS

3

Questão 3

Em cada uma das 3 tabelas, adicione os seguintes valores:

TABELA dCliente

ID_Cliente	Nome_Cliente	Data_de_Nascimento
1	André Martins	1989-02-12 00:00:00.000
2	Bárbara Campos	1992-05-07 00:00:00.000
3	Carol Freitas	1985-04-23 00:00:00.000
4	Diego Cardoso	1994-10-11 00:00:00.000
5	Eduardo Pereira	1988-11-09 00:00:00.000
6	Fabiana Silva	1989-09-02 00:00:00.000
7	Gustavo Barb...	1993-06-27 00:00:00.000
8	Helen Viana	1990-02-11 00:00:00.000

TABELA fContratos

ID_Contrato	Data_de_Assinatura	ID_Cliente	ID_Gerente	Valor_do_Contrato
1	2019-01-12 00:00:00.000	8	1	23000
2	2019-02-10 00:00:00.000	3	2	15500
3	2019-03-07 00:00:00.000	7	2	6500
4	2019-03-15 00:00:00.000	1	3	33000
5	2019-03-21 00:00:00.000	5	4	11100
6	2019-03-23 00:00:00.000	4	2	5500
7	2019-03-28 00:00:00.000	9	3	55000
8	2019-04-04 00:00:00.000	2	1	31000
9	2019-04-05 00:00:00.000	10	4	3400
10	2019-04-05 00:00:00.000	6	2	9200

TABELA dGerente

ID_Gerente	Nome_Gerente	Data_de_Contratacao	Salario
1	Lucas Sampaio	2015-03-21 00:00:00.000	6700
2	Mariana Padilha	2011-01-10 00:00:00.000	9900
3	Nathália Santos	2018-10-03 00:00:00.000	7200
4	Otávio Costa	2017-04-18 00:00:00.000	11000

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```sql
-- Questão 3

-- Inserindo valores na tabela dCliente
INSERT INTO dCliente(ID_Cliente, Nome_Cliente, Data_de_Nascimento)
VALUES
    (1,  'André Martins',   '12/02/1989'),
    (2,  'Bárbara Campos',  '07/05/1992'),
    (3,  'Carol Freitas',   '23/04/1985'),
    (4,  'Diego Cardoso',   '11/10/1994'),
    (5,  'Eduardo Pereira', '09/11/1988'),
    (6,  'Fabiana Silva',   '02/09/1989'),
    (7,  'Gustavo Barbosa', '27/06/1993'),
    (8,  'Helen Viana',     '11/02/1990'),
    (9,  'Igor Castro',     '21/08/1989'),
    (10, 'Juliana Pires',   '13/01/1991')

-- Inserindo valores na tabela dGerente
INSERT INTO dGerente(ID_Gerente, Nome_Gerente, Data_de_Contratacao, Salario)
VALUES
    (1,  'Lucas Sampaio',   '21/03/2015', 6700),
    (2,  'Mariana Padilha', '10/01/2011', 9900),
    (3,  'Nathália Santos', '03/10/2018', 7200),
    (4,  'Otávio Costa',    '18/04/2017', 11000)

-- Inserindo valores na tabela fContratos
INSERT INTO fContratos(ID_Contrato, Data_de_Assinatura, ID_Cliente, ID_Gerente, Valor_do_Contrato)
VALUES
    (1, '12/01/2019', 8, 1, 23000),
    (2, '10/02/2019', 3, 2, 15500),
    (3, '07/03/2019', 7, 2, 6500),
    (4, '15/03/2019', 1, 3, 33000),
    (5, '21/03/2019', 5, 4, 11100),
    (6, '23/03/2019', 4, 2, 5500),
    (7, '28/03/2019', 9, 3, 55000),
    (8, '04/04/2019', 2, 1, 31000),
    (9, '05/04/2019', 10, 4, 3400),
    (10, '05/04/2019', 6, 2, 9200)
```

GABARITOS

4

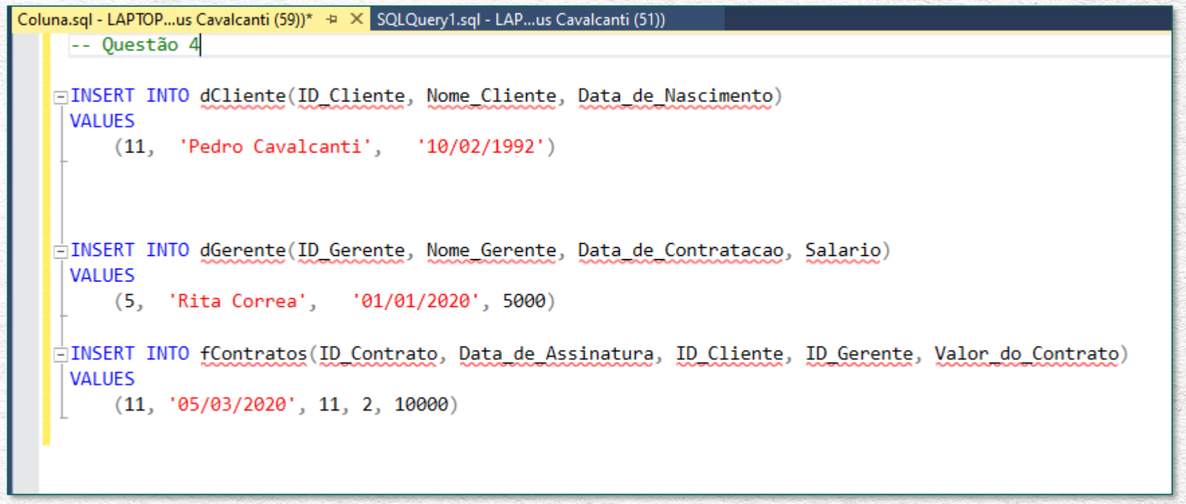
Questão 4

Novos dados deverão ser adicionados nas tabelas dCliente, dGerente e fContratos. Fique livre para adicionar uma nova linha em cada tabela contendo, respectivamente,

(1) um novo cliente (id cliente, nome e data de nascimento)

(2) um novo gerente (id gerente, nome, data de contratação e salário)

(3) um novo contrato (id, data assinatura, id cliente, id gerente, valor do contrato)



GABARITOS

5

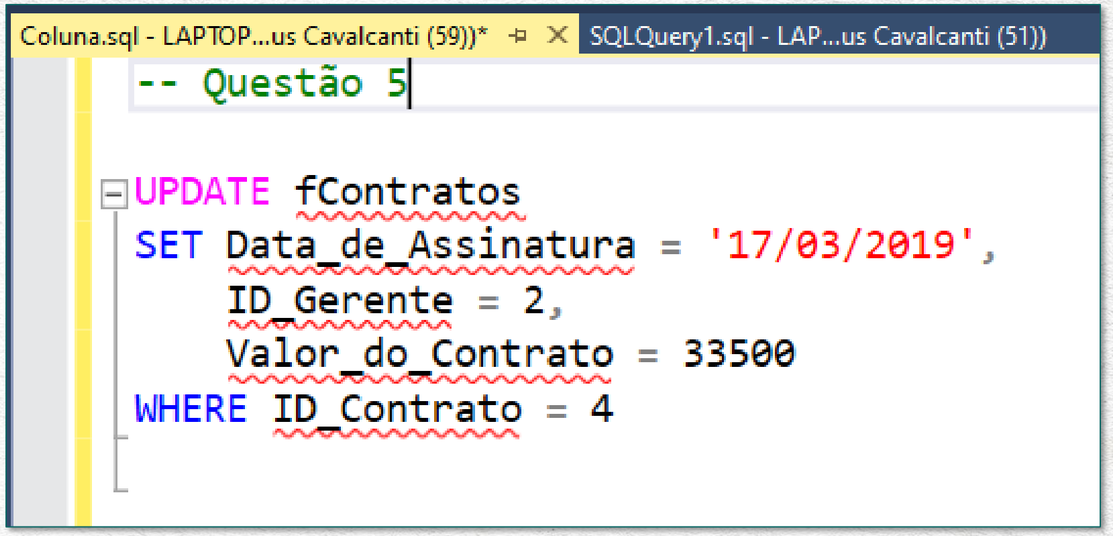
Questão 5

O contrato de ID igual a 4 foi registrado com alguns erros na tabela fContratos. Faça uma alteração na tabela atualizando os seguintes valores:

Data_de_Assinatura: 17/03/2019

ID_Gerente: 2

Valor_do_Contrato: 33500

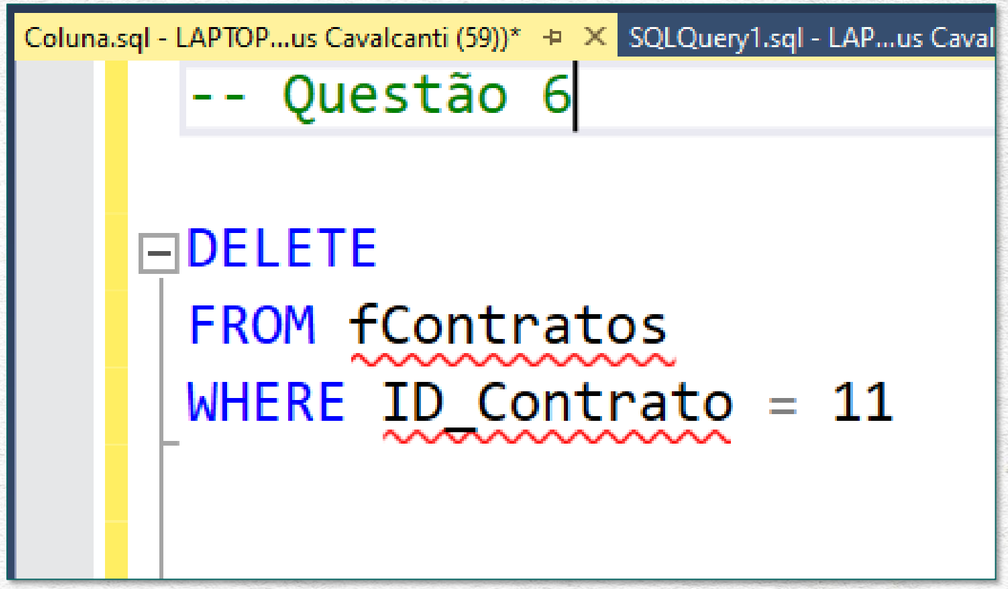


GABARITOS

6

Questão 6

Delete a linha da tabela fContratos que você criou na questão 4.



MÓDULO 14

SQL

SUBQUERIES E CTE'S

O QUE VEREMOS NESTE MÓDULO

MÓDULO 14 SUBQUERIES E CTE'S

01	Introdução	353
02	Onde utilizamos Subqueries	354
03	Subquery na prática: Aplicação com WHERE (Ex.1)	358
04	Subquery na prática: Aplicação com WHERE (Ex.2)	360
05	Subquery na prática: Aplicação com WHERE (Ex.3)	361
06	ANY e SOME	364
07	ALL	366
08	EXISTS	367
09	Subquery na prática: Aplicação com SELECT	368
10	Subquery na prática: Aplicação com FROM	369
11	Subqueries aninhadas	370
12	CTEs (Common Table Expression)	374
13	Calculando agregações com CTEs	375
14	Nomeando colunas de uma CTE	376
15	Criando múltiplas CTE's	377
16	EXERCÍCIOS	378
17	GABARITOS	383

Introdução

O QUE É UMA SUBQUERY?

Uma Subconsulta (ou Subquery ou SubSELECT) nada mais é do que uma consulta dentro de outra consulta. Ou seja, com uma subquery conseguimos utilizar o resultado de uma query (consulta) dentro de outra consulta.

O exemplo abaixo ilustra bem a ideia principal de uma subquery. Para descobrir os produtos que têm o UnitPrice maior do que a média, podemos adicionar um SELECT dentro de outro SELECT para criar uma consulta mais otimizada.

```
-- Descobrir os produtos que têm um UnitPrice
-- maior do que a média.
-- (Sem subquery)

-- 1) Primeiro descobrimos a média de UnitPrice
SELECT AVG(UnitPrice) FROM dimProduct

-- 2) Depois utilizamos o resultado dentro do filtro WHERE
SELECT
    *
FROM dimProduct
WHERE UnitPrice > 356.8301
```

```
-- Descobrir os produtos que têm um UnitPrice
-- maior do que a média.
-- (Com subquery)

-- 1) Podemos pular o passo 1 direto para a consulta principal
SELECT
    *
FROM dimProduct
WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM dimProduct)
```

Onde utilizamos Subqueries

Utilizamos subqueries nas seguintes situações:

1

Subqueries com WHERE (escalar)

Nesta situação, utilizamos o resultado de um SELECT para filtrar um outro SELECT.

Observe no print ao lado. Temos um SELECT mais interno sendo utilizado com o filtro WHERE de uma consulta principal.

Para este caso, utilizamos o termo “escalar” para dizer que o resultado do SELECT mais interno é um valor único, e não uma lista de valores.

```
SELECT
    Coluna1,
    Coluna2
FROM
    Tabela
WHERE Coluna1 = (SELECT)
```

Onde utilizamos subqueries

Utilizamos subqueries nas seguintes situações:

2

Subqueries com WHERE (lista)

Aqui temos uma situação semelhante à anterior, com a diferença de que o resultado da subquery seria uma lista de vários valores que poderiam ser passados no WHERE.

Neste caso, como o filtro seria de mais de 1 valor, utilizamos o IN.

```
SELECT
    Coluna1,
    Coluna2
FROM
    Tabela
WHERE Coluna1 IN (SELECT)
```

Onde utilizamos subqueries

Utilizamos subqueries nas seguintes situações:

3

Subqueries dentro de um SELECT

Outra opção seria utilizar o resultado de um SELECT como uma nova coluna de uma query principal.

```
SELECT
    Coluna1,
    Coluna2,
    (SELECT)
FROM
    Tabela
```

Onde utilizamos subqueries

Utilizamos subqueries nas seguintes situações:

4

Subqueries junto com um FROM

Podemos utilizar o resultado de um SELECT como uma tabela de um outro SELECT.

Neste caso, precisamos utilizar o comando AS para dar um nome para essa tabela.

```
SELECT
    Coluna1,
    Coluna2
FROM
    (SELECT) AS T
```

Subquery na prática: Aplicação com WHERE (Exemplo 1)

Para entender a ideia por trás das subqueries, vamos começar fazendo 3 exemplos.

No **Exemplo 1**, imagine que você queira fazer uma consulta à tabela DimProduct e considerar apenas os produtos com preços acima da média. Como poderíamos fazer isso?

O **primeiro passo** seria descobrir essa média de preço. Fazemos isso através da consulta simples abaixo:

```
SELECT
    AVG(UnitPrice)
FROM DimProduct
```

	(Nenhum nome de coluna)
1	356,8301

O **segundo passo** seria utilizar esse resultado em um filtro WHERE na tabela DimProduct.

```
SELECT
    *
FROM DimProduct
WHERE UnitPrice > 356.8301
```

Índice Geral

Índice deste Módulo

Subquery na prática: Aplicação com WHERE (Exemplo 1)

Fazer dessa forma não é muito automático, pois escrevemos manualmente a média de preço no filtro. Se os preços alterarem ou se tivermos adição ou exclusão de produtos, essa filtragem não será automática. O valor de 356.8301 continuará fixo mesmo que a média se altere.

O ideal então seria utilizar uma subquery, substituindo o valor fixo de 356.8301 pelo SELECT que criamos para descobrir essa média:

```
SELECT
    *

FROM DimProduct
WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM DimProduct)
```

Na solução acima, utilizamos uma subquery, ou seja, um SELECT dentro de outro SELECT. Dessa forma, o cálculo da média de preço ficará 100% automático, sem necessidade de atualizar manualmente o valor.

Vamos agora para o exemplo 2.

Subquery na prática: Aplicação com WHERE (Exemplo 2)

No **Exemplo 2** queremos filtrar nossa tabela DimProduct e retornar os produtos da categoria ‘Televisions’. Porém, não temos a informação de Nome da Subcategoria na tabela DimProduct.

Dessa forma, precisaremos criar um SELECT que descubra o ID da categoria ‘Televisions’, e passar esse resultado como o valor que queremos filtrar dentro do WHERE.

O resultado final ficaria da seguinte forma:

```
SELECT * FROM dimProduct
WHERE ProductSubcategoryKey =
    (SELECT ProductSubcategoryKey FROM DimProductSubcategory
    WHERE ProductSubcategoryName = 'Televisions')
```

Finalmente, vamos para o nosso **exemplo 3**.

Subquery na prática: Aplicação com WHERE (Exemplo 3)

No **Exemplo 3** queremos filtrar nossa tabela **FactSales_JUL2007** (código mostrado mais a frente) e mostrar apenas as vendas referentes às lojas com 100 ou mais funcionários.

O primeiro passo é descobrir os IDs dessas lojas, através da consulta abaixo:

```
SELECT
    StoreKey
FROM
    DimStore
WHERE EmployeeCount >= 100
```

155 %

Resultados | Mensagens

	StoreKey
1	199
2	200
3	280

Agora, vamos utilizar o resultado do SELECT anterior dentro do filtro da tabela factSales_JUL2007:

```
SELECT
    *
FROM factSales_JUL2007
WHERE StoreKey IN (199, 200, 280)
```

Subquery na prática: Aplicação com WHERE (Exemplo 3)

Obviamente, a consulta anterior não é a mais otimizada, pois os Ids das lojas com 100 ou mais funcionários estão fixados. Se uma nova loja passar a ter mais do que 100 funcionários, a forma como a gente fez não vai servir.

Portanto, utilizamos como uma subquery o SELECT onde descobrimos os Ids das lojas, tornando nossa consulta mais automática:

```
SELECT * FROM factSales_JUL2007
WHERE StoreKey IN (
    SELECT
        StoreKey
    FROM DimStore
    WHERE EmployeeCount >= 100
    )
ORDER BY StoreKey DESC
```

(Código para criar a factSales_JUL2007 na próxima página)

Subquery na prática: Aplicação com WHERE (Exemplo 3)


```sql
CREATE TABLE factSales_JUL2007(
    SalesKey int,
    DateKey datetime,
    channelKey int,
    StoreKey int,
    ProductKey int,
    UnitCost decimal(9,2),
    UnitPrice decimal(9,2),
    SalesQuantity int,
    TotalCost decimal(9,2),
    SalesAmount decimal(9,2)
)

INSERT INTO factSales_JUL2007(SalesKey, DateKey, channelKey, StoreKey, ProductKey, UnitCost, UnitPrice, SalesQuantity, TotalCost, SalesAmount)
SELECT SalesKey, DateKey, channelKey, StoreKey, ProductKey, UnitCost, UnitPrice, SalesQuantity, TotalCost, SalesAmount FROM FactSales
WHERE DateKey >= '20070701' AND DateKey <= '20070731'


select * from factSales_JUL2007
order by DateKey ASC

SELECT * FROM factSales_JUL2007
WHERE StoreKey IN (
    SELECT
        StoreKey
    FROM DimStore
    WHERE EmployeeCount >= 100
    )
ORDER BY StoreKey DESC
```

ANY e SOME

Os operadores ANY e SOME são equivalentes e permitem realizar uma comparação entre um único valor de uma coluna e um intervalo de outros valores. O =ANY e =SOME são equivalentes. Quando usados com o sinal lógico =, eles possuem a mesma finalidade do operador IN, permitindo retornar todas as linhas que sejam iguais à lista de valores especificados.

Exemplo:

=ANY(valor1, valor2, valor3)

Retorna todas as linhas da tabela que sejam iguais ao valor1 OU valor2 OU valor3.

O >ANY e >SOME retornam todas as linhas da tabela que tenham valores que sejam maiores a qualquer um dos valores da lista de valores especificados.

Exemplo:

>ANY(valor1, valor2, valor3)

Retorna todas as linhas da tabela com valores maiores que o valor1 OU valor2 OU valor3. Ou seja, maior que o menor de todos esses valores.

ANY e SOME

O <ANY e <SOME retornam todas as linhas da tabela que tenham valores que sejam menores que qualquer um dos valores da lista de valores especificados.

Exemplo:

<ANY(valor1, valor2, valor3)

Retorna todas as linhas da tabela com valores menores que o valor1 OU valor2 OU valor3. Ou seja, menor que o maior de todos esses valores.

ALL

O operador ALL também permite uma comparação entre valores especificados, mas para que a consulta retorne algum resultado, é necessário que o valor comparado seja maior ou menor do que TODOS os valores da lista. (Obs: dificilmente utilizamos =ALL pois estaríamos comparando um valor e esperando que ele fosse igual a todos os valores da lista. A menos que todos os valores da lista fossem iguais, o =ALL não faria sentido)

Exemplo:

>ALL(valor1, valor2, valor3)

Retorna todas as linhas da tabela que sejam maiores que os valores valor1 E valor2 E valor3. Ou seja, as linhas que possuem valores maiores que o máximo.

Exemplo:

<ALL(valor1, valor2, valor3)

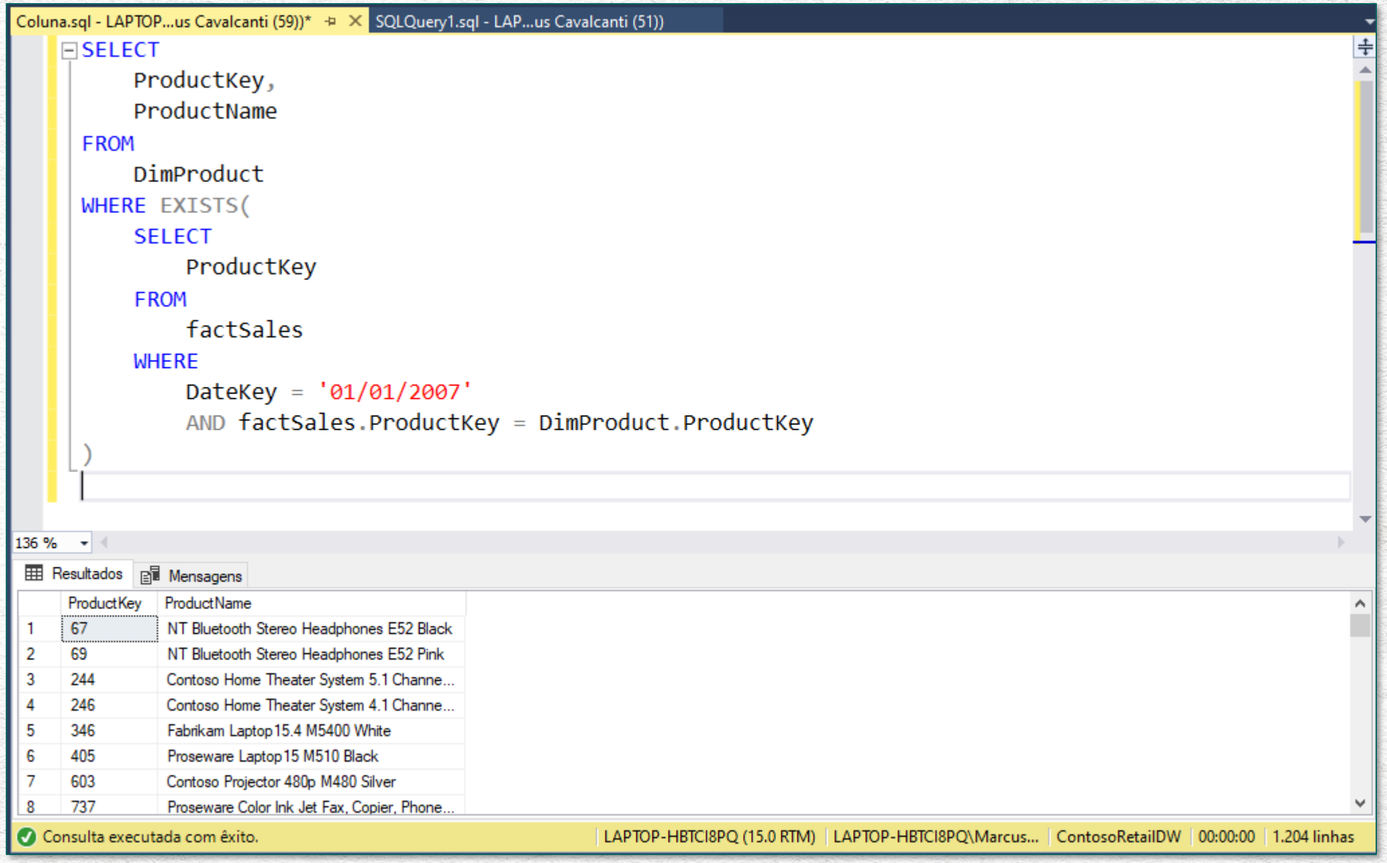
Retorna todas as linhas da tabela com valores menores que o valor1 E valor2 E valor3. Ou seja, menor que o menor de todos esses valores.

EXISTS

O operador EXISTS é usado para testar a existência de qualquer registro (linha) em uma subconsulta.

Pensando em um exemplo prático, imagina que a gente queira descobrir quais são os produtos que possuem vendas em um determinado dia, por exemplo, 01/01/2007.

Neste caso, poderíamos utilizar o EXISTS como mostrado ao lado.



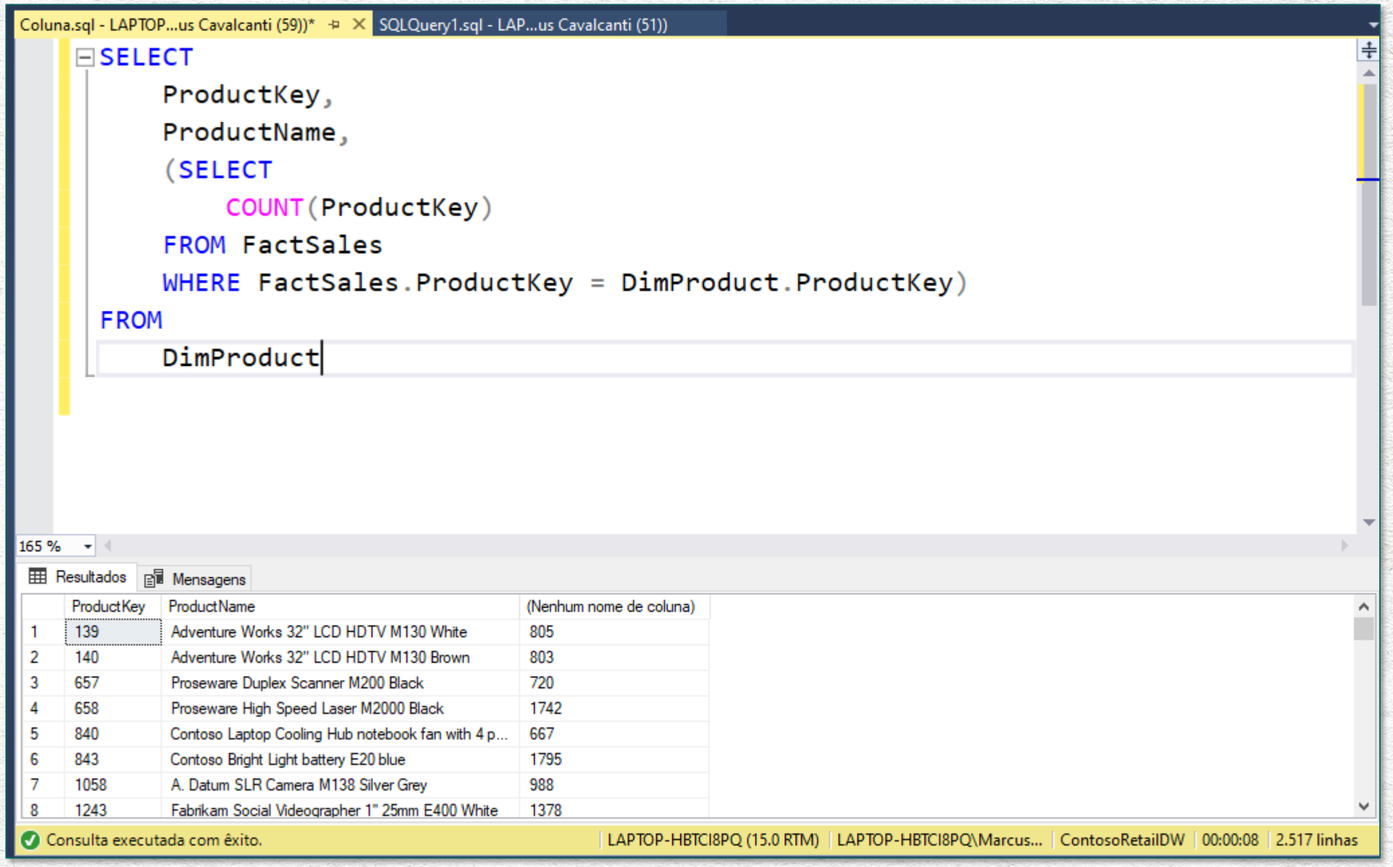
Subquery na prática: Aplicação com SELECT

Podemos também utilizar subqueries para criar novas colunas dentro de uma consulta.

Repare o exemplo ao lado: queremos descobrir qual foi o total de vendas de cada produto da nossa tabela DimProduct.

Para isso, adicionamos uma subquery (um novo SELECT) como sendo uma coluna extra da nossa consulta principal.

Assim, além de mostrar as colunas ProductKey e ProductName da tabela dimProduct, adicionamos também uma coluna que faz a contagem de vendas na tabela FactSales.



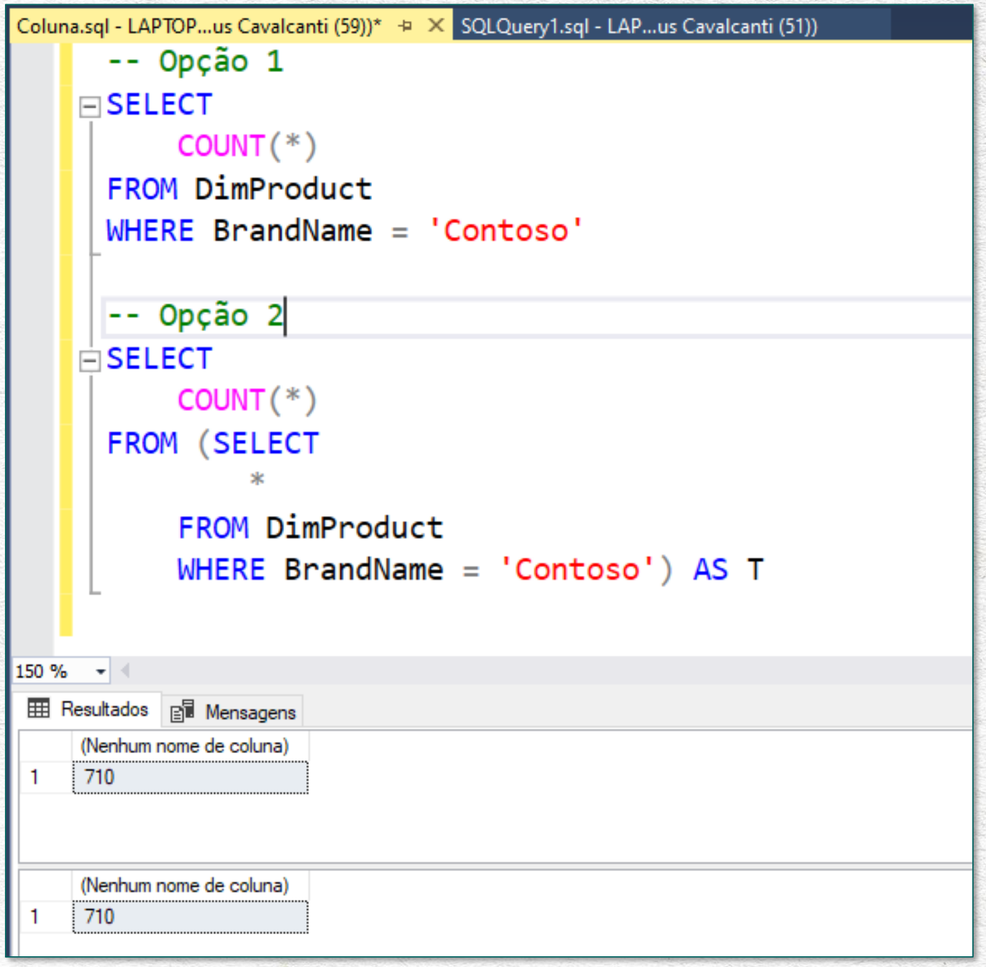
	ProductKey	ProductName	(Nenhum nome de coluna)
1	139	Adventure Works 32" LCD HDTV M130 White	805
2	140	Adventure Works 32" LCD HDTV M130 Brown	803
3	657	Proseware Duplex Scanner M200 Black	720
4	658	Proseware High Speed Laser M2000 Black	1742
5	840	Contoso Laptop Cooling Hub notebook fan with 4 p...	667
6	843	Contoso Bright Light battery E20 blue	1795
7	1058	A. Datum SLR Camera M138 Silver Grey	988
8	1243	Fabrikam Social Videographer 1" 25mm E400 White	1378

Subquery na prática: Aplicação com FROM

No exemplo ao lado, vemos duas formas de descobrir o total de produtos da marca Contoso.

Na opção 1, fizemos uma contagem na tabela DimProduct e por fim um filtro na coluna BrandName.

Já na opção 2, utilizamos a ideia de subqueries: a contagem foi feita não na tabela DimProduct, e sim em uma tabela que foi filtrada antes de ser aplicada no FROM.



Subqueries aninhadas

Também é possível criar subqueries aninhadas, ou seja, subqueries dentro de subqueries. Para entender como vai funcionar, vamos fazer o seguinte exemplo: descobrir os nomes dos clientes que ganham o segundo maior salário.

Poderíamos descobrir o segundo maior salário aplicando um TOP 2 + ORDER BY. Feito isso, descobriríamos que o segundo maior salário é $160.000 e poderíamos utilizar esse resultado para filtrar a nossa tabela DimCustomer e descobrir os clientes que recebem o 2º maior salário. **Mas como tornar isso mais automático?**

Coluna.sql - LAPTO...(59)) Executando...* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```sql
-- Descubra os nomes dos clientes que ganham o segundo maior salário

SELECT * FROM DimCustomer
WHERE CustomerType = 'Person'
ORDER BY YearlyIncome DESC

SELECT DISTINCT TOP(2) YearlyIncome FROM DimCustomer
WHERE CustomerType = 'Person'
ORDER BY YearlyIncome DESC

SELECT
    CustomerKey,
    FirstName,
    LastName,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome = 160000
```

Subqueries aninhadas

Para resolver esse problema utilizando subqueries, teríamos que dividi-lo em etapas:

```
--1. Descobrir o maior salário
--2. Descobrir o segundo maior salário
--3. Descobrir os nomes dos clientes que ganham o segundo maior salário
```

Subqueries aninhadas

A solução é mostrada ao lado. Repare que temos 3 SELECTs ao todo.

```sql
SELECT
    CustomerKey,
    FirstName,
    LastName,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome = (
    SELECT
        MAX(YearlyIncome)
    FROM DimCustomer
    WHERE
        YearlyIncome < (
            SELECT
                MAX(YearlyIncome)
            FROM DimCustomer
            WHERE CustomerType = 'Person'
    )
)
```

Subqueries aninhadas

O SELECT mais interno (3) é responsável por retornar pra gente o maior de todos os salários.

Com este resultado, criamos um SELECT intermediário (2) para descobrir qual é o salário máximo, mas que seja menor que o máximo descoberto em (3). Este seria exatamente o segundo maior salário.

Por fim, utilizamos o resultado para filtrar a coluna de YearlyIncome do SELECT (1), e finalmente chegamos no resultado desejado: a lista de clientes que recebem o segundo maior salário.

```
SELECT                                          1
    CustomerKey,
    FirstName,
    LastName,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome = (
    SELECT                                      2
        MAX(YearlyIncome)
    FROM DimCustomer
    WHERE
        YearlyIncome < (
            SELECT                              3
                MAX(YearlyIncome)
            FROM DimCustomer
            WHERE CustomerType = 'Person'
    )
)
```

CTEs (Common Table Expression)

Uma CTE se trata de uma tabela temporária que pode ser criada e reaproveitada em nosso código.

Ao lado, temos um exemplo de uma CTE. A estrutura para cria-la é bem simples. Utilizamos o comando WITH, seguido do nome da CTE, e em seguida o AS para especificar o que vai compor essa CTE. No exemplo ao lado, a CTE vai armazenar determinadas colunas da tabela DimProduct, mas considerando apenas a marca Contoso.

Em seguida, podemos selecionar essa CTE utilizando o comando SELECT.

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```sql
-- 1. Criando a CTE
WITH cte AS (
SELECT
    ProductKey,
    ProductName,
    BrandName,
    ColorName
    UnitPrice
FROM DimProduct
WHERE BrandName = 'Contoso'
)

-- 2. Selecionando a CTE
SELECT
    *
FROM cte
```

Calculando agregações com CTEs

Podemos armazenar consultas mais complexas dentro de uma CTE.

No exemplo ao lado, criamos uma CTE que é o agrupamento da tabela DimProduct, contendo o total de produtos por marca.

Por fim, podemos descobrir a média de produtos por marca, aplicando o AVG na coluna Total da nossa CTE.

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```sql
-- 1. Criando CTE com GROUP BY
WITH cte AS (
    SELECT
        BrandName,
        COUNT(*) AS 'Total'
    FROM
        DimProduct
    GROUP BY
        BrandName
)
-- 2. Calculando a média da coluna Total da nossa CTE
SELECT
    AVG(Total)
FROM
    cte
```

Nomeando colunas de uma CTE

Também seria possível criar uma CTE especificando os nomes das suas colunas.

No exemplo ao lado, temos a CTE com colunas chamadas ‘Marca’ e ‘Total’. Posteriormente, podemos fazer um SELECT referenciando o nome delas.

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```
-- 1. Criando CTE e nomeando as colunas
WITH cte(Marca, Total) AS (
    SELECT
        BrandName,
        COUNT(*)
    FROM
        DimProduct
    GROUP BY
        BrandName
)

-- 2. Selecionando a nossa CTE
SELECT
    Marca,
    Total
FROM
    cte
```

Criando múltiplas CTE's

Podemos criar múltiplas CTEs e em seguida criar uma relação entre elas.

Observe na consulta ao lado: criamos duas CTEs (produtos_contoso e vendas_top100) e por fim, realizamos um JOIN entre essas CTEs.

Coluna.sql - LAPTOP...us Cavalcanti (59))* | SQLQuery1.sql - LAP...us Cavalcanti (51))

```
-- 1. Criando a CTE produtos_contoso
WITH produtos_contoso AS (
    SELECT
        ProductKey,
        ProductName,
        BrandName
    FROM
        DimProduct
    WHERE
        BrandName = 'Contoso'
),
-- 2. Criando a CTE venads_top100
vendas_top100 AS (
    SELECT TOP(100)
        SalesKey,
        ProductKey,
        DateKey,
        SalesQuantity
    FROM
        FactSales
    ORDER BY
        DateKey DESC
)
-- 3. Realizando um JOIN entre as duas CTEs
SELECT * FROM vendas_top100
INNER JOIN produtos_contoso
    ON vendas_top100.ProductKey = produtos_contoso.ProductKey
```

EXERCÍCIOS

1

Questão 1

Para fins fiscais, a contabilidade da empresa precisa de uma tabela contendo todas as vendas referentes à loja 'Contoso Orlando Store'. Isso porque essa loja encontra-se em uma região onde a tributação foi modificada recente.

Portanto, crie uma consulta ao Banco de Dados para obter uma tabela FactSales contendo todas as vendas desta loja.

2

Questão 2

O setor de controle de produtos quer fazer uma análise para descobrir quais são os produtos que possuem um UnitPrice maior que o UnitPrice do produto de ID igual a 1893.

a) A sua consulta resultante deve conter as colunas ProductKey, ProductName e UnitPrice da tabela DimProduct.

b) Nessa query você também deve retornar uma coluna extra, que informe o UnitPrice do produto 1893.

EXERCÍCIOS

3

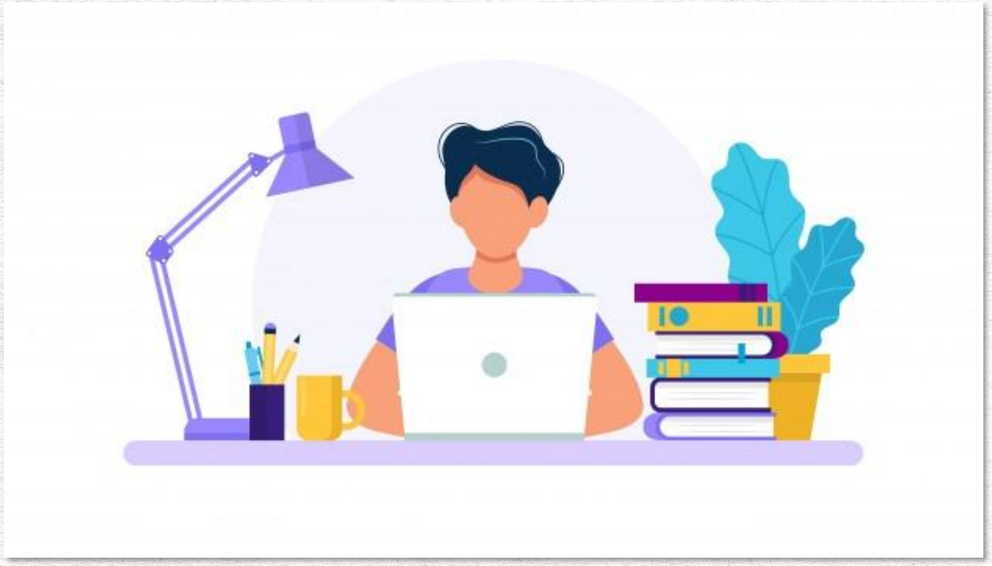
Questão 3

A empresa Contoso criou um programa de bonificação chamado "**Todos por 1**". Este programa consistia no seguinte: 1 funcionário seria escolhido ao final do ano como o funcionário destaque, só que a bonificação seria recebida por todos da área daquele funcionário em particular. O objetivo desse programa seria o de incentivar a colaboração coletiva entre os funcionários de uma mesma área. Desta forma, o funcionário destaque beneficiaria não só a si, mas também a todos os colegas de sua área.

Ao final do ano, o funcionário escolhido como destaque foi o **Miguel Severino**. Isso significa que todos os funcionários da área do Miguel seriam beneficiados com o programa.

O seu objetivo é realizar uma consulta à tabela **DimEmployee** e retornar todos os funcionários da área "vencedora" para que o setor Financeiro possa realizar os pagamentos das bonificações.

EXERCÍCIOS



4

Questão 4

Faça uma query que retorne os clientes que recebem um salário anual acima da média. A sua query deve retornar as colunas CustomerKey, FirstName, LastName, EmailAddress e YearlyIncome.

Obs: considere apenas os clientes que são 'Pessoas Físicas'.

5

Questão 5

A ação de desconto da Asian Holiday Promotion foi uma das mais bem sucedidas da empresa. Agora, a Contoso quer entender um pouco melhor sobre o perfil dos clientes que compraram produtos com essa promoção.

Seu trabalho é criar uma query que retorne a lista de clientes que compraram nessa promoção.

EXERCÍCIOS

6

Questão 6

A empresa implementou um programa de fidelização de clientes empresariais. Todos aqueles que comprarem mais de 3000 unidades de um mesmo produto receberá descontos em outras compras.

Você deverá descobrir as informações de CustomerKey e CompanyName destes clientes.

7

Questão 7

Você deverá criar uma consulta para o setor de vendas que mostre as seguintes colunas da tabela DimProduct:

ProductKey, ProductName, BrandName, UnitPrice, Média de UnitPrice.

8

Questão 8

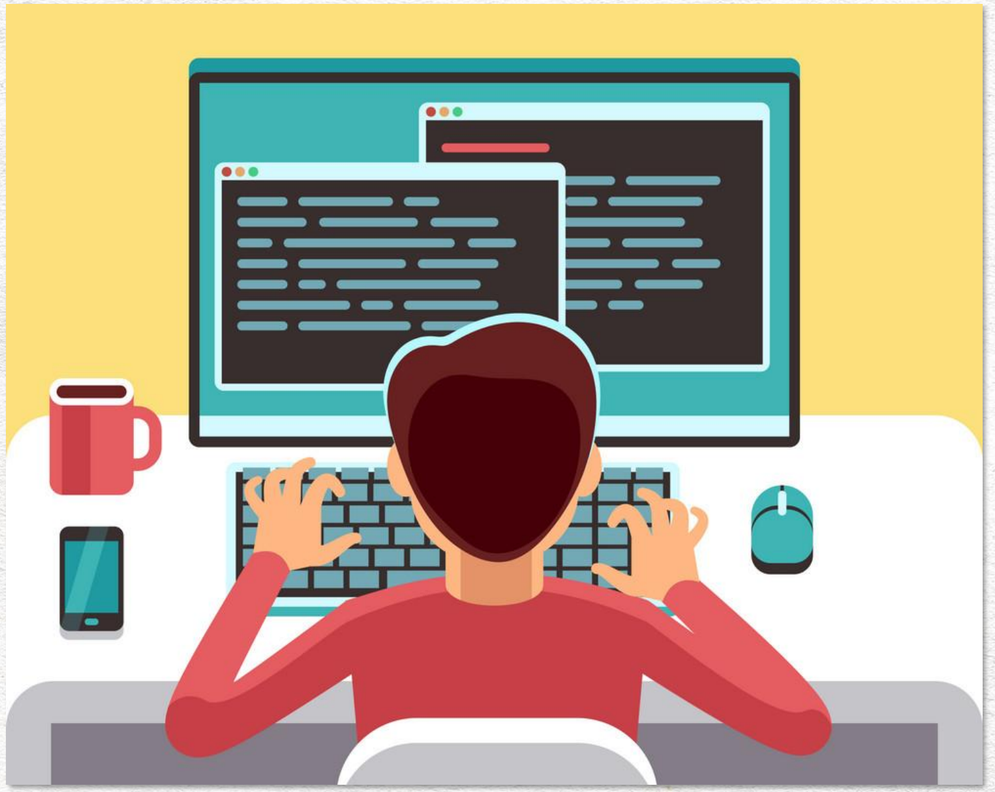
Faça uma consulta para descobrir os seguintes indicadores dos seus produtos:

Maior quantidade de produtos por marca

Menor quantidade de produtos por marca

Média de produtos por marca

EXERCÍCIOS



9

Questão 9

Crie uma CTE que seja o agrupamento da tabela **DimProduct**, armazenando o total de produtos por marca. Em seguida, faça um SELECT nesta CTE, descobrindo qual é a quantidade máxima de produtos para uma marca. Chame esta CTE de **CTE_QtdProdutosPorMarca**.

10

Questão 10

Crie duas CTEs:

(i) a primeira deve conter as colunas ProductKey, ProductName, ProductSubcategoryKey, BrandName e UnitPrice, da tabela DimProduct, mas apenas os produtos da marca Adventure Works. Chame essa CTE de CTE_ProdutosAdventureWorks.

(ii) a segunda deve conter as colunas ProductSubcategoryKey, ProductSubcategoryName, da tabela DimProductSubcategory mas apenas para as subcategorias 'Televisions' e 'Monitors'. Chame essa CTE de CTE_CategoriaTelevisionsERadio.

Faça um Join entre essas duas CTEs, e o resultado deve ser uma query contendo todas as colunas das duas tabelas. Observe nesse exemplo a diferença entre o LEFT JOIN e o INNER JOIN.

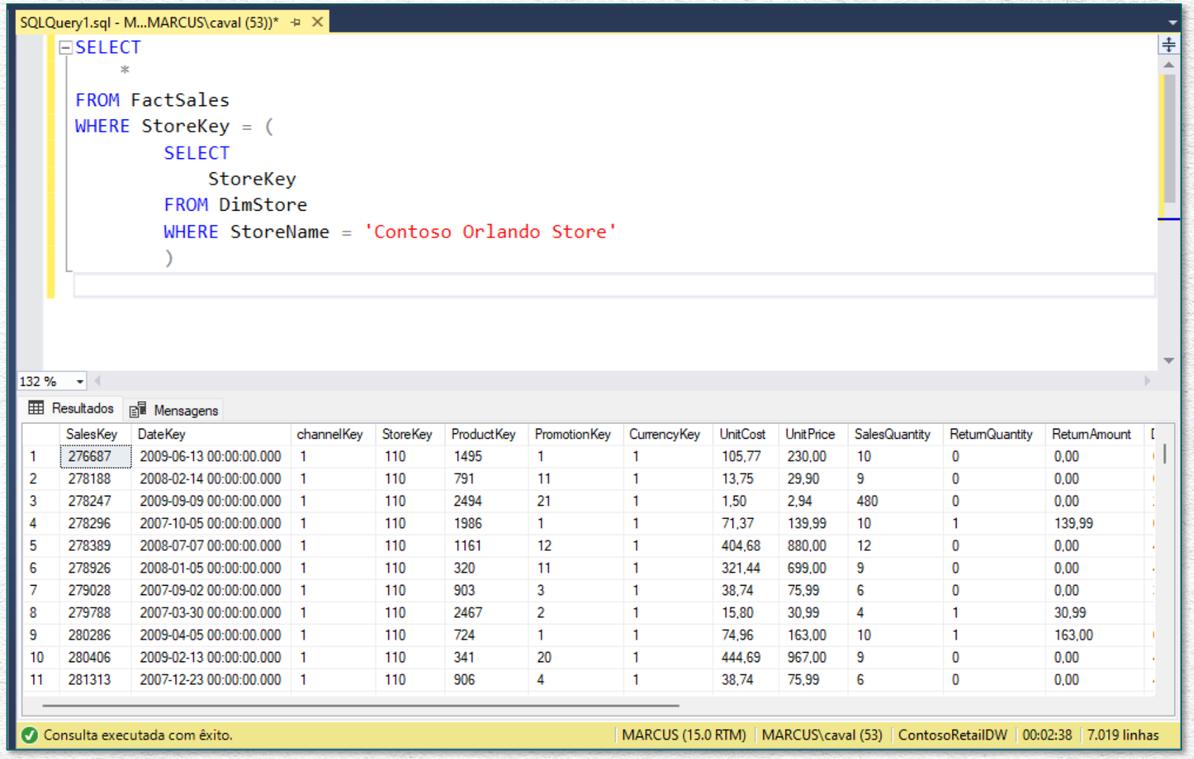
GABARITOS

1

Questão 1

Para fins fiscais, a contabilidade da empresa precisa de uma tabela contendo todas as vendas referentes à loja 'Contoso Orlando Store'. Isso porque essa loja encontra-se em uma região onde a tributação foi modificada recente.

Portanto, crie uma consulta ao Banco de Dados para obter uma tabela FactSales contendo todas as vendas desta loja.



	SalesKey	DateKey	channelKey	StoreKey	ProductKey	PromotionKey	CurrencyKey	UnitCost	UnitPrice	SalesQuantity	ReturnQuantity	ReturnAmount
1	276687	2009-06-13 00:00:00.000	1	110	1495	1	1	105,77	230,00	10	0	0,00
2	278188	2008-02-14 00:00:00.000	1	110	791	11	1	13,75	29,90	9	0	0,00
3	278247	2009-09-09 00:00:00.000	1	110	2494	21	1	1,50	2,94	480	0	0,00
4	278296	2007-10-05 00:00:00.000	1	110	1986	1	1	71,37	139,99	10	1	139,99
5	278389	2008-07-07 00:00:00.000	1	110	1161	12	1	404,68	880,00	12	0	0,00
6	278926	2008-01-05 00:00:00.000	1	110	320	11	1	321,44	699,00	9	0	0,00
7	279028	2007-09-02 00:00:00.000	1	110	903	3	1	38,74	75,99	6	0	0,00
8	279788	2007-03-30 00:00:00.000	1	110	2467	2	1	15,80	30,99	4	1	30,99
9	280286	2009-04-05 00:00:00.000	1	110	724	1	1	74,96	163,00	10	1	163,00
10	280406	2009-02-13 00:00:00.000	1	110	341	20	1	444,69	967,00	9	0	0,00
11	281313	2007-12-23 00:00:00.000	1	110	906	4	1	38,74	75,99	6	0	0,00

GABARITOS

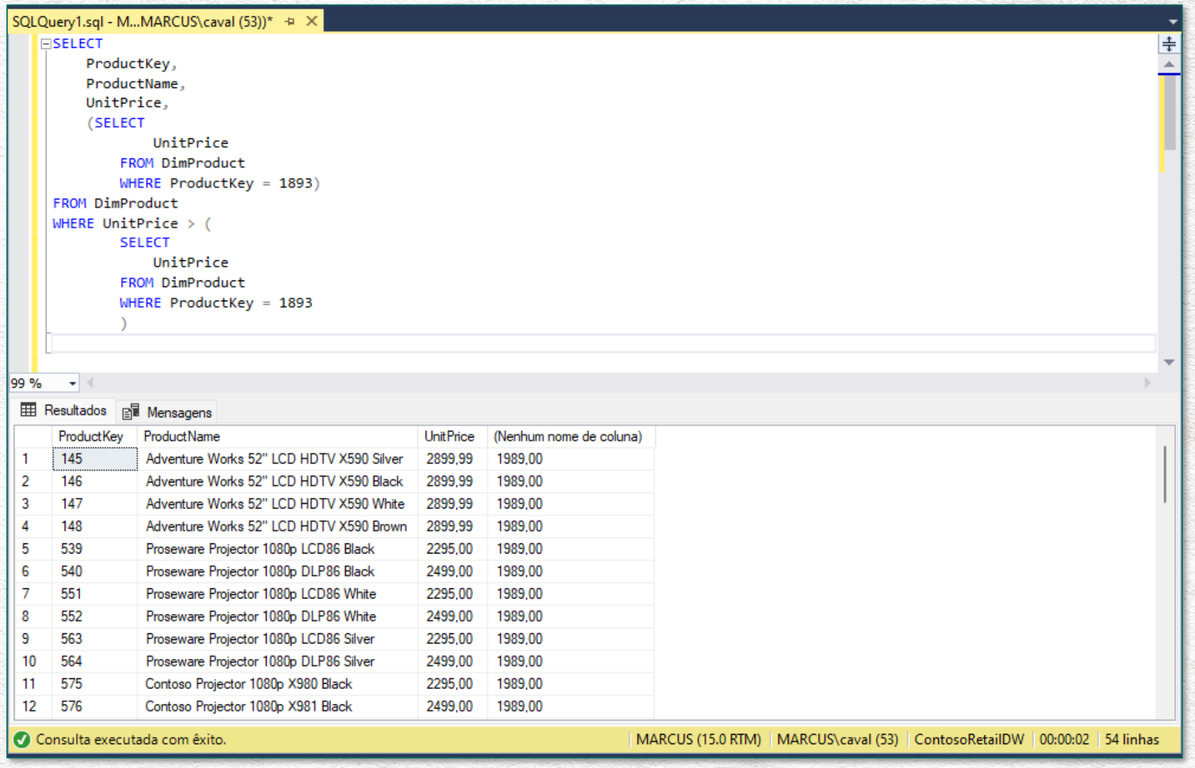
2

Questão 2

O setor de controle de produtos quer fazer uma análise para descobrir quais são os produtos que possuem um UnitPrice maior que o UnitPrice do produto de ID igual a 1893.

a) A sua consulta resultante deve conter as colunas ProductKey, ProductName e UnitPrice da tabela DimProduct.

b) Nessa query você também deve retornar uma coluna extra, que informe o UnitPrice do produto 1893.



	ProductKey	ProductName	UnitPrice	(Nenhum nome de coluna)
1	145	Adventure Works 52" LCD HDTV X590 Silver	2899,99	1989,00
2	146	Adventure Works 52" LCD HDTV X590 Black	2899,99	1989,00
3	147	Adventure Works 52" LCD HDTV X590 White	2899,99	1989,00
4	148	Adventure Works 52" LCD HDTV X590 Brown	2899,99	1989,00
5	539	Proseware Projector 1080p LCD86 Black	2295,00	1989,00
6	540	Proseware Projector 1080p DLP86 Black	2499,00	1989,00
7	551	Proseware Projector 1080p LCD86 White	2295,00	1989,00
8	552	Proseware Projector 1080p DLP86 White	2499,00	1989,00
9	563	Proseware Projector 1080p LCD86 Silver	2295,00	1989,00
10	564	Proseware Projector 1080p DLP86 Silver	2499,00	1989,00
11	575	Contoso Projector 1080p X980 Black	2295,00	1989,00
12	576	Contoso Projector 1080p X981 Black	2499,00	1989,00

GABARITOS

3

Questão 3

A empresa Contoso criou um programa de bonificação chamado "**Todos por 1**". Este programa consistia no seguinte: 1 funcionário seria escolhido ao final do ano como o funcionário destaque, só que a bonificação seria recebida por todos da área daquele funcionário em particular. O objetivo desse programa seria o de incentivar a colaboração coletiva entre os funcionários de uma mesma área. Desta forma, o funcionário destaque beneficiaria não só a si, mas também a todos os colegas de sua área.

Ao final do ano, o funcionário escolhido como destaque foi o **Miguel Severino**. Isso significa que todos os funcionários da área do Miguel seriam beneficiados com o programa.

O seu objetivo é realizar uma consulta à tabela **DimEmployee** e retornar todos os funcionários da área "vencedora" para que o setor Financeiro possa realizar os pagamentos das bonificações.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    *
FROM DimEmployee
WHERE DepartmentName = (
            SELECT
                DepartmentName
            FROM DimEmployee
            WHERE FirstName = 'Miguel'
            AND
            LastName = 'Severino'
            )
```

132 %

Resultados | Mensagens

	EmployeeKey	ParentEmployeeKey	FirstName	LastName	MiddleName	Title	HireDate	BirthDate	EmailAddress	Phone	MaritalSta
1	1	18	Kim	Abercrombie	NULL	Sales Region Manager	1996-07-31	1972-05-15	guy1@contoso.com	320-555-0195	M
2	9	4	Terry	Adams	NULL	Sales State Manager	1998-01-26	1946-02-16	jolynn0@contoso.com	903-555-0145	S
3	10	5	David	Probst	NULL	Sales State Manager	1998-02-06	1946-07-06	ruth0@contoso.com	145-555-0130	M
4	12	2	Michael	Raheem	NULL	Sales State Manager	1998-02-07	1946-04-27	barry0@contoso.com	206-555-0180	S
5	15	9	Kim	Akers	NULL	Sales Store Manager	1998-03-05	1946-10-01	sidney0@contoso.com	424-555-0189	M
6	16	1	Kate	Taneyhill	NULL	Sales State Manager	1998-03-11	1946-05-03	taylor0@contoso.com	508-555-0165	M
7	17	2	David	Alexander	NULL	Sales State Manager	1998-03-23	1946-08-12	jeffrey0@contoso.com	984-555-0185	S
8	19	4	Michelle	Alexander	NULL	Sales State Manager	1998-04-11	1946-05-06	doris0@contoso.com	328-555-0150	M
9	20	5	Rene	Valdes	NULL	Sales State Manager	1998-04-18	1946-09-08	john0@contoso.com	435-555-0113	M
10	21	1	Michael	Allen	NULL	Sales State Manager	1998-04-29	1946-04-30	diane0@contoso.com	202-555-0151	M
11	22	2	Rohinton	Wadia	NULL	Sales State Manager	1999-01-02	1967-06-15	steven0@contoso.com	925-555-0114	M

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 176 linhas

GABARITOS

4

Questão 4

Faça uma query que retorne os clientes que recebem um salário anual acima da média. A sua query deve retornar as colunas CustomerKey, FirstName, LastName, EmailAddress e YearlyIncome.

Obs: considere apenas os clientes que são 'Pessoas Físicas'.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    CustomerKey,
    FirstName,
    LastName,
    EmailAddress,
    YearlyIncome
FROM DimCustomer
WHERE YearlyIncome > (
            SELECT
                AVG(YearlyIncome)
            FROM DimCustomer
            WHERE CustomerType = 'Person'
            )
AND
CustomerType = 'Person'
ORDER BY YearlyIncome DESC
```

120 %

Resultados | Mensagens

	CustomerKey	FirstName	LastName	EmailAddress	YearlyIncome
1	81	Damien	Chander	damien32@adventure-works.com	170000,00
2	182	Devin	Martin	devin13@adventure-works.com	170000,00
3	245	Alexis	Coleman	alexis28@adventure-works.com	170000,00
4	251	Shannon	Liu	shannon4@adventure-works.com	170000,00
5	287	Hunter	Griffin	hunter16@adventure-works.com	170000,00
6	423	Dustin	Deng	dustin1@adventure-works.com	170000,00
7	435	Andre	Lopez	andre16@adventure-works.com	170000,00
8	468	Arturo	Zheng	arturo21@adventure-works.com	170000,00
9	1124	Wesley	Liang	wesley16@adventure-works.com	170000,00

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:09 | 9.858 linhas

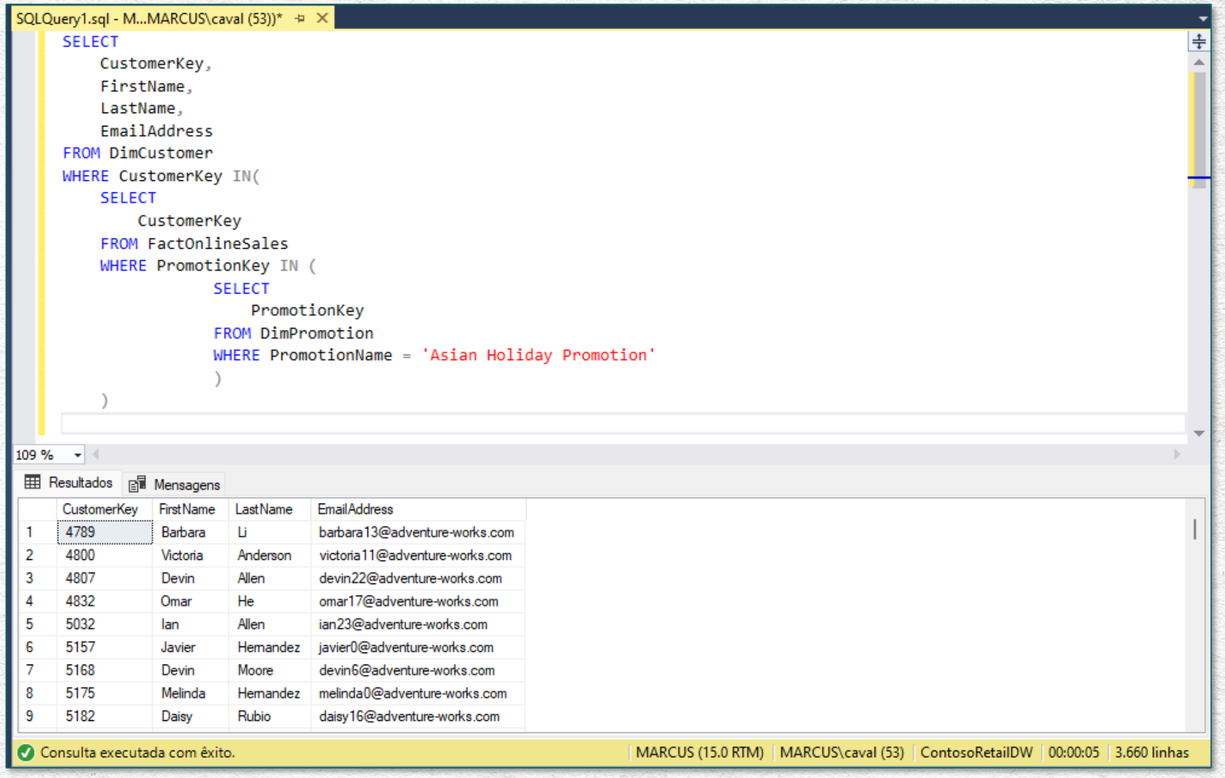
GABARITOS

5

Questão 5

A ação de desconto da Asian Holiday Promotion foi uma das mais bem sucedidas da empresa. Agora, a Contoso quer entender um pouco melhor sobre o perfil dos clientes que compraram produtos com essa promoção.

Seu trabalho é criar uma query que retorne a lista de clientes que compraram nessa promoção.



	CustomerKey	FirstName	LastName	EmailAddress
1	4789	Barbara	Li	barbara13@adventure-works.com
2	4800	Victoria	Anderson	victoria11@adventure-works.com
3	4807	Devin	Allen	devin22@adventure-works.com
4	4832	Omar	He	omar17@adventure-works.com
5	5032	Ian	Allen	ian23@adventure-works.com
6	5157	Javier	Hernandez	javier0@adventure-works.com
7	5168	Devin	Moore	devin6@adventure-works.com
8	5175	Melinda	Hernandez	melinda0@adventure-works.com
9	5182	Daisy	Rubio	daisy16@adventure-works.com

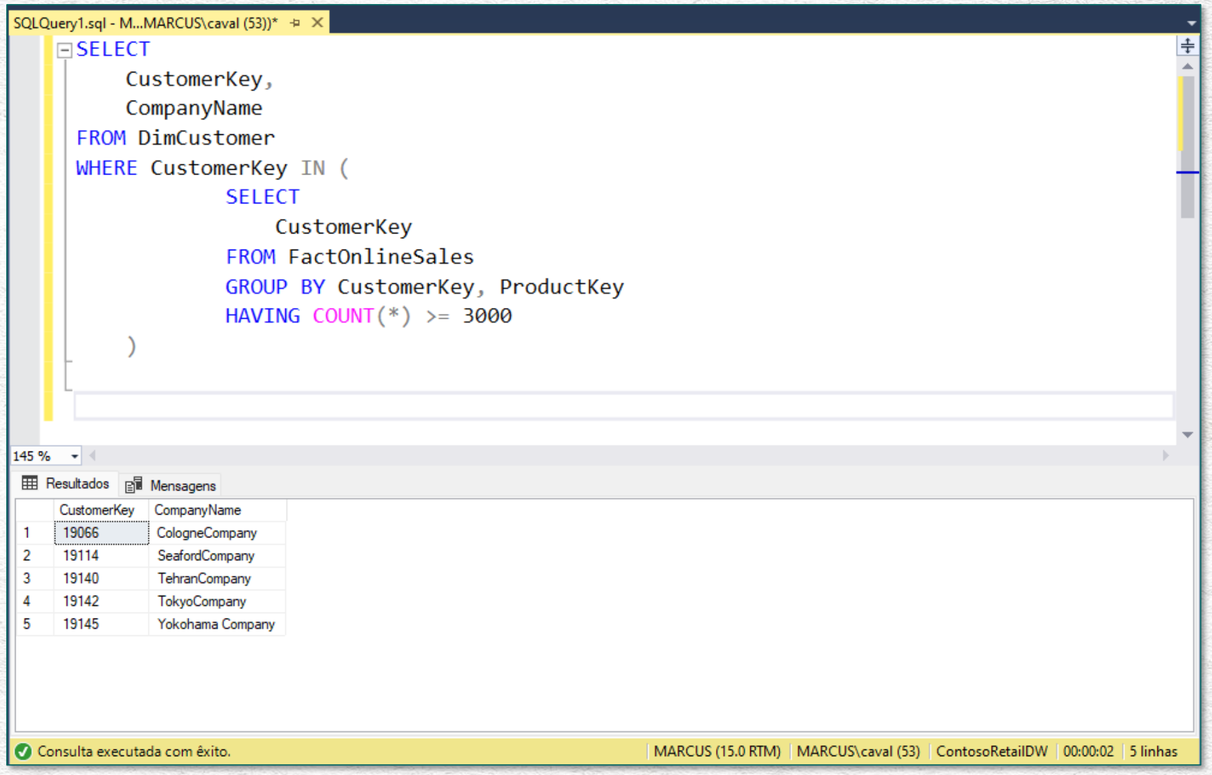
GABARITOS

6

Questão 6

A empresa implementou um programa de fidelização de clientes empresariais. Todos aqueles que comprarem mais de 3000 unidades de um mesmo produto receberá descontos em outras compras.

Você deverá descobrir as informações de CustomerKey e CompanyName destes clientes.



GABARITOS

7

Questão 7

Você deverá criar uma consulta para o setor de vendas que mostre as seguintes colunas da tabela DimProduct:

ProductKey, ProductName, BrandName, UnitPrice, Média de UnitPrice.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    ProductKey,
    ProductName,
    BrandName,
    UnitPrice,
    (SELECT AVG(UnitPrice) FROM DimProduct) AS 'Média de
     UnitPrice'
FROM DimProduct
```

193 %

Resultados | Mensagens

	ProductKey	ProductName	BrandName	UnitPrice	Média de UnitPrice
1	1	Contoso 512MB MP3 Player E51 Silver	Contoso	12,99	356,8301
2	2	Contoso 512MB MP3 Player E51 Blue	Contoso	12,99	356,8301
3	3	Contoso 1G MP3 Player E100 White	Contoso	14,52	356,8301
4	4	Contoso 2G MP3 Player E200 Silver	Contoso	21,57	356,8301
5	5	Contoso 2G MP3 Player E200 Red	Contoso	21,57	356,8301
6	6	Contoso 2G MP3 Player E200 Black	Contoso	21,57	356,8301
7	7	Contoso 2G MP3 Player E200 Blue	Contoso	21,57	356,8301
8	8	Contoso 4G MP3 Player E400 Silver	Contoso	59,99	356,8301
9	9	Contoso 4G MP3 Player E400 Black	Contoso	59,99	356,8301

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 2.517 linhas

GABARITOS

8

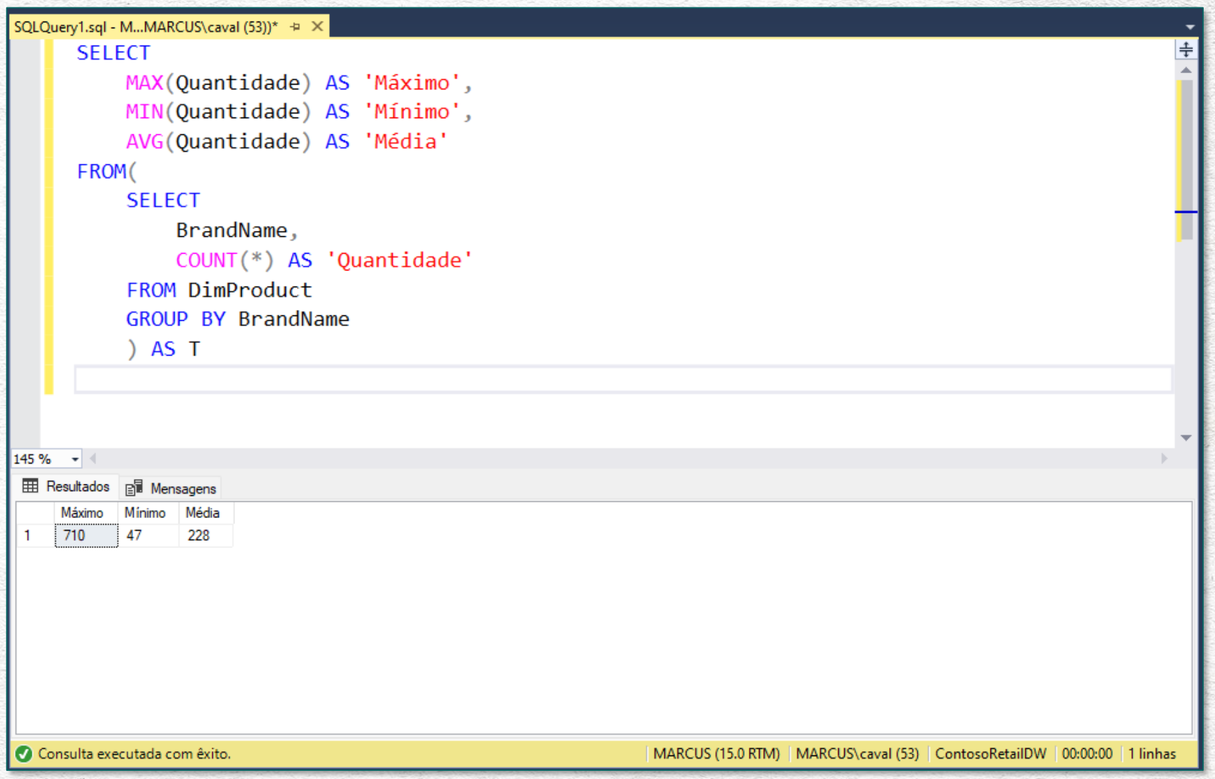
Questão 8

Faça uma consulta para descobrir os seguintes indicadores dos seus produtos:

Maior quantidade de produtos por marca

Menor quantidade de produtos por marca

Média de produtos por marca

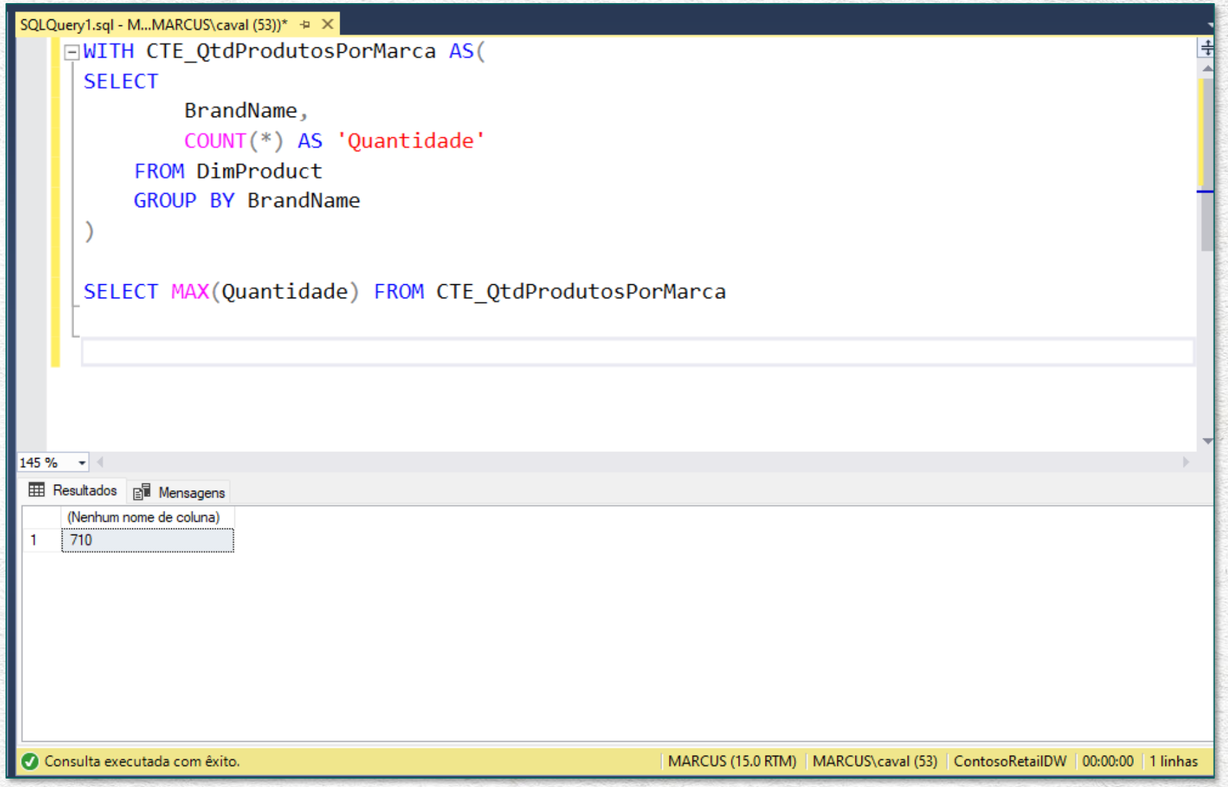


GABARITOS

9

Questão 9

Crie uma CTE que seja o agrupamento da tabela **DimProduct**, armazenando o total de produtos por marca. Em seguida, faça um SELECT nesta CTE, descobrindo qual é a quantidade máxima de produtos para uma marca. Chame esta CTE de **CTE_QtdProdutosPorMarca**.



GABARITOS

10

Questão 10

Crie duas CTEs:

(i) a primeira deve conter as colunas ProductKey, ProductName, ProductSubcategoryKey, BrandName e UnitPrice, da tabela DimProduct, mas apenas os produtos da marca Adventure Works. Chame essa CTE de CTE_ProdutosAdventureWorks.

(ii) a segunda deve conter as colunas ProductSubcategoryKey, ProductSubcategoryName, da tabela DimProductSubcategory mas apenas para as subcategorias 'Televisions' e 'Monitors'. Chame essa CTE de CTE_CategoriaTelevisionsERadio.

Faça um Join entre essas duas CTEs, e o resultado deve ser uma query contendo todas as colunas das duas tabelas. Observe nesse exemplo a diferença entre o LEFT JOIN e o INNER JOIN.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
WITH CTE_ProdutosAdventureWorks AS(
SELECT
    ProductKey,
    ProductName,
    ProductSubcategoryKey,
    BrandName,
    UnitPrice
FROM DimProduct
WHERE BrandName = 'Adventure Works'
),
CTE_CategoriaTelevisionsERadio AS(
SELECT
    ProductSubcategoryKey,
    ProductSubcategoryName
FROM DimProductSubcategory
WHERE ProductSubcategoryName IN ('Televisions', 'Monitors')
)

SELECT
    CTE_ProdutosAdventureWorks.*,
    CTE_CategoriaTelevisionsERadio.ProductSubcategoryName
FROM CTE_ProdutosAdventureWorks
LEFT JOIN CTE_CategoriaTelevisionsERadio
    ON CTE_ProdutosAdventureWorks.ProductSubcategoryKey = CTE_CategoriaTelevisionsERadio.ProductSubcategoryKey
```

99 %

Resultados | Mensagens

	ProductKey	ProductName	ProductSubcategoryKey	BrandName	UnitPrice	ProductSubcategoryName
1	116	Adventure Works 20" CRT TV E15 Silver	9	Adventure Works	169,99	Televisions
2	117	Adventure Works 20" CRT TV E15 Black	9	Adventure Works	169,99	Televisions
3	118	Adventure Works 20" CRT TV E15 White	9	Adventure Works	169,99	Televisions
4	119	Adventure Works 13" Color TV E25 Silver	9	Adventure Works	119,99	Televisions
5	120	Adventure Works 13" Color TV E25 Black	9	Adventure Works	119,99	Televisions

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 192 linhas

MÓDULO 15

SQL

LOOPS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 15 LOOPS

01	WHILE	395
02	WHILE - Exemplo	398
03	BREAK - Exemplo	399
04	CONTINUE - Exemplo	400
05	EXERCÍCIOS	401
06	GABARITOS	403

WHILE

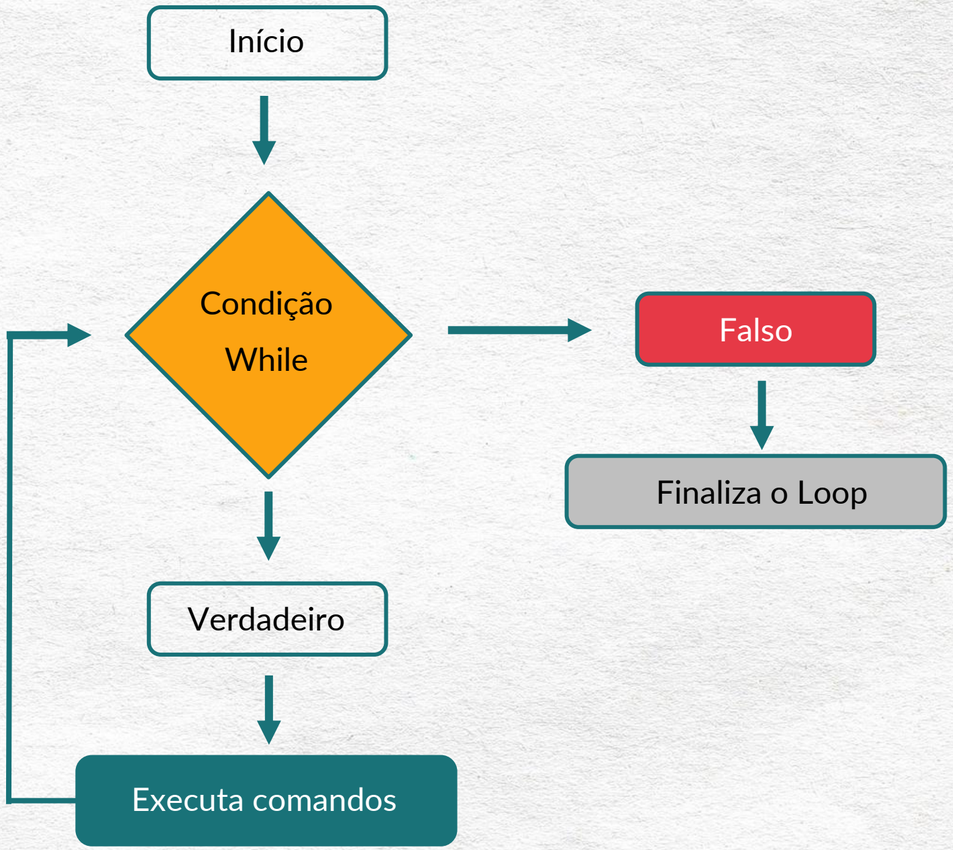
O QUE É UM LOOP

Para que servem?

Um loop em uma linguagem de programação tem como objetivo executar repetidas vezes uma série de comandos.

O While é uma estrutura de repetição que tem exatamente esse objetivo. Com ele, conseguimos executar diversas vezes um ou mais comandos, enquanto uma determinada condição é satisfeita. E a partir do momento que essa condição deixa de ser satisfeita, o loop finaliza.

Ao lado, podemos ver um esquema visual da lógica While.

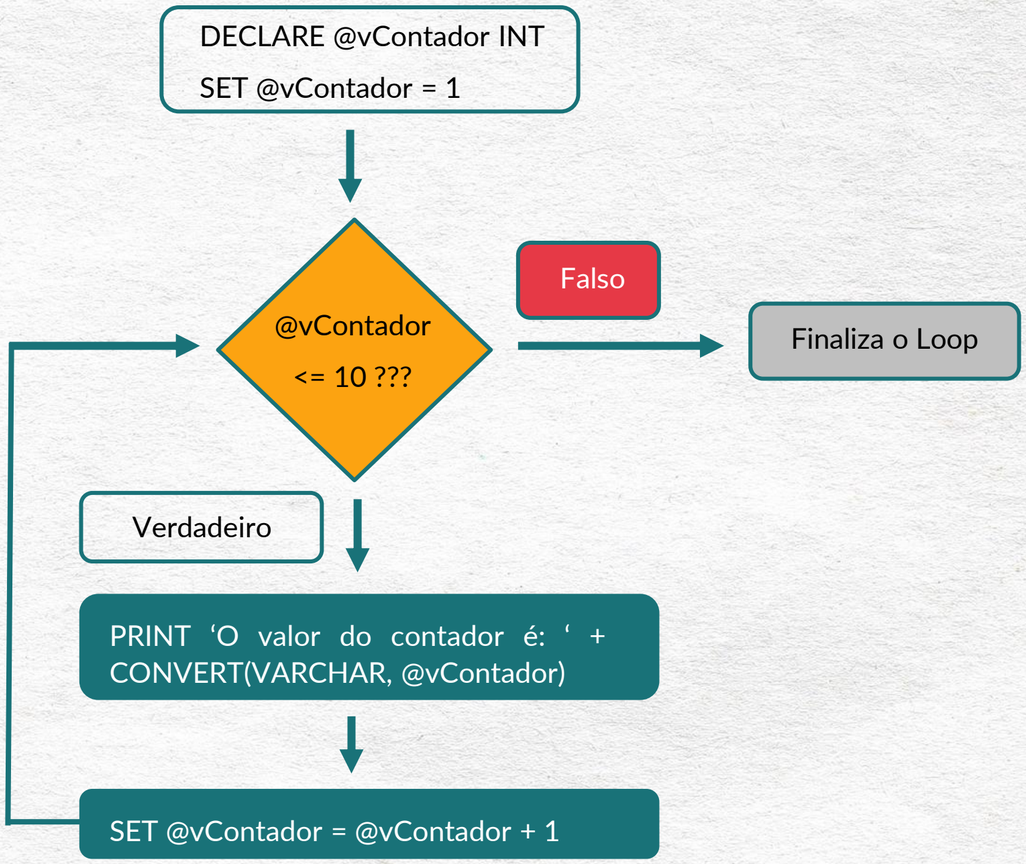


WHILE

EXEMPLO

Imagine que a gente queira criar um contador, que vai de 1 a 10, utilizando a lógica While.

Na imagem ao lado é possível visualizar a lógica.



WHILE

ESTRUTURA BÁSICA

A estrutura básica do WHILE é a seguinte:

```
WHILE condição
BEGIN
        comandos
END
```

Ou seja, enquanto uma determinada **condição** é satisfeita, todos os comandos dentro do bloco BEGIN.. END serão executados.

WHILE – Exemplo

Imagine que a gente queira criar um contador, de 1 até 10, que printasse na tela:

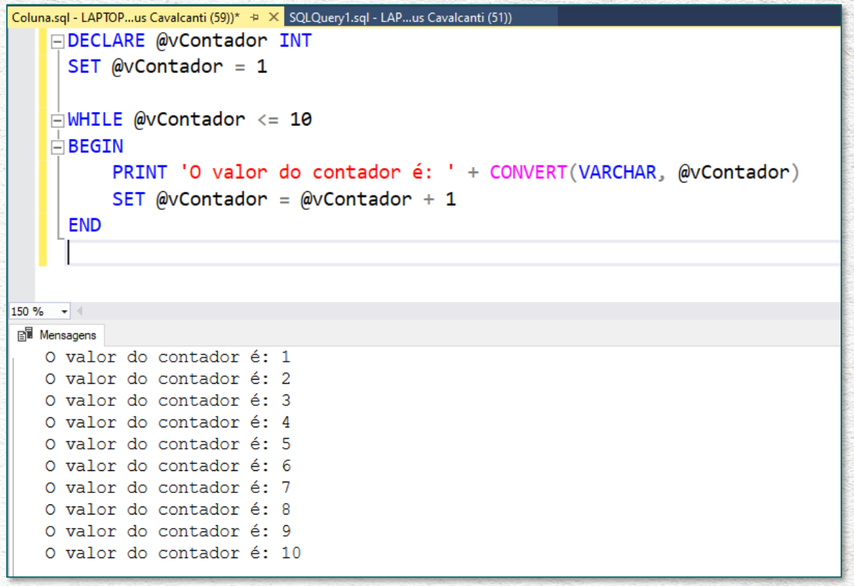
"O valor do contador é: __"

Poderíamos criar uma estrutura em Loop para executar o comando repetidas vezes, da forma como é mostrada no exemplo ao lado.

Iniciamos uma variável chamada @vContador = 1, e a nossa condição WHILE é executar o PRINT enquanto o valor de @vContador for menor ou igual a 10.

Observe que criamos uma linha para incrementar esta variável cada vez que finalizamos um loop (SET @vContador = @vContador + 1).

Desta forma, em algum momento, a condição @vContador não será mais satisfeita e consequentemente o WHILE será finalizado.



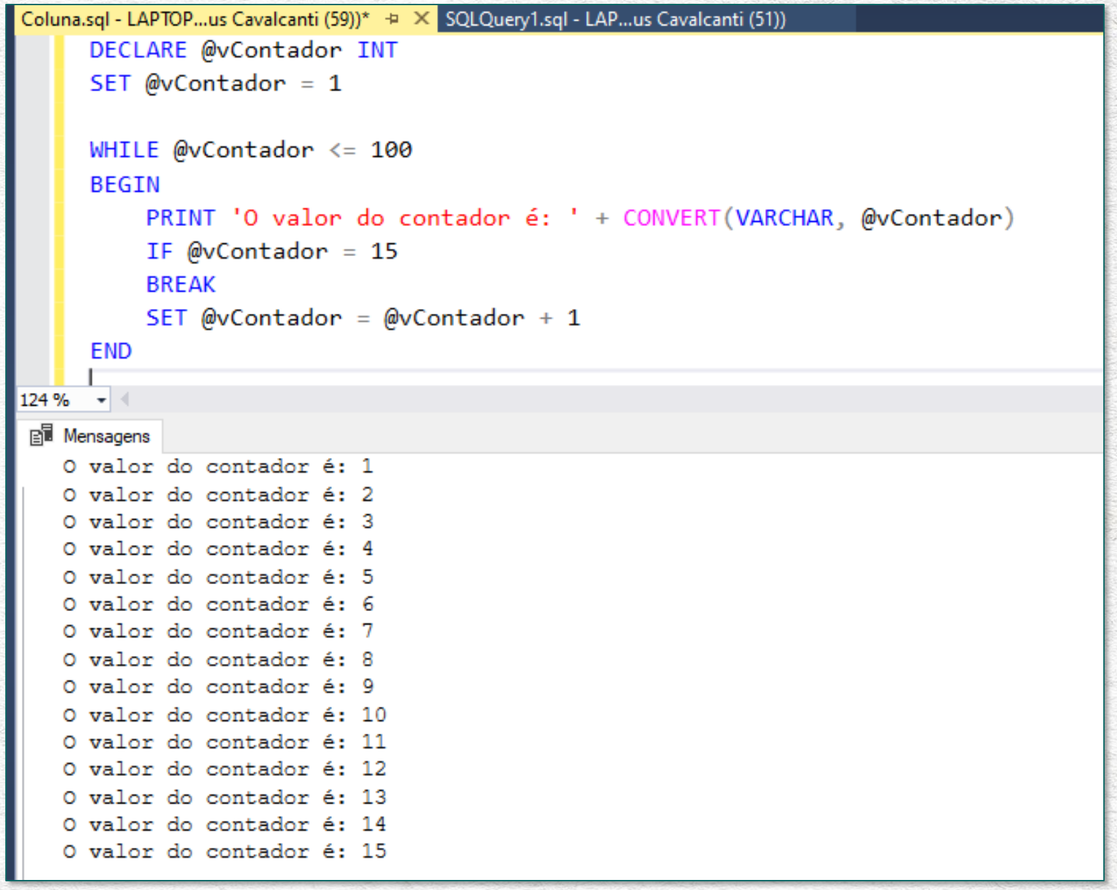
BREAK – Exemplo

Podemos utilizar o comando BREAK para finalizar um loop assim que uma condição intermediária for satisfeita.

Observe no exemplo ao lado que desejamos que o loop WHILE seja executado enquanto o momento em que @vContador for menor ou igual a 100.

Porém, criamos um teste IF para, a cada loop, verificar se o valor de @vContador é igual a 15. Caso seja, o loop é automaticamente interrompido e finalizado.

No exemplo ao lado, observe que o contador vai apenas até 15.



CONTINUE – Exemplo

O comando CONTINUE pode ser usado para pular repetições em um loop. Ou seja, cada vez que uma condição mais interna ao loop é atendida, o loop não executado, mas continua acontecendo até que a condição do WHILE seja atendida.

No exemplo ao lado, temos um contador que deve contar até 10 (WHILE @varContador <= 10). Porém, no meio do código, testamos dois números em particular: 3 e 6. Ou seja, sempre que @varContador for igual a 3 ou @varContador for igual a 6, o comando PRINT é pulado, porém a contagem se mantém funcionando até que @varContador seja igual a 10.

Observe que na contagem ao lado, na janela de mensagens, os números 3 e 6 foram pulados.

```
DECLARE @varContador INT
SET @varContador = 0

WHILE @varContador <= 10
BEGIN
    SET @varContador += 1
    IF @varContador = 3 OR @varContador = 6
        CONTINUE
    PRINT @varContador
END
```

22 %

Mensagens

```
1
2
4
5
7
8
9
10
11

Horário de conclusão: 2022-01-06T16:43:53.2218654-03:00
```

EXERCÍCIOS

1

Questão 1

Utilize o Loop While para criar um contador que comece em um valor inicial @ValorInicial e termine em um valor final @ValorFinal. Você deverá printar na tela a seguinte frase:

“O valor do contador é: “ + ___

2

Questão 2

Você deverá criar uma estrutura de repetição que printe na tela a quantidade de contratações para cada ano, desde 1996 até 2003. A informação de data de contratação encontra-se na coluna HireDate da tabela **DimEmployee**. Utilize o formato:

X contratações em 1996

Y contratações em 1997

Z contratações em 1998

...

...

N contratações em 2003

Obs: a coluna HireDate contém a data completa (dd/mm/aaaa). Lembrando que você deverá printar a quantidade de contratações **por ano**.

EXERCÍCIOS

3

Questão 3

Utilize um Loop While para criar uma tabela chamada Calendario, contendo uma coluna que comece com a data 01/01/2021 e vá até 31/12/2021.

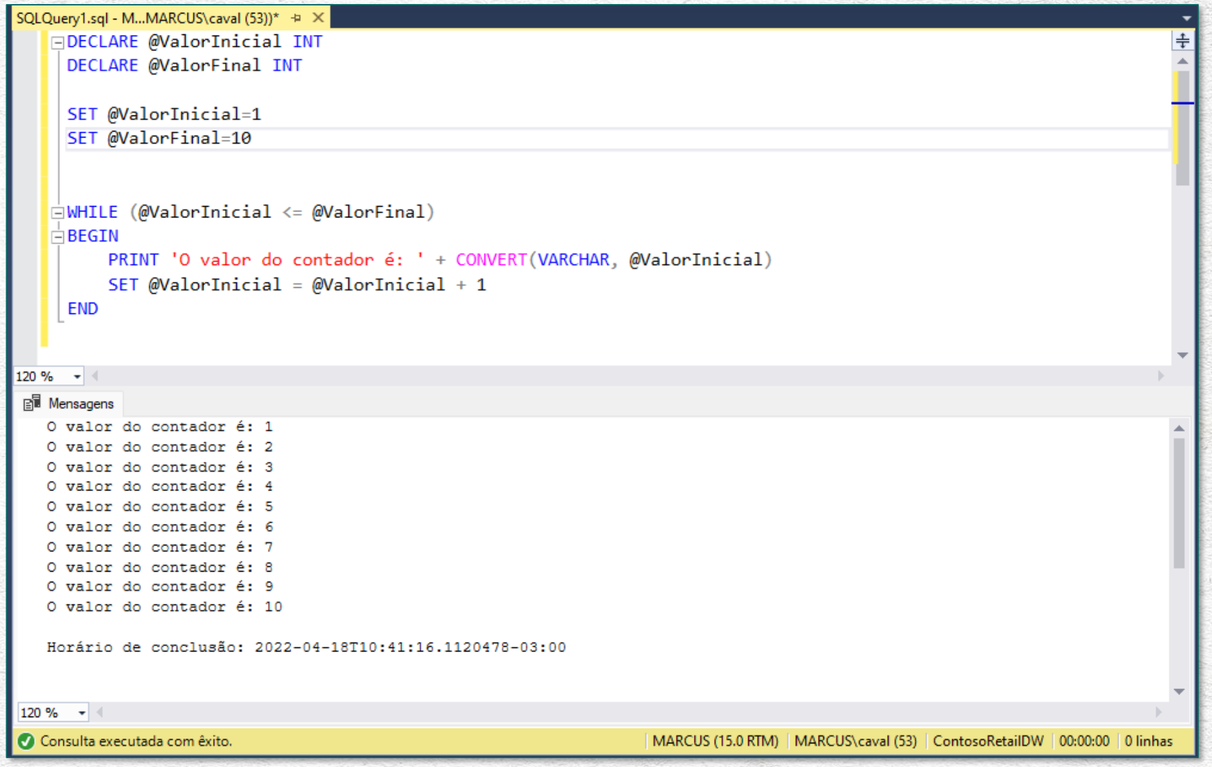
GABARITOS

1

Questão 1

Utilize o Loop While para criar um contador que comece em um valor inicial @ValorInicial e termine em um valor final @ValorFinal. Você deverá printar na tela a seguinte frase:

“O valor do contador é: “ + ___



GABARITOS

2

Questão 2

Você deverá criar uma estrutura de repetição que printe na tela a quantidade de contratações para cada ano, desde 1996 até 2003. A informação de data de contratação encontra-se na coluna HireDate da tabela **DimEmployee**. Utilize o formato:

X contratações em 1996

Y contratações em 1997

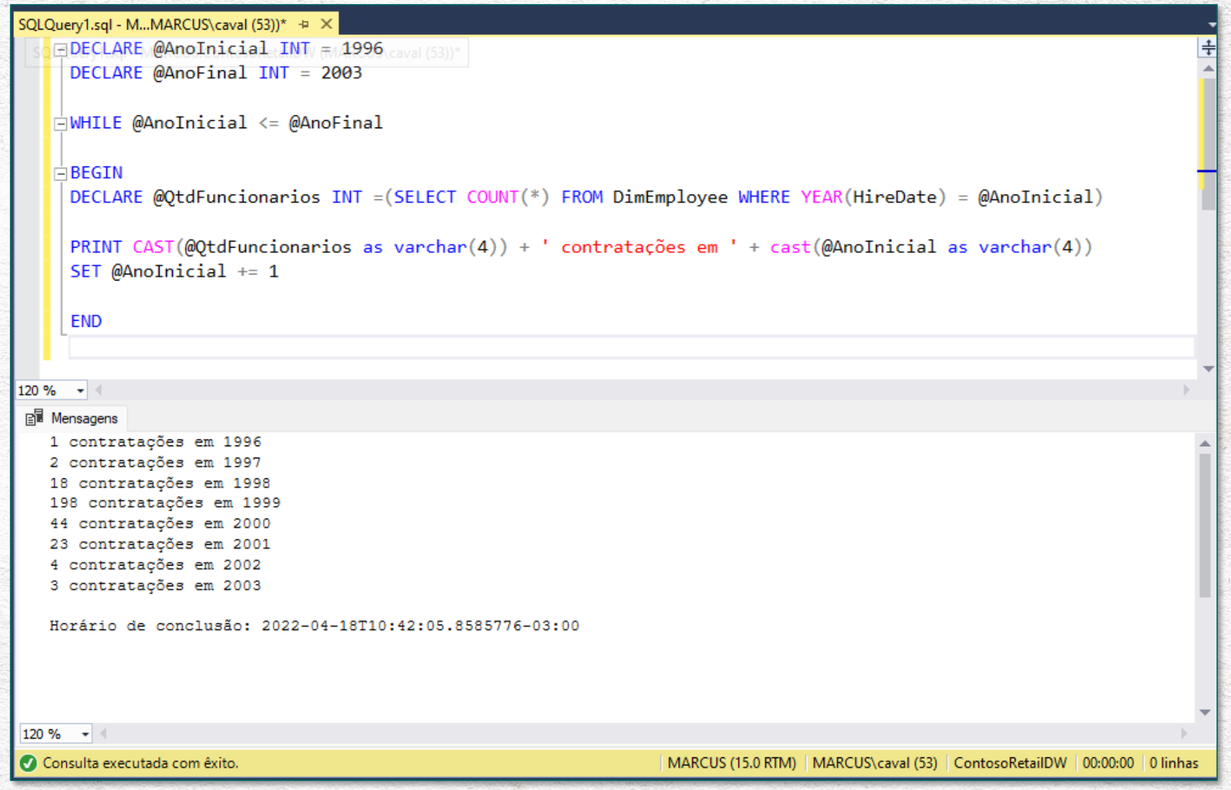
Z contratações em 1998

...

...

N contratações em 2003

Obs: a coluna HireDate contém a data completa (dd/mm/aaaa). Lembrando que você deverá printar a quantidade de contratações **por ano**.

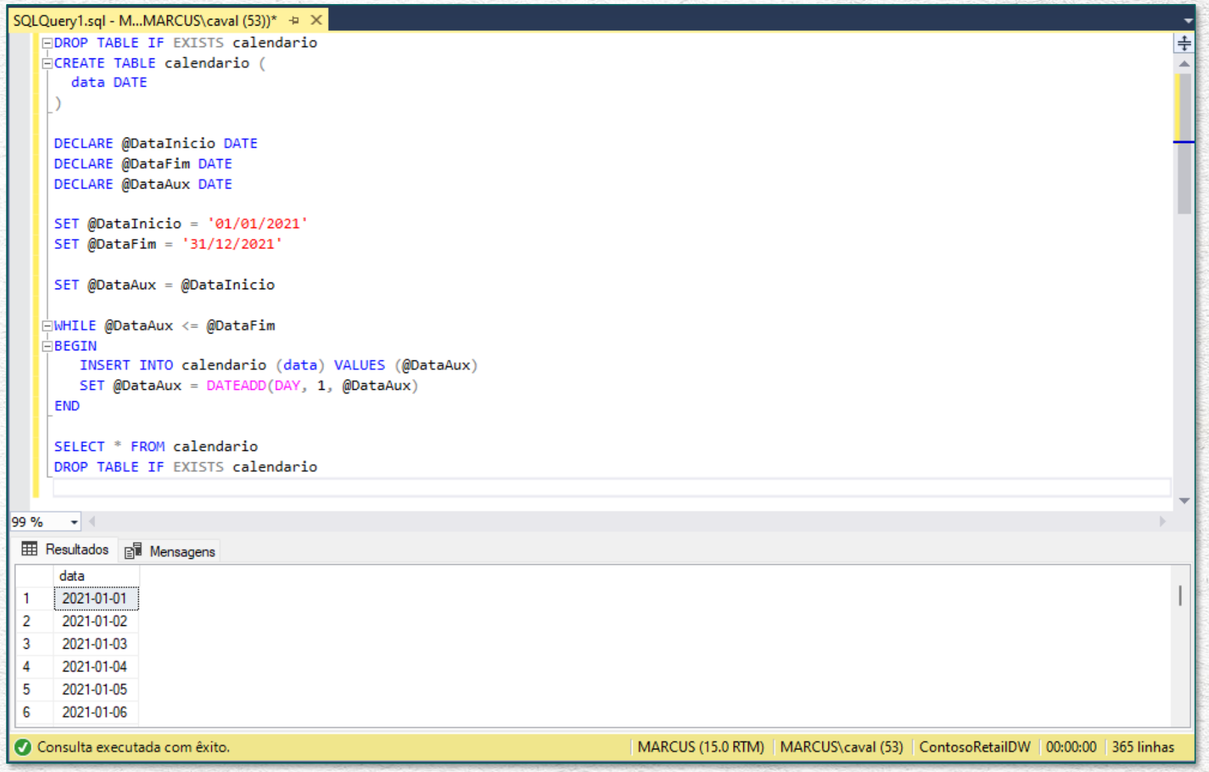


GABARITOS

3

Questão 3

Utilize um Loop While para criar uma tabela chamada Calendario, contendo uma coluna que comece com a data 01/01/2021 e vá até 31/12/2021.



MÓDULO 16

SQL

WINDOW FUNCTIONS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 16 WINDOW FUNCTIONS

01	WINDOW FUNCTIONS - Introdução	408
02	Código para criar a tabela lojas	417
03	Funções Agregação - SUM, COUNT, AVG, MIN, MAX	418
04	Cálculo de participação % (Parte 1)	421
05	Cálculo de participação % (Parte 2)	422
06	Funções de Classificação	423
07	ROW_NUMBER, RANK, DENSE_RANK e NTILE	424
08	Funções de Classificação com PARTITION BY	425
09	RANK com GROUP BY	426
10	Código para criar a tabela Resultado	427
11	Cálculo de soma móvel	428
12	Cálculo de média móvel	434
13	Cálculo de acumulado	435
14	Resumo das janelas	437
15	Funções de Offset	438
16	Função LAG	439
17	Função LEAD	442
18	Função NULLIF	443
19	Cálculo MoM	444
20	FIRST_VALUE e LAST_VALUE	449
21	DESAFIO 1 – EXERCÍCIOS	450
22	DESAFIO 2 - EXERCÍCIOS	453
23	DESAFIO 3 - EXERCÍCIOS	455
24	DESAFIO 1 – GABARITOS	457
25	DESAFIO 2 – GABARITOS	462
26	DESAFIO 3 - GABARITOS	465

WINDOW FUNCTIONS – Introdução

PARA QUE SERVEM?

- São utilizadas para cálculos mais avançados de análises de dados;
- Possuem um uso semelhante ao GROUP BY, só que mais avançado e personalizável;
- A instrução OVER permite definir qual será a “janela” (conjunto de linhas) a ser considerado no cálculo;
- A instrução PARTITION BY divide o conjunto em “partições” nas quais as funções de janela são aplicadas.

Window Functions – Introdução

QUAIS AS FINALIDADES?

As funções de janela possuem as seguintes finalidades:

1. Cálculos de agregação: COUNT, SUM, AVG, MIN, MAX
2. Cálculos de deslocamento: FIRST_VALUE, LAST_VALUE, LEAD, LAG
3. Cálculos estatísticos: RANK, DENSE_RANK, NTILE

Window Functions – Introdução

Imagine a tabela abaixo, com a rede de 10 lojas de uma determinada empresa.

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida
1	Botafogo Praia&Mar	Sudeste	1500
2	Lojas Vitoria	Sudeste	800
3	Emporio Mineirinho	Sudeste	2300
4	Central Paulista	Sudeste	1800
5	Casa Flor & Anópolis	Sul	2100
6	Pampas & Co	Sul	990
7	Paraná Papéis	Sul	2800
8	Amazonas Prime	Norte	4200
9	Pará Bens	Norte	3200
10	Tintas Rio Branco	Norte	2400

```
SELECT SUM(Qtd_Vendida) AS 'Total Vendido' FROM Lojas
```

Total Vendido
22090

Como poderíamos fazer para calcular o % vendido por cada loja?

Window Functions – Introdução

Imagine a tabela abaixo, com a rede de 10 lojas de uma determinada empresa. A imagem 1 mostra as 10 lojas e suas respectivas vendas. Já a figura 2 mostra o comando em SQL para calcular a soma total da quantidade vendida de todas essas lojas.

1

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida
1	Botafogo Praia&Mar	Sudeste	1500
2	Lojas Vitoria	Sudeste	800
3	Emporio Mineirinho	Sudeste	2300
4	Central Paulista	Sudeste	1800
5	Casa Flor & Anópolis	Sul	2100
6	Pampas & Co	Sul	990
7	Paraná Papéis	Sul	2800
8	Amazonas Prime	Norte	4200
9	Pará Bens	Norte	3200
10	Tintas Rio Branco	Norte	2400

2

```
SELECT SUM(Qtd_Vendida) AS 'Total Vendido' FROM Lojas
```

Total Vendido
22090

Como poderíamos fazer para calcular o % vendido por cada loja?

Window Functions – Introdução

Primeiro, teríamos que calcular o total vendido e incluir na nossa tabela.

Porém, ao executar o código ao lado, incluindo uma nova coluna a ser calculada pela função SUM, obtemos o erro abaixo:

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) AS 'Total Vendido'
FROM Lojas
```

Mensagens

```
Mensagem 8120, Nível 16, Estado 1, Linha 22
A coluna 'Lojas.ID_Loja' é inválida na lista de seleção porque não está contida em uma função de agregação nem na cláusula GROUP BY.

Horário de conclusão: 2021-09-21T13:00:52.2561016-03:00
```

O erro ocorreu porque quando incluímos o SUM dentro do SELECT, ele espera que a gente faça algum agrupamento com o GROUP BY.

Window Functions – Introdução

Uma solução que poderíamos pensar seria utilizar o GROUP BY, agrupando por todas as colunas da tabela, dado que queremos visualizar todas essas colunas.

Porém, ao fazer isso, o Total Vendido é calculado de forma diferente para cada linha, um resultado que não queremos. O que queremos na verdade é poder ter um valor total para todas as linhas, no caso 22090, para poder dividir os valores de Qtd_Vendida por este total.

Portanto, para resolver este problema, temos que usar funções de janela e a instrução OVER.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) AS 'Total Vendido'
FROM Lojas
GROUP BY ID_Loja, Nome_Loja, Regiao, Qtd_Vendida
```

04 %

Resultados | Mensagens

	ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido
1	1	Botafogo Praia&Mar	Sudeste	1500	1500
2	2	Lojas Vitoria	Sudeste	800	800
3	3	Emporio Mineirinho	Sudeste	2300	2300
4	4	Central Paulista	Sudeste	1800	1800
5	5	Casa Flor & Anópolis	Sul	2100	2100
6	6	Pampas & Co	Sul	990	990
7	7	Paraná Papéis	Sul	2800	2800
8	8	Amazonas Prime	Norte	4200	4200
9	9	Pará Bens	Norte	3200	3200
10	10	Tintas Rio Branco	Norte	2400	2400

Window Functions – Introdução

Simplesmente adicionamos o OVER() logo depois do SUM e o Total Vendido será calculado igual para todas as linhas, permitindo que a gente faça o cálculo do %.

Lembrando que a instrução OVER permite definir qual será a “janela” (conjunto de linhas) a ser considerado no cálculo.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER() AS 'Total Vendido'
FROM Lojas
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido
1	Botafogo Praia&Mar	Sudeste	1500	22090
2	Lojas Vitoria	Sudeste	800	22090
3	Emporio Mineirinho	Sudeste	2300	22090
4	Central Paulista	Sudeste	1800	22090
5	Casa Flor & Anópolis	Sul	2100	22090
6	Pampas & Co	Sul	990	22090
7	Paraná Papéis	Sul	2800	22090
8	Amazonas Prime	Norte	4200	22090
9	Pará Bens	Norte	3200	22090
10	Tintas Rio Branco	Norte	2400	22090

Window Functions – Introdução

E se a gente usasse o PARTITION BY junto com o OVER?

Lembrando que a instrução PARTITION BY divide o conjunto em "partições" nas quais as funções de janela são aplicadas.

Se a gente informar o ID_Loja junto com o PARTITION BY, a janela vai considerar os agrupamentos de ID_Loja. Como cada ID é um ID diferente, o resultado do Total Vendido não faz nenhum agrupamento, retornando o resultado diferente para cada loja.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER(PARTITION BY ID_Loja) AS 'Total Vendido'
FROM Lojas
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido
1	Botafogo Praia&Mar	Sudeste	1500	1500
2	Lojas Vitoria	Sudeste	800	800
3	Emporio Mineirinho	Sudeste	2300	2300
4	Central Paulista	Sudeste	1800	1800
5	Casa Flor & Anópolis	Sul	2100	2100
6	Pampas & Co	Sul	990	990
7	Paraná Papéis	Sul	2800	2800
8	Amazonas Prime	Norte	4200	4200
9	Pará Bens	Norte	3200	3200
10	Tintas Rio Branco	Norte	2400	2400

Window Functions – Introdução

Agora se a gente informar a Regiao junto com o PARTITION BY, a janela vai considerar os agrupamentos de Regiao.

Como as regiões podem se repetir para diferentes lojas, o cálculo da soma vai levar isso em consideração.

Ou seja, na linha de uma loja do Sudeste, o Total Vendido será calculado de acordo com aquela região.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Total Vendido'
FROM Lojas
ORDER BY ID_Loja
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido
1	Botafogo Praia&Mar	Sudeste	1500	6400
2	Lojas Vitoria	Sudeste	800	6400
3	Emporio Mineirinho	Sudeste	2300	6400
4	Central Paulista	Sudeste	1800	6400
5	Casa Flor & Anópolis	Sul	2100	5890
6	Pampas & Co	Sul	990	5890
7	Paraná Papéis	Sul	2800	5890
8	Amazonas Prime	Norte	4200	9800
9	Pará Bens	Norte	3200	9800
10	Tintas Rio Branco	Norte	2400	9800

Código para criar a tabela lojas

Para os exemplos e explicações das funções de janela, será necessário que você crie a tabela ao lado.

Portanto, copie o código e replique no seu SQL Server para que assim você possa acompanhar as próximas aulas.

```
CREATE TABLE Lojas(
ID_Loja INT,
Nome_Loja VARCHAR(100),
Regiao VARCHAR(100),
Qtd_Vendida FLOAT)


INSERT INTO Lojas(ID_Loja, Nome_Loja, Regiao, Qtd_Vendida)
VALUES
    (1, 'Botafogo Praia&Mar', 'Sudeste', 1800),
    (2, 'Lojas Vitoria', 'Sudeste', 800),
    (3, 'Emporio Mineirinho', 'Sudeste', 2300),
    (4, 'Central Paulista', 'Sudeste', 1800),
    (5, 'Rio 90 graus', 'Sudeste', 700),
    (6, 'Casa Flor & Anópolis', 'Sul', 2100),
    (7, 'Pampas & Co', 'Sul', 990),
    (8, 'Paraná Papéis', 'Sul', 2800),
    (9, 'Amazonas Prime', 'Norte', 4200),
    (10, 'Pará Bens', 'Norte', 3200),
    (11, 'Tintas Rio Branco', 'Norte', 1500),
    (12, 'Nordestemido Hall', 'Nordeste', 1910),
    (13, 'Cachoerinha Loft', 'Nordeste', 2380)
```

Funções de Agregação – SUM, COUNT, AVG, MIN, MAX

OVER

Os primeiros exemplos que vamos ver é com as funções de agregação. Conseguimos usar essas funções para cálculos de totais de contagens, somas, médias, mínimos e máximos.

Na imagem ao lado é possível visualizar o resultado de cada uma dessas funções de agregação aplicadas às funções de janela.

Um detalhe importante é que neste exemplo, utilizamos a instrução **OVER** apenas. Dessa forma, o cálculo do total será considerando todas as lojas. Se quiséssemos calcular totais levando em consideração cada região (por exemplo, o pra uma loja do Sudeste, calcular o total do sudeste) teríamos que utilizar a instrução **OVER** em conjunto com a **PARTITION BY**.

```sql
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    COUNT(*) OVER() AS 'Qtd. Lojas',
    SUM(Qtd_Vendida) OVER() AS 'Total Vendas',
    AVG(Qtd_Vendida) OVER() AS 'Média Vendas',
    MIN(Qtd_Vendida) OVER() AS 'Mínimo Vendas',
    MAX(Qtd_Vendida) OVER() AS 'Máximo Vendas'
FROM Lojas
ORDER BY ID_Loja
```

122 %

Resultados | Mensagens

	ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Qtd. Lojas	Total Vendas	Média Vendas	Mínimo Vendas	Máximo Vendas
1	1	Botafogo Praia&Mar	Sudeste	1800	13	26480	2036,92307692308	700	4200
2	2	Lojas Vitoria	Sudeste	800	13	26480	2036,92307692308	700	4200
3	3	Emporio Mineirinho	Sudeste	2300	13	26480	2036,92307692308	700	4200
4	4	Central Paulista	Sudeste	1800	13	26480	2036,92307692308	700	4200
5	5	Rio 90 graus	Sudeste	700	13	26480	2036,92307692308	700	4200
6	6	Casa Flor & Anópolis	Sul	2100	13	26480	2036,92307692308	700	4200
7	7	Pampas & Co	Sul	990	13	26480	2036,92307692308	700	4200
8	8	Paraná Papéis	Sul	2800	13	26480	2036,92307692308	700	4200
9	9	Amazonas Prime	Norte	4200	13	26480	2036,92307692308	700	4200
10	10	Pará Bens	Norte	3200	13	26480	2036,92307692308	700	4200
11	11	Tintas Rio Branco	Norte	1500	13	26480	2036,92307692308	700	4200
12	12	Nordestemido Hall	Norde...	1910	13	26480	2036,92307692308	700	4200
13	13	Cachoerinha Loft	Norde...	2380	13	26480	2036,92307692308	700	4200

Funções de Agregação – SUM, COUNT, AVG, MIN, MAX

OVER + PARTITION BY

Se quisermos calcular estas agregações para fazer análises considerando agrupamentos específicos (por região, por exemplo) apenas adicionamos dentro do OVER a instrução PARTITION BY e informamos por qual coluna queremos fazer um agrupamento. Teria uma lógica semelhante ao GROUP BY, sendo que não agrupamos uma tabela, e sim o cálculo.

Ou seja, os cálculos de contagem, soma, média, mínimo e máximo são feitos considerando um determinado grupo especificado. No caso do exemplo, o grupo é a coluna Regiao.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    COUNT(*) OVER(PARTITION BY Regiao) AS 'Qtd Lojas P/ Região',
    SUM(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Total Vendas P/ Região',
    AVG(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Média Vendas P/ Região',
    MIN(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Mínimo Vendas P/ Região',
    MAX(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Máximo Vendas P/ Região'
FROM Lojas
ORDER BY ID_Loja
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Qtd Lojas P/ Região	Total Vendas P/ Região	Média Vendas P/ Região	Mínimo Vendas P/ Região	Máximo Vendas P/ Região
1	Botafogo Praia&Mar	Sudeste	1800	5	7400	1480	700	2300
2	Lojas Vitoria	Sudeste	800	5	7400	1480	700	2300
3	Emporio Mineirinho	Sudeste	2300	5	7400	1480	700	2300
4	Central Paulista	Sudeste	1800	5	7400	1480	700	2300
5	Rio 90 graus	Sudeste	700	5	7400	1480	700	2300
6	Casa Flor & Anópolis	Sul	2100	3	5890	1963,33333333333	990	2800
7	Pampas & Co	Sul	990	3	5890	1963,33333333333	990	2800
8	Paraná Papéis	Sul	2800	3	5890	1963,33333333333	990	2800
9	Amazonas Prime	Norte	4200	3	8900	2966,66666666667	1500	4200
10	Pará Bens	Norte	3200	3	8900	2966,66666666667	1500	4200
11	Tintas Rio Branco	Norte	1500	3	8900	2966,66666666667	1500	4200
12	Nordestemido Hall	Nordeste	1910	2	4290	2145	1910	2380
13	Cachoerinha Loft	Nordeste	2380	2	4290	2145	1910	2380

Funções de Agregação – SUM, COUNT, AVG, MIN, MAX

OVER + PARTITION BY

Olhando mais atentamente para o exemplo anterior, vemos que a coluna 'Qtd Lojas P/ Região' nada mais é do que o total por região.

Para a linha 1, da loja Botafogo Praia&Mar, por exemplo, o valor 5 significa que há 5 lojas da região sudeste.

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Qtd Lojas P/ Região
1	Botafogo Praia&Mar	Sudeste	1800	5
2	Lojas Vitoria	Sudeste	800	5
3	Emporio Mineirinho	Sudeste	2300	5
4	Central Paulista	Sudeste	1800	5
5	Rio 90 graus	Sudeste	700	5
6	Casa Flor & Anópolis	Sul	2100	3
7	Pampas & Co	Sul	990	3
8	Paraná Papéis	Sul	2800	3
9	Amazonas Prime	Norte	4200	3
10	Pará Bens	Norte	3200	3
11	Tintas Rio Branco	Norte	1500	3
12	Nordestemido Hall	Nordeste	1910	2
13	Cachoerinha Loft	Nordeste	2380	2

Cálculo de participação % (Parte 1)

Sabendo como fazer agregamentos totais (OVER apenas) e agregamentos particionados (OVER + PARTITION BY), poderemos facilmente agora fazer cálculos de participação %.

Observe no exemplo ao lado. Se quisermos descobrir qual é a participação de cada loja em termos de total vendido, basta dividirmos a coluna Qtd_Vendida pelo bloco SUM(Qtd_Vendida) OVER().

Obs: utilizamos a função FORMAT apenas para formatar os resultados como percentual. Não haveria necessidade de incluí-lo.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER() AS 'Total Vendido',
    FORMAT(Qtd_Vendida/SUM(Qtd_Vendida) OVER(), '0.00%') AS '% Por Região'
FROM Lojas
ORDER BY ID_Loja
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido	% Por Região
1	Botafogo Praia&Mar	Sudeste	1800	26480	6,80%
2	Lojas Vitoria	Sudeste	800	26480	3,02%
3	Emporio Mineirinho	Sudeste	2300	26480	8,69%
4	Central Paulista	Sudeste	1800	26480	6,80%
5	Rio 90 graus	Sudeste	700	26480	2,64%
6	Casa Flor & Anópolis	Sul	2100	26480	7,93%
7	Pampas & Co	Sul	990	26480	3,74%
8	Paraná Papéis	Sul	2800	26480	10,57%
9	Amazonas Prime	Norte	4200	26480	15,86%
10	Pará Bens	Norte	3200	26480	12,08%
11	Tintas Rio Branco	Norte	1500	26480	5,66%
12	Nordestemido Hall	Nordeste	1910	26480	7,21%
13	Cachoerinha Loft	Nordeste	2380	26480	8,99%

Cálculo de participação % (Parte 2)

Fazendo uma alteração simples e incluindo o PARTITION BY, podemos calcular o % que cada loja tem de participação, considerando suas respectivas regiões.

Tome cuidado porque no exemplo ao lado, a soma de % não é 100% porque o cálculo está sendo considerando cada região.

Ou seja, a loja Botafogo Praia&Mar tem uma quantidade vendida que representa 6,8% do valor total, representa 24,32% das vendas da região Sudeste.

```
SELECT
    ID_Loja,
    Nome_Loja,
    Regiao,
    Qtd_Vendida,
    SUM(Qtd_Vendida) OVER(PARTITION BY Regiao) AS 'Total Vendido',
    FORMAT(Qtd_Vendida/SUM(Qtd_Vendida) OVER(PARTITION BY Regiao), '0.00%') AS '% Por Região'
FROM Lojas
ORDER BY ID_Loja
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	Total Vendido	% Por Região
1	Botafogo Praia&Mar	Sudeste	1800	7400	24,32%
2	Lojas Vitoria	Sudeste	800	7400	10,81%
3	Emporio Mineirinho	Sudeste	2300	7400	31,08%
4	Central Paulista	Sudeste	1800	7400	24,32%
5	Rio 90 graus	Sudeste	700	7400	9,46%
6	Casa Flor & Anópolis	Sul	2100	5890	35,65%
7	Pampas & Co	Sul	990	5890	16,81%
8	Paraná Papéis	Sul	2800	5890	47,54%
9	Amazonas Prime	Norte	4200	8900	47,19%
10	Pará Bens	Norte	3200	8900	35,96%
11	Tintas Rio Branco	Norte	1500	8900	16,85%
12	Nordestemido Hall	Nordeste	1910	4290	44,52%
13	Cachoerinha Loft	Nordeste	2380	4290	55,48%

Funções de Classificação

1

ROW_NUMBER

Cria uma coluna com a numeração das linhas da tabela. É como se fosse uma coluna de ID.

2

RANK

Cria uma coluna de ranking. Em caso de empates, os rankings dos números se repetem, mas o próximo ranking pula a sequência. Ex: dois valores na posição 8, o valor seguinte pula para a posição 10.

3

DENSE_RANK

Cria uma coluna de ranking. Em caso de empates, os rankings dos números se repetem, e o próximo ranking continua a sequência. Ex: dois valores na posição 8, o valor seguinte continua na posição 9.

4

NTILE

Distribui os valores de uma coluna em grupos.

Funções de Classificação – ROW_NUMBER, RANK, DENSE_RANK e NTILE

Para fins de entendimento e comparação, todas as funções de classificação são mostradas ao lado.

Observe que dentro da instrução OVER é utilizado o ORDER BY. O que faz sentido, pois de certa forma as funções de classificação têm como objetivo ordenar os valores.

Porém, com as funções de classificação, podemos potencializar a aplicação do ORDER BY (que aqui também pode ser usado com as opções ASC e DESC, fazendo a ordenação crescente e decrescente, respectivamente).

```
SELECT
    *,
    ROW_NUMBER() OVER(ORDER BY Qtd_Vendida DESC) AS 'rownumber',
    RANK() OVER(ORDER BY Qtd_Vendida DESC) AS 'rank',
    DENSE_RANK() OVER(ORDER BY Qtd_Vendida DESC) AS 'dense',
    NTILE(3) OVER(ORDER BY Qtd_Vendida DESC) AS 'ntile'
FROM
    Lojas
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	rownumber	rank	dense	ntile
9	Amazonas Prime	Norte	4200	1	1	1	1
10	Pará Bens	Norte	3200	2	2	2	1
8	Paraná Papéis	Sul	2800	3	3	3	1
13	Cachoerinha Loft	Nordeste	2380	4	4	4	1
3	Emporio Mineirinho	Sudeste	2300	5	5	5	1
6	Casa Flor & Anópolis	Sul	2100	6	6	6	2
12	Nordestemido Hall	Nordeste	1910	7	7	7	2
4	Central Paulista	Sudeste	1800	8	8	8	2
1	Botafogo Praia&Mar	Sudeste	1800	9	8	8	2
11	Tintas Rio Branco	Norte	1500	10	10	9	3
7	Pampas & Co	Sul	990	11	11	10	3
2	Lojas Vitoria	Sudeste	800	12	12	11	3
5	Rio 90 graus	Sudeste	700	13	13	12	3

Funções de Classificação com PARTITION BY

As funções de classificação também podem ser utilizadas dentro de repartições, ou seja, grupos internos a nossa tabela, por meio do PARTITION BY.

No exemplo ao lado, as funções de classificação são aplicadas particionando por regiões. Ou seja, é como se o PARTITION BY deixasse claro para o SQL que a tabela deve ser entendida como diversas tabelas separadas, fazendo essa divisão por região.

```
SELECT
    *,
    ROW_NUMBER() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'rownumber',
    RANK() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'rank',
    DENSE_RANK() OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'dense',
    NTILE(3) OVER(PARTITION BY Regiao ORDER BY Qtd_Vendida DESC) AS 'ntile'
FROM
    Lojas
```

Resultados | Mensagens

ID_Loja	Nome_Loja	Regiao	Qtd_Vendida	rownumber	rank	dense	ntile
13	Cachoerinha Loft	Nordeste	2380	1	1	1	1
12	Nordestemido Hall	Nordeste	1910	2	2	2	2
9	Amazonas Prime	Norte	4200	1	1	1	1
10	Pará Bens	Norte	3200	2	2	2	2
11	Tintas Rio Branco	Norte	1500	3	3	3	3
3	Emporio Mineirinho	Sudeste	2300	1	1	1	1
4	Central Paulista	Sudeste	1800	2	2	2	1
1	Botafogo Praia&Mar	Sudeste	1800	3	2	2	2
2	Lojas Vitoria	Sudeste	800	4	4	3	2
5	Rio 90 graus	Sudeste	700	5	5	4	3
8	Paraná Papéis	Sul	2800	1	1	1	1
6	Casa Flor & Anópolis	Sul	2100	2	2	2	2
7	Pampas & Co	Sul	990	3	3	3	3

RANK com GROUP BY

Podemos também utilizar a função RANK para criar rankings em dados agrupados.

Ou seja, criamos agrupamentos com o GROUP BY e fazemos um ranking considerando estes dados agrupados.

No exemplo ao lado, criamos um agrupamento por Região, e adicionamos a essa tabela agrupada uma coluna de ranking.

```
SELECT
    Regiao AS 'Região',
    SUM(Qtd_Vendida) AS 'Total Vendido',
    RANK() OVER(ORDER BY SUM(Qtd_Vendida) DESC) AS 'Rank'
FROM
    Lojas
GROUP BY Regiao
ORDER BY Rank
```

Resultados | Mensagens

Região	Total Vendido	Rank
Norte	8900	1
Sudeste	7400	2
Sul	5890	3
Nordeste	4290	4

Código para criar a tabela Resultado

Para os próximos exercícios, será importante ter esta tabela. Basta copiar o código e reproduzir no seu SSMS.

```
CREATE TABLE Resultado(
Data_Fechamento DATETIME,
Faturamento_MM FLOAT)


INSERT INTO Resultado(Data_Fechamento, Faturamento_MM)
VALUES
    ('01/01/2020', 8),
    ('01/02/2020', 10),
    ('01/03/2020', 6),
    ('01/04/2020', 9),
    ('01/05/2020', 5),
    ('01/06/2020', 4),
    ('01/07/2020', 7),
    ('01/08/2020', 11),
    ('01/09/2020', 9),
    ('01/10/2020', 12),
    ('01/11/2020', 11),
    ('01/12/2020', 10)
```

Cálculo de soma móvel

Uma outra análise que podemos fazer é a de soma móvel. A forma mais fácil de entender o funcionamento é observar o exemplo ao lado.

Na tabela, temos uma coluna com o faturamento de cada mês e outra coluna com a soma móvel.

129 %

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Soma móvel (em milhões)
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	16
4	2020-04-01 00:00:00.000	9	15
5	2020-05-01 00:00:00.000	5	14
6	2020-06-01 00:00:00.000	4	9
7	2020-07-01 00:00:00.000	7	11
8	2020-08-01 00:00:00.000	11	18
9	2020-09-01 00:00:00.000	9	20
10	2020-10-01 00:00:00.000	12	21
11	2020-11-01 00:00:00.000	11	23
12	2020-12-01 00:00:00.000	10	21

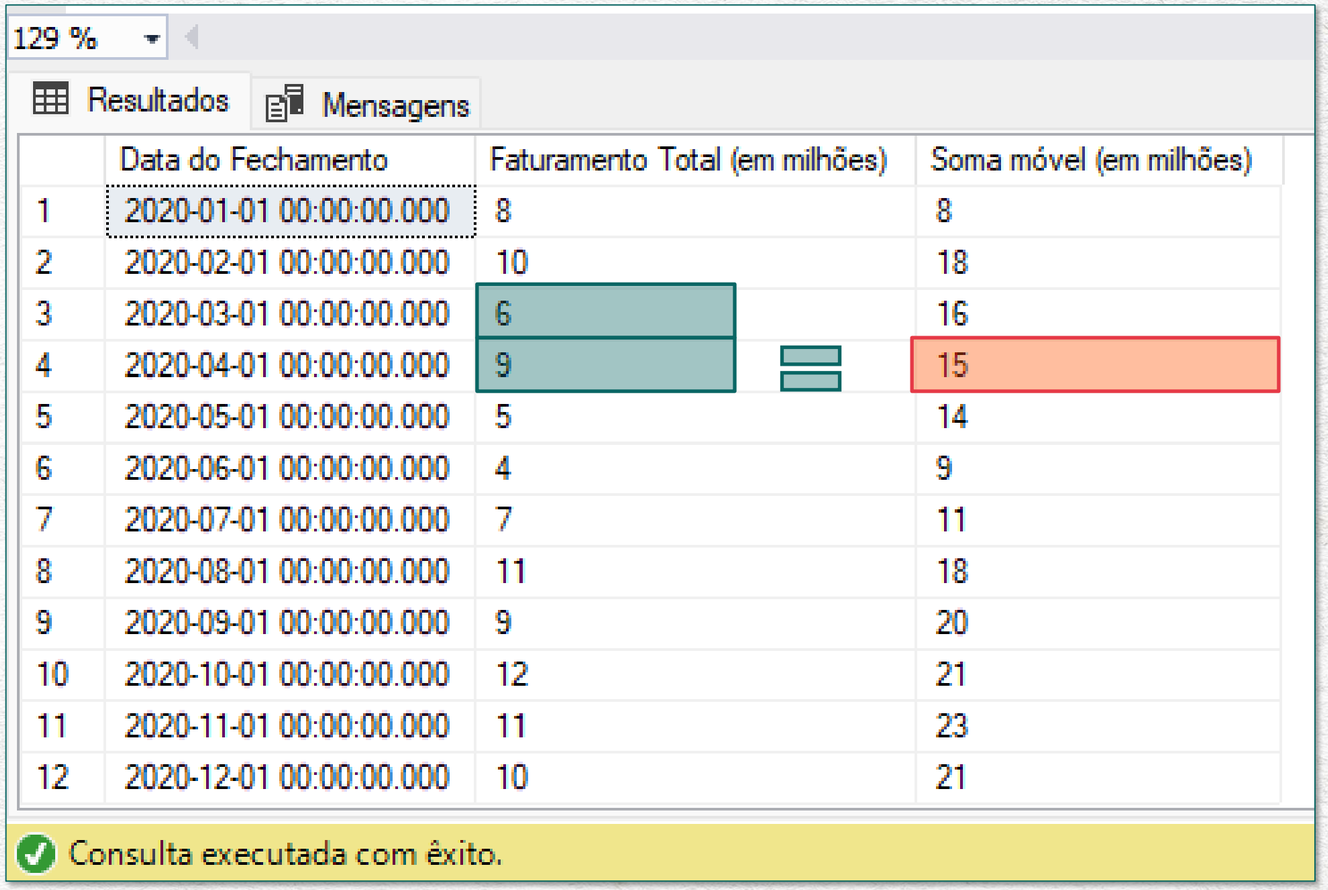
Consulta executada com êxito.

Cálculo de soma móvel

Se você observar bem, o **valor 15** marcado na imagem ao lado se trata do **valor 9** (Faturamento Total da linha 4) **somado** com o **valor 6** (Faturamento Total da linha 3).

Na verdade, qualquer valor da coluna Soma móvel é a soma do valor do Faturamento Total na sua linha mais o Faturamento Total na linha anterior. Isso é o que chamamos de soma móvel.

Vamos ver como é feito este cálculo usando comandos em SQL.



129 %

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Soma móvel (em milhões)
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	16
4	2020-04-01 00:00:00.000	9	15
5	2020-05-01 00:00:00.000	5	14
6	2020-06-01 00:00:00.000	4	9
7	2020-07-01 00:00:00.000	7	11
8	2020-08-01 00:00:00.000	11	18
9	2020-09-01 00:00:00.000	9	20
10	2020-10-01 00:00:00.000	12	21
11	2020-11-01 00:00:00.000	11	23
12	2020-12-01 00:00:00.000	10	21

Consulta executada com êxito.

Cálculo de soma móvel

Para fazer essa soma móvel, precisamos usar uma função de janela. Usamos a função SUM junto com a cláusula OVER para somar os valores de um determinado intervalo, como mostrado na figura ao lado.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
      'Soma móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

129 %

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Soma móvel (em milhões)
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	16
4	2020-04-01 00:00:00.000	9	15
5	2020-05-01 00:00:00.000	5	14
6	2020-06-01 00:00:00.000	4	9
7	2020-07-01 00:00:00.000	7	11
8	2020-08-01 00:00:00.000	11	18
9	2020-09-01 00:00:00.000	9	20
10	2020-10-01 00:00:00.000	12	21
11	2020-11-01 00:00:00.000	11	23
12	2020-12-01 00:00:00.000	10	21

Consulta executada com êxito.

MARCUS\caval (56) | Exercicios | 00:00:00 | 12 linhas

Cálculo de soma móvel


```
SELECT
     Data_Fechamento AS 'Data do Fechamento',
     Faturamento_MM AS 'Faturamento Total (em milhões)',
     SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
       'Soma móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Este intervalo de soma pode ser especificado pelo trecho marcado na imagem acima. Observe a parte:

ROWS BETWEEN 1 PRECEDING AND CURRENT ROW

Podemos traduzir esta parte literalmente por:

LINHAS ENTRE 1 ANTES E A LINHA ATUAL

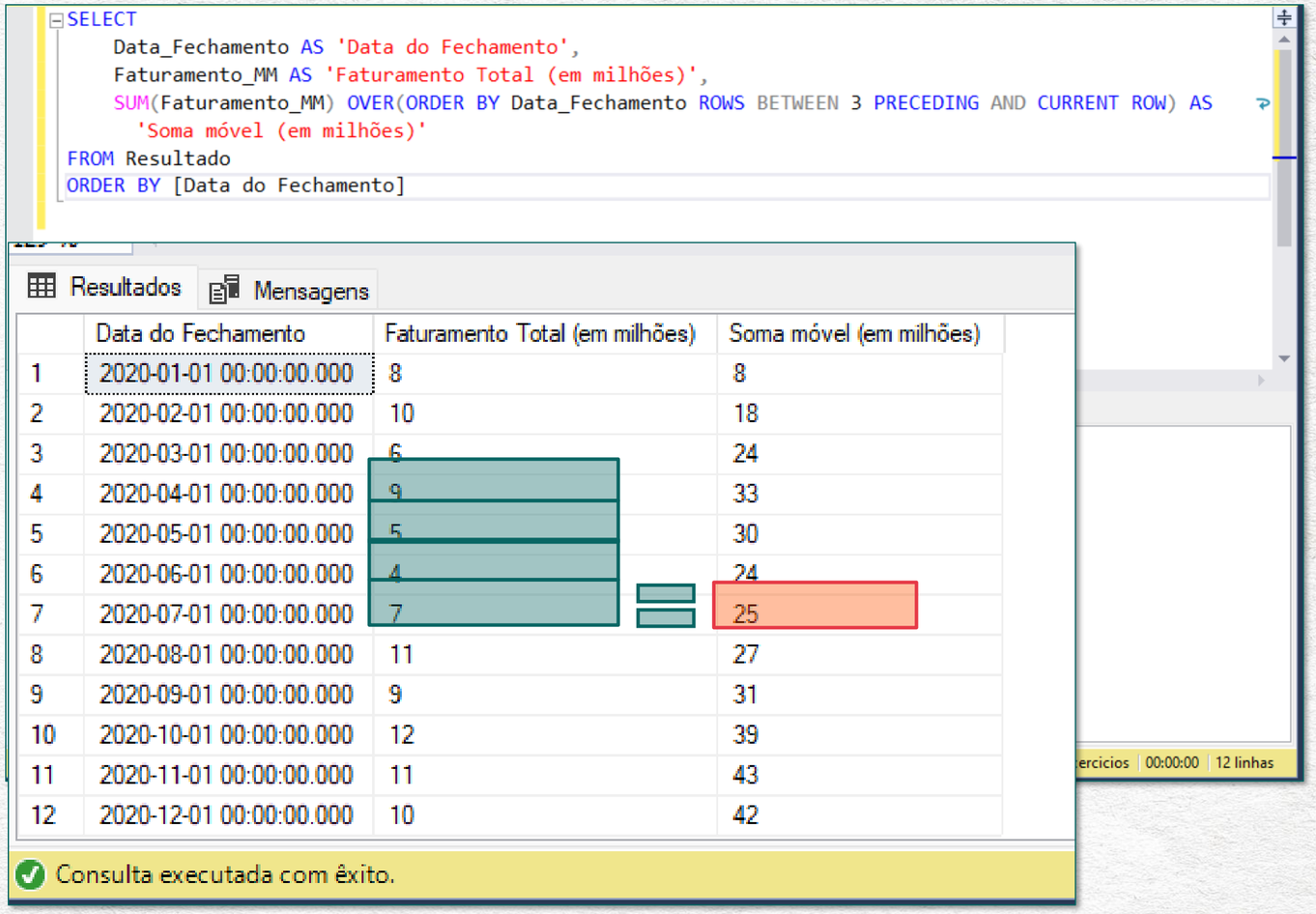
Como queremos fazer uma soma, isto significa que: a nossa janela de soma será entre as linhas anterior e atual. Ou seja, essa soma será feita considerando sempre os valores da linha atual + a linha anterior.

Cálculo de soma móvel

Podemos testar outros resultados personalizando a janela de soma. Dessa vez, se colocarmos

ROWS BETWEEN 3 PRECEDING AND CURRENT ROW

Ele vai fazer a soma da linha atual com as 3 linhas anteriores, conforme imagem ao lado.



```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 3 PRECEDING AND CURRENT ROW) AS
      'Soma móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Soma móvel (em milhões)
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	24
4	2020-04-01 00:00:00.000	9	33
5	2020-05-01 00:00:00.000	5	30
6	2020-06-01 00:00:00.000	4	24
7	2020-07-01 00:00:00.000	7	25
8	2020-08-01 00:00:00.000	11	27
9	2020-09-01 00:00:00.000	9	31
10	2020-10-01 00:00:00.000	12	39
11	2020-11-01 00:00:00.000	11	43
12	2020-12-01 00:00:00.000	10	42

Consulta executada com êxito.

ercicios | 00:00:00 | 12 linhas

Cálculo de soma móvel

Outra possibilidade é personalizar a janela para que ela considere valores antes da linha e depois da linha em questão. Por exemplo, na imagem ao lado, cada valor da coluna Soma móvel é a soma do Faturamento Total da linha anterior + Faturamento Total da linha atual + Faturamento Total da linha seguinte. Para conseguir estender a nossa janela para a linha seguinte, usamos o comando FOLLOWING.

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

Em resumo, podemos usar o PRECEDING ou o FOLLOWING para personalizar o “tamanho” da nossa janela de cálculo.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS
      'Soma móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Soma móvel (em milhões)
1	2020-01-01 00:00:00.000	8	18
2	2020-02-01 00:00:00.000	10	24
3	2020-03-01 00:00:00.000	6	25
4	2020-04-01 00:00:00.000	9	20
5	2020-05-01 00:00:00.000	5	18
6	2020-06-01 00:00:00.000	4	16
7	2020-07-01 00:00:00.000	7	22
8	2020-08-01 00:00:00.000	11	27
9	2020-09-01 00:00:00.000	9	32
10	2020-10-01 00:00:00.000	12	32
11	2020-11-01 00:00:00.000	11	33
12	2020-12-01 00:00:00.000	10	21

Consulta executada com êxito.

6) | Exercicios | 00:00:00 | 12 linhas

Cálculo de média móvel

Para fazer uma média móvel, seguimos exatamente a mesma lógica explicada anteriormente. A única diferença é que agora usamos o AVG no lugar do SUM.

Na verdade, podemos fazer qualquer análise móvel apenas trocando a função (COUNT, MIN, MAX, SUM, AVG).

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    AVG(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
      'Média Móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Média Móvel (em milhões)
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	9
3	2020-03-01 00:00:00.000	6	8
4	2020-04-01 00:00:00.000	9	7,5
5	2020-05-01 00:00:00.000	5	7
6	2020-06-01 00:00:00.000	4	4,5
7	2020-07-01 00:00:00.000	7	5,5
8	2020-08-01 00:00:00.000	11	9
9	2020-09-01 00:00:00.000	9	10
10	2020-10-01 00:00:00.000	12	10,5
11	2020-11-01 00:00:00.000	11	11,5
12	2020-12-01 00:00:00.000	10	10,5

Consulta executada com êxito.

ercicios | 00:00:00 | 12 linhas

Cálculo de acumulado

Para fazer um cálculo de acumulado, precisamos fazer com que a nossa janela de análise comece sempre na data mais antiga e vá até a linha atual.

Observe a imagem ao lado. O valor de Fat. Acumulado na linha onde o valor é 42 é a soma de Faturamento Total desde a primeira linha da tabela até aquela linha (8+10+6+9+5+4=42).

Isso significa que a janela de soma deve ser entre a linha mais anterior até a linha atual. Informar a linha atual nós sabemos: será por meio da CURRENT ROW.

Já para informar a linha mais anterior, usamos o UNBOUNDED PRECEDING.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
      ROW) AS 'Fat. Acumulado (em milhões'
FROM Resultado
ORDER BY [Data do Fechamento]
```

129 %

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Fat. Acumulado (em milhões
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	24
4	2020-04-01 00:00:00.000	9	33
5	2020-05-01 00:00:00.000	5	38
6	2020-06-01 00:00:00.000	4	42
7	2020-07-01 00:00:00.000	7	49
8	2020-08-01 00:00:00.000	11	60
9	2020-09-01 00:00:00.000	9	69
10	2020-10-01 00:00:00.000	12	81
11	2020-11-01 00:00:00.000	11	92
12	2020-12-01 00:00:00.000	10	102

Consulta executada com êxito.

(56) | Exercicios | 00:00:00 | 12 linhas

Cálculo de acumulado

Ou seja, sempre que precisarmos fazer um cálculo com uma janela que começa na primeira linha da tabela, o nosso intervalo começará com:

UNBOUNDED PRECEDING

De forma muito semelhante, se quisermos uma janela que vá até a última linha, usamos o UNBOUNDED FOLLOWING.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
      ROW) AS 'Fat. Acumulado (em milhões'
FROM Resultado
ORDER BY [Data do Fechamento]
```

129 %

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Fat. Acumulado (em milhões
1	2020-01-01 00:00:00.000	8	8
2	2020-02-01 00:00:00.000	10	18
3	2020-03-01 00:00:00.000	6	24
4	2020-04-01 00:00:00.000	9	33
5	2020-05-01 00:00:00.000	5	38
6	2020-06-01 00:00:00.000	4	42
7	2020-07-01 00:00:00.000	7	49
8	2020-08-01 00:00:00.000	11	60
9	2020-09-01 00:00:00.000	9	69
10	2020-10-01 00:00:00.000	12	81
11	2020-11-01 00:00:00.000	11	92
12	2020-12-01 00:00:00.000	10	102

Consulta executada com êxito.

(56) | Exercicios | 00:00:00 | 12 linhas

Resumo das janelas

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    SUM(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) AS
      'Soma móvel (em milhões)'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Você pode entender também a parte destacada acima como uma especificação do intervalo da nossa janela de cálculo. Ela terá a seguinte estrutura:

ROWS BETWEEN *inicio_do_intervalo* AND *fim_do_intervalo*

Exemplos desses intervalos podem ser:

- **ROWS BETWEEN *1 PRECEDING* AND *CURRENT ROW***: 1 linha antes e a linha atual.
- **ROWS BETWEEN *UNBOUNDED PRECEDING* AND *CURRENT ROW***: todas as linhas antes e a linha atual.
- **ROWS BETWEEN *1 PRECEDING* AND *1 FOLLOWING***: 1 linha antes e 1 linha depois da atual.
- **ROWS BETWEEN *UNBOUNDED PRECEDING* AND *UNBOUNDED FOLLOWING***: todas as linhas antes e todas as linhas depois da linha atual.

Funções de Offset

1

LAG

Faz um deslocamento nos valores, retornando um valor que está em uma posição anterior (1 linha anterior, 2 linhas anteriores, etc).

2

LEAD

Faz um deslocamento nos valores, retornando o valor que está em uma posição posterior (1 linha seguinte, 2 linhas seguintes, etc).

3

FIRST_VALUE

Cria uma coluna de ranking. Em caso de empates, os rankings dos números se repetem, e o próximo ranking continua a sequência. Ex: dois valores na posição 8, o valor seguinte continua na posição 9.

4

LAST_VALUE

Distribui os valores de uma coluna em grupos.

Função LAG

O funcionamento da função LAG é bem intuitivo. Ele permite acessar valores anteriores.

No exemplo ao lado, cada valor da coluna Valor anterior se trata do valor da linha anterior, na coluna Faturamento Total.

```
LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Valor anterior'
```

A função LAG pede 3 argumentos:

1) Qual é a coluna com os valores que queremos deslocar.
2) Quantas linhas pra trás queremos deslocar.
3) Quando não for encontrado um valor anterior, qual é o valor que deve aparecer.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Valor anterior'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Valor anterior
1	2020-01-01 00:00:00.000	8	0
2	2020-02-01 00:00:00.000	10	8
3	2020-03-01 00:00:00.000	6	10
4	2020-04-01 00:00:00.000	9	6
5	2020-05-01 00:00:00.000	5	9
6	2020-06-01 00:00:00.000	4	5
7	2020-07-01 00:00:00.000	7	4
8	2020-08-01 00:00:00.000	11	7
9	2020-09-01 00:00:00.000	9	11
10	2020-10-01 00:00:00.000	12	9
11	2020-11-01 00:00:00.000	11	12
12	2020-12-01 00:00:00.000	10	11

Consulta executada com êxito.

RTM) | MARCUS\caval (57) | Exercicios | 00:00:00 | 12 linhas

Função LAG

No exemplo ao lado, temos os seguintes argumentos para a função:

1º argumento) Expressão: Faturamento_MM - coluna que queremos acessar valores anteriores.

2º argumento) Offset: 1 - quantas linhas queremos deslocar para trás.

3º argumento) Default: 0 - valor padrão que queremos que retorne caso ele não encontre nenhum valor (na linha 1 da tabela ao lado, retornou zero porque não tem nenhum outro valor de Faturamento antes da linha 1).

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Valor anterior'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Valor anterior
1	2020-01-01 00:00:00.000	8	0
2	2020-02-01 00:00:00.000	10	8
3	2020-03-01 00:00:00.000	6	10
4	2020-04-01 00:00:00.000	9	6
5	2020-05-01 00:00:00.000	5	9
6	2020-06-01 00:00:00.000	4	5
7	2020-07-01 00:00:00.000	7	4
8	2020-08-01 00:00:00.000	11	7
9	2020-09-01 00:00:00.000	9	11
10	2020-10-01 00:00:00.000	12	9
11	2020-11-01 00:00:00.000	11	12
12	2020-12-01 00:00:00.000	10	11

RTM) | MARCUS\caval (57) | Exercicios | 00:00:00 | 12 linhas

Consulta executada com êxito.

Função LAG

Vejamos um outro exemplo:

1º argumento) Expressão: Faturamento_MM - coluna que queremos acessar valores anteriores.

2º argumento) Offset: 2 - quantas linhas queremos deslocar para trás.

3º argumento) Default: não informado.

Repare agora que a coluna Valor anterior está "pegando" o valor 2 casas para trás na coluna Faturamento Total. E como não informamos o default, os dois primeiros valores ficaram NULL (porque deslocando o Faturamento Total duas casas para trás, não tem nenhum valor disponível).

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 2) OVER(ORDER BY Data_Fechamento) AS 'Valor anterior'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Valor anterior
1	2020-01-01 00:00:00.000	8	NULL
2	2020-02-01 00:00:00.000	10	NULL
3	2020-03-01 00:00:00.000	6	8
4	2020-04-01 00:00:00.000	9	10
5	2020-05-01 00:00:00.000	5	6
6	2020-06-01 00:00:00.000	4	9
7	2020-07-01 00:00:00.000	7	5
8	2020-08-01 00:00:00.000	11	4
9	2020-09-01 00:00:00.000	9	7
10	2020-10-01 00:00:00.000	12	11
11	2020-11-01 00:00:00.000	11	9
12	2020-12-01 00:00:00.000	10	12

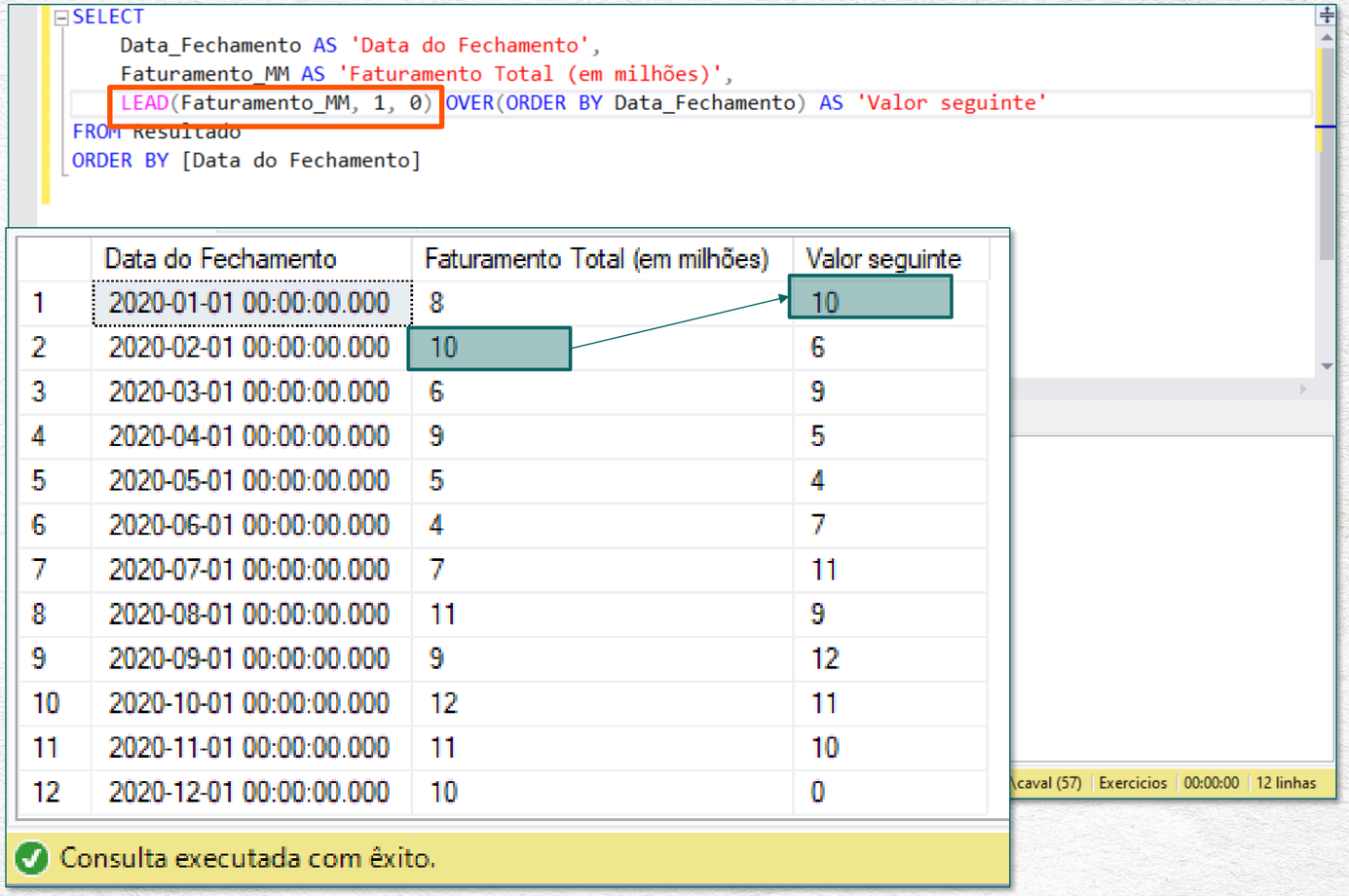
Consulta executada com êxito.

S\caval (57) | Exercicios | 00:00:00 | 12 linhas

Função LEAD

A função LEAD fará o oposto. Agora, o deslocamento será feito para frente. Então observe na imagem ao lado que ao usar o LEAD com o 2º argumento igual a 1, acessamos o valor de Faturamento Total da linha seguinte.

Ou seja, na linha 1, o valor 10 refere-se ao Faturamento Total da linha 2.



```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LEAD(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Valor seguinte'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Valor seguinte
1	2020-01-01 00:00:00.000	8	10
2	2020-02-01 00:00:00.000	10	6
3	2020-03-01 00:00:00.000	6	9
4	2020-04-01 00:00:00.000	9	5
5	2020-05-01 00:00:00.000	5	4
6	2020-06-01 00:00:00.000	4	7
7	2020-07-01 00:00:00.000	7	11
8	2020-08-01 00:00:00.000	11	9
9	2020-09-01 00:00:00.000	9	12
10	2020-10-01 00:00:00.000	12	11
11	2020-11-01 00:00:00.000	11	10
12	2020-12-01 00:00:00.000	10	0

Consulta executada com êxito.

caval (57) | Exercicios | 00:00:00 | 12 linhas

Função NULLIF

Vamos agora entender o funcionamento de uma função bem simples, mas muito útil: a função NULLIF.

Essa função recebe dois argumentos. Quando o valor dos dois argumentos é igual, ela retorna NULL. Quando o valor dos dois argumentos é diferente, ele retorna o valor do primeiro argumento.

Observe no exemplo ao lado. NULLIF(3, 3) retorna como resultado o valor NULL. Já o NULLIF(3, 5) retorna como resultado o valor 3, que é o valor do primeiro argumento.

Essa função tem uma funcionalidade muito simples, que será muito útil na nossa aplicação de cálculo do MoM a seguir.

```
SELECT
    NULLIF(3, 3) AS 'Igual',
    NULLIF(3, 5) AS 'Diferente'
```

29 %

Resultados | Mensagens

	Igual	Diferente
1	NULL	3

Cálculo MoM

O cálculo MoM (Month Over Month) nada mais é do que um cálculo de variação percentual de crescimento/decrescimento de um valor em relação ao mês anterior.

Por exemplo: **qual foi a variação percentual do Faturamento de fevereiro em relação ao mês anterior?** A fórmula é a seguinte:

Variação % MoM = (Faturamento Mês Atual/Faturamento Mês Anterior) – 1

O resultado para fevereiro portanto seria:

Variação % = (10/8) – 1 = **24,25%**

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Faturamento Mês Anterior'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Faturamento Mês Anterior
1	2020-01-01 00:00:00.000	8	0
2	2020-02-01 00:00:00.000	10	8
3	2020-03-01 00:00:00.000	6	10
4	2020-04-01 00:00:00.000	9	6
5	2020-05-01 00:00:00.000	5	9
6	2020-06-01 00:00:00.000	4	5
7	2020-07-01 00:00:00.000	7	4
8	2020-08-01 00:00:00.000	11	7
9	2020-09-01 00:00:00.000	9	11
10	2020-10-01 00:00:00.000	12	9
11	2020-11-01 00:00:00.000	11	12
12	2020-12-01 00:00:00.000	10	11

Consulta executada com êxito.

Cálculo MoM

Para fazer esse cálculo de variação % precisaremos trabalhar com funções de janela, mais especificamente a função de offset **LAG** pois precisamos fazer o Faturamento Total dividido pelo Faturamento do Mês Anterior.

Na imagem ao lado é possível ver o Faturamento Total e o Faturamento Mês Anterior lado a lado, graças à aplicação da função de janela LAG. Agora, fica fácil fazer um dividido pelo outro.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Faturamento Mês Anterior'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Faturamento Mês Anterior
1	2020-01-01 00:00:00.000	8	0
2	2020-02-01 00:00:00.000	10	8
3	2020-03-01 00:00:00.000	6	10
4	2020-04-01 00:00:00.000	9	6
5	2020-05-01 00:00:00.000	5	9
6	2020-06-01 00:00:00.000	4	5
7	2020-07-01 00:00:00.000	7	4
8	2020-08-01 00:00:00.000	11	7
9	2020-09-01 00:00:00.000	9	11
10	2020-10-01 00:00:00.000	12	9
11	2020-11-01 00:00:00.000	11	12
12	2020-12-01 00:00:00.000	10	11

Consulta executada com êxito.

Cálculo MoM

Ao fazer o cálculo de Faturamento_MM/Faturamento Mês Anterior, no entanto, temos um erro no resultado: Erro de divisão por zero.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Faturamento Mês Anterior',
    Faturamento_MM/LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) - 1 AS 'Crescimento MoM'
FROM Resultado
ORDER BY [Data do Fechamento]
```

```
Mensagem 8134, Nível 16, Estado 1, Linha 1
Erro de divisão por zero.

Horário de conclusão: 2022-05-25T00:08:47.8174571-03:00
```

Isso aconteceu porque especificamente porque para a primeira linha, o Faturamento Mês Anterior é zero, pois não temos nenhum valor antes de 2020-01-01. Por isso, ao fazer (8/0) – 1 temos uma divisão por zero neste caso.

	Data do Fechamento	Faturamento Total (em milhões)	Faturamento Mês Anterior
1	2020-01-01 00:00:00.000	8	0

Precisaremos portanto tratar essa divisão por zero. É ai que entra a função NULLIF.

Cálculo MoM

Usaremos a função NULLIF junto à função LAG, dado que ela retorna o valor zero na primeira linha da tabela, uma vez que não tem nenhum valor de Faturamento antes da primeira linha.

Lembrando que a função NULLIF retorna NULL se o resultado dos dois argumentos é igual, e retorna o resultado do primeiro argumento quando os dois valores são diferentes, ela será exatamente o que a gente precisa.

No primeiro mês, quando o Faturamento Mês Anterior é igual a zero, quanto tempos NULLIF(0, 0) o resultado é NULL. Assim, o cálculo de variação fica 8/NULL – 1. Qualquer valor dividido por NULL retorna NULL, então não teremos erro.

Agora quando o Faturamento Mês Anterior for diferente de zero, teremos NULLIF(Faturamento Mês Anterior, 0) e o resultado será Faturamento Mês Anterior.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Faturamento Mês Anterior',
    Faturamento_MM/NULLIF(LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento), 0) - 1 AS
      'Crescimento MoM'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Faturamento Mês Anterior	Crescimento MoM
1	2020-01-01 00:00:00.000	8	0	NULL
2	2020-02-01 00:00:00.000	10	8	0,25
3	2020-03-01 00:00:00.000	6	10	-0,4
4	2020-04-01 00:00:00.000	9	6	0,5
5	2020-05-01 00:00:00.000	5	9	-0,444444444444444
6	2020-06-01 00:00:00.000	4	5	-0,2
7	2020-07-01 00:00:00.000	7	4	0,75
8	2020-08-01 00:00:00.000	11	7	0,571428571428571
9	2020-09-01 00:00:00.000	9	11	-0,181818181818182
10	2020-10-01 00:00:00.000	12	9	0,333333333333333
11	2020-11-01 00:00:00.000	11	12	-0,0833333333333334
12	2020-12-01 00:00:00.000	10	11	-0,090909090909091

Consulta executada com êxito.

Cálculo MoM

Para completar, podemos usar a função FORMAT para deixar o valor do Crescimento MoM no formato percentual.

Um detalhe importante é que alguns dos valores do Crescimento MoM é negativo. Isso porque em alguns meses, o Faturamento reduziu em comparação com o mês anterior, por isso a variação é negativa.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento) AS 'Faturamento Mês Anterior',
    FORMAT(Faturamento_MM/NULLIF(LAG(Faturamento_MM, 1, 0) OVER(ORDER BY Data_Fechamento), 0) - 1,
      '0.00%') AS 'Crescimento MoM'
FROM Resultado
ORDER BY [Data do Fechamento]
```

Resultados | Mensagens

	Data do Fechamento	Faturamento Total (em milhões)	Faturamento Mês Anterior	Crescimento MoM
1	2020-01-01 00:00:00.000	8	0	NULL
2	2020-02-01 00:00:00.000	10	8	25,00%
3	2020-03-01 00:00:00.000	6	10	-40,00%
4	2020-04-01 00:00:00.000	9	6	50,00%
5	2020-05-01 00:00:00.000	5	9	-44,44%
6	2020-06-01 00:00:00.000	4	5	-20,00%
7	2020-07-01 00:00:00.000	7	4	75,00%
8	2020-08-01 00:00:00.000	11	7	57,14%
9	2020-09-01 00:00:00.000	9	11	-18,18%
10	2020-10-01 00:00:00.000	12	9	33,33%
11	2020-11-01 00:00:00.000	11	12	-8,33%
12	2020-12-01 00:00:00.000	10	11	-9,09%

12 linhas

Consulta executada com êxito.

FIRST_VALUE e LAST_VALUE

Fechamos com as funções FIRST_VALUE e LAST_VALUE. Estas funções permitem retornar o primeiro valor e o último valor de uma determinada coluna.

Ao lado, temos a aplicação dessas duas funções de janela. Para ambos, consideramos uma janela que vai do valor UNBOUNDED PRECEDING até o UNBOUNDED FOLLOWING, ou seja, todos os valores.

No exemplo ao lado, o FIRST_VALUE (primeiro valor) de todo o Faturamento_MM é o 8. Já o LAST_VALUE (último valor) de todo o Faturamento_MM é o 10.

```
SELECT
    Data_Fechamento AS 'Data do Fechamento',
    Faturamento_MM AS 'Faturamento Total (em milhões)',
    FIRST_VALUE(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED PRECEDING AND
      UNBOUNDED FOLLOWING) AS 'Primeiro valor',
    LAST_VALUE(Faturamento_MM) OVER(ORDER BY Data_Fechamento ROWS BETWEEN UNBOUNDED PRECEDING AND
      UNBOUNDED FOLLOWING) AS 'Último valor'
FROM Resultado
ORDER BY [Data do Fechamento]
```

	Data do Fechamento	Faturamento Total (em milhões)	Primeiro valor	Último valor
1	2020-01-01 00:00:00.000	8	8	10
2	2020-02-01 00:00:00.000	10	8	10
3	2020-03-01 00:00:00.000	6	8	10
4	2020-04-01 00:00:00.000	9	8	10
5	2020-05-01 00:00:00.000	5	8	10
6	2020-06-01 00:00:00.000	4	8	10
7	2020-07-01 00:00:00.000	7	8	10
8	2020-08-01 00:00:00.000	11	8	10
9	2020-09-01 00:00:00.000	9	8	10
10	2020-10-01 00:00:00.000	12	8	10
11	2020-11-01 00:00:00.000	11	8	10
12	2020-12-01 00:00:00.000	10	8	10

00:00:00 | 12 linhas

Consulta executada com êxito.

DESAFIO 1

Para resolver os exercícios 1 a 4, crie uma View chamada **vwProdutos**, que contenha o agrupamento das colunas BrandName, ColorName e os totais de quantidade vendida por marca/cor e também o total de receita por marca/cor.

EXERCÍCIOS

1 Questão 1

Utilize a View vwProdutos para criar uma coluna extra calculando a quantidade total vendida dos produtos.

2 Questão 2

Crie mais uma coluna na consulta anterior, incluindo o total de produtos vendidos para cada marca.

3 Questão 3

Calcule o % de participação do total de vendas de produtos por marca.

Ex: A marca A. Datum teve uma quantidade total de vendas de 199.041 de um total de 3.406.089 de vendas. Isso significa que a da marca A. Datum é 199.041/3.406.089 = 5,84%.

EXERCÍCIOS

4

Questão 4

Crie uma consulta à View **vwProdutos**, selecionando as colunas Marca, Cor, Quantidade_Vendida e também criando uma coluna extra de Rank para descobrir a posição de cada Marca/Cor. Você deve obter o resultado abaixo. Obs: Sua consulta deve ser filtrada para que seja mostrada apenas a marca Contoso.

Marca	Cor	Quantidade_Por_Produto	Rank
Contoso	Black	239810	1
Contoso	White	230199	2
Contoso	Silver	149081	3
Contoso	Grey	133244	4
Contoso	Blue	52819	5
Contoso	Red	50168	6
Contoso	Pink	21709	7
Contoso	Brown	21014	8
Contoso	Green	19617	9
Contoso	Orange	10105	10
Contoso	Yellow	9295	11
Contoso	Gold	7502	12
Contoso	Silver Grey	6419	13
Contoso	Purple	1752	14
Contoso	Transparent	1655	15

DESAFIO 2

Para responder os próximos 2 exercícios, você precisará criar uma View auxiliar.

A sua view deve se chamar vwHistoricoLojas e deve conter um histórico com a quantidade de lojas abertas a cada Ano/Mês (como mostrado na figura à direita). Os desafios são:

(1) Criar uma coluna de ID para essa View
(2) Relacionar as tabelas DimDate e DimStore

Dicas:

1- A coluna de ID será criada a partir de uma função de janela. Você deverá se atentar a forma como essa coluna deverá ser ordenada, pensando que queremos visualizar uma ordem de Ano/Mês que seja: 2005/january, 2005/February... e não 2005/April, 2005/August...

2- As colunas Ano, Mês e Qtd_Lojas correspondem, respectivamente, às seguintes colunas: CalendarYear e CalendarMonthLabel da tabela DimDate e uma contagem da coluna OpenDate da tabela DimStore.

ID	Ano	Mês	Qtd_Lojas
1	2005	January	0
2	2005	February	0
3	2005	March	0
4	2005	April	0
5	2005	May	2
6	2005	June	4
7	2005	July	1
8	2005	August	0
9	2005	September	0
10	2005	October	0
11	2005	November	1
12	2005	December	0
13	2006	January	2
14	2006	February	3
15	2006	March	0
16	2006	April	1
17	2006	May	0
18	2006	June	0
19	2006	July	1
20	2006	August	1
21	2006	September	0
22	2006	October	0

EXERCÍCIOS

5 Questão 5

A partir da view criada no **DESAFIO 2**, você deverá fazer uma soma móvel considerando sempre o mês atual + 2 meses para trás.

6 Questão 6

Utilize a **vwHistoricoLojas** para calcular o acumulado de lojas abertas a cada ano/mês.

DESAFIO 3

Neste desafio, você terá que criar suas próprias tabelas e views para conseguir resolver os exercícios 7 e 8. Os próximos exercícios envolverão análises de novos clientes. Para isso, será necessário criar uma nova tabela e uma nova view.

Abaixo, temos um passo a passo para resolver o problema por partes.

PASSO 1: Crie um novo banco de dados chamado Desafio e selecione esse banco de dados criado.

PASSO 2: Crie uma tabela de datas entre o dia 1 de janeiro do ano com a compra (DateFirstPurchase) mais antiga e o dia 31 de dezembro do ano com a compra mais recente.

Obs1: Chame essa tabela de Calendario.

Obs2: A princípio, essa tabela deve conter apenas 1 coluna, chamada data e do tipo DATE.

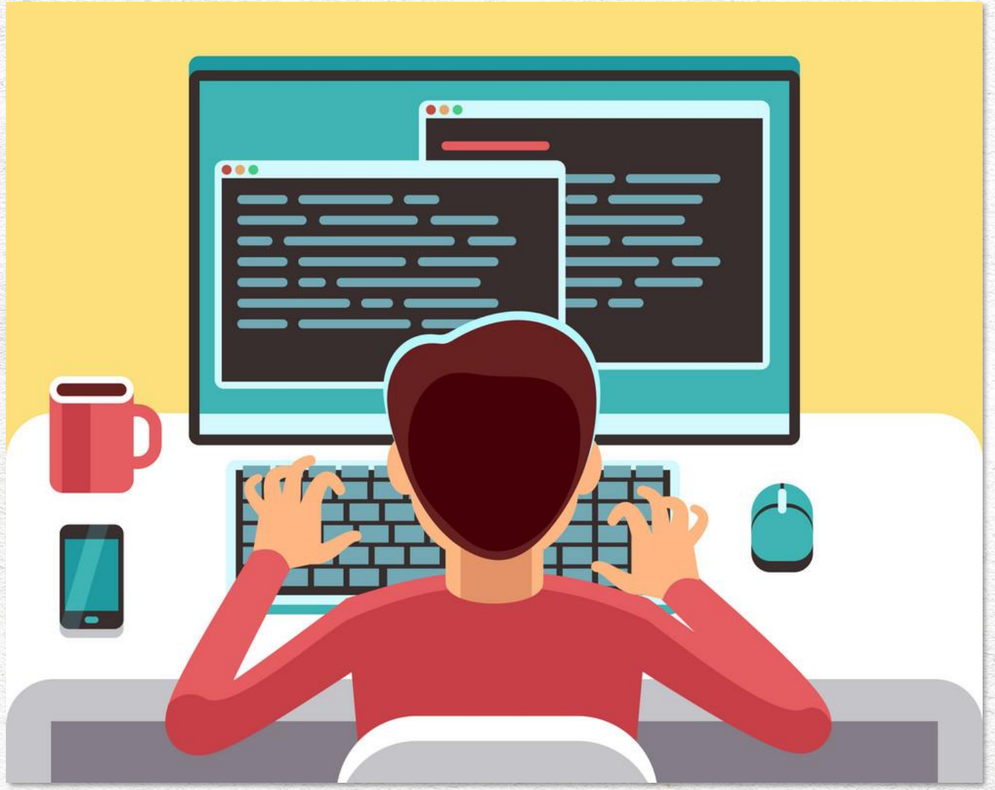
PASSO 3: Crie colunas auxiliares na tabela Calendario chamadas: Ano, Mes, Dia, AnoMes e NomeMes. Todas do tipo INT.

PASSO 4: Adicione na tabela os valores de Ano, Mês, Dia, AnoMes e NomeMes (nome do mês em português). Dica: utilize a instrução CASE para verificar o mês e retornar o nome certo.

PASSO 5: Crie a View vwNovosClientes, que deve ter as colunas mostradas abaixo.

ID	Ano	NomeMes	Novos_Clientes
1	2001	Janeiro	0
2	2001	Fevereiro	0
3	2001	Março	0
4	2001	Abril	0
5	2001	Maio	0
6	2001	Junho	0
7	2001	Julho	146
8	2001	Agosto	156
9	2001	Setembro	146
10	2001	Outubro	161
11	2001	Novembro	169
12	2001	Dezembro	235
13	2002	Janeiro	188
14	2002	Fevereiro	171
15	2002	Março	199
16	2002	Abril	207
17	2002	Maio	214
18	2002	Junho	214
19	2002	Julho	253
20	2002	Agosto	281
21	2002	Setembro	198
22	2002	Outubro	229
23	2002	Novembro	193
24	2002	Dezembro	330
25	2003	Janeiro	244
26	2003	Fevereiro	272
27	2003	Março	272
28	2003	Abril	294
29	2003	Maio	335
30	2003	Junho	321
31	2003	Julho	202
32	2003	Agosto	1210
33	2003	Setembro	1112

EXERCÍCIOS



7

Questão 7

Faça um cálculo de soma móvel de novos clientes nos últimos 2 meses.

b) Faça um cálculo de média móvel de novos clientes nos últimos 2 meses.

c) Faça um cálculo de acumulado dos novos clientes ao longo do tempo.

d) Faça um cálculo de acumulado intra-ano, ou seja, um acumulado que vai de janeiro a dezembro de cada ano, e volta a fazer o cálculo de acumulado no ano seguinte.

8

Questão 8

Faça os cálculos de MoM e YoY para avaliar o percentual de crescimento de novos clientes, entre o mês atual e o mês anterior, e entre um mês atual e o mesmo mês do ano anterior.

DESAFIO 1

Para resolver os exercícios 1 a 4, crie uma View chamada **vwProdutos**, que contenha o agrupamento das colunas BrandName, ColorName e os totais de quantidade vendida por marca/cor e também o total de receita por marca/cor.

SQLQuery1.sql - M...MARCUS\caval (53))*

```sql
CREATE VIEW vwProdutos AS
SELECT
    BrandName AS 'Marca',
    ColorName AS 'Cor',
    COUNT(*) AS 'Quantidade_Vendida',
    ROUND(SUM(SalesAmount), 2) AS 'Receita_Total'
FROM DimProduct
INNER JOIN FactSales
    ON DimProduct.ProductKey = FactSales.ProductKey
GROUP BY BrandName, ColorName
```

GABARITOS

1

Questão 1

Utilize a View vwProdutos para criar uma coluna extra calculando a quantidade total vendida dos produtos

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd Total Produtos'
FROM vwProdutos
ORDER BY Marca
```

145 %

Resultados | Mensagens

	Marca	Cor	Quantidade_Vendida	Receita_Total	Qtd Total Produtos
1	A. Datum	Blue	5601	28597778,95	3406089
2	A. Datum	Gold	4221	24740930,28	3406089
3	A. Datum	Azure	20837	51328936,53	3406089
4	A. Datum	Silver	25036	72671174,86	3406089
5	A. Datum	Green	21350	52020952,44	3406089
6	A. Datum	Grey	24279	79225719,70	3406089
7	A. Datum	Black	28253	90942577,13	3406089
8	A. Datum	Silv...	9990	47717252,60	3406089
9	A. Datum	Ora...	30637	93022209,61	3406089
10	A. Datum	Pink	28837	79536221,46	3406089
11	Advent	Brown	16812	130429712	3406089

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 111 linhas

GABARITOS

2 Questão 2

Crie mais uma coluna na consulta anterior, incluindo o total de produtos vendidos para cada marca.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd Total Produtos',
    SUM(Quantidade_Vendida) OVER(PARTITION BY Marca) AS 'Qtd Total Por Marca'
FROM vwProdutos
ORDER BY Marca
```

145 %

Resultados | Mensagens

	Marca	Cor	Quantidade_Vendida	Receita_Total	Qtd Total Produtos	Qtd Total Por Marca
1	A. Datum	Blue	5601	28597778,95	3406089	199041
2	A. Datum	Gold	4221	24740930,28	3406089	199041
3	A. Datum	Azure	20837	51328936,53	3406089	199041
4	A. Datum	Silver	25036	72671174,86	3406089	199041
5	A. Datum	Green	21350	52020952,44	3406089	199041
6	A. Datum	Grey	24279	79225719,70	3406089	199041
7	A. Datum	Black	28253	90942577,13	3406089	199041
8	A. Datum	Silver Grey	9990	47717252,60	3406089	199041
9	A. Datum	Orange	30637	93022209,61	3406089	199041
10	A. Datum	Pink	28837	79536221,46	3406089	199041
11	Adventure Works	Brown	16812	130429712,63	3406089	262788

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 111 linhas

GABARITOS

3

Questão 3

Calcule o % de participação do total de vendas de produtos por marca.

Ex: A marca A. Datum teve uma quantidade total de vendas de 199.041 de um total de 3.406.089 de vendas. Isso significa que a da marca A. Datum é 199.041/3.406.089 = 5,84%.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    *,
    SUM(Quantidade_Vendida) OVER() AS 'Qtd1',
    SUM(Quantidade_Vendida) OVER(PARTITION BY Marca) AS 'Qtd2',
    FORMAT(1.0*(SUM(Quantidade_Vendida) OVER(PARTITION BY Marca))/SUM
      (Quantidade_Vendida) OVER(), '0.00%')
FROM vwProdutos
ORDER BY Marca
```

145 %

Resultados | Mensagens

	Marca	Cor	Quantidade_Vendida	Receita_Total	Qtd1	Qtd2	(Nenhum nome de coluna)
1	A. Datum	Blue	5601	28597778,95	3406089	199041	5,84%
2	A. Datum	Gold	4221	24740930,28	3406089	199041	5,84%
3	A. Datum	Azure	20837	51328936,53	3406089	199041	5,84%
4	A. Datum	Silver	25036	72671174,86	3406089	199041	5,84%
5	A. Datum	Green	21350	52020952,44	3406089	199041	5,84%
6	A. Datum	Grey	24279	79225719,70	3406089	199041	5,84%
7	A. Datum	Black	28253	90942577,13	3406089	199041	5,84%
8	A. Datum	Silver Grey	9990	47717252,60	3406089	199041	5,84%
9	A. Datum	Orange	30637	93022209,61	3406089	199041	5,84%
10	A. Datum	Pink	28837	79536221,46	3406089	199041	5,84%
11	Adventure Works	Brown	16812	130429712,63	3406089	262788	7,72%

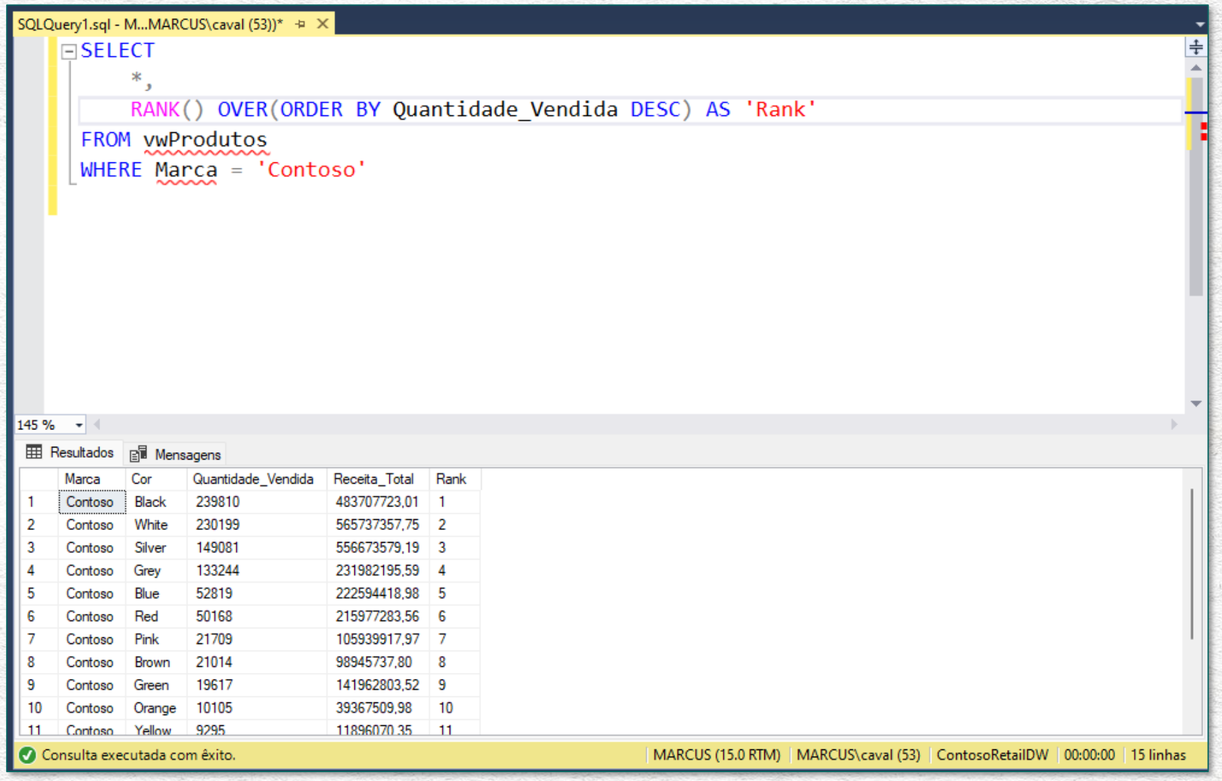
Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:01 | 111 linhas

GABARITOS

4

Questão 4

Crie uma consulta à View **vwProdutos**, selecionando as colunas Marca, Cor, Quantidade_Vendida e também criando uma coluna extra de Rank para descobrir a posição de cada Marca/Cor. Você deve obter o resultado abaixo. Obs: Sua consulta deve ser filtrada para que seja mostrada apenas a marca Contoso.



	Marca	Cor	Quantidade_Vendida	Receita_Total	Rank
1	Contoso	Black	239810	483707723,01	1
2	Contoso	White	230199	565737357,75	2
3	Contoso	Silver	149081	556673579,19	3
4	Contoso	Grey	133244	231982195,59	4
5	Contoso	Blue	52819	222594418,98	5
6	Contoso	Red	50168	215977283,56	6
7	Contoso	Pink	21709	105939917,97	7
8	Contoso	Brown	21014	98945737,80	8
9	Contoso	Green	19617	141962803,52	9
10	Contoso	Orange	10105	39367509,98	10
11	Contoso	Yellow	9295	11896070,35	11

DESAFIO 2

Para responder os próximos 2 exercícios, você precisará criar uma View auxiliar.

A sua view deve se chamar vwHistoricoLojas e deve conter um histórico com a quantidade de lojas abertas a cada Ano/Mês (como mostrado na figura à direita). Os desafios são:

(1) Criar uma coluna de ID para essa View
(2) Relacionar as tabelas DimDate e DimStore

Dicas:

1- A coluna de ID será criada a partir de uma função de janela. Você deverá se atentar a forma como essa coluna deverá ser ordenada, pensando que queremos visualizar uma ordem de Ano/Mês que seja: 2005/january, 2005/February... e não 2005/April, 2005/August...

2- As colunas Ano, Mês e Qtd_Lojas correspondem, respectivamente, às seguintes colunas: CalendarYear e CalendarMonthLabel da tabela DimDate e uma contagem da coluna OpenDate da tabela DimStore.

SQLQuery1.sql - M...MARCUS\caval (53))*

```sql
CREATE VIEW vwHistoricoLojas AS
SELECT
    ROW_NUMBER() OVER(ORDER BY CalendarMonth) AS 'ID',
    CalendarYear AS 'Ano',
    CalendarMonthLabel AS 'Mês',
    COUNT(DimStore.OpenDate) AS 'Qtd_Lojas'
FROM DimDate
LEFT JOIN DimStore
    ON DimDate.Datekey = DimStore.OpenDate
GROUP BY CalendarYear, CalendarMonthLabel, CalendarMonth
```

GABARITOS

5

Questão 5

A partir da view criada no **DESAFIO 2**, você deverá fazer uma soma móvel considerando sempre o mês atual + 2 meses para trás.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
SELECT
    *,
    SUM(Qtd_Lojas) OVER(ORDER BY ID ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
FROM vwHistoricoLojas
```

145 %

Resultados | Mensagens

	ID	Ano	Mês	Qtd_Lojas	(Nenhum nome de coluna)
1	1	2005	January	0	0
2	2	2005	February	0	0
3	3	2005	March	0	0
4	4	2005	April	0	0
5	5	2005	May	2	2
6	6	2005	June	4	6
7	7	2005	July	1	7
8	8	2005	August	0	5
9	9	2005	September	0	1
10	10	2005	October	0	0
11	11	2005	November	1	1

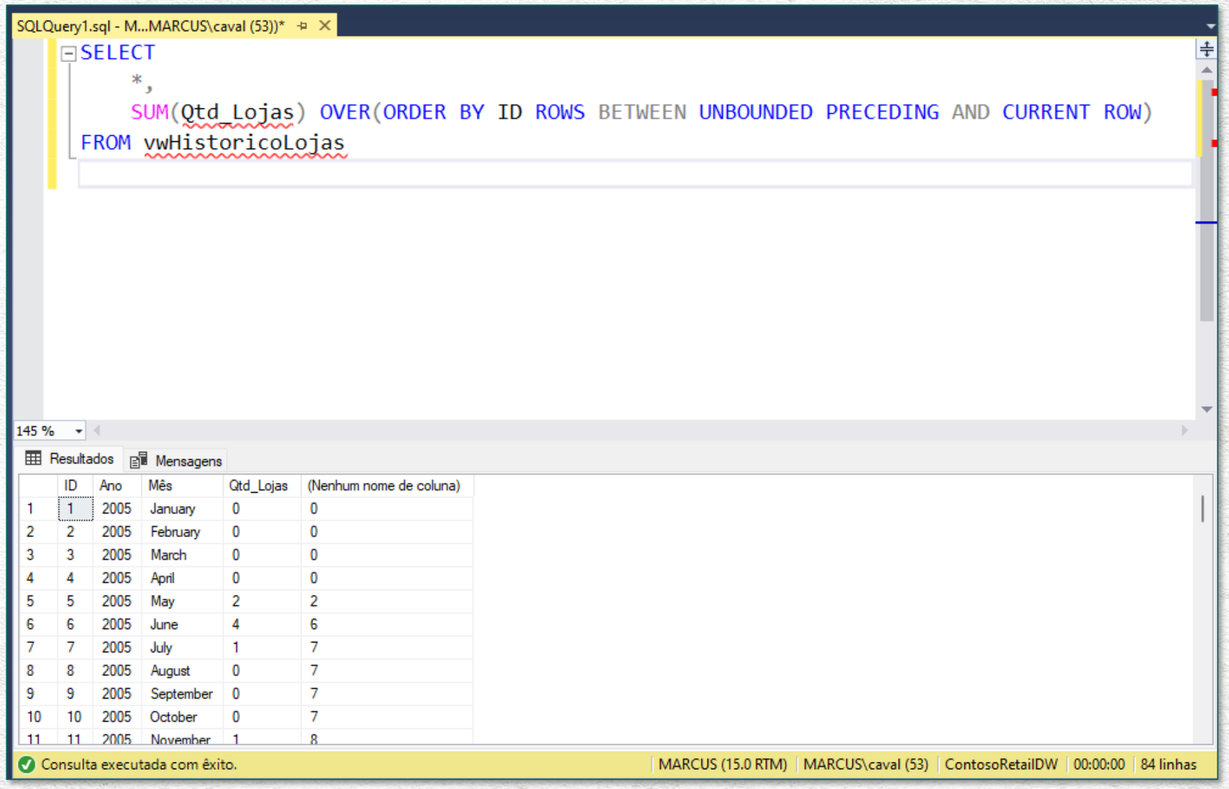
Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | ContosoRetailDW | 00:00:00 | 84 linhas

GABARITOS

6

Questão 6

Utilize a **vwHistoricoLojas** para calcular o acumulado de lojas abertas a cada ano/mês.



	ID	Ano	Mês	Qtd_Lojas	(Nenhum nome de coluna)
1	1	2005	January	0	0
2	2	2005	February	0	0
3	3	2005	March	0	0
4	4	2005	April	0	0
5	5	2005	May	2	2
6	6	2005	June	4	6
7	7	2005	July	1	7
8	8	2005	August	0	7
9	9	2005	September	0	7
10	10	2005	October	0	7
11	11	2005	November	1	8

DESAFIO 3

Neste desafio, você terá que criar suas próprias tabelas e views para conseguir resolver os exercícios 7 e 8. Os próximos exercícios envolverão análises de novos clientes. Para isso, será necessário criar uma nova tabela e uma nova view.

Abaixo, temos um passo a passo para resolver o problema por partes.

PASSO 1: Crie um novo banco de dados chamado Desafio e selecione esse banco de dados criado.

PASSO 2: Crie uma tabela de datas entre o dia 1 de janeiro do ano com a compra (DateFirstPurchase) mais antiga e o dia 31 de dezembro do ano com a compra mais recente.

Obs1: Chame essa tabela de Calendario.

Obs2: A princípio, essa tabela deve conter apenas 1 coluna, chamada data e do tipo DATE.

CONTINUA NA PRÓXIMA PÁGINA

SQLQuery1.sql - M...MARCUS\caval (53))*

```sql
--1) Passo 1: Criar o Banco de Dados Desafio

CREATE DATABASE Desafio
USE Desafio

--2) Passo 2: Criar uma tabela de datas entre o dia 1 de janeiro do ano com a primeira compra de cliente
  mais antiga até o dia 31 de dezembro do ano com a última compra

CREATE TABLE Calendario (
  data DATE
)


DECLARE @varAnoInicial INT = YEAR((SELECT MIN(DateFirstPurchase) FROM ContosoRetailDW.dbo.DimCustomer))
DECLARE @varAnoFinal INT = YEAR((SELECT MAX(DateFirstPurchase) FROM ContosoRetailDW.dbo.DimCustomer))

DECLARE @varDataInicial DATE = DATEFROMPARTS(@varAnoInicial, 1, 1)
DECLARE @varDataFinal DATE = DATEFROMPARTS(@varAnoFinal, 12, 31)


WHILE @varDataInicial <= @varDataFinal
BEGIN
    INSERT INTO Calendario(data) VALUES(@varDataInicial)
    SET @varDataInicial = DATEADD(DAY, 1, @varDataInicial)
END

SELECT * FROM Calendario
```

120 %

DESAFIO 3

Neste desafio, você terá que criar suas próprias tabelas e views para conseguir resolver os exercícios 7 e 8. Os próximos exercícios envolverão análises de novos clientes. Para isso, será necessário criar uma nova tabela e uma nova view.

Abaixo, temos um passo a passo para resolver o problema por partes.

PASSO 3: Crie colunas auxiliares na tabela Calendario chamadas: Ano, Mes, Dia, AnoMes e NomeMes. Todas do tipo INT.

PASSO 4: Adicione na tabela os valores de Ano, Mês, Dia, AnoMes e NomeMes (nome do mês em português). Dica: utilize a instrução CASE para verificar o mês e retornar o nome certo.

CONTINUA NA PRÓXIMA PÁGINA

SQLQuery1.sql - M...MARCUS\caval (53))*

```sql
--3) Passo 3: Criar colunas auxiliares na tabela Calendario

ALTER TABLE Calendario
ADD Ano INT,
    Mes INT,
    Dia INT,
    AnoMes INT,
    NomeMes VARCHAR(50)

--4) Passo 4: Adicionar os valores às colunas

UPDATE Calendario SET Ano = YEAR(data)
UPDATE Calendario SET Mes = MONTH(data)
UPDATE Calendario SET Dia = DAY(data)
UPDATE Calendario SET AnoMes = CONCAT(YEAR(data), FORMAT(MONTH(data), '00'))
UPDATE Calendario SET NomeMes =
        CASE
            WHEN MONTH(data) = 1 THEN 'Janeiro'
            WHEN MONTH(data) = 2 THEN 'Fevereiro'
            WHEN MONTH(data) = 3 THEN 'Março'
            WHEN MONTH(data) = 4 THEN 'Abril'
            WHEN MONTH(data) = 5 THEN 'Maio'
            WHEN MONTH(data) = 6 THEN 'Junho'
            WHEN MONTH(data) = 7 THEN 'Julho'
            WHEN MONTH(data) = 8 THEN 'Agosto'
            WHEN MONTH(data) = 9 THEN 'Setembro'
            WHEN MONTH(data) = 10 THEN 'Outubro'
            WHEN MONTH(data) = 11 THEN 'Novembro'
            WHEN MONTH(data) = 12 THEN 'Dezembro'
        END
```

DESAFIO 3

Neste desafio, você terá que criar suas próprias tabelas e views para conseguir resolver os exercícios 7 e 8. Os próximos exercícios envolverão análises de novos clientes. Para isso, será necessário criar uma nova tabela e uma nova view.

Abaixo, temos um passo a passo para resolver o problema por partes.

PASSO 5: Crie a View vwNovosClientes, que deve ter as colunas mostradas abaixo.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
--5) Passo 5: Crie uma View

CREATE VIEW vwNovosClientes AS
SELECT
    ROW_NUMBER() OVER(ORDER BY AnoMes) AS 'ID',
    Ano,
    NomeMes,
    COUNT(DimCustomer.DateFirstPurchase) AS 'Novos_Clientes'
FROM Calendario
LEFT JOIN ContosoRetailDW.dbo.DimCustomer
    ON Calendario.data = DimCustomer.DateFirstPurchase
GROUP BY Ano, NomeMes, AnoMes

SELECT * FROM vwNovosClientes
```

132 %

Resultados | Mensagens

	ID	Ano	NomeMes	Novos_Clientes
1	1	2001	Janeiro	0
2	2	2001	Fevereiro	0
3	3	2001	Março	0
4	4	2001	Abril	0
5	5	2001	Maio	0
6	6	2001	Junho	0
7	7	2001	Julho	146
8	8	2001	Agosto	156

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | Desafio | 00:00:00 | 48 linhas

GABARITOS

7

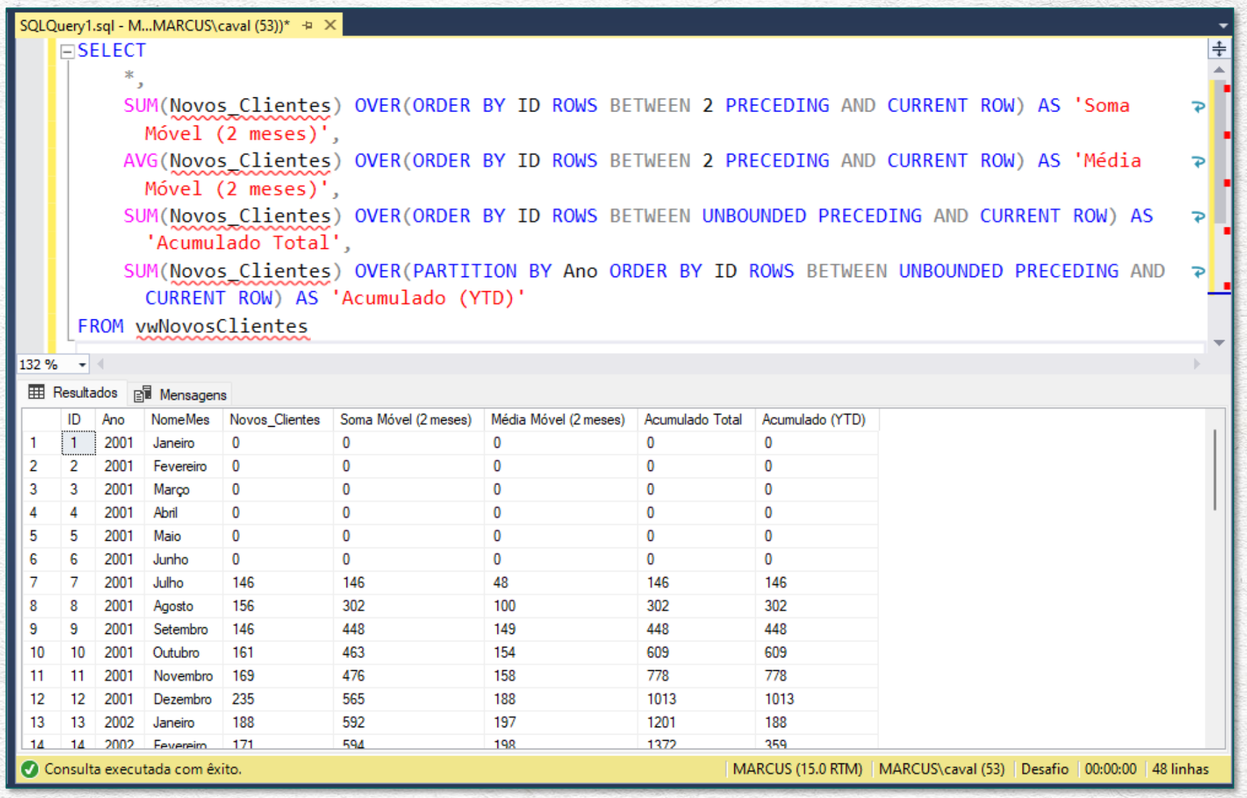
Questão 7

Faça um cálculo de soma móvel de novos clientes nos últimos 2 meses.

b) Faça um cálculo de média móvel de novos clientes nos últimos 2 meses.

c) Faça um cálculo de acumulado dos novos clientes ao longo do tempo.

d) Faça um cálculo de acumulado intra-ano, ou seja, um acumulado que vai de janeiro a dezembro de cada ano, e volta a fazer o cálculo de acumulado no ano seguinte.



	ID	Ano	NomeMes	Novos_Clientes	Soma Móvel (2 meses)	Média Móvel (2 meses)	Acumulado Total	Acumulado (YTD)
1	1	2001	Janeiro	0	0	0	0	0
2	2	2001	Fevereiro	0	0	0	0	0
3	3	2001	Março	0	0	0	0	0
4	4	2001	Abril	0	0	0	0	0
5	5	2001	Maio	0	0	0	0	0
6	6	2001	Junho	0	0	0	0	0
7	7	2001	Julho	146	146	48	146	146
8	8	2001	Agosto	156	302	100	302	302
9	9	2001	Setembro	146	448	149	448	448
10	10	2001	Outubro	161	463	154	609	609
11	11	2001	Novembro	169	476	158	778	778
12	12	2001	Dezembro	235	565	188	1013	1013
13	13	2002	Janeiro	188	592	197	1201	188
14	14	2002	Fevereiro	171	594	198	1372	359

GABARITOS

8

Questão 8

Faça os cálculos de MoM e YoY para avaliar o percentual de crescimento de novos clientes, entre o mês atual e o mês anterior, e entre um mês atual e o mesmo mês do ano anterior.

```
SELECT
    *,
    FORMAT(CONVERT(FLOAT, Novos_Clientes)/NULLIF(LAG(Novos_Clientes, 1) OVER(ORDER BY ID), 0) -
      1, '0.00%') AS '% MoM',
    FORMAT(CONVERT(FLOAT, Novos_Clientes)/NULLIF(LAG(Novos_Clientes, 12) OVER(ORDER BY ID), 0) -
      1, '0.00%') AS '% YoY'
FROM vwNovosClientes
```

Resultados | Mensagens

	ID	Ano	NomeMes	Novos_Clientes	% MoM	% YoY
1	1	2001	Janeiro	0	NULL	NULL
2	2	2001	Fevereiro	0	NULL	NULL
3	3	2001	Março	0	NULL	NULL
4	4	2001	Abril	0	NULL	NULL
5	5	2001	Maio	0	NULL	NULL
6	6	2001	Junho	0	NULL	NULL
7	7	2001	Julho	146	NULL	NULL
8	8	2001	Agosto	156	6,85%	NULL
9	9	2001	Setembro	146	-6,41%	NULL
10	10	2001	Outubro	161	10,27%	NULL
11	11	2001	Novembro	169	4,97%	NULL
12	12	2001	Dezembro	235	39,05%	NULL
13	13	2002	Janeiro	188	-20,00%	NULL
14	14	2002	Fevereiro	171	-9,04%	NULL
15	15	2002	Março	199	16,37%	NULL
16	16	2002	Abril	207	4,02%	NULL
17	17	2002	Maio	214	3,38%	NULL

Consulta executada com êxito. | MARCUS (15.0 RTM) | MARCUS\caval (53) | Desafio | 00:00:00 | 48 linhas

MÓDULO 17

SQL

REGEX
(REGULAR EXPRESSIONS)

O QUE VEREMOS NESTE MÓDULO

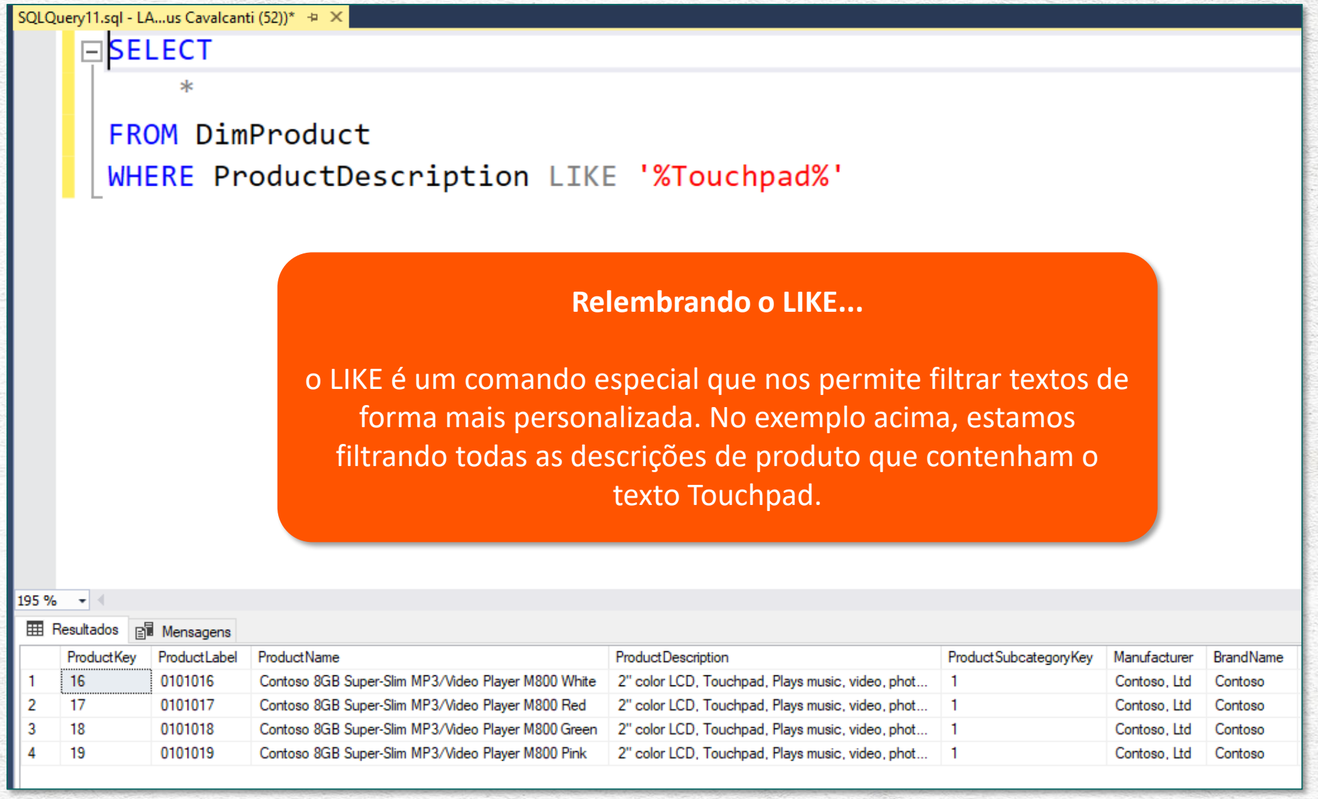
MÓDULO 17 REGEX (REGULAR EXPRESSIONS)

01	REGULAR EXPRESSIONS - Introdução	472
02	COLLATE – Introdução	473
03	Um pouco mais sobre COLLATE	476
04	LIKE – Case Sensitive	480
05	LIKE – Filtrando os primeiros caracteres	481
06	LIKE – Filtrando 1os caracteres + case sensitive	482
07	LIKE – Aplicando filtro mais personalizado	483
08	LIKE – Utilizando o operador de negação	484
09	LIKE – Identificando caracteres especiais	485
10	LIKE – Aplicações com números	486

REGULAR EXPRESSIONS – Introdução

O QUE É E PARA QUE SERVEM?

- A expressão regular (conhecida como regex ou regexp, do inglês *regular expression*) permite uma forma de identificar cadeias de caracteres de interesse, como caracteres específicos, palavras ou padrões de caracteres.
- Em resumo, a expressão regular é uma forma de permitir realizar, de forma simples, operações bastante complexas com strings, que possivelmente exigiriam várias condições para tratar cada caso.
- O SQL Server já possui uma opção para tratar casos especiais de textos, por meio do comando LIKE, que já vimos anteriormente. Porém, neste módulo vamos ver diversas aplicações mais avançadas.



	ProductKey	ProductLabel	ProductName	ProductDescription	ProductSubcategoryKey	Manufacturer	BrandName
1	16	0101016	Contoso 8GB Super-Slim MP3/Video Player M800 White	2" color LCD, Touchpad, Plays music, video, phot...	1	Contoso, Ltd	Contoso
2	17	0101017	Contoso 8GB Super-Slim MP3/Video Player M800 Red	2" color LCD, Touchpad, Plays music, video, phot...	1	Contoso, Ltd	Contoso
3	18	0101018	Contoso 8GB Super-Slim MP3/Video Player M800 Green	2" color LCD, Touchpad, Plays music, video, phot...	1	Contoso, Ltd	Contoso
4	19	0101019	Contoso 8GB Super-Slim MP3/Video Player M800 Pink	2" color LCD, Touchpad, Plays music, video, phot...	1	Contoso, Ltd	Contoso

COLLATE – Introdução

COLLATION é um conjunto de regras que informam ao mecanismo de banco de dados como comparar e classificar os dados de caracteres no SQL Server.

Em resumo, ele serve para indicar se um determinado campo será CASE INSENSITIVE e também a forma como interpreta a acentuação das palavras. Observe os exemplos abaixo.

1

ID	NOME
1	MARCUS
2	marcus
3	André
4	andre

Temos uma tabela com dois nomes: Marcus e André. Só que cada nome é escrito de uma forma diferente: MARCUS e marcus, André e andre. Como será que filtros de texto se comportam com essas diferenciações?

2

```
SELECT
    ID,
    NOME
FROM
    Teste
WHERE NOME = 'MARCUS'
```

09 % | Resultados | Mensagens

	ID	NOME
1	1	MARCUS
2	2	marcus

Filtrando o nome 'MARCUS', temos as duas linhas com esse nome como resultado, mesmo que uma delas esteja escrita em letras minúsculas.

3

```
SELECT
    ID,
    NOME
FROM
    Teste
WHERE NOME = 'andre'
```

09 % | Resultados | Mensagens

	ID	NOME
1	4	andre

Filtrando 'andre', apenas 1 linha é retornada, pois o outro nome está escrito com acento.

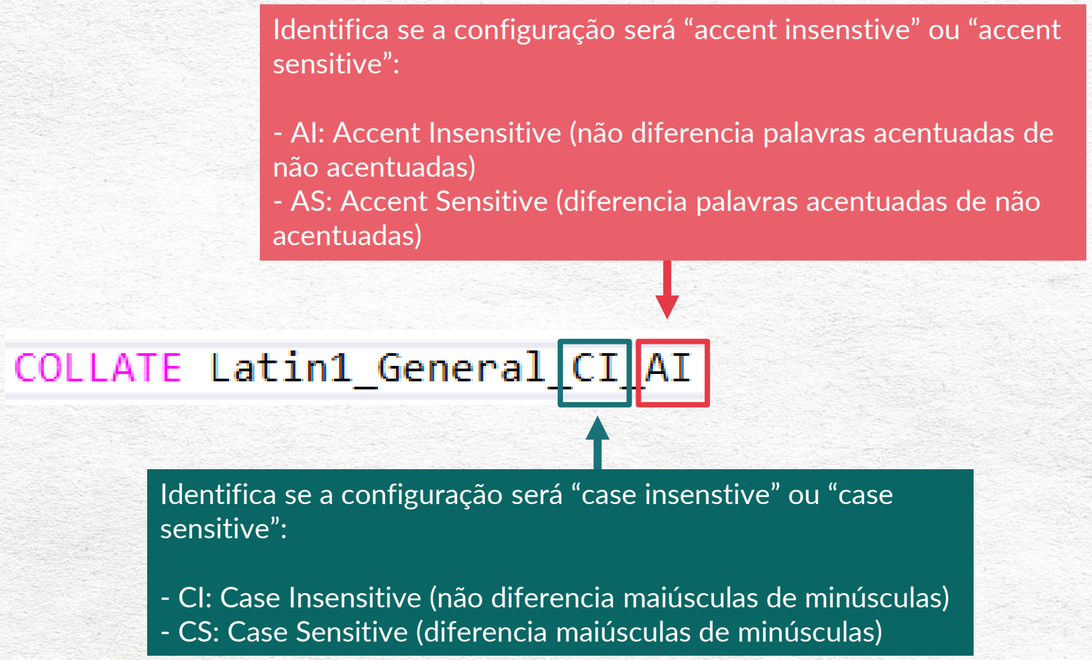
Índice Geral
Índice deste Módulo

COLLATE – Introdução

Quando usamos o COLLATE, conseguimos configurar duas coisas na hora de criar os filtros de textos:

(i) Se haverá ou não diferenciação entre letras maiúsculas e minúsculas.

(ii) Se haverá ou não diferenciação entre letras acentuadas e não acentuadas.



COLLATE – Introdução

Observe agora uma nova situação, dessa vez usando o COLLATE.

1

ID	NOME
1	MARCUS
2	marcus
3	André
4	andre

Temos a mesma tabela com dois nomes: Marcus e André.

2

```
SELECT
    ID,
    NOME
FROM
    Teste
WHERE NOME COLLATE Latin1_General_CS_AI = 'Marcus'
```

Resultados | Mensagens

ID	NOME

Filtrando o nome 'Marcus', ele não retorna nenhuma linha, pois nenhuma está escrita exatamente dessa forma (considerando maiúsculas e minúsculas).

3

```
SELECT
    ID,
    NOME
FROM
    Teste
WHERE NOME COLLATE Latin1_General_CI_AI = 'Andre'
```

Resultados | Mensagens

	ID	NOME
1	3	André
2	4	andre

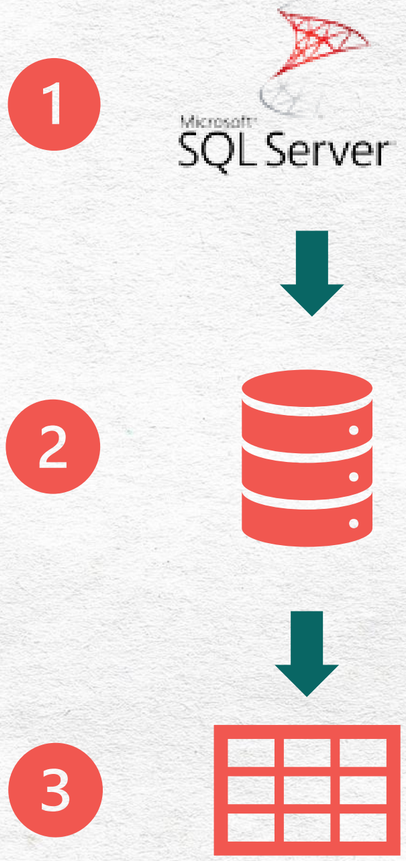
Filtrando 'Andre', as duas linhas são retornadas, mesmo que em uma delas tenhamos o nome com acento.

Um pouco mais sobre COLLATE

O COLLATION nos permite configurar se teremos diferenciação entre maiúsculas e minúsculas, ou entre palavras acentuadas.

Ele pode ser definido em níveis diferentes no SQL Server. Abaixo estão os três níveis:

1. A nível de programa (SQL Server)
2. A nível de Bancos de Dados
3. A nível de tabelas/colunas



Um pouco mais sobre COLLATE

1. A NÍVEL DE SQL SERVER

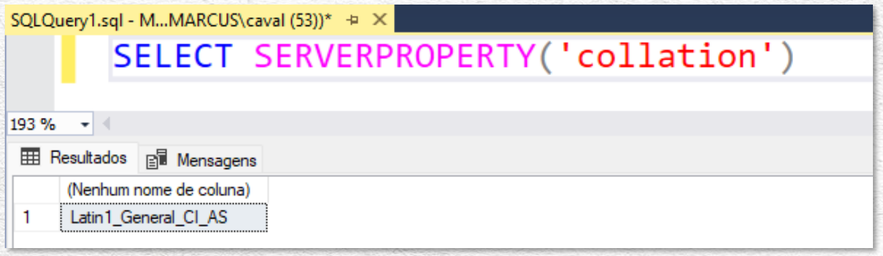
O **COLLATION** a princípio é definido durante a instalação do programa.

Por padrão, o **COLLATION** padrão é o seguinte:

Latin1_General_CI_AS

Onde CI significa Case Insensitive (não diferencia maiúsculas de minúsculas) e AS significa Accent Sensitive (sensível ao sotaque).

Para descobrir o **COLLATION** configurado, podemos utilizar o comando abaixo:



Um pouco mais sobre COLLATE

2. A NÍVEL DE BANCO DE DADOS

Por padrão, todos os bancos de dados vão herdar a configuração do COLLATION feita durante a instalação do SQL Server.

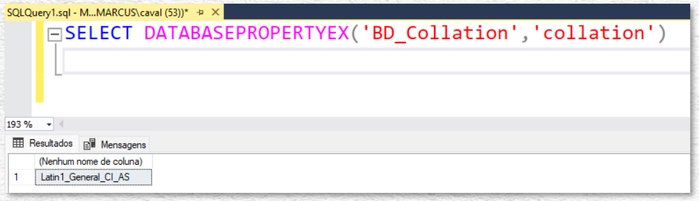
Nós podemos também especificar o COLLATION do Banco de Dados no momento da sua criação.

```
CREATE DATABASE BD_Collation
COLLATE Latin1_General_CS_AS
```

Para alterar o COLLATE após criar um banco de dados, usamos o comando abaixo:*/

```
ALTER DATABASE BD_Collation
COLLATE Latin1_General_CI_AS
```

Para saber o COLLATION de um Banco de Dados específico, podemos usar o comando abaixo:



Índice Geral

Índice deste Módulo

Um pouco mais sobre COLLATE

3. A NÍVEL DE TABELA/COLUNA

Por padrão, uma nova coluna de tipo VARCHAR herda o COLLATION do banco de dados, a menos que você especifique o COLLATION explicitamente ao criar a tabela.

Para criar uma coluna com um COLLATION diferente, você pode especificar usando o comando Collate SQL.

```
CREATE TABLE Nomes(
ID INT,
Nome1 VARCHAR(100),
Nome2 VARCHAR(100) COLLATE Latin1_General_CS_AS)
```

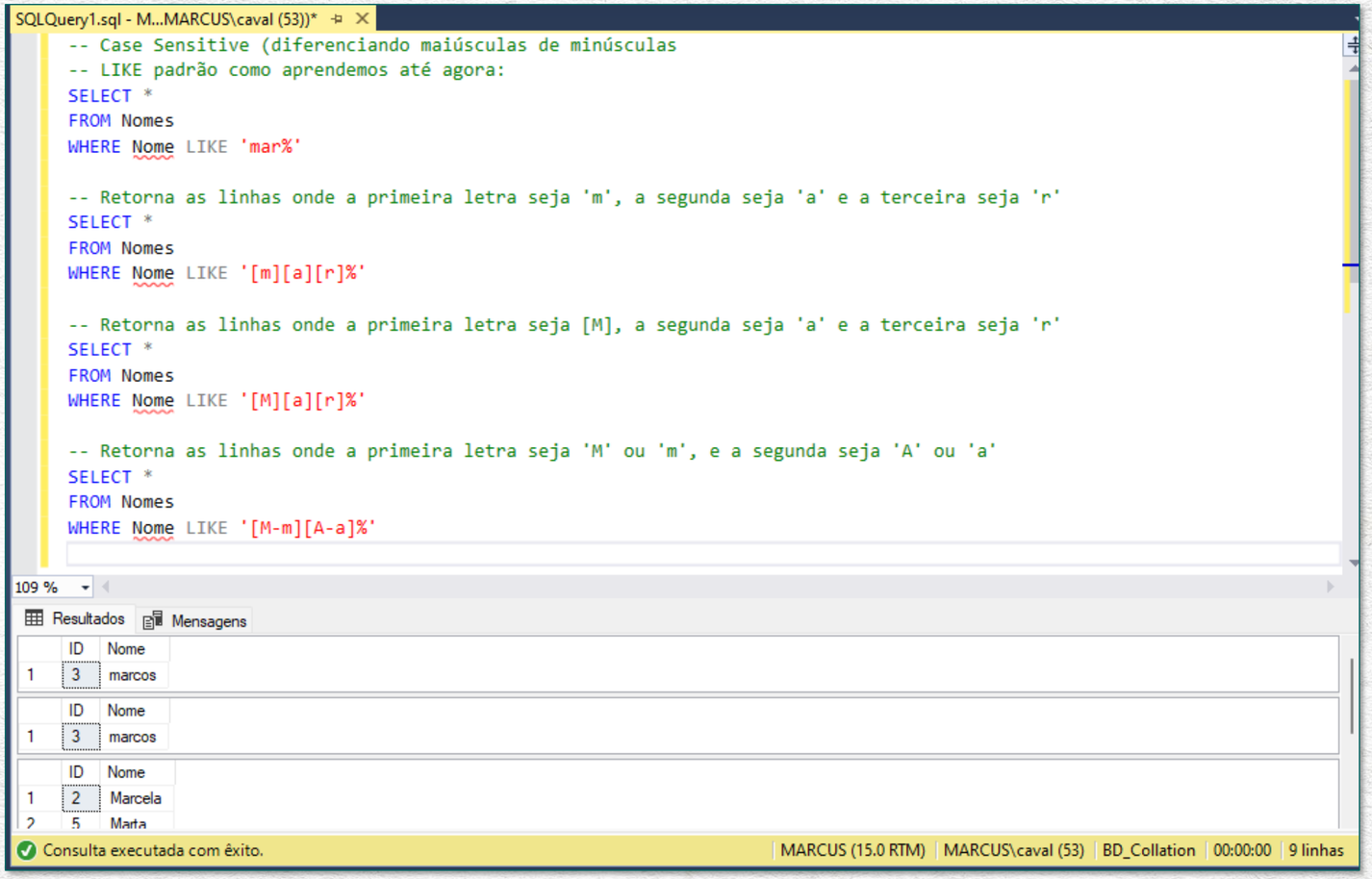
Podemos ver o COLLATION de cada coluna da tabela usando o comando abaixo:

```
sp_help Nomes
```

LIKE – Case Sensitive

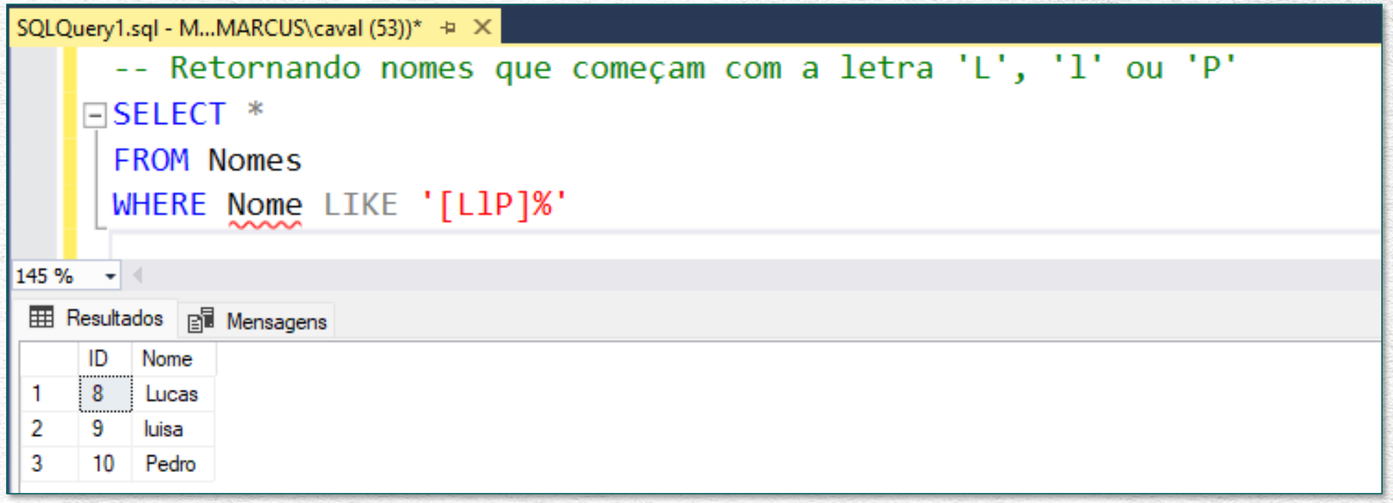
No exemplo ao lado, temos vários exemplos de aplicação do LIKE e suas respectivas descrições.

Como podemos ver na imagem ao lado, podemos usar colchetes como forma de criar filtros ainda mais personalizados.



LIKE – Filtrando os primeiros caracteres

No próximo exemplo vemos como é possível filtrar uma coluna a partir do primeiro caractere da palavra.



LIKE – Filtrando os primeiros caracteres + case sensitive

Agora vamos trabalhar com uma outra tabela, que vamos chamar de **Textos**.

Neste exemplo vamos ver mais algumas aplicações de filtros especiais, como pode ser visto na imagem ao lado.

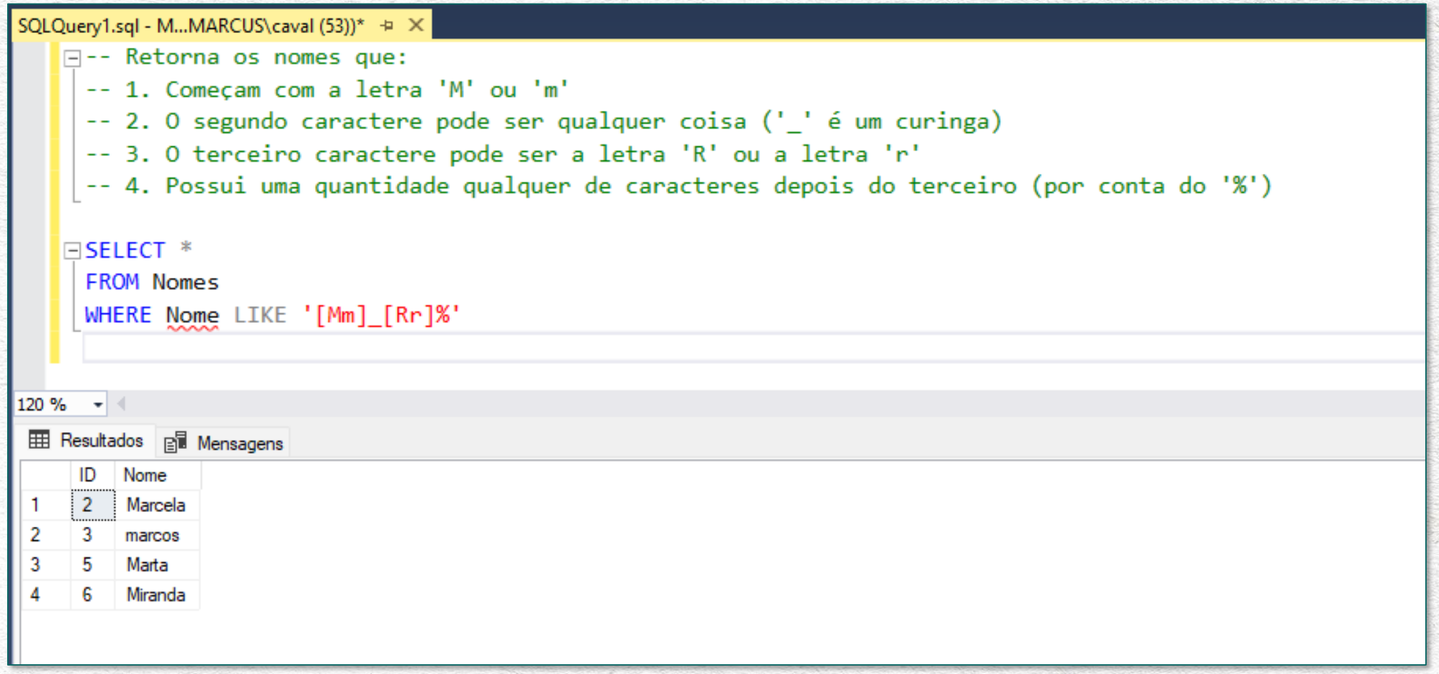
SQLQuery1.sql - M...MARCUS\caval (53))*

```
USE BD_Collation
CREATE TABLE Textos(
ID INT,
Texto VARCHAR(100) COLLATE SQL_Latin1_General_CP1_CS_AS)

INSERT INTO Textos(ID, Texto)
VALUES
    (1, 'Marcos'), (2, 'Excel'), (3, 'leandro'), (4, 'K'), (5, 'X7'), (6, 'l9'), (7, '#M'), (8, '@9'), (9,
      'M'), (10, 'RT')

-- Retornando nomes que possuem apenas 1 caracteres
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z]'

-- Retornando nomes que possuem apenas 2 caracteres
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z][A-z]'

-- Retornando nomes que possuem apenas 2 caracteres: o primeiro uma letra, o segundo um número
SELECT *
FROM Textos
WHERE Texto LIKE '[A-z][0-9]'
```

LIKE – Aplicando um filtro ainda mais personalizado

Conseguimos personalizar ainda mais o filtro combinando o que vimos até agora.

No exemplo ao lado, usamos o colchetes, o underline (para especificar 'qualquer caractere' naquela posição) e também o sinal de percentual, que significa que temos uma quantidade de texto qualquer no final da palavra.



LIKE – Utilizando o operador de negação

Neste exemplo vemos a aplicação do operador ^, que faz uma negação no nosso filtro de textos.

Ao lado, é possível visualizar a aplicação deste operador.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
-- Retorna nomes que não começa com as letras 'L' ou 'l'
SELECT *
FROM Nomes
WHERE Nome LIKE '[^Ll]%'

-- Retorna nomes que não começa com as letras 'L' ou 'l'
SELECT *
FROM Nomes
WHERE Nome LIKE '_[^Ee]%'
```

120 %

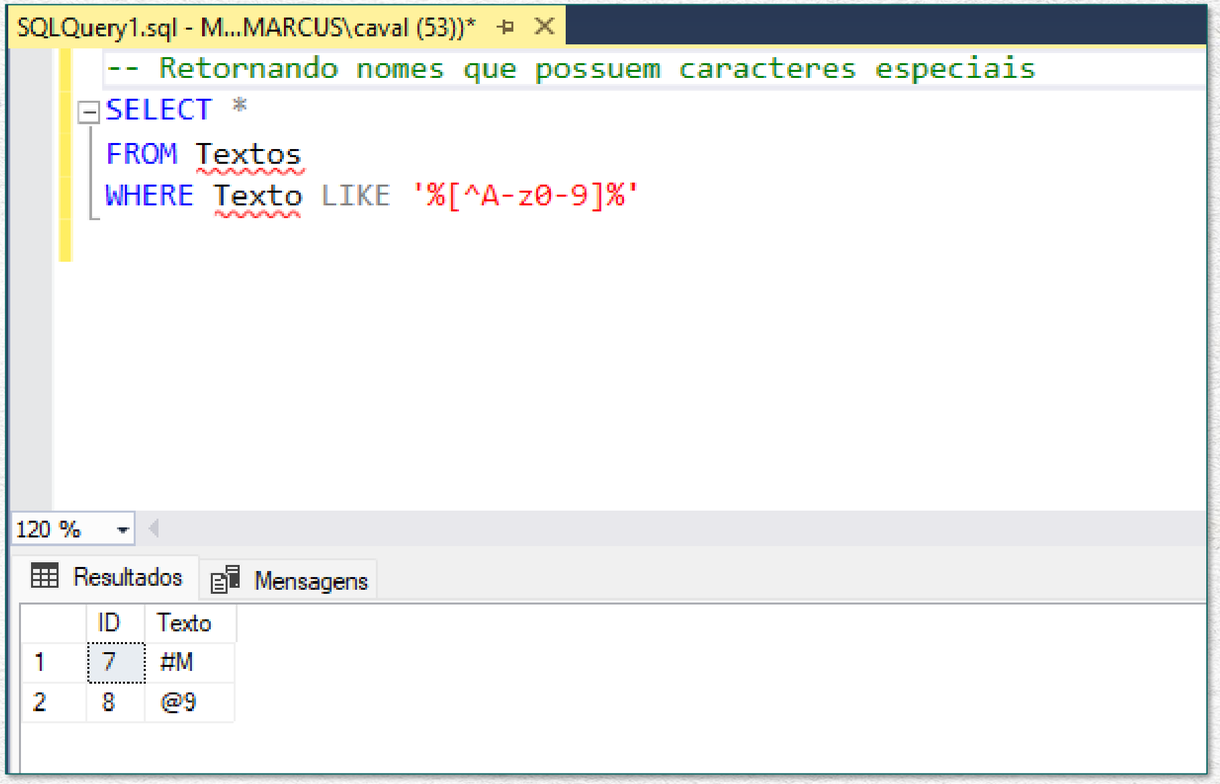
Resultados | Mensagens

	ID	Nome
1	1	Matheus
2	2	Marcela
3	3	marcos
4	4	MAuricio
5	5	Marta
6	6	Miranda
7	7	Melissa
8	10	Pedro

	ID	Nome
1	1	Matheus
2	2	Marcela
3	3	marcos
4	4	MAuricio
5	5	Marta
6	6	Miranda

LIKE – Identificando caracteres especiais

É possível fazer um filtro para visualizar todos os textos que possuem caracteres especiais. A aplicação é mostrada ao lado.



LIKE – Aplicações com números

Por fim, temos uma aplicação especial para números, como mostrado nos exemplos ao lado.

SQLQuery1.sql - M...MARCUS\caval (53))*

```
USE BD_Collation
CREATE TABLE Numeros(
Numero DECIMAL(20, 2))

INSERT INTO Numeros(Numero)
VALUES
    (50), (30.23), (9), (100.54), (15.9), (6.5), (10), (501.76), (1000.56), (31)

-- Retornando os números que possuem 2 dígitos na parte inteira
SELECT *
FROM Numeros
WHERE Numero LIKE '[0-9][0-9].[0][0]'

-- Retornando linhas que:
-- 1. Possuem 3 dígitos na parte inteira, sendo o primeiro dígito igual a 5
-- 2. O primeiro número da parte decimal seja 7.
-- 3. O segundo número da parte decimal seja um número entre 0 e 9.

SELECT *
FROM Numeros
WHERE Numero LIKE '[5]__.[7][0-9]'
```

99 %

Resultados | Mensagens

	Numero
1	50.00
2	10.00
3	31.00

	Numero
1	501.76

MÓDULO 18

SQL

Constraints

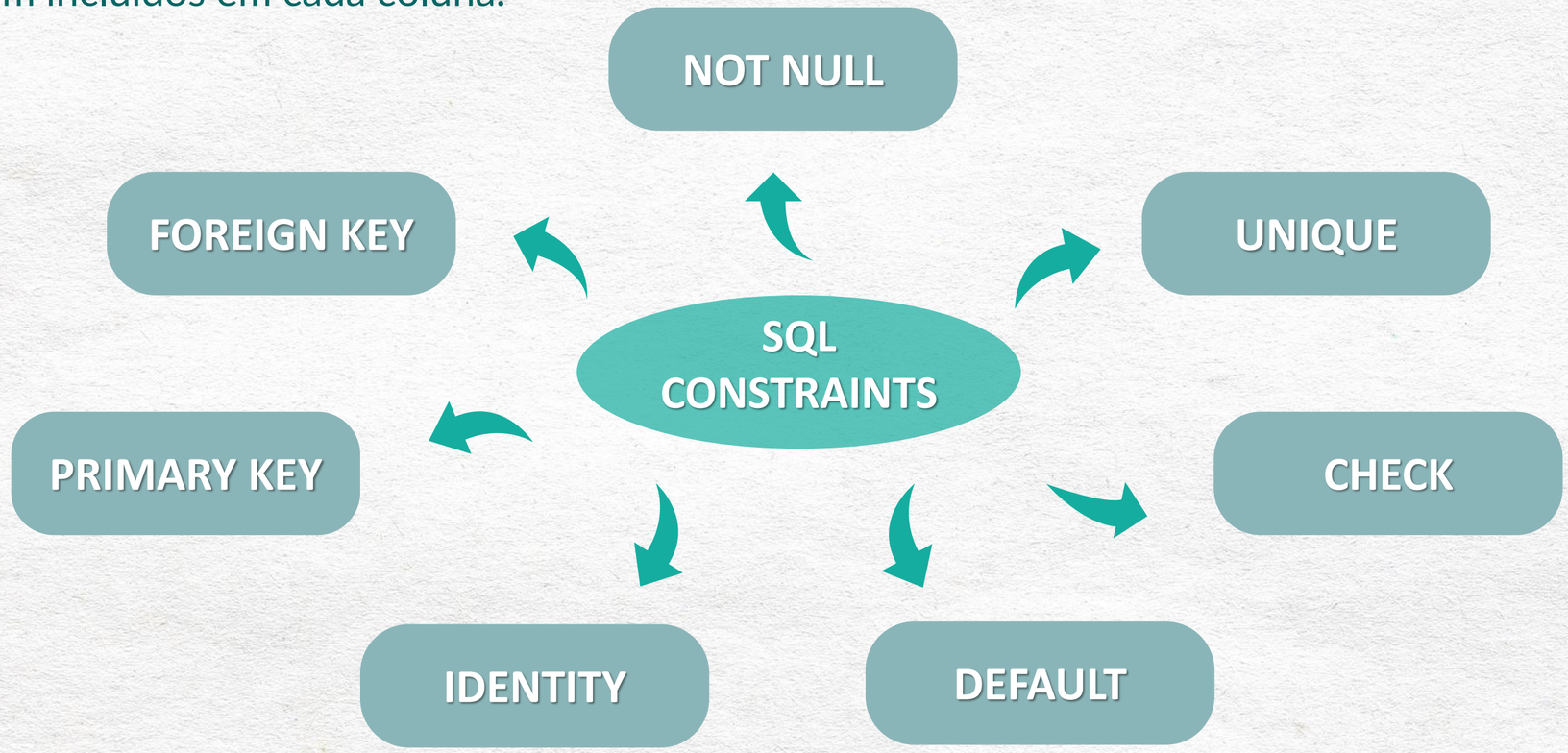
O QUE VEREMOS NESTE MÓDULO

MÓDULO 18 CONSTRAINTS

01	O que são CONSTRAINTS?	489
02	Constraints: NOT NULL	490
03	Constraints: UNIQUE	491
04	Constraints: CHECK	492
05	Constraints: DEFAULT	493
06	Constraints: IDENTITY	494
07	Constraints: PRIMARY KEY	495
08	Constraints: FOREIGN KEY	496
09	Violação de Constraints: NOT NULL	497
10	Violação de Constraints: CHECK	499
11	Violação de Constraints: UNIQUE	501
12	Violação de Constraints: PK e FK	503
13	Gerenciando Constraints: ADICIONAR	506
14	Gerenciando Constraints: RENOMEAR	507
15	Gerenciando Constraints: REMOVER	508
16	EXERCÍCIOS	509
17	GABARITOS	512

O que são CONSTRAINTS?

CONSTRAINTS no SQL são regras (restrições) que podemos definir para as colunas de uma tabela. Essas regras garantem integridade ao Banco de Dados, pois é através delas que garantimos que apenas os valores que atendam às regras pré-estabelecidas sejam incluídos em cada coluna.



Constraints: NOT NULL

A CONSTRAINT NOT NULL não permite que sejam adicionados valores nulos à coluna.

Repare ao lado na criação da tabela dCliente:

Definimos a restrição NOT NULL para as colunas nome_cliente, genero, data_de_nascimento e cpf.

Isso significa que, quando formos inserir linhas na tabela dCliente, não podemos deixar de informar dados para estas colunas.

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

Constraints: UNIQUE

A CONSTRAINT UNIQUE identifica uma coluna de forma única, sem permitir valores duplicados (mas, permite NULL).

Repare ao lado na criação da tabela dCliente:

Definimos a restrição UNIQUE para a coluna cpf.

Isso significa que, quando formos inserir linhas na tabela dCliente, não podemos informar o mesmo CPF para dois ou mais clientes.

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

Constraints: CHECK

A CONSTRAINT CHECK verifica se o valor adicionado na coluna atende a uma determinada condição.

Repare ao lado na criação da tabela dGerente:

Definimos a restrição CHECK para a coluna salario, para verificar se os valores informados são maiores que 0.

Isso significa que, quando formos inserir linhas na tabela dGerente, não podemos informar valores negativos ou iguais a 0 para a coluna salario.

```
CREATE TABLE dGerente(
    id_gerente INT IDENTITY(1, 1),
    nome_gerente VARCHAR(100) NOT NULL,
    data_contratacao VARCHAR(100) NOT NULL,
    salario FLOAT NOT NULL,
    CONSTRAINT dgerente_id_gerente_pk PRIMARY KEY(id_gerente),
    CONSTRAINT dgerente_salario_ck CHECK(salario > 0)
)
```

Constraints: DEFAULT

A CONSTRAINT DEFAULT retorna um valor default (padrão) caso a coluna não seja preenchida.

Repare na criação da tabela fContratos:

Definimos a restrição DEFAULT SYSDATE para a coluna data_assinatura.

Isso significa que, quando formos inserir linhas na tabela fContratos, caso não seja informada nenhuma data para a coluna data_assinatura, ela será preenchida, por padrão, com a data atual.

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

Constraints: IDENTITY

A CONSTRAINT IDENTITY permite que uma coluna siga uma auto numeração (usada em colunas de ID).

Repare na criação da tabela fContratos:

Definimos a restrição IDENTITY para a coluna id_contrato.

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

Isso significa que, quando formos inserir linhas na tabela fContratos, não precisaremos preencher a coluna id_contrato, pois a restrição IDENTITY irá auto incrementar esta coluna.

Note que é necessário informar, no primeiro argumento da IDENTITY o número a ser considerado como valor inicial da numeração (aqui começamos com o número 1); e no segundo argumento, o passo desta incrementação: no caso deste exemplo, de 1 em 1.

Resultados | Mensagens

	id_contrato	data_assinatura	id_cliente	id_gerente	valor_contrato
1	1	2019-01-12	8	1	23000
2	2	2019-02-10	3	2	15500
3	3	2019-03-07	7	2	6500
4	4	2019-03-15	1	3	33000
5	5	2019-03-21	5	4	11100
6	6	2019-03-23	4	2	5500
7	7	2019-03-28	9	3	55000
8	8	2019-04-04	2	1	31000
9	9	2019-04-05	10	4	3400
10	10	2019-04-05	6	2	9200

Coluna id_contrato auto incrementada pela Constraint IDENTITY

Índice Geral
Índice deste Módulo

Constraints: PRIMARY KEY

A CONSTRAINT PRIMARY KEY torna uma coluna a chave primária da tabela. É uma coluna que identifica de forma única as linhas de uma tabela.

Nenhum dos valores de uma coluna de chave primária deve ser nulo ou se repetir. Será através dessa coluna que criaremos relações entre as tabelas.

Repare na criação da tabela fContratos:

Definimos a restrição PRIMARY KEY para a coluna id_contrato.

Isso significa que esta coluna é a chave primária da tabela fContratos e será utilizada para se relacionar com outras tabelas, que farão referência a ela como uma FOREIGN KEY (chave estrangeira).

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

Constraints: FOREIGN KEY

A CONSTRAINT FOREIGN KEY torna uma coluna uma chave estrangeira da tabela, que será relacionada à chave primária (PRIMARY KEY) de outra tabela.

Repare na criação da tabela fContratos:

Definimos a restrição FOREIGN KEY para as colunas id_cliente e id_gerente.

Isso significa que estas colunas se relacionarão com as PRIMARY KEYS (chaves primárias) id_cliente e id_gerente das tabelas dCliente e dGerente, respectivamente.

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

Violação de Constraints: NOT NULL

Recapitulando, a CONSTRAINT NOT NULL não permite que sejam adicionados valores nulos à coluna.

Agora, repare na criação da nossa tabela dCliente:

Podemos observar que definimos a CONSTRAINT NOT NULL para diversas colunas, entre elas a coluna nome_cliente.

Isto significa que a coluna nome_cliente precisa ser devidamente preenchida quando inserimos um novo registro (linha) na tabela dCliente.

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

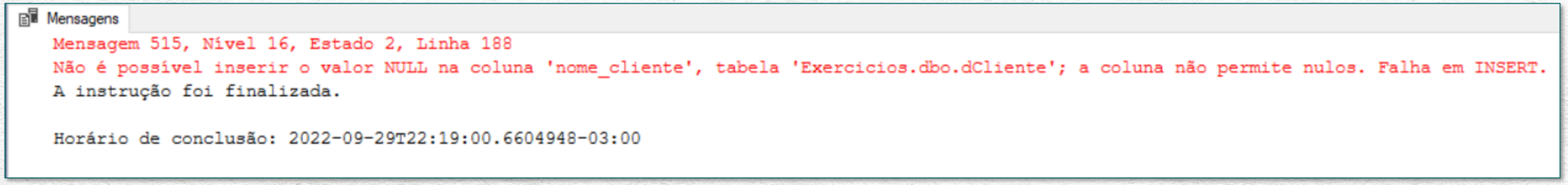
Violação de Constraints: NOT NULL

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos um problema. Consegue perceber por quê?

```
INSERT INTO dCliente(Genero, Data_de_Nascimento, CPF) VALUES
    ('M', '20/03/1989', '123.456.789-00');
```

Se você respondeu que é porque não especificamos a coluna nome_cliente no nosso código, acertou!

Quando deixamos de informar uma coluna na hora de inserir novos dados, o sistema entende que aquela coluna deve ser deixada vazia (preenchida com null). Como definimos, quando criamos nossa tabela dCliente, a restrição NOT NULL para a coluna nome_cliente, o sistema vai retornar o seguinte erro ao tentar executar o código acima:



Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: CHECK

Recapitulando, a CONSTRAINT CHECK verifica se o valor adicionado na coluna atende a uma determinada condição.

Agora, repare na criação da nossa tabela dCliente:

Podemos observar que definimos a CONSTRAINT CHECK para a coluna genero, estabelecendo a condição:

genero IN (‘M’, ‘F’, ‘O’, ‘PND’)

Isto significa que a coluna genero só pode ser preenchida com os valores ‘M’ (para masculino), ‘F’ (para feminino), ‘O’ (para outros) ou ‘PND’ (para prefiro não declarar) quando inserimos um novo registro (linha) na tabela dCliente.

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

Violação de Constraints: CHECK

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos outro problema. Só que desta vez será porque tentamos cadastrar um valor diferente de 'M', 'F', 'O' ou 'PND' na coluna genero:

```
INSERT INTO dCliente(Nome_Cliente, Genero, Data_de_Nascimento, CPF) VALUES
    ('Lucia Campos', 'Feminino', '23/10/1994', '999.234.812-20');
```

Quando violamos uma restrição CHECK, o sistema retorna um erro informando os nomes da constraint, do banco de dados, da tabela e da coluna violada:

Mensagens

```
Mensagem 547, Nível 16, Estado 0, Linha 191
A instrução INSERT conflitou com a restrição do CHECK "dcliente_genero_ck". O conflito ocorreu no banco de dados "Exercicios", tabela "dbo.dCliente", column 'genero'.
A instrução foi finalizada.

Horário de conclusão: 2022-09-29T22:29:24.3698948-03:00
```

Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: UNIQUE

Recapitulando, a CONSTRAINT UNIQUE identifica uma coluna de forma única, sem permitir valores duplicados.

Agora, repare na criação da nossa tabela dCliente:

Podemos observar que definimos a CONSTRAINT UNIQUE para a coluna cpf.

Isto significa que a coluna cpf não pode receber um valor que já conste na coluna como pertencente a algum outro registro (cliente) da tabela dCliente.

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

Violação de Constraints: UNIQUE

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos mais um problema...

```
INSERT INTO dCliente(Nome_Cliente, Genero, Data_de_Nascimento, CPF) VALUES
    ('Lucia Campos',  'F', '23/10/1994', '839.283.190-00');
```

... pois o CPF informado já foi informado como sendo de outro cliente. Veja:

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00

Quando violamos uma restrição UNIQUE, o sistema retorna um erro informando os nomes da constraint e da tabela, assim como o valor da chave duplicada:

Mensagens

```
Mensagem 2627, Nível 14, Estado 1, Linha 198
Violação da restrição UNIQUE KEY 'dcliente_cpf_un'. Não é possível inserir a chave duplicada no objeto 'dbo.dCliente'. O valor de chave duplicada é (839.283.190-00).
A instrução foi finalizada.

Horário de conclusão: 2022-09-29T23:52:50.4784075-03:00
```

Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Recapitulando, a CONSTRAINT PRIMARY KEY torna uma coluna a chave primária da tabela, ao passo que a CONSTRAINT FOREIGN KEY torna uma coluna uma chave estrangeira da tabela, que será relacionada à chave primária (PRIMARY KEY) de outra tabela.

Agora, repare na criação da nossa tabela fContratos:

Podemos observar que definimos a CONSTRAINT FOREIGN KEY para a coluna id_gerente, que faz referência à PRIMARY KEY id_gerente estabelecida na tabela dGerente.

Isto significa que a coluna id_gerente da tabela fContratos não pode receber um ID de um gerente que não exista na coluna id_gerente da tabela dGerente.

```
CREATE TABLE fContratos(
    id_contrato INT IDENTITY(1, 1),
    data_assinatura DATE DEFAULT GETDATE(),
    id_cliente INT,
    id_gerente INT,
    valor_contrato FLOAT,
    CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato),
    CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente),
    CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
    CONSTRAINT fcontratos_valor_contrato_ck CHECK(valor_contrato > 0)
)
```

```
CONSTRAINT fcontratos_id_gerente_fk FOREIGN KEY(id_gerente) REFERENCES dGerente(id_gerente),
```

Foreign Key da tabela fContratos

Referência à Primary Key da tabela dGerente

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Portanto, se tentarmos executar, por exemplo, o INSERT INTO abaixo, teremos outro problema...

```
INSERT INTO fContratos(Data_Assinatura, ID_Cliente, ID_Gerente, Valor_Contrato) VALUES
    ('12/03/2022', 4, 10, 1500);
```

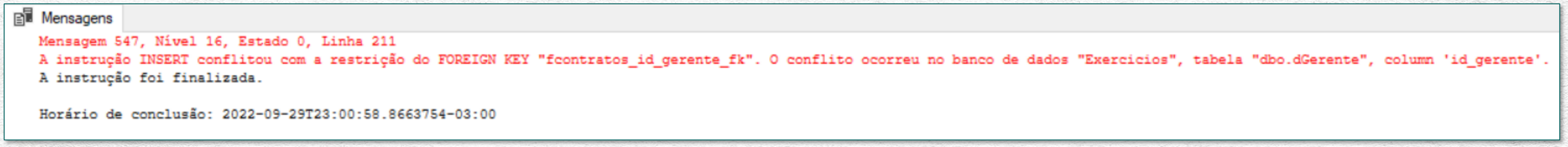
... pois o id_gerente informado (10) não existe na tabela dGerente. Veja:

Resultados | Mensagens

	id_gerente	nome_gerente	data_contratacao	salario
1	1	Lucas Sampaio	21/03/2015	6700
2	2	Mariana Padilha	10/01/2011	9900
3	3	Nathália Santos	03/10/2018	7200
4	4	Otávio Costa	18/04/2017	11000

Violação de Constraints: PRIMARY KEY e FOREIGN KEY

Quando violamos as restrições PRIMARY KEY e FOREIGN KEY, o sistema retorna um erro informando os nomes da constraint, do banco de dados, da tabela e da coluna violada:



Desta forma, a tentativa de inserção de novo registro é considerada falha, e nenhum dado deste novo registro é salvo no banco de dados, nem mesmo nas outras colunas que atendiam às especificações.

Gerenciando Constraints: ADICIONAR

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para ADICIONAR uma nova CONSTRAINT a uma tabela, utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
ADD CONSTRAINT   nome_da_nova_constraint    tipo_da_nova_constraint (nome_da_coluna);
```

Exemplos:

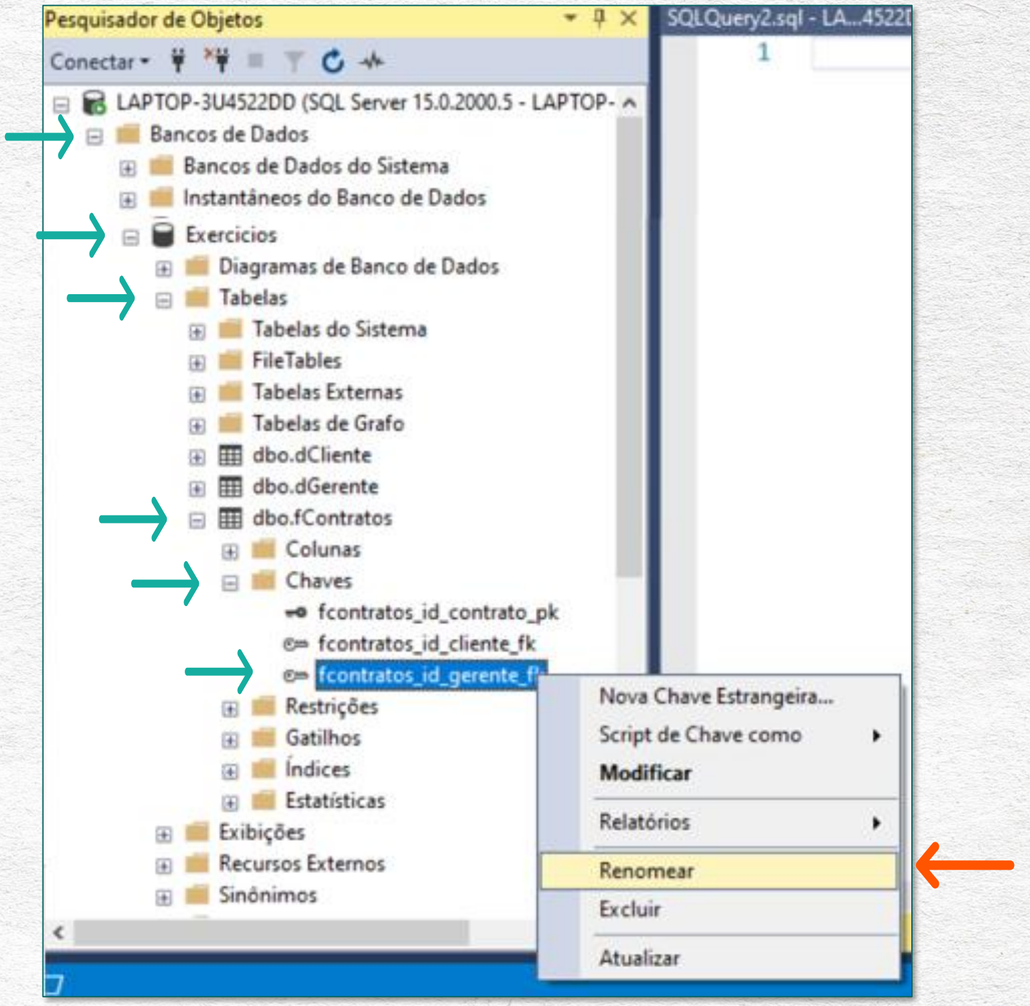
```
-- Adicione a constraint PK id_contrato na tabela fContratos.
ALTER TABLE fContratos
ADD CONSTRAINT fcontratos_id_contrato_pk PRIMARY KEY(id_contrato)
```

```
-- Adicione a constraint FK id_cliente na tabela fContratos.
ALTER TABLE fContratos
ADD CONSTRAINT fcontratos_id_cliente_fk FOREIGN KEY(id_cliente) REFERENCES dCliente(id_cliente)
```

Gerenciando Constraints: RENOMEAR

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para RENOMEAR uma CONSTRAINT de uma tabela, devemos clicar com o botão direito sobre o nome da constraint no menu lateral esquerdo da tela (Bancos de Dados > Nome_do_Banco_de_Dados > Tabelas > Nome_da_Tabela > Chaves > Nome_da_Constraint), após em Renomear:



Gerenciando Constraints: REMOVER

Vejamos agora como fazemos para adicionar, renomear e remover CONSTRAINTS.

Para REMOVER uma CONSTRAINT de uma tabela, utilizamos a seguinte sintaxe:

```
ALTER TABLE   nome_da_tabela
DROP CONSTRAINT   nome_da_constraint;
```

Exemplos:

```
-- Remova a constraint PK da tabela fContratos.
ALTER TABLE fContratos
DROP CONSTRAINT fcontratos_id_contrato_pk
```

```
-- Remova a constraint FK id_cliente da tabela fContratos.
ALTER TABLE fContratos
DROP CONSTRAINT fcontratos_id_cliente_fk
```

EXERCÍCIOS

1.a

Questão 1.a

Você está responsável por criar um Banco de Dados com algumas tabelas que vão armazenar informações associadas ao aluguel de carros de uma locadora.

a) O primeiro passo é criar um banco de dados chamado **AlugaFacil**.

1.b

Questão 1.b – Tabela 1

b) O seu banco de dados deve conter 3 tabelas e a descrição de cada uma delas é mostrada abaixo:

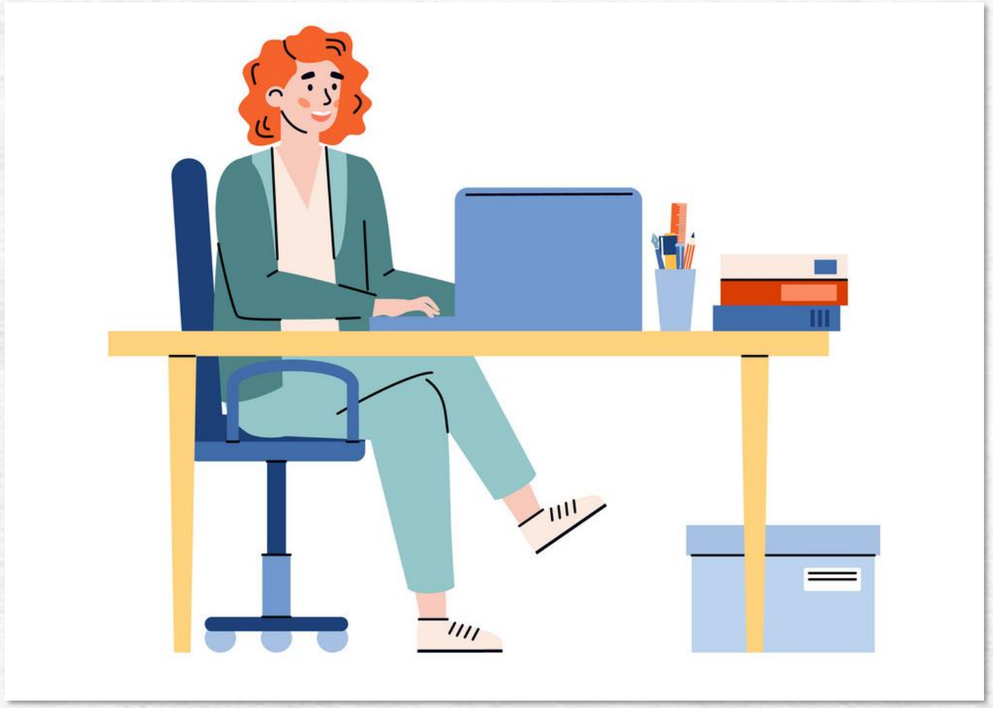
Obs.: você identificará as restrições das tabelas a partir de suas descrições.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

- A tabela Cliente possui 4 colunas.
- A coluna id_cliente deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- As colunas nome_cliente, cnh e cartao não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.
- Por fim, a coluna cnh não pode aceitar valores duplicados.

EXERCÍCIOS



1.b

Questão 1.b – Tabela 2

Tabela 2: Carro

- id _carro
- placa
- modelo
- tipo

- A tabela Carro possui 3 colunas.
- A coluna id_carro deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- As colunas modelo, tipo e placa não podem aceitar valores nulos.
- Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.
- Por fim, a coluna placa não pode aceitar valores duplicados.

EXERCÍCIOS

1.b

Questão 1.b – Tabela 3

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

- A tabela Locacoes possui 5 colunas.
- A coluna id_locacao deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- Nenhuma das demais colunas devem aceitar valores nulos.
- As colunas id_carro e id_cliente são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.

2

Questão 2

Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs.: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.

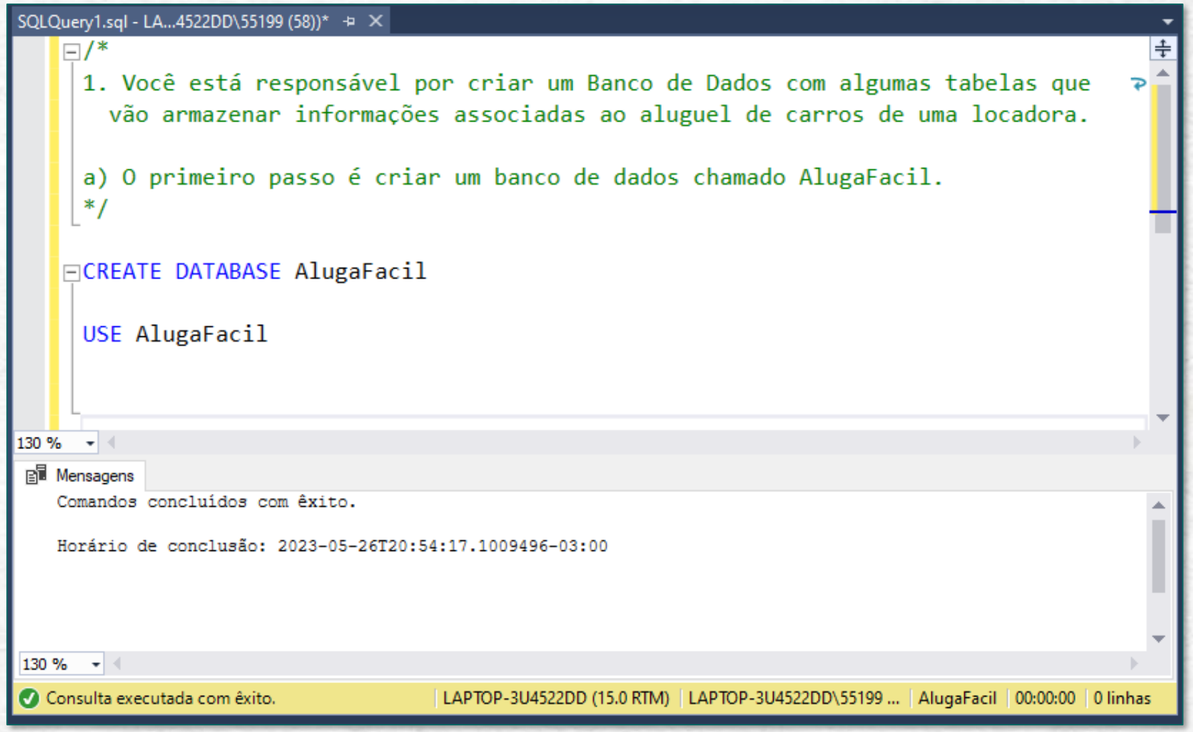
GABARITOS

1.a

Questão 1.a

Você está responsável por criar um Banco de Dados com algumas tabelas que vão armazenar informações associadas ao aluguel de carros de uma locadora.

a) O primeiro passo é criar um banco de dados chamado **AlugaFacil**.



GABARITOS

1.b

Questão 1.b – Tabela 1

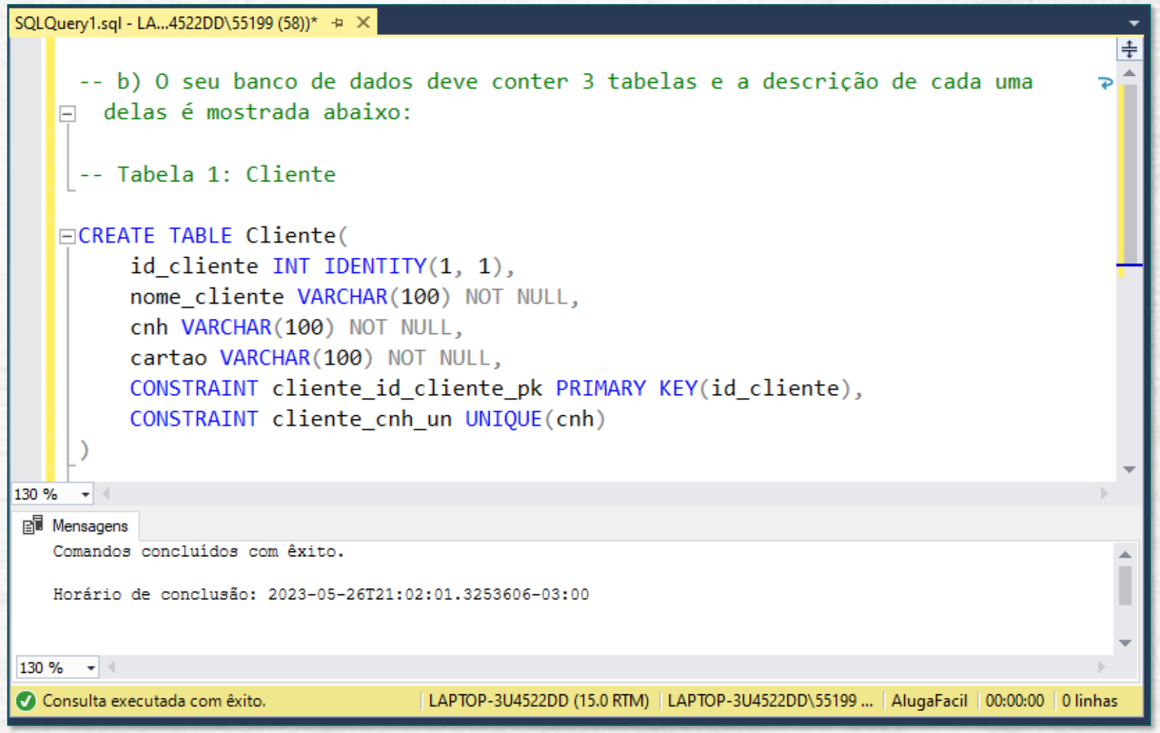
b) O seu banco de dados deve conter 3 tabelas e a descrição de cada uma delas é mostrada abaixo:

Obs.: você identificará as restrições das tabelas a partir de suas descrições.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

- A tabela Cliente possui 4 colunas.
- A coluna id_cliente deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- As colunas nome_cliente, cnh e cartao não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.
- Por fim, a coluna cnh não pode aceitar valores duplicados.



GABARITOS

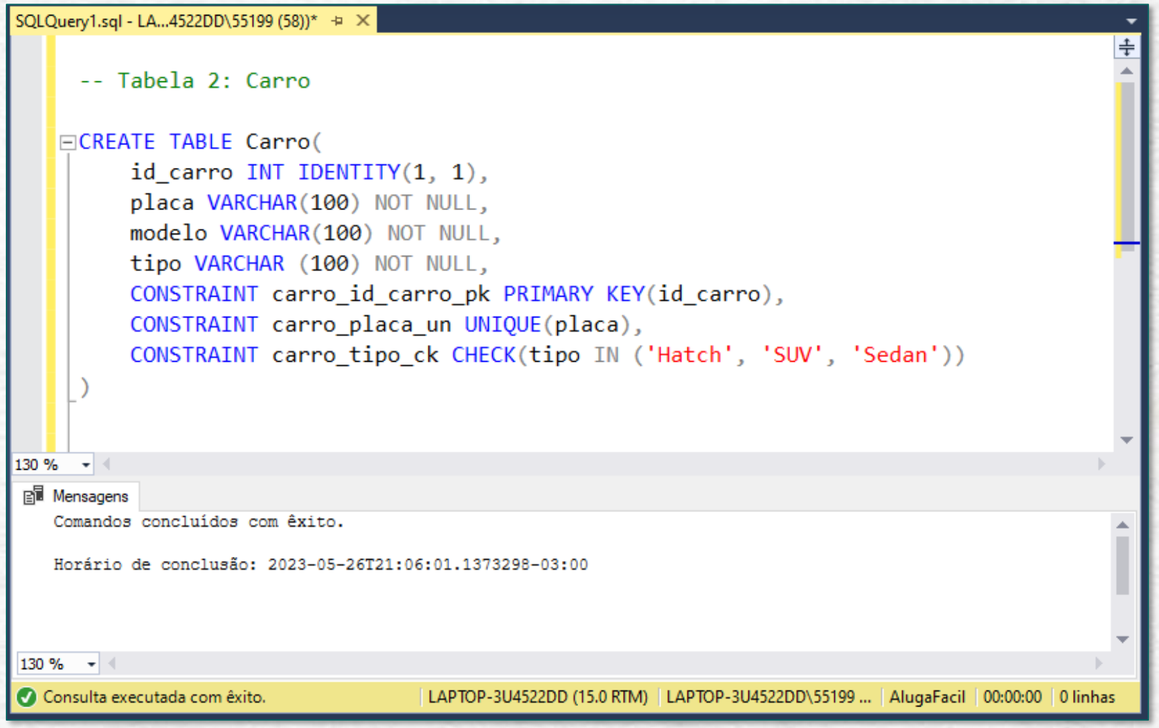
1.b

Questão 1.b – Tabela 2

Tabela 2: Carro

- id _carro
- placa
- modelo
- tipo

- A tabela Carro possui 3 colunas.
- A coluna id_carro deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- As colunas modelo, tipo e placa não podem aceitar valores nulos.
- Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.
- Por fim, a coluna placa não pode aceitar valores duplicados.



GABARITOS

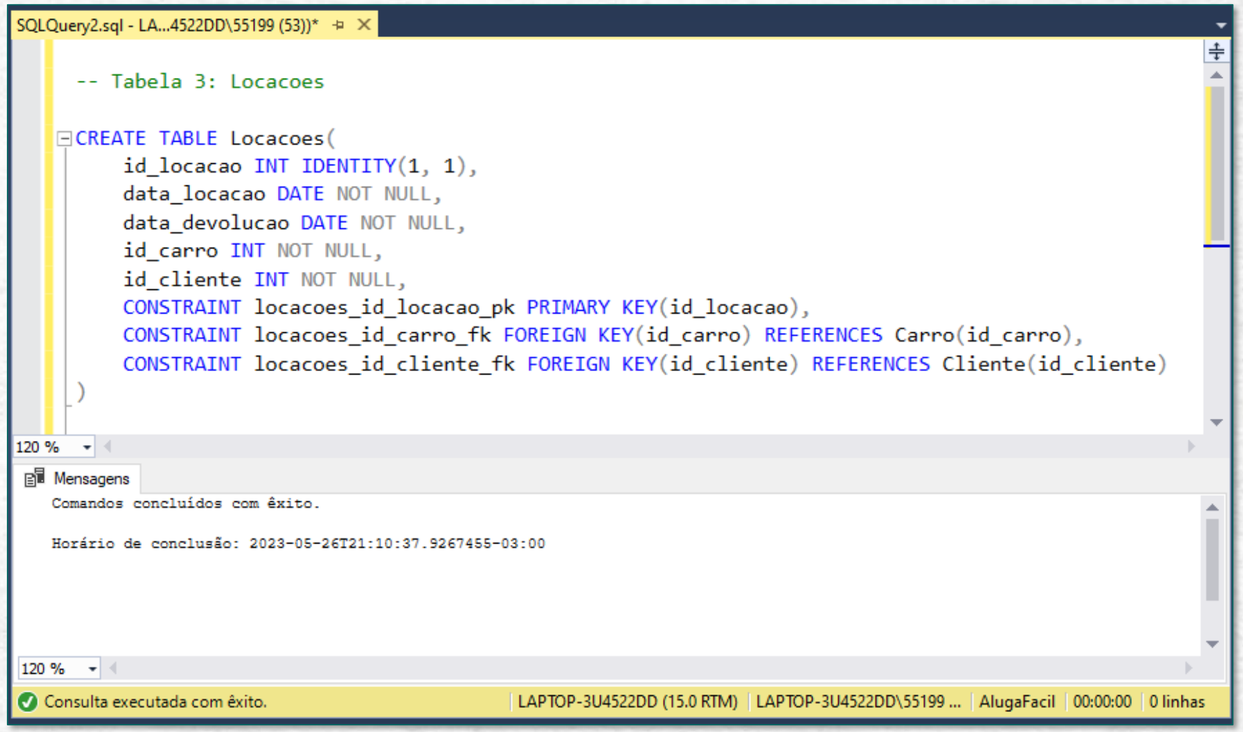
1.b

Questão 1.b – Tabela 3

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

- A tabela Locacoes possui 5 colunas.
- A coluna id_locacao deve ser a chave primária da tabela, além de ser autoincrementada de forma automática.
- Nenhuma das demais colunas devem aceitar valores nulos.
- As colunas id_carro e id_cliente são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.



GABARITOS

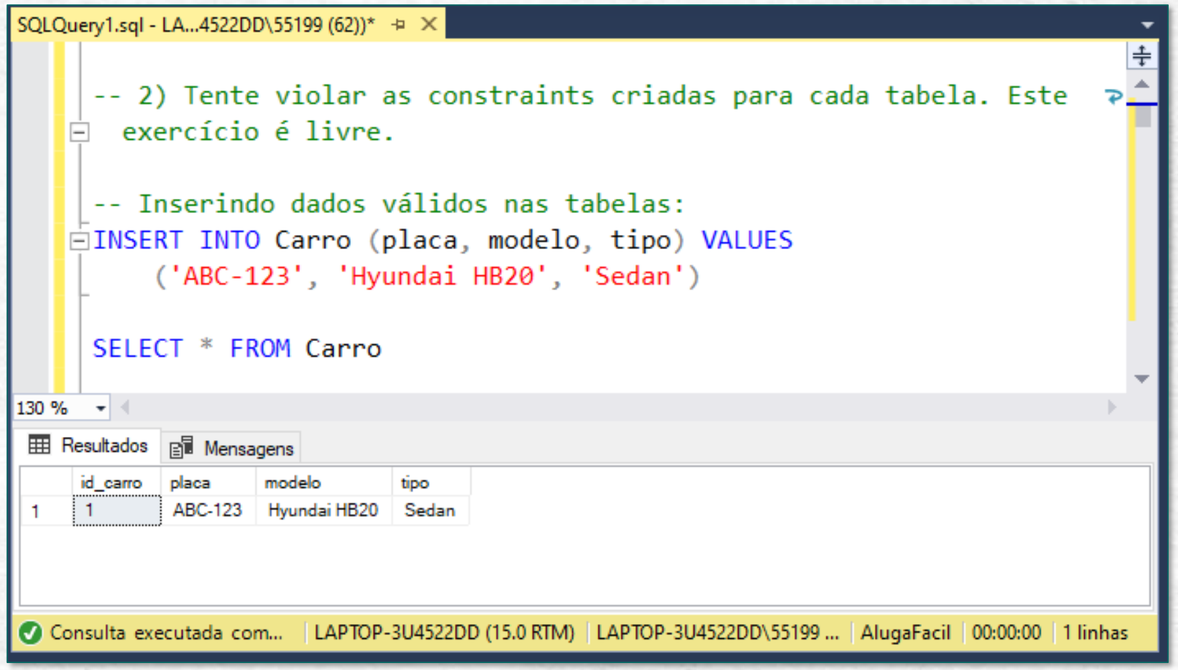
2

Questão 2

Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs.: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.



GABARITOS

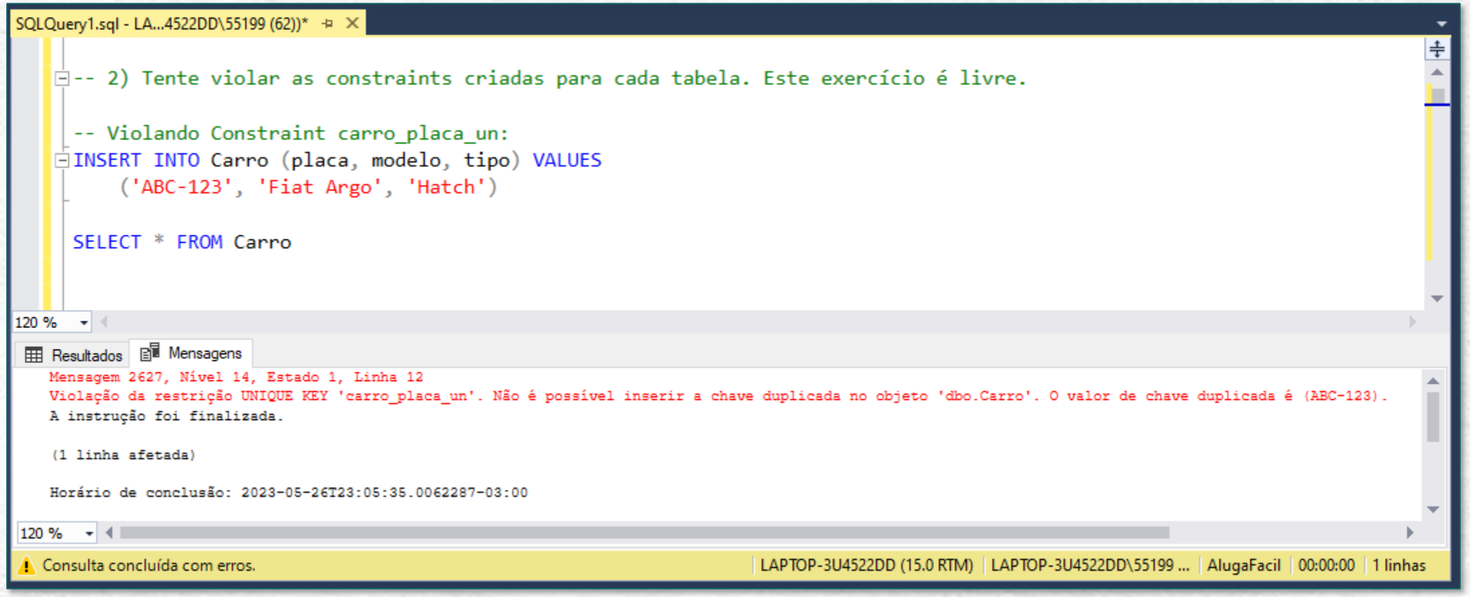
2

Questão 2

Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs.: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.



GABARITOS

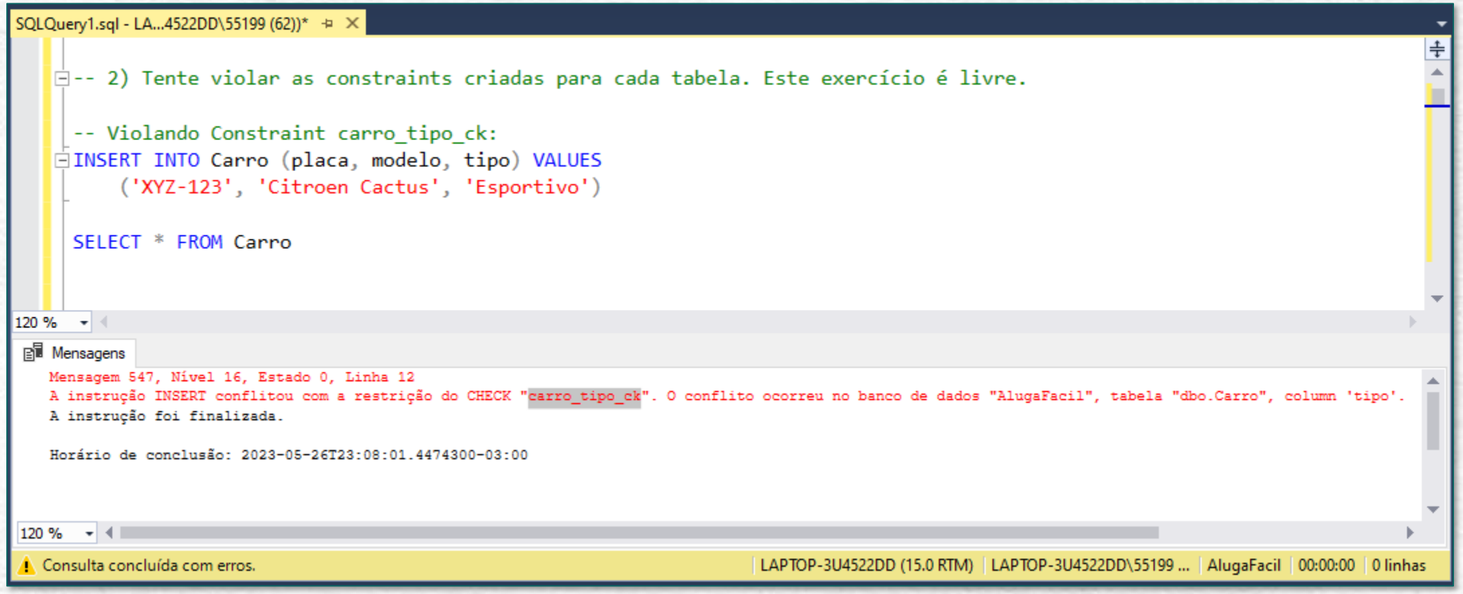
2

Questão 2

Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs.: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.



GABARITOS

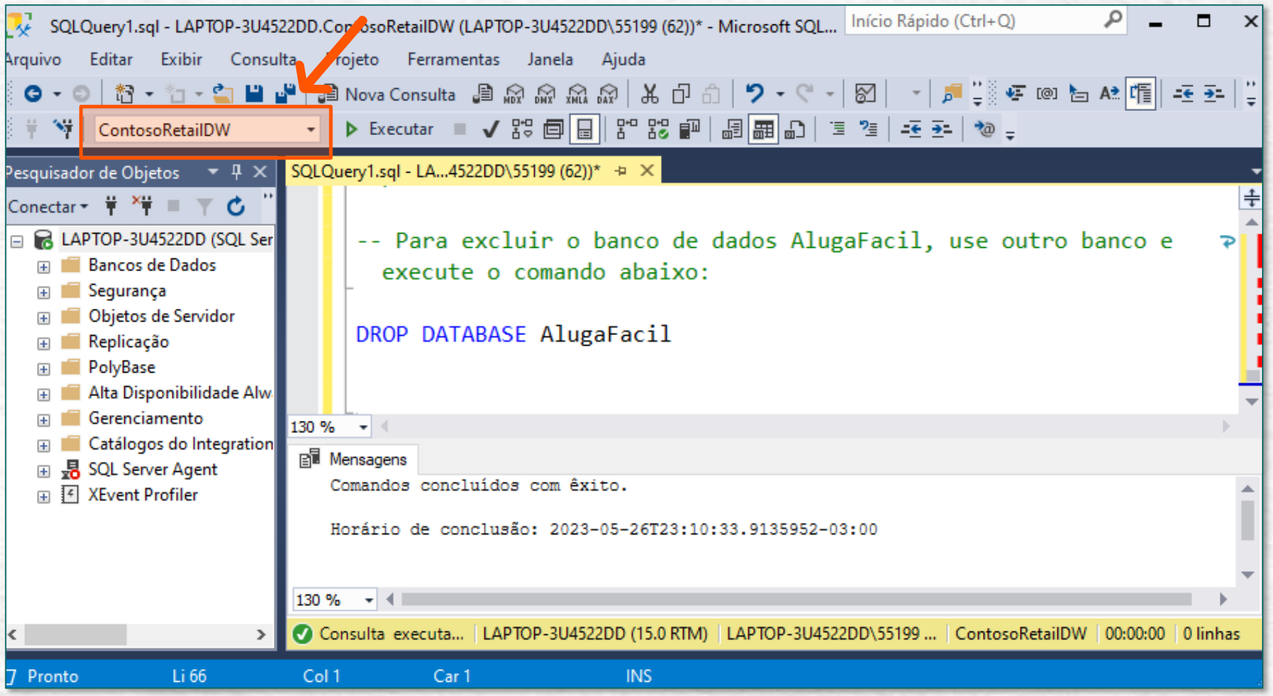
2

Questão 2

Tente violar as constraints criadas para cada tabela. Este exercício é livre.

Obs.: Para fazer o exercício de violação de constraints basta utilizar o comando INSERT INTO para adicionar valores nas tabelas que não respeitem as restrições (constraints) estabelecidas na criação das tabelas.

Ao final, exclua o banco de dados criado.



MÓDULO 19

SQL

Sequences

O QUE VEREMOS NESTE MÓDULO

MÓDULO 19 SEQUENCES

01	O que são SEQUENCES?	522
02	Como criar SEQUENCES	524
03	Descobrindo o próximo valor da SEQUENCE	525
04	Como excluir SEQUENCES	526
05	Como utilizar uma SEQUENCE na prática	527
06	EXERCÍCIOS	532
07	GABARITOS	536

O que são SEQUENCES?

Uma sequência (SEQUENCE) é um objeto que utilizamos para criação de números sequenciais automáticos.

São usadas especialmente para gerar valores sequenciais únicos para as chaves primárias das tabelas.

Dessa forma, não precisamos ficar preenchendo a sequência de IDs manualmente (como fizemos até então). Podemos gerá-los automaticamente por meio de uma SEQUENCE.

Nas próximas páginas, vamos entender sua sintaxe e como utilizá-la.

O que são SEQUENCES?

SINTAXE:

CREATE SEQUENCE nome_sequencia	Criar sequência
AS tipo	Informar tipo de dados a ser utilizado (INT, DECIMAL, etc.)
START WITH n	Começar a sequência a partir de qual número
INCREMENT BY n	Incrementar de quantos em quantos números (de 1 em 1, de 10 em 10, etc.)
MAXVALUE n | **NO MAXVALUE**	A sequência terá valor máximo ou não
MINVALUE n | **NO MINVALUE**	A sequência terá valor mínimo ou não
CYCLE | **NO CYCLE**;	Quando atingir o valor máximo, deve voltar ao começo ou não

Como criar SEQUENCES

EXEMPLO:

Vamos supor que queremos criar uma sequência para utilizar na tabela dCliente. Para isso, utilizamos um código que:

```
CREATE SEQUENCE clientes_seq
AS INT
START WITH 1
INCREMENT BY 1
MAXVALUE 9999999
NO CYCLE
```

Criará uma sequência que será chamada de clientes_seq;

Será do tipo INT;

Começará pelo número 1;

Será incrementada de 1 em 1;

Terá valor máximo de 9999999;

E não voltará ao início.

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-09-30T19:42:19.2696280-03:00
```

Descobrindo o próximo valor da SEQUENCE

Para descobrir qual é o próximo valor da sequência, utilize SELECT NEXT VALUE FOR:

```
SELECT NEXT VALUE FOR clientes_seq
```

Cada vez que você executar o código acima, ele retornará o próximo valor da sequência:

	(Nenhum nome de coluna)
1	1

	(Nenhum nome de coluna)
1	2

	(Nenhum nome de coluna)
1	3

	(Nenhum nome de coluna)
1	4

Como excluir SEQUENCES

Por fim, para excluir uma SEQUENCE, utilize o comando DROP SEQUENCE:

```
DROP SEQUENCE clientes_seq
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-09-30T19:51:21.6968762-03:00
```

Como utilizar uma SEQUENCE na prática

Vejamos agora na prática como utilizar uma sequence.

Vamos supor que a gente queira criar uma sequência para uma tabela chamada dProjeto, que criaremos logo mais. Para tanto, vamos utilizar o seguinte código, estabelecendo que a nossa sequence:

```
CREATE SEQUENCE projetos_seq
AS INT
START WITH 1
INCREMENT BY 1
NO MAXVALUE
NO CYCLE
```

Será chamada de projetos_seq;

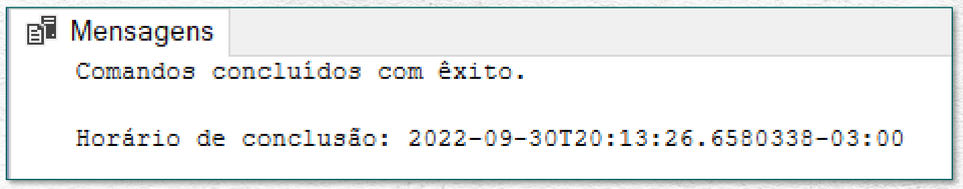
Será do tipo INT;

Começará pelo número 1;

Será incrementada de 1 em 1;

Não terá valor máximo;

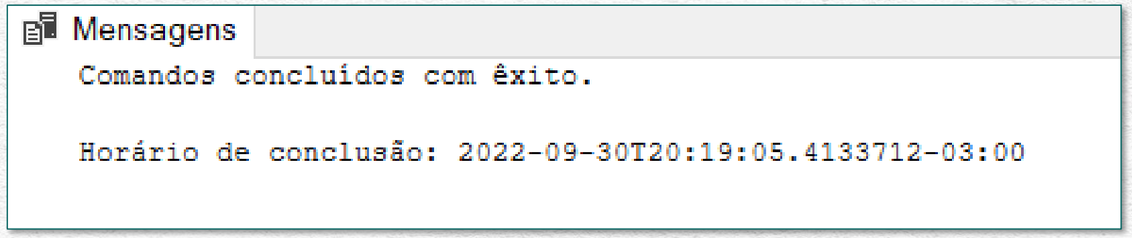
Não voltará ao início.



Como utilizar uma SEQUENCE na prática

Criada a sequence projetos_seq, vamos agora criar a tabela dProjeto que a utilizará:

```
CREATE TABLE dProjeto(
    id_projeto INT,
    nome_projeto VARCHAR(100) NOT NULL,
    CONSTRAINT dareas_id_area_pk PRIMARY KEY(id_projeto)
)
```



Como utilizar uma SEQUENCE na prática

Por fim, vamos inserir os registros (linhas) na nova tabela criada.

Repare que para inserir os valores da coluna id_projeto, agora que temos uma sequence, não precisamos informar número de ID por número de ID, manualmente. Basta informarmos nas linhas a serem inseridas o comando NEXT VALUE FOR e o nome da sequence, conforme abaixo:

```
INSERT INTO dProjeto(id_projeto, nome_projeto) VALUES
    (NEXT VALUE FOR projetos_seq, 'Planejamento Estratégico'),
    (NEXT VALUE FOR projetos_seq, 'Desenvolvimento de App'),
    (NEXT VALUE FOR projetos_seq, 'Plano de Negócios'),
    (NEXT VALUE FOR projetos_seq, 'Visualização 3D')
```

Mensagens

```
(4 linhas afetadas)

Horário de conclusão: 2022-09-30T20:23:05.1512283-03:00
```

Como utilizar uma SEQUENCE na prática

Se consultarmos nossa tabela dProjeto agora, veremos que quatro registros foram inseridos, e a coluna id_projeto foi devidamente preenchida com os valores automaticamente gerados pela sequence projetos_seq:

```
SELECT * FROM dProjeto
```

110 %

Resultados | Mensagens

	id_projeto	nome_projeto
1	1	Planejamento Estratégico
2	2	Desenvolvimento de App
3	3	Plano de Negócios
4	4	Visualização 3D

Como utilizar uma SEQUENCE na prática

Repare que se continuarmos inserindo novas linhas, estas também seguirão a ordem da sequence utilizada.

A título de exemplo, vamos adicionar mais um registro:

```
INSERT INTO dProjeto(id_projeto, nome_projeto) VALUES
    (NEXT VALUE FOR projetos_seq, 'Mapeamento de Processos')
```

Mensagens

```
(1 linha afetada)

Horário de conclusão: 2022-09-30T20:31:01.5920240-03:00
```

Veja que o ID gerado automaticamente para a coluna id_projeto do novo registro seguiu normalmente a numeração da sequência:

Resultados | Mensagens

	id_projeto	nome_projeto
1	1	Planejamento Estratégico
2	2	Desenvolvimento de App
3	3	Plano de Negócios
4	4	Visualização 3D
5	5	Mapeamento de Processos

EXERCÍCIOS

0 Questão 0

Crie o Banco de Dados AlugaFacil, onde serão criadas as sequences e tabelas dos exercícios.

1 Questão 1

Vamos criar Sequences que serão utilizadas nas tabelas: Carro, Cliente e Locacoes.

Essas sequences serão chamadas de: cliente_seq, carro_seq e locaçoes_seq.

Todas essas sequences devem começar pelo número 1, incrementar de 1 em 1 e não ter valor máximo.

EXERCÍCIOS

2

Questão 2 – Tabela Cliente

Utilize as sequences nas 3 tabelas: Carro, Cliente e Locacoes.

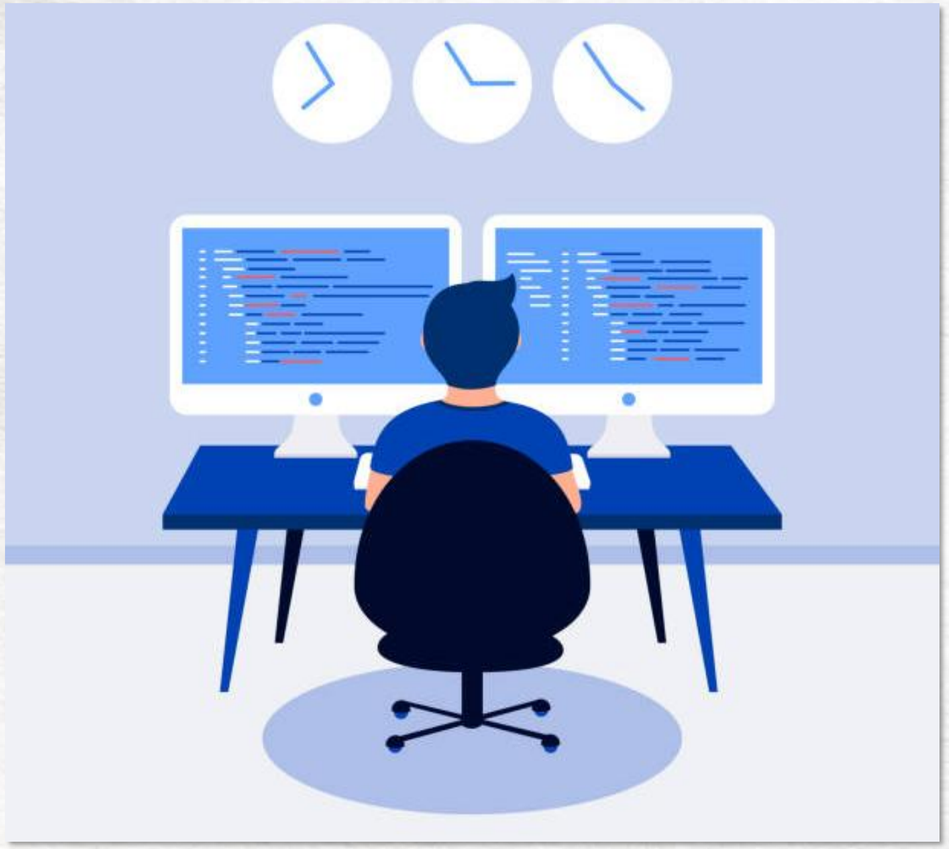
Você deve excluir as tabelas existentes e recriá-las. Lembre-se que não é necessário utilizar a constraint IDENTITY nas colunas de chave primária, uma vez que nelas serão usadas as Sequences.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

- A tabela Cliente possui 4 colunas.
- A coluna **id_cliente** deve ser a chave primária da tabela.
- As colunas **nome_cliente**, **cnh** e **cartao** não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.
- Por fim, a coluna **cnh** não pode aceitar valores duplicados.

EXERCÍCIOS



2

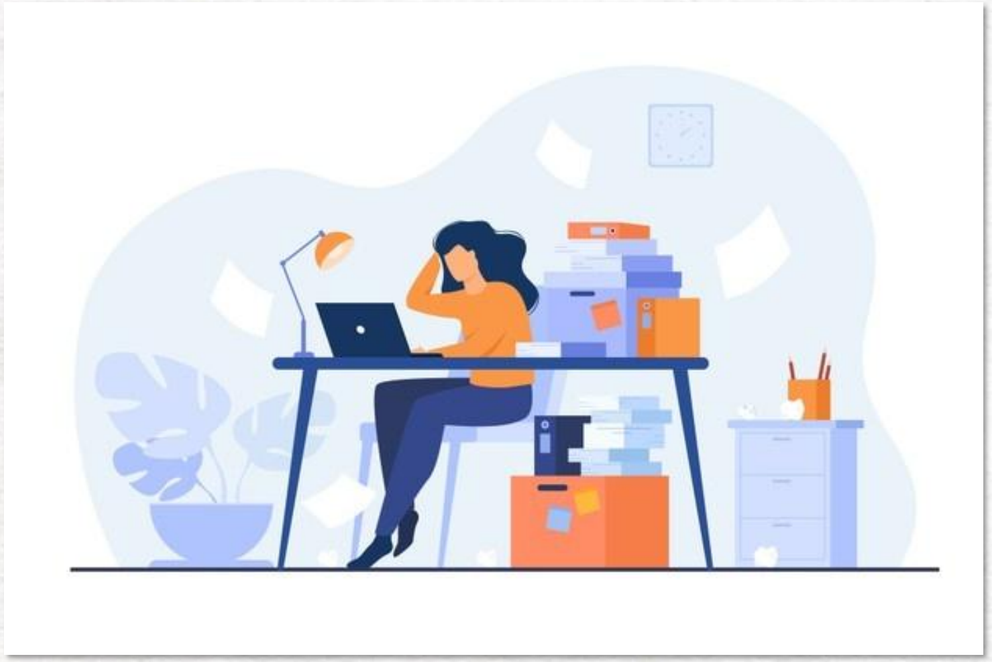
Questão 2 – Tabela Carro

Tabela 2: Carro

- id _carro
- placa
- modelo
- tipo

- A tabela Carro possui 3 colunas.
- A coluna **id_carro** deve ser a chave primária da tabela.
- As colunas **modelo**, **tipo** e **placa** não podem aceitar valores nulos.
- Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.
- Por fim, a coluna **placa** não pode aceitar valores duplicados.

EXERCÍCIOS



2 Questão 2 – Tabela Locacoes

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

- A tabela Locacoes possui 5 colunas.
- A coluna **id_locacao** deve ser a chave primária da tabela.
- Nenhuma das demais colunas devem aceitar valores nulos.
- As colunas **id_carro** e **id_cliente** são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.

3 Questão 3

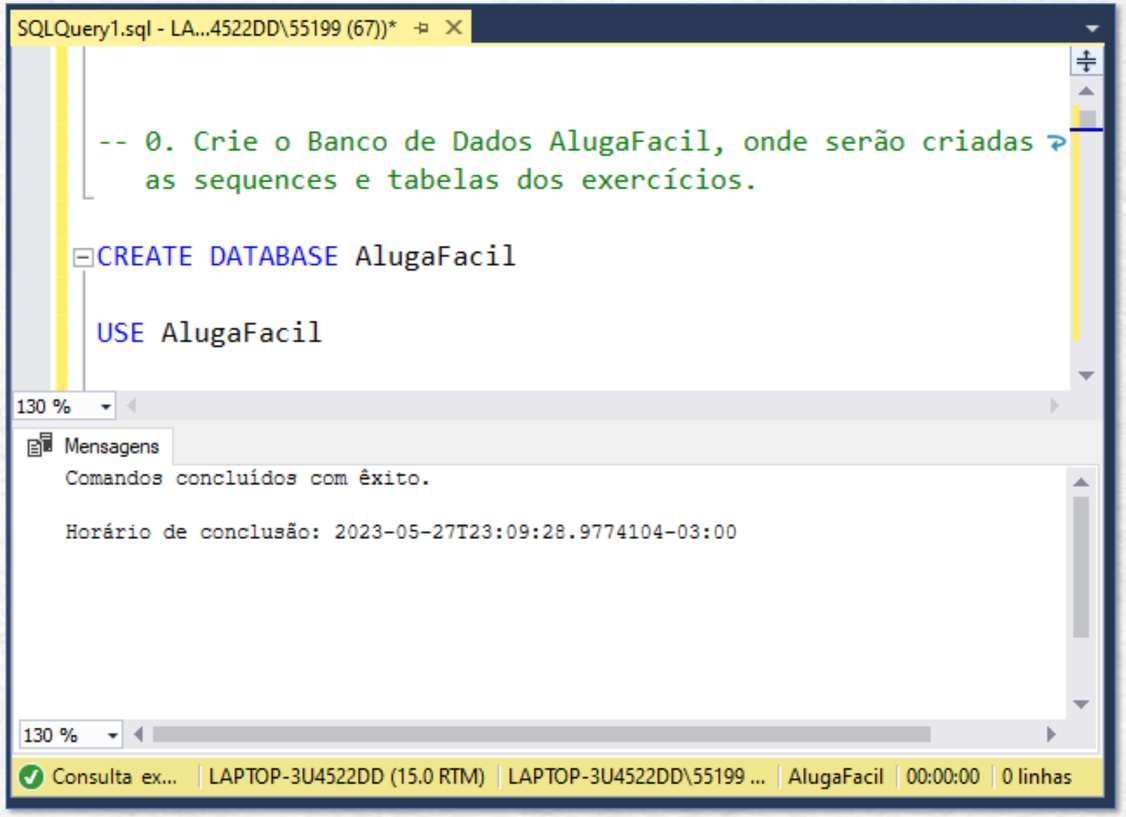
Exclua as sequences criadas.

GABARITOS

0

Questão 0

Crie o Banco de Dados AlugaFacil, onde serão criadas as sequences e tabelas dos exercícios.



GABARITOS

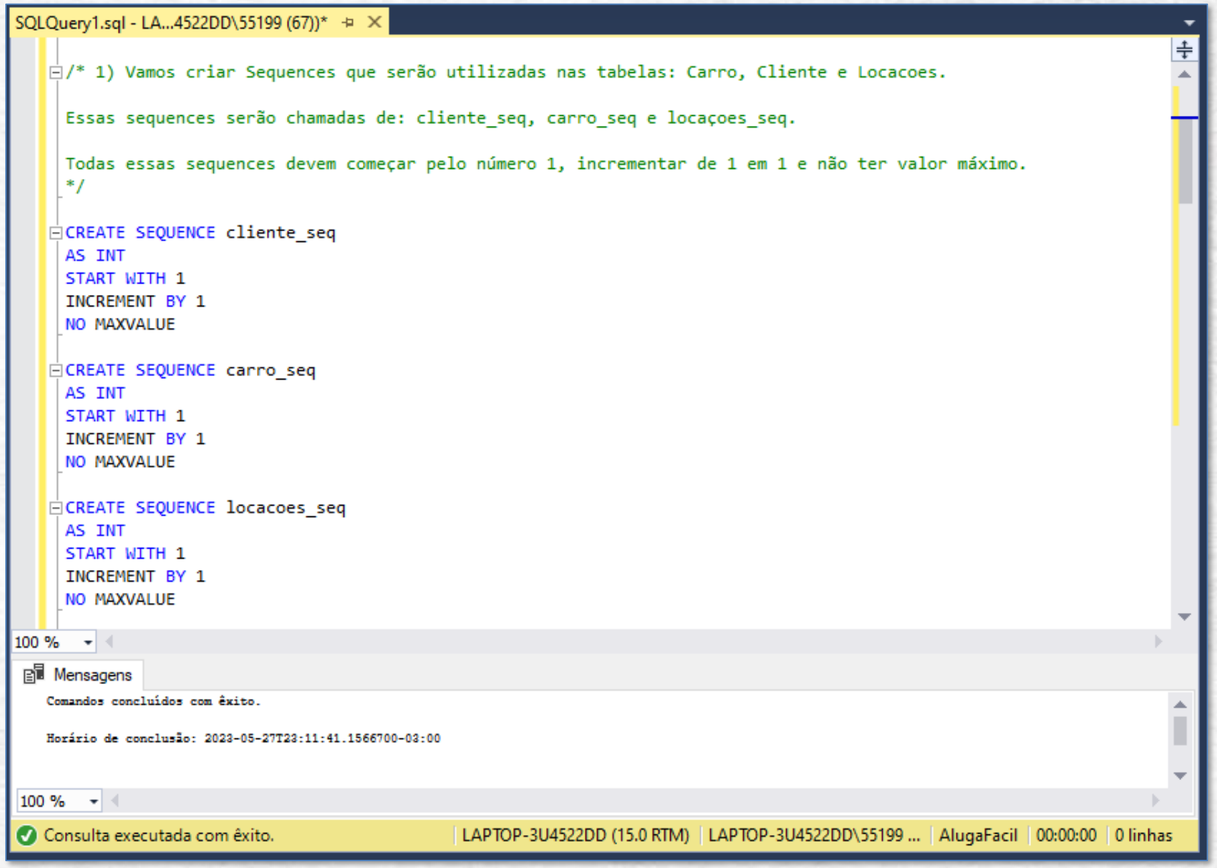
1

Questão 1

Vamos criar Sequences que serão utilizadas nas tabelas: Carro, Cliente e Locacoes.

Essas sequences serão chamadas de: cliente_seq, carro_seq e locaçoes_seq.

Todas essas sequences devem começar pelo número 1, incrementar de 1 em 1 e não ter valor máximo.



GABARITOS

2

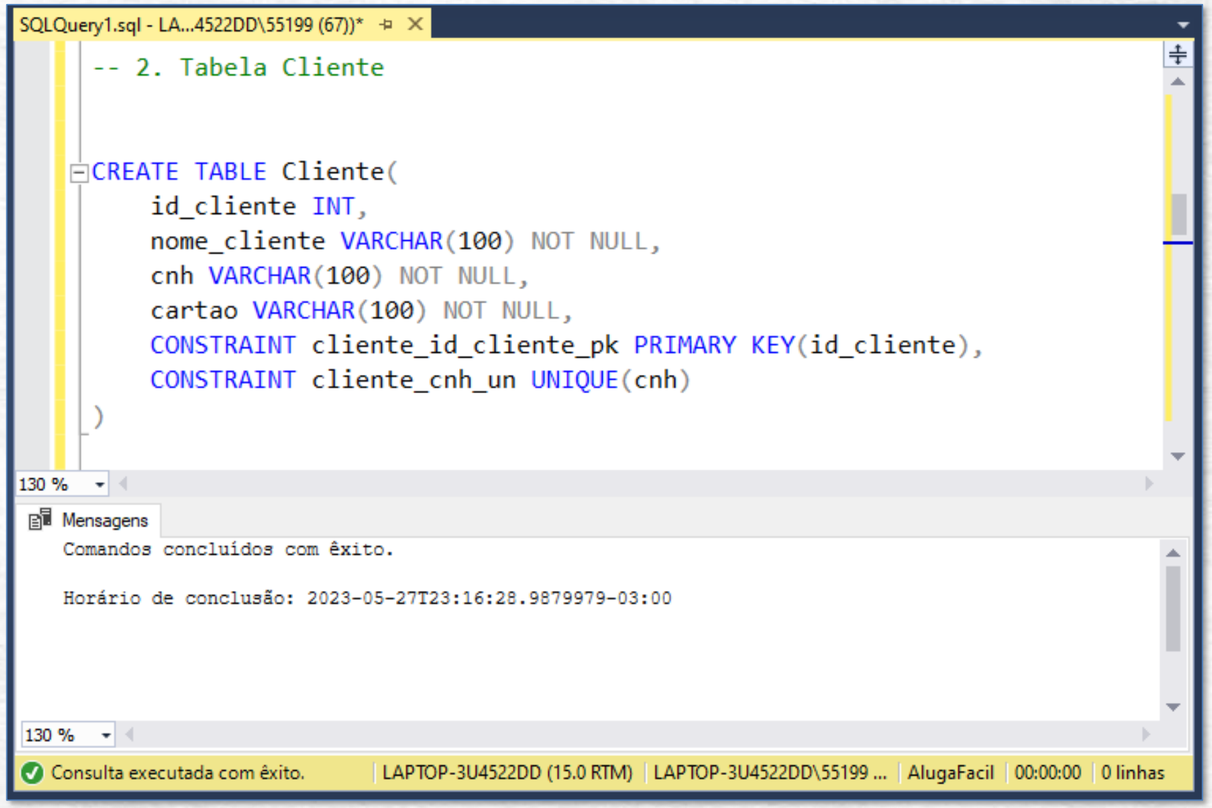
Questão 2 – Tabela Cliente

Você deve excluir as tabelas existentes e recriá-las. Lembre-se que não é necessário utilizar a constraint IDENTITY nas colunas de chave primária, uma vez que nelas serão usadas as Sequences.

Tabela 1: Cliente

- id_cliente
- nome_cliente
- cnh
- cartao

- A tabela Cliente possui 4 colunas.
- A coluna **id_cliente** deve ser a chave primária da tabela.
- As colunas **nome_cliente**, **cnh** e **cartao** não podem aceitar valores nulos, ou seja, para todo cliente estes campos devem necessariamente ser preenchidos.
- Por fim, a coluna **cnh** não pode aceitar valores duplicados.



GABARITOS

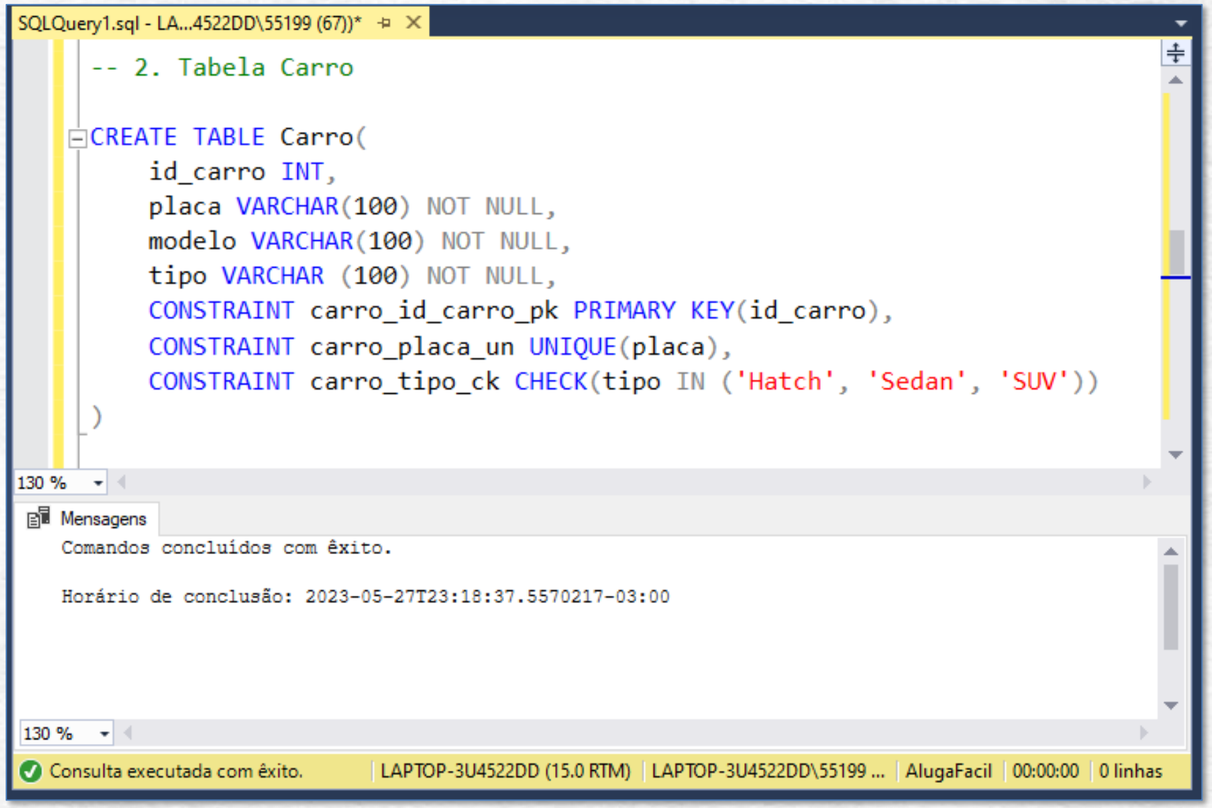
2

Questão 2 – Tabela Carro

Tabela 2: Carro

- id _carro
- placa
- modelo
- tipo

- A tabela Carro possui 3 colunas.
- A coluna **id_carro** deve ser a chave primária da tabela.
- As colunas **modelo**, **tipo** e **placa** não podem aceitar valores nulos.
- Os tipos de carros cadastrados devem ser: Hatch, Sedan, SUV.
- Por fim, a coluna **placa** não pode aceitar valores duplicados.



GABARITOS

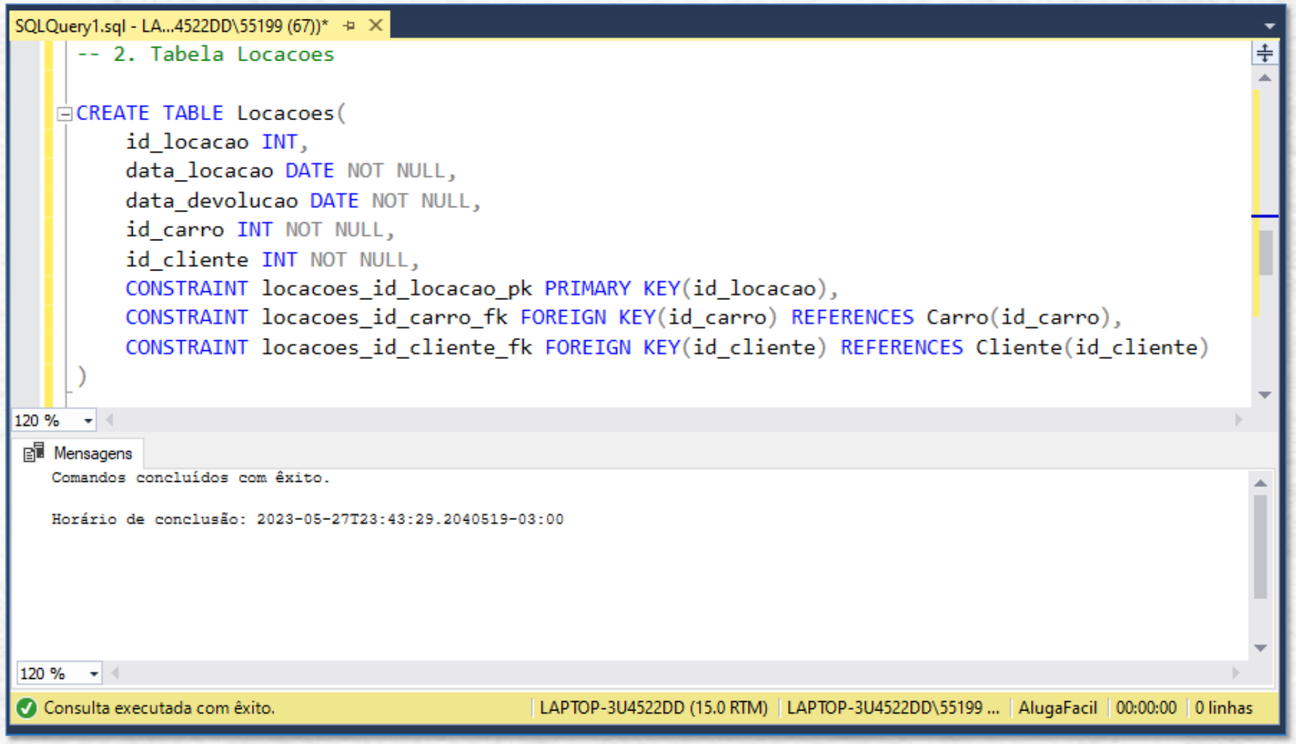
2

Questão 2 – Tabela Locacoes

Tabela 3: Locacoes

- id_locacao
- data_locacao
- data_devolucao
- id_carro
- id_cliente

- A tabela Locacoes possui 5 colunas.
- A coluna **id_locacao** deve ser a chave primária da tabela.
- Nenhuma das demais colunas devem aceitar valores nulos.
- As colunas **id_carro** e **id_cliente** são chaves estrangeiras que permitirão a relação da tabela Locacoes com as tabelas Carro e Cliente.

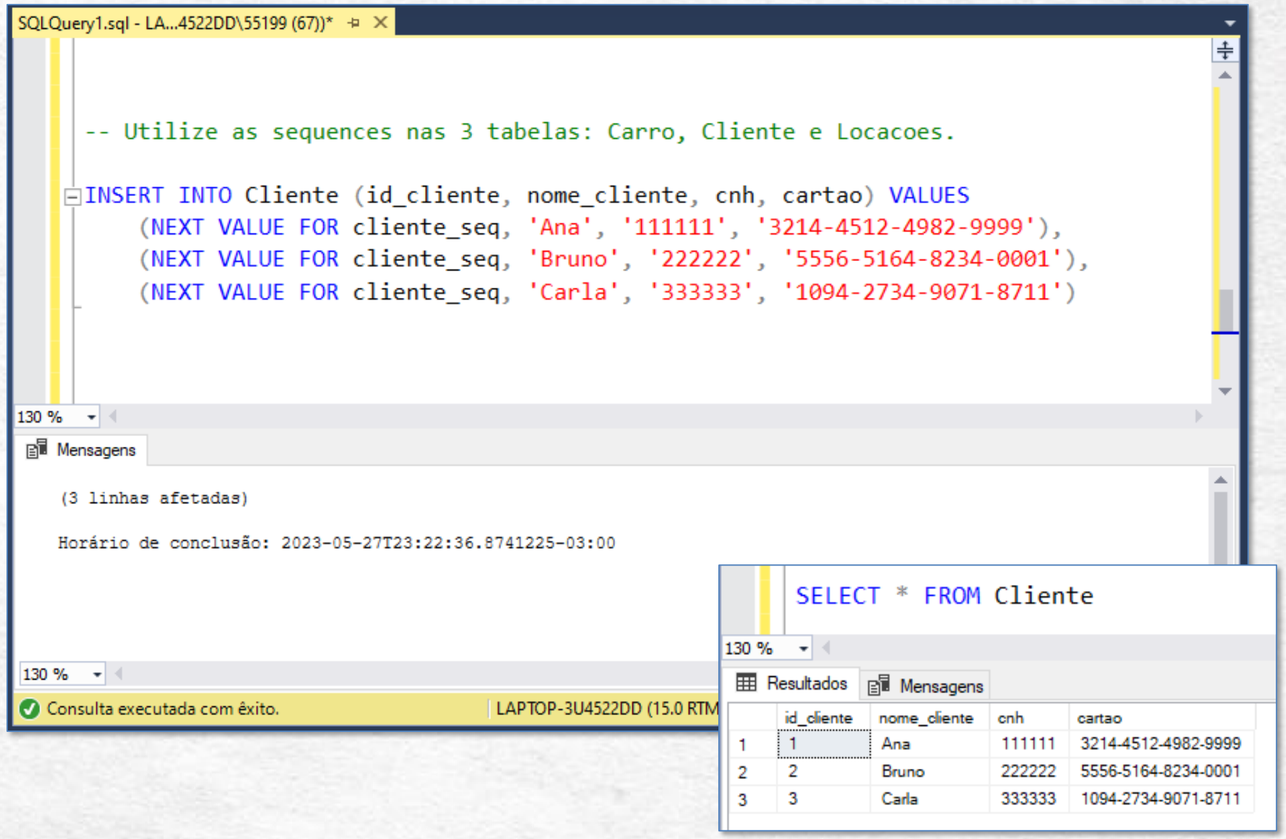


GABARITOS

2

Questão 2

Utilize as sequences nas 3 tabelas: Carro, Cliente e Locacoes.

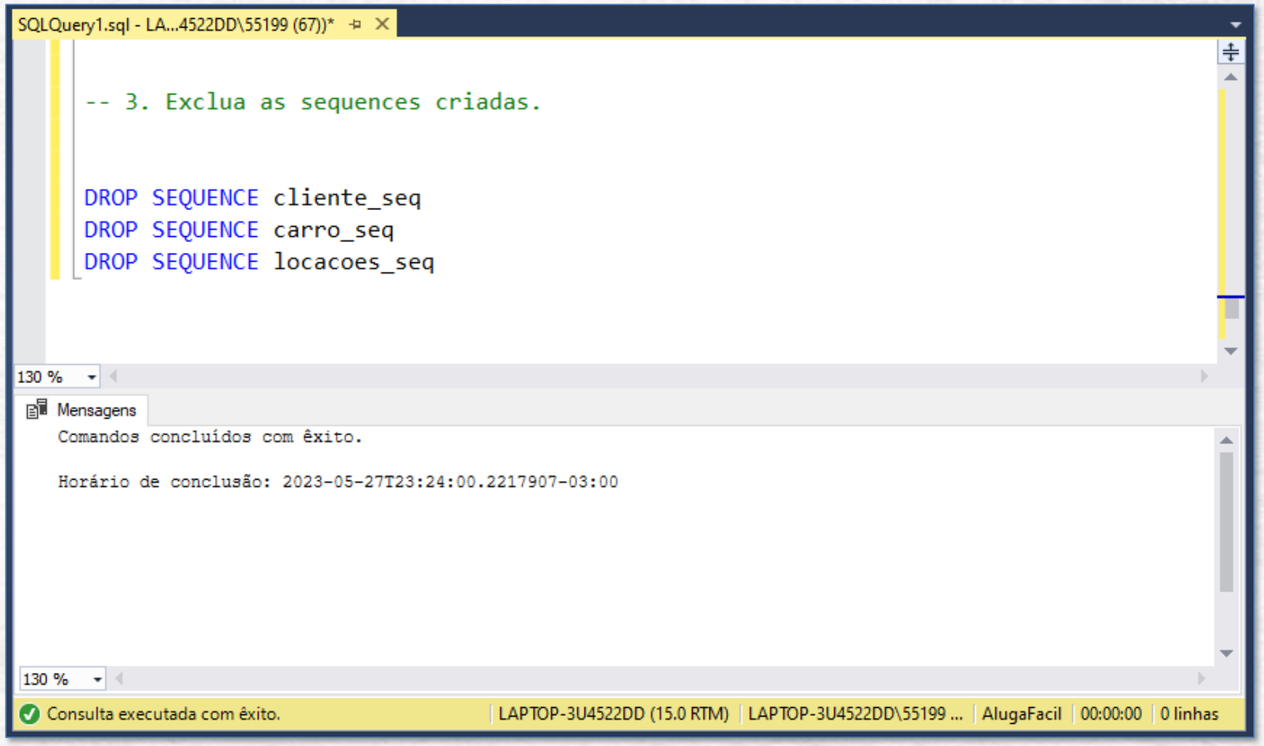


GABARITOS

3

Questão 3

Exclua as sequences criadas.



MÓDULO 20

SQL Transactions

O QUE VEREMOS NESTE MÓDULO

MÓDULO 20 TRANSACTIONS

01	O que são TRANSACTIONS?	545
02	BEGIN TRANSACTION, COMMIT e ROLLBACK	548
03	Criando TRANSACTIONS nomeadas	559
04	COMMIT e ROLLBACK condicionais	560
05	Tratamento de erros (TRY e CATCH)	568
06	Variável @@TRANCOUNT	577
07	Transações Aninhadas	581
08	EXERCÍCIOS	585
09	GABARITOS	586

O que são TRANSACTIONS?

Uma TRANSACTION é uma ação realizada dentro do banco de dados. Essa ação pode ser uma inserção (INSERT INTO), uma atualização (UPDATE), ou uma exclusão (DELETE) de dados do banco. Precisamos de transações quando estamos alterando o banco de dados de alguma forma, seja inserindo, atualizando ou excluindo dados.

Normalmente, não temos muito "controle" sobre transações, a menos que a gente **explicite** no nosso código que queremos fazer isso.

Assim, a ideia de uma transação é agrupar um conjunto de instruções a serem executadas no banco de dados, e ter a flexibilidade de:

A. Se algo der errado, desfazer essa transação;
B. Se tudo der certo, salvar essa transação.

O que são TRANSACTIONS?

Vimos até agora como inserir, alterar e excluir dados no banco de dados com comandos como estes aqui:

```sql
-- 1. Adicionamos novas linhas (registros) na tabela
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Maria Julia', 'F', '30/04/1995', '987.654.321-00')


-- 2. Atualizamos linhas (registros) da tabela
UPDATE cliente_aux
    SET nome_cliente = LOWER(nome_cliente)


-- 3. Excluimos linhas (registros) da tabela
DELETE FROM dCliente
    WHERE id_cliente = 1
```

Quando executamos as ações acima desta forma, não temos controle sobre desfazê-las ou salvá-las no banco, pois as transações efetuadas estão implícitas (não declaradas). Mas, quando usamos transações explícitas, conseguimos controlar melhor essas ações.

O que são TRANSACTIONS?

E este controle é possível por meio dos comandos:

BEGIN TRANSACTION: com ele informamos ao sistema que estamos iniciando uma transação;

ROLLBACK TRANSACTION: comando TCL que usamos para desfazer uma transação no banco;

COMMIT (TRANSACTION): comando TCL por meio do qual salvamos uma transação no banco.

BEGIN TRANSACTION, COMMIT e ROLLBACK

Para exemplificarmos como utilizamos os comandos apresentados na página anterior, inicialmente, vamos criar uma nova tabela em nosso banco de dados, chamada cliente_aux que conterá os mesmos dados da tabela dCliente:

```
SELECT *
INTO cliente_aux
FROM dCliente
```

Assim, poderemos modificar à vontade nossa nova tabelinha sem “bagunçar” a tabela original!

Vejamos como ficou:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20

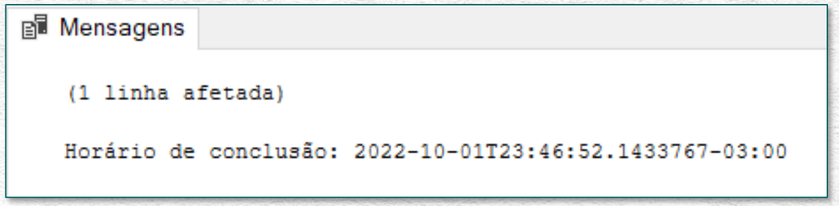
BEGIN TRANSACTION, COMMIT e ROLLBACK

Vamos supor que queremos criar uma transaction para inserir um novo registro na tabela cliente_aux.

Para explicitarmos ao banco de dados que queremos iniciar uma transação, devemos utilizar o comando BEGIN TRANSACTION (ou BEGIN TRAN, que também funciona da mesma forma).

Em seguida, acrescentamos o código do que queremos fazer:

```
BEGIN TRANSACTION
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Maria Julia', 'F', '30/04/1995', '987.654.321-00')
```



BEGIN TRANSACTION, COMMIT e ROLLBACK

Se consultarmos novamente a tabela cliente_aux, veremos que o registro foi inserido:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	17	Maria Julia	F	1995-04-30	987.654.321-00

BEGIN TRANSACTION, COMMIT e ROLLBACK

Porém, ele não está definitivamente salvo, pois o inserimos por meio de uma transação explícita que iniciamos com o BEGIN TRANSACTION.

Desta forma, temos a oportunidade de verificar se este registro está correto antes de executarmos a próxima ação. E agora, temos duas opções:

Verificando que a inserção dos dados foi efetuada de acordo com o esperado, podemos **salvá-la definitivamente** no banco de dados por meio de um COMMIT. Ou, percebendo que algo errado aconteceu, podemos **desfazer** esta inserção por meio de um ROLLBACK.

BEGIN TRANSACTION, COMMIT e ROLLBACK

Vamos supor que verificamos que algo não saiu conforme o esperado e desejamos desfazer esta inserção.

Para isso, devemos utilizar o comando ROLLBACK TRANSACTION:

```
ROLLBACK TRANSACTION
```

Mensagens

```
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-01T23:56:19.3067672-03:00
```

Consultando novamente a tabela cliente_aux, repare que o registro que havíamos inserido foi excluído:

```
SELECT * FROM cliente_aux
```

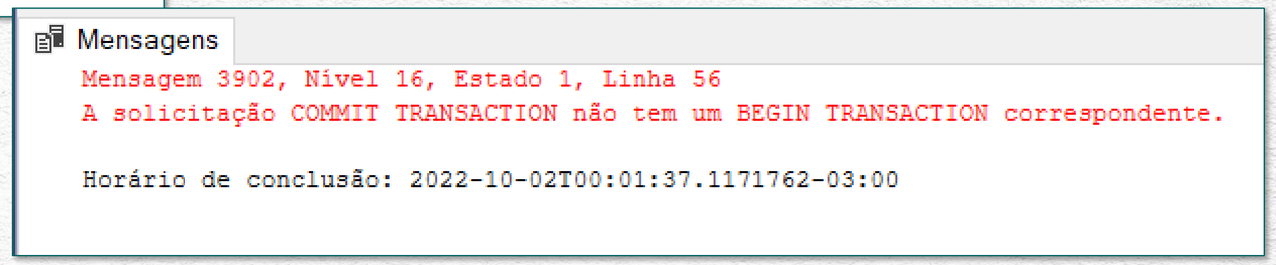
Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20

BEGIN TRANSACTION, COMMIT e ROLLBACK

Note que, uma vez que demos um ROLLBACK, não conseguimos efetuar um COMMIT para salvar essa mesma inserção de dados:

```
COMMIT TRANSACTION
```



Isto acontece porque, uma transação, uma vez iniciada, pode ser salva com um COMMIT ou desfeita com um ROLLBACK. Enquanto não for efetuada nenhuma das duas ações, a transação permanece em aberto.

Porém, após executado um COMMIT ou um ROLLBACK, essa transação é encerrada. Portanto, se, após executado um ROLLBACK, tentarmos efetuar um COMMIT, o sistema retornará o erro acima, pois a transação já havia sido encerrada, não sendo mais possível efetuar nenhuma ação em relação a ela.

O contrário também vale: após um COMMIT, a transação é encerrada, não sendo possível efetuar um ROLLBACK.

BEGIN TRANSACTION, COMMIT e ROLLBACK

Agora, vamos executar novamente o nosso BEGIN TRANSACTION para efetuar a inserção do mesmo registro. Assim, iniciaremos uma nova transação:

```
BEGIN TRANSACTION
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Maria Julia', 'F', '30/04/1995', '987.654.321-00')
```

Mensagens

```
(1 linha afetada)

Horário de conclusão: 2022-10-02T00:09:21.4310349-03:00
```

BEGIN TRANSACTION, COMMIT e ROLLBACK

Repare que o registro foi mais uma vez inserido na tabela cliente_aux:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	18	Maria Julia	F	1995-04-30	987.654.321-00

BEGIN TRANSACTION, COMMIT e ROLLBACK

Porém, como já abordado anteriormente, sabemos que ele não está definitivamente salvo, pois o inserimos por meio de uma transação explícita que iniciamos com o BEGIN TRANSACTION.

Desta forma, temos a oportunidade de verificar se este registro está correto antes de executarmos a próxima ação. E agora, temos novamente duas opções:

Verificando que a inserção dos dados foi efetuada de acordo com o esperado, podemos **salvá-la definitivamente** no banco de dados por meio de um COMMIT. Ou, percebendo que algo errado aconteceu, podemos **desfazer** esta inserção por meio de um ROLLBACK.

BEGIN TRANSACTION, COMMIT e ROLLBACK

No exemplo anterior, havíamos desfeito a inserção com um ROLLBACK.

Agora, vimos que tudo saiu conforme o esperado, e por isso queremos salvar essa inserção definitivamente na nossa tabela. Para isso, executamos um COMMIT:

```
COMMIT TRANSACTION
```

Mensagens

```
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-02T00:14:20.3045687-03:00
```

Consultando novamente a tabela cliente_aux, repare que o registro permanece lá, pois foi definitivamente salvo:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	18	Maria Julia	F	1995-04-30	987.654.321-00

BEGIN TRANSACTION, COMMIT e ROLLBACK

Note que, uma vez que demos um COMMIT, não conseguimos efetuar um ROLLBACK para excluir essa mesma inserção de dados:

```
ROLLBACK TRANSACTION
```

Mensagens

```
Mensagem 3903, Nível 16, Estado 1, Linha 59
A solicitação ROLLBACK TRANSACTION não tem um BEGIN TRANSACTION correspondente.

Horário de conclusão: 2022-10-02T00:20:07.6216095-03:00
```

Isto acontece porque, uma transação, uma vez iniciada, pode ser salva com um COMMIT ou desfeita com um ROLLBACK. Enquanto não for efetuada nenhuma das duas ações, a transação permanece em aberto.

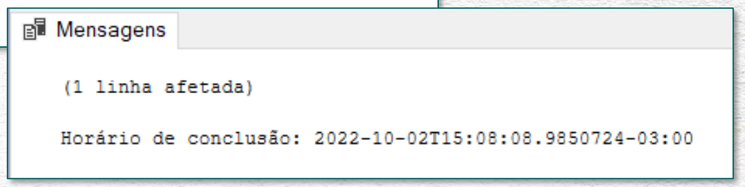
Porém, após executado um COMMIT ou um ROLLBACK, essa transação é encerrada. Portanto, se, após executado um COMMIT, tentarmos efetuar um ROLLBACK, o sistema retornará o erro acima, pois a transação já havia sido encerrada, não sendo mais possível efetuar nenhuma ação em relação a ela.

O contrário também vale: após um ROLLBACK, a transação é encerrada, não sendo possível efetuar um COMMIT.

Criando TRANSACTIONS nomeadas

Veremos agora que podemos criar transações nomeadas. O processo é o mesmo de se criar uma transação sem nome. Devemos apenas acrescentar o nome que desejamos dar a ela logo após o BEGIN TRANSACTION, desta forma:

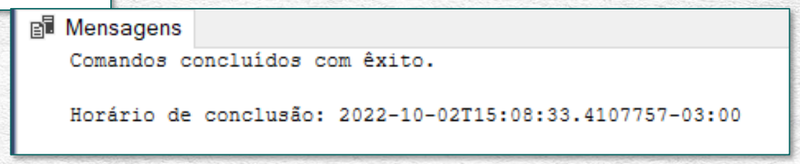
```
BEGIN TRANSACTION T1
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Naldo Reis', 'M', '10/02/1992', '412.889.311-90')
```



No exemplo acima, chamamos nossa transação de T1.

Assim, quando formos efetuar um COMMIT ou um ROLLBACK nesta transação, informamos o nome dela junto ao comando a ser executado:

```
COMMIT TRANSACTION T1
```



COMMIT e ROLLBACK condicionais

Vamos agora a um exemplo um pouquinho mais complexo que os anteriores, em que estabeleceremos, dentro de uma transação, uma **condição** para que uma inserção de dados seja ou salva de forma definitiva no banco de dados (com um COMMIT), ou desfeita (com um ROLLBACK).

Suponhamos que a gente queira inserir no banco de dados uma cliente chamada Ruth Campos.

Só que, antes de executar um COMMIT, primeiro nós queremos saber se o cadastro dessa cliente já existe.

Para isso, podemos estabelecer uma condição: se o cadastro dessa cliente já existir, desfaça a transação; se não existir, salve a transação.

Vejamos, na próxima página, como podemos fazer isso na prática.

COMMIT e ROLLBACK condicionais

Primeiramente, apenas para fins didáticos, vamos consultar os registros da nossa tabela cliente_aux:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	19	Maria Julia	F	1995-04-30	987.654.321-00
14	20	Naldo Reis	M	1992-02-10	412.889.311-90

Note que não temos nenhuma cliente chamada Ruth Campos cadastrada na tabela cliente_aux.

Vamos, então, ao código da nossa transaction.

COMMIT e ROLLBACK condicionais

O código completo que executaremos será este aqui:

```
DECLARE @contador INT

BEGIN TRANSACTION T1
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Ruth Campos', 'F', '23/03/1992', '324.731.903-89')

SELECT @contador = COUNT(*) FROM cliente_aux WHERE nome_cliente = 'Ruth Campos'


IF @contador = 1
    BEGIN

        COMMIT TRANSACTION T1
        PRINT 'Ruth Campos cadastrada com sucesso.'

    END
ELSE
    BEGIN

        ROLLBACK TRANSACTION T1
        PRINT 'Ruth Campos já foi cadastrada na tabela. Insert abortado.'

    END
```

Fique tranquilo(a), veremos agora seu passo a passo nas próximas páginas!

COMMIT e ROLLBACK condicionais

1. Antes de tudo, declaramos uma variável chamada @contador que mais a frente receberá a contagem de quantas clientes chamadas “Ruth Campos” existem na tabela cliente_aux;

2. Iniciamos a transação com um BEGIN TRANSACTION, dando a ela o nome de T1;

3. Efetuamos um INSERT INTO com os dados da cliente Ruth Campos;

```
DECLARE @contador INT 1

BEGIN TRANSACTION T1 2
INSERT INTO cliente_aux(nome_cliente, genero, data_de_nascimento, cpf) VALUES 3
    ('Ruth Campos', 'F', '23/03/1992', '324.731.903-89')
```

COMMIT e ROLLBACK condicionais

4 Fazemos a contagem na tabela cliente_aux de todas as clientes que se chamam “Ruth Campos”, e armazenamos essa contagem na variável @contador que criamos no início.
Repare que, conforme vimos anteriormente, até agora, não tínhamos nenhuma cliente chamada Ruth Campos cadastrada na tabela cliente_aux.
Quando iniciamos a transação T1, inserimos um registro com os dados de uma cliente chamada Ruth Campos.
Portanto, agora, a nossa contagem (COUNT) verificará que existe o registro de uma cliente chamada Ruth Campos na tabela cliente_aux, por isso armazenará na varíavel @contador o valor 1.

```
SELECT @contador = COUNT(*) FROM cliente_aux WHERE nome_cliente = 'Ruth Campos'  4
```

COMMIT e ROLLBACK condicionais

5 Continuamos agora estabelecendo a condição: se a variável @contador for igual a 1, devemos salvar definitivamente esse novo registro executando um COMMIT na transação T1. Além disso, imprimimos na tela a mensagem “Ruth Campos cadastrada com sucesso.”

6 Caso contrário, ou seja, se a variável @contador tiver recebido um valor maior que 1, significa que esse registro já existia na tabela, portanto, devemos desfazer a inserção dele executando um ROLLBACK na transação T1. Além disso, imprimimos na tela a mensagem “Ruth Campos já foi cadastrada na tabela. Insert abortado.”

```
IF @contador = 1 5
    BEGIN

        COMMIT TRANSACTION T1
        PRINT 'Ruth Campos cadastrada com sucesso.'

    END
ELSE
    BEGIN

        ROLLBACK TRANSACTION T1 6
        PRINT 'Ruth Campos já foi cadastrada na tabela. Insert abortado.'

    END
```

COMMIT e ROLLBACK condicionais

Note que, como ainda não havia registro na tabela cliente_aux com o nome_cliente = “Ruth Campos”, na primeira vez que executamos essa transação condicional, a inserção foi cadastrada com sucesso 1 :

1

Mensagens

```
(1 linha afetada)
Ruth Campos cadastrada com sucesso.

Horário de conclusão: 2022-10-02T16:03:33.5436543-03:00
```

Repare que, agora, a cliente Ruth Campos encontra-se devidamente cadastrada 2 :

2

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	19	Maria Julia	F	1995-04-30	987.654.321-00
14	20	Naldo Reis	M	1992-02-10	412.889.311-90
15	21	Ruth Campos	F	1992-03-23	324.731.903-89

COMMIT e ROLLBACK condicionais

Porém, se tentarmos inserir novamente um registro com o nome_cliente = “Ruth Campos”, o sistema efetuará o ROLLBACK estabelecido na condição que criamos, retornando a mensagem “Ruth Campos já foi cadastrada na tabela. Insert abortado.”, conforme esperado 1:

Mensagens

```
(1 linha afetada)
Ruth Campos já foi cadastrada na tabela. Insert abortado.

Horário de conclusão: 2022-10-02T16:12:07.1141148-03:00
```

1

Repare que continua registrada na tabela cliente_aux apenas uma linha com nome_cliente = “Ruth Campos” 2 :

Resultados | Mensagens

2

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	19	Maria Julia	F	1995-04-30	987.654.321-00
14	20	Naldo Reis	M	1992-02-10	412.889.311-90
15	21	Ruth Campos	F	1992-03-23	324.731.903-89

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Lembra-se que, no início deste módulo, para criar nossa tabela cliente_aux, nós utilizamos as informações contidas na tabela dCliente?

```
SELECT *
INTO cliente_aux
FROM dCliente
```

Pois bem. Se repararmos, agora, no código que foi utilizado para criação da tabela dCliente, veremos que a coluna data_de_nascimento foi definida como do tipo DATE:

```
CREATE TABLE dCliente(
    id_cliente INT IDENTITY(1, 1),
    nome_cliente VARCHAR(100) NOT NULL,
    genero VARCHAR(100) NOT NULL,
    data_de_nascimento DATE NOT NULL,
    cpf VARCHAR(100) NOT NULL,
    CONSTRAINT dcliente_id_cliente_pk PRIMARY KEY(id_cliente),
    CONSTRAINT dcliente_genero_ck CHECK(genero IN ('M', 'F', 'O', 'PND')),
    CONSTRAINT dcliente_cpf_un UNIQUE(cpf)
)
```

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Isso significa que a coluna data_de_nascimento da nossa tabela cliente_aux também é do tipo DATE, já que ela foi criada com base em toda a estrutura e registros da tabela dCliente.

Vamos imaginar que descobrimos que a data de nascimento do cliente de ID número 4 estava incorreta e, portanto, precisamos corrigir essa informação:

Resultados | Mensagens

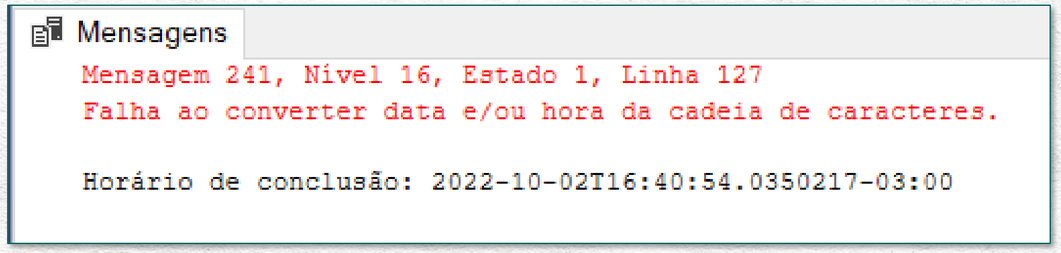
	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Sabendo que a data de nascimento correta deste cliente é 15 de março de 1992, tentamos efetuar a seguinte alteração:

```
BEGIN TRANSACTION T1

    UPDATE cliente_aux
    SET data_de_nascimento = '15 de março 1992'
    WHERE id_cliente = 4

COMMIT TRANSACTION T1
```

Porém, conforme vimos, a coluna data_de_nascimento é do tipo DATE. Portanto, ela não aceitou a nova data de nascimento informada no código acima, pois, da forma como ela foi passada, o sistema entendeu que se trata de uma string (texto), não uma data. O correto seria ter informado 15/03/1992. Assim sendo, o sistema retornou o seguinte erro:



Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Todo sistema é passível de apresentar erros, por isso, é muito importante que sejamos capazes de prever esses erros, para que possamos criar mecanismos para trata-los caso aconteçam. Assim, o sistema não será abortado, pois saberá exatamente como proceder.

Portanto, vamos pensar em como poderíamos ter tratado o erro apresentado na página anterior, utilizando os comandos TRY e CATCH.

Uma forma de ter resolvido esse problema, poderia ter sido por meio de um ROLLBACK: informaríamos ao sistema que ele deveria tentar efetuar o UPDATE desejado (dentro de um bloco TRY) e, caso não fosse possível, deveria ser efetuado um ROLLBACK (informado dentro de um bloco CATCH).

Vejamos ao lado como ficaria o código:

```
BEGIN TRY

    BEGIN TRANSACTION T1

        UPDATE cliente_aux
        SET data_de_nascimento = '15 de março 1992'
        WHERE id_cliente = 4

    COMMIT TRANSACTION T1
    PRINT 'Data atualizada com sucesso'

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION T1
    PRINT 'Data cadastrada inválida'

END CATCH
```

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Vamos percorrer agora seu o passo a passo:

1. Iniciamos o bloco TRY com o comando BEGIN TRY;
2. Iniciamos a transação com o comando BEGIN TRANSACTION, nomeando-a T1;
3. Inserimos dentro da transação o UPDATE que queremos efetuar;
4. Finalizamos a transação com um COMMIT, imprimindo na tela, em seguida, a informação “Data atualizada com sucesso”;
5. Finalizamos o bloco TRY com o comando END TRY;

```
BEGIN TRY 1

    BEGIN TRANSACTION T1 2

        UPDATE cliente_aux 3
        SET data_de_nascimento = '15 de março 1992'
        WHERE id_cliente = 4

    COMMIT TRANSACTION T1 4
    PRINT 'Data atualizada com sucesso'

END TRY 5
```

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Para o caso do comando inserido dentro do bloco TRY apresentar algum erro, criamos agora um bloco com o comando CATCH, que tratará o erro eventualmente retornado pelo código contido no bloco TRY.

6 Para isso, iniciamos o bloco CATCH com o comando BEGIN CATCH;

7 Inserimos dentro deste bloco o comando ROLLBACK para desfazer a transação T1 criada dentro do bloco TRY, imprimindo na tela, em seguida, a informação “Data cadastrada inválida”;

8 Finalizamos o bloco CATCH com o comando END CATCH;

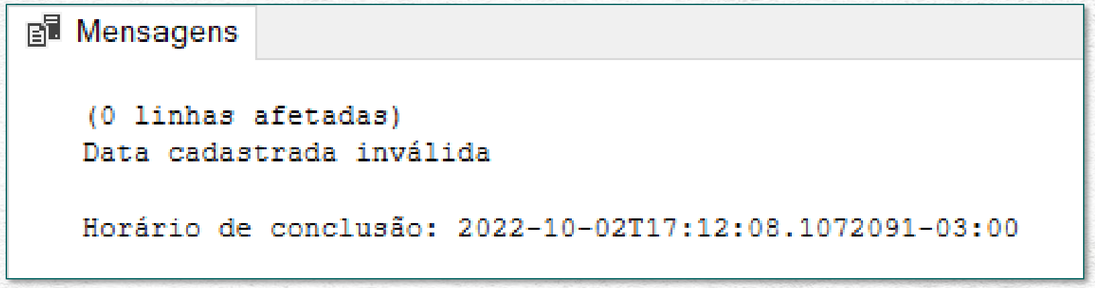
```
BEGIN CATCH 6

    ROLLBACK TRANSACTION T1 7
    PRINT 'Data cadastrada inválida'

END CATCH 8
```

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Note que, agora, se tentarmos executar este código que criamos, tentando inserir a data com a string ‘15 de março de 1992’, o sistema retornará a mensagem abaixo.

Repare que 0 linhas foram afetadas, pois o ROLLBACK definido no bloco CATCH foi efetuado.

E a mensagem “Data cadastrada inválida”, também definida no bloco CATCH, foi apresentada na tela, conforme esperado.



Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Se consultarmos a tabela cliente_aux, verificaremos que a data de nascimento do cliente de ID número 4 permanece inalterada:

```
SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	11	Katia Lopes	F	1989-01-10	123.234.812-20
12	16	Lucia Campos	F	1994-10-23	999.234.812-20
13	19	Maria Julia	F	1995-04-30	987.654.321-00
14	20	Naldo Reis	M	1992-02-10	412.889.311-90
15	21	Ruth Campos	F	1992-03-23	324.731.903-89

Tratamento de erros em TRANSACTIONS (TRY e CATCH)

Se alterarmos a data informada no nosso código para o formato correto, para que o sistema a reconheça como sendo do tipo DATE, a alteração será devidamente efetuada e salva com um COMMIT, conforme determinado pela transação contida dentro do bloco TRY:

1

```
BEGIN TRY

    BEGIN TRANSACTION T1

        UPDATE cliente_aux
        SET data_de_nascimento = '15/03/1992'
        WHERE id_cliente = 4

    COMMIT TRANSACTION T1
    PRINT 'Data atualizada com sucesso'

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION T1
    PRINT 'Data cadastrada inválida'

END CATCH
```

2

Mensagens

```
(1 linha afetada)
Data atualizada com sucesso

Horário de conclusão: 2022-10-02T17:21:56.3303218-03:00
```

3

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1992-03-15	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82

Variável @@TRANCOUNT

Conforme visto, quando iniciamos uma transação explícita, como temos feito neste módulo, ela fica em aberto aguardando que executemos um COMMIT para salvá-la definitivamente no banco de dados, ou um ROLLBACK para desfazê-la.

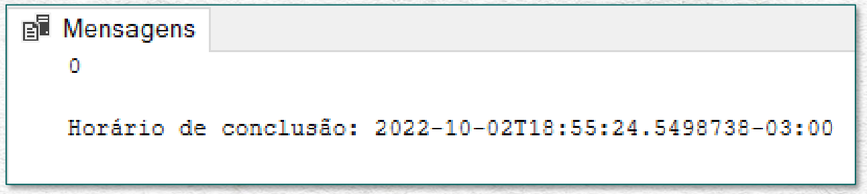
A variável @@TRANCOUNT pode ser muito útil quando queremos saber quantas transações temos em aberto aguardando que executemos um COMMIT ou um ROLLBACK.

Vejamos:

Para iniciar, vamos utilizar a variável @@TRANCOUNT para nos mostrar que não existem transações em aberto:

```
PRINT @@TRANCOUNT
```

Repare que o print acima retornou o número 0, indicando que realmente não temos transações em aberto:



Variável @@TRANCOUNT

Agora, vamos iniciar duas transações: a T1 e a T2.

```
BEGIN TRANSACTION T1
BEGIN TRANSACTION T2
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-02T19:00:42.0521858-03:00
```

Imprimindo novamente a variável @@TRANCOUNT, veremos que, agora, o sistema retornará o valor 2. Isso significa que temos duas transações em aberto, que são exatamente a T1 e a T2 que inicializamos acima:

```
PRINT @@TRANCOUNT
```

```
Mensagens
2

Horário de conclusão: 2022-10-02T19:01:17.2331234-03:00
```

Variável @@TRANCOUNT

Vamos executar um COMMIT na transação T2:

```
COMMIT TRANSACTION T2
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-02T19:02:59.6356074-03:00
```

Agora, se imprimirmos novamente a variável @@TRANCOUNT, veremos que o sistema retornará o valor 1. Isso significa que temos apenas uma transação em aberto, que é a T1, já que a T2 acabamos de salvar e finalizar com o COMMIT acima:

```
PRINT @@TRANCOUNT
```

```
Mensagens
1

Horário de conclusão: 2022-10-02T19:04:55.7479322-03:00
```

Variável @@TRANCOUNT

Por fim, vamos executar um ROLLBACK na transação T1:

```
ROLLBACK TRANSACTION T1
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-02T19:06:29.5764010-03:00
```

Note que, se imprimirmos novamente a variável @@TRANCOUNT, veremos que o sistema retornará o valor 0. Isso significa que finalmente não temos mais transações em aberto, pois a última transação que aguardava um fechamento, a T1, foi desfeita e finalizada com o ROLLBACK que executamos acima:

```
PRINT @@TRANCOUNT
```

```
Mensagens
0

Horário de conclusão: 2022-10-02T19:08:39.2821874-03:00
```

Transações Aninhadas

TRANSACTIONS, além dos diversos tipos de comandos capazes de inserir, alterar ou excluir informações em um banco de dados, também podem conter outras transações dentro de seu bloco de comandos. A essas transações "uma dentro da outra", chamamos de "transações aninhadas" (ou, em inglês, de *nested transactions*).

A grande diferença das transações aninhadas para as transações feitas separadamente (que temos visto até aqui) é que, com as transações aninhadas, as transações mais internas, mesmo que contenham COMMITs ou ROLLBACKs, só serão de fato salvas ou desfeitas quando a transação principal (mais externa) for salva com um COMMIT.

Caso a transação externa venha a ser desfeita com um ROLLBACK, tudo que tiver sido efetuado pelas transações internas não será salvo, mesmo que estas transações internas contenham seus próprios COMMITs.

Em resumo, as transações internas são dependentes da transação externa.

Para ilustrar, vejamos um exemplo na próxima página.

Transações Aninhadas

Vamos executar o código ao lado:

Repare que nele temos uma transação externa **(T1)** e dentro dela, temos outra transação **(T2)**.

1. Com a T1, efetuamos um UPDATE alterando o CPF do cliente de ID número 1 da tabela cliente_aux para “999.999.999-99”.

2. Já com a T2, efetuamos um UPDATE alterando o CPF do cliente de ID número 2 da tabela cliente_aux para “111.111.111-11”.

3. Para finalizar a T2 (interna), efetuamos um COMMIT.

4. Porém, para finalizar a T1 (externa) executamos um ROLLBACK.

```
BEGIN TRAN T1 <-- T1                    -- Início transação 1

    UPDATE cliente_aux
    SET cpf = '999.999.999-99' 1
    WHERE id_cliente = 1

    SELECT * FROM cliente_aux

    BEGIN TRAN T2 <-- T2                -- Início transação 2

        UPDATE cliente_aux
        SET cpf = '111.111.111-11' 2
        WHERE id_cliente = 2

        SELECT * FROM cliente_aux

    COMMIT TRAN T2 3                    -- Commit transação 2

ROLLBACK TRAN T1 4                      -- Rollback transação 1

SELECT * FROM cliente_aux
```

Transações Aninhadas

Vejamos agora o passo a passo de como as informações de CPF desses dois clientes (IDs números 1 e 2) foram sendo alteradas conforme o código da página anterior foi sendo executado.

Antes de tudo, vamos relembrar as informações dos clientes de IDs número 1 e 2:

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02

Ao alterarmos o CPF do cliente de ID número 1 para 999.999.999-99, esta alteração foi inserida na tabela, aguardando o COMMIT para ser salva de maneira definitiva, ou o ROLLBACK para ser desfeita:

```
BEGIN TRAN T1

    UPDATE cliente_aux
    SET cpf = '999.999.999-99'
    WHERE id_cliente = 1

    SELECT * FROM cliente_aux
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	999.999.999-99
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02

Transações Aninhadas

Continuando na execução do nosso código, ao alterarmos o CPF do cliente de ID número 2 para 111.111.111-11, esta alteração foi inserida na tabela e aparentemente salva, já que efetuamos um COMMIT ao final da T2:

```
BEGIN TRAN T2

    UPDATE cliente_aux
    SET cpf = '111.111.111-11'
    WHERE id_cliente = 2

    SELECT * FROM cliente_aux

COMMIT TRAN T2
```

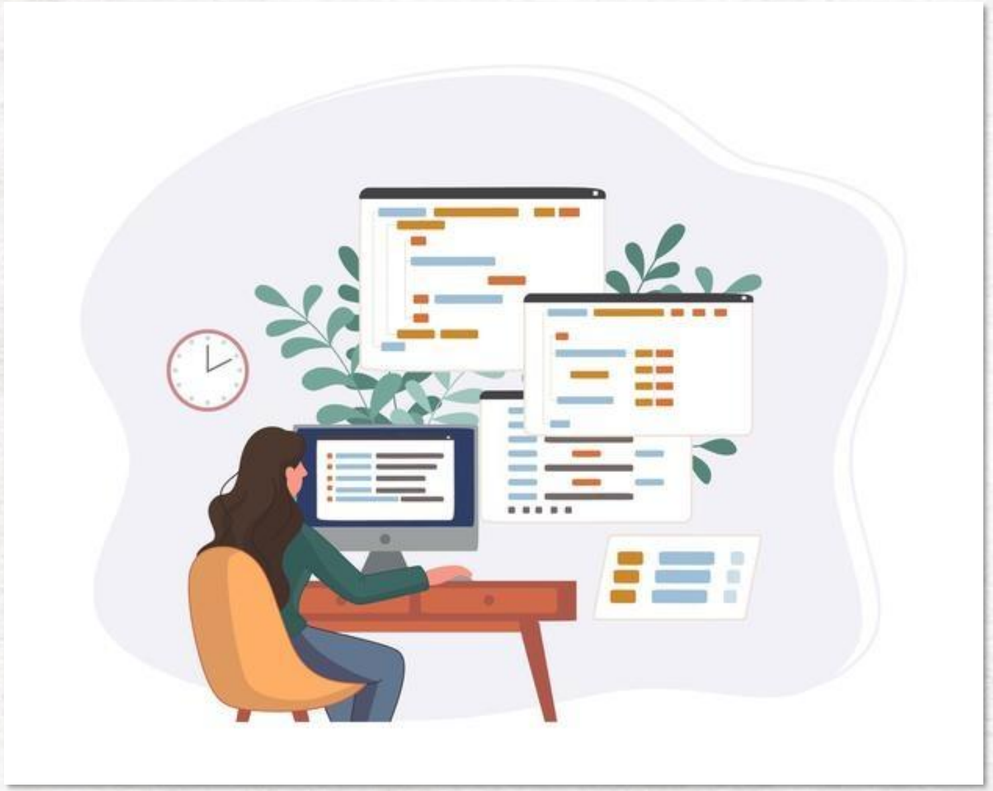
	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	999.999.999-99
2	2	Bárbara Campos	F	1992-05-07	111.111.111-11

Porém, para finalizar a T1, que é a nossa transação externa, na qual a T2 acima está contida, utilizamos um ROLLBACK. Assim, ambas as alterações de números de CPFs que haviam sido efetuadas foram desfeitas, inclusive a do ID número 2 (que foi efetuada e salva com um COMMIT pela T2), pois a T2 interna é dependente da T1 externa. Assim, uma vez efetuado um ROLLBACK na T1, também foi efetuado um ROLLBACK na T2:

```
ROLLBACK TRAN T1

SELECT * FROM cliente_aux
```

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02

EXERCÍCIOS



1 Questão 1

Crie uma tabela chamada Carro com os dados abaixo.

	id_carro	placa	modelo	tipo
1	1	DAS-1412	Hyundai HB20	Hatch
2	2	JHG-3902	Fiat Cronos	Sedan
3	3	IPW-9018	Citroen C4 Cactus	SUV
4	4	JKR-8891	Nissa Kicks	SUV
5	5	TRF-5904	Chevrolet Onix	Sedan

Obs.: não se preocupe com constraints, pode criar uma tabela simples.

2 Questão 2

Execute as seguintes transações no banco de dados, sempre na tabela Carro. Lembre-se de dar um COMMIT para efetivar cada uma das transações.

a) Inserir uma nova linha com os seguintes valores:
- id_carro = 6
- placa = CDR-0090
- modelo = Fiat Argo
- tipo = Hatch

b) Atualizar o tipo do carro de id = 1 de Hatch para Sedan.

c) Deletar a linha referente ao carro de id = 6.

GABARITOS

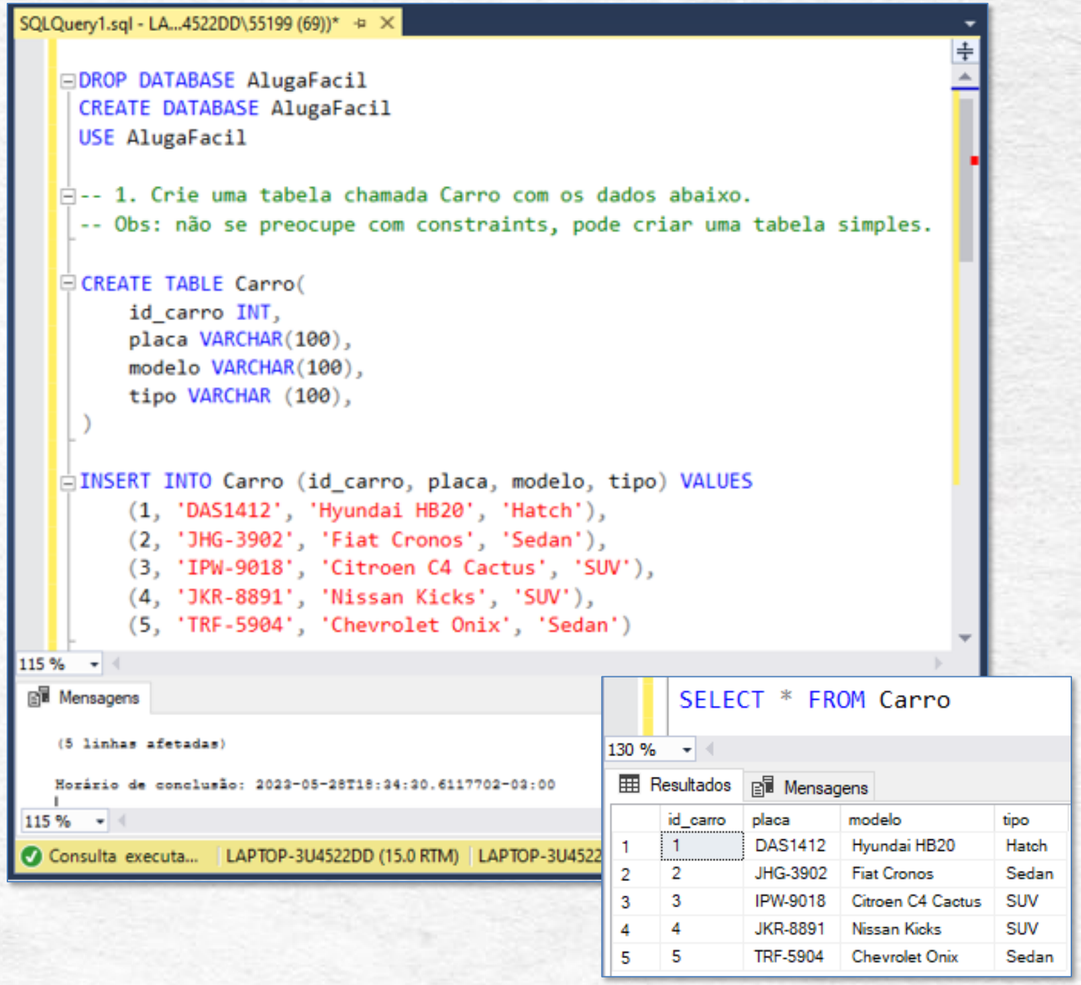
1

Questão 1

Crie uma tabela chamada Carro com os dados abaixo.

	id_carro	placa	modelo	tipo
1	1	DAS-1412	Hyundai HB20	Hatch
2	2	JHG-3902	Fiat Cronos	Sedan
3	3	IPW-9018	Citroen C4 Cactus	SUV
4	4	JKR-8891	Nissa Kicks	SUV
5	5	TRF-5904	Chevrolet Onix	Sedan

Obs.: não se preocupe com constraints, pode criar uma tabela simples.



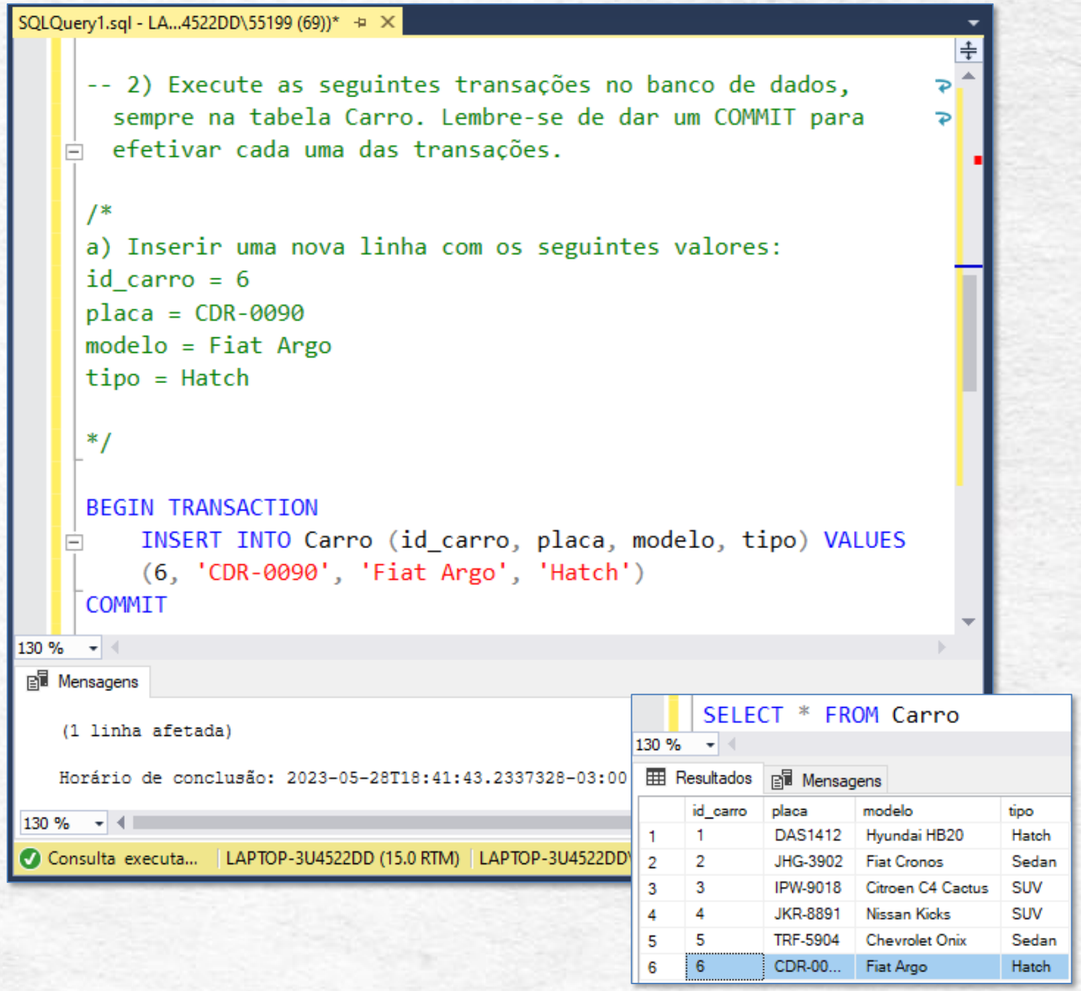
GABARITOS

2.a

Questão 2.a

Execute as seguintes transações no banco de dados, sempre na tabela Carro. Lembre-se de dar um COMMIT para efetivar cada uma das transações.

a) Inserir uma nova linha com os seguintes valores:
- id_carro = 6
- placa = CDR-0090
- modelo = Fiat Argo
- tipo = Hatch

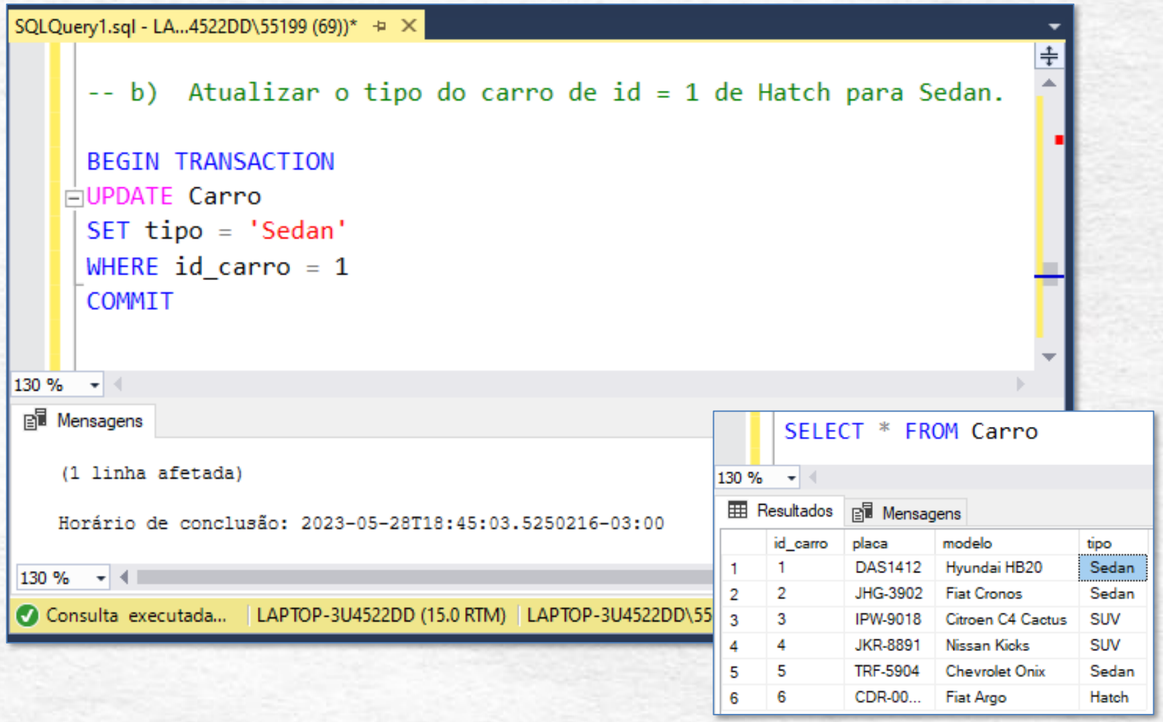


GABARITOS

2.b

Questão 2.b

b) Atualizar o tipo do carro de id = 1 de Hatch para Sedan.

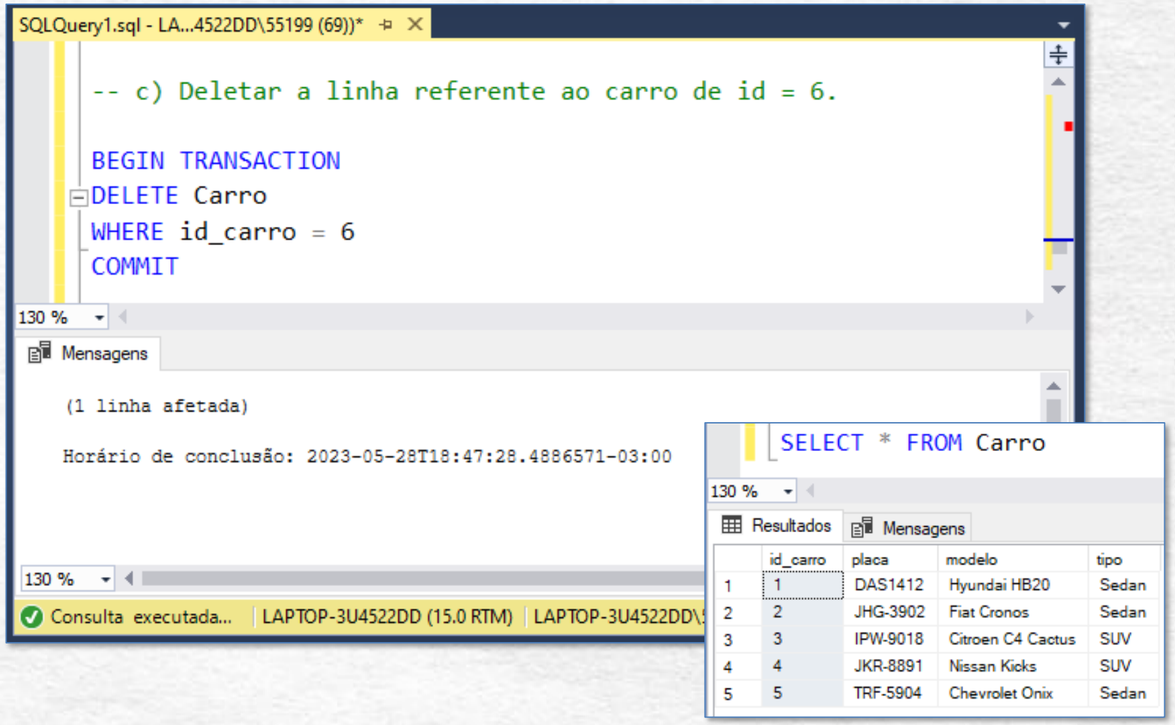


GABARITOS

2.c

Questão 2.c

c) Deletar a linha referente ao carro de id = 6.



MÓDULO 21

SQL Functions

O QUE VEREMOS NESTE MÓDULO

MÓDULO 21 FUNCTIONS

01	O que são FUNCTIONS?	592
02	Como criar e utilizar uma FUNCTION	594
03	Como alterar uma FUNCTION	599
04	Como excluir uma FUNCTION	602
05	EXERCÍCIOS	603
06	GABARITOS	605

O que são FUNCTIONS?

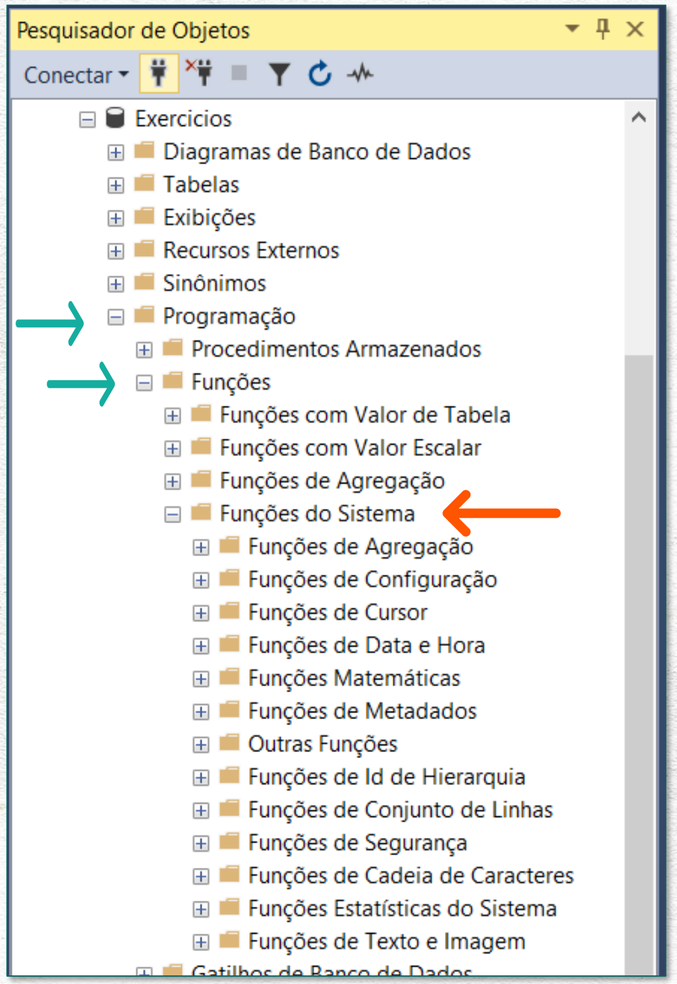
Uma FUNCTION é um conjunto de comandos que executam ações e retorna um valor escalar. As functions ajudam a simplificar um código.

Por exemplo, se você tem um cálculo complexo que aparece diversas vezes no seu código, em vez de repetir várias vezes aquela série de comandos, você pode simplesmente criar uma função e reaproveitá-la sempre que precisar.

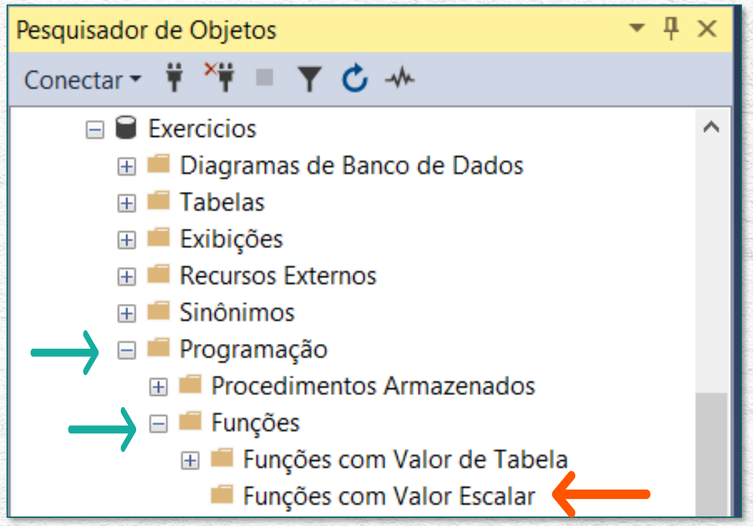
O próprio SQL tem diversas funções do sistema (prontas) e, até agora, já vimos vários exemplos de funções deste tipo, como funções de data, texto, etc.

O que são FUNCTIONS?

Podemos visualizar as funções do sistema na pasta Programação > Funções > Funções do Sistema:



Conforme formos criando nossas próprias funções, elas ficarão salvas na pasta Programação > Funções > Funções com Valor Escalar:



Como criar e utilizar uma FUNCTION

Vamos imaginar que a gente queira fazer uma formatação diferenciada na coluna data_de_nascimento da tabela dCliente. Em vez de retornar apenas a data conforme ela está armazenada (por exemplo, '1989-02-12'), queremos retorná-la por extenso (assim: 'Domingo, 12 de Fevereiro de 1989').

Vimos no módulo de funções single row que podemos fazer isso por meio da função DATENAME, desta forma:

```
SELECT
    nome_cliente,
    data_de_nascimento,
    DATENAME(DW, data_de_nascimento) + ', ' +
    DATENAME(D, data_de_nascimento) + ' de ' +
    DATENAME(M, data_de_nascimento) + ' de ' +
    DATENAME(YY, data_de_nascimento)
FROM
    dCliente
```

Resultados | Mensagens

	nome_cliente	data_de_nascimento	(Nenhum nome de coluna)
1	André Martins	1989-02-12	Domingo, 12 de Fevereiro de 1989
2	Bárbara Campos	1992-05-07	Quinta-Feira, 7 de Maio de 1992
3	Carol Freitas	1985-04-23	Terça-Feira, 23 de Abril de 1985
4	Diego Cardoso	1994-10-11	Terça-Feira, 11 de Outubro de 1994
5	Eduardo Pereira	1988-11-09	Quarta-Feira, 9 de Novembro de 1988
6	Fabiana Silva	1989-09-02	Sábado, 2 de Setembro de 1989
7	Gustavo Barbosa	1993-06-27	Domingo, 27 de Junho de 1993
8	Helen Viana	1990-02-11	Domingo, 11 de Fevereiro de 1990
9	Igor Castro	1989-08-21	Segunda-Feira, 21 de Agosto de 1989
10	Juliana Pires	1991-01-13	Domingo, 13 de Janeiro de 1991

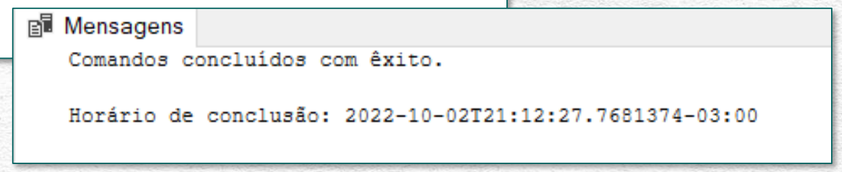
Como criar e utilizar uma FUNCTION

Porém, para facilitar para gente, nós podemos criar uma function que utilize a DATENAME e retorne este mesmo resultado para qualquer coluna de data de qualquer tabela.

Assim, quando formos efetuar a consulta a uma tabela que contenha uma coluna de data que precisemos retornar por extenso (como no caso da coluna data_de_nascimento da tabela dCliente), nós não precisamos recriar todo o código com a DATENAME. Tudo que precisamos fazer é “chamar” dentro do nosso SELECT a função que tivermos criado.

Vamos portanto, criar essa função, que chamaremos de fnDataCompleta, usando o código ao lado:

```
CREATE FUNCTION fnDataCompleta(@data AS DATE)
RETURNS VARCHAR(MAX)
AS
BEGIN

    RETURN DATENAME(DW, @Data) + ', ' +
           DATENAME(D, @Data) + ' de ' +
           DATENAME(M, @Data) + ' de ' +
           DATENAME(YY, @Data)

END
```



Como criar e utilizar uma FUNCTION

1. Iniciamos com o comando CREATE FUNCTION, damos um nome à function (fnDataCompleta), assim como ao parâmetro que será recebido por ela (no caso @data, que será do tipo DATE);

2. Em seguida, informamos o tipo de dado que a function deverá retornar que, neste caso, será do tipo VARCHAR. Repare que no parâmetro a function receberá um dado do tipo DATE (YYYY-MM-DD), fará o tratamento necessário nesse dado (cujos comandos serão passados logo mais dentro do bloco de execução que inicia com BEGIN) e retornará um dado do tipo VARCHAR (a string da data por extenso);

3. Depois, abrimos o bloco de execução e, dentro dele, passamos os comandos do que a função deve ser capaz de fazer (no caso, transformar uma data em uma string desta data por extenso);

4. Finalizamos o bloco de execução com o comando END.

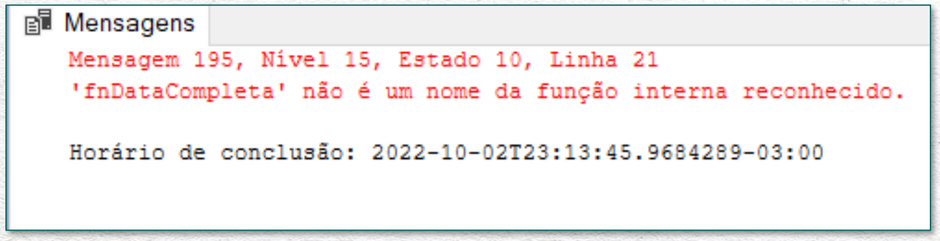
```
CREATE FUNCTION fnDataCompleta(@data AS DATE) 1
RETURNS VARCHAR(MAX) 2
AS
BEGIN 3

    RETURN DATENAME(DW, @Data) + ', ' +
           DATENAME(D, @Data) + ' de ' +
           DATENAME(M, @Data) + ' de ' +
           DATENAME(YY, @Data)

END 4
```

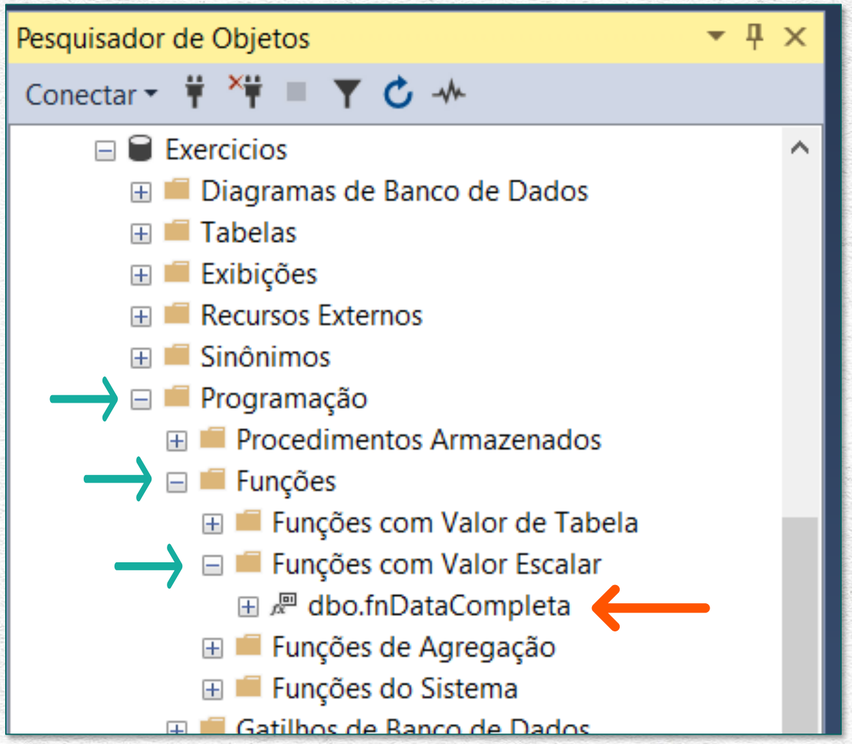
Como criar e utilizar uma FUNCTION

Repare que nossa function recém-criada agora está salva na pasta Programação > Funções > Funções com Valor Escalar, como havíamos mencionado anteriormente.

Note que, ao nome da function, o sistema acrescentou o prefixo “dbo”. Este termo significa “database owner” (proprietário do banco de dados) e é anexado a todo objeto criado pelo administrador do banco de dados.

Por isso, em se tratando de functions, quando formos utilizá-las nas consultas que efetuarmos, devemos sempre nos lembrar de acrescentar o prefixo “dbo.” ao seu nome, caso contrário, o sistema retornará o erro abaixo:





Como criar e utilizar uma FUNCTION

Criada nossa function fnDataCompleta, agora podemos utilizá-la todas as vezes que precisarmos pegar um valor do tipo DATE e transformá-lo em uma string dessa data por extenso.

Então, para executar esse procedimento com a coluna data_de_nascimento da tabela dCliente, podemos fazer como no código ao lado.

Perceba que informamos o nome da função (dbo.fnDataCompleta) e, entre parênteses, passamos o nome da coluna que servirá de parâmetro para a função. Neste caso, é a coluna data_de_nascimento da tabela dCliente 1.

Assim, a function pegará essa coluna informada como parâmetro na nossa consulta ao lado, passará para o parâmetro @data da function, e executará os comandos constantes em seu bloco de execução.

Ao final, nossa *query* retornará o resultado desejado, com a nova coluna “criada” pela function 2:

```sql
SELECT
    nome_cliente,
    data_de_nascimento,
    dbo.fnDataCompleta(data_de_nascimento)
FROM
    dcliente
```

1

Resultados | Mensagens

2

	nome_cliente	data_de_nascimento	(Nenhum nome de coluna)
1	André Martins	1989-02-12	Domingo, 12 de Fevereiro de 1989
2	Bárbara Campos	1992-05-07	Quinta-Feira, 7 de Maio de 1992
3	Carol Freitas	1985-04-23	Terça-Feira, 23 de Abril de 1985
4	Diego Cardoso	1994-10-11	Terça-Feira, 11 de Outubro de 1994
5	Eduardo Pereira	1988-11-09	Quarta-Feira, 9 de Novembro de 1988
6	Fabiana Silva	1989-09-02	Sábado, 2 de Setembro de 1989
7	Gustavo Barbosa	1993-06-27	Domingo, 27 de Junho de 1993
8	Helen Viana	1990-02-11	Domingo, 11 de Fevereiro de 1990
9	Igor Castro	1989-08-21	Segunda-Feira, 21 de Agosto de 1989
10	Juliana Pires	1991-01-13	Domingo, 13 de Janeiro de 1991

Como alterar uma FUNCTION

Para alterar uma function, devemos, na primeira linha do nosso código, utilizar o comando ALTER FUNCTION:

```
ALTER FUNCTION fnDataCompleta(@data AS DATE)
```

Entretanto, como uma boa prática, aconselhamos unir os comandos CREATE e ALTER sempre que formos criar ou alterar uma FUNCTION.

Desta forma, não corremos o risco de querer criar uma function que já existe e retornar um erro, ou querer alterar uma function que ainda não existe, incorrendo em novo erro:

```
CREATE OR ALTER FUNCTION fnDataCompleta(@data AS DATE)
```

Com o comando unificado CREATE OR ALTER FUNCTION, o sistema será capaz de verificar:

A. Se a function não existe, então ele criará uma nova function por meio do comando CREATE;
B. Se a function já existe, assim ele alterará a function já existente por meio do comando ALTER.

Como alterar uma FUNCTION

Dito isto, vamos alterar a function que criamos para incluir na nossa string de data por extenso, também a informação de semestre.

Para isso, vamos precisar acrescentar uma estrutura condicional: o CASE.

Conforme demonstrado ao lado, com o CASE verificamos o seguinte:

Caso o mês seja menor ou igual a 6, retornamos o termo '(1º Semestre)' junto à string da data por extenso;
Caso contrário (ou seja, caso o mês seja maior que 6), retornamos o termo '(2º Semestre)' junto à string da data por extenso.

```
CREATE OR ALTER FUNCTION fnDataCompleta(@data AS DATE)
RETURNS VARCHAR(MAX)
AS
BEGIN

    RETURN DATENAME(DW, @Data) + ', ' +
           DATENAME(D, @Data) + ' de ' +
           DATENAME(M, @Data) + ' de ' +
           DATENAME(YY, @Data) + ' - ' +
           CASE
               WHEN MONTH(@Data) <= 6 THEN
                   '(1º Semestre)'
               ELSE
                   '(2º Semestre)'
           END

END
```

Mensagens

```
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-02T23:57:52.1827678-03:00
```

Como alterar uma FUNCTION

Repare agora, com a alteração efetuada na nossa function, como será o resultado apresentado pela consulta à tabela dCliente:

```
SELECT
    nome_cliente,
    data_de_nascimento,
    dbo.fnDataCompleta(data_de_nascimento)
FROM
    dCliente
```

Foi incluída na coluna "criada" pela function fnDataCompleta a informação de semestre junto à data por extenso:

Resultados | Mensagens

	nome_cliente	data_de_nascimento	(Nenhum nome de coluna)
1	André Martins	1989-02-12	Domingo, 12 de Fevereiro de 1989 - (1º Semestre)
2	Bárbara Campos	1992-05-07	Quinta-Feira, 7 de Maio de 1992 - (1º Semestre)
3	Carol Freitas	1985-04-23	Terça-Feira, 23 de Abril de 1985 - (1º Semestre)
4	Diego Cardoso	1994-10-11	Terça-Feira, 11 de Outubro de 1994 - (2º Semestre)
5	Eduardo Pereira	1988-11-09	Quarta-Feira, 9 de Novembro de 1988 - (2º Semestre)
6	Fabiana Silva	1989-09-02	Sábado, 2 de Setembro de 1989 - (2º Semestre)
7	Gustavo Barbosa	1993-06-27	Domingo, 27 de Junho de 1993 - (1º Semestre)
8	Helen Viana	1990-02-11	Domingo, 11 de Fevereiro de 1990 - (1º Semestre)
9	Igor Castro	1989-08-21	Segunda-Feira, 21 de Agosto de 1989 - (2º Semestre)
10	Juliana Pires	1991-01-13	Domingo, 13 de Janeiro de 1991 - (1º Semestre)

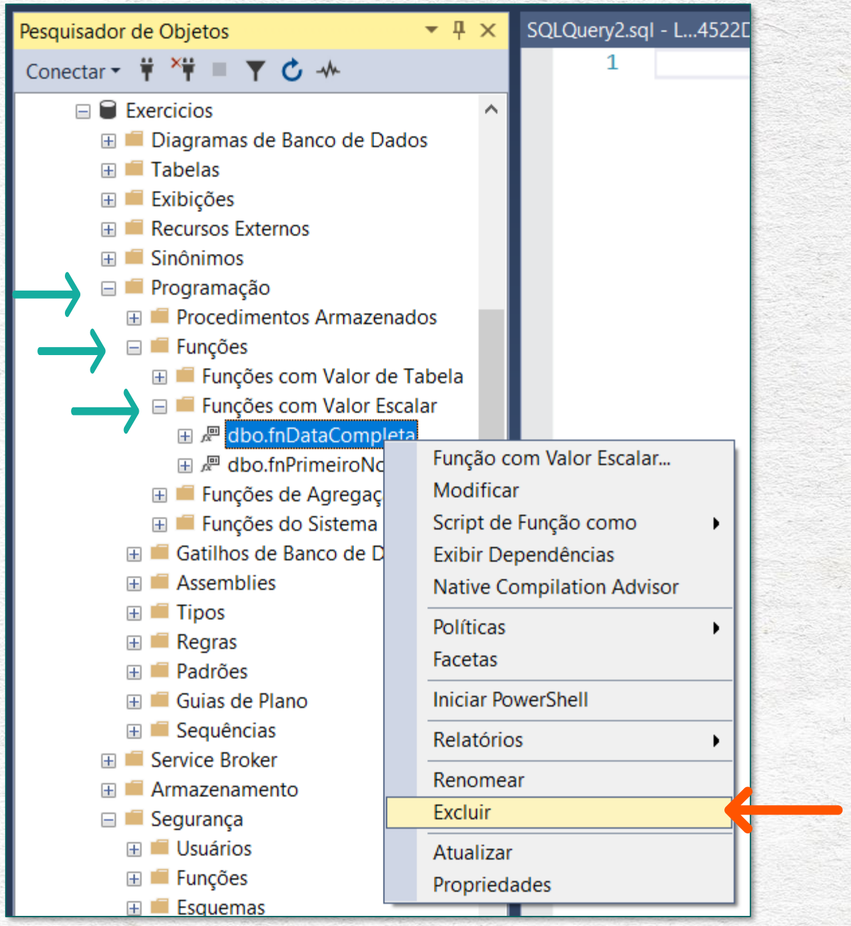
Como excluir uma FUNCTION

Para excluir uma function é muito simples, basta executarmos o comando DROP FUNCTION, conforme abaixo:

```
DROP FUNCTION fnDataCompleta
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-03T00:04:05.3563334-03:00
```

Ou, manualmente, clicando com o botão direito sobre o nome da function armazenada na pasta Programação > Funções > Funções com Valor Escalar, e depois selecionar a opção Excluir (conforme ao lado):



EXERCÍCIOS

1 Questão 1

Crie uma Function que calcule o tempo (em anos) entre duas datas.

Essa function deve receber dois argumentos: data_inicial e data_final.

Caso a data_final não seja informada, a function deve automaticamente considerar a data atual do sistema.

Essa function será usada para calcular o tempo de casa de cada funcionário.

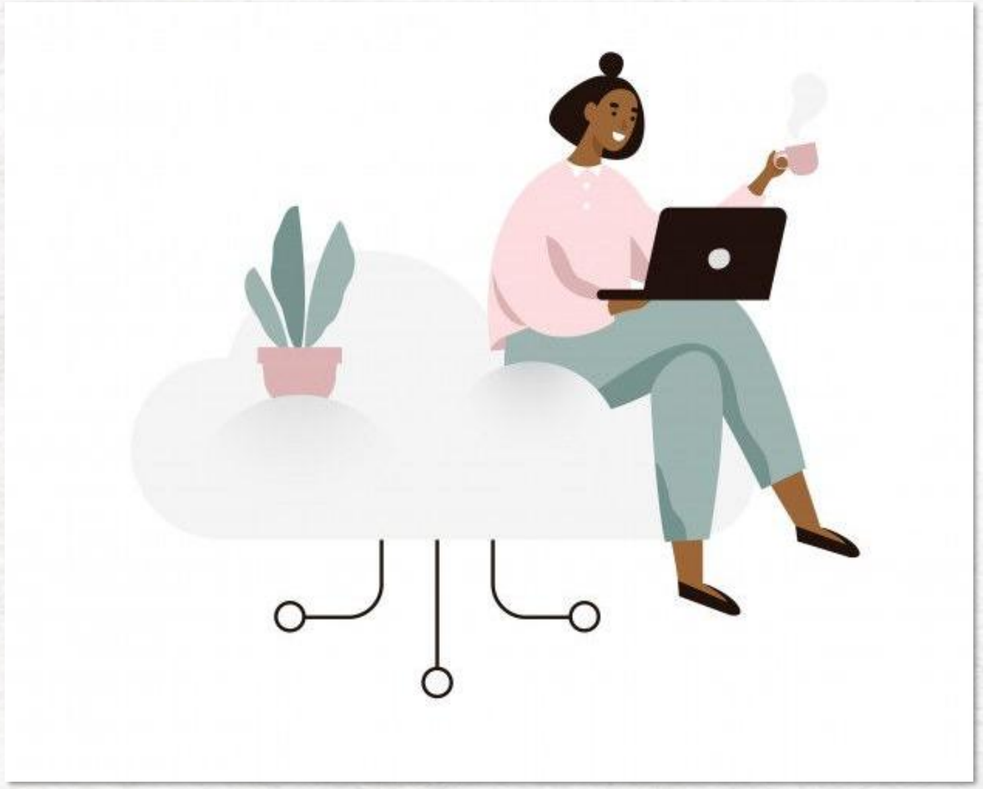
Obs.: a função DATEDIFF não é suficiente para resolver este problema.

2 Questão 2

Crie uma function que calcula a bonificação de cada funcionário (5% a mais em relação ao BaseRate).

Porém, tome cuidado! Nem todos os funcionários deverão receber bônus...

EXERCÍCIOS



3 Questão 3

Crie uma Function que retorna uma tabela. Esta function deverá receber como parâmetro o gênero do cliente e retornar todos os clientes que são do gênero informado na function.

Observe que esta function será utilizada particularmente na tabela DimCustomer.

4 Questão 4

Crie uma Function que retorna uma tabela-resumo com o total de produtos por cores.

Sua function deve receber 1 argumento, em que será possível especificar de qual marca você deseja o resumo.

GABARITOS

1

Questão 1

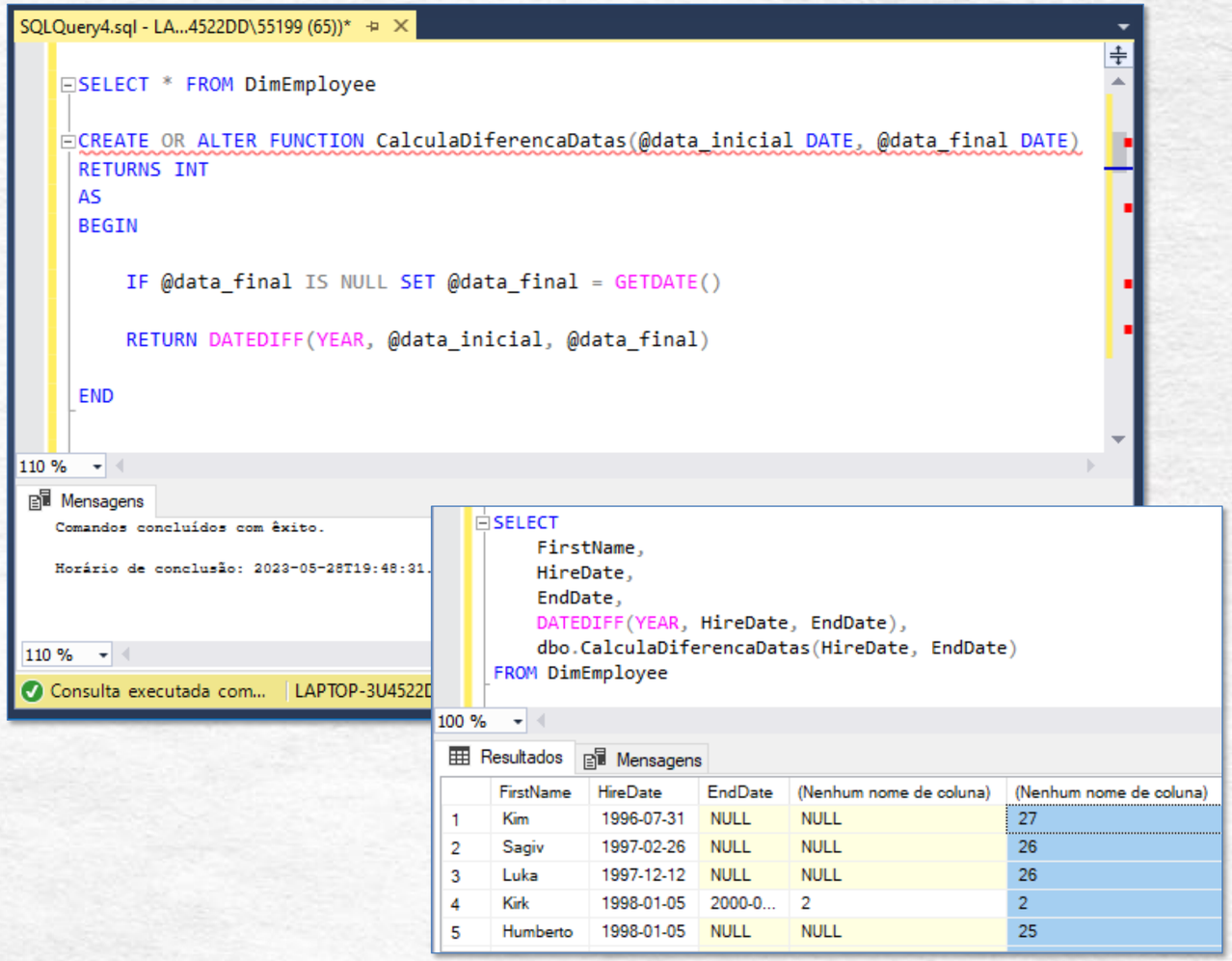
Crie uma Function que calcule o tempo (em anos) entre duas datas.

Essa function deve receber dois argumentos: data_inicial e data_final.

Caso a data_final não seja informada, a function deve automaticamente considerar a data atual do sistema.

Essa function será usada para calcular o tempo de casa de cada funcionário.

Obs.: a função DATEDIFF não é suficiente para resolver este problema.



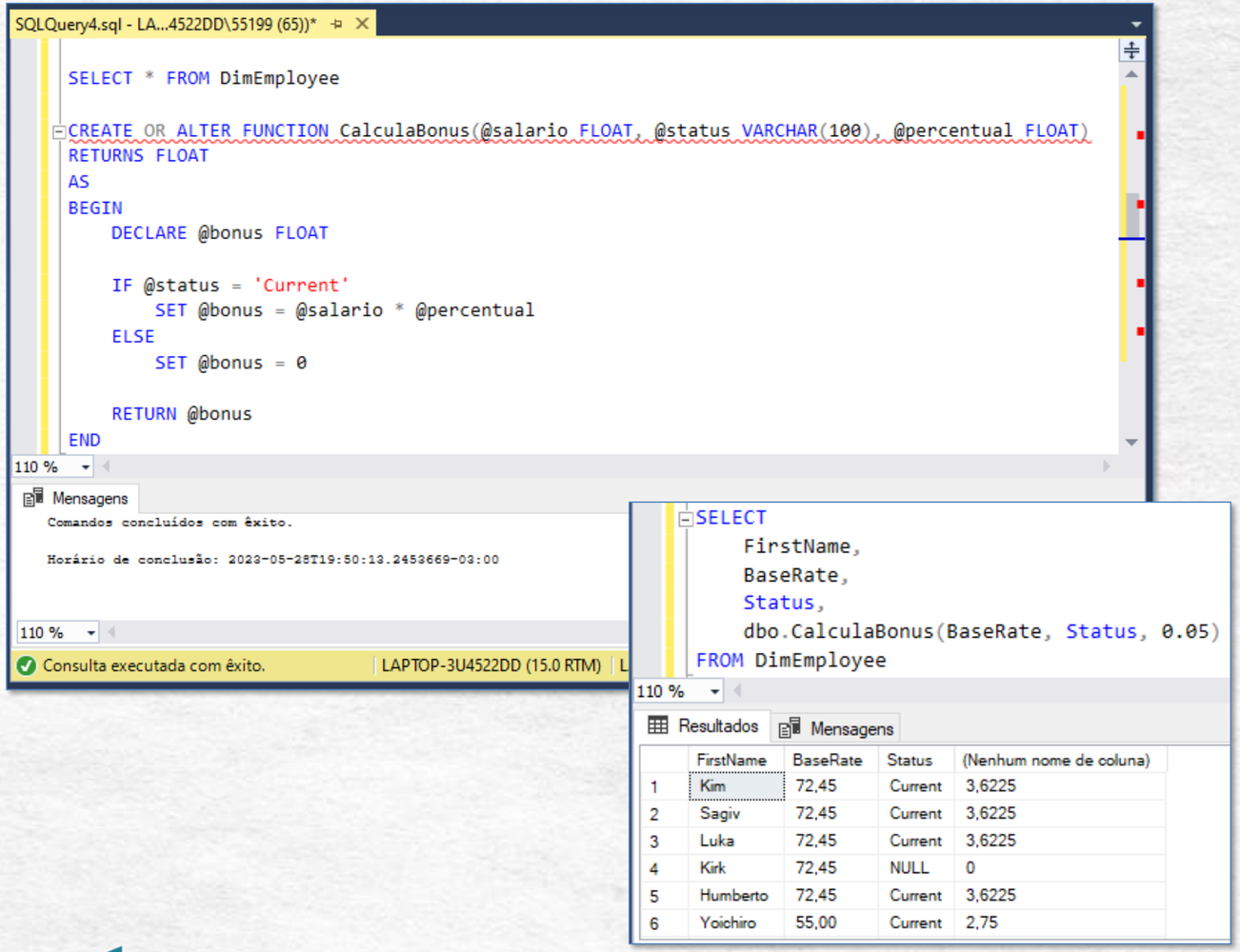
GABARITOS

2

Questão 2

Crie uma function que calcula a bonificação de cada funcionário (5% a mais em relação ao BaseRate).

Porém, tome cuidado! Nem todos os funcionários deverão receber bônus...



```
SELECT * FROM DimEmployee

CREATE OR ALTER FUNCTION CalculaBonus(@salario FLOAT, @status VARCHAR(100), @percentual FLOAT)
RETURNS FLOAT
AS
BEGIN
    DECLARE @bonus FLOAT

    IF @status = 'Current'
        SET @bonus = @salario * @percentual
    ELSE
        SET @bonus = 0

    RETURN @bonus
END
```

```
Comandos concluídos com êxito.

Horário de conclusão: 2023-05-28T19:50:13.2453669-03:00
```

```
SELECT
    FirstName,
    BaseRate,
    Status,
    dbo.CalculaBonus(BaseRate, Status, 0.05)
FROM DimEmployee
```

	FirstName	BaseRate	Status	(Nenhum nome de coluna)
1	Kim	72,45	Current	3,6225
2	Sagiv	72,45	Current	3,6225
3	Luka	72,45	Current	3,6225
4	Kirk	72,45	NULL	0
5	Humberto	72,45	Current	3,6225
6	Yoichiro	55,00	Current	2,75

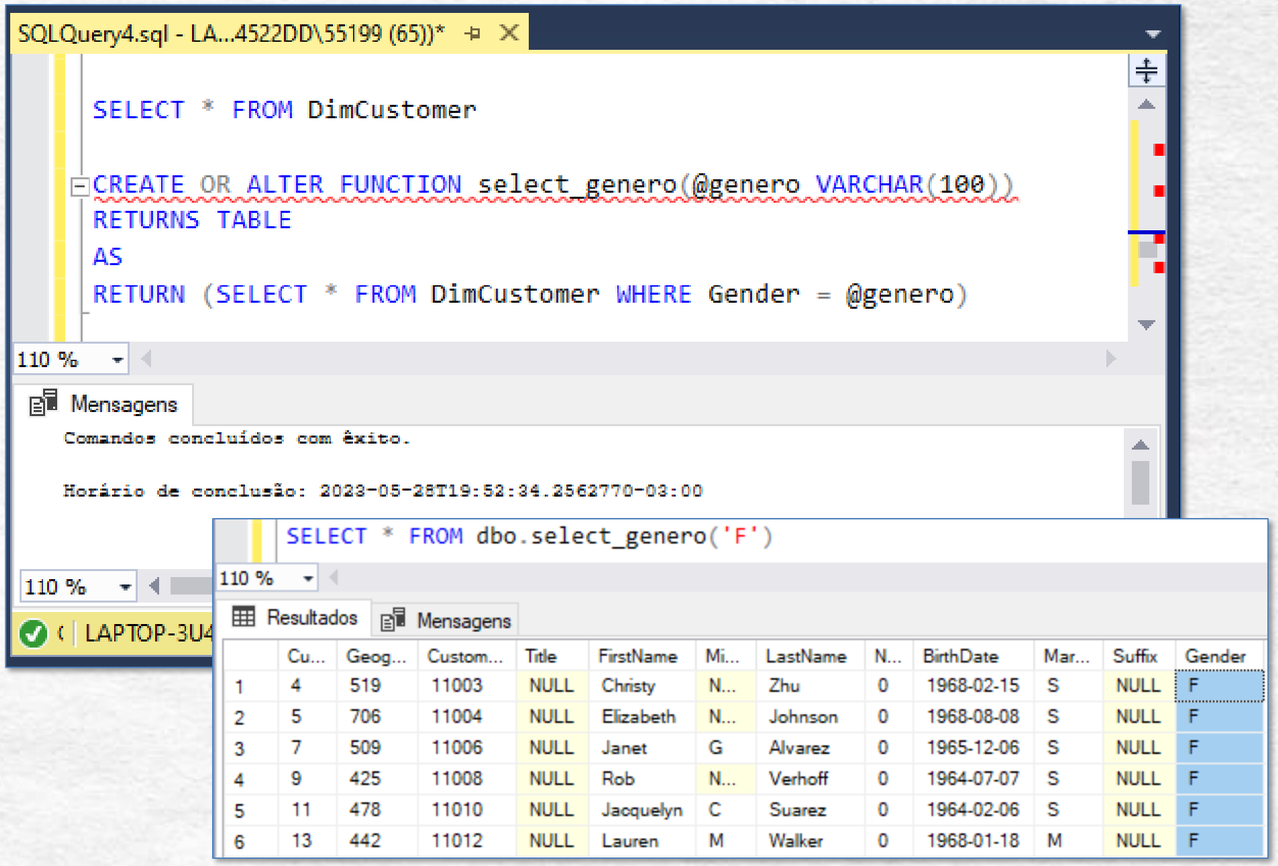
GABARITOS

3

Questão 3

Crie uma Function que retorna uma tabela. Esta function deverá receber como parâmetro o gênero do cliente e retornar todos os clientes que são do gênero informado na function.

Observe que esta function será utilizada particularmente na tabela DimCustomer.



	Cu...	Geog...	Custom...	Title	FirstName	Mi...	LastName	N...	BirthDate	Mar...	Suffix	Gender
1	4	519	11003	NULL	Christy	N...	Zhu	0	1968-02-15	S	NULL	F
2	5	706	11004	NULL	Elizabeth	N...	Johnson	0	1968-08-08	S	NULL	F
3	7	509	11006	NULL	Janet	G	Alvarez	0	1965-12-06	S	NULL	F
4	9	425	11008	NULL	Rob	N...	Verhoff	0	1964-07-07	S	NULL	F
5	11	478	11010	NULL	Jacquelyn	C	Suarez	0	1964-02-06	S	NULL	F
6	13	442	11012	NULL	Lauren	M	Walker	0	1968-01-18	M	NULL	F

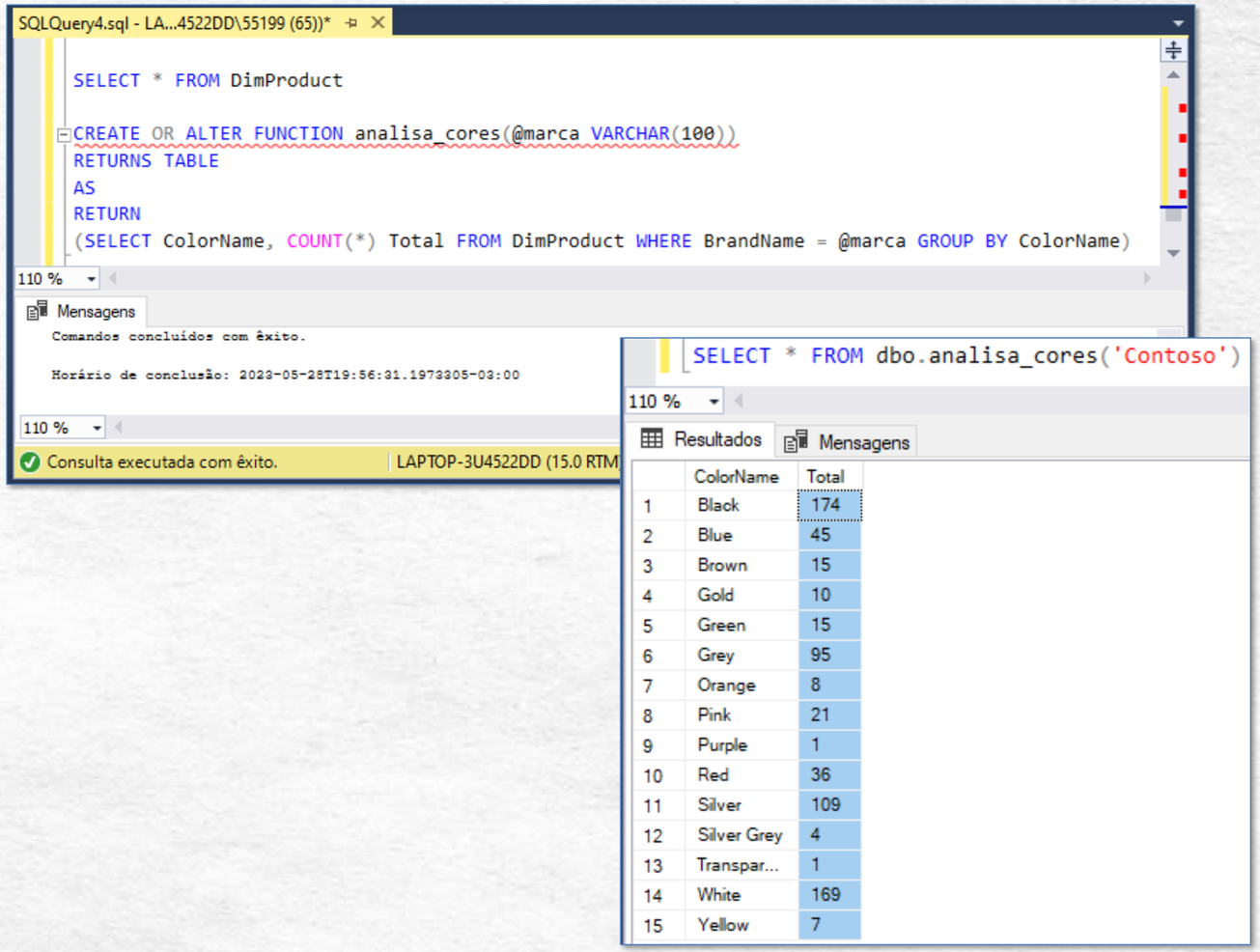
GABARITOS

4

Questão 4

Crie uma Function que retorna uma tabela-resumo com o total de produtos por cores.

Sua function deve receber 1 argumento, em que será possível especificar de qual marca você deseja o resumo.



MÓDULO 22

SQL Procedures

O QUE VEREMOS NESTE MÓDULO

MÓDULO 22 PROCEDURES

01	O que são PROCEDURES?	611
02	Como criar PROCEDURES	613
03	Como criar PROCEDURES SEM PARÂMETROS	614
04	Como criar PROCEDURES COM UM PARÂMETRO	617
05	Como criar PROCEDURES COM MAIS DE UM PARÂMETRO	620
06	Como criar PROCEDURES COM PARÂMETRO DEFAULT	623
07	Como criar PROCEDURES COMPLEXAS	628
08	Como excluir uma PROCEDURE	641
09	FUNCTIONS vs. PROCEDURES	642
10	EXERCÍCIOS	643
11	GABARITOS	646

O que são PROCEDURES?

Uma PROCEDURE é um bloco de código que possui um nome e pode ser armazenado no banco de dados. Ele pode incluir uma série de comandos SQL para executar alguma tarefa.

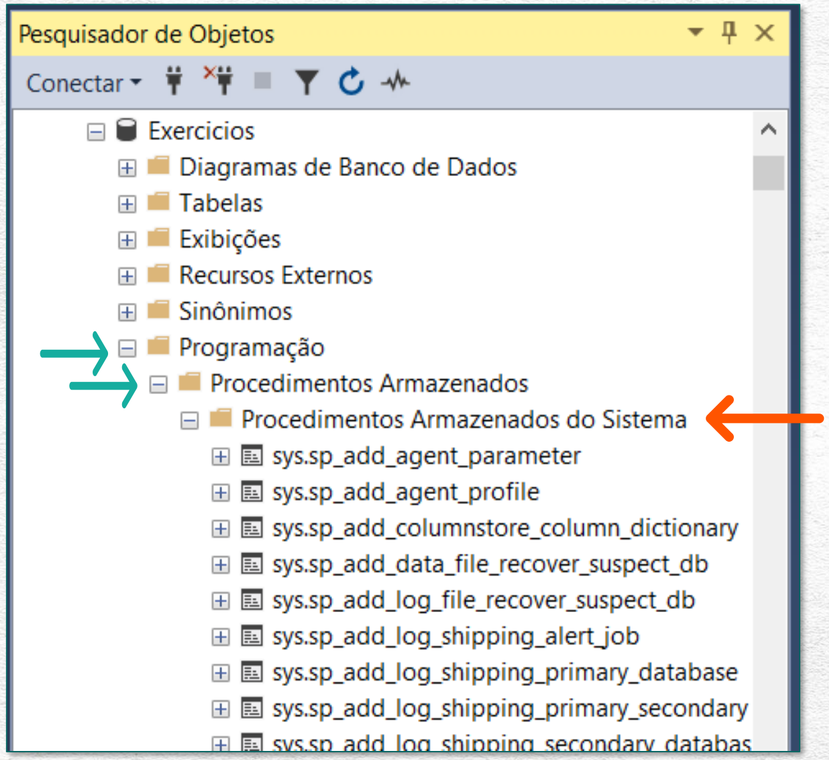
Procedures são usadas para fazer tarefas repetitivas que não são possíveis em queries do SQL ou que dariam muito trabalho.

Elas podem incluir estruturas de controle e comandos.

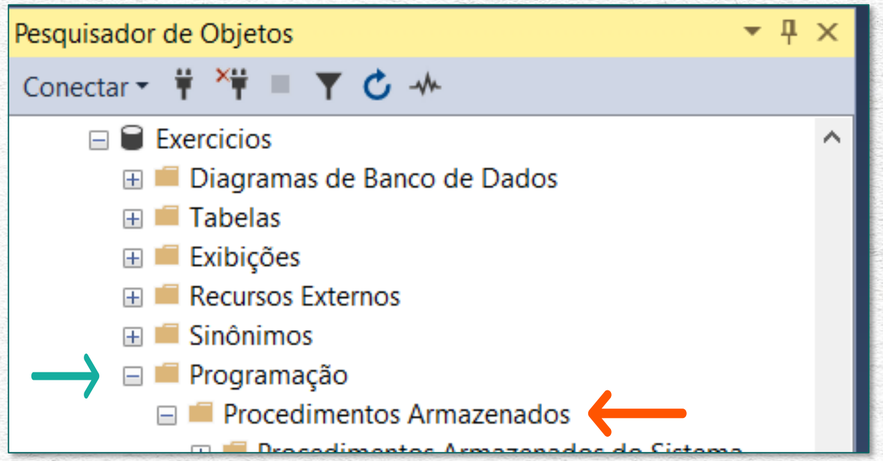
Uma Procedure pode ou não aceitar parâmetros de entrada.

O que são PROCEDURES?

Podemos visualizar as procedures do sistema na pasta Programação > Procedimentos Armazenados > Procedimentos Armazenados do Sistema:



Conforme formos criando nossas próprias procedures, elas ficarão salvas na pasta Programação > Procedimentos Armazenados:



Como criar PROCEDURES

Existem duas formas de se criar uma PROCEDURE: **sem parâmetros** e **com parâmetros**.

Na PROCEDURE sem parâmetros, todas as informações necessárias para sua execução serão atribuídas dentro da própria PROCEDURE.

Já na PROCEDURE com parâmetros, essas informações serão fornecidas pelo usuário.

Vejamos a seguir exemplos de cada uma delas.

Como criar PROCEDURES SEM PARÂMETROS

Vamos iniciar com um exemplo bem simples de como criar uma PROCEDURE SEM PARÂMETROS.

Suponhamos que a gente queira criar uma procedure para armazenar uma consulta à tabela dGerente.
O código ficaria assim:

```
/* Exemplo 1. Crie uma Procedure que
executa um SELECT simples (sem parâmetros). */

CREATE PROCEDURE prOrdenaGerentes
AS
BEGIN

    SELECT
        id_gerente,
        nome_gerente,
        salario
    FROM dGerente
    ORDER BY salario DESC

END
```

Vejamos seu passo a passo na próxima página.

Como criar PROCEDURES SEM PARÂMETROS

1. Iniciamos a procedure com o comando CREATE PROCEDURE e damos um nome a ela;
2. Abrimos o bloco de execução com o comando BEGIN;
3. Dentro do bloco de execução, passamos todas as instruções da procedure. No caso, o nosso SELECT à tabela dGerente;
4. Finalizamos o bloco de execução e a procedure com o comando END.

```
CREATE PROCEDURE prOrdenaGerentes 1
AS
BEGIN 2

    SELECT 3
        id_gerente,
        nome_gerente,
        salario
    FROM dGerente
    ORDER BY salario DESC

END 4
```

Mensagens

```
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-04T23:24:13.0630035-03:00
```

Como criar PROCEDURES SEM PARÂMETROS

Para executar nossa procedure, utilizamos o seguinte comando:

```
EXECUTE prOrdenaGerentes
```

Executando-a, temos o seguinte resultado:

Resultados | Mensagens

	id_gerente	nome_gerente	salario
1	4	Otávio Costa	11000
2	2	Mariana Padilha	9900
3	3	Nathália Santos	7200
4	1	Lucas Sampaio	6700

Agora, toda vez que precisarmos visualizar o resultado dessa mesma consulta, basta utilizarmos o comando acima e executar a procedure prOrdenaGerentes, em vez de ter que recriar todo o código novamente.

Como criar PROCEDURES COM UM PARÂMETRO

Vamos agora avançar um pouquinho aprendendo a criar uma PROCEDURE COM UM PARÂMETRO.

Suponhamos que a gente queira criar uma procedure para armazenar uma consulta à tabela dCliente que retorne apenas os clientes ou do sexo masculino, ou do sexo feminino, a depender do que for informado pelo usuário.
O código ficaria conforme ao lado.

Repare que iniciamos com a junção dos comandos CREATE OR ALTER PROCEDURE. Como explicado no módulo de Functions, fazendo desta forma, o sistema será capaz de verificar:

A. Se a procedure não existe, então ele criará uma nova procedure por meio do comando CREATE;
B. Se a procedure já existe, assim ele alterará a procedure já existente por meio do comando ALTER.

Vejamos seu passo a passo na próxima página.

```
/* Exemplo 2. Crie uma Procedure que execute um SELECT que
receba um parâmetro de entrada para filtrar a tabela
dClientes de acordo com o gênero informado. */

CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX))
AS
BEGIN

    SELECT
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen


END
```

Como criar PROCEDURES COM UM PARÂMETRO

1. Iniciamos a procedure com o comando CREATE OR ALTER PROCEDURE, a nomeamos, e declaramos o nome e o tipo de dado do parâmetro que receberá a informação de gênero que será passada pelo usuário;

2. Abrimos o bloco de execução com o comando BEGIN;

3. Dentro do bloco de execução, passamos todas as instruções da procedure. No caso, o nosso SELECT à tabela dCliente (observe que no filtro que fizemos com o comando WHERE, em vez de estabelecer que genero = 'M' ou genero = 'F', informamos o nome do parâmetro que criamos na primeira linha do nosso código, junto ao comando CREATE OR ALTER PROCEDURE: @gen);

5. Finalizamos o bloco de execução e a procedure com o comando END.

```
CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX))  1
AS
BEGIN  2

    SELECT  3
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen  4


END  5
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-04T23:43:15.1091171-03:00
```

Como criar PROCEDURES COM UM PARÂMETRO

Para executar nossa procedure, utilizamos o comando EXECUTE seguido do nome da procedure e da informação que deverá ser passada ao parâmetro @gen (no caso, ‘M’ para retornar todos os clientes do gênero masculino ou ‘F’ para retornar todas as clientes do gênero feminino):

```
EXECUTE prListaClientes 'M'
```

Resultados | Mensagens

	nome_cliente	genero	data_de_nascimento	cpf
1	André Martins	M	1989-02-12	839.283.190-00
2	Diego Cardoso	M	1994-10-11	192.371.081-17
3	Eduardo Pereira	M	1988-11-09	193.174.192-82
4	Gustavo Barbosa	M	1993-06-27	240.174.171-76
5	Igor Castro	M	1989-08-21	184.148.102-29

OU

```
EXECUTE prListaClientes 'F'
```

Resultados | Mensagens

	nome_cliente	genero	data_de_nascimento	cpf
1	Bárbara Campos	F	1992-05-07	351.391.410-02
2	Carol Freitas	F	1985-04-23	139.274.921-12
3	Fabiana Silva	F	1989-09-02	231.298.471-98
4	Helen Viana	F	1990-02-11	193.129.183-01
5	Juliana Pires	F	1991-01-13	416.209.192-47

Agora, toda vez que precisarmos visualizar o resultado dessa mesma consulta, basta utilizarmos o comando acima e executar a procedure prListaClientes, em vez de ter que recriar todo o código novamente.

Como criar PROCEDURES COM MAIS DE UM PARÂMETRO

Avancemos novamente mais um pouquinho para aprender como podemos criar uma PROCEDURE COM MAIS DE UM PARÂMETRO.

Vamos imaginar que a gente queira, lá na nossa procedure prListasClientes, além de poder filtrar nossos clientes por gênero, também escolher o ano de sua data de nascimento. Vamos precisar incluir mais um parâmetro, então, concorda?

Para isso, a gente precisa alterar alguns detalhes no nosso código, que vai ficar igual ao lado:

```
/* Exemplo 3. Crie uma Procedure que executa um SELECT que recebe
um parâmetro de entrada para filtrar a tabela dClientes de acordo
com o gênero informado e ano de nascimento informado. */

CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX), @ano INT)
AS
BEGIN

    SELECT
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano

END
```

Vejamos seu passo a passo na próxima página.

Como criar PROCEDURES COM MAIS DE UM PARÂMETRO

1. Iniciamos a procedure com o comando CREATE OR ALTER PROCEDURE, a nomeamos, e declaramos os nomes e os tipos de dados dos parâmetros que receberão as informações de gênero e ano de nascimento que serão passados pelo usuário;

2. Abrimos o bloco de execução com o comando BEGIN;

3. Dentro do bloco de execução, passamos todas as instruções da procedure. No caso, o nosso SELECT à tabela dCliente (observe que no filtro que fizemos com o comando WHERE, em vez de estabelecer que genero = 'M' ou genero = 'F', informamos o nome do parâmetro que criamos na primeira linha do nosso código, junto ao comando CREATE OR ALTER PROCEDURE: @gen. O mesmo vale para o filtro YEAR(data_de_nascimento), pois não informamos um ano qualquer, mas sim o parâmetro que criamos: @ano);

4.

5. Finalizamos o bloco de execução e a procedure com o comando END.

```
CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX), @ano INT)  1
AS
BEGIN  2

    SELECT  3
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano  4

END  5
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-05T18:48:44.5279823-03:00
```

Como criar PROCEDURES COM MAIS DE UM PARÂMETRO

Para executar nossa procedure, utilizamos o comando EXECUTE seguido do nome da procedure e das informações que deverão ser passadas aos parâmetros @gen e @ano. A título de exemplo, informamos 'M' para o parâmetro @gen e 1989 para o parâmetro @ano; assim retornaremos todos os clientes do gênero masculino que nasceram no ano de 1989) :

```
EXECUTE prListaClientes 'M', 1989
```

Resultados | Mensagens

	nome_cliente	genero	data_de_nascimento	cpf
1	André Martins	M	1989-02-12	839.283.190-00
2	Igor Castro	M	1989-08-21	184.148.102-29

Agora, toda vez que precisarmos visualizar o resultado dessa mesma consulta, basta utilizarmos o comando acima e executar a procedure prListaClientes, em vez de ter que recriar todo o código novamente.

Como criar PROCEDURES COM PARÂMETRO DEFAULT

E continuemos avançando! Agora, vamos ver como podemos passar um valor padrão (default) para um parâmetro. Esse valor default será sempre utilizado caso o usuário não informe nenhum valor para o parâmetro em questão.

Vamos imaginar que a gente queira alterar novamente a nossa procedure prListasClientes, para podermos incluir o valor “M” como default do parâmetro @gen. Assim, caso o usuário não informe nenhum valor, sempre retornaremos os clientes do gênero masculino na consulta efetuada pela procedure.

Para isso, a gente precisa alterar um detalhe no nosso código, que vai ficar igual ao lado:

```sql
/* Exemplo 3. Crie uma Procedure que executa um SELECT que recebe
um parâmetro de entrada para filtrar a tabela dClientes de acordo
com o gênero informado e ano de nascimento informado. */

CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX)='M', @ano INT)
AS
BEGIN

    SELECT
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano

END
```

Vejamos seu passo a passo na próxima página.

Como criar PROCEDURES COM PARÂMETRO DEFAULT

1. Iniciamos a procedure com o comando CREATE OR ALTER PROCEDURE, a nomeamos, e declaramos os nomes e os tipos de dados dos parâmetros que receberão as informações de gênero e ano de nascimento que serão passados pelo usuário. Repare que, para o parâmetro @gen, estabelecemos o valor default “M”. Assim, caso o usuário não informe o gênero, a consulta executada dentro da procedure, por padrão, retornará os clientes do gênero masculino;

2. Abrimos o bloco de execução com o comando BEGIN;

3. Dentro do bloco de execução, passamos todas as instruções da procedure. No caso, o nosso SELECT à tabela dCliente;

4. Finalizamos o bloco de execução e a procedure com o comando END.

```
CREATE OR ALTER PROCEDURE prListaClientes(@gen VARCHAR(MAX)='M', @ano INT) 1
AS
BEGIN 2

    SELECT 3
        nome_cliente,
        genero,
        data_de_nascimento,
        cpf
    FROM dCliente
    WHERE genero = @gen AND YEAR(data_de_nascimento) = @ano

END 4
```

Mensagens

```
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-05T19:16:13.5956446-03:00
```

Como criar PROCEDURES COM PARÂMETRO DEFAULT

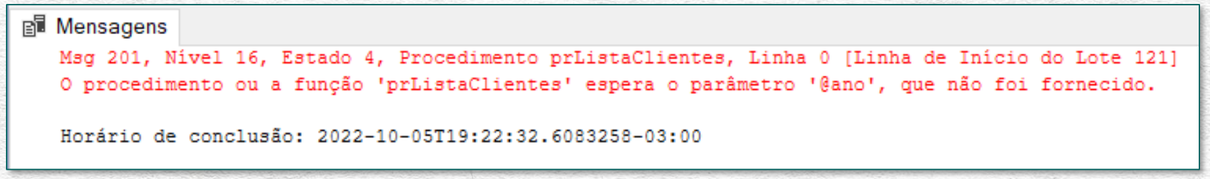
Para executar nossa procedure, utilizamos o comando EXECUTE seguido do nome da procedure e das informações que deverão ser passadas aos parâmetros @gen e @ano.

Porém, agora, como definimos um valor default para o parâmetro @gen, não somos obrigados a informar novamente o valor “M” caso a gente queira retornar clientes do gênero masculino. Em um caso assim, basta informarmos o ano de nascimento. Só precisamos informar o gênero se quisermos que apareçam as clientes do gênero feminino (F) em vez dos clientes do gênero masculino (M).

Então, se quisermos retornar os clientes do gênero masculino que nasceram no ano de 1989 é só escrever o código abaixo, certo?

```
EXECUTE prListaClientes 1989
```

Errado!
Repare que, se executarmos o código acima, o sistema retornará o seguinte erro:



Como criar PROCEDURES COM PARÂMETRO DEFAULT

Isso acontece porque, como a nossa procedure prListaClientes possui dois parâmetros, quando informamos apenas um valor ao comando EXECUTE, o sistema não sabe para qual parâmetro ele deve passar aquele valor. Ele não entende que, uma vez que existe um parâmetro com um valor default, se informamos apenas um valor, este valor deve ser passado ao parâmetro que não tem valor default.

Então, para que o erro da página anterior não ocorra, temos que informar ao EXECUTE a qual parâmetro aquele valor se refere, desta forma:

```
EXECUTE prListaClientes @ano = 1989
```

Perceba que, agora, o sistema entendeu e retornou todos os clientes do gênero masculino que nasceram em 1989:

Resultados | Mensagens

	nome_cliente	genero	data_de_nascimento	cpf
1	André Martins	M	1989-02-12	839.283.190-00
2	Igor Castro	M	1989-08-21	184.148.102-29

Como criar PROCEDURES COM PARÂMETRO DEFAULT

E para retornar as clientes do gênero feminino que nasceram em 1989, por exemplo?

Neste caso, como teremos que, obrigatoriamente, informar valores tanto para o parâmetro @gen quanto para o parâmetro @ano, não faz diferença se informamos o nome do parâmetro no comando EXECUTE ou não:

```
EXECUTE prListaClientes 'F', 1989
```

OU

```
EXECUTE prListaClientes @gen = 'F', @ano = 1989
```

Resultados | Mensagens

	nome_cliente	genero	data_de_nascimento	cpf
1	Fabiana Silva	F	1989-09-02	231.298.471-98

Perceba que, agora, o sistema retornou todas as clientes do gênero feminino que nasceram em 1989.
Portanto, o parâmetro default passado dentro da procedure só é utilizado quando o usuário não informa nenhum valor para aquele parâmetro. Caso o usuário informe um valor diferente, é este valor que prevalece, não o default.

Como criar PROCEDURES COMPLEXAS

Agora, vamos ver um exemplo de uma procedure um pouco mais complexa que as anteriores.

Com esta nova procedure, nós poderemos efetuar cadastros de assinatura de novos contratos na tabela fContratos.

Como exemplo, ao final, executada a procedure, nós cadastraremos os seguintes dados informados pelo usuário para um novo contrato:

Gerente: Lucas Sampaio

Cliente: Gustavo Barbosa

Valor do Contrato: 5000

Como criar PROCEDURES COMPLEXAS

Repare novamente nos valores que cadastraremos:

Gerente: Lucas Sampaio
Cliente: Gustavo Barbosa
Valor do Contrato: 5000

Façamos uma consulta a nossa tabela fContratos:

Resultados | Mensagens

	id_contrato	data_assinatura	id_cliente	id_gerente	valor_contrato
1	1	2019-01-12	8	1	23000
2	2	2019-02-10	3	2	15500
3	3	2019-03-07	7	2	6500
4	4	2019-03-15	1	3	33000
5	5	2019-03-21	5	4	11100
6	6	2019-03-23	4	2	5500
7	7	2019-03-28	9	3	55000
8	8	2019-04-04	2	1	31000
9	9	2019-04-05	10	4	3400
10	10	2019-04-05	6	2	9200

Notou que ela não tem as colunas de nome do cliente e nome do gerente, somente as colunas de id_cliente e id_gerente?

Como, então, vamos conseguir cadastrar esse novo registro, se o usuário vai informar para gente exatamente os **nomes** do gerente e do cliente, não seus IDs?

Como criar PROCEDURES COMPLEXAS

Vamos ter que encontrar uma forma de extrair essas informações das tabelas dCliente e dGerente, pois lá temos seus nomes registrados. Veja:

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47

dCliente

Resultados | Mensagens

	id_gerente	nome_gerente	data_contratacao	salario
1	1	Lucas Sampaio	21/03/2015	6700
2	2	Mariana Padilha	10/01/2011	9900
3	3	Nathália Santos	03/10/2018	7200
4	4	Otávio Costa	18/04/2017	11000

dGerente

Como criar PROCEDURES COMPLEXAS

Como sabemos, as colunas de id_cliente e id_gerente das tabelas dCliente e dGerente são as chaves primárias dessas tabelas, às quais as chaves estrangeiras id_cliente e id_gerente da tabela fContratos fazem referência:

Resultados | Mensagens

	id_contrato	data_assinatura	id_cliente	id_gerente	valor_contrato
1	1	2019-01-12	8	1	23000
2	2	2019-02-10	3	2	15500
3	3	2019-03-07	7	2	6500
4	4	2019-03-15	1	3	33000
5	5	2019-03-21	5	4	11100
6	6	2019-03-23	4	2	5500
7	7	2019-03-28	9	3	55000
8	8	2019-04-04	2	1	31000
9	9	2019-04-05	10	4	3400
10	10	2019-04-05	6	2	9200

fContratos

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47

dCliente

Resultados | Mensagens

	id_gerente	nome_gerente	data_contratacao	salario
1	1	Lucas Sampaio	21/03/2015	6700
2	2	Mariana Padilha	10/01/2011	9900
3	3	Nathália Santos	03/10/2018	7200
4	4	Otávio Costa	18/04/2017	11000

dGerente

Como criar PROCEDURES COMPLEXAS

Portanto, se queremos que os nomes informados pelo usuário para cliente e gerente sejam considerados para cadastrarmos os IDs desses nomes corretamente na tabela fContratos, precisamos, por meio dos nomes informados pelo usuário, conseguir extrair os IDs das tabelas dCliente e dGerente correspondentes a esses nomes para poder salvá-los no registro do novo contrato assinado.

E é isso que a nossa nova procedure vai ser capaz de fazer. Para tanto, precisaremos seguir o seguinte passo a passo:

1º Passo: Definir as variáveis a serem utilizadas;

2º Passo: Armazenar em uma das variáveis definidas no 1º passo o valor de id_gerente da tabela dGerente de acordo com o nome do gerente informado pelo usuário;

3º Passo: Armazenar na outra variável definida no 1º passo o valor de id_cliente da tabela dCliente de acordo com o nome do cliente informado pelo usuário;

4º Passo: Armazenar a data da assinatura como sendo a data atual do sistema;

5º Passo: Utilizar o INSERT INTO para inserir os dados na tabela fContratos.

Como criar PROCEDURES COMPLEXAS

Observe o código que vamos criar:

```
CREATE OR ALTER PROCEDURE prRegistraContrato(@gerente VARCHAR(MAX), @cliente VARCHAR(MAX), @valor FLOAT)
AS
BEGIN

    DECLARE
        @vIdGerente INT,
        @vIdCliente INT

    SELECT
        @vIdGerente = id_gerente
    FROM dGerente
    WHERE nome_gerente = @gerente

    SELECT
        @vIdCliente = id_cliente
    FROM dCliente
    WHERE nome_cliente = @cliente

    INSERT INTO fContratos(data_assinatura, id_cliente, id_gerente, valor_contrato) VALUES
        (GETDATE(), @vIdCliente, @vIdGerente, @valor)

    PRINT 'Contrato registrado com sucesso!'

END
```

Vejamos seu passo a passo nas próximas páginas.

Como criar PROCEDURES COMPLEXAS

1. Iniciamos a procedure com o comando CREATE OR ALTER PROCEDURE, a nomeamos, e declaramos os nomes e os tipos de dados dos parâmetros que receberão as informações de nome do gerente, nome do cliente e valor do contrato que serão informados pelo usuário;

2. Abrimos o bloco de execução com o comando BEGIN;

3. Dentro do bloco de execução, declaramos as variáveis que vamos precisar. No caso, utilizaremos duas: uma para armazenar o id_gerente da tabela dGerente que estiver associado ao nome do gerente informado pelo usuário; e outra para armazenar o id_cliente da tabela dCliente que estiver associado ao nome do cliente informado pelo usuário;

```
CREATE OR ALTER PROCEDURE prRegistraContrato(@gerente VARCHAR(MAX), @cliente VARCHAR(MAX), @valor FLOAT) 1
AS
BEGIN 2

    DECLARE 3
        @vIdGerente INT,
        @vIdCliente INT
```

Como criar PROCEDURES COMPLEXAS

Continuando o bloco de execução, vamos agora incluir dois SELECTs:

4 O primeiro vai extrair o id_gerente da tabela dGerente de acordo com o nome_gerente informado pelo usuário (parâmetro @gerente), e armazenará esse id_gerente na variável que criamos na página anterior: @vIdGerente;

5 Já o segundo SELECT extrairá o id_cliente da tabela dCliente de acordo com o nome_cliente informado pelo usuário (parâmetro @cliente), e armazenará esse id_cliente na outra variável que criamos na página anterior: @vIdCliente;

```
SELECT 4
    @vIdGerente = id_gerente
FROM dGerente
WHERE nome_gerente = @gerente

SELECT 5
    @vIdCliente = id_cliente
FROM dCliente
WHERE nome_cliente = @cliente
```

Como criar PROCEDURES COMPLEXAS

6 Em seguida, inserimos este novo registro na tabela fContratos, puxando a data atual (GETDATE()) para a coluna data_assinatura, o valor armazenado na variável @vIdCliente para a coluna id_cliente, o valor armazenado na variável @vIdGerente para a coluna id_gerente, e o parâmetro @valor (informado pelo usuário) para a coluna valor_contrato;

7 Por fim, imprimimos na tela a mensagem “Contrato registrado com sucesso!”;

8 Finalizamos o bloco de execução e a procedure com o comando END.

```
    INSERT INTO fContratos(data_assinatura, id_cliente, id_gerente, valor_contrato) VALUES 6
        (GETDATE(), @vIdCliente, @vIdGerente, @valor)

    PRINT 'Contrato registrado com sucesso!' 7

END 8
```

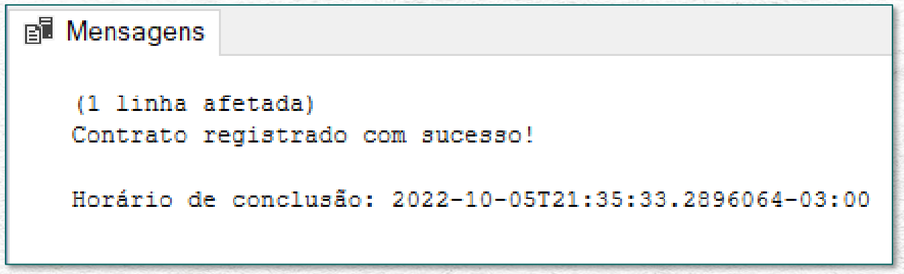
```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-05T21:31:43.5465628-03:00
```

Como criar PROCEDURES COMPLEXAS

Ao executarmos o comando EXECUTE com os valores informados pelo usuário...

```
EXECUTE prRegistraContrato @gerente='Lucas Sampaio', @cliente='Gustavo Barbosa', @valor=5000
```

... note que a mensagem configurada no PRINT da procedure prRegistraContrato foi impressa na tela, conforme esperado...



Como criar PROCEDURES COMPLEXAS

... e o novo registro foi devidamente incluído na tabela fContratos:

```
SELECT * FROM fContratos
```

Resultados | Mensagens

	id_contrato	data_assinatura	id_cliente	id_gerente	valor_contrato
1	1	2019-01-12	8	1	23000
2	2	2019-02-10	3	2	15500
3	3	2019-03-07	7	2	6500
4	4	2019-03-15	1	3	33000
5	5	2019-03-21	5	4	11100
6	6	2019-03-23	4	2	5500
7	7	2019-03-28	9	3	55000
8	8	2019-04-04	2	1	31000
9	9	2019-04-05	10	4	3400
10	10	2019-04-05	6	2	9200
11	11	2022-10-05	7	1	5000

Como criar PROCEDURES COMPLEXAS

Vejamos nas tabelas dCliente e dGerente se extraímos os IDs corretos.

Primeiramente, repare que o novo registro puxou:
a) Da tabela dCliente, o ID nº 7 para a coluna id_cliente da tabela fContratos;
b) Da tabela dGerente, o ID nº 1 para a coluna id_gerente da tabela fContratos:

	id_contrato	data_assinatura	id_cliente	id_gerente	valor_contrato
1	11	2022-10-05	7	1	5000

Portanto, na tabela dCliente o cliente de ID nº 7 tem que ser o Gustavo Barbosa; e na tabela dGerente o gerente de ID nº 1 precisa ser o Lucas Sampaio, pois foram estes os nomes informados pelo usuário aos parâmetros @cliente e @gerente no nosso EXECUTE, lembra-se? Veja:

```
EXECUTE prRegistraContrato @gerente='Lucas Sampaio', @cliente='Gustavo Barbosa', @valor=5000
```

Como criar PROCEDURES COMPLEXAS

Vamos ver se é isso mesmo:

```
SELECT * FROM dCliente
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47

```
SELECT * FROM dGerente
```

Resultados | Mensagens

	id_gerente	nome_gerente	data_contratacao	salario
1	1	Lucas Sampaio	21/03/2015	6700
2	2	Mariana Padilha	10/01/2011	9900
3	3	Nathália Santos	03/10/2018	7200
4	4	Otávio Costa	18/04/2017	11000

Tudo certo, nossa procedure prRegistraContrato está funcionando perfeitamente!

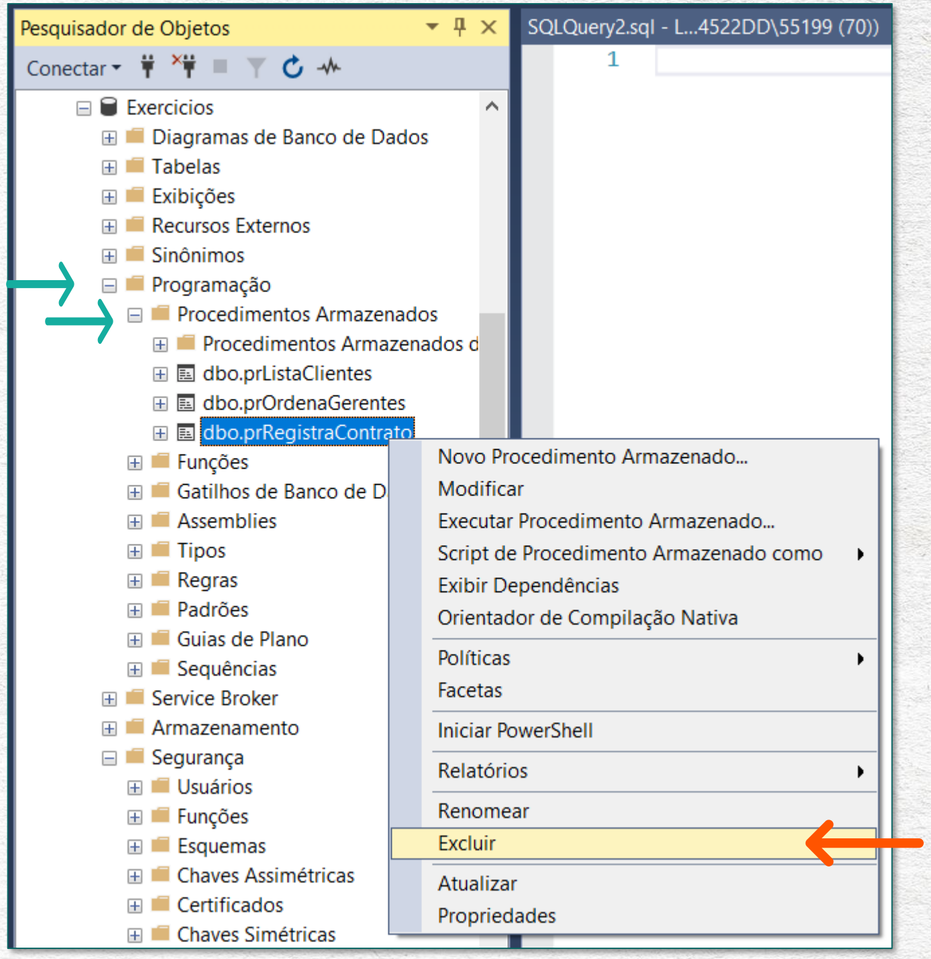
Como excluir uma PROCEDURE

Para excluir uma procedure é muito simples, basta executarmos o comando DROP PROCEDURE, conforme abaixo:

```
DROP PROCEDURE prRegistraContrato
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-05T23:09:46.9272582-03:00
```

Ou, manualmente, clicando com o botão direito sobre o nome da procedure armazenada na pasta Programação > Procedimentos Armazenados, e depois selecionar a opção Excluir (conforme ao lado):



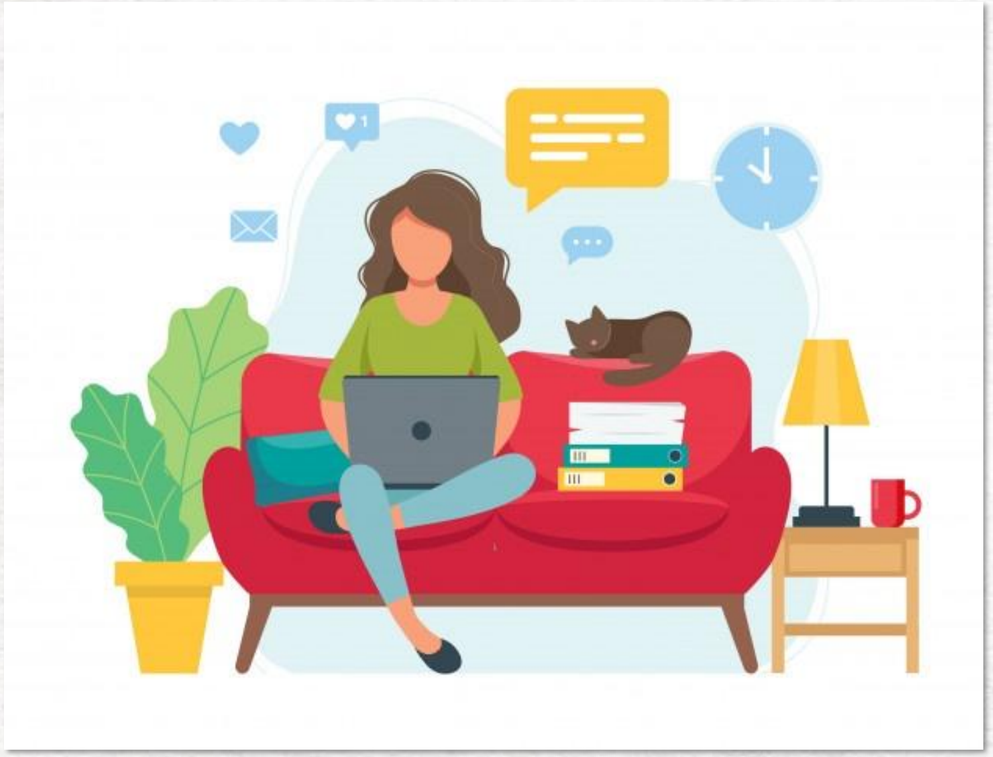
FUNCTIONS vs. PROCEDURES

Muita gente se pergunta quais são as diferenças entre FUNCTIONS e PROCEDURES.

Abaixo, listamos as principais delas:

DIFERENÇAS	PROCEDURES	FUNCTIONS
1	São usadas para executar um processo, uma sequência de comandos e blocos SQL.	São usadas para fazer cálculos.
2	Não podem ser “chamadas” dentro da cláusula SELECT.	Podem ser “chamadas” dentro da cláusula SELECT (desde que não contenham comandos SELECT).
3	Não precisam retornar valor.	Precisam retornar algum valor.

EXERCÍCIOS



1 Questão 1

Crie uma Procedure que resume o total de produtos por nome da categoria. Essa Procedure deve solicitar ao usuário qual marca deve ser considerada na análise.

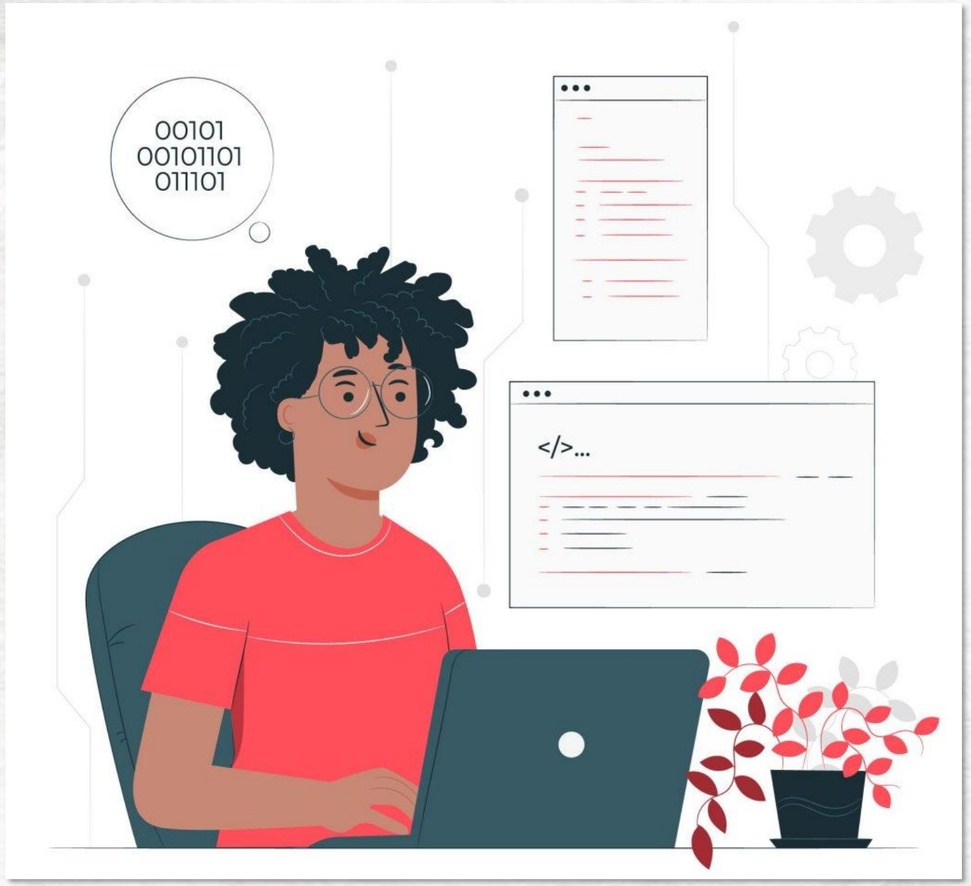
2 Questão 2

Crie uma Procedure que lista os top N clientes de acordo com a data de primeira compra. O valor de N deve ser um parâmetro de entrada da sua Procedure.

3 Questão 3

Crie uma Procedure que recebe 2 argumentos: MÊS (de 1 a 12) e ANO (1996 a 2003). Sua Procedure deve listar todos os funcionários que foram contratados no mês/ano informado.

EXERCÍCIOS



Observação:

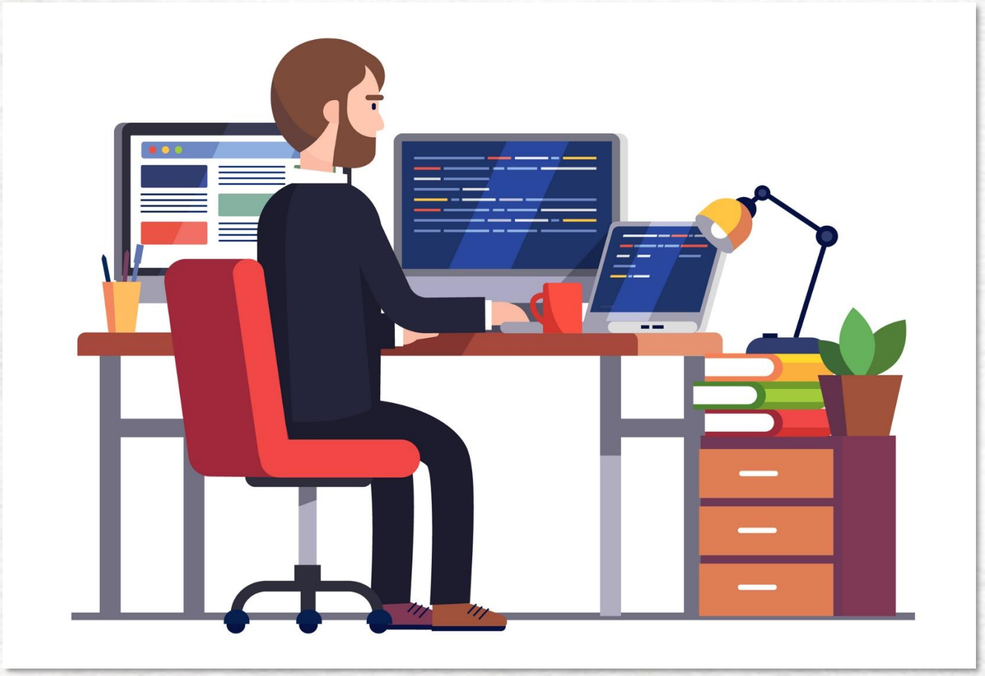
Para os exercícios 4, 5 e 6 utilize a tabela abaixo:

```
DROP DATABASE AlugaFacil
CREATE DATABASE AlugaFacil
USE AlugaFacil


CREATE TABLE Carro(
    id_carro INT,
    placa VARCHAR(100) NOT NULL,
    modelo VARCHAR(100) NOT NULL,
    tipo VARCHAR (100) NOT NULL,
    valor FLOAT NOT NULL,
    CONSTRAINT carro_id_carro_pk PRIMARY KEY(id_carro)
)

INSERT INTO Carro (id_carro, placa, modelo, tipo, valor) VALUES
    (1, 'CRU-1111', 'Chevrolet Cruze', 'Sedan', 140000),
    (2, 'ARG-2222', 'Fiat Argo', 'Hatch', 80000),
    (3, 'COR-3333', 'Toyota Corolla', 'Sedan', 170000),
    (4, 'TIG-4444', 'Caoa Chery Tiggo', 'SUV', 190000)

SELECT * FROM Carro
```

Resultados | Mensagens

	id_carro	placa	modelo	tipo	valor
1	1	CRU-1111	Chevrolet Cruze	Sedan	140000
2	2	ARG-2222	Fiat Argo	Hatch	80000
3	3	COR-3333	Toyota Corolla	Sedan	170000
4	4	TIG-4444	Caoa Chery Tiggo	SUV	190000

EXERCÍCIOS



4 Questão 4

Crie uma Procedure que insere uma nova linha na tabela Carro. Essa nova linha deve conter os seguintes dados:

- id = 5
- placa = GOL-5555
- modelo = Volkswagen Gol
- tipo = Hatch
- valor = 80000

5 Questão 5

Crie uma Procedure que altera o valor de venda de um carro. A Procedure deve receber como parâmetros o id_carro e o novo valor.

6 Questão 6

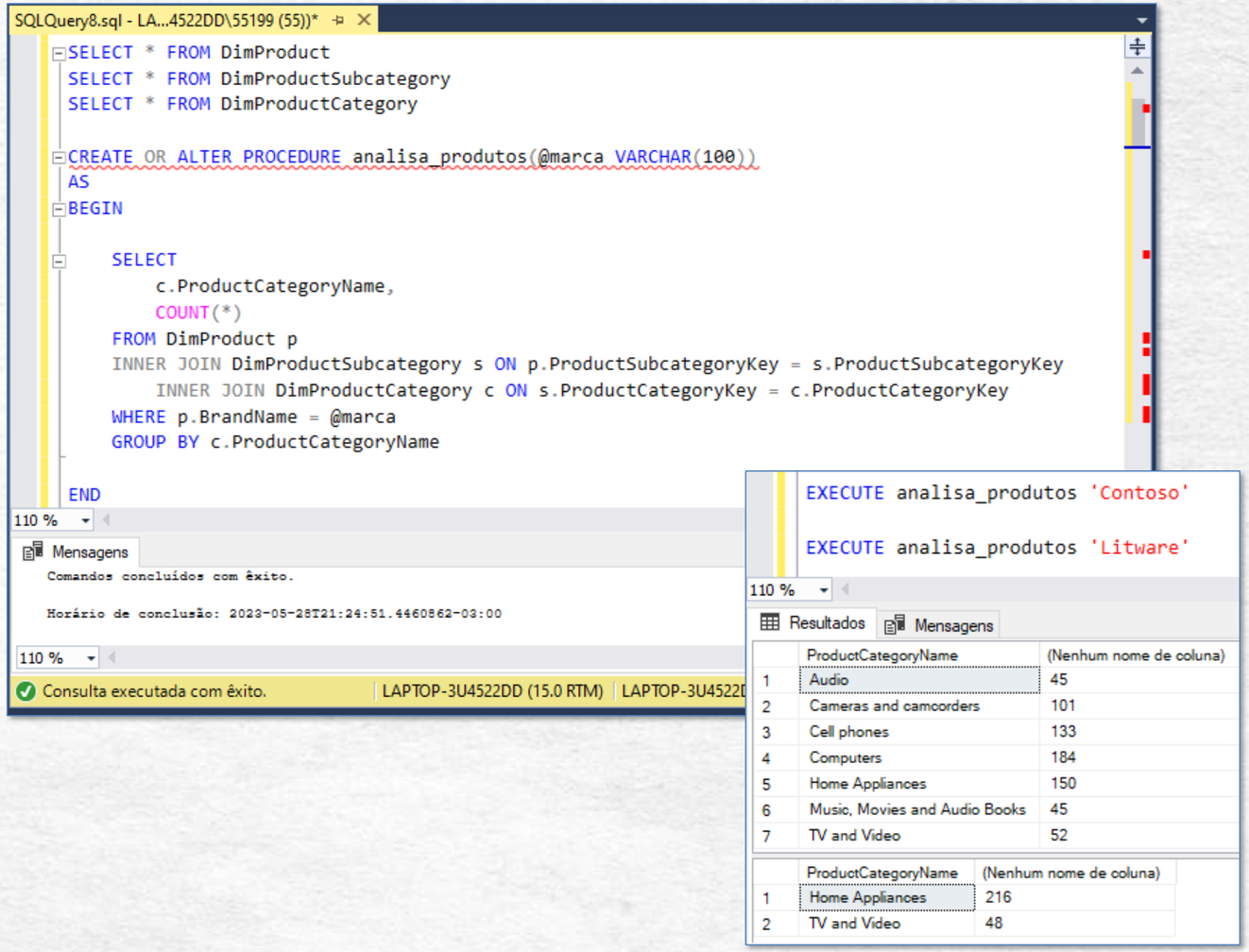
Crie uma Procedure que exclui um carro a partir do id informado.

GABARITOS

1

Questão 1

Crie uma Procedure que resume o total de produtos por nome da categoria. Essa Procedure deve solicitar ao usuário qual marca deve ser considerada na análise.

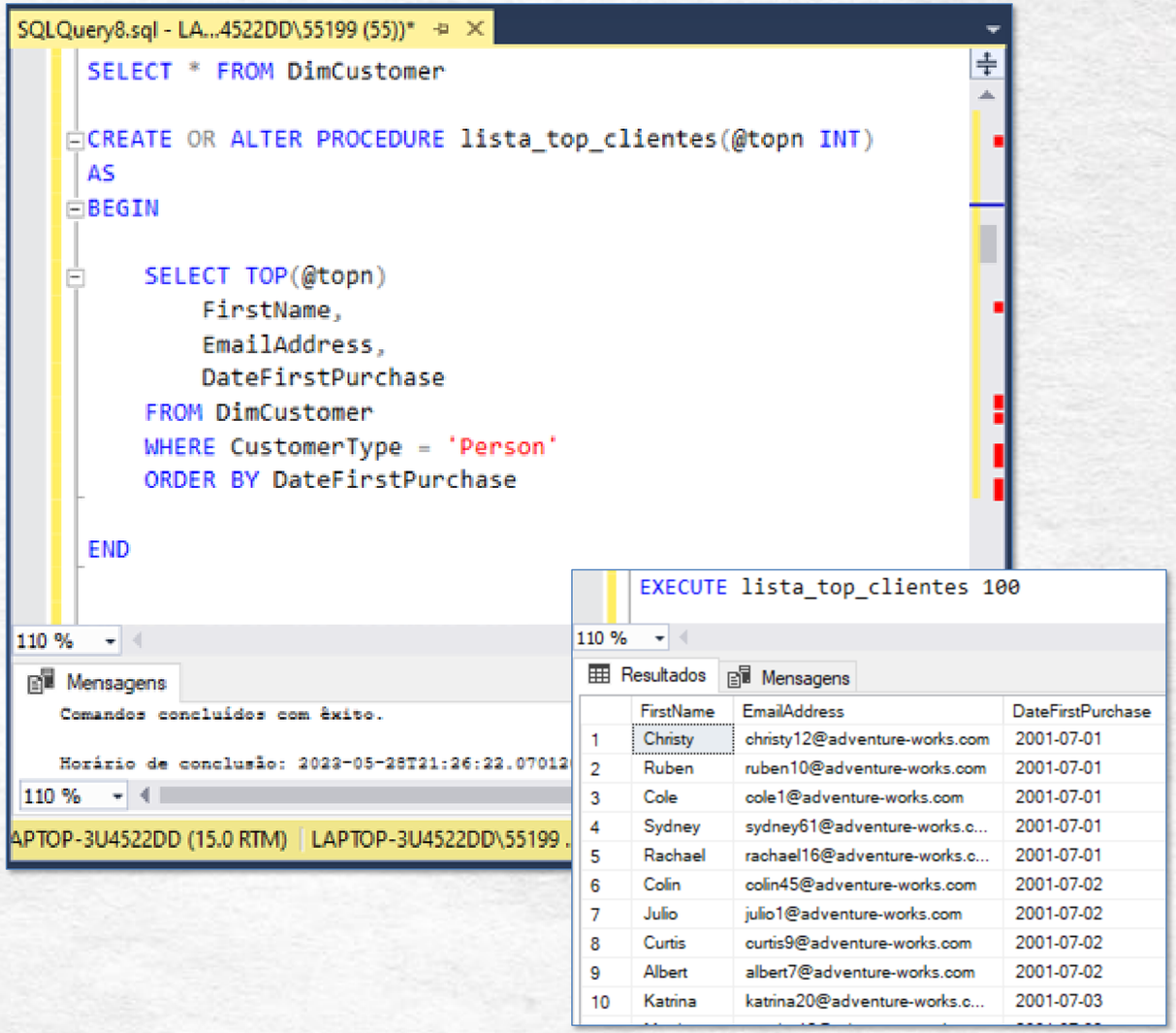


GABARITOS

2

Questão 2

Crie uma Procedure que lista os top N clientes de acordo com a data de primeira compra. O valor de N deve ser um parâmetro de entrada da sua Procedure.

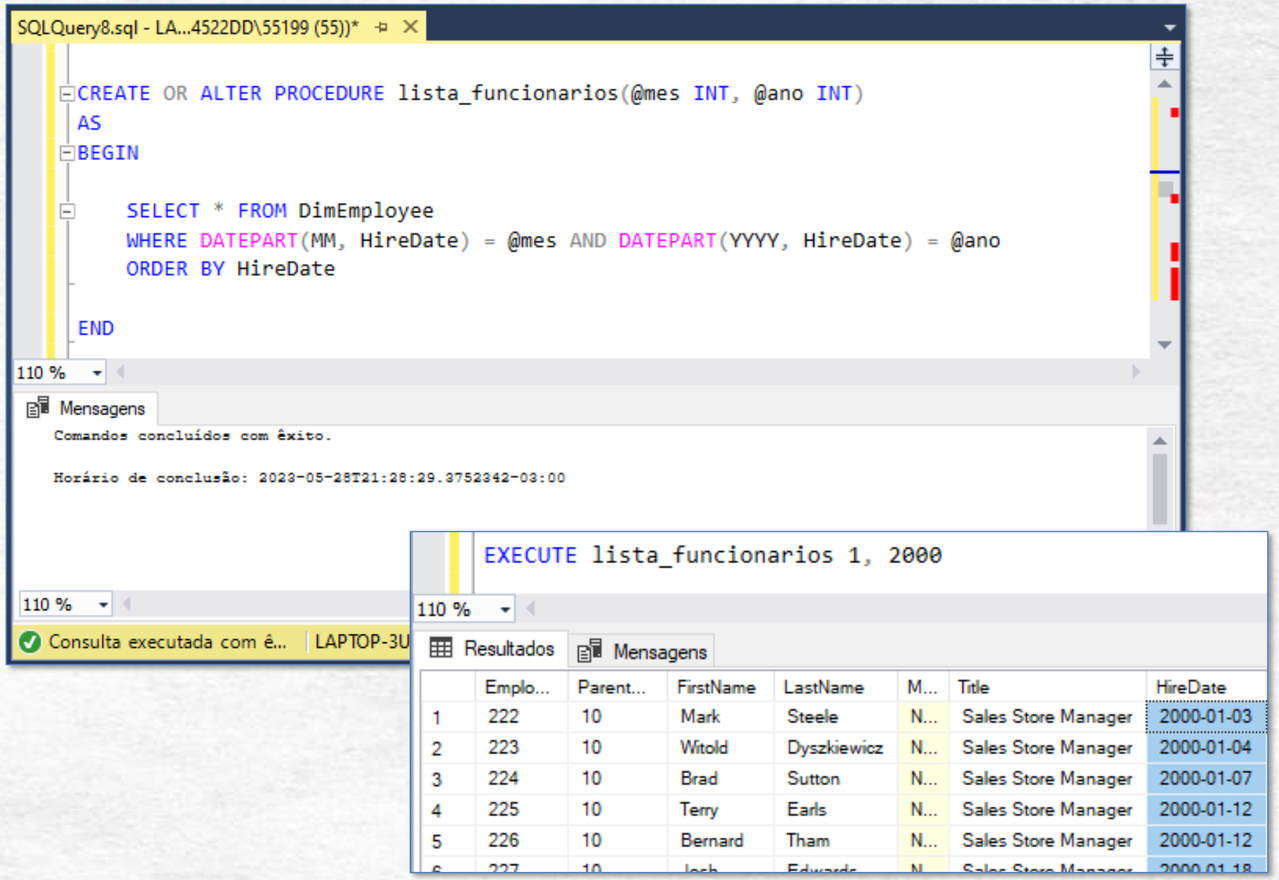


GABARITOS

3

Questão 3

Crie uma Procedure que recebe 2 argumentos: MÊS (de 1 a 12) e ANO (1996 a 2003). Sua Procedure deve listar todos os funcionários que foram contratados no mês/ano informado.



	Emplo...	Parent...	FirstName	LastName	M...	Title	HireDate
1	222	10	Mark	Steele	N...	Sales Store Manager	2000-01-03
2	223	10	Witold	Dyszkiewicz	N...	Sales Store Manager	2000-01-04
3	224	10	Brad	Sutton	N...	Sales Store Manager	2000-01-07
4	225	10	Terry	Earls	N...	Sales Store Manager	2000-01-12
5	226	10	Bernard	Tham	N...	Sales Store Manager	2000-01-12

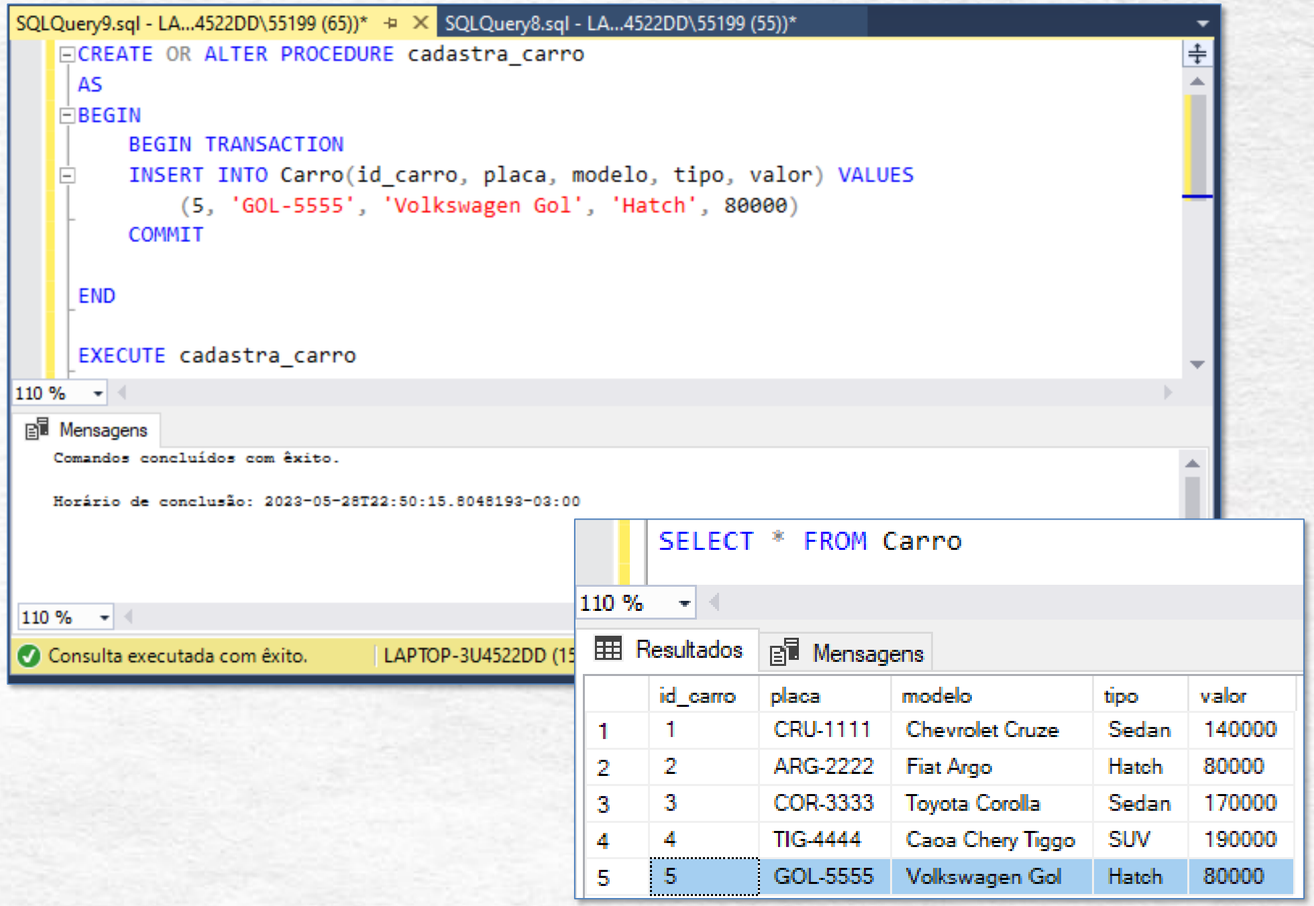
GABARITOS

4

Questão 4

Crie uma Procedure que insere uma nova linha na tabela Carro. Essa nova linha deve conter os seguintes dados:

- id = 5
- placa = GOL-5555
- modelo = Volkswagen Gol
- tipo = Hatch
- valor = 80000



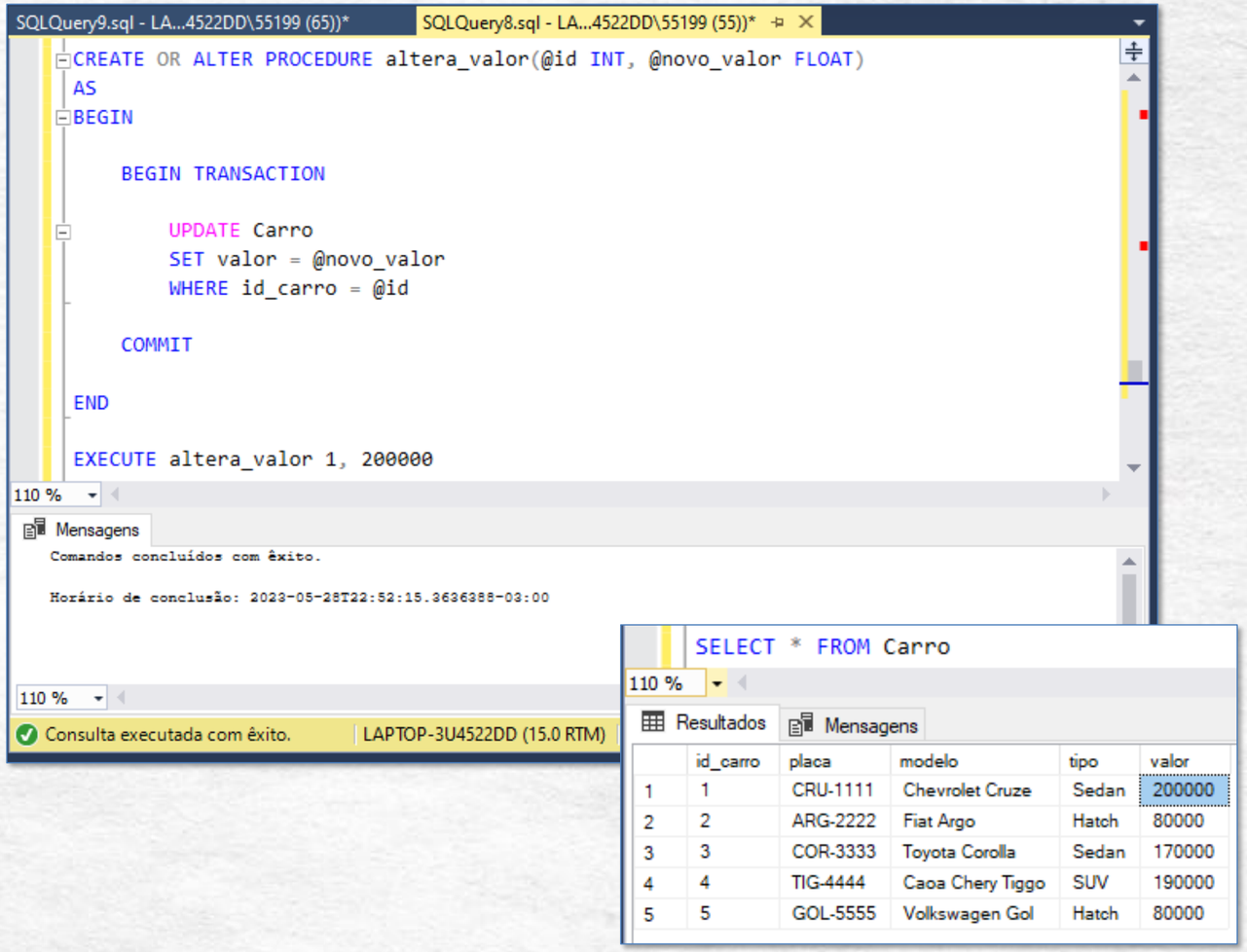
	id_carro	placa	modelo	tipo	valor
1	1	CRU-1111	Chevrolet Cruze	Sedan	140000
2	2	ARG-2222	Fiat Argo	Hatch	80000
3	3	COR-3333	Toyota Corolla	Sedan	170000
4	4	TIG-4444	Caoa Chery Tiggo	SUV	190000
5	5	GOL-5555	Volkswagen Gol	Hatch	80000

GABARITOS

5

Questão 5

Crie uma Procedure que altera o valor de venda de um carro. A Procedure deve receber como parâmetros o id_carro e o novo valor.

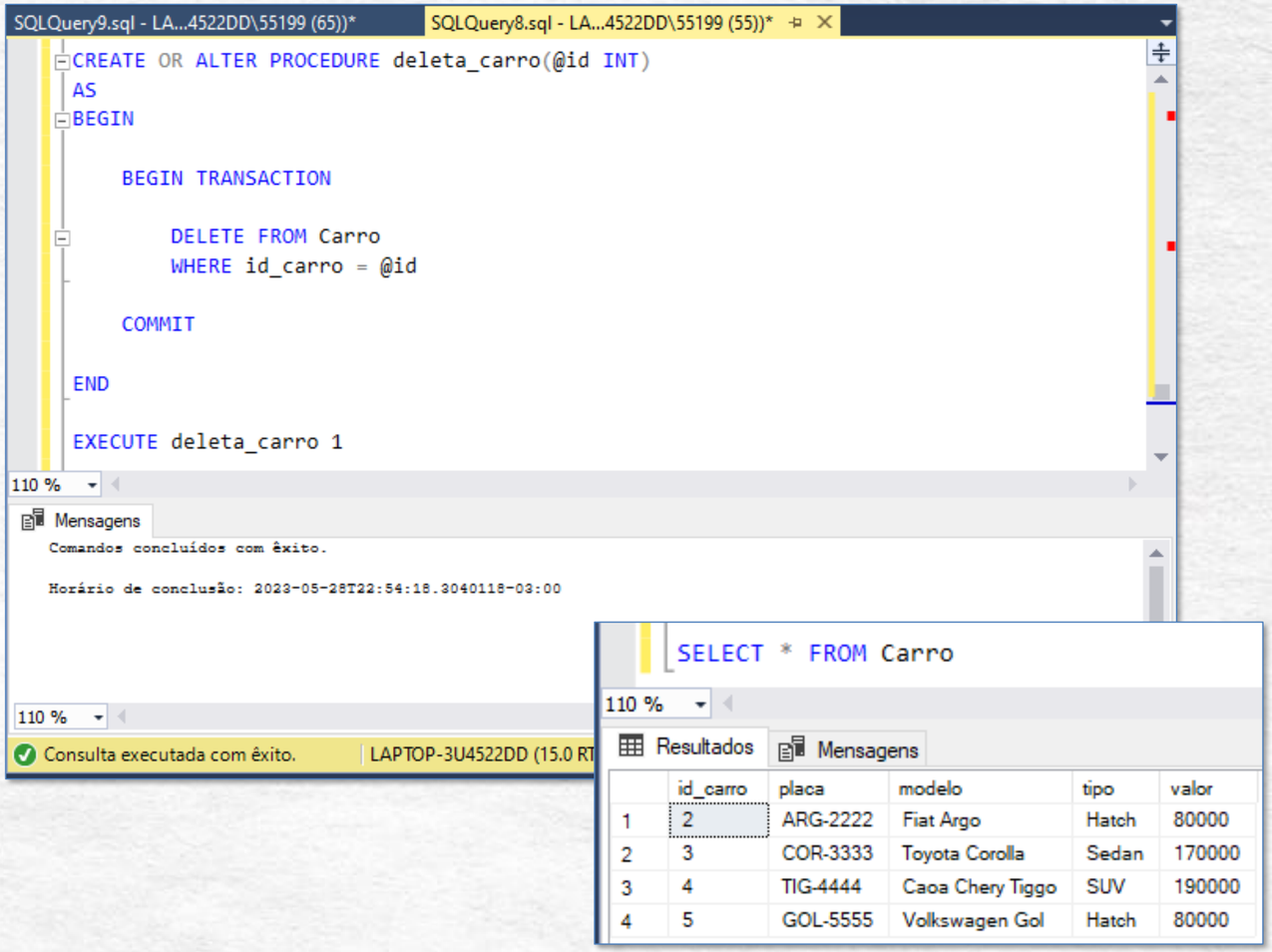


GABARITOS

6

Questão 6

Crie uma Procedure que exclui um carro a partir do id informado.



MÓDULO 23

SQL

Triggers DML

O QUE VEREMOS NESTE MÓDULO

MÓDULO 23 TRIGGERS DML

01	O que são TRIGGERS DML?	654
02	Como criar uma TRIGGER DML (AFTER)	655
03	Tabelas INSERTED e DELETED	660
04	Como criar uma TRIGGER DML (INSTEAD OF)	676
05	Como habilitar / desabilitar / excluir uma TRIGGER DML	683

O que são TRIGGERS DML?

Uma TRIGGER é um gatilho que será disparado automaticamente quando acontecer um evento.

Uma TRIGGER DML é disparada quando um comando INSERT, UPDATE ou DELETE é executado em uma Tabela ou View.

Antes de continuarmos, um parêntese: existem também as Triggers DDL que são disparadas quando utilizamos comandos DDL (CREATE, ALTER, DROP). Mas este será assunto para o próximo módulo. Neste aqui, vamos focar nas Triggers DML.

Prossigamos: na hora de criar uma TRIGGER DML, podemos definir alguns **elementos**:

TIPO (em relação ao evento): AFTER (depois) ou INSTEAD OF (no lugar de);

EVENTO: INSERT, UPDATE, DELETE;

CORPO: Bloco de Comandos.

Como criar uma TRIGGER DML (AFTER)

Vamos iniciar criando uma Trigger DML do tipo AFTER bem simples.

Essa trigger será disparada **após** (AFTER) um evento INSERT, UPDATE ou DELETE ser executado na tabela dCliente e imprimirá na tela uma mensagem informando que os dados dessa tabela foram alterados.

Para isso, utilizaremos o código ao lado:

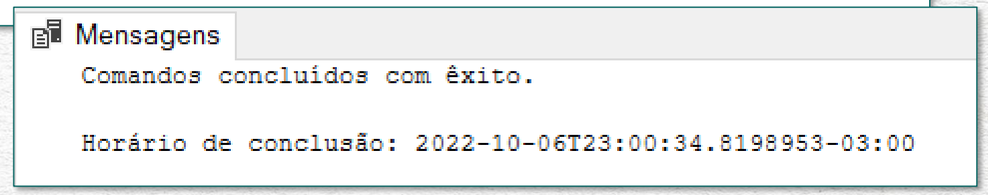
```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT, UPDATE, DELETE
AS
BEGIN

    PRINT 'Os dados da tabela dCliente foram alterados!'

END
```

Vejamos na próxima página o seu passo a passo.

Como criar uma TRIGGER DML (AFTER)

1. Iniciamos a nossa trigger com o comando CREATE OR ALTER TRIGGER e damos um nome a ela;
2. Informamos o nome da tabela que fará a trigger ser disparada toda vez que for alterada. No caso, será a dCliente;
3. Determinamos se a trigger será do tipo AFTER ou INSTEAD OF (no caso, será do tipo AFTER, pois queremos que ela dispare **depois** que uma alteração for efetuada) e qual(is) o(s) comando(s) que a disparará(ão) quando for(em) executado(s). No caso, será depois dos três comandos DML: INSERT, UPDATE e DELETE;

```
CREATE OR ALTER TRIGGER tgClienteAlterado 1
ON dCliente 2
AFTER INSERT, UPDATE, DELETE 3
AS
BEGIN 4

    PRINT 'Os dados da tabela dCliente foram alterados!' 5

END 6
```



4. Iniciamos o bloco de comandos com o comando BEGIN;
5. Incluímos todos os comandos que deverão ser executados quando a trigger for disparada. Aqui, apenas um PRINT na tela com a mensagem “Os dados da tabela dCliente foram alterados!”;
6. Finalizamos o bloco de comandos com o comando END.

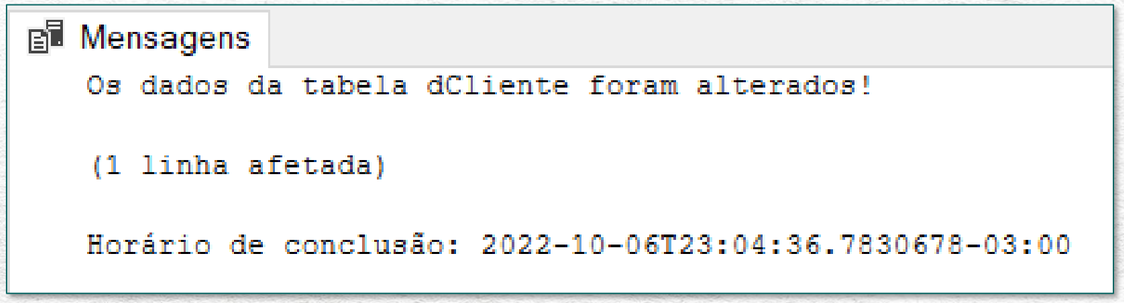
Como criar uma TRIGGER DML (AFTER)

Com a nossa trigger tgClienteAlterado criada, repare que toda vez que efetuarmos um INSERT, um UPDATE ou um DELETE na tabela dCliente, aparecerá na tela a mensagem configurada no PRINT.

Vejamos um exemplo com o comando INSERT. Vamos incluir o seguinte registro na tabela dCliente:

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Zacarias Neto', 'M', '13/02/1999', '139.543.189-00')
```

Repare que a mensagem configurada apareceu na tela:



Como criar uma TRIGGER DML (AFTER)

Observemos outro exemplo com o comando UPDATE. Vamos atualizar o registro que acabamos de incluir na tabela dCliente para alterar seu CPF:

```
UPDATE dCliente
SET cpf = '130.451.892-10'
WHERE id_cliente = 11
```

Veja que a mensagem configurada novamente apareceu na tela:

Mensagens

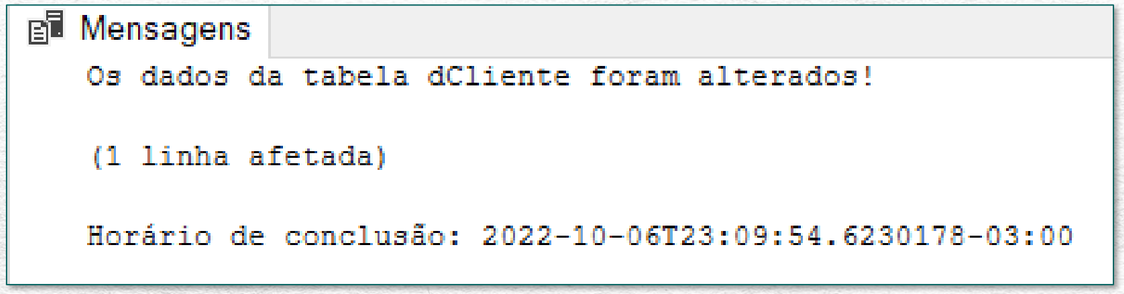
```
Os dados da tabela dCliente foram alterados!

(1 linha afetada)

Horário de conclusão: 2022-10-06T23:08:14.0115593-03:00
```

Como criar uma TRIGGER DML (AFTER)

Vejamos, por fim, um exemplo com o comando DELETE. Vamos excluir o registro que acabamos de incluir na tabela dCliente:

```
DELETE FROM dCliente
WHERE id_cliente = 11
```

Note que a mensagem configurada mais uma vez apareceu na tela:



Tabelas INSERTED e DELETED

Como poderíamos deixar nossa trigger tgClienteAlterado, criada no exemplo anterior, ainda mais clara? E se quiséssemos, em vez de apenas informar na tela que os dados da tabela dCliente foram alterados, apresentar uma mensagem diferente para cada tipo de ação?

Por exemplo:

Quando efetuássemos um INSERT INTO, aparecesse a mensagem “Os dados foram inseridos na tabela!”;
Já quando dados fossem atualizados com um UPDATE, mostrássemos na tela “Os dados foram atualizados na tabela!”;
E, por fim, quando registros fossem excluídos por meio de um DELETE, avisássemos que “Os dados foram excluídos da tabela!”.

Nossa trigger ficaria mais interessante, concorda?

Só que, para fazer isso, precisamos antes entender sobre a existência das tabelas INSERTED e DELETED.

Tabelas INSERTED e DELETED

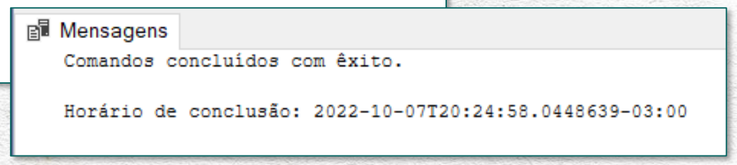
No SQL Server, existem duas tabelas pré-configuradas, chamadas INSERTED e DELETED que, todas as vezes que efetuamos um comando DML (INSERT, UPDATE ou DELETE), elas recebem os dados que foram ou inseridos, ou atualizados, ou excluídos, a depender do comando que foi utilizado.

Vamos entender melhor fazendo alguns exemplos.

Suponhamos que a gente queira alterar a trigger tgClienteAlterado para, em vez de apresentar na tela aquela mensagem genérica que os dados foram alterados, mostrar os próprios dados que foram inseridos na tabela dCliente toda vez que executarmos um INSERT.

Ela ficará assim:

```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT
AS
BEGIN

    SELECT * FROM INSERTED

END
```



Repare que agora, no bloco de comandos, retiramos o PRINT e inserimos em seu lugar um SELECT * FROM INSERTED.

Tabelas INSERTED e DELETED

Com essa alteração na nossa trigger, se efetuarmos o INSERT abaixo...

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Zacarias Neto', 'M', '13/02/1999', '139.543.189-00')
```

... repare que o resultado serão os próprios dados que foram inseridos na tabela dCliente:

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	13	Zacarias Neto	M	1999-02-13	139.543.189-00

Isso aconteceu porque, toda vez que inserimos um dado em alguma tabela, esse registro também fica temporariamente armazenado na tabela INSERTED. Como incluímos um SELECT a essa tabela (INSERTED) no bloco de comandos da trigger tgClienteAlterado, quando disparada, ela mostrou como resultado o registro que a tabela INSERTED acabou de temporariamente armazenar.

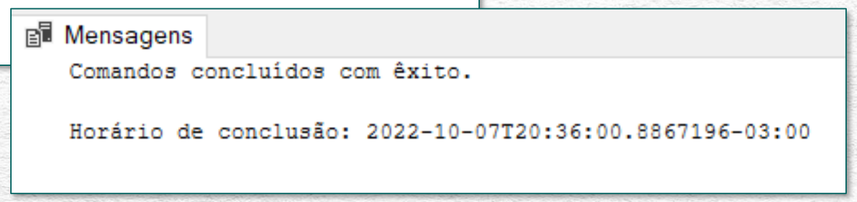
Tabelas INSERTED e DELETED

Agora, vamos supor que queremos novamente alterar a trigger tgClienteAlterado para, a partir de agora, apresentar na tela os dados que foram excluídos da tabela dCliente toda vez que executarmos um DELETE.

Ela ficará assim:

```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER DELETE
AS
BEGIN

    SELECT * FROM DELETED

END
```

Repare que agora, no bloco de comandos, inserimos um SELECT * FROM DELETED.



Tabelas INSERTED e DELETED

Com essa alteração na nossa trigger, se efetuarmos o DELETE abaixo...

```
DELETE FROM dCliente
WHERE id_cliente = 13
```

... repare que o resultado serão os próprios dados que foram excluídos na tabela dCliente:

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	13	Zacarias Neto	M	1999-02-13	139.543.189-00

Isso aconteceu porque, toda vez que excluímos algum dado de uma tabela, esse registro também fica temporariamente armazenado na tabela DELETED. Como incluímos um SELECT a essa tabela (DELETED) no bloco de comandos da trigger tgClienteAlterado, quando disparada, ela mostrou como resultado o registro que a tabela DELETED acabou de temporariamente armazenar.

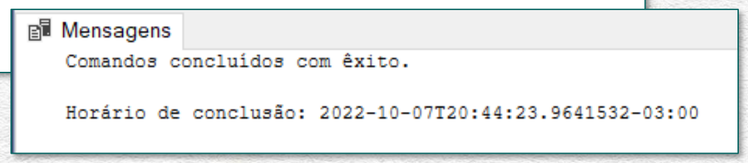
Tabelas INSERTED e DELETED

Já parou para pensar que quando atualizamos uma tabela, temos dois registros? O novo que foi inserido na tabela, e o antigo que foi excluído da tabela para dar lugar ao novo.

Com isso em mente, vamos imaginar que queremos, por fim, alterar a trigger tgClienteAlterado para, a partir de agora, apresentar na tela os dados que foram inseridos na tabela dCliente e os que foram excluídos da tabela dCliente toda vez que executarmos um UPDATE.

Ela ficará assim:

```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER UPDATE
AS
BEGIN

    SELECT * FROM INSERTED
    SELECT * FROM DELETED

END
```



Repare que agora, no bloco de comandos, inserimos duas consultas: um SELECT * FROM INSERTED (para retornar o novo registro) e um SELECT * FROM DELETED (para retornar o registro que foi excluído).

Tabelas INSERTED e DELETED

Vamos inserir novamente o registro que excluímos no último exemplo para podermos, posteriormente, atualizá-lo:

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Zacarias Neto', 'M', '13/02/1999', '139.543.189-00')
```

Mensagens

```
(1 linha afetada)

Horário de conclusão: 2022-10-07T20:48:42.4075488-03:00
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	14	Zacarias Neto	M	1999-02-13	139.543.189-00

Tabelas INSERTED e DELETED

Agora, note que, com a alteração na nossa trigger, se efetuarmos o UPDATE abaixo...

```
UPDATE dCliente
SET cpf = '130.451.892-10'
WHERE id_cliente = 14
```

... o resultado será o registro que foi incluído na tabela dCliente e também o que foi excluído da tabela dCliente:

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	14	Zacarias Neto	M	1999-02-13	130.451.892-10

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	14	Zacarias Neto	M	1999-02-13	139.543.189-00

← **Novo CPF incluído pelo UPDATE efetuado acima.**

← **Antigo CPF que foi excluído para dar lugar ao novo CPF informado no UPDATE acima.**

Isso aconteceu porque, toda vez que atualizamos algum dado de uma tabela, o novo registro também fica temporariamente armazenado na tabela INSERTED, enquanto que o antigo registro (que foi excluído) fica temporariamente armazenado na tabela DELETED. Como incluímos um SELECT tanto à tabela INSERTED quanto à tabela DELETED no bloco de comandos da trigger tgClienteAlterado, quando disparada, ela mostrou como resultado os registros que as tabelas INSERTED e DELETED acabaram de temporariamente armazenar.

Tabelas INSERTED e DELETED

Portanto, em resumo, podemos concluir que:

1) Se a trigger for de INSERT, a tabela INSERTED terá registros e a tabela DELETED ficará vazia;

2) Se a trigger for de DELETE, a tabela INSERTED ficará vazia e a tabela DELETED terá registros;

3) Se a trigger for de UPDATE, a tabela INSERTED e a tabela DELETED terão registros.

Tabelas INSERTED e DELETED

Sabendo agora como funcionam as tabelas INSERTED e DELETED, podemos finalmente personalizar as mensagens apresentadas na tela, de acordo com o comando DML utilizado, quando a tabela dCliente for alterada e a nossa trigger tgClienteAlterado disparada.

Vamos relembrar o que queremos fazer:

Quando efetuarmos um INSERT, queremos que apareça na tela a mensagem “Os dados foram **inseridos** na tabela!”;

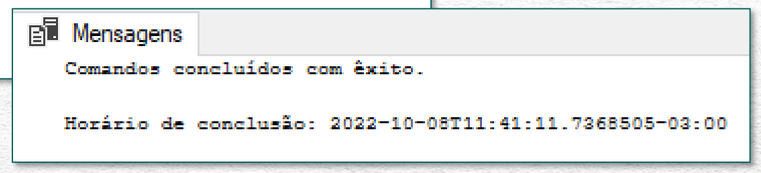
Já quando atualizarmos algum registro com um UPDATE, queremos mostrar na tela que “Os dados foram **atualizados** na tabela!”;

E, por fim, quando excluirmos dados por meio de um DELETE, queremos avisar que “Os dados foram **excluídos** da tabela!”.

Tabelas INSERTED e DELETED

Para isso, vamos alterar a trigger tgClienteAlterado para deixa-la da seguinte forma:

```
CREATE OR ALTER TRIGGER tgClienteAlterado
ON dCliente
AFTER INSERT, UPDATE, DELETE
AS
BEGIN

    IF EXISTS (SELECT * FROM INSERTED) AND EXISTS (SELECT * FROM DELETED)
            PRINT 'Os dados foram atualizados na tabela!'
    ELSE IF EXISTS (SELECT * FROM INSERTED)
            PRINT 'Os dados foram inseridos na tabela!'
    ELSE IF EXISTS (SELECT * FROM DELETED)
            PRINT 'Os dados foram excluídos da tabela!'

END
```



Vamos entender seu funcionamento nas próximas páginas.

Tabelas INSERTED e DELETED

1. Iniciamos a nossa trigger com o comando CREATE OR ALTER TRIGGER e damos um nome a ela;
2. Informamos o nome da tabela que fará a trigger ser disparada toda vez que for alterada. No caso, será a dCliente;
3. Determinamos se a trigger será do tipo AFTER ou INSTEAD OF (no caso, será do tipo AFTER, pois queremos que ela dispare **depois** que uma alteração for efetuada) e qual(is) o(s) comando(s) que a disparará(ão) quando for(em) executado(s). No caso, será depois dos três comandos DML: INSERT, UPDATE e DELETE;

```
CREATE OR ALTER TRIGGER tgClienteAlterado 1
ON dCliente 2
AFTER INSERT, UPDATE, DELETE 3
AS
```

Tabelas INSERTED e DELETED

```
BEGIN 4

    IF EXISTS (SELECT * FROM INSERTED) AND EXISTS (SELECT * FROM DELETED) 5
            PRINT 'Os dados foram atualizados na tabela!'
    ELSE IF EXISTS (SELECT * FROM INSERTED) 6
            PRINT 'Os dados foram inseridos na tabela!'
    ELSE IF EXISTS (SELECT * FROM DELETED) 7
            PRINT 'Os dados foram excluídos da tabela!'

END 8
```

4 Iniciamos o bloco de comandos com o comando BEGIN;

5 Dentro do bloco de comandos, fazemos um IF EXISTS para verificar se, após executado um dos comandos DML (INSERT, UPDATE ou DELETE), há dados tanto na tabela INSERTED quanto na DELETED; se houver, significa que o comando DML utilizado foi um UPDATE, já que, conforme vimos, o UPDATE é o único comando que insere dados tanto na INSERTED (novos dados) quanto na DELETED (dados antigos que foram excluídos para que os novos fossem inseridos em seu lugar). Sendo um UPDATE, aparecerá na tela a mensagem “Os dados foram atualizados na tabela!”;

6 Se no IF acima for verificado que ou a INSERTED ou a DELETED está vazia, significa que o comando DML utilizado não foi um UPDATE, então, prosseguimos para a próxima linha do código e verificamos no primeiro ELSE IF EXISTS se há dados na tabela INSERTED; se houver, significa que o comando DML utilizado foi um INSERT, portanto, imprimimos na tela a mensagem “Os dados foram inseridos na tabela!”;

7 Caso contrário, ou seja, se a tabela INSERTED estiver vazia, fazemos a última verificação para saber se há dados na tabela DELETED; se houver, significa que o comando DML utilizado foi um DELETE, portanto, imprimimos na tela que “Os dados foram excluídos da tabela!”.

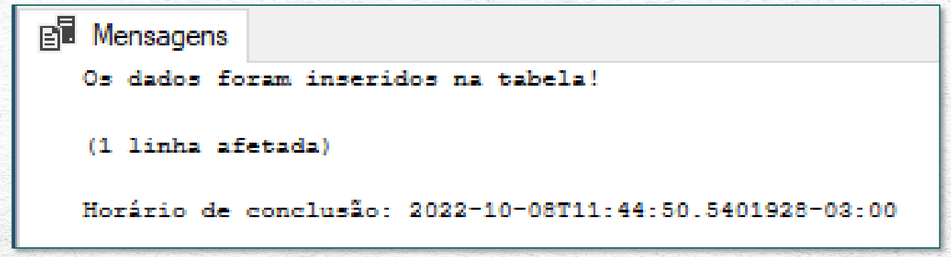
8 Finalizamos o bloco de comandos com o comando END.

Tabelas INSERTED e DELETED

Vamos testar agora um comando DML por vez. Iniciemos com o comando INSERT:

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Eleonora Batista', 'F', '13/02/2003', '453.543.189-00')
```

Veja só que a mensagem “Os dados foram inseridos na tabela!” apareceu na tela, conforme esperado:

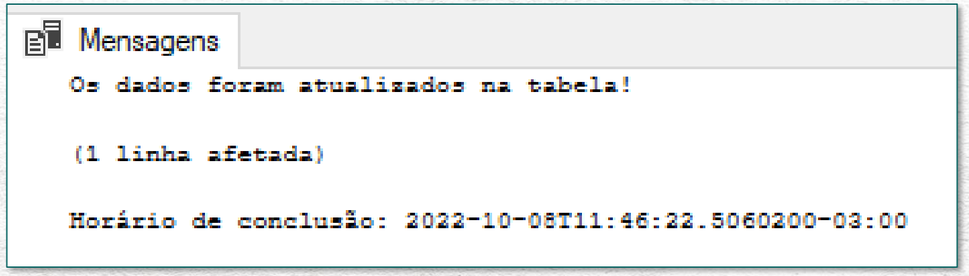


Tabelas INSERTED e DELETED

Vamos testar agora o comando UPDATE:

```
UPDATE dCliente
SET cpf = '678.333.111-11'
WHERE id_cliente = 15
```

Repare que a mensagem “Os dados foram atualizados na tabela!” apareceu na tela, conforme esperado:

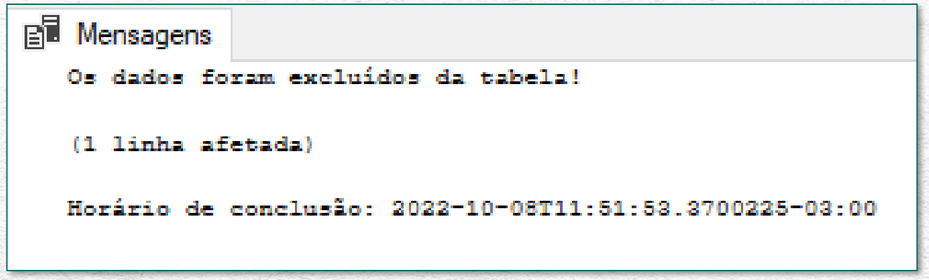


Tabelas INSERTED e DELETED

Por fim, testemos o comando DELETE:

```
DELETE FROM dCliente
WHERE id_cliente = 15
```

Note que a mensagem “Os dados foram excluídos da tabela!” apareceu na tela, conforme esperado:



Como criar uma TRIGGER DML (INSTEAD OF)

Vamos partir agora para a criação de uma Trigger DML do tipo INSTEAD OF. Este tipo de trigger efetua alguma ação **no lugar do comando DML** especificado, toda vez que uma determinada condição for atendida.

No nosso exemplo, vamos imaginar que a gente queira permitir a inserção de dados na tabela dCliente apenas de segunda à sexta. Se for sábado ou domingo, nossa trigger deve gerar um erro, apresentá-lo na tela e não permitir a inserção dos novos dados. Porém, se for dia útil (segunda à sexta), a trigger deve permitir a inserção dos novos dados na tabela dCliente normalmente.

Para criá-la, utilizaremos o código ao lado.

Vejamos nas próximas páginas o seu passo a passo.

```
CREATE OR ALTER TRIGGER tgControleRegistros
ON dCliente
INSTEAD OF INSERT
AS
BEGIN

    IF FORMAT(GETDATE(), 'dddd') IN ('domingo')
    BEGIN

        RAISERROR('O cadastro de clientes só pode ser feito de segunda à sexta', 1, 1)
        ROLLBACK

    END

    ELSE
    BEGIN

        INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf)
        SELECT nome_cliente, genero, data_de_nascimento, cpf FROM INSERTED

    END

END
```

```
Mensagens
Comandos concluídos com êxito.

Horário de conclusão: 2022-10-08T18:48:43.0602574-03:00
```

Como criar uma TRIGGER DML (INSTEAD OF)

1. Iniciamos a nossa trigger com o comando CREATE OR ALTER TRIGGER e damos um nome a ela;
2. Informamos o nome da tabela que fará a trigger ser disparada quando for alterada. No caso, será a dCliente;
3. Determinamos se a trigger será do tipo AFTER ou INSTEAD OF (no caso, será do tipo INSTEAD OF, pois queremos que ela informe um erro **no lugar do INSERT**, não permitindo que ele seja efetuado, caso o usuário tente inserir novos dados na tabela dCliente no final de semana (sábado ou domingo).

```
CREATE OR ALTER TRIGGER tgControleRegistros 1
ON dCliente 2
INSTEAD OF INSERT 3
AS
```

Como criar uma TRIGGER DML (INSTEAD OF)

4 Iniciamos o bloco de comandos com o comando BEGIN;

5 Dentro do bloco de comandos, executamos um IF para verificar se o dia de hoje (GETDATE) é sábado ou domingo:

6 Se for, utilizamos a instrução RAISERROR para exibir um erro na tela com a seguinte mensagem: “O cadastro de clientes só pode ser feito de segunda à sexta”, e efetuamos um ROLLBACK para que o INSERT efetuado seja desfeito;

7 Se não for sábado nem domingo, efetuamos o INSERT desejado na tabela dCliente. Observe que os dados a serem inseridos na tabela dCliente serão extraídos da tabela INSERTED, que armazenou temporariamente o registro incluído pelo usuário;

8 Finalizamos o bloco de comandos com o comando END.

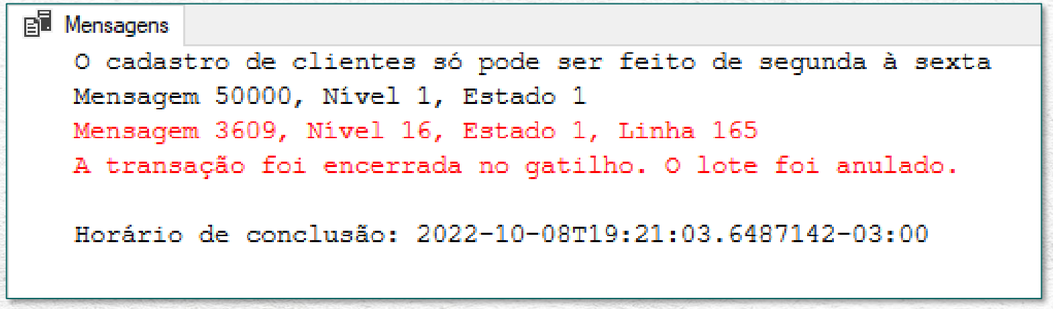
```
BEGIN 4

    IF FORMAT(GETDATE(), 'dddd') IN ('sábado', 'domingo') 5
    BEGIN

        RAISERROR('O cadastro de clientes só pode ser feito de segunda à sexta', 1, 1) 6
        ROLLBACK

    END

    ELSE
    BEGIN

        INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) 7
        SELECT nome_cliente, genero, data_de_nascimento, cpf FROM INSERTED

    END

END 8
```

Como criar uma TRIGGER DML (INSTEAD OF)

Testemos agora a nossa trigger tgControleRegistros. Vamos tentar efetuar o seguinte INSERT na tabela dCliente:

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Lucio Dantas', 'M', '13/02/2003', '111.222.333-00')
```

Se hoje for sábado ou domingo, repare que a trigger disparada retornará o erro configurado, conforme esperado:



Como criar uma TRIGGER DML (INSTEAD OF)

Além disso, como foi efetuado o ROLLBACK configurado na trigger, o novo registro também será desfeito, não sendo salvo na tabela dCliente. Veja:

```
SELECT * FROM dCliente
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	14	Zacarias Neto	M	1999-02-13	130.451.892-10

Como criar uma TRIGGER DML (INSTEAD OF)

Porém, caso hoje não seja sábado nem domingo, repare que, executado o INSERT...

```
INSERT INTO dCliente(nome_cliente, genero, data_de_nascimento, cpf) VALUES
    ('Lucio Dantas', 'M', '13/02/2003', '111.222.333-00')
```

... os dados serão normalmente inseridos na tabela:

Mensagens

```
Os dados foram inseridos na tabela!

(1 linha afetada)

(1 linha afetada)

Horário de conclusão: 2022-10-08T19:25:25.5551400-03:00
```

Como criar uma TRIGGER DML (INSTEAD OF)

Se consultarmos a tabela dCliente, veremos o novo registro devidamente inserido:

```
SELECT * FROM dCliente
```

Resultados | Mensagens

	id_cliente	nome_cliente	genero	data_de_nascimento	cpf
1	1	André Martins	M	1989-02-12	839.283.190-00
2	2	Bárbara Campos	F	1992-05-07	351.391.410-02
3	3	Carol Freitas	F	1985-04-23	139.274.921-12
4	4	Diego Cardoso	M	1994-10-11	192.371.081-17
5	5	Eduardo Pereira	M	1988-11-09	193.174.192-82
6	6	Fabiana Silva	F	1989-09-02	231.298.471-98
7	7	Gustavo Barbosa	M	1993-06-27	240.174.171-76
8	8	Helen Viana	F	1990-02-11	193.129.183-01
9	9	Igor Castro	M	1989-08-21	184.148.102-29
10	10	Juliana Pires	F	1991-01-13	416.209.192-47
11	14	Zacarias Neto	M	1999-02-13	130.451.892-10
12	18	Lucio Dantas	M	2003-02-13	111.222.333-00

Como habilitar / desabilitar / excluir uma TRIGGER DML

Além de podermos excluir uma TRIGGER DML, também temos a opção de apenas desabilitá-la. Desta forma, ela continua salva no banco de dados, porém, deixa de ser disparada. Se porventura precisarmos, podemos habilitá-la novamente.

Para desabilitar uma trigger DML utilizamos o comando DISABLE:

DISABLE TRIGGER nome_da_trigger **ON** nome_da_tabela

```
DISABLE TRIGGER tgControleRegistros ON dCliente
```

Para habilitar uma trigger DML novamente, utilizamos o comando ENABLE:

ENABLE TRIGGER nome_da_trigger **ON** nome_da_tabela

```
ENABLE TRIGGER tgControleRegistros ON dCliente
```

Como habilitar / desabilitar / excluir uma TRIGGER DML

Para desabilitar todas as triggers DML de uma tabela utilizamos a seguinte sintaxe:

DISABLE TRIGGER ALL **ON** nome_da_tabela

```
DISABLE TRIGGER ALL ON dCliente
```

Para habilitar novamente todas as triggers DML de uma tabela utilizamos a seguinte sintaxe:

ENABLE TRIGGER ALL **ON** nome_da_tabela

```
ENABLE TRIGGER ALL ON dCliente
```

Por fim, para excluir uma trigger DML, utilizamos a seguinte sintaxe:

DROP TRIGGER nome_da_trigger

```
DROP TRIGGER tgControleRegistros
```

MÓDULO 24

SQL

Triggers DDL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 24 TRIGGERS DDL

01	O que são TRIGGERS DDL?	687
02	Como criar TRIGGERS DDL	688
03	Como habilitar / desabilitar / excluir uma TRIGGER DDL	693

O que são TRIGGERS DDL?

Como sabemos, uma TRIGGER é um gatilho que será disparado automaticamente quando acontecer um evento.

Vimos no módulo anterior as Triggers DML, que podem ser disparadas quando os comandos DML INSERT, UPDATE ou DELETE são executados em uma Tabela ou View.

Agora, vamos entender como funcionam as TRIGGERS DDL, que podem ser disparadas quando executamos os comandos DDL CREATE, ALTER ou DROP.

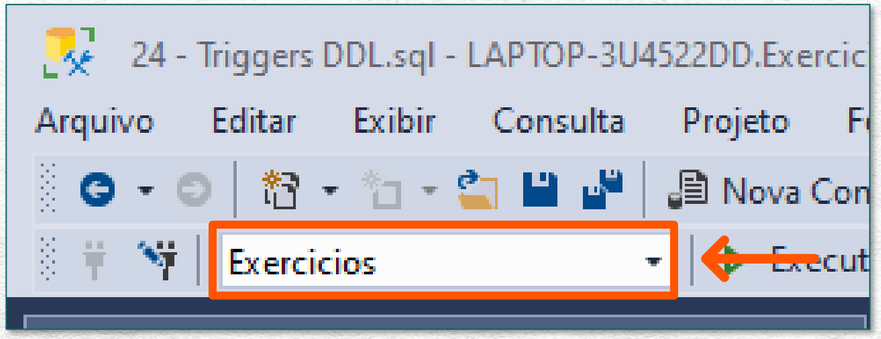
Como criar TRIGGERS DDL

Enquanto que, para configurar as triggers DML, nós visamos uma tabela, para criar triggers DDL, nós levamos em consideração o banco de dados em si.

Vamos a um exemplo:

Queremos agora criar uma trigger que impeça a inclusão, alteração ou exclusão de tabelas no banco de dados Exercícios (que viemos utilizando nos últimos módulos).

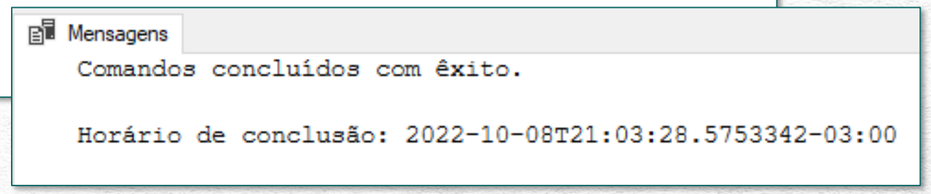
Portanto, primeiro devemos ter a certeza que o banco de dados Exercícios está selecionado. Para isso, verifique na barra Editor SQL se aparece o nome Exercícios, conforme abaixo:



Como criar TRIGGERS DDL

Feito isso, vamos criar o seguinte código:

```
CREATE OR ALTER TRIGGER tgRecusarTabelas
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN

    PRINT 'Não é permitido criação, alteração ou exclusão de tabelas'
    ROLLBACK

END
```



Vejamos o seu passo a passo na próxima página.

Como criar TRIGGERS DDL


```
CREATE OR ALTER TRIGGER tgRecusarTabelas 1
ON DATABASE 2
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE 3
AS
BEGIN 4

    PRINT 'Não é permitido criação, alteração ou exclusão de tabelas' 5
    ROLLBACK

END 6
```

1. Iniciamos a nossa trigger com o comando CREATE OR ALTER TRIGGER e damos um nome a ela;
2. Incluímos o comando ON DATABASE que indica que se trata de uma trigger DDL (não precisamos informar o nome do banco de dados, pois a trigger será criada dentro do banco de dados que estiver selecionado);
3. Acrescentamos o comando FOR e informamos os comandos DDL que serão capazes de disparar a nossa trigger quando forem executados. No caso CREATE_TABLE, ALTER_TABLE e DROP_TABLE;
4. Iniciamos o bloco de comandos com o comando BEGIN;
5. Inserimos, dentro do bloco de comandos, um PRINT para informar na tela que “Não é permitido criação, alteração ou exclusão de tabelas”; além disso, efetuamos um ROLLBACK para desfazer a tentativa de criação, alteração ou exclusão de tabela que havia sido executada, causando o disparo da trigger;
6. Finalizamos o bloco de comandos com o comando END.

Como criar TRIGGERS DDL

Para testar nossa trigger tgRecusarTabelas, vamos tentar criar a tabela abaixo:

```
CREATE TABLE teste
    (ID INT)
```

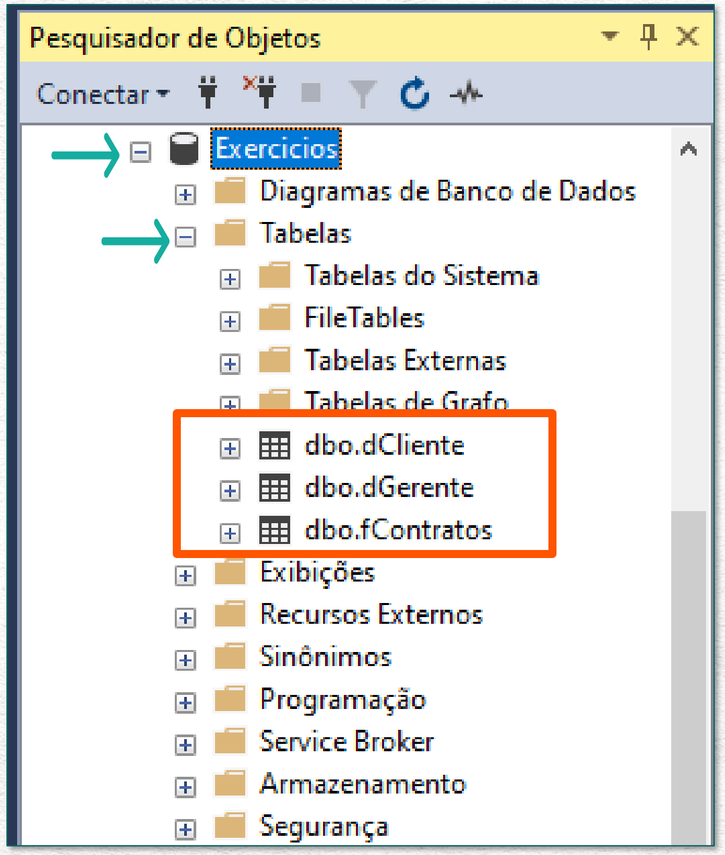
Repare que a mensagem configurada no PRINT da trigger apareceu na tela:

Mensagens

```
Não é permitido criação, alteração ou exclusão de tabelas
Mensagem 3609, Nível 16, Estado 2, Linha 34
A transação foi encerrada no gatilho. O lote foi anulado.

Horário de conclusão: 2022-10-08T21:05:35.1475171-03:00
```

Como criar TRIGGERS DDL

Se visualizarmos as tabelas do banco de dados Exercícios, perceberemos que realmente a tabela teste da página anterior não foi criada:



Como habilitar / desabilitar / excluir uma TRIGGER DDL

Além de podermos excluir uma TRIGGER DDL, também temos a opção de apenas desabilitá-la. Desta forma, ela continua salva no banco de dados, porém, deixa de ser disparada. Se porventura precisarmos, podemos habilitá-la novamente.

Para desabilitar uma trigger DDL utilizamos o comando DISABLE:

DISABLE TRIGGER nome_da_trigger **ON DATABASE**

```
DISABLE TRIGGER tgRecusarTabelas ON DATABASE
```

Para habilitar uma trigger DDL novamente, utilizamos o comando ENABLE:

ENABLE TRIGGER nome_da_trigger **ON DATABASE**

```
ENABLE TRIGGER tgRecusarTabelas ON DATABASE
```

Como habilitar / desabilitar / excluir uma TRIGGER DDL

Para desabilitar todas as triggers DDL de um banco de dados utilizamos a seguinte sintaxe:

DISABLE TRIGGER ALL **ON DATABASE**

```
DISABLE TRIGGER ALL ON DATABASE
```

Para habilitar novamente todas as triggers DDL de um banco de dados utilizamos a seguinte sintaxe:

ENABLE TRIGGER ALL **ON DATABASE**

```
ENABLE TRIGGER ALL ON DATABASE
```

Por fim, para excluir uma trigger DDL, utilizamos a seguinte sintaxe:

DROP TRIGGER nome_da_trigger **DATABASE**

```
DROP TRIGGER tgRecusarTabelas ON DATABASE
```

MÓDULO 25

SQL

Pivot Table

O QUE VEREMOS NESTE MÓDULO

MÓDULO 25 PIVOT TABLE

01	O que é uma PIVOT TABLE?	697
02	Como criar uma PIVOT TABLE	698
03	Adicionando mais linhas / colunas à PIVOT TABLE	702
04	Limitações de uma PIVOT TABLE	704
05	Corrigindo as limitações de uma PIVOT TABLE	705

O que é uma PIVOT TABLE?

A PIVOT TABLE é uma tabela criada a partir da inversão de uma outra tabela.

Basicamente, com o uso do operador PIVOT, conseguimos transformar o que eram linhas de uma tabela em colunas da PIVOT TABLE, ao passo que as colunas de outra tabela se transformam nas linhas da PIVOT TABLE.

Um exemplo: com o operador PIVOT conseguimos transformar a consulta à tabela DimEmployee do BD Contoso (1) ...

... nesta PIVOT TABLE (2) :

1

Resultados | Mensagens

	DepartmentName	Total_Funcionarios
1	Document Control	5
2	Engineering	6
3	Executive	3
4	Facilities and Maintenance	7
5	Finance	10
6	Human Resources	6
7	'Human Resources Contral	1
8	Information Services	10
9	Marketing	10
10	Production	176
11	Production Control	7
12	Purchasing	13
13	Quality Assurance	6
14	Research and Developm...	4
15	Sales	18
16	Shipping and Receiving	6
17	Tool Design	5

2

Resultados | Mensagens

	Document Control	Engineering	Executive	Facilities and Maintenance	Finance	'Human Resources Contral	Human Resources	Information Services	Marketing	Production Control	Production	Purchasing	Quality Assurance	Research and Development	Sales	Shipping and Receiving	Tool Design
1	5	6	3	7	10	1	6	10	10	7	176	13	6	4	18	6	5

Como criar uma PIVOT TABLE

Vamos entender como conseguimos transformar a consulta mostrada na página anterior naquela PIVOT TABLE.

Primeiramente, efetuamos a seguinte consulta à tabela DimEmployee:

```
SELECT
    DepartmentName,
    COUNT(EmployeeKey) Total_Funcionarios
FROM DimEmployee
GROUP BY DepartmentName
```

A consulta acima, como vimos, gera o resultado ao lado:

Resultados | Mensagens

	DepartmentName	Total_Funcionarios
1	Document Control	5
2	Engineering	6
3	Executive	3
4	Facilities and Maintenance	7
5	Finance	10
6	Human Resources	6
7	'Human Resources Contral	1
8	Information Services	10
9	Marketing	10
10	Production	176
11	Production Control	7
12	Purchasing	13
13	Quality Assurance	6
14	Research and Developm...	4
15	Sales	18
16	Shipping and Receiving	6
17	Tool Design	5

Como criar uma PIVOT TABLE

Depois, para criar a PIVOT TABLE, precisamos executar o código ao lado:

Repare que precisamos de uma *subquery* que para ser utilizada como tabela da nossa consulta principal. Nesta *subquery*, fazemos uma consulta às colunas dos dados que queremos ver na PIVOT TABLE: EmployeeKey e DepartmentName 1 .

Após isso, incluímos o operador PIVOT 2 e passamos as informações necessárias para ele, que são:

O cálculo que desejamos efetuar (uma contagem da quantidade de funcionários): COUNT(EmployeeKey) 3 ;

Por qual coluna devemos agrupar nossa PIVOT TABLE: neste caso, se queremos uma contagem da quantidade de funcionários por departamento, devemos agrupar pela coluna DepartmentName. Então, incluímos a linha FOR DepartmentName 4 ;

Para o operador IN, passamos os nomes de todos os departamentos (DepartmentName) que servirão de colunas para a nossa PIVOT TABLE 5 . Repare que cada departamento precisa estar entre colchetes [], separados por vírgulas.

```
SELECT * FROM
    (SELECT
        EmployeeKey,
        DepartmentName
    FROM DimEmployee) AS Dados
PIVOT(
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN([Document Control]
        ,[Engineering]
        ,[Executive]
        ,[Facilities and Maintenance]
        ,[Finance]
        ,['Human Resources Contral]
        ,[Human Resources]
        ,[Information Services]
        ,[Marketing]
        ,[Production Control]
        ,[Production]
        ,[Purchasing]
        ,[Quality Assurance]
        ,[Research and Development]
        ,[Sales]
        ,[Shipping and Receiving]
        ,[Tool Design])
) AS PivotTable
```

Como criar uma PIVOT TABLE

Para não precisarmos digitar departamento por departamento nesta parte, existem algumas formas de conseguir obtê-los diretamente da coluna DepartmentName.

Neste primeiro exemplo, vamos fazer isso de uma forma um tanto quanto manual, apenas para entendermos melhor como tudo funciona. Mais para frente, aprenderemos a fazer essa obtenção de forma dinâmica.

Fazendo o SELECT abaixo, obteremos o seguinte resultado:

```
SELECT DISTINCT
    ',' + QUOTENAME(TRIM(DepartmentName))
FROM DimEmployee
```

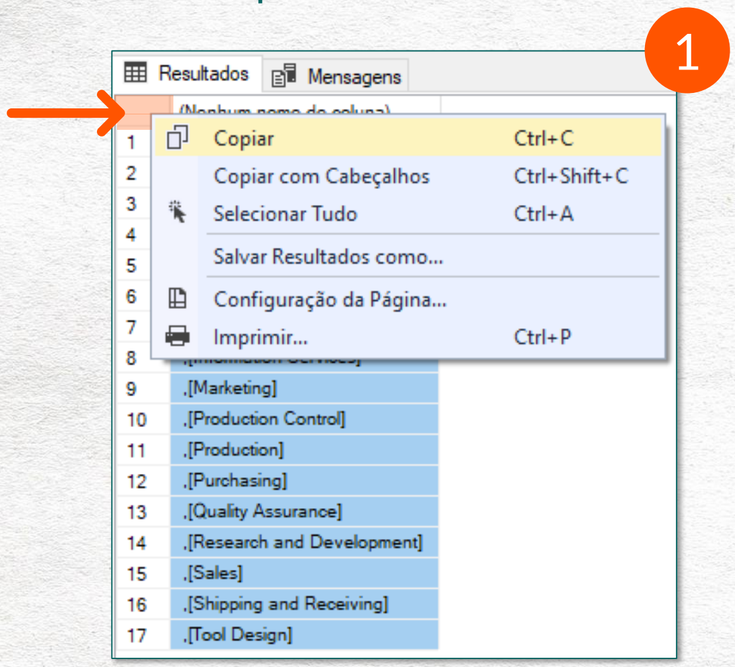
Resultados | Mensagens

	(Nenhum nome de coluna)
1	,[Document Control]
2	,[Engineering]
3	,[Executive]
4	,[Facilities and Maintenance]
5	,[Finance]
6	,['Human Resources Contral]
7	,[Human Resources]
8	,[Information Services]
9	,[Marketing]
10	,[Production Control]
11	,[Production]
12	,[Purchasing]
13	,[Quality Assurance]
14	,[Research and Development]
15	,[Sales]
16	,[Shipping and Receiving]
17	,[Tool Design]

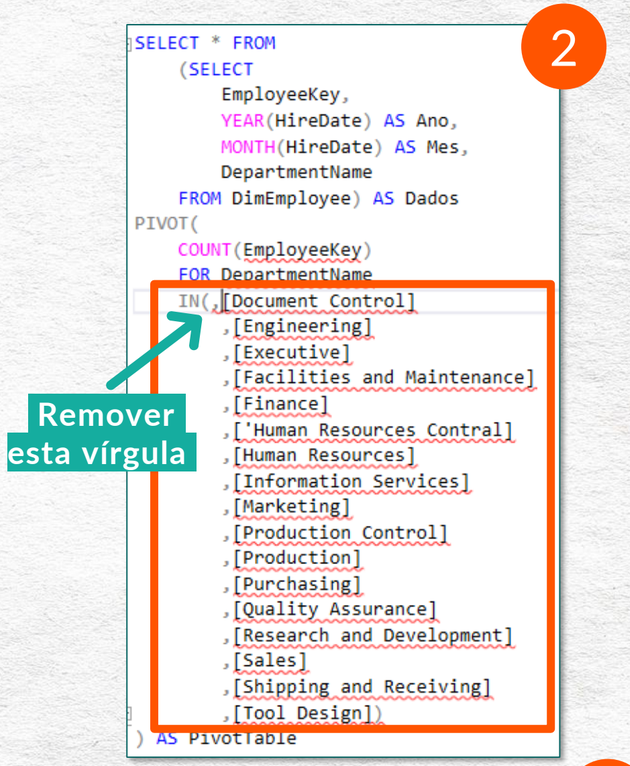
Note que, como concatenamos as vírgulas e utilizamos a função QUOTENAME para acrescentar os colchetes, assim como fizemos um TRIM na coluna DepartmentName para que ela retorne os nomes dos departamentos sem espaços no início e no fim, nosso resultado já vem praticamente pronto, pois é exatamente o que precisamos passar para o operador IN no nosso PIVOT mostrado da página anterior.

Como criar uma PIVOT TABLE

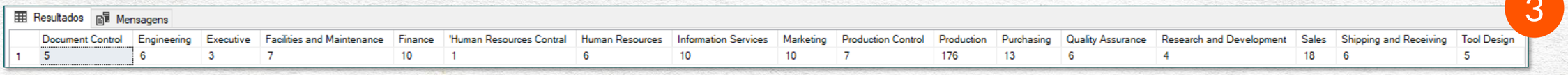
Agora, basta clicarmos com o botão direito no quadradinho da intersecção das linhas e colunas (apontado abaixo) e clicar em Copiar:



Feito isso, colamos esse resultado dentro dos parênteses do operador IN, precisando apenas remover a vírgula que aparece na frente do primeiro departamento, conforme apontada no código ao lado:



Executado o código, temos a nossa PIVOT TABLE:



	Document Control	Engineering	Executive	Facilities and Maintenance	Finance	'Human Resources Contral	Human Resources	Information Services	Marketing	Production Control	Production	Purchasing	Quality Assurance	Research and Development	Sales	Shipping and Receiving	Tool Design
1	5	6	3	7	10	1	6	10	10	7	176	13	6	4	18	6	5

Adicionando mais linhas / colunas à PIVOT TABLE

Imagine agora que queremos acrescentar à nossa PIVOT TABLE as colunas Ano e Mes, para detalhar melhor quando os funcionários de cada departamento foram contratados. Suponhamos que a gente queira que ela fique assim:

Resultados | Mensagens

	Ano	Mes	Document Control	Engineering	Executive	Facilities and Ma...	Finance	'Human Reso...	Human Resources	Information...	Marketing	Production Control	Production	Purchasing	Quality Assurance	Research and Development	Sales	Shipping and Receiving	Tool Design
1	1996	7	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
2	1997	2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
3	1997	12	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	1998	1	0	0	0	0	0	0	0	0	2	0	1	0	0	0	0	0	3
5	1998	2	0	2	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0
6	1998	3	0	1	0	0	0	1	0	0	0	0	3	0	0	0	0	0	0
7	1998	4	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0
8	1999	1	1	0	0	1	2	0	3	3	1	2	40	0	2	1	0	3	0
9	1999	2	2	0	1	0	3	0	1	3	2	3	38	0	1	1	0	2	0
10	1999	3	1	0	2	0	3	0	2	4	0	1	45	1	1	1	0	1	0
11	1999	4	1	0	0	0	2	0	0	0	1	1	12	1	1	0	0	0	0
12	1999	6	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
13	2000	1	0	0	0	2	0	0	0	0	0	0	9	1	0	0	0	0	0
14	2000	2	0	0	0	1	0	0	0	0	0	0	8	3	0	0	0	0	0
15	2000	3	0	0	0	1	0	0	0	0	0	0	11	1	1	0	0	0	0
16	2000	4	0	0	0	2	0	0	0	0	0	0	3	1	0	0	0	0	0
17	2001	1	0	1	0	0	0	0	0	0	0	0	0	2	0	0	0	0	2
18	2001	2	0	1	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
19	2001	3	0	0	0	0	0	0	0	0	2	0	0	3	0	0	1	0	0
20	2001	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0
21	2002	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
22	2002	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0
23	2002	11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
24	2003	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
25	2003	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0

Adicionando mais linhas / colunas à PIVOT TABLE

Para isso, vamos precisar fazer pequenas alterações no nosso código.

Vejamos:

Primeiramente, vamos ter que acrescentar na *subquery* que utilizamos como tabela (aquela que retorna as colunas com os dados que queremos ver na PIVOT TABLE) as novas colunas: Ano (extraída da coluna HireDate por meio da função YEAR) e Mes (extraída da coluna HireDate por meio da função MONTH) 1;

Ao final do código, podemos também acrescentar um ORDER BY pelas novas colunas (Ano e Mes), para que nossa PIVOT TABLE fique ordenada por ordem de Ano/Mês de contratação 2.

Executando o código ao lado, teremos a PIVOT TABLE mostrada na página anterior.

```
SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM DimEmployee) AS Dados
PIVOT(
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN([Document Control]
        ,[Engineering]
        ,[Executive]
        ,[Facilities and Maintenance]
        ,[Finance]
        ,['Human Resources Contral]
        ,[Human Resources]
        ,[Information Services]
        ,[Marketing]
        ,[Production Control]
        ,[Production]
        ,[Purchasing]
        ,[Quality Assurance]
        ,[Research and Development]
        ,[Sales]
        ,[Shipping and Receiving]
        ,[Tool Design])
) AS PivotTable
ORDER BY Ano, Mes
```

1

2

Limitações de uma PIVOT TABLE

Lembra-se que utilizamos o código abaixo para conseguir retornar os nomes dos departamentos que queríamos ver na PIVOT TABLE?

```
SELECT DISTINCT
    ',' + QUOTENAME(TRIM(DepartmentName))
FROM DimEmployee
```

Resultados | Mensagens

	(Nenhum nome de coluna)
1	,[Document Control]
2	,[Engineering]
3	,[Executive]
4	,[Facilities and Maintenance]
5	,[Finance]
6	,['Human Resources Contral]
7	,[Human Resources]
8	,[Information Services]
9	,[Marketing]
10	,[Production Control]
11	,[Production]
12	,[Purchasing]
13	,[Quality Assurance]
14	,[Research and Development]
15	,[Sales]
16	,[Shipping and Receiving]
17	,[Tool Design]

Lembra-se também que dissemos que esta era uma forma manual de fazer a extração dos dados de uma coluna?

Obtendo os dados desta forma, por não ser dinâmica, caso ocorra alguma alteração na tabela original, essa mudança não será repassada à PIVOT TABLE que criamos.

Portanto, é importante que a gente aprenda como fazer essa obtenção de forma mais automática e otimizada, para que a nossa PIVOT TABLE não fique limitada aos dados extraídos no momento da criação de seu código.

Corrigindo as limitações de uma PIVOT TABLE

Inicialmente, precisamos encontrar uma forma dinâmica de pegar os nomes de todos os departamentos da coluna DepartmentName da tabela DimEmployee e armazená-los em uma variável, em formato de string (texto). Para isso, vamos fazer o seguinte:

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''  1

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','  2
FROM
    (SELECT DISTINCT  3
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)  4
```

Vamos declarar uma variável que receberá os nomes de todos os departamentos: @NomeColunas, do tipo NVARCHAR 1 ;

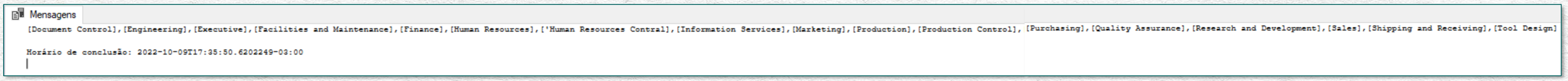
Depois, faremos um SELECT que retornará a variável @NomeColunas para que ela vá recebendo departamento distinto por departamento distinto e vá armazenando, um a um, seguido de uma vírgula concatenada 2 . Repare que, aqui, utilizamos uma *subquery* para retornar os departamentos distintos da coluna DepartmentName da tabela DimEmployee e a utilizamos como a tabela da nossa consulta principal 3 .

Por fim, vamos retirar a última vírgula concatenada após o último departamento armazenado na variável @NomeColunas. Para isso, basta utilizarmos a função LEFT 4 .

Corrigindo as limitações de uma PIVOT TABLE

Se fizermos um PRINT da variável @NomeColunas, repare que será retornada uma string com os nomes de todos os departamentos entre colchetes, separados por vírgulas, exatamente como precisamos passar ao operador IN do código da PIVOT TABLE:

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','
FROM
    (SELECT DISTINCT
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)

PRINT @NomeColunas
```



Corrigindo as limitações de uma PIVOT TABLE

Só que não podemos simplesmente pegar a variável @NomeColunas e passar para o operador IN do código da nossa PIVOT TABLE, pois não é aceito. Veja que, se executarmos o código dessa forma, será retornado um erro:

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','
FROM
    (SELECT DISTINCT
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)


SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN (@NomeColunas)
        ) AS PivotTable
ORDER BY Ano, Mes
```

Mensagens

```
Mensagem 102, Nível 15, Estado 1, Linha 187
Sintaxe incorreta próxima a '@NomeColunas'.

Horário de conclusão: 2022-10-09T17:42:09.6982380-03:00
```

Corrigindo as limitações de uma PIVOT TABLE

Para que o nosso código da PIVOT TABLE funcione, vamos ter que conseguir transformar o próprio código (da PIVOT TABLE) em uma string (texto) para poder concatená-lo com a variável @NomeColunas, e armazenar tudo isso em uma outra variável, que vamos chamar de @SQL.

Para isso, vamos ter que, inicialmente, declarar essa variável:

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''
DECLARE @SQL NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','
FROM
    (SELECT DISTINCT
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)
```

Corrigindo as limitações de uma PIVOT TABLE

Feito isso, precisamos armazenar na variável @SQL essa concatenação que mencionamos na página anterior. Repare que, colocando o código da PIVOT TABLE entre aspas simples, o concatenamos com a variável @NomeColunas utilizando os sinais de + :

```
SET @SQL=
'SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN (' + @NomeColunas + ')
        ) AS PivotTable
ORDER BY Ano, Mes'
```

Corrigindo as limitações de uma PIVOT TABLE

Agora, se dermos um PRINT na variável @SQL...

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''
DECLARE @SQL NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','
FROM
    (SELECT DISTINCT
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)

SET @SQL=
'SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN (' + @NomeColunas + ')
        ) AS PivotTable
ORDER BY Ano, Mes'

PRINT @SQL
```

Índice Geral

Índice deste Módulo

Corrigindo as limitações de uma PIVOT TABLE

... veremos que o texto retornado na mensagem é exatamente o código que precisamos executar para mostrar nossa PIVOT TABLE na tela:

Mensagens

```
SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN ([Document Control],[Engineering],[Executive],[Facilities and Maintenance],[Finance],[Human Resources],['Human Resources Contral],[Information Services],[Marketing],[Production],[Production Control],[Purchasing],[Quality Assurance],[Research and Development],[Sales],[Shipping and Receiving],[Tool Design])
        ) AS PivotTable
ORDER BY Ano, Mes

Horário de conclusão: 2022-10-09T18:31:11.0236630-03:00
```

Mensagens

```
SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN ([Document Control],[Engineering],[Executive],[Facilities and Maintenance],[Finance],[Human Resources],['Human Resources Contral],[Information Services],[Marketing],[Production],
        ) AS PivotTable
ORDER BY Ano, Mes

Horário de conclusão: 2022-10-09T18:31:11.0236630-03:00
```

,[Production Control],[Purchasing],[Quality Assurance],[Research and Development],[Sales],[Shipping and Receiving],[Tool Design])

Corrigindo as limitações de uma PIVOT TABLE

Agora temos o nosso código em formato de texto armazenado dentro da variável @SQL. Como fazemos para executá-lo?

Bom, para isso, precisamos passar a variável @SQL como parâmetro a uma procedure do sistema chamada **sp_executesql**, dessa forma:

```
EXECUTE sp_executesql @SQL
```

Corrigindo as limitações de uma PIVOT TABLE

Executando todo o código (desde a declaração das variáveis, até o EXECUTE da procedure sp_executesql)...

```
DECLARE @NomeColunas NVARCHAR(MAX) = ''
DECLARE @SQL NVARCHAR(MAX) = ''

SELECT @NomeColunas += QUOTENAME(TRIM(DepartmentName)) + ','
FROM
    (SELECT DISTINCT
        Departmentname
    FROM DimEmployee) AS Aux

SET @NomeColunas = LEFT(@NomeColunas, LEN(@NomeColunas) - 1)

SET @SQL=
'SELECT * FROM
    (SELECT
        EmployeeKey,
        YEAR(HireDate) AS Ano,
        MONTH(HireDate) AS Mes,
        DepartmentName
    FROM
        DimEmployee) AS Dados
PIVOT (
    COUNT(EmployeeKey)
    FOR DepartmentName
    IN (' + @NomeColunas + ')
        ) AS PivotTable
ORDER BY Ano, Mes'

EXECUTE sp_executesql @SQL
```

Índice Geral

Índice deste Módulo

Corrigindo as limitações de uma PIVOT TABLE

... teremos, finalmente, nossa PIVOT TABLE!

Resultados | Mensagens

	Ano	Mes	Document Control	Engineering	Executive	Facilities and M...	Finance	Human Resources	'Human Resour...	Information Services	Marketing	Production	Production Control	Purchasing	Quality Assurance	Research and Development	Sales	Shipping and Receiving	Tool Design
1	1996	7	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
2	1997	2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
3	1997	12	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	1998	1	0	0	0	0	0	0	0	0	2	1	0	0	0	0	0	0	3
5	1998	2	0	2	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0
6	1998	3	0	1	0	0	0	0	1	0	0	3	0	0	0	0	0	0	0
7	1998	4	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0
8	1999	1	1	0	0	1	2	3	0	3	1	40	2	0	2	1	0	3	0
9	1999	2	2	0	1	0	3	1	0	3	2	38	3	0	1	1	0	2	0
10	1999	3	1	0	2	0	3	2	0	4	0	45	1	1	1	1	0	1	0
11	1999	4	1	0	0	0	2	0	0	0	1	12	1	1	1	0	0	0	0
12	1999	6	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
13	2000	1	0	0	0	2	0	0	0	0	0	9	0	1	0	0	0	0	0
14	2000	2	0	0	0	1	0	0	0	0	0	8	0	3	0	0	0	0	0
15	2000	3	0	0	0	1	0	0	0	0	0	11	0	1	1	0	0	0	0
16	2000	4	0	0	0	2	0	0	0	0	0	3	0	1	0	0	0	0	0
17	2001	1	0	1	0	0	0	0	0	0	0	0	0	2	0	0	0	0	2
18	2001	2	0	1	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
19	2001	3	0	0	0	0	0	0	0	0	2	0	0	3	0	0	1	0	0
20	2001	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0
21	2002	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
22	2002	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0
23	2002	11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
24	2003	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
25	2003	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0

Agora nosso código está totalmente dinâmico. Assim, caso seja alterada qualquer informação da tabela DimEmployee, seja em sua estrutura, sejam nos dados armazenados, tudo será automaticamente atualizado toda vez que executarmos o código da página anterior. Dessa forma, teremos nossa PIVOT TABLE sempre atualizada.

SQL IMPRESSIONADOR

Apostila Completa de SQL Server